



SQL MASTERY

Zero to Advanced SQL

The complete practical guide to databases, query writing, performance engineering, and interviews.

● BEGINNER

● ADVANCED

● EXPERT

PRIMARY DIALECT

PostgreSQL 16, with labelled differences for MySQL 8, SQL Server 2022 and Oracle 19c+.

THIS VOLUME

Front matter, the full 40-part map, Part 1 — Database Fundamentals, and Part 2 — SQL Environment Setup.

About this edition

This book is a self-study curriculum. It is written to be read in order, front to back, with a live database open beside it. Every SQL statement in the text was written against PostgreSQL 16 syntax and against the practice schema defined in Chapter 2.9, so you can run any example verbatim.

DISCLAIMER

Software versions move. Where a behaviour depends on a version or a configuration setting, the text says so. Vendor documentation is the final authority for your specific installation — this book teaches the model and the reasoning, not the release notes.

USE ON REAL SYSTEMS

Examples that modify or delete data (`UPDATE` , `DELETE` , `DROP` , `TRUNCATE` , `VACUUM FULL`) are written for a throwaway practice database. Never run them against production data you cannot restore.

Volume plan

The complete work is 40 parts. It is published in volumes so each is a finished, usable book rather than a draft of the whole.

Volume	Contents	Status
I	Front matter · Parts 1–2 — Database fundamentals, environment setup, the practice database	This volume
II	Parts 3–8 — SQL basics, data types, operators, filtering, functions, aggregation	Planned
III	Parts 9–14 — Joins, subqueries, CTEs, set operations, window functions, advanced SQL	Planned
IV	Parts 15–19 — Design, normalization, constraints, transactions, isolation	Planned
V	Parts 20–26 — Indexes, performance, views, routines, triggers, JSON	Planned
VI	Parts 27–31 — Applications, security, administration, partitioning, warehousing	Planned
VII	Parts 32–40 — Projects, interviews, 500 problems, patterns, debugging, dialects, references, capstone	Planned

How to use this book

Read this page. It is the shortest chapter and it saves the most time.

The one rule

Type every query. Do not read them. SQL is a language you learn in your fingers — the difference between a reader who finishes this book and a practitioner who finishes it is roughly 4,000 keystrokes. Break each example on purpose afterwards: remove a `GROUP BY`, misspell a column, drop a join condition. The error messages you learn to read here are the same ones you will read for the rest of your career.

How a concept is taught

Every significant concept follows the same nine-beat structure, so you always know where you are:

1. **What is it** — plain language, then an analogy.
2. **Why it exists** — the problem it solves.
3. **Syntax** — the canonical form.
4. **Example** — on the practice database.
5. **Output** — the rows you should see.
6. **How it works** — execution, step by step.
7. **Real-world use** — where professionals reach for it.
8. **Mistakes & best practice** — what breaks, and the habit that prevents it.
9. **Interview & practice** — the question, then the drills.

Difficulty labels

Every exercise, problem and interview question carries one of five labels. They are absolute, not relative to the chapter — a Beginner problem in Part 13 really is a beginner problem.

● **BEGINNER** ● **EASY** ● **INTERMEDIATE** ● **ADVANCED** ● **EXPERT**

Callout boxes

NOTE

Background, precision, or a definition worth pinning down. Skippable on a first pass.

BEST PRACTICE

What experienced engineers actually do. Adopt these as habits, not as rules to recite.

COMMON MISTAKE

A trap that produces wrong answers rather than errors — the expensive kind. Read every one of these.

INTERVIEW

A question that is genuinely asked, with the thinking an interviewer is listening for.

DIALECT

Where PostgreSQL, MySQL, SQL Server and Oracle disagree. Portable SQL is called out explicitly.

Code and output

Statements appear in sand-coloured blocks; the rows they return appear in sage blocks directly beneath. Prompts (`=#` , `>`) are shown only when the shell itself is the subject, so you can copy code without editing it.

```
SELECT department_id, count(*) AS headcount
FROM   employees
GROUP  BY department_id
ORDER  BY headcount DESC;
```

```
department_id | headcount
-----+-----
              3 |          4
              1 |          3
              2 |          3
(3 rows)
```

Practice system

Each part closes with the same five-stage review, and each group of parts closes with a cumulative test.

Stage	Size	Purpose
Quick Check	5–10	Recall. Answer from memory, in one sentence each.
Practice	10–20	Write and run SQL. The bulk of your learning happens here.
Challenge	3–5	Problems that need two ideas combined.
Interview Drill	5	Answer out loud, in under two minutes, without a keyboard.
Chapter Test	10–20	Timed, closed-book. Under 70% means re-read the part.

ON SOLUTIONS

Solutions live in a separate **Solutions** section at the end of each part, never beside the question. Sit with a problem for ten minutes before you turn the page; the struggle is the mechanism.

Reading paths

You are	Read
A complete beginner	Everything, in order. Do not skip Part 1 — the relational model is the whole game.
A developer who writes some SQL	Skim Parts 1–8 through the Quick Checks; slow down at Parts 9, 13, 20 and 21.
Interviewing in two weeks	Parts 9, 10, 11, 13, 16, 20, 33, 34, 35 — then the cheat sheets in Part 39.
A data analyst	Parts 3–14 and 31, plus the pattern library in Part 35.
A backend engineer	Parts 15–22 and 27–30, plus Part 28 on injection and privileges.

Contents

The complete 40-part curriculum. Parts published in this volume are linked; the rest are the confirmed plan.

-
- 01 Database Fundamentals** — data · databases · DBMS vs RDBMS · the relational model · rows, columns, records · keys · constraints · relationships · schemas · client–server architecture · how a query executes · SQL vs NoSQL · OLTP vs OLAP

 - 02 SQL Environment Setup** — choosing an engine · installing PostgreSQL · psql · GUI clients · MySQL, SQL Server, SQLite, Oracle · creating databases and schemas · running scripts · import and export · **loading the practice database**

 - 03 SQL Basics** — CREATE · INSERT · SELECT · UPDATE · DELETE · DROP · ALTER · TRUNCATE · WHERE · ORDER BY · LIMIT · OFFSET · DISTINCT · aliases · comments

 - 04 Data Types** — integers · exact and approximate numerics · boolean · character types · date, time, timestamp, timestamptz, interval · UUID · JSON and JSONB · arrays · enums · binary

 - 05 Operators & Expressions** — arithmetic · comparison · logical · BETWEEN · IN · LIKE and ILIKE · IS NULL · CASE · COALESCE · NULLIF · three-valued logic

 - 06 Filtering & Sorting** — WHERE in depth · multi-column ORDER BY · NULLS FIRST/LAST · collation · LIMIT/OFFSET pagination · keyset pagination

 - 07 SQL Functions** — string · numeric · date and time · NULL handling · type casting · function composition

 - 08 Aggregations** — COUNT, SUM, AVG, MIN, MAX · GROUP BY · HAVING · WHERE vs HAVING · GROUP BY vs DISTINCT · grouping sets, ROLLUP, CUBE

 - 09 Joins — in depth** — the concept before the syntax · INNER · LEFT · RIGHT · FULL · CROSS · SELF · NATURAL · three or more tables · join duplication · many-to-many · semi and anti joins · **100 join problems**

 - 10 Subqueries** — scalar · single-row · multi-row · correlated · EXISTS and NOT EXISTS · IN and NOT IN with NULLs · subqueries in SELECT, FROM and WHERE · JOIN vs IN vs EXISTS

 - 11 Common Table Expressions** — WITH · multiple and chained CTEs · materialization · recursive CTEs · hierarchies · graph traversal · cycle detection

 - 12 Set Operations** — UNION · UNION ALL · INTERSECT · EXCEPT · duplicate semantics · type resolution · performance

 - 13 Window Functions — in depth** — OVER · PARTITION BY · frames · ROW_NUMBER, RANK, DENSE_RANK, NTILE · LAG, LEAD · FIRST_VALUE, LAST_VALUE, NTH_VALUE · running totals · moving averages · top-N per group · gaps and islands · sessionization · percentiles
-

14	Advanced SQL — complex CASE · FILTER · DISTINCT ON · LATERAL joins · advanced CTE patterns · pivot and unpivot · conditional aggregation · crosstab · dynamic SQL
15	Database Design — entities · attributes · relationships · ER diagrams · cardinality and optionality · candidate, natural, surrogate, composite keys · design reviews
16	Normalization — functional dependency · 1NF · 2NF · 3NF · BCNF · 4NF · 5NF · denormalization · when normalization costs performance
17	Constraints — PRIMARY KEY · FOREIGN KEY · UNIQUE · NOT NULL · CHECK · DEFAULT · CASCADE, RESTRICT, SET NULL · deferrable constraints · naming
18	Transactions — BEGIN, COMMIT, ROLLBACK · SAVEPOINT · ACID · failure modes · transaction scope in applications
19	Transaction Isolation — read uncommitted · read committed · repeatable read · serializable · snapshot isolation · dirty, non-repeatable and phantom reads · lost updates · write skew · deadlocks
20	Indexes — why indexes work · B-tree internals · hash, GIN, GiST, BRIN · composite and covering indexes · partial and expression indexes · selectivity and cardinality · maintenance cost · when an index hurts
21	Query Performance — the planner and optimizer · EXPLAIN · EXPLAIN ANALYZE · sequential, index and bitmap scans · nested loop, hash and merge joins · sorts and aggregation strategies · cost estimation · reading a plan
22	Advanced Performance Engineering — index and join tuning · pagination at scale · the N+1 problem · SELECT * · statistics and ANALYZE · VACUUM and bloat · partitioning · materialized views · caching · pooling · read replicas
23	Views — creating · updatable views · limitations · materialized views and refresh strategies · views as a security boundary
24	Stored Procedures & Functions — SQL functions · procedures · parameters and return values · variables · control flow · loops · exception handling · PL/pgSQL
25	Triggers — BEFORE and AFTER · row and statement level · INSERT, UPDATE, DELETE · audit logging · validation · pitfalls and cost
26	JSON & Semi-structured Data — JSON vs JSONB · operators · extraction and nesting · arrays · aggregation · GIN indexing · relational vs document design
27	SQL & Application Development — Python, Java, Node.js · Spring Boot, Django, Flask · parameterized and prepared statements · pooling · ORMs · transaction boundaries in services
28	SQL Security — injection anatomy and defence · authentication · roles and privileges · GRANT and REVOKE · least privilege · row-level security · encryption and masking

29	Database Administration Concepts — backup and restore · replication · failover and high availability · monitoring · logs · connection management · storage and maintenance
30	Partitioning — why partition · range, list and hash · pruning · partitioned indexes · maintenance and rotation
31	Data Warehousing SQL — OLTP vs OLAP · fact and dimension tables · star and snowflake schemas · slowly changing dimensions · analytical SQL
32	Real-world Projects — fifteen complete databases from students and libraries to e-commerce, banking, hospital, food delivery, ticketing, social, inventory, jobs, LMS, expenses, transport and SaaS
33	Interview Preparation — 100 beginner · 150 intermediate · 150 advanced · 100 scenario · 200 query-writing questions, each with the expected thinking, the solution, the common wrong approach and the tip
34	Coding Challenges — 500+ problems across five difficulty levels, each with schema, sample data, expected output, hint, solution and alternatives
35	SQL Patterns — top-N per group · latest per user · duplicate detection · missing records · consecutive dates · gaps and islands · running totals · ranking · Nth highest · above-average · retention · cohorts · funnels · sessionization
36	Debugging SQL — syntax vs logic errors · wrong joins · duplicate rows · NULL traps · broken aggregation · date filtering · window misuse · <i>find the bug</i> exercises
37	Dialect Comparison — LIMIT/TOP/FETCH · strings · dates · auto-increment · booleans · upsert · JSON · windows · CTEs · procedures · indexing · portable vs specific SQL
38	Best Practices — naming · formatting · readability · maintainability · security · performance · index strategy · transactions · error handling · documentation, each as bad SQL beside good SQL
39	Cheat Sheets — SELECT · WHERE · JOIN · GROUP BY · HAVING · subqueries · CTEs · windows · aggregation · dates · strings · constraints · indexes · transactions, plus one per dialect
40	Mastery Roadmap — 30, 60 and 90-day plans · beginner-to-advanced track · interview track · project track · practice and revision strategy · career roadmap · glossary · quick reference · capstone project

Database Fundamentals

Before the first `SELECT`, you need the model the language is built on. Everything difficult in SQL later — a join that duplicates rows, an index that is never used, a report that quietly drops customers — is a consequence of the ideas in this part. Ten pages of patience here saves a hundred pages of confusion later.

IN THIS PART

- | | |
|-----------------------------------|------------------------------------|
| 1.1 What is data? | 1.8 Schemas and database objects |
| 1.2 What is a database? | 1.9 What is SQL? |
| 1.3 Database, DBMS, RDBMS | 1.10 Client–server architecture |
| 1.4 The relational model | 1.11 How a query actually executes |
| 1.5 Keys | 1.12 SQL vs NoSQL |
| 1.6 Constraints | 1.13 OLTP vs OLAP |
| 1.7 Relationships and cardinality | Review · Solutions |

1.1 What is data?

WHAT IS IT?

Data is a recorded fact. The number `4200` is data. On its own it is useless: 4,200 what? Rupees, steps, milliseconds, employees? **Information** is data plus the context that makes it mean something — “employee 104 earns 4,200 per month”. A database's entire job is to store data together with enough structure that it stays information.

The structure comes in three grades, and knowing which one you are holding tells you which tool to reach for.

Grade	What it means	Examples
Structured	Every record has the same named fields with declared types. You can validate it before it is stored and query it precisely afterwards.	A salary table, a bank ledger, an order line
Semi-structured	Self-describing but irregular — records carry their own field names and may differ from one another.	JSON API payloads, XML, log events
Unstructured	No queryable internal shape. The database stores it and stores facts <i>about</i> it.	Photographs, PDFs, audio, free text

WHY IT MATTERS

SQL is a language for structured data, and it is extremely good at it precisely because it assumes the shape is known in advance. Postgres can also store and index semi-structured JSON (Part 26) and hold unstructured bytes, but the moment you stop declaring structure you give up most of what SQL gives you: validation, planning, and the optimizer's ability to reason about your data.

THREE WORDS USED LOOSELY EVERYWHERE

Data — recorded facts. **Information** — data in context, answering a question. **Knowledge** — patterns across information that let you decide. This book takes you from writing queries (information) to designing systems (knowledge).

REAL-WORLD USE CASE

A food-delivery company records one row per order — structured. The driver's GPS trace arrives as a JSON array of points — semi-structured. The customer's photo of a damaged meal is a file — unstructured. All three live in the same product, and a competent engineer stores each in the form that fits: the order in a table, the trace in a `jsonb` column or a time-series store, the photo in object storage with its URL and checksum in a table.

COMMON MISTAKE

Storing images or large documents as bytes inside a row “because it is simpler”. It inflates every backup, every replica and every sequential scan of that table. Store the file elsewhere; store the *reference* and its metadata in the database.

1.2 What is a database?

WHAT IS IT?

A **database** is an organised collection of related data, stored so that it can be retrieved, updated and protected reliably by many users at once. Note the three demands hidden in that sentence: *organised* (there is a schema), *reliably* (it survives crashes), *many users at once* (it handles concurrency). A spreadsheet satisfies the first and fails the other two.

WHY IT EXISTS — THE WORLD BEFORE DATABASES

Before databases, each program owned its own files. Payroll had `staff.dat`; HR had `people.csv`; the office directory had a printed list. The consequences were predictable and are worth naming, because every one of them is something a database is specifically engineered to prevent:

Problem	What went wrong	How a database answers it
Redundancy	The same address stored in four files, wasting space and drifting apart	Store each fact once; reference it (Part 16)
Inconsistency	An employee resigns; three of the four files still list them	Single source of truth, foreign keys (1.5)
No integrity rules	A salary of <code>-500</code> , a hire date in 2190, a blank name	Constraints checked on write (1.6)
Concurrency damage	Two clerks save the file; the second overwrites the first	Transactions and isolation (Parts 18–19)
No recovery	Power loss mid-write leaves a half-written file	Write-ahead logging, atomic commit
Rigid access	Every new question needs a new program	A declarative query language — SQL
No security model	File permissions, all or nothing	Roles, privileges, row-level security (Part 28)

BEST PRACTICE

When you evaluate any storage technology — a new database, a queue, a cache — walk that list. Ask which of the seven it solves for you and which it hands back to your application code. That single habit will outlive every specific product you learn.

ANALOGY, THEN THE TECHNICAL PICTURE

Think of a well-run library. Books are the data. The shelving system is the schema. The catalogue is the index. The librarian, who alone is allowed to move books and who enforces the borrowing rules, is the DBMS. You never reach behind the desk yourself — you ask, in a language the librarian understands, and the librarian decides how to find it.

Technically: the data lives in files on disk organised into fixed-size **pages** (8 KB in PostgreSQL). A **buffer cache** in memory holds hot pages. Changes are written first to a sequential **write-ahead log** so a crash can be replayed, then to the data pages later. Indexes are separate structures pointing at row locations. You never touch any of this directly; you send SQL, and the engine chooses how to satisfy it.

FIGURE 1.2 — WHAT SITS BETWEEN A QUESTION AND THE BYTES

Your application or SQL client — asks a question in SQL

↓ SQL text over a connection

DBMS — parses, plans, enforces constraints and privileges, manages transactions and concurrency

↓ page reads and writes

Storage — data files, indexes, write-ahead log, buffer cache

The database is the bottom layer. The DBMS is the layer you talk to. People say “database” for both; when precision matters, this book does not.

1.3 Database, DBMS, RDBMS

Three words that interviewers separate and beginners merge. Learn the distinction once, in one table.

Term	Definition	Concretely
Database	The organised data itself, plus its schema and indexes.	The <code>shopdb</code> data directory on disk
DBMS	The software that creates, reads, updates and protects databases, and enforces their rules.	The running PostgreSQL server process
RDBMS	A DBMS built on the <i>relational</i> model: data in tables of typed columns, related by key values, queried with SQL.	PostgreSQL, MySQL, SQL Server, Oracle

Not every DBMS is relational. MongoDB is a document DBMS, Redis a key-value store, Neo4j a graph DBMS, Cassandra a wide-column store. They are all DBMSs; none is an RDBMS. The relational ones share a common core — the model in 1.4 and the language in 1.9 — which is why the skills in this book transfer across all four dialects and to engines you have not met yet.

WHAT AN RDBMS GIVES YOU

- **A declarative language.** You state the result you want; the engine decides the algorithm. You will spend Part 21 learning to read those decisions.
- **Declared integrity.** Rules live in the database, so every application and every ad-hoc script obeys them — including the one written next year by someone who never read your documentation.
- **Transactions.** Groups of changes that either all happen or none do.
- **An optimizer with statistics.** The same query gets a different, better plan as your data grows.
- **Durability and recovery.** Committed data survives a power cut.
- **Concurrency without corruption.** Thousands of sessions reading and writing the same tables safely.

INTERVIEW · BEGINNER

Q. What is the difference between a database and a DBMS?

What they are listening for: that you separate the data from the software managing it, and can name one thing the DBMS does that the data cannot do for itself. A complete answer in two sentences: “The database is the structured data and its schema; the DBMS is the software that stores, retrieves and protects it — enforcing constraints, managing transactions and coordinating concurrent access. PostgreSQL is a DBMS; `shopdb` running inside it is a database.”

THE FOUR ENGINES THIS BOOK COVERS

Engine	Character	Where you meet it
PostgreSQL	Open source, standards-led, deep feature set (window functions, CTEs, JSONB, extensions). The book's default because what you learn here is closest to standard SQL.	Startups to banks; the default choice for new systems
MySQL	Open source, ubiquitous in web hosting, historically permissive about data; modern versions (8.0+) added CTEs and window functions.	WordPress, PHP stacks, large web properties
SQL Server	Microsoft's engine with T-SQL, strong tooling and BI integration.	Enterprise .NET environments
Oracle	The enterprise incumbent with PL/SQL and decades of specialised features.	Banking, telecom, ERP, government

DIALECT

Roughly 80% of the SQL in this book is portable across all four with no change. The remaining 20% — pagination, string and date functions, auto-increment, upsert, JSON, procedures — is exactly where interviews and migrations go wrong, so it is called out at every occurrence and tabulated in Part 37.

1.4 The relational model

Proposed by E. F. Codd in 1970, the relational model is the single idea underneath every RDBMS: **represent all data as values in tables, and derive new tables from them with operations that are themselves defined on tables.** No pointers, no navigation paths, no reliance on physical layout — just values and the relationships between them.

VOCABULARY — FORMAL AND EVERYDAY

Formal term	Everyday term	Meaning
Relation	Table	A named set of rows sharing the same columns
Tuple	Row / record	One instance of the thing the table describes — one employee
Attribute	Column / field	One named, typed property — salary
Domain	Data type	The set of values an attribute may take — numeric(10,2)
Cardinality	Row count	How many rows the table currently holds
Degree	Column count	How many columns the table has
Relation schema	Table definition	Name plus columns, types and constraints
Relation instance	Table contents	The rows in it at this moment

COMMON MISTAKE

“Relational” does not mean “tables have relationships”. It comes from *relation*, the mathematical name for a table. The relationships are a consequence of the model, not the source of its name — a favourite interview trap.

ANATOMY OF A TABLE

FIGURE 1.4 — EMPLOYEES , ANNOTATED

employee_id	full_name	department_id	salary	hire_date	manager_id
101	Asha Menon	1	92000.00	2019-03-11	NULL
102	Rahul Iyer	1	64000.00	2021-07-01	101
103	Meera Nair	2	71500.00	2020-01-20	101

Highlighted row — one tuple: everything known about Rahul Iyer.

Degree — 6 columns. **Cardinality** — 3 rows.

The salary column — one attribute across all rows; its domain is numeric(10,2) .

NULL — not zero, not blank: *unknown or not applicable*. Asha has no manager.

THE RULES THAT MAKE IT WORK

1. **Rows are unordered.** A table is a set. There is no “first row” until you write `ORDER BY`. Without it, the engine may return rows in any order — and will change that order as data grows or the plan changes.
2. **Columns are identified by name, not position.** `SELECT *` hides this and is a real source of bugs (Part 22).
3. **Every value is atomic.** One cell holds one value from its domain — not a comma-separated list of three phone numbers. This is first normal form (Part 16), and violating it destroys your ability to query, constrain and index.
4. **No duplicate rows** in a properly keyed table. This is what a primary key guarantees.
5. **Access is by value.** You find a row by what it contains, never by where it sits on disk.

BEST PRACTICE

Whenever you feel tempted to store `"maths,physics,chem"` in one column, stop: you want another table. That instinct, applied consistently, prevents most of the schema disasters in Part 16.

INTERVIEW · BEGINNER

Q. What is the difference between a row, a record and a tuple? — In practice, nothing: three names for one horizontal slice of a table. Tuple is the relational-theory term, record the file-processing term, row the SQL term. Say that, then add the distinction that matters: a *row* is data, a *column* is structure.

1.5 Keys

A key is a column or set of columns used to **identify** rows or to **connect** tables. Almost every design error in Part 15 is a key chosen badly, so the taxonomy is worth learning properly now.

Key	Definition	In <code>employees</code>
Super key	Any column set whose values are unique per row — including sets with unnecessary columns.	<code>(employee_id, full_name)</code>
Candidate key	A minimal super key: remove any column and uniqueness breaks. A table may have several.	<code>employee_id</code> ; also <code>email</code> if present
Primary key	The one candidate key you choose as <i>the</i> identifier. Unique and never <code>NULL</code> . One per table.	<code>employee_id</code>
Alternate key	The candidate keys you did not choose — still enforce them with <code>UNIQUE</code> .	<code>email</code>
Composite key	A key made of two or more columns.	<code>(order_id, product_id)</code> in <code>order_items</code>
Natural key	A key with real-world meaning.	An ISBN, a national ID, an email
Surrogate key	A meaningless generated identifier, usually an integer or UUID.	<code>employee_id</code> as a generated value
Foreign key	A column whose values must exist as a key in another table. The mechanism of every relationship.	<code>department_id</code> → <code>departments</code>

SYNTAX

```
CREATE TABLE departments (  
    department_id    integer PRIMARY KEY,  
    department_name  text NOT NULL UNIQUE  
);  
  
CREATE TABLE employees (  
    employee_id      integer PRIMARY KEY,  
    full_name        text    NOT NULL,  
    email            text    UNIQUE,                -- alternate key  
    department_id    integer REFERENCES departments, -- foreign key  
    salary           numeric(10,2) CHECK (salary > 0),  
    hire_date        date    NOT NULL DEFAULT current_date,  
    manager_id       integer REFERENCES employees    -- self-reference  
);
```

Read the last line again: a foreign key may point at the same table. That is how a management hierarchy is stored, and it is what makes the self-join (Part 9) and the recursive CTE (Part 11) possible.

NATURAL OR SURROGATE?

	Natural key	Surrogate key
For	Meaningful; no extra column; joins can avoid a lookup	Never changes; compact and fast; never leaks business meaning; always available at insert time
Against	Real-world values change (people change email and surname); may be long, or not actually unique; a change cascades to every referencing row	An extra column; two rows can be identical apart from the key unless you also enforce the natural key

BEST PRACTICE

Use a surrogate primary key *and* a `UNIQUE` constraint on the natural key. You get stable joins and real integrity. The failure mode of surrogate-only design is duplicate people with different ids — which is far more expensive than one extra column.

COMMON MISTAKE

A primary key on a column that can repeat later — an invoice number that resets each financial year, a phone number that gets reassigned. If a value can ever be reused, it is not an identifier.

INTERVIEW · EASY

Q. Can a table have two primary keys? — No. It can have one primary key, and that key may be *composite* (several columns). It can also have many `UNIQUE` constraints. Candidates who answer “yes, a composite key” are being tested on exactly this vocabulary; be precise.

1.6 Constraints

A **constraint** is a rule the database enforces on every write, from every client, forever. It is the difference between data you trust and data you hope about. Part 17 covers each in depth; here is the working set.

Constraint	Guarantees	Example
NOT NULL	The column always has a value	<code>full_name text NOT NULL</code>
UNIQUE	No two rows share this value	<code>email text UNIQUE</code>
PRIMARY KEY	<code>UNIQUE</code> + <code>NOT NULL</code> ; the row identifier	<code>employee_id integer</code> <code>PRIMARY KEY</code>
FOREIGN KEY	The referenced row exists — <i>referential integrity</i>	<code>REFERENCES departments</code>
CHECK	An arbitrary boolean condition holds	<code>CHECK (salary > 0)</code>
DEFAULT	A value supplied when the insert omits the column	<code>DEFAULT current_date</code>

NOTE

`DEFAULT` is strictly not a constraint — it constrains nothing, it fills a gap. It is grouped here because it is declared in the same place and answers the same design question: “what must always be true of this column?”

WHY NOT ENFORCE RULES IN THE APPLICATION?

Because there is never exactly one application. There is the API, plus the mobile backend, plus a nightly import job, plus the migration script, plus a colleague in `psql` at 23:00 fixing an incident. Application checks are advisory; a constraint is a guarantee. The database is the only place every writer must pass through.

```
-- Both rules are enforced no matter who writes:
INSERT INTO employees (employee_id, full_name, salary)
VALUES (110, 'Test User', -100);
-- ERROR: new row for relation "employees" violates check
constraint "employees_salary_check"

INSERT INTO employees (employee_id, full_name, department_id, salary)
VALUES (111, 'Test User', 99, 50000);
-- ERROR: insert or update on table "employees" violates foreign key constraint
-- DETAIL: Key (department_id)=(99) is not present in table "departments".
```

BEST PRACTICE

Name your constraints explicitly (`CONSTRAINT employees_salary_positive CHECK (salary > 0)`). Generated names appear in error messages your users may see and in migration scripts your team must read; a good name makes an incident five minutes shorter.

1.7 Relationships and cardinality

A relationship is an association between rows in two tables, expressed with keys. There are exactly three shapes.

FIGURE 1.7 — THE THREE RELATIONSHIP SHAPES

One-to-one (1:1) — one employee has one passport record.

employees —1—1— employee_passports

Implementation: the child's foreign key is also `UNIQUE`, or is itself the primary key. Use it to split rarely-read or sensitive columns off a hot table.

One-to-many (1:N) — one department has many employees. *The common case.*

departments —1—∞— employees

Implementation: the foreign key lives on the **many** side. `employees.department_id` → `departments.department_id`. Never a list of employee ids on the department.

Many-to-many (M:N) — an order contains many products; a product appears in many orders.

orders —1—∞— order_items —∞—1— products

Implementation: a **junction table** holding one foreign key to each side, with a composite primary key across both — and often extra columns of its own (`quantity`, `unit_price`).

A relational database cannot store a many-to-many relationship directly, and that is not a limitation — the junction table is where the interesting attributes of the association live. The quantity of a product in an order belongs to neither the order nor the product; it belongs to the pairing.

CARDINALITY AND OPTIONALITY

Cardinality is *how many*; optionality is *whether it is required*. They are independent, and both must be decided per relationship — because optionality is what determines whether a column is nullable, and nullable columns are what force you to choose between `INNER JOIN` and `LEFT JOIN` in Part 9.

Reading	Means	Design consequence
Exactly one (1..1)	Every employee must have a department	<code>department_id ... NOT NULL REFERENCES</code>
Zero or one (0..1)	An employee may have no manager	Nullable foreign key; queries need <code>LEFT JOIN</code>
One or more (1..*)	An order must have at least one line	Not expressible as a column constraint — enforce in a transaction or trigger

Reading	Means	Design consequence
Zero or more (0..*)	A customer may have no orders	The default; nothing extra to declare

COMMON MISTAKE

Making a foreign key `NOT NULL` because “it should always be set”, then discovering the import job has rows where it genuinely is not known. Decide optionality from the business rule, not from tidiness — and if the rule is “unknown is possible”, allow the `NULL` and handle it in your joins.

INTERVIEW · EASY

Q. How do you model a many-to-many relationship? — With a third table holding a foreign key to each side and a composite primary key across the pair, plus any attributes of the association itself. Volunteer the example: `students`, `courses`, and `enrollments(student_id, course_id, enrolled_on, grade)`. Naming the extra columns is what separates a memorised answer from an understood one.

1.8 Schemas and database objects

The word **schema** carries two meanings, and mixing them causes real confusion:

1. **Schema as design** — the whole structure of your database: the tables, their columns and types, their keys and constraints. “Review the schema before we ship.”
2. **Schema as namespace** — a named container *inside* a database that groups objects, in PostgreSQL and SQL Server. “Put the reporting views in the `analytics` schema.”

PostgreSQL's containment is: **cluster** → **database** → **schema** → **object**. One server (a cluster) holds many databases; you connect to exactly one and cannot query across to another without extra machinery. Inside it, schemas namespace your tables. Every database starts with a schema called `public`.

```
-- Fully qualified name: schema.object
SELECT * FROM sales.orders;

CREATE SCHEMA analytics;
CREATE TABLE analytics.daily_revenue (...);

-- The search_path decides what an unqualified name resolves to
SET search_path TO analytics, public;
SELECT * FROM daily_revenue; -- resolves to analytics.daily_revenue
```

DIALECT

In **MySQL**, “schema” and “database” are synonyms — `CREATE SCHEMA` and `CREATE DATABASE` do the same thing, and there is no namespace layer between them. In **SQL Server**, a database contains schemas exactly as in PostgreSQL (`dbo` is the default), and cross-database queries are permitted. In **Oracle**, a schema *is* a user — the objects owned by that account.

THE OBJECTS A SCHEMA HOLDS

Object	What it is	Part
Table	Stored rows — the only object that holds data	3
View	A named query, evaluated on each use	23
Materialized view	A named query whose results are stored and refreshed	23
Index	A lookup structure that speeds up access	20
Sequence	A generator of increasing numbers, behind identity columns	4
Constraint	A declared rule attached to a table	17
Function / procedure	Code that runs inside the database	24

Object	What it is	Part
Trigger	Code fired automatically by a data change	25
Type / domain	A user-defined type, enum, or constrained type	4

BEST PRACTICE

Use schemas as a boundary, not decoration: one per bounded domain (`sales` , `billing` , `analytics`), with privileges granted per schema. It makes least-privilege access (Part 28) a one-line grant instead of a table-by-table audit.

1.9 What is SQL?

SQL — Structured Query Language — is the standard language for defining and manipulating relational data. Two properties define its character:

- **It is declarative.** You describe the result, not the algorithm. `SELECT ... WHERE country = 'IN'` says nothing about scanning, index use or join order; the optimizer decides, and its decision changes as your data changes. This is why SQL rewards understanding the data more than understanding the engine.
- **It is set-based.** Operations apply to whole sets of rows at once. Beginners coming from Python write loops; the SQL habit is to express the transformation once and let the engine iterate. Every performance chapter in this book comes back to that sentence.

THE FIVE SUBLANGUAGES

SQL commands are conventionally grouped into five families. Interviewers ask for these by name.

Family	Commands	Purpose & transaction behaviour
DDL definition	<code>CREATE</code> <code>ALTER</code> <code>DROP</code> <code>TRUNCATE</code> <code>RENAME</code>	Defines structure. In PostgreSQL and SQL Server, DDL is transactional — you can roll back a <code>CREATE TABLE</code> . In MySQL and Oracle it is not; Oracle commits implicitly.
DML manipulation	<code>INSERT</code> <code>UPDATE</code> <code>DELETE</code> <code>MERGE</code>	Changes data. Always transactional; always reversible before commit.
DQL query	<code>SELECT</code>	Reads data. The one command you will write ten thousand times.
DCL control	<code>GRANT</code> <code>REVOKE</code>	Manages privileges (Part 28).
TCL transaction	<code>BEGIN</code> <code>COMMIT</code> <code>ROLLBACK</code> <code>SAVEPOINT</code>	Groups statements into all-or-nothing units (Part 18).

INTERVIEW · BEGINNER

Q. Is `TRUNCATE` DDL or DML? What is the difference from `DELETE` ? — `TRUNCATE` is DDL. It removes all rows by deallocating storage rather than deleting them row by row, so it is far faster on large tables, cannot carry a `WHERE` clause, does not fire row-level triggers, and resets identity sequences in some engines. `DELETE` is DML: row-by-row, filterable, trigger-firing, and logged per row. Add the nuance that earns the point: in PostgreSQL `TRUNCATE` is still transactional and can be rolled back; in MySQL and Oracle it cannot.

NOTE · STANDARDS

SQL is an ISO standard (SQL:1999 through SQL:2023), but no engine implements all of it and every engine adds its own extensions. “Standard SQL” is therefore a useful target, not a description of any product. Write to the standard where you can; know where you have left it.

1.10 Client–server architecture

Almost every RDBMS is a **server**: a long-running process that owns the data files, and that clients reach over a network connection. Your SQL client, your web application and your reporting tool are all just clients.

FIGURE 1.10 — THE ROUND TRIP OF ONE STATEMENT

- 1 Connect.** The client opens a TCP connection, authenticates, and selects a database. Connections are expensive — this is why applications pool them (Part 22).
- 2 Send.** The SQL text (with separate parameter values, if you are writing safe code — Part 28) crosses the wire.
- 3 Parse.** The server checks syntax and resolves every name against the catalog, producing a parse tree.
- 4 Rewrite.** Views are expanded, rules applied.
- 5 Plan.** The optimizer enumerates strategies, costs each using table statistics, and picks the cheapest. This is the step you learn to inspect in Part 21.
- 6 Execute.** The plan runs, reading pages from the buffer cache or disk, enforcing privileges and constraints, honouring the transaction's isolation level.
- 7 Return.** Rows stream back to the client, often in batches rather than all at once.

Two practical consequences fall straight out of this picture. First, **latency is per round trip**, not per row: one query returning 1,000 rows is dramatically cheaper than 1,000 queries returning one row each — the N+1 problem of Part 22. Second, **the server is shared**: your careless query competes for the same CPU, cache and locks as everyone else's.

NOTE · EMBEDDED DATABASES

SQLite is the exception: no server, no connection, no network. It is a library that reads and writes a single file inside your process. That makes it perfect for tests, phones and desktop apps, and unsuitable for many concurrent writers.

1.11 How a query actually executes

This is the most valuable page in Part 1. SQL is *written* in one order and *evaluated* in another, and almost every beginner error — “why can't I use my alias in `WHERE`?”, “why does `HAVING` exist?” — dissolves once you know the evaluation order.

WRITTEN ORDER

```
SELECT
FROM
JOIN
WHERE
GROUP BY
HAVING
ORDER BY
LIMIT
```

LOGICAL EVALUATION ORDER

```
1 FROM      pick the tables
2 JOIN      combine them
3 WHERE      filter rows
4 GROUP BY   form groups
5 HAVING     filter groups
6 SELECT     compute output columns
7 DISTINCT   remove duplicates
8 ORDER BY   sort the result
9 LIMIT      take a slice
```

Read the right column carefully and three rules become obvious rather than arbitrary:

1. A **SELECT** alias cannot be used in **WHERE**. **WHERE** runs at step 3; the alias is not created until step 6. It *can* be used in **ORDER BY**, which runs at step 8. This is why the identical-looking pair below behaves differently.
2. **WHERE** filters rows, **HAVING** filters groups. One runs before grouping, the other after. That is the whole distinction, and it is asked in almost every SQL interview.
3. **LIMIT** without **ORDER BY** is not deterministic. The slice is taken at step 9 from whatever order step 8 left — and with no **ORDER BY**, that order is undefined.

```
-- Fails: annual_salary does not exist yet at WHERE
SELECT full_name, salary * 12 AS annual_salary
FROM employees
WHERE annual_salary > 800000;
-- ERROR: column "annual_salary" does not exist

-- Works: repeat the expression...
SELECT full_name, salary * 12 AS annual_salary
FROM employees
WHERE salary * 12 > 800000
ORDER BY annual_salary DESC;  -- ...but the alias is fine here
```

NOTE · LOGICAL VS PHYSICAL

This is the *logical* order — the semantics the result must obey. The engine is free to execute differently as long as the answer is identical, and it does: it will push a `WHERE` filter down into a table scan, use an index to satisfy `ORDER BY` without sorting, or stop early because of `LIMIT`. Part 21 reads those physical plans. Reason about correctness with the logical order; reason about speed with the plan.

INTERVIEW · INTERMEDIATE

Q. Why can you reference a column alias in `ORDER BY` but not in `WHERE` ? — Because `WHERE` is evaluated before the `SELECT` list produces the alias, while `ORDER BY` is evaluated after. State the order, and note the workaround: wrap the query in a subquery or CTE and filter outside, or repeat the expression. **Common wrong approach:** claiming it is a dialect quirk — MySQL's tolerance in `HAVING` is the extension, not the rule.

1.12 SQL vs NoSQL

“NoSQL” is not one thing — it names four unrelated families that happen to share the property of not being relational. Comparing “SQL vs NoSQL” as if it were one choice is the mistake; the real question is always *which access pattern*.

Family	Model	Examples	Fits
Key-value	Opaque value under a key	Redis, DynamoDB	Caches, sessions, counters
Document	Self-describing nested documents	MongoDB, Couchbase	Varying shapes read whole
Wide-column	Row keys with sparse column families	Cassandra, HBase	Huge write volume, known access path
Graph	Nodes and edges as first-class	Neo4j	Deep, variable-length traversal

Dimension	Relational	Typical NoSQL
Schema	Declared and enforced on write	Flexible; interpreted on read
Query language	SQL — declarative, ad-hoc, joinable	API or engine-specific; joins limited or absent
Joins	Native and optimized	Usually done in the application
Transactions	ACID across many rows and tables	Often single-document; multi-document varies
Consistency	Strong by default	Frequently eventual, tunable
Scaling	Vertical first, then read replicas and partitioning	Horizontal sharding as a design premise
Cost of a new question	Write a query	May require a new index, or a data migration

BEST PRACTICE

Default to relational for data with relationships and correctness requirements — orders, money, users, inventory. Reach for a specialised store when the access pattern is genuinely the specialised one: a cache, a document you always read whole, a write-firehose, a deep graph traversal. Most production systems use both, and PostgreSQL's `jsonb` (Part 26) covers a surprising amount of the “we need flexibility” case without a second system to operate.

COMMON MISTAKE

Choosing a document store “for scale” at a thousand users, then rebuilding joins, uniqueness and transactions by hand in application code — slower, buggier, and unenforced. Scale is a real reason; it is rarely *your* reason yet.

INTERVIEW · INTERMEDIATE

Q. When would you choose NoSQL over a relational database? — Name a concrete access pattern, not a slogan. Good answer: “A session cache with millions of tiny reads and no relationships — Redis. Or event ingestion at a scale where I need write throughput more than ad-hoc queries, with a known query key — Cassandra. For anything with entities that relate to each other and invariants I need enforced, relational, because I get joins, constraints and transactions for free.”

1.13 OLTP vs OLAP

The same SQL language serves two workloads with almost opposite characteristics. Knowing which one you are in tells you how to model, how to index and what “fast” means.

	OLTP — transactional	OLAP — analytical
Question shape	“Place this order.” “Show me my last 10 orders.”	“Revenue by category by month for three years.”
Rows touched	A handful, by key	Millions, aggregated
Read/write mix	Heavy writes, short transactions	Read-mostly, bulk loads
Target latency	Milliseconds	Seconds to minutes
Modelling	Normalized (Part 16)	Star / snowflake schemas (Part 31)
Indexing	Selective B-trees on lookup keys	Fewer, wider; partitioning; columnar storage
Concurrency	Thousands of short sessions	Few long-running queries
Typical engines	PostgreSQL, MySQL, SQL Server, Oracle	Snowflake, BigQuery, Redshift, ClickHouse

The friction is real and predictable: a normalized OLTP schema needs six joins to answer an analytical question, and the analytical query's long-running scans compete with the checkout path for cache and CPU. The standard resolution is separation — replicate or ETL the transactional data into an analytical store, or at minimum route reporting to a read replica (Part 22).

NOTE · HTAP

Some systems claim to serve both from one store (“hybrid transactional/analytical processing”), and PostgreSQL with materialized views and partitioning goes a long way for mid-sized workloads. Treat it as a spectrum, and let the numbers decide: when your reporting query starts affecting checkout latency, it is time to separate.

INTERVIEW · EASY

Q. Why not run analytics directly on the production database? — Because long scans evict the buffer cache the transactional path depends on, hold locks and snapshots open, and consume CPU your checkout needs; and because the normalized schema is the wrong shape for aggregation. The mitigation ladder: read replica → materialized views → a separate analytical store.

Part 1 Review

Work through all five stages before starting Part 2. Solutions begin on the next page — resist them.

Quick Check ● BEGINNER

One sentence each, from memory.

- 1 Define data, information and knowledge, with one example each.
- 2 Name four problems that separate files caused and that a DBMS solves.
- 3 State the difference between a database, a DBMS and an RDBMS.
- 4 Where does the word “relational” come from?
- 5 Define degree and cardinality for a table.
- 6 What exactly does a primary key guarantee, in two properties?
- 7 On which side of a one-to-many relationship does the foreign key live, and why?
- 8 List the five SQL sublanguages and one command from each.
- 9 Give the logical evaluation order of a `SELECT` statement, all nine steps.
- 10 Name three differences between OLTP and OLAP workloads.

Practice ● EASY

Design and reasoning exercises — pen and paper is fine; no database needed yet.

- 1 For a school, list the entities you would store and mark each relationship 1:1, 1:N or M:N.
- 2 A table stores `student_name` and `subjects = 'maths,physics,chem'`. Name three questions this makes hard to answer, and redesign it.
- 3 Given `books(isbn, title, author, publisher, publisher_city)`, list every candidate key and every problem you can see.
- 4 Choose primary keys for: a bank account, a country, an order line, a login session. Justify natural vs surrogate for each.
- 5 Write the constraints (in words) you would attach to `employees` beyond its primary key.
- 6 Model an M:N relationship between `doctors` and `patients` that also records the appointment time and fee. Name the junction table's columns and its primary key.
- 7 Explain why `SELECT ... LIMIT 10` without `ORDER BY` may return different rows on two consecutive runs.
- 8 An employee's `manager_id` is nullable. Give one query that would silently lose rows if written with `INNER JOIN`.
- 9 Decide the optionality (0..1, 1..1, 0..*, 1..*) of: order→customer, customer→order, employee→manager, order→order line.

- 10 For a photo-sharing app, decide where each of these belongs — relational table, JSONB column, or object storage: user profile, follower edges, image bytes, EXIF metadata, feed cache.
- 11 Sketch the round trip of one `UPDATE` from client to storage, naming the seven stages.
- 12 Rewrite `SELECT name, salary*12 AS annual FROM employees WHERE annual > 500000` two different ways so it runs.

Challenge ● INTERMEDIATE

- 1 A team stores each customer as one JSON document to “avoid migrations”. Twelve months on, list five concrete problems they now have, and the smallest change to each that a relational design would have prevented.
- 2 Design the tables for a cinema booking system where a seat may be sold only once per showing. Which constraint enforces that, and on which table?
- 3 You must record an employee's *history* of departments, not just the current one. Change the design, and say what the primary key of the new table is.
- 4 Reporting queries have begun slowing down checkout. Give three mitigations in increasing order of cost, and the signal that justifies moving to the next one.

Interview Drill ● ADVANCED

Out loud, under two minutes each, no keyboard.

- 1 Explain the relational model to a non-technical manager, then to a senior engineer.
- 2 What is referential integrity, and what breaks without it?
- 3 Why enforce a rule in the database rather than the application?
- 4 Walk through what happens between pressing enter on a query and seeing rows.
- 5 When is a relational database the wrong choice? Be specific about the access pattern.

Chapter Test ● INTERMEDIATE

Closed book, 20 minutes. Below 70% — re-read the part.

- 1 True or false: a table can have two primary keys. Explain.
- 2 Which is DDL — `TRUNCATE` or `DELETE` ? Give three further differences.
- 3 What does `NULL` mean, and what does it not mean?
- 4 Distinguish super key, candidate key, primary key and alternate key.
- 5 Why can a relational database not store an M:N relationship directly?
- 6 In PostgreSQL, name the four levels of containment from server to table.
- 7 Which two steps of the logical order explain why `HAVING` exists?

- 8** Name two things a `CHECK` constraint can do that a data type cannot.
- 9** Why is one query returning 500 rows usually faster than 500 queries returning one row?
- 10** Give one case each where a natural key and a surrogate key is the better primary key.
- 11** What does “declarative” mean, and what does it let the database do for you?
- 12** Define atomicity of a column value, and give a violation.
- 13** What is a junction table and what is usually its primary key?
- 14** Name the three grades of data structure and which one SQL is designed for.
- 15** Explain the difference between the two meanings of “schema”.

Part 1 Solutions

Quick Check

- 1 Data is a recorded fact (`4200`); information is data in context (“employee 104 earns 4,200 monthly”); knowledge is the pattern across information (“our junior salaries sit 18% below market”).
- 2 Any four of: redundancy, inconsistency, no integrity rules, concurrency damage, no crash recovery, rigid access requiring new programs, no security model.
- 3 The database is the structured data and its schema; the DBMS is the software managing and protecting it; an RDBMS is a DBMS implementing the relational model and queried with SQL.
- 4 From *relation* — the mathematical term for a table. Not from “relationships between tables”.
- 5 Degree = number of columns. Cardinality = number of rows. (Cardinality also names relationship multiplicity — context decides.)
- 6 Uniqueness and non-nullability: every row is identifiable, and the identifier is always present. It also implies an index in every major engine.
- 7 On the many side. One column can hold one value, so the side that has exactly one partner holds the reference; putting a list on the one side would violate atomicity.
- 8 DDL `CREATE` ; DML `INSERT` ; DQL `SELECT` ; DCL `GRANT` ; TCL `COMMIT` .
- 9 `FROM` → `JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT` .
- 10 Any three of: rows touched per query, read/write mix, latency target, modelling style, indexing strategy, concurrency profile.

Practice — worked

2 • The comma-separated subjects column. Hard to answer: “how many students take physics?” (needs string matching, and `LIKE '%chem%'` also matches `biochem`); “which subject is most popular?” (no way to aggregate); “rename physics to Physics I” (must rewrite every row); and you cannot constrain a subject to exist, nor index the relationship. Redesign into three tables:

```
CREATE TABLE students (student_id integer PRIMARY KEY, full_name text NOT NULL);
CREATE TABLE subjects (subject_id integer PRIMARY KEY, subject_name text NOT NULL UNIQUE);
CREATE TABLE student_subjects (
    student_id integer REFERENCES students,
    subject_id integer REFERENCES subjects,
    PRIMARY KEY (student_id, subject_id)
);
```

The composite primary key does double duty: it identifies the pairing and it prevents enrolling the same student in the same subject twice.

3 • books . Candidate key: `isbn` . Problems: `author` as a single text column cannot handle two authors (atomicity), and `publisher_city` depends on `publisher` , not on `isbn` — a transitive dependency. Change the publisher's city and you must update every one of their books, and if you miss one the data

disagrees with itself. Fix: `publishers(publisher_id, name, city)`, `books(isbn, title, publisher_id)`, and `book_authors(isbn, author_id)`. This is third normal form arriving early; Part 16 formalises it.

4 • Primary keys. Bank account — surrogate `account_id`, with `UNIQUE` on the account number (which is natural but formatted, reissued across branches, and sometimes changed by migration). Country — natural `ISO 3166-1 alpha-2` code is defensible: short, stable, universally recognised; codes do occasionally change, so accept the small risk knowingly. Order line — composite `(order_id, product_id)`, or a surrogate if the same product may legitimately appear twice on one order. Login session — surrogate, and specifically a UUID or cryptographically random token, because a guessable sequential id is a security hole.

6 • Doctors and patients. `appointments(appointment_id, doctor_id, patient_id, scheduled_at, fee)`. The primary key is the surrogate `appointment_id` — not `(doctor_id, patient_id)`, since the same pair meets repeatedly. Add `UNIQUE (doctor_id, scheduled_at)` so one doctor cannot be double-booked; that is the constraint an interviewer is listening for.

7 • LIMIT without ORDER BY. A table is an unordered set, and the returned order is a side effect of the plan: which index was used, whether the scan started mid-table from a previous position, whether rows were physically rewritten by an update, or whether a parallel scan interleaved workers. Any of those can change between runs. If order matters, say so — and make the `ORDER BY` unique (add the primary key as a tie-break) or ties will still shuffle.

8 • The nullable manager_id. “List every employee with their manager's name” written as `FROM employees e JOIN employees m ON e.manager_id = m.employee_id` silently drops the CEO, who has no manager — and any employee whose manager is not yet recorded. `LEFT JOIN` keeps them with a `NULL` manager name. This is the single most common cause of “the totals don't match” in reporting.

12 • The alias in WHERE. Repeat the expression: `WHERE salary * 12 > 500000`. Or filter outside a subquery or CTE, where the alias is a real column:

```
SELECT * FROM (
    SELECT full_name, salary * 12 AS annual FROM employees
) AS e
WHERE annual > 500000;
```

Prefer repeating the expression for a single condition; prefer the CTE when the derived value is used several times or the query is already long.

Challenge — worked

2 • Cinema booking. `movies`, `screens`, `showings(showing_id, movie_id, screen_id, starts_at)`, `seats(seat_id, screen_id, row_label, seat_number)`, `bookings(booking_id, showing_id, customer_id, booked_at)`, and the table that carries the rule: `booking_seats(booking_id, seat_id)` with `UNIQUE (showing_id, seat_id)`. Because `showing_id` lives on `bookings`, either denormalize it onto `booking_seats` to make that constraint declarable, or hold the uniqueness on a

`showing_seats(showing_id, seat_id, booking_id)` table where `(showing_id, seat_id)` is the primary key. The second is cleaner: the seat-per-showing is the real entity, and the primary key *is* the business rule. Enforcing it in application code instead loses the race between two customers clicking at once.

3 • Department history. Replace `employees.department_id` with `employee_departments(employee_id, department_id, valid_from, valid_to)`, primary key `(employee_id, valid_from)` — an employee starts at most one assignment per day. Use a `NULL` `valid_to` for the current row, and a `CHECK (valid_to IS NULL OR valid_to > valid_from)`. Current department becomes a view. This is a slowly changing dimension (Part 31) and an exclusion-constraint exercise (Part 17).

4 • Reporting hurting checkout. (i) Route reports to a read replica — cheap, no schema change, accepts small replication lag. Move on when the replica itself cannot keep up or reports need cross-source data. (ii) Materialize the expensive aggregates and refresh on a schedule — bounded cost, accepts staleness. Move on when the number of distinct report shapes outgrows what you can pre-compute. (iii) A separate analytical store with an ETL pipeline — highest cost in operations and expertise, but the right shape for the workload. The signal for each step is the same: the cheaper tier's latency or freshness has stopped meeting the requirement, measured, not guessed.

Chapter Test

- 1** False. One primary key per table; it may be composite. Additional uniqueness is expressed with `UNIQUE` constraints.
- 2** `TRUNCATE` is DDL. It takes no `WHERE`, does not fire row triggers, is far faster on large tables, and may reset identity sequences; in PostgreSQL it is still transactional, in MySQL and Oracle it is not.
- 3** `NULL` means unknown or not applicable. It is not zero, not an empty string, and not false — and it is not equal to another `NULL`, which is why `IS NULL` exists.
- 4** Super key: any unique column set. Candidate key: a minimal super key. Primary key: the chosen candidate. Alternate key: the remaining candidates.
- 5** Because a column holds one atomic value, so neither side can hold a list of the other's keys. The relationship becomes its own table.
- 6** Cluster (server) → database → schema → object.
- 7** `GROUP BY` (step 4) runs after `WHERE` (step 3), so a filter on an aggregate needs a clause that runs after grouping — `HAVING` (step 5).
- 8** Express a relationship between two columns in the same row (`CHECK (valid_to > valid_from)`), and restrict a range inside a type (`CHECK (rating BETWEEN 1 AND 5)`).
- 9** Because network latency, parsing and planning are paid per round trip, not per row, and the engine can satisfy one set-based query with a single efficient plan rather than 500 independent ones.
- 10** Natural: a country's ISO code — short, stable, meaningful. Surrogate: a person — every “natural” identifier about a person can change or be duplicated.
- 11** You state the result, not the algorithm. That lets the optimizer choose scans, join order and join algorithms, and change them as statistics change — without you rewriting the query.

- 12** One cell holds exactly one value from its domain. Violation: `phone = '9876543210, 9123456789'`.
- 13** A table implementing an M:N relationship, holding a foreign key to each side; its primary key is usually the composite of those two keys, or a surrogate when the pairing may repeat.
- 14** Structured, semi-structured, unstructured. SQL is designed for structured data (and handles semi-structured via JSON types).
- 15** Schema-as-design: the whole structure of the database. Schema-as-namespace: a named container of objects inside a database.

SQL Environment Setup

You cannot learn SQL by reading it. This part gets a real database running on your machine, teaches you the handful of client commands you will use every day, and loads the practice schema that every example in the remaining thirty-eight parts is written against. Budget an hour; you will use it for years.

IN THIS PART

- | | |
|---------------------------------------|----------------------------------|
| 2.1 Choosing an engine | 2.6 Databases, schemas, users |
| 2.2 Installing PostgreSQL | 2.7 Running SQL scripts |
| 2.3 psql — the commands that matter | 2.8 Importing and exporting data |
| 2.4 GUI clients | 2.9 The practice database |
| 2.5 MySQL, SQL Server, SQLite, Oracle | Review · Solutions |

2.1 Choosing an engine

Use **PostgreSQL** for this book. Three reasons, in order of importance: it is the closest mainstream engine to standard SQL, so what you learn transfers rather than needing unlearning; it implements every feature this book covers, including window functions, recursive CTEs, `FILTER`, `LATERAL`, `DISTINCT ON` and JSONB; and its error messages are the most informative of the four, which matters enormously while you are learning.

If you must use	What still works	What to watch
MySQL 8+	Nearly everything: CTEs, window functions and <code>FILTER</code> -equivalent conditional aggregation are all available	<code>LIMIT</code> syntax matches; no <code>DISTINCT ON</code> , no <code>LATERAL</code> before 8.0.14, different string and date functions, <code>AUTO_INCREMENT</code> instead of <code>GENERATED</code>
SQL Server 2016+	Window functions, CTEs, <code>MERGE</code> , JSON functions	<code>TOP</code> / <code>OFFSET...FETCH</code> instead of <code>LIMIT</code> , <code>ISNULL</code> vs <code>COALESCE</code> , <code>GETDATE()</code> , T-SQL procedures
Oracle 12c+	Everything, plus features the others lack	<code>FETCH FIRST</code> , no native boolean in SQL until 23c, <code>DUAL</code> , <code>NVL</code> , <code>SYSDATE</code> , empty string equals <code>NULL</code> , PL/SQL
SQLite	Surprisingly much: CTEs, window functions, JSON	Dynamic typing, no <code>RIGHT</code> / <code>FULL JOIN</code> before 3.39, minimal <code>ALTER TABLE</code> , single writer

BEST PRACTICE

Learn one engine deeply before sampling others. Depth in PostgreSQL makes MySQL a weekend of differences; breadth across four engines at once makes all of them feel arbitrary.

2.2 Installing PostgreSQL

Pick *one* of the four routes below. Docker is the fastest to a clean state and the easiest to throw away; a native install is the most convenient if this will be your daily database.

ROUTE A — DOCKER (ANY OS)

```
docker run --name pg16 \  
  -e POSTGRES_PASSWORD=secret \  
  -p 5432:5432 \  
  -v pgdata:/var/lib/postgresql/data \  
  -d postgres:16  
  
# open a shell inside the container  
docker exec -it pg16 psql -U postgres
```

The named volume `pgdata` is what makes your data survive `docker rm`. Without it, removing the container removes the database — occasionally what you want, usually not.

ROUTE B — MACOS

```
brew install postgresql@16  
brew services start postgresql@16  
psql postgres          # your macOS username is the default superuser
```

Postgres.app is the graphical alternative: drag to Applications, click start, done — and it bundles the command-line tools if you add them to your `PATH`.

ROUTE C — WINDOWS

Run the EDB installer, accept port 5432, and set a password for the `postgres` superuser — write it down; there is no recovery path that is quicker than reinstalling. The installer includes pgAdmin and the Stack Builder. Then, from a terminal:

```
psql -U postgres -h localhost
```

ROUTE D — LINUX

```
# Debian / Ubuntu  
sudo apt install postgresql postgresql-client  
sudo -u postgres psql  
  
# Fedora / RHEL  
sudo dnf install postgresql-server postgresql  
sudo postgresql-setup --initdb  
sudo systemctl enable --now postgresql
```

VERIFY THE INSTALL

```
SELECT version();  
SELECT current_database(), current_user, now();
```

version

PostgreSQL 16.2 on x86_64-pc-linux-gnu, compiled by gcc 12.2.0, 64-bit
(1 row)

COMMON MISTAKE · FOUR ERRORS YOU WILL MEET IN THE FIRST TEN MINUTES

`connection refused` — the server is not running, or is on another port. Start the service; check with `pg_isready`.

`role "you" does not exist` — psql defaults to your OS username as the database role. Pass `-U postgres`.

`database "you" does not exist` — psql also defaults the database name to your username. Pass the database explicitly.

`peer authentication failed` — on Linux, socket connections authenticate by OS user. Use `sudo -u postgres psql`, or connect over TCP with `-h localhost`.

2.3 psql — the commands that matter

`psql` is PostgreSQL's terminal client. Learn these twenty lines and you will rarely need a GUI. Commands beginning with a backslash are psql's own **meta-commands**; they are not SQL and do not travel to other clients.

Meta-command	Does
<code>\l</code>	List databases
<code>\c dbname</code>	Connect to another database
<code>\dn</code>	List schemas
<code>\dt</code>	List tables in the search path
<code>\dt sales.*</code>	List tables in one schema
<code>\d employees</code>	Describe a table — columns, types, indexes, constraints. The one you will use most.
<code>\d+ employees</code>	The same, plus storage and size
<code>\di</code> / <code>\dv</code> / <code>\df</code>	List indexes / views / functions
<code>\du</code>	List roles
<code>\x</code>	Toggle expanded output — essential for wide rows
<code>\timing</code>	Toggle query timing
<code>\i file.sql</code>	Run a script file
<code>\o out.txt</code>	Send output to a file (<code>\o</code> alone returns to screen)
<code>\copy</code>	Import/export CSV using <i>client-side</i> paths (see 2.8)
<code>\e</code>	Edit the last query in your <code>\$EDITOR</code>
<code>\set AUTOCOMMIT off</code>	Require explicit <code>COMMIT</code> — a good habit on shared systems
<code>\pset null 'Ø'</code>	Render <code>NULL</code> visibly instead of as blank
<code>\watch 2</code>	Re-run the last query every 2 seconds
<code>\?</code> / <code>\h SELECT</code>	Meta-command help / syntax help for a SQL command
<code>\q</code>	Quit

BEST PRACTICE · A STARTER `~/ .PSQLRC`

```
\set QUIET 1
\pset null 'Ø'
\timing on
\set COMP_KEYWORD_CASE upper
\set HISTSIZE 5000
\set PROMPT1 '%[%033[1;33m%]n@%/%[%033[0m%]%# '
\set QUIET 0
```

Rendering `NULL` as a visible glyph will save you from a class of bug where an empty string and a `NULL` look identical on screen.

COMMON MISTAKE

Forgetting the semicolon. psql shows a continuation prompt (`dbname-#`) and waits — it has not hung, it is waiting for you to finish the statement. Type `;` and press enter, or `\r` to reset the buffer.

2.4 GUI clients

A GUI is genuinely better for exploring an unfamiliar schema, reading a wide result set, and viewing an execution plan as a diagram. It is worse for reproducibility: what you did lives in a window, not in a file. Use both — GUI to explore, script to commit.

Client	Character	Cost
pgAdmin 4	Official, comprehensive, browser-based; bundled with the Windows installer	Free
DBeaver	Connects to everything — Postgres, MySQL, SQL Server, Oracle, SQLite — one tool for all four dialects in Part 37	Free (CE)
DataGrip	The best SQL editor: refactoring, real completion, inspections	Paid
TablePlus	Fast, minimal, macOS-first	Free tier
VS Code + SQLTools	Queries beside your application code	Free

BEST PRACTICE

Keep every schema change in a numbered `.sql` file in version control, even while learning. “I clicked something in pgAdmin last Tuesday” is not a schema history; `003_add_orders_index.sql` is.

2.5 The other engines, briefly

You will meet these in interviews and inherited systems. Enough to connect and orient yourself:

MYSQL

```
mysql -u root -p
SHOW DATABASES;
USE shopdb;
SHOW TABLES;
DESCRIBE employees;      -- psql's \d
SHOW CREATE TABLE employees\G
```

SQL SERVER

```
sqlcmd -S localhost -U sa -P 'Passw0rd!'
SELECT name FROM sys.databases;
GO
USE shopdb;
SELECT name FROM sys.tables;
GO
EXEC sp_help 'dbo.employees';
GO
```

T-SQL batches are terminated by `GO`, which is a client directive rather than SQL. Docker image:

`mcr.microsoft.com/mssql/server:2022-latest`.

SQLITE

```
sqlite3 practice.db
.tables
.schema employees
.headers on
.mode column
.quit
```

No server, no user, no password — the database is the file. Ideal for the practice problems in Part 34 when you are away from your machine.

ORACLE

```
sqlplus system/password@localhost:1521/FREEPDB1
SELECT table_name FROM user_tables;
DESC employees;
SELECT sysdate FROM dual;
```


DIALECT · DUAL

Oracle historically required a `FROM` clause, so it ships a one-row table called `DUAL` for expressions with no table: `SELECT 1+1 FROM dual`. PostgreSQL, MySQL and SQL Server allow `SELECT 1+1` with no `FROM` at all. Oracle 23c now permits it too.

2.6 Databases, schemas and users

```
-- as a superuser
CREATE DATABASE sqlmastery
  WITH ENCODING 'UTF8'
      LC_COLLATE 'en_US.UTF-8'
      LC_CTYPE 'en_US.UTF-8'
      TEMPLATE template0;

CREATE ROLE learner WITH LOGIN PASSWORD 'learner';
GRANT CONNECT ON DATABASE sqlmastery TO learner;

\c sqlmastery

CREATE SCHEMA core AUTHORIZATION learner;
GRANT USAGE ON SCHEMA core TO learner;
ALTER ROLE learner SET search_path TO core, public;
```

Three details in that script are worth understanding rather than copying. `TEMPLATE template0` gives you a pristine database when you are also setting a non-default collation. **Collation** decides sort order — `'Z' < 'a'` under the C locale but not under `en_US` — and it therefore changes the results of `ORDER BY` and the usability of an index for text comparisons (Part 20). And `search_path` is what lets you write `employees` instead of `core.employees`.

COMMON MISTAKE

Doing all your learning as a superuser, then being surprised in production when a privilege is missing. Create the `learner` role and use it. The five minutes cost you here is the Part 28 chapter you will not have to learn under pressure.

DIALECT

MySQL: `CREATE DATABASE shopdb CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;` then `CREATE USER 'learner'@'%' IDENTIFIED BY '...'; GRANT ALL ON shopdb.* TO 'learner'@'%';` — note that a MySQL user is a name *plus a host*. **SQL Server:** a server-level *login* maps to a database-level *user*; two objects, two steps. **Oracle:** `CREATE USER` creates the schema, and you must `GRANT CREATE SESSION` before the account can even connect.

2.7 Running SQL scripts

```
# from your shell
psql -U learner -d sqlmastery -f schema.sql

# stop at the first error instead of ploughing on
psql -U learner -d sqlmastery -v ON_ERROR_STOP=1 -f schema.sql

# wrap the whole file in one transaction: all of it, or none
psql -U learner -d sqlmastery --single-transaction -f schema.sql

# from inside psql
\i schema.sql
```

BEST PRACTICE

Always run migration scripts with `ON_ERROR_STOP=1` and `--single-transaction`. Without them, a script that fails on statement 4 of 40 leaves your database in a state that exists nowhere in your version control — the worst place a schema can be.

Write scripts to be idempotent where you reasonably can, so re-running one is safe:

```
CREATE TABLE IF NOT EXISTS departments (...);
DROP TABLE IF EXISTS staging_orders;
CREATE INDEX IF NOT EXISTS idx_orders_customer ON orders (customer_id);
ALTER TABLE orders ADD COLUMN IF NOT EXISTS shipped_at timestamptz;
```

2.8 Importing and exporting data

PostgreSQL has two spellings of the same idea, and confusing them is the single most common import failure.

	COPY (SQL)	\copy (psql)
Runs on	The server	The client
Path is	A path on the server's filesystem	A path on your machine
Needs	<code>pg_read_server_files</code> privilege or superuser	Only normal table privileges
Use when	The file is already on the database host	Almost always, while learning

```
-- import (client-side; note the leading backslash and no trailing semicolon needed)
\copy customers FROM '~/data/customers.csv' WITH (FORMAT csv, HEADER true)

-- import a subset of columns, with a custom NULL marker
\copy orders (order_id, customer_id, order_date, total_amount)
      FROM '~/data/orders.csv' WITH (FORMAT csv, HEADER true, NULL 'NA')

-- export a query result
\copy (SELECT * FROM orders WHERE order_date >= '2024-01-01')
      TO '~/out/orders_2024.csv' WITH (FORMAT csv, HEADER true)
```

BACKUP AND RESTORE

```
pg_dump -U learner -d sqlmastery -Fc -f sqlmastery.dump # custom format
pg_restore -U learner -d sqlmastery_copy sqlmastery.dump

pg_dump -U learner -d sqlmastery -f sqlmastery.sql # plain SQL
pg_dump -U learner -d sqlmastery -s -f schema_only.sql # schema only
pg_dump -U learner -d sqlmastery -a -t orders -f orders.sql # data of one table
```

BEST PRACTICE · THE STAGING-TABLE PATTERN

Never load a messy CSV straight into a constrained table — the first bad row aborts the whole load and tells you little. Load into a staging table whose every column is `text`, inspect and clean with SQL, then `INSERT ... SELECT` with casts into the real table. You keep the constraints *and* get a readable list of exactly which rows are wrong.

COMMON MISTAKE · FOUR CSV TRAPS

Encoding. A file saved as Windows-1252 loaded as UTF-8 produces `invalid byte sequence`. Convert first: `iconv -f WINDOWS-1252 -t UTF-8`.

Date format. `03/04/2024` is ambiguous. Set `SET datestyle = 'ISO, DMY';` for the session, or fix the file.

Empty strings vs NULL. In CSV, an unquoted empty field imports as `NULL`; `""` imports as an empty string. They are different values and they sort, compare and aggregate differently.

Thousands separators. `"1,234.50"` will not cast to `numeric`. Strip in the staging step: `replace(amount, ',', '')::numeric`.

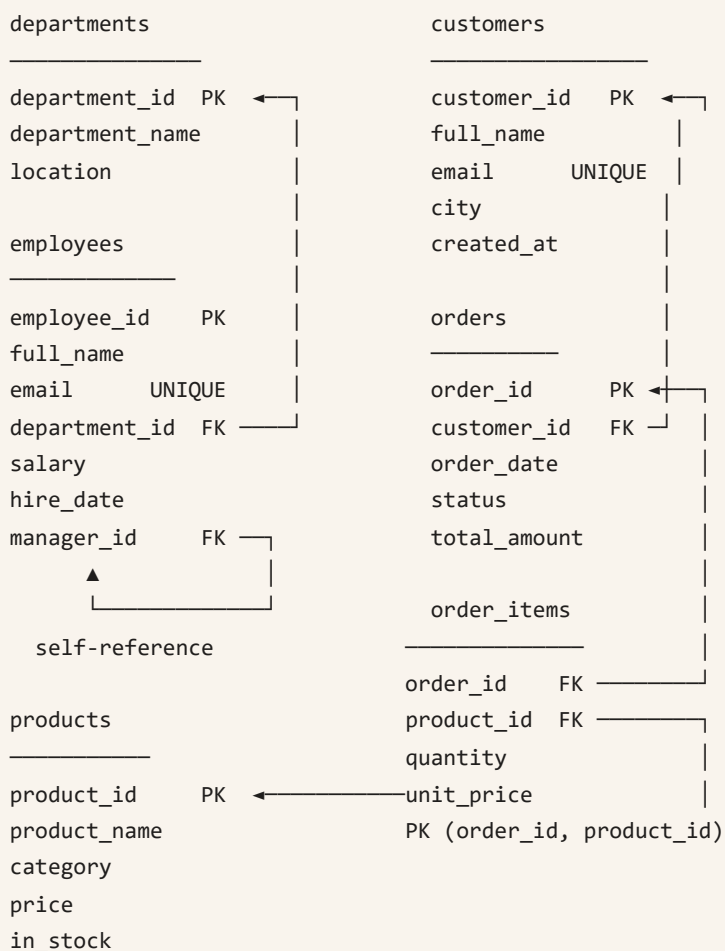
DIALECT

MySQL: `LOAD DATA [LOCAL] INFILE ... INTO TABLE ... FIELDS TERMINATED BY ','` — `LOCAL` makes it client-side, and it must be enabled on both client and server. **SQL Server:** `BULK INSERT`, or the `bcp` utility. **Oracle:** external tables or `SQL*Loader`.

2.9 The practice database

Every example, exercise and solution in this book runs against the six tables below. Learning one schema deeply is a deliberate choice: by Part 13 you will know it well enough to read a query and predict the answer, which is exactly the skill an interview whiteboard tests.

FIGURE 2.9 — THE PRACTICE SCHEMA



Six tables, five relationships: four one-to-many, one self-referencing hierarchy, and one many-to-many resolved by `order_items`.

```
DROP TABLE IF EXISTS order_items, orders, products, employees, departments, customers;

CREATE TABLE departments (
    department_id    integer PRIMARY KEY,
    department_name  text NOT NULL UNIQUE,
    location         text
);

CREATE TABLE employees (
    employee_id      integer PRIMARY KEY,
    full_name        text NOT NULL,
    email            text UNIQUE,
    department_id    integer REFERENCES departments (department_id),
    salary           numeric(10,2) CONSTRAINT employees_salary_positive CHECK (salary > 0),
    hire_date        date NOT NULL DEFAULT current_date,
    manager_id       integer REFERENCES employees (employee_id)
);

CREATE TABLE customers (
    customer_id      integer PRIMARY KEY,
    full_name        text NOT NULL,
    email            text UNIQUE NOT NULL,
    city             text,
    created_at       timestampz NOT NULL DEFAULT now()
);

CREATE TABLE products (
    product_id       integer PRIMARY KEY,
    product_name     text NOT NULL,
    category         text NOT NULL,
    price            numeric(10,2) NOT NULL CHECK (price >= 0),
    in_stock         integer NOT NULL DEFAULT 0 CHECK (in_stock >= 0)
);

CREATE TABLE orders (
    order_id         integer PRIMARY KEY,
    customer_id      integer NOT NULL REFERENCES customers (customer_id),
    order_date       date NOT NULL,
    status           text NOT NULL CHECK (status IN
('pending','paid','shipped','delivered','cancelled')),
    total_amount     numeric(12,2) NOT NULL CHECK (total_amount >= 0)
);

CREATE TABLE order_items (
    order_id         integer NOT NULL REFERENCES orders (order_id) ON DELETE CASCADE,
    product_id       integer NOT NULL REFERENCES products (product_id),
    quantity         integer NOT NULL CHECK (quantity > 0),
    unit_price       numeric(10,2) NOT NULL CHECK (unit_price >= 0),
```

```
PRIMARY KEY (order_id, product_id)
);
```

Read that script as a design document, not just as setup. Every choice in it is an argument made in Part 1: `manager_id` is nullable because the CEO has no manager; `order_items` has a composite primary key because the same product appears once per order; `status` carries a `CHECK` rather than a lookup table because the set is small and stable; `ON DELETE CASCADE` appears on order items because a line has no meaning without its order, and appears nowhere else because nothing else should vanish silently.


```
INSERT INTO departments (department_id, department_name, location) VALUES
(1, 'Engineering', 'Bengaluru'),
(2, 'Sales', 'Mumbai'),
(3, 'Support', 'Pune'),
(4, 'Finance', 'Bengaluru'),
(5, 'Research', NULL); -- a department with no employees, on purpose
```

```
INSERT INTO employees (employee_id, full_name, email, department_id, salary, hire_date,
manager_id) VALUES
(101, 'Asha Menon', 'asha@example.com', 1, 92000.00, '2019-03-11', NULL),
(102, 'Rahul Iyer', 'rahul@example.com', 1, 64000.00, '2021-07-01', 101),
(103, 'Meera Nair', 'meera@example.com', 2, 71500.00, '2020-01-20', 101),
(104, 'Vikram Rao', 'vikram@example.com', 2, 58000.00, '2022-11-15', 103),
(105, 'Sneha Das', 'sneha@example.com', 3, 47000.00, '2023-02-06', 103),
(106, 'Imran Khan', 'imran@example.com', 1, 88000.00, '2018-06-25', 101),
(107, 'Divya Pillai', 'divya@example.com', 3, 52000.00, '2021-09-13', 105),
(108, 'Arjun Shetty', 'arjun@example.com', 4, 76000.00, '2020-05-04', 101),
(109, 'Nisha Verma', NULL, NULL, 45000.00, '2024-01-08', 108); -- no dept,
no email
```

```
INSERT INTO customers (customer_id, full_name, email, city, created_at) VALUES
(1, 'Kavya Reddy', 'kavya@example.com', 'Hyderabad', '2023-01-14'),
(2, 'Rohan Gupta', 'rohan@example.com', 'Delhi', '2023-03-02'),
(3, 'Fatima Sheikh', 'fatima@example.com', 'Mumbai', '2023-06-19'),
(4, 'Daniel Fernandes', 'daniel@example.com', 'Goa', '2024-02-27'),
(5, 'Priya Anand', 'priya@example.com', 'Hyderabad', '2024-05-30'); -- never orders
```

```
INSERT INTO products (product_id, product_name, category, price, in_stock) VALUES
(10, 'Mechanical Keyboard', 'Peripherals', 5400.00, 42),
(11, '27-inch Monitor', 'Displays', 18900.00, 15),
(12, 'USB-C Hub', 'Peripherals', 2300.00, 0),
(13, 'Laptop Stand', 'Accessories', 1750.00, 88),
(14, 'Noise-cancelling Headset', 'Audio', 9200.00, 7),
(15, 'Webcam 1080p', 'Peripherals', 3100.00, 23);
```

```
INSERT INTO orders (order_id, customer_id, order_date, status, total_amount) VALUES
(1001, 1, '2024-01-05', 'delivered', 24300.00),
(1002, 2, '2024-01-19', 'delivered', 5400.00),
(1003, 1, '2024-02-11', 'shipped', 11000.00),
(1004, 3, '2024-03-03', 'cancelled', 9200.00),
(1005, 2, '2024-03-28', 'paid', 21200.00),
(1006, 4, '2024-06-14', 'pending', 3100.00),
(1007, 1, '2024-07-02', 'delivered', 1750.00);
```

```
INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES
(1001, 10, 1, 5400.00), (1001, 11, 1, 18900.00),
(1002, 10, 1, 5400.00),
(1003, 14, 1, 9200.00), (1003, 13, 1, 1750.00),
```

```
(1004,14,1,9200.00),
(1005,11,1,18900.00),(1005,13,1,1750.00),(1005,12,1,2300.00),
(1006,15,1,3100.00),
(1007,13,1,1750.00);
```

The data is small on purpose — nine employees, five customers, seven orders — so you can verify any result by hand. It is also **deliberately awkward** in six places, and each awkwardness exists to teach something specific later:

Deliberate quirk	Teaches	Part
Department 5 has no employees	Why <code>LEFT JOIN</code> and <code>INNER JOIN</code> disagree	9
Employee 109 has no department and no email	<code>NULL</code> in joins, <code>COUNT(col)</code> vs <code>COUNT(*)</code>	8, 9
Employee 101 has no manager	Self-joins and recursive hierarchy traversal	9, 11
Customer 5 has never ordered	<code>NOT EXISTS</code> vs <code>NOT IN</code> vs anti-join	10
Order 1004 is cancelled	Why revenue queries need a status filter	6, 8
Product 12 has zero stock; product 12 is on a paid order	Filtering vs aggregating, and conditional aggregation	8, 14

COMMON MISTAKE

`total_amount` on `orders` duplicates the sum of its items — a denormalization. It is here on purpose: real systems store the charged total because prices change, and Part 16 discusses exactly when that is correct. Do not assume the two always agree; order 1003's total is deliberately not the sum of its lines, and Part 36 asks you to find that.

LOAD IT

```
psql -U learner -d sqlmastery -v ON_ERROR_STOP=1 --single-transaction \
-f practice_schema.sql -f practice_data.sql
```

VERIFY IT — YOUR FIRST THREE QUERIES

Run these now. They confirm the load and give you the shape of every table's row count.

```

SELECT count(*) AS employees FROM employees;
SELECT count(*) AS orders, sum(total_amount) AS gross FROM orders;

-- one row per table, with its live row count
SELECT relname AS table_name, n_live_tup AS approx_rows
FROM   pg_stat_user_tables
ORDER  BY relname;

```

```

employees
-----
          9

orders | gross
-----+-----
       7 | 75950.00

table_name | approx_rows
-----+-----
customers  |           5
departments |           5
employees  |           9
order_items |          11
orders     |           7
products   |           6

```

BEST PRACTICE

Keep `practice_schema.sql` and `practice_data.sql` in a folder you can re-run in one command. You will break this database on purpose many times — during the transaction experiments of Part 18, the locking demonstrations of Part 19, and the index rebuilds of Part 20. A ten-second reset removes all fear of experimenting, and fearless experimenting is how the rest of this book works.

INTERVIEW · BEGINNER

Q. How would you explore a database you have never seen before? — A practical answer beats a theoretical one: list the tables and their sizes; read the largest few with `\d` to see keys, foreign keys and indexes; follow the foreign keys to reconstruct the entity map; then `SELECT * ... LIMIT 20` on each to see what the data actually looks like versus what the types allow. Mention that you check `NULL` rates and distinct counts on the columns you intend to filter on, because that is what determines whether an index will help.

Part 2 Review

This part is hands-on, so the review is too. Do it at a keyboard, with the practice database loaded.

Quick Check ● BEGINNER

- 1 Which `psql` meta-command shows a table's columns, indexes and constraints?
- 2 What is the difference between `COPY` and `\copy` ?
- 3 What does `ON_ERROR_STOP=1` change, and why does it matter for migrations?
- 4 Why does `psql` sometimes report that a database named after you does not exist?
- 5 What does collation affect, and name two places you would notice it.
- 6 What is `search_path` for?
- 7 Name MySQL's, SQL Server's and Oracle's equivalents of `\d tablename` .
- 8 In a CSV, how do an empty field and `""` import differently?

Practice ● EASY

- 1 Install PostgreSQL by any route, then prove it: report the output of `SELECT version(), current_user, current_database();`
- 2 Create the `sqlmastery` database and the `learner` role, and connect as `learner` .
- 3 Load the practice schema and data. Confirm the six row counts match the output in 2.9.
- 4 Run `\d employees` . List every constraint you can see, and match each one to its line in the schema script.
- 5 Write a `~/.psqlrc` with `\timing` on and a visible `NULL` marker. Prove it works by selecting employee 109.
- 6 Export `orders` to CSV with headers, open it in a text editor, and confirm what a `NULL` looks like in the file.
- 7 Deliberately break the load: change a `department_id` in `practice_data.sql` to 99 and re-run. Read the error and name the constraint that caught it.
- 8 Take a `pg_dump` in custom format, drop the database, and restore it.
- 9 Create a `staging_orders` table with all- `text` columns, load the CSV from exercise 6 into it, then `INSERT ... SELECT` with casts into a new real table.
- 10 Create a schema called `scratch` , move nothing into it, and set your `search_path` so that `scratch` is searched first. Explain what now happens if you `CREATE TABLE employees (...)` .

Challenge ● INTERMEDIATE

- 1 Write a single shell command that drops, recreates and reloads the practice database from scratch, stopping on any error. This is the reset you will use for the rest of the book.

- 2 You are handed a 2 GB CSV with inconsistent date formats and thousands separators in the amount column. Describe your load procedure step by step, and say where each of the four CSV traps is handled.
- 3 Load the same practice schema into SQLite. List every statement you had to change and why.

Interview Drill ● ADVANCED

- 1 How do you explore an unfamiliar production database safely?
- 2 Why should schema changes live in version-controlled files rather than a GUI?
- 3 What would you check before running an import against a production table?
- 4 Explain the difference between a role, a user and a login across PostgreSQL, MySQL and SQL Server.
- 5 Why avoid doing routine work as a database superuser?

Chapter Test ● INTERMEDIATE

- 1 Which of `COPY` / `\copy` needs a server-side privilege, and why?
- 2 Give the four levels of PostgreSQL containment and where a foreign key can and cannot reach across them.
- 3 Name three things `--single-transaction` protects you from.
- 4 What is `DUAL` and which engine needs it?
- 5 Write the idempotent form of `CREATE INDEX`, `DROP TABLE` and `ADD COLUMN`.
- 6 Why is a MySQL user identified by a name *and* a host?
- 7 In the practice schema, why is `manager_id` nullable but `orders.customer_id` not?
- 8 Why does `order_items` carry `ON DELETE CASCADE` while `orders.customer_id` does not?
- 9 Name two reasons the practice data includes a department with no employees.
- 10 What is a staging table and what problem does it solve?

Part 2 Solutions

Quick Check

- 1 `\d tablename`, or `\d+` for storage and size as well.
- 2 `COPY` runs on the server and reads server paths, requiring superuser or the `pg_read_server_files` role. `\copy` is a psql meta-command that reads your local filesystem and streams the data over the connection, needing only table privileges.
- 3 Without it, psql reports an error and continues with the next statement, leaving a half-applied schema. With it, the script aborts at the first failure — and combined with `--single-transaction`, rolls back entirely.
- 4 Because psql defaults both the role and the database name to your operating-system username. Pass `-U` and `-d` explicitly.
- 5 Collation determines string comparison and sort order. You notice it in `ORDER BY` results (case and accent handling) and in whether a B-tree index can serve a `LIKE 'abc%'` or a range comparison — an index built under one collation is unusable for comparisons under another.
- 6 It is the ordered list of schemas searched when you write an unqualified object name, and it also decides which schema a `CREATE TABLE` without a schema prefix lands in.
- 7 MySQL: `DESCRIBE t;` or `SHOW CREATE TABLE t;`. SQL Server: `EXEC sp_help 'dbo.t';`. Oracle: `DESC t;`.
- 8 An unquoted empty field becomes `NULL`; `""` becomes a zero-length string. `NULL` is skipped by aggregates and fails equality; an empty string is a real value that compares and counts.

Practice — worked

4 • **Reading** `\d employees`. You should be able to point at each of: the primary key on `employee_id`; a unique constraint (and its implicit index) on `email`; two foreign keys, one to `departments` and one back to `employees`; a `NOT NULL` on `full_name` and on `hire_date`; a default of `current_date` on `hire_date`; and the named check `employees_salary_positive`. Note what is *not* there: no index on `department_id`, even though it is a foreign key. PostgreSQL indexes the referenced side automatically, never the referencing side — a frequent cause of slow joins and slow cascading deletes, and a Part 20 exercise.

7 • **The broken load.** The error names the constraint directly:

```
ERROR: insert or update on table "employees" violates foreign key constraint
       "employees_department_id_fkey"
DETAIL: Key (department_id)=(99) is not present in table "departments".
```

Two things to take from it. First, the generated constraint name follows the pattern `table_column_fkey` — which is why explicitly naming your important constraints (as the schema does for `employees_salary_positive`) makes errors self-documenting. Second, with `--single-transaction` the whole load rolled back, so the database is exactly as it was. Confirm that: `SELECT count(*) FROM employees;` returns 0, not 8.

9 · The staging pattern.

```
CREATE TABLE staging_orders (  
    order_id text, customer_id text, order_date text, status text, total_amount text  
);  
\copy staging_orders FROM '~/out/orders.csv' WITH (FORMAT csv, HEADER true)  
  
-- inspect before trusting: which rows would fail a cast?  
SELECT * FROM staging_orders WHERE order_date !~ '^\d{4}-\d{2}-\d{2}$';  
SELECT * FROM staging_orders WHERE total_amount ~ '^[^0-9.]';  
  
-- then load for real  
INSERT INTO orders (order_id, customer_id, order_date, status, total_amount)  
SELECT order_id::integer,  
       customer_id::integer,  
       order_date::date,  
       status,  
       replace(total_amount, ',', '')::numeric(12,2)  
FROM   staging_orders;
```

The value of the pattern is in the two `SELECT` s in the middle. A direct `\copy` into `orders` aborts on the first bad row and tells you about that row only; staging gives you the complete list of problems in one pass, before you have touched the real table.

10 · `search_path` and `CREATE TABLE`. `CREATE TABLE employees (...)` creates `scratch.employees`, because unqualified creation targets the *first* schema in the path. You now have two tables called `employees`, and every unqualified query resolves to the empty one in `scratch` — which looks exactly like your data having vanished. This is the mechanism behind a real security issue too: a writable schema early in the search path lets an attacker shadow a function you call. Part 28 returns to it.

Challenge — worked

1 · **The reset command.** Keep this in a file called `reset.sh`; you will run it dozens of times.

```
#!/usr/bin/env bash  
set -euo pipefail  
dropdb --if-exists sqlmastery  
createdb sqlmastery  
psql -d sqlmastery -v ON_ERROR_STOP=1 --single-transaction \  
    -f practice_schema.sql -f practice_data.sql  
echo "practice database reset"
```

`set -euo pipefail` is the shell's equivalent of `ON_ERROR_STOP`: without it, a failed `createdb` is followed cheerfully by a load into the wrong database.

2 • The 2 GB messy CSV. (i) Inspect the first and last few hundred lines and confirm the encoding with `file`; convert with `iconv` if it is not UTF-8 — *trap 1 handled*. (ii) Create an all-`text` staging table and `\copy` into it; at this size, drop indexes on the target first and recreate them after. (iii) Profile the mess with SQL: count rows failing a date regex, count rows with non-numeric characters in the amount, count distinct values in every low-cardinality column. (iv) Normalise inside the database: `to_date(order_date, 'DD/MM/YYYY')` for the known-ambiguous format rather than relying on `datestyle`, which is a session setting a colleague can change — *trap 2 handled*; `nullif(trim(col), '')` to make blanks explicit — *trap 3*; `replace(amount, ',', '')::numeric` — *trap 4*. (v) `INSERT ... SELECT` into the real table inside a transaction, in batches if you need to bound lock duration. (vi) Reconcile: row counts and a checksum sum of the amount column, staging versus target. Step (vi) is the one people skip and the one that catches silent losses.

3 • The same schema in SQLite. Changes required: `text` and `integer` are fine, but `numeric(10,2)` is accepted and then ignored — SQLite has dynamic typing and no fixed-point type, so money becomes `REAL` or an integer count of paise. `timestampz` does not exist; store ISO-8601 text or a Unix epoch integer. `DEFAULT current_date` becomes `DEFAULT CURRENT_DATE` (supported) but `now()` becomes `CURRENT_TIMESTAMP`. Foreign keys are *declared but not enforced* unless you run `PRAGMA foreign_keys = ON`; in every connection — the single most surprising difference, and the reason a schema can look correct in SQLite and still admit orphan rows. The multi-table `DROP TABLE IF EXISTS a, b, c` must be split into separate statements. The lesson worth carrying: portability is mostly about types and enforcement, not about syntax.

Chapter Test

- 1** `COPY`, because it reads and writes files on the database host — an unrestricted version would let any user read the server's filesystem.
- 2** Cluster → database → schema → object. A foreign key may cross schemas within one database; it may not cross databases.
- 3** A partially applied schema; an inconsistent state that exists in no version-controlled file; and having to write a manual undo script under pressure.
- 4** A one-row dummy table in Oracle, used when an expression needs a `FROM` clause. PostgreSQL, MySQL and SQL Server do not need it.
- 5** `CREATE INDEX IF NOT EXISTS ...`, `DROP TABLE IF EXISTS ...`, `ALTER TABLE ... ADD COLUMN IF NOT EXISTS ...`.
- 6** Because MySQL's access control is per name-and-origin: `'app'@'10.0.%'` and `'app'@'localhost'` are two distinct accounts with independent passwords and privileges.
- 7** Optionality follows the business rule. Some employee has no manager (the top of the hierarchy), so the column must allow `NULL`. Every order has a customer by definition, so that column is `NOT NULL`.
- 8** An order line has no meaning without its order, so cascading the delete is correct. A customer's orders are financial records that must not disappear because an account was removed — deleting such a customer should fail, forcing an explicit decision.

- 9 To make `INNER JOIN` and `LEFT JOIN` produce visibly different results, and to expose the `COUNT(*)` - returns-1-for-an-empty-group trap in Part 8.
- 10 An unconstrained all-`text` table used as a landing zone for imported data. It lets you find every bad row with SQL before casting, instead of discovering them one aborted load at a time.

Where you are, and what comes next

You now hold the two things that make the rest of this book possible: the relational model, and a working database with a schema you will use for the next thirty-eight parts. Before moving on, confirm all three of the following are true — if any is not, the fix is cheaper now than in Part 9.

1. You can state the nine-step logical evaluation order of a `SELECT` without looking.
2. You can draw the practice schema's six tables and five relationships from memory.
3. You can reset the practice database to a known state in one command.

NEXT

Volume II — Parts 3 to 8. The SQL basics, in the nine-beat format used throughout: `CREATE`, `INSERT`, `SELECT`, `UPDATE`, `DELETE` and the DDL family; the full data-type reference; operators and three-valued logic; filtering, sorting and both styles of pagination; the string, numeric, date and `NULL` function libraries; and aggregation up to grouping sets. It ends with the first cumulative test covering Parts 1 to 8.

Say the word and I will write it in the same format and at the same depth.



SQL MASTERY

Volume II — Core SQL

Parts 3 to 8. The statements, the type system, the logic, and the two operations — filtering and aggregation — that everything after this is built on.

-
- 03** **SQL Basics** — the statements you will write for the rest of your career

 - 04** **Data Types** — choosing the right one, and what it costs when you don't

 - 05** **Operators & Expressions** — three-valued logic, `CASE`, `COALESCE`

 - 06** **Filtering & Sorting** — `WHERE`, `ORDER BY`, and pagination that survives scale

 - 07** **Functions** — string, numeric, date/time, and `NULL` handling

 - 08** **Aggregations** — `GROUP BY`, `HAVING`, grouping sets

 - ★ **Cumulative Test** — Parts 1 to 8, with full solutions

Every example runs on the six-table practice database from Volume I, §2.9. If you have not loaded it, do that first — this volume assumes it.

SQL Basics

Twelve statements cover ninety percent of all SQL ever written. This part teaches each one properly — not just its syntax, but what it locks, what it logs, how it fails, and how to write it so that a colleague reading it at 2 a.m. understands it immediately.

IN THIS PART

3.1 SELECT — the shape of every query

3.2 Aliases and comments

3.3 DISTINCT

3.4 WHERE, ORDER BY, LIMIT — first pass

3.5 INSERT

3.6 UPDATE

3.7 DELETE and TRUNCATE

3.8 CREATE TABLE

3.9 ALTER TABLE

3.10 DROP and the destructive family

3.11 A style guide for readable SQL

Review · Solutions

3.1 SELECT — the shape of every query

`SELECT` is the only statement that reads data, and every other idea in this book is a clause bolted onto it. Learn its skeleton once and you can read any query, however long, by finding the clauses in order.

```
SELECT  [DISTINCT] select_list
FROM    table_or_join
WHERE    row_condition
GROUP BY grouping_columns
HAVING  group_condition
ORDER BY sort_columns [ASC|DESC] [NULLS FIRST|LAST]
LIMIT   n OFFSET m;
```

Only `SELECT` is mandatory. `SELECT 1 + 1;` is a complete, valid statement in PostgreSQL, MySQL and SQL Server — useful for testing an expression without a table in the way.

EXAMPLE — THE FOUR THINGS A SELECT LIST CAN HOLD

```
SELECT full_name,          -- 1. a column
       salary * 12,        -- 2. an expression
       upper(city),        -- 3. a function call
       'active'             -- 4. a literal
FROM   customers
LIMIT 3;
```

full_name	?column?	upper	?column?
Kavya Reddy		HYDERABAD	active
Rohan Gupta		DELHI	active
Fatima Sheikh		MUMBAI	active

Note `?column?`. PostgreSQL cannot name a computed column, so it emits a placeholder. Every computed column should be aliased — that is §3.2, and it is not optional in professional work.

ON `SELECT *`

`SELECT *` is right for exactly one thing: exploring a table by hand. In anything saved to a file it is a defect, for four concrete reasons.

- **It breaks silently on schema change.** A colleague adds a column; your application receives an extra field, your `INSERT INTO ... SELECT *` now mismatches, your CSV export gains a column no downstream consumer expects.
- **It moves data you do not need.** A `text` column holding 40 KB of description is fetched across the network for a query that only wanted the id.

- **It defeats covering indexes.** An index containing every column you asked for can answer a query without touching the table at all (Part 20). `*` guarantees you touch the table.
- **It hides intent.** A reader cannot tell which columns matter, so nobody can safely drop the unused one.

INTERVIEW · BEGINNER

Q. Why avoid `SELECT *` in production code? — Give two of the four reasons above, and then the nuance that marks experience: “I use it constantly in `psql` while exploring; I never commit it. The exception is `SELECT count(*)`, which is not the same thing at all — the star there means ‘rows’, not ‘columns’, and it is the correct idiom.”

REAL-WORLD USE CASE

An order-confirmation email needs six fields from three tables. Written with explicit columns, the query documents the email's data contract; a year later, someone can delete an unused column with confidence because `grep` finds every place it is read. Written with `*`, nothing is ever safely deletable, and schemas accumulate dead columns forever.

3.2 Aliases and comments

Aliases rename columns and tables for the duration of a query. They are the cheapest readability tool SQL has.

```
SELECT e.full_name      AS employee,  
       e.salary * 12    AS annual_salary,  
       d.department_name AS department  
FROM   employees AS e  
JOIN   departments AS d ON d.department_id = e.department_id  
ORDER BY annual_salary DESC;
```

The `AS` keyword is optional for both column and table aliases in PostgreSQL and MySQL (`employees e` works). Keep it for columns, where it makes the boundary obvious; drop it for tables if your team prefers. Consistency matters more than which you pick.

COMMON MISTAKE · THE MISSING COMMA

`SELECT full_name salary FROM employees` is not an error. SQL reads it as “column `full_name`, aliased `salary`” and returns one column of names under the heading `salary`. A dropped comma silently deletes a column from your result. When a query returns fewer columns than you expected, look for this first.

WHEN AN ALIAS NEEDS QUOTES

```
SELECT full_name AS "Employee Name",    -- spaces or mixed case need double quotes  
       salary    AS "Monthly Salary (INR)"  
FROM   employees;
```

Double quotes make an identifier case-sensitive and literal. Single quotes are for *string values* only — `WHERE city = 'Mumbai'`. Confusing the two produces the beginner's most common error message, `column "Mumbai" does not exist`, which is the database telling you it read your double-quoted value as a column name.

DIALECT

MySQL uses backticks (``Employee Name``) by default, though it accepts double quotes in ANSI mode. SQL Server uses square brackets (`[Employee Name]`) or double quotes. PostgreSQL and Oracle use double quotes and *fold unquoted identifiers to lower case* (Oracle to upper case) — which is why `SELECT FullName` can fail against a table created as `"FullName"`. Avoid quoted identifiers entirely; use `snake_case` and the problem never arises.

COMMENTS

```
-- single-line comment, to end of line

/* multi-line comment
   spanning several lines */

SELECT full_name          -- who
FROM   employees
WHERE  hire_date >= '2022-01-01';  /* joined in the last two years */
```

BEST PRACTICE

Comment the *why*, never the *what*. `-- select the name` is noise. `-- exclude cancelled: finance` counts them as never having happened is the sentence that saves the next reader an hour and a wrong report.

3.3 DISTINCT

`DISTINCT` removes duplicate rows from a result. The critical point, and the one interviews probe: **it applies to the whole select list, not to one column.**

```
SELECT DISTINCT city FROM customers ORDER BY city;
```

```
city
-----
Delhi
Goa
Hyderabad
Mumbai
(4 rows)
```

Five customers, four cities — Hyderabad appears twice and is collapsed. Now add a second column:

```
SELECT DISTINCT city, full_name FROM customers ORDER BY city;
```

```
city | full_name
-----+-----
Delhi | Rohan Gupta
Goa   | Daniel Fernandes
Hyderabad | Kavya Reddy
Hyderabad | Priya Anand
Mumbai | Fatima Sheikh
(5 rows)
```

Hyderabad is back twice, because the *pairs* differ. Nothing is wrong; `DISTINCT` did exactly what it says. But a beginner who wanted “one row per city, with a name” has silently got the wrong answer — and this is the mechanism behind the majority of “my count is too high” bugs. The correct tools for one-row-per-group are `GROUP BY` (Part 8) or `DISTINCT ON` (Part 14).

COMMON MISTAKE

`SELECT DISTINCT(city), full_name ...` looks like it distincts only `city`. It does not — the parentheses are just grouping an expression, and the statement is identical to the one above.

`DISTINCT` is never a function.

BEST PRACTICE

Treat `DISTINCT` appearing in a query you are debugging as a smell, not a fix. It usually means a join is producing duplicates you have not understood (Part 9). Removing the duplicates hides the bug; understanding the join removes it — and is faster, since `DISTINCT` forces a sort or hash over the entire result.

INTERVIEW · EASY

Q. `DISTINCT` vs `GROUP BY` — is there a difference? — For deduplication alone, they produce the same result and usually the same plan. The difference is intent and capability: `GROUP BY` forms groups you can then aggregate over and filter with `HAVING`; `DISTINCT` only removes duplicate output rows. Use `DISTINCT` when you want unique values, `GROUP BY` when you want per-group calculations.

3.4 WHERE, ORDER BY, LIMIT — first pass

Part 6 treats these in depth. Here is the working minimum, so the rest of Part 3 has something to filter with.

```
SELECT full_name, salary, hire_date
FROM   employees
WHERE  salary > 60000
      AND hire_date < '2022-01-01'
ORDER BY salary DESC
LIMIT 3;
```

full_name	salary	hire_date
Asha Menon	92000.00	2019-03-11
Imran Khan	88000.00	2018-06-25
Arjun Shetty	76000.00	2020-05-04

Three rules to internalise now, each explained fully later:

1. **WHERE runs before SELECT**. You cannot filter on a select-list alias (Volume I, §1.11).
2. **LIMIT without ORDER BY is non-deterministic**. And an **ORDER BY** with ties is only partly deterministic — add the primary key as a final tie-break when the order must be reproducible.
3. **NULL never satisfies a comparison**. **WHERE department_id != 1** excludes employee 109, whose department is **NULL** — not because 109 is in department 1, but because **NULL != 1** is *unknown*, not true. §5.5 makes this precise; it is the single most expensive misunderstanding in SQL.

DIALECT · ROW LIMITING

Engine	Syntax
PostgreSQL	<code>LIMIT 10 OFFSET 20</code> — also supports the standard <code>OFFSET 20 ROWS FETCH FIRST 10 ROWS ONLY</code>
MySQL	<code>LIMIT 10 OFFSET 20</code> , or the reversed <code>LIMIT 20, 10</code>
SQL Server	<code>SELECT TOP 10 ...</code> , or <code>ORDER BY ... OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY</code> (the <code>ORDER BY</code> is required)
Oracle 12c+	<code>OFFSET 20 ROWS FETCH FIRST 10 ROWS ONLY</code> ; older code uses <code>WHERE ROWNUM <= 10</code>

3.5 INSERT

SYNTAX

```
INSERT INTO table (column_list) VALUES (value_list);
```

Always write the column list. Without it, the statement depends on the physical column order of the table — so adding a column anywhere breaks every columnless `INSERT` in your codebase, and reordering silently loads the wrong values into the wrong columns.

```
-- single row
INSERT INTO departments (department_id, department_name, location)
VALUES (6, 'Marketing', 'Chennai');

-- many rows in one statement: one round trip, one transaction, far faster
INSERT INTO products (product_id, product_name, category, price, in_stock) VALUES
  (16, 'Ergonomic Mouse', 'Peripherals', 2900.00, 31),
  (17, 'Desk Lamp', 'Accessories', 1450.00, 12),
  (18, 'Cable Organiser', 'Accessories', 399.00, 0);

-- from a query: the workhorse of every data pipeline
INSERT INTO archived_orders (order_id, customer_id, order_date, total_amount)
SELECT order_id, customer_id, order_date, total_amount
FROM orders
WHERE order_date < '2024-01-01';
```

RETURNING — GET THE GENERATED KEY BACK

```
INSERT INTO customers (customer_id, full_name, email, city)
VALUES (6, 'Ananya Bose', 'ananya@example.com', 'Kolkata')
RETURNING customer_id, created_at;
```

```
customer_id |          created_at
-----+-----
          6 | 2024-08-11 09:14:22.481+05:30
```

This is one of PostgreSQL's most useful extensions. Without it, inserting a parent row and then its children requires a second query to discover the generated id — two round trips and a race condition if you use `max(id)` to find it. `RETURNING` works on `UPDATE` and `DELETE` too.

CONFLICT HANDLING — UPSERT

```
-- ignore the row if it already exists
INSERT INTO customers (customer_id, full_name, email, city)
VALUES (1, 'Kavya Reddy', 'kavya@example.com', 'Hyderabad')
ON CONFLICT (customer_id) DO NOTHING;

-- or update the existing row: "upsert"
INSERT INTO products (product_id, product_name, category, price, in_stock)
VALUES (10, 'Mechanical Keyboard', 'Peripherals', 5100.00, 50)
ON CONFLICT (product_id)
DO UPDATE SET price      = EXCLUDED.price,
               in_stock = EXCLUDED.in_stock;
```

EXCLUDED is the pseudo-table holding the row you *tried* to insert. That is what lets you write “use the new price” without repeating the value.

DIALECT · UPSERT

PostgreSQL: `ON CONFLICT ... DO UPDATE` . **MySQL:** `INSERT ... ON DUPLICATE KEY UPDATE price = VALUES(price)` — note it reacts to *any* unique key, not one you name, which makes it less predictable on multi-key tables. **SQL Server / Oracle:** the standard `MERGE` statement. **SQLite:** follows PostgreSQL's syntax. There is no portable upsert; this is a genuine migration hazard.

COMMON MISTAKE

Inserting rows one at a time in a loop from application code. Ten thousand single-row inserts is ten thousand round trips and ten thousand transactions; the same data in batched multi-row inserts is typically 50–100× faster. If you are loading a file, use `COPY` instead — faster again by another order of magnitude.

INTERVIEW · INTERMEDIATE

Q. How do you insert a row only if it does not already exist? — `ON CONFLICT (key) DO NOTHING` . The follow-up that catches people: *why not* `SELECT first, then INSERT if absent?` Because between the two statements another session can insert the same key — a check-then-act race. `ON CONFLICT` is atomic; the two-statement version needs a lock or a retry loop.

3.6 UPDATE

```
UPDATE table SET column = value [, ...] [FROM other] WHERE condition;
```

```
-- a 10% raise for engineering
UPDATE employees
SET    salary = salary * 1.10
WHERE  department_id = 1;
-- UPDATE 3

-- several columns at once
UPDATE customers
SET    city      = 'Bengaluru',
       created_at = now()
WHERE  customer_id = 4
RETURNING customer_id, city;
```

UPDATE WITH A JOIN

Setting one table's values from another is a daily task and the syntax differs sharply between engines.

```
-- PostgreSQL: UPDATE ... FROM
UPDATE orders o
SET    total_amount = i.line_total
FROM  (SELECT order_id, sum(quantity * unit_price) AS line_total
       FROM    order_items
       GROUP BY order_id) AS i
WHERE  i.order_id = o.order_id
AND    o.total_amount <> i.line_total;
```

DIALECT

MySQL: `UPDATE orders o JOIN (...) i ON i.order_id = o.order_id SET o.total_amount = i.line_total;` — the join goes before `SET`. **SQL Server:** `UPDATE o SET total_amount = i.line_total FROM orders o JOIN (...) i ON ...`. **Oracle:** no `FROM` clause at all — use a correlated scalar subquery in `SET`, or `MERGE`. This is one of the least portable statements in SQL.

COMMON MISTAKE · THE MISSING WHERE

`UPDATE employees SET salary = 50000;` is valid SQL. It updates every row, and the database will not ask whether you meant it. This has ended careers. Two habits prevent it permanently:

```
-- 1. Write it as a SELECT first, and read the row count.
SELECT count(*) FROM employees WHERE department_id = 1;

-- 2. Wrap it, verify, then decide.
BEGIN;
UPDATE employees SET salary = salary * 1.10 WHERE department_id = 1;
-- UPDATE 3    <- is that the number you expected?
COMMIT;    -- or ROLLBACK;
```

BEST PRACTICE

Make destructive statements *idempotent* where you can. `SET status = 'shipped' WHERE status = 'paid'` is safe to run twice; `SET attempts = attempts + 1` is not. When a script may be retried after a network failure, that distinction decides whether the retry is harmless.

INTERVIEW · INTERMEDIATE

Q. An `UPDATE` on a million-row table is holding locks for minutes. What do you do? — Batch it: update in chunks of a few thousand rows keyed on the primary key, committing between batches, so no single transaction holds locks long. Mention the secondary effects: in PostgreSQL every update writes a new row version, so a mass update bloats the table and the indexes and needs a `VACUUM` afterwards (Part 22); and if the column is indexed, every batch also rewrites index entries.

3.7 DELETE and TRUNCATE

```
DELETE FROM orders WHERE status = 'cancelled';
-- DELETE 1

DELETE FROM orders
WHERE customer_id IN (SELECT customer_id FROM customers WHERE city = 'Goa')
RETURNING order_id;

TRUNCATE TABLE staging_orders; -- all rows, instantly
TRUNCATE TABLE orders, order_items RESTART IDENTITY; -- several tables, reset sequences
```

	DELETE	TRUNCATE
Family	DML	DDL
Filter rows	Yes — <code>WHERE</code>	No — all rows or nothing
Speed on large tables	Slow: row by row, each logged	Near-instant: deallocates storage
Row triggers	Fire	Do not fire (statement triggers may)
Identity/sequence	Untouched	Optionally reset
Reclaims space	Not immediately (needs <code>VACUUM</code>)	Immediately
Rollback	Always	PostgreSQL and SQL Server yes; MySQL and Oracle no
Foreign keys	Blocked or cascaded per the constraint	Refused unless referencing tables are truncated too

COMMON MISTAKE

Expecting `DELETE` to free disk space. In PostgreSQL a deleted row is only marked dead; space is reclaimed by `VACUUM` and returned to the operating system only by `VACUUM FULL` (which locks the table exclusively). Deleting 90% of a huge table and finding the disk unchanged is not a bug — it is Part 22.

BEST PRACTICE · SOFT DELETES

For anything a user can undo or an auditor may ask about, do not delete: add `deleted_at timestampz` and filter it out. You keep history and referential integrity. The cost is that *every* query must remember the filter — so put it behind a view, and be honest that forgetting it once means showing deleted data to a customer.

3.8 CREATE TABLE

```
CREATE TABLE invoices (  
    invoice_id    bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    order_id      integer NOT NULL REFERENCES orders (order_id),  
    issued_on     date NOT NULL DEFAULT current_date,  
    due_on        date NOT NULL,  
    amount        numeric(12,2) NOT NULL CHECK (amount >= 0),  
    currency      char(3) NOT NULL DEFAULT 'INR',  
    notes         text,  
    CONSTRAINT invoices_due_after_issue CHECK (due_on >= issued_on),  
    CONSTRAINT invoices_order_unique    UNIQUE (order_id)  
);
```

Read it top to bottom as a set of decisions. `GENERATED ALWAYS AS IDENTITY` is the SQL-standard auto-increment and the modern replacement for `serial`. Column-level constraints sit inline; anything spanning two columns (`due_on >= issued_on`) must be a table-level constraint, which is also where you name things explicitly.

DIALECT · AUTO-INCREMENT

Engine	Syntax
PostgreSQL	<code>bigint GENERATED ALWAYS AS IDENTITY</code> (preferred) or <code>bigserial</code> (legacy)
MySQL	<code>bigint AUTO_INCREMENT</code> — must be a key
SQL Server	<code>bigint IDENTITY(1,1)</code>
Oracle 12c+	<code>number GENERATED ALWAYS AS IDENTITY</code> ; older code uses a sequence plus a trigger

TEMPORARY AND DERIVED TABLES

```
CREATE TEMP TABLE session_scratch (id integer, note text);    -- dropped at session end  
  
CREATE TABLE high_earners AS                                -- CTAS: structure + data  
from a query  
SELECT employee_id, full_name, salary FROM employees WHERE salary > 70000;  
  
CREATE TABLE employees_backup (LIKE employees INCLUDING ALL); -- structure only,  
with constraints
```

COMMON MISTAKE

`CREATE TABLE ... AS SELECT` copies column names and types but **no constraints, no keys, no indexes, no defaults**. A “backup” made this way will happily accept the duplicate and orphan rows the original refused. Use `LIKE ... INCLUDING ALL`, or `pg_dump`, when you mean a real copy.

3.9 ALTER TABLE

```
ALTER TABLE employees ADD COLUMN phone text;
ALTER TABLE employees ADD COLUMN status text NOT NULL DEFAULT 'active';
ALTER TABLE employees DROP COLUMN phone;
ALTER TABLE employees RENAME COLUMN full_name TO employee_name;
ALTER TABLE employees ALTER COLUMN salary TYPE numeric(12,2);
ALTER TABLE employees ALTER COLUMN email SET NOT NULL;
ALTER TABLE employees ALTER COLUMN hire_date DROP DEFAULT;

ALTER TABLE employees
  ADD CONSTRAINT employees_salary_cap CHECK (salary <= 1000000);

ALTER TABLE employees DROP CONSTRAINT employees_salary_cap;
ALTER TABLE employees RENAME TO staff;
```

WHAT IT COSTS — THE PART BEGINNERS MISS

Every `ALTER TABLE` takes a lock. Which lock, and for how long, decides whether the statement is invisible or an outage.

Operation (PostgreSQL 11+)	Cost
Add a nullable column	Instant — catalog change only
Add a column with a constant <code>DEFAULT</code>	Instant since PG 11; a full table rewrite before that
Add a column with a <i>volatile</i> default (e.g. <code>now()</code>)	Full table rewrite
Drop a column	Instant — the column is marked dead, space reclaimed later
Change a type (<code>integer</code> → <code>bigint</code>)	Full rewrite, exclusive lock for the duration
Add <code>NOT NULL</code>	Full scan to verify; blocks writes
Add a <code>CHECK</code> or foreign key	Full scan — unless added <code>NOT VALID</code> , then <code>VALIDATE CONSTRAINT</code> separately

BEST PRACTICE · THE TWO-STEP CONSTRAINT

```
ALTER TABLE orders
  ADD CONSTRAINT orders_amount_positive CHECK (total_amount >= 0) NOT VALID;
-- takes a brief lock; new and updated rows are checked from now on

ALTER TABLE orders VALIDATE CONSTRAINT orders_amount_positive;
-- scans existing rows without blocking writes
```

The same pattern applies to foreign keys. On a table with a hundred million rows it is the difference between a minute of downtime and none.

INTERVIEW · ADVANCED

Q. How would you add a `NOT NULL` column to a 500-million-row table with no downtime? — Add it nullable (instant). Backfill in batches with pauses, so no long transaction and no lock queue. Add a `CHECK (col IS NOT NULL) NOT VALID`, then `VALIDATE` it. In PostgreSQL 12+, you can then set `NOT NULL` cheaply because the validated check proves it. **Common wrong answer:** “add it with a default in one statement” — correct only for a constant default on PG 11+, and wrong on older versions and on other engines.

3.10 DROP and the destructive family

```
DROP TABLE order_items;
DROP TABLE IF EXISTS order_items;
DROP TABLE orders CASCADE;      -- also drops views and constraints depending on it
DROP DATABASE sqlmastery;
DROP SCHEMA analytics CASCADE;
DROP INDEX idx_orders_customer;
DROP VIEW active_customers;
```

COMMON MISTAKE · CASCADE

`CASCADE` silently removes every dependent object — views, foreign keys, and the tables on the other side of a `DROP SCHEMA CASCADE`. Before using it, run the drop without it and read the error: PostgreSQL lists exactly what depends on the object. That list is information you want *before* the objects are gone, not after.

BEST PRACTICE · THE THREE-QUESTION RULE

Before any `DROP`, `TRUNCATE`, or unqualified `UPDATE` / `DELETE` on a shared database, answer all three out loud: **Which database am I connected to?** (`SELECT current_database();` — the classic accident is the right command in the wrong window.) **Do I have a restorable backup from within the hour?** **Have I run the `SELECT` version and checked the row count?** If any answer is no, stop.

3.11 A style guide for readable SQL

SQL is read far more often than it is written, usually by someone under pressure. These conventions are used throughout this book; adopt them or your team's, but adopt something.

BAD

```
select e.full_name,d.department_name,e.salary
from employees e,departments d where
e.department_id=d.department_id and
e.salary>50000
order by 3 desc
```

GOOD

```
SELECT e.full_name,
       d.department_name,
       e.salary
FROM   employees   AS e
JOIN   departments AS d
      ON d.department_id = e.department_id
WHERE  e.salary > 50000
ORDER BY e.salary DESC;
```

Rule	Why
Keywords upper, identifiers lower	The eye finds clause boundaries without reading words
One column per line in a long select list	Diffs show the column that changed, not the whole line
Explicit <code>JOIN ... ON</code> , never comma joins	A comma join with a missing condition is a silent cross join; a missing <code>ON</code> is a syntax error
Qualify every column when more than one table is in play	An unqualified <code>id</code> becomes ambiguous the moment someone adds a column
Meaningful aliases (<code>emp</code> , not <code>e1</code>)	Single letters stop working at four tables
Never <code>ORDER BY 3</code>	Positional references break when the select list changes
Trailing semicolons, always	Scripts concatenate; a missing one merges two statements
<code>snake_case</code> , plural table names, singular columns	Consistency; avoids quoted identifiers entirely
Never name a column <code>order</code> , <code>user</code> , <code>date</code> , <code>type</code>	Reserved words force quoting forever after

Part 3 Review

Quick Check ● BEGINNER

- 1 Which clause of a `SELECT` is mandatory?
- 2 Give three reasons not to commit `SELECT *`.
- 3 What does `DISTINCT` apply to?
- 4 Single quotes vs double quotes in SQL — what is each for?
- 5 Why write the column list in an `INSERT`?
- 6 What does `RETURNING` do, and name one problem it solves.
- 7 Name four differences between `DELETE` and `TRUNCATE`.
- 8 What does `CREATE TABLE ... AS SELECT` fail to copy?
- 9 Which two `ALTER TABLE` operations force a full table rewrite?
- 10 What is `EXCLUDED` in an `ON CONFLICT` clause?

Practice ● EASY

On the practice database. Reset it first.

- 1 List every employee's name and salary, highest paid first.
- 2 Show each product's name and its price including 18% GST, with the computed column aliased `price_with_gst`.
- 3 List the distinct order statuses present in `orders`.
- 4 List the distinct `(city, created_at::date)` pairs from `customers`. Explain why the row count differs from exercise 3's logic.
- 5 Insert a new department, Marketing in Chennai, using an explicit column list.
- 6 Insert three products in a single statement, and return their ids.
- 7 Re-run exercise 5 unchanged. Read the error, then rewrite it so re-running is harmless.
- 8 Give every Support employee a 5% raise, inside a transaction, and verify the row count before committing.
- 9 Set every product with zero stock to category `'Discontinued'`, returning the affected rows.
- 10 Delete all cancelled orders. What happened to their `order_items`, and which clause of the schema decided that?
- 11 Create an `invoices` table with an identity primary key, a foreign key to `orders`, and a named check that the due date is not before the issue date.
- 12 Add a nullable `phone` column to `customers`, then rename it to `contact_phone`, then drop it.

- 13 Add a `CHECK` constraint to `employees` capping salary at 1,000,000 — first `NOT VALID`, then validate it.
- 14 Create a structural copy of `employees` including all constraints, and prove the constraints came across by attempting an invalid insert.
- 15 Reformat this to the book's style: `select p.product_name,p.price from products p where p.price>3000 order by 2 desc`

Challenge ● INTERMEDIATE

- 1 Write one statement that corrects every `orders.total_amount` that disagrees with the sum of its `order_items`, and returns the rows it changed with both the old and new value.
- 2 Write an upsert that loads a batch of product price updates, inserting products that do not exist and updating those that do — without touching `in_stock` on existing rows.
- 3 You must delete 40 million rows from a 100-million-row table on a live system. Describe your approach and say why a single `DELETE` is wrong.
- 4 Design a soft-delete scheme for `customers`. Give the column, the view, and the one thing that will eventually go wrong.

Interview Drill ● ADVANCED

- 1 Walk through what the database does when you run an `UPDATE`, from parse to commit.
- 2 Why is `ON CONFLICT` safer than checking for existence first?
- 3 How do you add a column to a huge table without downtime?
- 4 When is `TRUNCATE` the wrong choice even though it is faster?
- 5 Argue for and against soft deletes.

Part 3 Solutions

Quick Check

- 1 Only `SELECT` . `SELECT 1+1;` is complete.
- 2 Breaks on schema change; transfers unneeded data; defeats covering indexes; hides which columns are actually used.
- 3 The entire select list — the whole output row, never a single column.
- 4 Single quotes delimit string *values*. Double quotes delimit *identifiers* and make them case-sensitive.
- 5 Without it the statement depends on physical column order, so adding or reordering a column breaks or corrupts it.
- 6 Returns values from the rows a write statement affected. It eliminates the follow-up `SELECT` for a generated key — one round trip instead of two, and no race.
- 7 DML vs DDL; filterable vs all-rows; row triggers fire vs not; identity untouched vs resettable; slow vs instant; space reclaimed later vs immediately.
- 8 Constraints, primary and foreign keys, indexes, defaults and identity properties.
- 9 Changing a column's type, and adding a column with a volatile default.
- 10 A pseudo-table holding the row that failed to insert, so the `DO UPDATE` can reference the proposed new values.

Practice — selected

2 ·

```
SELECT product_name,  
       price,  
       round(price * 1.18, 2) AS price_with_gst  
FROM   products  
ORDER BY price_with_gst DESC;
```

`round()` matters here: `numeric` arithmetic would otherwise carry more decimal places than a currency should ever display, and inconsistent rounding between the display layer and the database is a classic source of one-rupee discrepancies in invoices.

7 · The duplicate insert. The error is `duplicate key value violates unique constraint "departments_pkey"`. The safe rewrite:

```
INSERT INTO departments (department_id, department_name, location)  
VALUES (6, 'Marketing', 'Chennai')  
ON CONFLICT (department_id) DO NOTHING;
```

Note what this does *not* do: if row 6 exists with different values, it is left alone and you get no warning. If you want the values reconciled, use `DO UPDATE`; if you want to know about the discrepancy, add `RETURNING` and check whether you got a row back.

8 ·

```
BEGIN;
UPDATE employees
SET    salary = round(salary * 1.05, 2)
WHERE  department_id = (SELECT department_id FROM departments
                        WHERE department_name = 'Support');
-- UPDATE 2    <- Sneha and Divya. Expected? Yes.
COMMIT;
```

The subquery is better than hard-coding `department_id = 3`: it survives the ids changing, and it documents the intent. It also fails loudly if the department name is wrong, rather than quietly updating the wrong department.

10 · Deleting cancelled orders. Order 1004's single item disappeared with it, because `order_items.order_id` carries `ON DELETE CASCADE`. Had it not, the delete would have been refused with a foreign key violation — which is the correct default for most relationships and the reason cascade should be a deliberate, rare choice.

13 ·

```
ALTER TABLE employees
  ADD CONSTRAINT employees_salary_cap CHECK (salary <= 1000000) NOT VALID;
ALTER TABLE employees VALIDATE CONSTRAINT employees_salary_cap;
```

On nine rows this is theatre. The habit is the point: on nine million rows, the one-step version blocks writes for the length of a full scan and the two-step version does not.

Challenge — worked

1 · Reconciling totals.

```
UPDATE orders o
SET    total_amount = i.line_total
FROM  (SELECT order_id, sum(quantity * unit_price) AS line_total
       FROM  order_items
       GROUP BY order_id) AS i
WHERE  i.order_id = o.order_id
  AND  o.total_amount IS DISTINCT FROM i.line_total
RETURNING o.order_id, i.line_total AS new_total;
```

Two details carry the answer. `IS DISTINCT FROM` rather than `<>`, so a `NULL` total is also treated as different and corrected — with `<>`, a `NULL` comparison yields unknown and the row is skipped, which is exactly the bug you were asked to fix. And the `WHERE` restricting to genuinely differing rows: without it you rewrite every row in the table, generating dead tuples and index churn for no change in value.

Note the limitation of `RETURNING` here — it can show the new value but not the old, since the row is already updated. To capture both, use a CTE with `UPDATE ... RETURNING` joined against a pre-read snapshot (Part 11).

2 · Selective upsert.

```
INSERT INTO products (product_id, product_name, category, price, in_stock)
VALUES (10, 'Mechanical Keyboard', 'Peripherals', 5100.00, 0),
      (19, 'Trackpad', 'Peripherals', 7400.00, 25)
ON CONFLICT (product_id) DO UPDATE
SET    price          = EXCLUDED.price,
      product_name = EXCLUDED.product_name
WHERE  products.price IS DISTINCT FROM EXCLUDED.price;
```

`in_stock` is deliberately absent from the `SET` list, so existing stock is preserved while the placeholder `0` is still required by the `NOT NULL` column for genuinely new rows. The `WHERE` on the `DO UPDATE` — a feature many people do not know exists — suppresses no-op updates entirely, so unchanged products generate no row versions.

3 · The 40-million-row delete. A single statement is wrong for four reasons: it holds a transaction open for hours, blocking `VACUUM` from cleaning *any* table; it accumulates 40 million row locks; it writes an enormous volume of WAL, which can fill the disk and stall replicas; and if it fails at 95% you get nothing. Instead, loop in batches:

```
DELETE FROM events
WHERE event_id IN (SELECT event_id FROM events
                  WHERE occurred_at < '2023-01-01'
                  ORDER BY event_id
                  LIMIT 10000);
```

Run until it reports zero rows, with a short pause between batches so autovacuum and replication can keep up. If the deletion is by a natural range and you can plan ahead, the far better answer is that the table should have been *partitioned* by time (Part 30): dropping a partition is instantaneous and produces no dead rows at all. Mentioning that unprompted is what distinguishes a senior answer.

4 · Soft deletes. `ALTER TABLE customers ADD COLUMN deleted_at timestamptz;`, then `CREATE VIEW active_customers AS SELECT * FROM customers WHERE deleted_at IS NULL;` and point the application at the view. What will eventually go wrong: the `UNIQUE` constraint on `email`. A deleted customer still occupies their email address, so the same person cannot re-register. The fix is a partial unique index — `CREATE UNIQUE INDEX ON customers (email) WHERE deleted_at IS NULL;` — which enforces uniqueness

only among live rows (Part 20). Second failure mode: any query written against the base table instead of the view shows deleted customers, and there is no mechanism that prevents that except discipline or row-level security (Part 28).

Data Types

A type is a promise about what a column can hold and how it behaves. Choose well and the database validates your data for free, stores it compactly and compares it correctly. Choose badly and you get money that loses a paisa per transaction, timestamps that shift by five and a half hours, and an index that cannot be used.

IN THIS PART

4.1 Integers

4.2 Exact and approximate numerics

4.3 Boolean

4.4 Character types

4.5 Dates, times and intervals

4.6 UUID

4.7 JSON and JSONB

4.8 Arrays and enums

4.9 Binary and large objects

4.10 Casting · Review · Solutions

4.1 Integers

Type	Size	Range	Use for
<code>smallint</code>	2 bytes	-32,768 to 32,767	Small enumerated codes, ages, quantities
<code>integer</code>	4 bytes	±2.1 billion	The default choice for counts and most keys
<code>bigint</code>	8 bytes	±9.2 quintillion	Keys on high-volume tables, epoch milliseconds, money in minor units

The 4-byte saving of `integer` over `bigint` is real but small; the cost of running out is enormous. A table growing at a million rows a day exhausts `integer` in about six years — and the fix is a full table rewrite under an exclusive lock, on your largest table, at the worst possible moment.

BEST PRACTICE

Use `bigint` for the primary key of any table you expect to exceed ten million rows, or whose growth you cannot predict. Use `integer` for lookup tables that will always be small. The decision is nearly free at design time and expensive later — asymmetry decides it.

COMMON MISTAKE

Storing a phone number as `bigint`. Leading zeros vanish, `+` and country codes cannot be represented, and you gain arithmetic you will never use. Identifiers that merely look numeric — phone numbers, postcodes, PAN, order references — are `text`. The test: would you ever add two of them together? If not, it is not a number.

SERIAL AND IDENTITY

```
-- modern, SQL-standard, preferred
id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY

-- older PostgreSQL shorthand; still common in the wild
id bigserial PRIMARY KEY
```

`GENERATED ALWAYS` forbids inserting an explicit id, which prevents an application from accidentally colliding with the sequence. `GENERATED BY DEFAULT` permits it — useful for data migration where you must preserve original ids.

NOTE · SEQUENCES HAVE GAPS

A sequence value consumed by a transaction that rolls back is never reused. Gaps are normal and correct: sequences guarantee *uniqueness*, not contiguity. If your business requires gapless numbering — invoice numbers in some jurisdictions — you need a separate counter table updated inside the transaction, and you must accept the serialisation cost that implies.

4.2 Exact and approximate numerics

This is the most consequential type decision in the book, and it takes one paragraph to get right.

Type	Representation	Use for
<code>numeric(p,s)</code> / <code>decimal(p,s)</code>	Exact, base-10, arbitrary precision. <code>p</code> total digits, <code>s</code> after the point.	Money, tax, anything audited
<code>real</code> (<code>float4</code>)	Approximate, binary, ~6 significant digits	Sensor readings, coordinates
<code>double precision</code> (<code>float8</code>)	Approximate, binary, ~15 significant digits	Scientific computation, statistics

Binary floating point cannot represent 0.1 exactly, any more than decimal can represent one third. Errors accumulate across additions:

```
SELECT 0.1::float8 + 0.2::float8      AS float_sum,
       0.1::numeric + 0.2::numeric   AS numeric_sum,
       (0.1::float8 + 0.2::float8) = 0.3 AS float_equals;
```

float_sum	numeric_sum	float_equals
0.3000000000000004	0.3	f

Now imagine that last column deciding whether a payment balances. Over a million transactions the drift is not theoretical — it is a reconciliation report that never sums to zero and an auditor who wants an explanation.

BEST PRACTICE · MONEY

Two acceptable approaches. **(1)** `numeric(12,2)` — readable, exact, arithmetic in base 10; use this by default. **(2)** `bigint` holding minor units (paise, cents) — fastest and impossible to mis-round, at the cost of dividing by 100 in every display. Never `float`. Never PostgreSQL's `money` type either: it depends on a session locale setting, which makes the same bytes mean different amounts to different clients.

INTERVIEW · INTERMEDIATE

Q. Why should you not store money as a float? — Because binary floating point cannot exactly represent most decimal fractions, so values drift under accumulation and equality comparisons fail unpredictably. Use exact `numeric` / `decimal`, or integers of the smallest currency unit. Strong follow-up to volunteer: *rounding* — SQL `round()` on `numeric` rounds half away from zero, but on `double precision` it rounds half to even, so the same expression can give different answers depending on the column type.

PRECISION AND SCALE IN PRACTICE

```
SELECT 1234.5678::numeric(6,2); -- 1234.57 (rounded to scale)
SELECT 1234.5678::numeric(5,2); -- ERROR: numeric field overflow
SELECT 10 / 3; -- 3 (integer division!)
SELECT 10 / 3.0; -- 3.3333333333333333
SELECT 10::numeric / 3; -- 3.3333333333333333
```

COMMON MISTAKE · INTEGER DIVISION

`SELECT count(paid) / count(*)` returns `0`, not `0.43`, because both operands are integers and SQL truncates. Cast one side: `count(paid)::numeric / count(*)`. This silently produces zero percentages in dashboards and is one of the most common reporting bugs in existence. **Dialect:** MySQL and SQLite return a float here; PostgreSQL, SQL Server and Oracle truncate — so the same query gives different answers on different engines.

4.3 Boolean

```
SELECT true, false, NULL::boolean;  
SELECT 'yes'::boolean, 't'::boolean, '1'::boolean, 'on'::boolean;  -- all true
```

A SQL boolean has **three** values: true, false and unknown (`NULL`). §5.5 covers the consequences; the design consequence here is simply that `is_active boolean` without `NOT NULL` gives you three states when you meant two — and every query that says `WHERE NOT is_active` silently omits the unknown ones.

```
SELECT full_name FROM employees WHERE department_id IS NOT NULL;  -- explicit  
SELECT product_name FROM products WHERE in_stock > 0;           -- derive,  
don't duplicate
```

BEST PRACTICE

Declare booleans `NOT NULL DEFAULT false` unless the third state is genuinely meaningful. And name them as predicates — `is_active` , `has_shipped` , `can_edit` — so `WHERE is_active` reads as English and needs no `= true` .

DIALECT · BOOLEANS BARELY EXIST ELSEWHERE

MySQL: `BOOLEAN` is an alias for `tinyint(1)` ; `true` is `1` . Any non-zero counts as true, so `2` is truthy but `= true` is false. **SQL Server:** no boolean type at all — use `bit` (0/1) and note you cannot write `WHERE is_active` , only `WHERE is_active = 1` . **Oracle:** no SQL boolean before 23c — the convention is `char(1)` with a `CHECK (flag IN ('Y', 'N'))` . Portable code should compare explicitly rather than rely on truthiness.

4.4 Character types

Type	Behaviour	Use for
<code>char(n)</code>	Fixed length, blank-padded to <code>n</code>	Genuinely fixed codes — <code>char(3)</code> for ISO currency
<code>varchar(n)</code>	Variable, rejected beyond <code>n</code>	When a business rule really caps the length
<code>varchar</code> / <code>text</code>	Variable, unlimited (1 GB in PostgreSQL)	Almost everything else

In PostgreSQL, `text` and `varchar(n)` perform identically. There is no storage or speed advantage to a length limit; the limit is a constraint, nothing more. So the question is only ever “is there a real rule that this cannot exceed `n` characters?” Usually there is not, and inventing one produces the classic bug of a name field that rejects a legitimate customer.

COMMON MISTAKE · CHAR(N) PADDING

```
SELECT 'IN'::char(5) = 'IN'      AS equal,      -- t: comparison ignores padding
       length('IN'::char(5))    AS len,        -- 2: length() strips it
       length('IN'::char(5)::text) AS as_text;  -- 5: the padding is really stored
```

The padding is invisible until it is not — it reappears on export to CSV, in string concatenation, and in application code that does a byte-exact comparison. Use `text` unless the value is genuinely fixed-width.

DIALECT

MySQL: `varchar` requires a length; `text` is a separate type family (`tinytext` ... `longtext`) with real restrictions — `text` columns cannot be fully indexed, only prefix-indexed. Also use `utf8mb4`, never `utf8`: MySQL's `utf8` is a three-byte subset that cannot store emoji or many CJK characters.

SQL Server: `nvarchar` for Unicode, `varchar` for a single-byte code page; `nvarchar(max)` for large values. **Oracle:** `varchar2`, and note that an empty string is stored as `NULL` — a genuine trap when porting.

INTERVIEW · EASY

Q. `char` vs `varchar` vs `text` ? — `char` is fixed-width and blank-padded; `varchar(n)` is variable with a maximum; `text` is variable and unbounded. In PostgreSQL all three have identical performance, so the choice is about constraints, not speed. Add the engine-specific caveat that in MySQL and SQL Server the choice does affect storage and indexing — knowing that the answer differs by engine is the point of the question.

4.5 Dates, times and intervals

Type	Holds	Example
date	A calendar day, no time	2024-03-15
time	A time of day, no date	14:30:00
timestamp	Date and time, no zone — a wall-clock reading	2024-03-15 14:30:00
timestamptz	An absolute instant; stored as UTC, displayed in the session's zone	2024-03-15 14:30:00+05:30
interval	A duration	3 days 04:00:00

TIMESTAMP VS TIMESTAMPTZ — THE DISTINCTION THAT CAUSES PRODUCTION INCIDENTS

Despite the name, `timestamptz` does not store a time zone. It stores an absolute point in time (UTC internally) and renders it in whatever zone your session declares. `timestamp` stores the digits you gave it with no notion of where on Earth they were read.

```
SET timezone = 'Asia/Kolkata';
SELECT '2024-03-15 14:30'::timestamp AS plain,
       '2024-03-15 14:30'::timestamptz AS with_zone;
```

plain		with_zone
-----+-----		
2024-03-15 14:30:00		2024-03-15 14:30:00+05:30

```
SET timezone = 'UTC'; -- same rows, read again from a different session
SELECT '2024-03-15 14:30'::timestamp AS plain,
       '2024-03-15 14:30'::timestamptz AS with_zone;
```

plain		with_zone
-----+-----		
2024-03-15 14:30:00		2024-03-15 09:00:00+00

The `timestamptz` value moved, correctly — it is the same instant described from a different place. The plain `timestamp` did not, because it never meant an instant at all.

BEST PRACTICE

Use `timestampz` for everything that records when something happened — created, updated, logged in, paid. Use plain `timestamp` only for a wall-clock intention detached from any location: “the alarm rings at 07:00 local, wherever the user is”. Store the user's zone as a separate `text` column when you need to reconstruct their local view. The default should be `timestampz`; deviating from it should require a sentence of justification in the schema comment.

COMMON MISTAKE · THE DATE-RANGE OFF-BY-ONE

```
-- WRONG: silently drops everything after midnight on the 31st
WHERE created_at BETWEEN '2024-03-01' AND '2024-03-31'
```

```
-- RIGHT: half-open range, correct for any precision and any time zone
WHERE created_at >= '2024-03-01'
      AND created_at < '2024-04-01'
```

`'2024-03-31'` as a timestamp means `2024-03-31 00:00:00`, so an event at 09:15 on the 31st is excluded. Half-open ranges (`>= start AND < next_start`) are always correct, never need an end-of-day fudge, and remain index-friendly. Make it a reflex.

INTERVAL ARITHMETIC

```
SELECT current_date + interval '30 days'           AS in_30_days,
       age(current_date, '1995-08-14'::date)      AS exact_age,
       '2024-03-15'::date - '2024-01-01'::date    AS days_between,
       date_trunc('month', now())                  AS month_start,
       extract(dow FROM date '2024-03-15')        AS day_of_week;
```

NOTE · WHY INTERVALS ARE NOT SIMPLY SECONDS

`interval '1 month'` is not 30 days: adding it to 31 January gives 28 or 29 February, and adding it to 31 March gives 30 April. Similarly `interval '1 day'` across a daylight-saving boundary is 23 or 25 hours, not 24. This is correct calendar behaviour, and it is why you should add intervals rather than compute with seconds — but also why `interval '1 month' = interval '30 days'` is false.

DIALECT · CURRENT TIME

Engine	Now	Add a month
PostgreSQL	<code>now()</code> , <code>current_timestamp</code>	<code>d + interval '1 month'</code>
MySQL	<code>NOW()</code> , <code>CURDATE()</code>	<code>DATE_ADD(d, INTERVAL 1 MONTH)</code>
SQL Server	<code>GETDATE()</code> , <code>SYSDATETIMEOFFSET()</code>	<code>DATEADD(month, 1, d)</code>
Oracle	<code>SYSDATE</code> , <code>SYSTIMESTAMP</code>	<code>ADD_MONTHS(d, 1)</code>

Almost nothing about date handling is portable. Part 37 tabulates the rest.

4.6 UUID

```
CREATE TABLE api_keys (  
  key_id      uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  owner_id    integer NOT NULL REFERENCES customers (customer_id),  
  created_at  timestamptz NOT NULL DEFAULT now()  
);
```

	UUID	bigint identity
For	Generated anywhere without coordination; safe to expose publicly; no cross-shard collision; merging datasets never conflicts	8 bytes; sequential, so inserts append to one index page; human-readable in logs and support tickets
Against	16 bytes, and random values scatter inserts across the whole index — more page splits, worse cache locality, larger indexes	Sequential ids leak volume (a competitor reads your order count) and are guessable in URLs

BEST PRACTICE

Use `bigint` internally and expose a separate UUID or short random token publicly when you need unguessable URLs. If you want UUIDs as the primary key, prefer a time-ordered variant (UUIDv7) so inserts stay sequential and you keep the index locality — the random-v4-as-primary-key pattern is a measurable write-throughput cost on large tables.

4.7 JSON and JSONB

	json	jsonb
Stored as	Exact input text	A parsed binary tree
Preserves	Whitespace, key order, duplicate keys	Nothing cosmetic; last duplicate key wins
Write	Faster (no parsing)	Slower (parsed on input)
Read/query	Slower (re-parsed each access)	Much faster
Indexable	No	Yes — GIN
Verdict	Only when you must reproduce the document byte-for-byte	Use this

```
CREATE TABLE events (  
    event_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    payload jsonb NOT NULL  
);  
  
INSERT INTO events (payload) VALUES  
    ('{"type":"login","user":{"id":42,"plan":"pro"},"tags":["web","mobile"]}');  
  
SELECT payload -> 'user'          AS user_obj,    -- -> returns json  
       payload ->> 'type'          AS event_type,  -- ->> returns text  
       payload #>> '{user,plan}' AS plan,          -- #>> path, as text  
       payload @> '{"type":"login"}'::jsonb AS is_login -- containment  
FROM events;
```

user_obj	event_type	plan	is_login
-----+-----+-----+-----			
{"id": 42, "plan":"pro"}	login	pro	t

Remember the pair: `->` keeps the value as JSON (so you can chain), `->>` extracts it as text (so you can compare and cast). Reaching for `->` when you need `->>` gives you `"pro"` with quotes, which then fails to equal `'pro'` — a five-minute debugging session that everyone has exactly once.

BEST PRACTICE

Model what you know as columns; use `jsonb` for what genuinely varies. A `users` table with `email` and `created_at` as real columns and a `preferences jsonb` alongside is a good design. A `users` table that is one `jsonb` column is a document database you have to operate yourself, with none of a document database's tooling — and no constraint can protect it.

COMMON MISTAKE

Filtering on a JSON field without an index. `WHERE payload ->> 'type' = 'login'` scans and re-parses every row. Fix with an expression index for a specific key — `CREATE INDEX ON events ((payload ->> 'type'));` — or a GIN index for general containment queries: `CREATE INDEX ON events USING gin (payload jsonb_path_ops);`, which then supports the `@>` operator. Part 26 goes deeper.

4.8 Arrays and enums

```
CREATE TABLE articles (  
    article_id integer PRIMARY KEY,  
    title      text NOT NULL,  
    tags       text[] NOT NULL DEFAULT '{}'  
);  
  
INSERT INTO articles VALUES (1, 'Indexing basics', ARRAY['sql','performance']);  
  
SELECT title FROM articles WHERE 'sql' = ANY(tags);  
SELECT title FROM articles WHERE tags @> ARRAY['sql','performance'];  
SELECT title, unnest(tags) AS tag FROM articles;      -- one row per tag  
SELECT array_length(tags, 1) FROM articles;
```

Arrays are a genuine PostgreSQL strength and a genuine trap. They are right when the list is small, read as a unit, and has no attributes of its own — tags, feature flags, a fixed set of scores. They are wrong the moment you need to attach anything to an element (when was this tag applied? by whom?), enforce a foreign key on it, or count usage across rows efficiently. That is a junction table, and Volume I §1.7 already told you so.

DIALECT

Arrays are PostgreSQL and Oracle only. MySQL and SQL Server have no array type — the portable model is a junction table, or a JSON array with the query limitations that implies.

ENUMS

```
CREATE TYPE order_status AS ENUM ('pending','paid','shipped','delivered','cancelled');  
ALTER TABLE orders ALTER COLUMN status TYPE order_status USING status::order_status;  
  
ALTER TYPE order_status ADD VALUE 'refunded' AFTER 'delivered';  -- adding is easy  
-- removing or reordering a value is NOT supported; you must recreate the type
```

Approach	For	Against
<code>CHECK (col IN (...))</code>	Simple, portable, easy to change	Values not discoverable as data; each table repeats the list
<code>ENUM</code> type	Compact storage, sorts in declared order, shared across tables	Values cannot be removed or reordered; <code>ALTER TYPE ADD VALUE</code> could not run inside a transaction before PG 12
Lookup table + FK	Fully dynamic; can carry a label, sort order, active flag; changed by ordinary <code>INSERT</code>	A join to read; slightly more schema

BEST PRACTICE

A `CHECK` for a genuinely fixed, tiny set (a two-state flag, a currency triplet). A lookup table for anything the business might extend — order statuses, roles, categories. Enums sit awkwardly between the two: their inflexibility surprises teams a year in, when someone asks to rename a status.

4.9 Binary data, and 4.10 casting

`bytea` stores raw bytes. Use it for small binaries with a genuine reason to live in the row — an encryption nonce, a hash digest, a thumbnail of a few kilobytes. Do not use it as a file store: every byte is copied into every backup, every replica, and every sequential scan of the table, and none of it can be served by a CDN. Files belong in object storage, with the URL, size, content type and checksum in a table.

CASTING

```
SELECT '42'::integer,           -- PostgreSQL shorthand
       CAST('42' AS integer),   -- SQL standard, portable
       '2024-03-15'::date,
       42::text,
       '3.7'::numeric::integer AS rounded; -- 4 – rounds, does not truncate
```

Kind	When it happens	Risk
Implicit	The engine converts silently — <code>integer</code> to <code>numeric</code> in mixed arithmetic	Can prevent index use; differs sharply between engines
Explicit	You wrote <code>::</code> or <code>CAST</code>	Fails loudly on bad data, which is what you want

COMMON MISTAKE · A CAST THAT KILLS AN INDEX

```
-- Index on created_at cannot be used: the function is applied to the column
WHERE created_at::date = '2024-03-15'

-- Index-friendly: the column is left bare, the constants are computed
WHERE created_at >= '2024-03-15' AND created_at < '2024-03-16'
```

The rule generalises: **never wrap an indexed column in a function or cast on the left of a comparison**. Transform the constant instead. When you genuinely cannot, build an expression index on exactly that expression (Part 20).

BEST PRACTICE · SAFE CASTING

PostgreSQL 16 has no `TRY_CAST`. To convert dirty data without aborting, filter first with a regular expression — `WHERE col ~ '^\\d+$'` — or write a small `PL/pgSQL` function with an exception block (Part 24). SQL Server has `TRY_CAST` and `TRY_CONVERT`; MySQL has none.

Part 4 Review

Quick Check ● BEGINNER

- 1 Which type for money, and why not `float` ?
- 2 What does `timestampz` actually store?
- 3 Why is `SELECT 10/3` equal to 3?
- 4 Difference between `->` and `->>` ?
- 5 Why store a phone number as `text` ?
- 6 In PostgreSQL, is `varchar(50)` faster than `text` ?
- 7 Name two reasons a random UUID primary key can slow down inserts.
- 8 Why does `char(5)` surprise people?
- 9 Which is preferred for a query, `json` or `jsonb` , and why?
- 10 What is wrong with `BETWEEN '2024-03-01' AND '2024-03-31'` on a timestamp column?

Practice ● EASY

- 1 Prove the floating-point problem: show that `0.1 + 0.2 <> 0.3` in `float8` but not in `numeric` .
- 2 Compute the percentage of orders that are delivered, correct to two decimal places. Show the wrong integer-division version first.
- 3 Show the same `timestampz` value under two different session time zones.
- 4 List all orders from March 2024 using a half-open range.
- 5 For each employee, compute exact tenure using `age()` , and tenure in whole years using `extract` .
- 6 Add a `preferences jsonb` column to `customers` , set one customer's preferences, and query customers who prefer email notifications.
- 7 Create an expression index supporting that query, and confirm with `EXPLAIN` that it is used.
- 8 Convert `orders.status` to an enum type, then add a `'refunded'` value.
- 9 Demonstrate that `'IN'::char(5)` compares equal to `'IN'` but has a different byte length when cast to text.
- 10 Write two versions of a filter on `created_at` for a single day — one that defeats an index and one that does not — and explain the difference.

Challenge ● INTERMEDIATE

- 1 Design the type for every column of a payments table: amount, currency, status, external reference, created time, settled time, metadata. Justify each choice in one sentence.

- 2 A team stored all timestamps as plain `timestamp` in server-local time, and the servers have moved from Mumbai to a UTC region. Describe the damage and the migration.
- 3 A table stores `tags text` as a comma-separated string. Write the migration to `text[]`, then argue whether a junction table would be better.

Interview Drill ● ADVANCED

- 1 How do you store money, and defend it against three alternatives?
- 2 Explain `timestamp` vs `timestampz` to someone who has been burned by it.
- 3 When would you put JSON in a relational database, and when would you refuse?
- 4 UUID or bigint primary keys — argue both sides and then decide.
- 5 Give three ways a data type choice can prevent an index from being used.

Part 4 Solutions

Quick Check

- 1 `numeric(p,s)`, or `bigint` in minor units. Binary floats cannot represent decimal fractions exactly, so values drift and equality fails.
- 2 An absolute instant, held as UTC. It does not store a zone; it renders in the session's zone.
- 3 Both operands are integers, so PostgreSQL performs integer division and truncates. Cast one side to `numeric`.
- 4 `->` returns a JSON value (chainable); `->>` returns text (comparable and castable).
- 5 Leading zeros, `+` prefixes and country codes are not numeric, and you never do arithmetic on it.
- 6 No — identical performance. The length is a constraint only.
- 7 16 bytes rather than 8 in every index entry, and random values scatter writes across the whole index causing page splits and poor cache locality.
- 8 It blank-pads to the declared width. Comparison and `length()` hide the padding, but export, concatenation and byte-level comparison reveal it.
- 9 `jsonb` — pre-parsed, far faster to read, and the only one that can be indexed.
- 10 `'2024-03-31'` becomes midnight, so the whole of the 31st after 00:00:00 is excluded. Use `>= '2024-03-01' AND < '2024-04-01'`.

Practice — selected

2 • The percentage.

```
-- Wrong: integer division truncates to 0
SELECT count(*) FILTER (WHERE status = 'delivered') / count(*) AS pct FROM orders;

-- Right
SELECT round(100.0 * count(*) FILTER (WHERE status = 'delivered') / count(*), 2)
       AS delivered_pct
FROM   orders;
```

```
delivered_pct
-----
         42.86
```

Multiplying by `100.0` rather than `100` is what forces numeric arithmetic — a one-character difference between a correct dashboard and one that reads 0%. `FILTER` is PostgreSQL's clean form of conditional aggregation; the portable equivalent is `sum(CASE WHEN status='delivered' THEN 1 ELSE 0 END)`, and both appear in Part 8.

5 • Tenure.

```
SELECT full_name,
       age(current_date, hire_date)           AS exact_tenure,
       extract(year FROM age(current_date, hire_date)) AS whole_years
FROM   employees
ORDER BY hire_date;
```

`age()` returns a calendar-aware interval ("5 years 4 mons 27 days"), not a day count. Do not compute years as `(current_date - hire_date)/365` — it drifts by a day every four years and will eventually award someone a work anniversary on the wrong date.

6 & 7 · JSONB with an index.

```
ALTER TABLE customers ADD COLUMN preferences jsonb NOT NULL DEFAULT '{}';

UPDATE customers
SET   preferences = '{"notify":{"email":true,"sms":false},"theme":"dark"}'
WHERE customer_id = 1;

SELECT full_name
FROM   customers
WHERE  preferences @> '{"notify":{"email":true}}';

CREATE INDEX idx_customers_prefs ON customers USING gin (preferences jsonb_path_ops);
EXPLAIN SELECT full_name FROM customers WHERE preferences @> '{"notify":{"email":true}}';
```

On five rows the planner will still choose a sequential scan — correctly, because reading five rows is cheaper than consulting an index. That is not a failure; it is the optimizer doing its job, and Part 21 explains how to verify index usage on a table large enough for it to matter (`SET enable_seqscan = off` is the debugging trick, never a production setting).

Note also `jsonb_path_ops` versus the default GIN operator class: it is smaller and faster but supports only the containment operator `@>`. If you need key-existence operators (`? , ?|`), use the default class.

10 · Two filters.

```
-- Defeats the index on created_at: the column is wrapped in a cast
WHERE created_at::date = DATE '2024-03-15'

-- Uses it: the column stands bare, the boundaries are constants
WHERE created_at >= TIMESTAMPTZ '2024-03-15 00:00+05:30'
   AND created_at <  TIMESTAMPTZ '2024-03-16 00:00+05:30'
```

A B-tree index stores the values of `created_at`, not of `created_at::date`, so it cannot answer a question about the derived value. Note too that the first version is *also semantically fragile*: the cast uses the session time zone, so the same query returns different rows for a user in Mumbai and a user in London. The explicit-boundary version says exactly which instants it means.

Challenge — worked

1 · The payments table.

```
CREATE TABLE payments (  
  payment_id    bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  public_ref    uuid NOT NULL DEFAULT gen_random_uuid() UNIQUE,  
  order_id      bigint NOT NULL REFERENCES orders (order_id),  
  amount_minor  bigint NOT NULL CHECK (amount_minor > 0),  
  currency      char(3) NOT NULL CHECK (currency ~ '^[A-Z]{3}$'),  
  status        text NOT NULL REFERENCES payment_statuses (status),  
  gateway_ref   text UNIQUE,  
  created_at    timestamptz NOT NULL DEFAULT now(),  
  settled_at    timestamptz,  
  metadata      jsonb NOT NULL DEFAULT '{}',  
  CONSTRAINT payments_settled_after_created  
    CHECK (settled_at IS NULL OR settled_at >= created_at)  
);
```

`amount_minor bigint` — exact, impossible to mis-round, and currency-agnostic once you know the currency. `currency char(3)` — genuinely fixed-width, one of the few honest uses of `char`. `status` as a foreign key to a lookup table rather than an enum, because payment statuses grow. `gateway_ref text` — an opaque external identifier, never an integer however numeric it looks, and `UNIQUE` so a webhook delivered twice cannot create two payments. `settled_at` nullable, because unsettled is a real state, with a cross-column check that only a table-level constraint can express. `metadata jsonb` for gateway-specific fields you cannot enumerate in advance — and `NOT NULL DEFAULT '{}'` so every query can assume an object exists rather than guarding for `NULL`.

2 · The timestamp migration. The damage: existing rows hold wall-clock readings in IST with nothing recording that fact, while new rows written after the move hold UTC readings — in the same column, indistinguishable. Every report spanning the cutover is wrong by 5 hours 30 minutes for part of its range, and no query can tell which rows are which except by date. The migration: (i) establish the exact cutover instant from deployment records; (ii) add a new `timestamptz` column; (iii) backfill in two passes — rows before the cutover with `old_col AT TIME ZONE 'Asia/Kolkata'`, rows after with `AT TIME ZONE 'UTC'`; (iv) verify against an external source of truth, such as payment gateway timestamps, on a sample from each side; (v) swap the columns and drop the old one. The lesson: this is unrecoverable without external evidence, which is precisely why `timestamptz` should be the default and not a decision each developer makes per column.

3 · Comma-separated tags.

```
ALTER TABLE articles ADD COLUMN tags_arr text[];
UPDATE articles
SET     tags_arr = string_to_array(regexp_replace(tags, '\s*,\s*', ', ', 'g'), ',')
WHERE  tags IS NOT NULL AND tags <> '';
ALTER TABLE articles DROP COLUMN tags;
ALTER TABLE articles RENAME COLUMN tags_arr TO tags;
ALTER TABLE articles ALTER COLUMN tags SET DEFAULT '{}';
ALTER TABLE articles ALTER COLUMN tags SET NOT NULL;
```

The `regexp_replace` matters: `'sql, performance'` would otherwise produce an element with a leading space, and `' performance'` is a different tag from `'performance'` forever after.

Is an array better than a junction table? It is better if tags are free-form, small in number, and only ever read with the article. A junction table wins the moment you need any of: a canonical tag list preventing `'SQL'` and `'sql'` from diverging; attributes on the tagging (who applied it, when); an efficient “top 20 tags by article count” query; or rename-a-tag-everywhere as a single `UPDATE`. Most systems eventually need all four — which is the honest answer: the array is a reasonable start and a predictable migration.

Operators & Expressions

The centre of this part is §5.5, three-valued logic. More production bugs trace to a misunderstanding of `NULL` than to any other single cause in SQL — and unlike a syntax error, this one returns a plausible wrong answer instead of an error message.

IN THIS PART

5.1 Arithmetic operators

5.2 Comparison operators

5.3 Logical operators and precedence

5.4 BETWEEN

5.5 NULL and three-valued logic

5.6 IN and NOT IN

5.7 LIKE and pattern matching

5.8 CASE

5.9 COALESCE, NULLIF, GREATEST

Review · Solutions

5.1 Arithmetic, and 5.2 comparison

```
SELECT 7 + 3, 7 - 3, 7 * 3, 7 / 3, 7 % 3, 2 ^ 10, | / 16, @ -5;
--      10      4      21      2      1      1024      4      5
--
--              ^ integer division              ^ absolute value
```

Operator	Meaning	Note
=	Equal	Never true when either side is <code>NULL</code>
<> or !=	Not equal	<> is the standard spelling
< > <= >=	Ordering	Text ordering depends on collation
IS NULL	Is unknown	The only way to test for <code>NULL</code>
IS DISTINCT FROM	Not equal, treating <code>NULL</code> as a comparable value	The <code>NULL</code> -safe <>; MySQL's equivalent is <=> (inverted)
BETWEEN	Inclusive range	Both ends included — see §5.4

Comparisons are not only for numbers. Dates, timestamps, booleans and text all order, and one comparison in particular deserves attention: `'apple' < 'Banana'` is **true** under an `en_US` collation and **false** under the C locale, because C compares raw bytes and every uppercase letter sorts before every lowercase one. The same query, the same data, two answers, decided by a setting made when the database was created.

5.3 Logical operators and precedence

```
-- Precedence, highest to lowest:  
--  () -> arithmetic -> comparison -> NOT -> AND -> OR
```

AND binds tighter than OR, and forgetting it is one of the most expensive one-line bugs in SQL because it produces no error at all.

```
-- Intent: pending or paid orders belonging to customer 1  
-- WRONG – parsed as: (status='pending') OR (status='paid' AND customer_id=1)  
SELECT * FROM orders  
WHERE  status = 'pending' OR status = 'paid' AND customer_id = 1;  
  
-- RIGHT  
SELECT * FROM orders  
WHERE  (status = 'pending' OR status = 'paid') AND customer_id = 1;  
  
-- BETTER – no precedence to reason about at all  
SELECT * FROM orders  
WHERE  status IN ('pending','paid') AND customer_id = 1;
```

The wrong version returns every pending order in the system, including other customers'. In a report it looks merely surprising; in a permission check it is a data leak.

BEST PRACTICE

Parenthesise every mixed **AND / OR** even when precedence is on your side. The parentheses cost nothing, remove the need for the reader to recall the rule, and survive the next person editing the clause.

5.4 BETWEEN

```
WHERE salary BETWEEN 50000 AND 80000
-- identical to:
WHERE salary >= 50000 AND salary <= 80000
```

BETWEEN is **inclusive at both ends**. That is fine for discrete values like integers and dates, and dangerous for anything continuous — timestamps and **numeric** — where the upper bound cuts in the middle of the last unit. §4.5 covered the timestamp case; the rule is the same one: **use half-open ranges for continuous types**, and reserve **BETWEEN** for discrete ones.

COMMON MISTAKE

BETWEEN 80000 AND 50000 returns zero rows, not the range reversed. The operator does not sort your bounds. If either bound comes from a parameter, an accidental swap silently yields an empty result set — which reads as “no matching data” rather than as an error.

5.5 NULL and three-valued logic

`NULL` means *unknown*. It is not zero, not an empty string, and not false. Once you accept that it is a statement about your knowledge rather than a value, every rule below becomes obvious instead of arbitrary.

FIGURE 5.5 — THREE-VALUED TRUTH TABLES

AND	T	F	NULL	OR	T	F	NULL
T	T	F	NULL	T	T	T	T
F	F	F	F	F	T	F	NULL
NULL	NULL	F	NULL	NULL	T	NULL	NULL

Read them as knowledge: `false AND unknown` is **false**, because a false conjunct settles it whatever the unknown turns out to be. `true OR unknown` is **true**, for the mirror reason. Everything else stays unknown.

THE FIVE RULES

```
SELECT NULL = NULL      AS eq,      -- NULL, not true
      NULL <> NULL      AS ne,      -- NULL, not true
      NULL IS NULL      AS isnull,  -- t  <- the only working test
      NULL + 1          AS arith,    -- NULL: any arithmetic with NULL is NULL
      'abc' || NULL     AS concat,  -- NULL: concatenation too
      NULL IS DISTINCT FROM NULL AS dist;-- f  <- NULL-safe comparison
```

- `NULL` is never equal to anything, including itself. Two unknown values may or may not be the same; the database refuses to guess.
- Any expression containing `NULL` is `NULL` — arithmetic, concatenation, most functions.
- `WHERE` keeps only rows where the condition is **true**. Unknown is discarded exactly like false — which is why a row can escape both `WHERE x = 1` and `WHERE x <> 1`.
- Aggregates skip `NULL` s — except `count(*)`, which counts rows rather than values. §8.2.
- `NULL` s group and sort together, even though they do not compare equal. `GROUP BY` puts them in one group; `ORDER BY` places them all at one end. Two exceptions to rule 1, for pragmatic reasons.

RULE 3, DEMONSTRATED ON THE PRACTICE DATA

```
SELECT count(*) FROM employees;           -- 9
SELECT count(*) FROM employees WHERE department_id = 1; -- 3
SELECT count(*) FROM employees WHERE department_id <> 1; -- 5, not 6!
SELECT count(*) FROM employees WHERE department_id IS NULL; -- 1
```

3 + 5 = 8, and there are nine employees. Nisha Verma (109) has no department, so `NULL <> 1` is unknown and she is excluded from both. Neither query is wrong; the mental model “= 1 and <> 1 partition the table” is. The correction is explicit:

```
WHERE department_id <> 1 OR department_id IS NULL
-- or, more compactly and NULL-safely:
WHERE department_id IS DISTINCT FROM 1
```

COMMON MISTAKE · NOT IN WITH NULL

```
-- Returns ZERO rows if the subquery yields even one NULL
SELECT full_name FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders WHERE total_amount
> 100000);
```

Why: `x NOT IN (a, b, NULL)` expands to `x <> a AND x <> b AND x <> NULL`. The last conjunct is unknown, so the whole expression can never be true — at best unknown, which `WHERE` discards. Every row disappears. Use `NOT EXISTS` instead, which has no such problem (Part 10), or add `WHERE customer_id IS NOT NULL` inside the subquery. **This is the single most asked NULL question in SQL interviews.**

INTERVIEW · ADVANCED

Q. Why does `NOT IN` with a subquery sometimes return nothing? — Because a `NULL` in the subquery makes every comparison unknown, and `WHERE` keeps only true. **Expected thinking:** expand `NOT IN` into its chain of `<>` conjuncts out loud — that is what the interviewer wants to hear. **Common wrong approach:** “`NOT IN` is just slower than `NOT EXISTS`” — a performance answer to a correctness question. **Tip:** finish by saying you default to `NOT EXISTS` for anti-joins precisely so this class of bug cannot occur.

DIALECT

Oracle treats the empty string as `NULL`, so `WHERE name = ''` never matches anything — code ported from PostgreSQL or MySQL breaks silently. **MySQL** offers `<=>`, a `NULL`-safe equality (the inverse of `IS DISTINCT FROM`). **SQL Server** has neither by default; you write `(a = b OR (a IS NULL AND b IS NULL))`. **Sorting** differs too: PostgreSQL and Oracle place `NULL`s last ascending, MySQL and SQL Server first — always write `NULLS FIRST` or `NULLS LAST` explicitly if it matters.

5.6 IN, and 5.7 pattern matching

```
WHERE status IN ('pending','paid','shipped')
WHERE customer_id IN (SELECT customer_id FROM customers WHERE city = 'Hyderabad')
```

`IN` with a list is shorthand for a chain of `OR` s and is both clearer and safe. `NOT IN` with a *subquery* is the hazard of §5.5; `NOT IN` with a literal list you wrote yourself is fine, because you know there is no `NULL` in it.

LIKE

Pattern	Matches
'a%'	Starts with a — can use an index
'%a'	Ends with a — cannot use a normal B-tree index
'%abc%'	Contains abc — full scan unless you add a trigram index
'_bc'	<code>_</code> is exactly one character
'100\%'	A literal percent sign, escaped

```
SELECT full_name FROM customers WHERE full_name LIKE 'R%';      -- case-sensitive
SELECT full_name FROM customers WHERE full_name ILIKE 'r%';      -- case-insensitive (PG)
SELECT full_name FROM customers WHERE full_name ~* '^(r|k)';     -- regex, case-insensitive
```

COMMON MISTAKE

A leading wildcard defeats a B-tree index, so a search box wired to `LIKE '%term%'` is a full table scan on every keystroke. For real substring search use a trigram index (`CREATE EXTENSION pg_trgm;` `CREATE INDEX ... USING gin (col gin_trgm_ops);`) or full-text search. This is one of the most common causes of a page that is fast in development and unusable in production.

DIALECT

`ILIKE` is PostgreSQL only. MySQL's default collations are case-insensitive, so plain `LIKE` already ignores case — which surprises people migrating to PostgreSQL, where it does not. SQL Server depends on the column's collation. The portable form is `WHERE lower(col) LIKE lower('r%')`, which then needs an expression index on `lower(col)` to stay fast.

5.8 CASE

`CASE` is SQL's if/then/else, and it is an *expression* — it produces a value, so it can appear anywhere a value can: in the select list, in `WHERE`, in `ORDER BY`, inside an aggregate.

```
-- Searched form: any condition
SELECT full_name,
       salary,
       CASE WHEN salary >= 85000 THEN 'senior'
            WHEN salary >= 60000 THEN 'mid'
            ELSE 'junior'
       END AS band
FROM   employees
ORDER BY salary DESC;

-- Simple form: equality against one expression
SELECT order_id,
       CASE status
            WHEN 'delivered' THEN 'complete'
            WHEN 'cancelled' THEN 'closed'
            ELSE 'in progress'
       END AS phase
FROM   orders;
```

full_name	salary	band
Asha Menon	92000.00	senior
Imran Khan	88000.00	senior
Arjun Shetty	76000.00	mid
Meera Nair	71500.00	mid
...		

Branches are evaluated **in order**, and the first true one wins — so the ordering above is load-bearing. Reverse the two `WHEN` s and everyone earning 92,000 is classified as “mid”, because `92000 >= 60000` matches first. Write the most specific condition first, always.

COMMON MISTAKE

Omitting `ELSE`. An unmatched `CASE` returns `NULL`, which then propagates silently through every downstream expression. Write an explicit `ELSE` even if it is `ELSE 'unknown'` — it converts a silent hole into a visible one.

THREE PROFESSIONAL USES OF CASE

-- 1. Conditional aggregation – a pivot without a pivot operator (Part 14)

```
SELECT count(*) FILTER (WHERE status = 'delivered')           AS delivered,  
       sum(CASE WHEN status = 'cancelled' THEN 1 ELSE 0 END)   AS cancelled,  
       sum(CASE WHEN status = 'paid' THEN total_amount ELSE 0 END) AS paid_value  
FROM   orders;
```

-- 2. Custom sort order

```
ORDER BY CASE status WHEN 'pending' THEN 1 WHEN 'paid' THEN 2 ELSE 3 END, order_date;
```

-- 3. Guarding a division by zero

```
SELECT CASE WHEN count(*) = 0 THEN NULL  
         ELSE round(sum(total_amount) / count(*), 2) END AS avg_order  
FROM   orders;
```

BEST PRACTICE

Prefer `FILTER (WHERE ...)` over `sum(CASE ...)` in PostgreSQL — it is clearer and marginally faster. Use the `CASE` form when the query must run on MySQL, SQL Server or Oracle, none of which support `FILTER`.

5.9 COALESCE, NULLIF and friends

```
SELECT coalesce(NULL, NULL, 'fallback')    AS coalesce_,    -- first non-NULL
       nullif(0, 0)                        AS nullif_,        -- NULL if the two are equal
       greatest(3, 9, 5)                   AS greatest_,      -- 9
       least(3, 9, 5)                      AS least_;         -- 3
```

Function	Returns	Classic use
<code>coalesce(a,b,...)</code>	The first argument that is not <code>NULL</code>	Defaults for display; summing nullable columns
<code>nullif(a,b)</code>	<code>NULL</code> if <code>a = b</code> , else <code>a</code>	Turning a sentinel into a real <code>NULL</code> ; guarding division
<code>greatest</code> / <code>least</code>	Max/min <i>across columns in one row</i>	Clamping; latest of several dates

```
-- Division that cannot fail: the denominator becomes NULL, and NULL/x is NULL
SELECT round(sum(total_amount) / nullif(count(*), 0), 2) AS avg_order FROM orders;
```

```
-- Display defaults
```

```
SELECT full_name,
       coalesce(email, 'no email on file')           AS email,
       coalesce(department_id::text, 'unassigned')  AS dept
FROM   employees;
```

```
-- Sum a nullable column correctly inside a row
```

```
SELECT coalesce(base_pay, 0) + coalesce(bonus, 0) AS total_pay FROM payroll;
```

BEST PRACTICE · `NULLIF` FOR DIVISION

`x / nullif(y, 0)` is the idiomatic division guard: no `CASE`, no error, and a `NULL` result that honestly says “undefined” rather than a misleading zero. Learn it as one token.

COMMON MISTAKE

`greatest` and `least` compare *across columns within one row*, `max` and `min` aggregate *down a column across rows*. Reaching for `max(a, b)` gives “function max(integer, integer) does not exist” — which is the database telling you that you want `greatest`.

DIALECT

`COALESCE` and `NULLIF` are standard and portable. The two-argument shortcuts are not: MySQL has `IFNULL`, SQL Server has `ISNULL`, Oracle has `NVL` and `NVL2`. Prefer `COALESCE` everywhere — it works on all four and takes any number of arguments. Also note that `GREATEST` / `LEAST` return `NULL` if any argument is `NULL` in PostgreSQL and Oracle, but *ignore* `NULL` s in MySQL and SQL Server 2022 — a genuine behavioural split.

Part 5 Review

Quick Check ● BEGINNER

- 1 What does `NULL` mean, and why does `NULL = NULL` not return true?
- 2 Which binds tighter, `AND` or `OR`?
- 3 Is `BETWEEN` inclusive or exclusive?
- 4 Why can `NOT IN` return zero rows unexpectedly?
- 5 What does `WHERE` do with a condition that evaluates to unknown?
- 6 Difference between `coalesce` and `nullif`?
- 7 Difference between `greatest` and `max`?
- 8 Which `LIKE` patterns can use a B-tree index?
- 9 In what order are `CASE` branches evaluated?
- 10 What does `IS DISTINCT FROM` do that `<>` does not?

Practice ● EASY

- 1 Count employees with `department_id = 1`, with `<> 1`, and in total. Explain the discrepancy and fix it two ways.
- 2 List employees whose email is missing, using the only test that works.
- 3 Reproduce the `NOT IN` zero-rows bug on the practice data, then fix it with `NOT EXISTS`.
- 4 Write a query with a mixed `AND / OR` that gives the wrong answer, then parenthesise it correctly. Show both row counts.
- 5 Band employees into senior/mid/junior with `CASE`, then deliberately reverse the branches and explain the result.
- 6 Display every employee's email, substituting a message when it is missing.
- 7 Compute the average order value using `nullif` so the query is safe on an empty table. Prove it by filtering to a status that matches nothing.
- 8 Sort orders so that pending comes first, then paid, then everything else, and within each group by date.
- 9 Count orders by status in a single row, using both `FILTER` and `sum(CASE ...)`. Confirm the two agree.
- 10 Find customers whose name starts with R, case-insensitively, in three different ways. Say which can use an index.
- 11 Show that `'abc' || NULL` is `NULL`, and write the version that treats the missing part as empty.

- 12 Use `greatest` to find, for each order, the later of its order date and 1 March 2024.

Challenge ● INTERMEDIATE

- 1 A colleague reports that “the report is missing rows but nothing errors”. The `WHERE` clause is `status <> 'cancelled' AND discount_code <> 'NONE'`. Diagnose it and give two fixes with different semantics.
- 2 Write a single expression that returns an employee's manager's name, or `'no manager'`, or `'manager record missing'` — three distinguishable outcomes. Explain why `coalesce` alone is not enough.
- 3 Build a status summary row for `orders`: count and total value per status, all in one row, plus a percentage column, with no division-by-zero risk.
- 4 Explain why `WHERE lower(email) = 'x@y.com'` may be slow, and give two fixes.

Interview Drill ● ADVANCED

- 1 Explain three-valued logic and give a bug it causes.
- 2 Why do experienced engineers prefer `NOT EXISTS` to `NOT IN`?
- 3 How do aggregates treat `NULL`, and what is the one exception?
- 4 When is `BETWEEN` the wrong operator?
- 5 Give three ways a `WHERE` clause can prevent an index from being used.

Part 5 Solutions

Quick Check

- 1 Unknown. Two unknowns cannot be shown to be the same value, so the comparison is itself unknown rather than true.
- 2 `AND`.
- 3 Inclusive at both ends.
- 4 A `NULL` in the value list makes one conjunct unknown, so the whole expression can never be true.
- 5 Discards the row — `WHERE` keeps only true.
- 6 `coalesce` replaces `NULL` with a value; `nullif` replaces a value with `NULL`. Inverses of each other.
- 7 `greatest` works across columns in one row; `max` aggregates down a column across rows.
- 8 Only those anchored at the start, like `'abc%'`. A leading wildcard cannot.
- 9 Top to bottom; the first true branch wins and the rest are not evaluated.
- 10 Treats `NULL` as a comparable value, so `NULL IS DISTINCT FROM 1` is true where `NULL <> 1` is unknown.

Practice — selected

1 · 3, 5 and 9. The missing row is employee 109, whose `department_id` is `NULL`: `NULL <> 1` is unknown, so she satisfies neither predicate. Two fixes, and they mean different things — choose deliberately:

```
WHERE department_id <> 1 OR department_id IS NULL    -- explicit: "not dept 1, or unknown"
WHERE department_id IS DISTINCT FROM 1              -- same result, one predicate
```

If the business meaning is “employees we know are not in Engineering”, the original `<> 1` was correct and the count of 5 is right. The bug is never in the SQL alone; it is in the mismatch between the SQL and the sentence someone said in a meeting.

3 · The `NOT IN` bug.

```
-- Returns 0 rows, because employee 109 has a NULL department_id
SELECT department_name FROM departments
WHERE department_id NOT IN (SELECT department_id FROM employees);

-- Correct: departments with no employees
SELECT d.department_name
FROM departments d
WHERE NOT EXISTS (SELECT 1 FROM employees e
                  WHERE e.department_id = d.department_id);
```



```
department_name
-----
Research
```

`NOT EXISTS` is immune because it asks “did the subquery produce any row?”, a question with only two answers. This is why the practice data deliberately contains both an employee with no department and a department with no employees — the pair makes the difference visible.

7 • Safe average.

```
SELECT round(sum(total_amount) / nullif(count(*), 0), 2) AS avg_order
FROM   orders
WHERE  status = 'refunded';      -- matches nothing
```

```
avg_order
-----
```

One row, `NULL` — not an error, and honestly “undefined” rather than a misleading `0.00`. Note that `avg(total_amount)` would also have returned `NULL` here without any guard; the `nullif` pattern matters when you are computing a ratio of two aggregates yourself.

10 • Three case-insensitive prefix searches.

```
WHERE full_name ILIKE 'r%'          -- PostgreSQL only
WHERE lower(full_name) LIKE 'r%'    -- portable
WHERE full_name ~* '^r'             -- regex, PostgreSQL
```

All three scan by default. Only the second becomes index-friendly, and only with a matching expression index: `CREATE INDEX ON customers (lower(full_name) text_pattern_ops);`. The `text_pattern_ops` operator class is the detail people miss — under a non-C collation, a plain B-tree index cannot serve `LIKE` prefix matching, and adding it is what makes the index usable. `ILIKE` can be served by a trigram index instead.

Challenge — worked

1 • The missing rows. `discount_code <> 'NONE'` silently drops every order where `discount_code IS NULL` — which is almost certainly most of them, since “no discount” is usually recorded as `NULL` rather than the string `'NONE'`. The same trap applies to the status test if `status` is nullable. Two fixes with genuinely different meanings:

```
-- (a) "orders with no discount code, or a code other than NONE"
WHERE status <> 'cancelled'
  AND (discount_code IS NULL OR discount_code <> 'NONE')

-- (b) same, expressed NULL-safely
WHERE status IS DISTINCT FROM 'cancelled'
  AND discount_code IS DISTINCT FROM 'NONE'
```

The deeper fix is a schema one: having both `NULL` and `'NONE'` mean “no discount” is the actual defect. Pick one representation — `NULL`, and enforce it with a check — and this class of bug disappears rather than being patched.

2 • Three distinguishable outcomes. `coalesce` alone cannot do it because both “no manager” and “manager row missing” arrive as a `NULL` name from the left join — `coalesce` sees one `NULL` and cannot tell which caused it. You must test the *cause*, not the result:

```
SELECT e.full_name,
       CASE WHEN e.manager_id IS NULL THEN 'no manager'
            WHEN m.employee_id IS NULL THEN 'manager record missing'
            ELSE m.full_name
       END AS manager
FROM   employees e
LEFT   JOIN employees m ON m.employee_id = e.manager_id;
```

The order of the branches is the whole answer: check the foreign key first, then the join result. This distinction — “the value is absent” versus “the reference is broken” — is exactly what a `LEFT JOIN` plus `CASE` is for, and a good interviewer will ask for it precisely because `coalesce` is the tempting wrong answer.

3 • The summary row.

```
SELECT count(*)                AS total_orders,
       count(*) FILTER (WHERE status = 'delivered') AS delivered,
       count(*) FILTER (WHERE status = 'cancelled') AS cancelled,
       coalesce(sum(total_amount) FILTER (WHERE status <> 'cancelled'), 0) AS realised_value,
       round(100.0 * count(*) FILTER (WHERE status = 'delivered')
            / nullif(count(*), 0), 1) AS delivered_pct
FROM   orders;
```

Three defensive details. `100.0` forces numeric division. `nullif(count(*),0)` makes an empty table return `NULL` rather than raising. And `coalesce(sum(...), 0)` because `sum` over zero matching rows returns `NULL`, not `0` — a distinction that turns a financial dashboard blank instead of showing a legitimate zero.

4 • `lower(email)` . Wrapping the column in a function means a B-tree index on `email` cannot be used — the index holds the original values, not the lowercased ones, so the planner must scan and compute. Two fixes: build an expression index on exactly the expression used, `CREATE INDEX ON customers (lower(email));` ; or change the column's type to `citext` (a case-insensitive text type from the `citext` extension), after which plain `=` is both case-insensitive and index-friendly. The third, best answer for email specifically: normalise to lower case *on write*, so the stored value is already canonical and no function is needed on read at all.

Filtering & Sorting

Two clauses, and one of them contains the single most common scalability failure in web applications: `OFFSET`-based pagination. This part ends by replacing it with something that stays fast at page ten thousand.

IN THIS PART

6.1 WHERE in depth — sargability

6.2 ORDER BY and multi-column sorts

6.3 NULL ordering and collation

6.4 LIMIT and OFFSET

6.5 Why OFFSET pagination fails

6.6 Keyset pagination · Review

6.1 WHERE in depth — sargability

A predicate is **sargable** (from “Search ARGument able”) if the database can use an index to satisfy it. The rule is one sentence: **keep the indexed column bare on one side of the comparison**. Everything below is that sentence applied.

Not sargable	Sargable rewrite
WHERE upper(city) = 'MUMBAI'	WHERE city = 'Mumbai' , or an index on upper(city)
WHERE created_at::date = '2024-03-15'	WHERE created_at >= '2024-03-15' AND created_at < '2024-03-16'
WHERE salary * 12 > 800000	WHERE salary > 800000 / 12
WHERE extract(year FROM hire_date) = 2021	WHERE hire_date >= '2021-01-01' AND hire_date < '2022-01-01'
WHERE full_name LIKE '%nair%'	Trigram or full-text index — no rewrite possible
WHERE order_id::text = '1001'	WHERE order_id = 1001 — never cast the column to match a parameter

BEST PRACTICE

When a filter must transform a column, move the transformation to the *constant* side. When that is impossible, create an index on exactly the expression you use — but note the index only serves that expression, character for character, so `lower(x)` and `upper(x)` need separate indexes. Pick one convention per column.

PREDICATE ORDER DOES NOT MATTER (AND WHY PEOPLE THINK IT DOES)

Writing your most selective condition first has no effect: the planner reorders predicates based on statistics, not on the order you typed. What *does* matter is which conditions are indexed and how selective they actually are. Optimise by reading the plan (Part 21), never by rearranging text.

6.2 ORDER BY, 6.3 NULLs and collation

```
SELECT full_name, department_id, salary
FROM   employees
ORDER  BY department_id ASC NULLS LAST,
        salary          DESC,
        employee_id     ASC;           -- deterministic tie-break
```

Three points, in order of how often they bite.

1. **Sort direction is per column.** `ORDER BY a, b DESC` sorts `a` ascending and `b` descending — `DESC` does not carry backwards. Nearly everyone assumes otherwise once.
2. **Ties are not stable.** Two rows with equal sort keys may appear in either order, and the order may change between runs. Any query whose output feeds pagination, a diff, or a test assertion needs a unique final sort key — the primary key is always available and always works.
3. **NULL placement is engine-specific.** PostgreSQL and Oracle sort `NULL` s last ascending; MySQL and SQL Server sort them first. `NULLS FIRST / NULLS LAST` is standard and supported by PostgreSQL and Oracle; MySQL and SQL Server require the workaround `ORDER BY (col IS NULL), col`.

COMMON MISTAKE · SORTING NUMBERS AS TEXT

A `text` column containing `'10'`, `'9'`, `'100'` sorts as `'10'`, `'100'`, `'9'` — lexicographic, not numeric. If a column sorts strangely, check its type before you check your query. The fix is the type, not `ORDER BY col::integer`, which is both slow and unsargable.

NOTE · ORDER BY COSTS

A sort of a result larger than `work_mem` spills to disk and dominates the query's runtime. An index whose column order matches your `ORDER BY` lets the engine read rows already in order and skip the sort entirely — visible in `EXPLAIN` as the absence of a `Sort` node. This is why the index that helps a query is often chosen for its `ORDER BY`, not its `WHERE`.

6.4 LIMIT and OFFSET, 6.5 why they fail

```
-- page 1
SELECT order_id, order_date, total_amount
FROM   orders ORDER BY order_date DESC, order_id DESC LIMIT 20;

-- page 500
SELECT order_id, order_date, total_amount
FROM   orders ORDER BY order_date DESC, order_id DESC LIMIT 20 OFFSET 9980;
```

The second query looks like the first. It is not. **OFFSET n** does not skip to row n — it computes the first n rows and throws them away. Page 500 therefore does 500 pages of work to return 20 rows, and page 5,000 does fifty times more again. Latency grows linearly with page number, which is why the last page of a long list times out while the first page is instant.

FIGURE 6.5 — COST OF **OFFSET** VERSUS **KEYSET**, 10-MILLION-ROW TABLE

Page	OFFSET pagination	Keyset pagination
1	~1 ms	~1 ms
100	~15 ms	~1 ms
10,000	~1,400 ms	~1 ms
500,000	Effectively unusable	~1 ms

Indicative figures on an indexed sort column. The shape is what matters: **OFFSET** is $O(n)$ in the page number, keyset is $O(1)$.

There is a second, subtler defect. Between the user loading page 2 and page 3, rows are inserted or deleted. Offsets shift, so a row that was going to appear on page 3 moves to page 2 and is **never shown**, or is shown twice. For an infinite-scroll feed this manifests as items mysteriously missing — a bug that is almost impossible to reproduce and is inherent to the technique.

6.6 Keyset pagination

Also called cursor or seek pagination. Instead of “skip 9,980 rows”, say “give me the rows after the last one I saw”. The database seeks straight into the index.

```
-- Page 1: unchanged
SELECT order_id, order_date, total_amount
FROM   orders
ORDER  BY order_date DESC, order_id DESC
LIMIT  20;

-- Next page: pass the last row's sort key back as the cursor
SELECT order_id, order_date, total_amount
FROM   orders
WHERE  (order_date, order_id) < (DATE '2024-03-28', 1005)  -- row-value comparison
ORDER  BY order_date DESC, order_id DESC
LIMIT  20;
```

The row-value comparison `(a, b) < (x, y)` is the key trick. It means exactly “ $a < x$, or ($a = x$ and $b < y$)” — lexicographic, matching the sort order precisely — and PostgreSQL can satisfy it with a single index seek on `(order_date DESC, order_id DESC)`. Written out longhand as `a < x OR (a = x AND b < y)`, the planner often cannot use the index as cleanly.

	OFFSET	Keyset
Cost at page n	Grows linearly	Constant
Stable under writes	No — rows skip and repeat	Yes
Jump to arbitrary page	Yes	No — next/previous only
Total page count	Available (at the cost of a count query)	Not naturally available
Fits	Small admin tables with real page numbers	Feeds, infinite scroll, APIs, exports

BEST PRACTICE

Use keyset for anything user-facing that can grow. Encode the cursor as an opaque base64 token rather than exposing raw column values — it keeps the API free to change its sort key later, and stops clients constructing cursors by hand. Keep `OFFSET` for internal admin screens over small tables, where jumping to page 7 genuinely matters.

COMMON MISTAKE

Keyset pagination on a non-unique sort key. If you paginate by `order_date` alone and forty orders share a date, the cursor cannot distinguish them and rows are skipped or repeated at the boundary.

The sort key must be unique — always append the primary key, both to the `ORDER BY` and to the row-value comparison.

INTERVIEW · ADVANCED

Q. Your API's page 5,000 takes eight seconds. Why, and how do you fix it? — `OFFSET` computes and discards 100,000 rows. Switch to keyset pagination with a unique compound sort key and a row-value comparison, backed by a matching index. **Interview tip:** volunteer the trade-off unprompted — you lose “jump to page N” and the total count. Naming what you give up is what makes the answer credible; a candidate who presents keyset as free has not shipped it.

Part 6 Review

Quick Check ● BEGINNER

- 1 What does sargable mean?
- 2 Does `ORDER BY a, b DESC` sort `a` descending?
- 3 Where do `NULL`s sort in PostgreSQL ascending, and in MySQL?
- 4 Why is `OFFSET 10000` slow?
- 5 Name the two problems `OFFSET` pagination has.
- 6 What must be true of a keyset pagination sort key?
- 7 What does `(a,b) < (x,y)` mean?
- 8 Does writing your most selective predicate first help?

Practice ● EASY

- 1 Rewrite six non-sargable predicates from §6.1 against the practice tables and confirm each returns the same rows.
- 2 List employees sorted by department ascending with unassigned last, then salary descending, with a deterministic tie-break.
- 3 List all 2024 orders using a half-open date range, newest first.
- 4 Return orders 3–5 by date using `OFFSET`, then the same rows using keyset. Show both queries.
- 5 Create a text column containing `'9'`, `'10'`, `'100'`, sort it, and explain the order.
- 6 Write a query that would skip a row if a new order were inserted between two `OFFSET` pages. Explain exactly which row.
- 7 Sort products by category, then price descending, with a custom category order putting Peripherals first.
- 8 Find every customer created in the first half of 2024, sargably.

Challenge ● INTERMEDIATE

- 1 Design a keyset-paginated API endpoint for an order feed sorted by date descending. Specify the cursor's contents, the query, the index, and the behaviour at the last page.
- 2 A screen must show “page 3 of 847”. Keyset gives no total. Give three ways to satisfy the requirement and the cost of each.
- 3 Users can sort the feed by date, amount, or status. Explain what this does to your index strategy and your cursor design.

Interview Drill ● ADVANCED

- 1 Explain sargability with three examples.

- 2 Walk through converting an `OFFSET` endpoint to keyset, including what you tell the frontend team.
- 3 Why can a query be correct and still return different rows on two runs?
- 4 When is `OFFSET` pagination the right choice?
- 5 How does an index remove a sort from a plan?

Part 6 Solutions

Quick Check

- 1 A predicate the engine can satisfy with an index, because the indexed column appears bare on one side of the comparison.
- 2 No. `DESC` applies only to `b`; `a` remains ascending.
- 3 Last in PostgreSQL (and Oracle); first in MySQL (and SQL Server). Always be explicit.
- 4 Because it computes the first 10,000 rows and discards them; it does not seek.
- 5 Cost grows linearly with page number, and concurrent inserts or deletes shift offsets so rows are skipped or duplicated.
- 6 It must be unique — append the primary key if the natural sort column is not.
- 7 Lexicographic comparison: `a < x`, or `a = x` and `b < y`.
- 8 No. The planner reorders predicates using statistics.

Practice — selected

2 ·

```
SELECT full_name, department_id, salary
FROM   employees
ORDER BY department_id ASC NULLS LAST,
        salary DESC,
        employee_id ASC;
```

Nisha Verma lands at the bottom rather than the top, which is what a reader expects from “unassigned”. The `employee_id` tie-break does nothing on this data — no two employees share a department and a salary — and that is exactly why it should be there: it costs nothing and guarantees the order stays fixed when the data changes.

4 · Both paginations.

```
-- OFFSET
SELECT order_id, order_date FROM orders
ORDER BY order_date, order_id LIMIT 3 OFFSET 2;

-- Keyset, given that row 2 was (2024-01-19, 1002)
SELECT order_id, order_date FROM orders
WHERE (order_date, order_id) > (DATE '2024-01-19', 1002)
ORDER BY order_date, order_id LIMIT 3;
```

order_id	order_date
1003	2024-02-11
1004	2024-03-03
1005	2024-03-28

Identical results. The difference is invisible at seven rows and decisive at seven million — which is the pedagogical problem with pagination: the correct technique looks like unnecessary complexity right up until the moment it is an incident.

6 • The skipped row. Page 1 is `ORDER BY order_date LIMIT 3 OFFSET 0` → orders 1001, 1002, 1003. Now someone inserts an order dated 2024-01-10, which sorts into position 2. Page 2 (`OFFSET 3`) now begins at what is *currently* the fourth row — which is order 1003, already shown — and order 1004 is pushed to position 5. The user sees 1003 twice and, at the far end of the list, one row is pushed off entirely. Deletions do the reverse: a row is skipped and never displayed. No amount of care in the query fixes this; only changing the technique does.

7 •

```
SELECT product_name, category, price
FROM   products
ORDER  BY CASE category WHEN 'Peripherals' THEN 1
              WHEN 'Displays' THEN 2
              ELSE 3 END,
        category,
        price DESC;
```

The trailing `category` sort matters: everything in the `ELSE` bucket shares rank 3, so without it the order within that bucket is arbitrary. Note also that this `CASE` makes the sort unsargable — no index can serve it, so the engine always sorts. On a large table, the better design is a `sort_order` column on a categories lookup table, which *can* be indexed.

Challenge — worked

1 • The endpoint. Cursor: base64 of `{"d": "2024-03-28", "id": 1005}` — the complete sort key, never an offset. **Query:** the row-value comparison shown in §6.6, with `LIMIT 21` rather than 20. **The 21st row is the trick:** if it comes back, there is a next page and you return a cursor; if it does not, you return `null` and the client stops. This gives you has-more without a second count query. **Index:** `CREATE INDEX ON orders (order_date DESC, order_id DESC);` — the direction must match the query, or PostgreSQL will scan the index backwards (which it can do, but which prevents combining with other conditions cleanly). **Last page:** fewer than 21 rows, cursor `null`. **Also validate the cursor** — it is user input, and decoding it straight into SQL without parameterisation is an injection vector (Part 28).

2 • “Page 3 of 847”. Three options, cheapest first. **(a) Don't.** Replace it with “load more” or an approximate count. Most product requirements for an exact total do not survive being asked why. **(b) An estimated count** from the planner's own statistics — `SELECT reltuples FROM pg_class WHERE relname = 'orders'`,

or parse the row estimate from `EXPLAIN` for a filtered query. Free, and accurate to within a few percent on a well-analyzed table; display it as “about 17,000 results”. **(c) An exact count**, cached. Run `count(*)` with the same filters, cache it against the filter set for 30–60 seconds, and accept staleness. Costly on large tables because a filtered count must examine every matching row — this is the option to choose only when the product genuinely requires exactness, such as an export that must state a definite row count.

3 • User-selectable sort. Each sort option needs its own index, because a keyset cursor is meaningful only within one ordering: `(order_date DESC, order_id DESC)`, `(total_amount DESC, order_id DESC)`, `(status, order_id)`. That is three indexes with their write cost, which is the real argument for limiting the number of sortable columns — a UI decision with a storage bill. The cursor must therefore record *which* sort it belongs to, so switching sort order invalidates it: encode `{"sort": "amount_desc", "v": [21200.00, 1005]}` and reject a cursor whose sort does not match the request rather than silently returning nonsense. And every sort key must still end in the primary key — amounts and statuses tie constantly, and a tied keyset cursor loses rows.

SQL Functions

A working reference, not a catalogue. Each function appears with the case it is actually used for; the ones you will reach for weekly are marked, and the dialect differences — which are extensive here — are tabulated rather than scattered.

IN THIS PART

7.1 String functions

7.2 Numeric functions

7.3 Date and time functions

7.4 NULL functions · Review

7.1 String functions

Function	Returns	Example → result
<code>a b</code>	Concatenation. NULL anywhere makes the whole result NULL .	<code>'a' 'b' → ab</code>
<code>concat(a,b,...)</code>	Concatenation that treats NULL as empty	<code>concat('a',NULL) → a</code>
<code>concat_ws(sep,...)</code>	Joins with a separator, skipping NULL s	<code>concat_ws(' ', 'Pune',NULL) → Pune</code>
<code>length(s)</code>	Characters (not bytes)	<code>length('नमस्ते') → 6</code>
<code>lower</code> / <code>upper</code> / <code>initcap</code>	Case conversion	<code>initcap('asha menon') → Asha Menon</code>
<code>trim</code> / <code>ltrim</code> / <code>rtrim</code>	Removes leading/trailing characters (spaces by default)	<code>trim(' x ') → x</code>
<code>substring(s FROM p FOR n)</code>	Substring; 1-indexed	<code>substring('abcdef' FROM 2 FOR 3) → bcd</code>
<code>left(s,n)</code> / <code>right(s,n)</code>	First / last n characters	<code>left('abcdef',3) → abc</code>
<code>replace(s,from,to)</code>	Replaces every occurrence	<code>replace('a-b','-','_') → a_b</code>
<code>position(sub IN s)</code>	1-based index, or 0 if absent	<code>position('b' IN 'abc') → 2</code>
<code>split_part(s,sep,n)</code>	The nth field	<code>split_part('a@b.com','@',2) → b.com</code>
<code>lpad</code> / <code>rpadd</code>	Pads to a width	<code>lpad('7',3,'0') → 007</code>
<code>regexp_replace(s,p,r,'g')</code>	Regex replace; 'g' for all occurrences	<code>regexp_replace('a1b2','\d','','g') → ab</code>
<code>string_agg(s,sep)</code>	Aggregate — joins values down a group (Part 8)	<code>string_agg(name, ' , ')</code>

COMMON MISTAKE · || VERSUS CONCAT

```
SELECT full_name || ', ' || city AS with_pipes,      -- NULL if city is NULL
       concat_ws(', ', full_name, city) AS with_ws   -- 'Nisha Verma'
FROM   employees;
```

A single missing middle name silently blanks an entire address label built with `||`. `concat_ws` is almost always what you meant. The exception: when a `NULL` *should* propagate — you would rather show nothing than a half-built string — `||` is the honest choice.

WORKED EXAMPLE – PARSING AND NORMALISING

```
SELECT full_name,
       lower(split_part(email, '@', 1)) AS local_part,
       split_part(email, '@', 2) AS domain,
       initcap(trim(full_name)) AS normalised,
       lpad(employee_id::text, 6, '0') AS padded_id,
       regexp_replace(full_name, '\s+', ' ', 'g') AS single_spaced
FROM   employees
WHERE  email IS NOT NULL
ORDER BY domain, local_part;
```

DIALECT · STRINGS

Need	PostgreSQL	MySQL	SQL Server	Oracle
Concat	<code> </code> , <code>concat</code>	<code>CONCAT</code> (<code> </code> is OR!)	<code>+</code> , <code>CONCAT</code>	<code> </code> , <code>CONCAT</code>
Length	<code>length</code>	<code>CHAR_LENGTH</code>	<code>LEN</code>	<code>LENGTH</code>
Substring	<code>substring</code>	<code>SUBSTRING</code>	<code>SUBSTRING</code>	<code>SUBSTR</code>
Position	<code>position</code> , <code>strpos</code>	<code>LOCATE</code>	<code>CHARINDEX</code>	<code>INSTR</code>
Group concat	<code>string_agg</code>	<code>GROUP_CONCAT</code>	<code>STRING_AGG</code>	<code>LISTAGG</code>

Note the trap: in MySQL's default mode `||` means logical OR, so `'a' || 'b'` returns `0`, not `'ab'`. Porting concatenation from PostgreSQL to MySQL produces numbers instead of strings, with no error.

7.2 Numeric functions

Function	Returns	Example → result
<code>round(n, d)</code>	Rounds to d decimals	<code>round(2.567, 2)</code> → <code>2.57</code>
<code>ceil</code> / <code>floor</code>	Up / down to an integer	<code>ceil(2.1)</code> → <code>3</code>
<code>trunc(n, d)</code>	Truncates — does not round	<code>trunc(2.99, 1)</code> → <code>2.9</code>
<code>abs(n)</code>	Absolute value	<code>abs(-5)</code> → <code>5</code>
<code>mod(a,b)</code> / <code>a % b</code>	Remainder	<code>mod(7,3)</code> → <code>1</code>
<code>power(a,b)</code> / <code>a ^ b</code>	Exponent	<code>power(2,10)</code> → <code>1024</code>
<code>sqrt</code> , <code>exp</code> , <code>ln</code> , <code>log</code>	The usual	<code>sqrt(16)</code> → <code>4</code>
<code>random()</code>	Float in [0,1)	Sampling, shuffling
<code>sign(n)</code>	-1, 0 or 1	<code>sign(-3)</code> → <code>-1</code>
<code>width_bucket(v,lo,hi,n)</code>	Histogram bucket number	Bucketing salaries

```
-- A salary histogram in four bands, without a CASE ladder
SELECT width_bucket(salary, 40000, 100000, 4) AS band,
       count(*)                               AS employees,
       min(salary), max(salary)
FROM   employees
GROUP BY band
ORDER BY band;

-- A reproducible random sample
SELECT setseed(0.42);
SELECT * FROM orders ORDER BY random() LIMIT 3;
```

COMMON MISTAKE · ROUNDING DIFFERS BY TYPE

```
SELECT round(2.5::numeric),          -- 3  (half away from zero)
       round(2.5::double precision); -- 2  (banker's rounding, half to even)
```

The same literal, the same function, two answers. For money, always operate on `numeric` so rounding is predictable and matches what an accountant expects. And round **once**, at the point of display — rounding intermediate values and then summing produces a total that does not match the sum of the displayed rows, which is the most common complaint auditors raise about a report.

BEST PRACTICE

`ORDER BY random() LIMIT n` sorts the whole table to pick `n` rows — fine on thousands, catastrophic on millions. For a large sample use `TABLESAMPLE SYSTEM (1)`, which reads a random fraction of pages instead.

7.3 Date and time functions

Function	Returns	Example
<code>current_date</code> , <code>now()</code>	Today; the current instant	<code>now()</code>
<code>age(a, b)</code>	A calendar-aware interval between two dates	<code>5 years 4 mons</code>
<code>date_trunc(unit, ts)</code>	Floors a timestamp to a unit — the workhorse of time-series SQL	<code>date_trunc('month', now())</code>
<code>extract(field FROM ts)</code>	One numeric component	<code>extract(year FROM d)</code>
<code>date_part(field, ts)</code>	Same thing, function syntax	<code>date_part('dow', d)</code>
<code>to_char(ts, fmt)</code>	Formats a timestamp as text	<code>to_char(d, 'DD Mon YYYY')</code>
<code>to_date</code> / <code>to_timestamp</code>	Parses text with an explicit format	<code>to_date(s, 'DD/MM/YYYY')</code>
<code>ts AT TIME ZONE 'zone'</code>	Converts between zoned and unzoned	<code>now() AT TIME ZONE 'UTC'</code>
<code>generate_series(a,b,step)</code>	A set of dates — essential for gap-free reports	See below

THE TWO YOU WILL USE MOST

```
-- date_trunc: monthly revenue
SELECT date_trunc('month', order_date)::date AS month,
       count(*)                               AS orders,
       sum(total_amount)                       AS revenue
FROM   orders
WHERE  status <> 'cancelled'
GROUP BY month
ORDER BY month;
```

month	orders	revenue
2024-01-01	2	29700.00
2024-02-01	1	11000.00
2024-03-01	1	21200.00
2024-06-01	1	3100.00
2024-07-01	1	1750.00

April and May are missing — not because the query is wrong, but because there were no orders and `GROUP BY` cannot invent rows that do not exist. A chart drawn from this jumps from March to June and misrepresents the trend. The fix is to generate the calendar and left-join onto it:

```

SELECT m.month::date,
       count(o.order_id)           AS orders,
       coalesce(sum(o.total_amount), 0) AS revenue
FROM   generate_series(DATE '2024-01-01', DATE '2024-07-01', INTERVAL '1 month')
AS m(month)
LEFT   JOIN orders o
       ON o.order_date >= m.month
       AND o.order_date < m.month + INTERVAL '1 month'
       AND o.status <> 'cancelled'
GROUP BY m.month
ORDER BY m.month;

```

month	orders	revenue
2024-01-01	2	29700.00
2024-02-01	1	11000.00
2024-03-01	1	21200.00
2024-04-01	0	0.00
2024-05-01	0	0.00
2024-06-01	1	3100.00
2024-07-01	1	1750.00

BEST PRACTICE · THE GAP-FILLING PATTERN

Generate the time series, `LEFT JOIN` the data onto it, `coalesce` the aggregates. Learn this as one unit — it is the correct answer to a large fraction of all reporting requirements, and Part 35 names it as a pattern. Note the two details: the join condition uses a half-open range (§4.5), and the status filter sits **in the `ON` clause, not in `WHERE`** — putting it in `WHERE` would turn the left join back into an inner one and re-create the gaps (Part 9).

COMMON MISTAKE

`extract(dow FROM d)` returns 0 for Sunday in PostgreSQL, 1 for Sunday in MySQL's `DAYOFWEEK`, and depends on `DATEFIRST` in SQL Server. Never hard-code a weekday number across engines, and never assume week boundaries agree — `date_trunc('week', ...)` starts Monday in PostgreSQL, which is not universal either.

7.4 NULL functions, recapped

§5.9 covered `coalesce`, `nullif`, `greatest` and `least`. Two additional forms complete the set:

```
-- IS NOT DISTINCT FROM: NULL-safe equality, useful in joins on nullable columns
SELECT * FROM a JOIN b ON a.code IS NOT DISTINCT FROM b.code;
```

```
-- coalesce in an ORDER BY, to control where unknowns land
ORDER BY coalesce(shipped_at, order_date::timestampz) DESC;
```

COMMON MISTAKE

`coalesce` in a `WHERE` or `JOIN` condition wraps the column and destroys sargability — `WHERE coalesce(status, 'pending') = 'pending'` cannot use an index on `status`. Prefer `WHERE status = 'pending' OR status IS NULL`, which can. Reserve `coalesce` for the select list and for `ORDER BY`, where the cost is bounded by the rows you are already returning.

Part 7 Review

Quick Check ● BEGINNER

- 1 Difference between `||`, `concat` and `concat_ws` when an argument is `NULL`?
- 2 Is `substring` 0-indexed or 1-indexed?
- 3 Why does `round(2.5)` give different answers on different types?
- 4 What does `date_trunc('month', ts)` return?
- 5 Why does a monthly report skip months with no data, and how do you fix it?
- 6 What does `age()` return, and why not subtract dates instead?
- 7 Why is `ORDER BY random() LIMIT 10` slow on a big table?
- 8 In MySQL, what does `'a' || 'b'` evaluate to?

Practice ● EASY

- 1 Extract the domain from every employee email and count employees per domain.
- 2 Build a display label `"Name – City"` for every customer, correct when city is missing. Show both the broken and the correct version.
- 3 Produce zero-padded six-digit order references from `order_id`.
- 4 Compute each employee's tenure in whole years and in months.
- 5 Monthly order counts for 2024 with no gaps, including months with zero orders.
- 6 Bucket employees into four salary bands with `width_bucket` and label each band.
- 7 Format order dates as `"05 Jan 2024"`, then parse that string back to a date.
- 8 Find products whose name contains a digit, using a regex.
- 9 Show `now()` in IST, UTC and New York in a single row.
- 10 Normalise a messy name column: trim, collapse repeated spaces, title-case it.
- 11 Compute the day of week for every order date and explain why the number is not portable.
- 12 Round each product's price to the nearest 50 rupees.

Challenge ● INTERMEDIATE

- 1 Build a daily revenue report for March 2024 with every day present, running in one query, and explain each defensive element.
- 2 A CSV column holds dates as a mix of `15/03/2024`, `2024-03-15` and `Mar 15 2024`. Write a robust parse and say what you do with rows that still fail.
- 3 Write a query that reports, per customer, their first order date, last order date, and days between — handling customers with exactly one order and with none.

Part 7 Solutions

Quick Check

- 1 `||` returns `NULL` if any operand is `NULL`; `concat` treats `NULL` as empty; `concat_ws` does the same and also omits the separator for skipped values.
- 2 1-indexed.
- 3 `numeric` rounds half away from zero; `double precision` uses banker's rounding, half to even.
- 4 The first instant of that month, as a timestamp — not a month number.
- 5 `GROUP BY` can only group rows that exist. Generate the series and `LEFT JOIN` the data onto it, then `coalesce` the aggregates.
- 6 A calendar-aware interval. Subtracting dates gives a day count, which drifts when converted to years and mishandles month lengths.
- 7 It assigns a random value to every row and sorts the entire table before taking ten.
- 8 `0` — in MySQL's default mode `||` is logical OR, not concatenation.

Practice — selected

2 • The display label.

```
SELECT full_name || ' - ' || city          AS broken,  
       concat_ws(' - ', full_name, city)  AS correct  
FROM   customers;
```

Add a customer with a `NULL` city and the `broken` column goes entirely blank — not “Name —”, but `NULL`, losing the name too. This is the failure mode worth remembering: `||` does not degrade gracefully, it erases.

5 • Gapless monthly counts.

```
SELECT m.month::date, count(o.order_id) AS orders  
FROM   generate_series(DATE '2024-01-01', DATE '2024-12-01', INTERVAL '1 month')  
AS m(month)  
LEFT   JOIN orders o  
       ON o.order_date >= m.month  
       AND o.order_date <  m.month + INTERVAL '1 month'  
GROUP  BY m.month  
ORDER  BY m.month;
```

Use `count(o.order_id)`, not `count(*)`. For a month with no orders the left join produces one row with all-`NULL` order columns, and `count(*)` would count that row and report 1. `count(column)` skips `NULL`s and correctly reports 0. This is the single most common bug in gap-filled reports, and Part 8 explains the mechanism.

9 • Three time zones.


```
SELECT now() AS session_tz,
       now() AT TIME ZONE 'UTC' AS utc,
       now() AT TIME ZONE 'Asia/Kolkata' AS ist,
       now() AT TIME ZONE 'America/New_York' AS nyc;
```

`AT TIME ZONE` applied to a `timestampz` returns a plain `timestamp` — the local wall-clock reading in that zone. Applied to a plain `timestamp` it does the opposite, interpreting the value as being in that zone and returning a `timestampz`. Two operators sharing one syntax, distinguished by the input type; reading the direction wrong is a common source of doubly-shifted timestamps.

10 · Normalising names.

```
SELECT initcap(regexp_replace(trim(full_name), '\s+', ' ', 'g')) AS clean
FROM   customers;
```

Order matters: trim the ends, collapse interior runs, then case. Note the honest limitation — `initcap` produces `Mcdonald` and `O'brien`, and will mangle `d'Souza`. Name casing is not solvable with a function; the correct engineering answer is usually to store what the user typed and not normalise display names at all.

Challenge — worked

1 · Daily revenue, March 2024.

```
SELECT d.day::date AS day,
       count(o.order_id) AS orders,
       coalesce(sum(o.total_amount), 0)::numeric(12,2) AS revenue
FROM   generate_series(DATE '2024-03-01', DATE '2024-03-31', INTERVAL '1 day') AS d(day)
LEFT   JOIN orders o
      ON o.order_date = d.day
      AND o.status <> 'cancelled'
GROUP BY d.day
ORDER BY d.day;
```

Five defensive elements, each preventing a specific wrong answer. **(i)** `generate_series` guarantees all 31 days appear. **(ii)** `LEFT JOIN`, so days with no orders survive. **(iii)** the status filter is in `ON`, not `WHERE` — in `WHERE` it would eliminate the all-`NULL` rows produced by the left join and the gaps would return. **(iv)** `count(o.order_id)` rather than `count(*)`, so empty days report 0. **(v)** `coalesce(sum(...), 0)`, because `sum` of no rows is `NULL`, and a revenue chart with `NULL`s renders as a broken line rather than a zero.

2 · Mixed date formats. Do not attempt one clever expression. Test formats in order with a `CASE` guarded by regexes, so each branch parses only strings it can actually handle:

```

SELECT raw_date,
       CASE
         WHEN raw_date ~ '^\\d{4}-\\d{2}-\\d{2}$'      THEN to_date(raw_date, 'YYYY-MM-DD')
         WHEN raw_date ~ '^\\d{2}/\\d{2}/\\d{4}$'     THEN to_date(raw_date, 'DD/MM/YYYY')
         WHEN raw_date ~ '^\\[A-Za-z]{3} \\d{1,2} \\d{4}$' THEN to_date(raw_date, 'Mon DD YYYY')
       END AS parsed
FROM   staging_orders;

```

Rows matching nothing yield `NULL` from the absent `ELSE` — deliberately, so `WHERE parsed IS NULL` gives you the exact list of unparseable values to inspect. Do not guess at them and do not drop them silently; quarantine them in a rejects table and report the count. The genuinely dangerous case is `03/04/2024`, which is valid under both `DD/MM` and `MM/DD` — no amount of code resolves that ambiguity, so it must be settled by asking the data's owner. Saying so is part of the correct answer.

3 · Per-customer order span.

```

SELECT c.customer_id,
       c.full_name,
       min(o.order_date)           AS first_order,
       max(o.order_date)           AS last_order,
       max(o.order_date) - min(o.order_date) AS days_span,
       count(o.order_id)           AS order_count
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id
GROUP  BY c.customer_id, c.full_name
ORDER  BY order_count DESC, c.customer_id;

```

customer_id	full_name	first_order	last_order	days_span	order_count
1	Kavya Reddy	2024-01-05	2024-07-02	179	3
2	Rohan Gupta	2024-01-19	2024-03-28	69	2
3	Fatima Sheikh	2024-03-03	2024-03-03	0	1
4	Daniel Fernandes	2024-06-14	2024-06-14	0	1
5	Priya Anand				0

The `LEFT JOIN` keeps Priya, who has never ordered — an `INNER JOIN` would silently drop her, and “customers with no orders” is precisely the segment a retention report exists to find. Single-order customers correctly show a span of 0 rather than `NULL`, because `min` and `max` over one row are the same date. And `count(o.order_id)` gives Priya 0 where `count(*)` would give her 1.

Aggregations

Aggregation collapses many rows into one. It is where SQL stops being a way to fetch records and starts being a way to answer questions — and where the first genuinely conceptual jump in the language happens.

IN THIS PART

8.1 The five aggregate functions

8.2 COUNT and NULL

8.3 GROUP BY

8.4 The grouping rule

8.5 WHERE vs HAVING

8.6 Conditional aggregation and FILTER

8.7 GROUPING SETS, ROLLUP, CUBE

Review · Cumulative Test

8.1 The five, and 8.2 COUNT and NULL

Function	Returns	NULL behaviour
<code>count(*)</code>	Number of rows	Counts every row, including all- <code>NULL</code> ones
<code>count(col)</code>	Number of non- <code>NULL</code> values	Skips <code>NULL</code> s
<code>count(DISTINCT col)</code>	Number of distinct non- <code>NULL</code> values	Skips <code>NULL</code> s
<code>sum(col)</code>	Total	Skips <code>NULL</code> s; returns <code>NULL</code> (not 0) over zero rows
<code>avg(col)</code>	Mean	Skips <code>NULL</code> s — they are excluded from the denominator too
<code>min</code> / <code>max</code>	Extremes	Skip <code>NULL</code> s; work on text and dates too

```
SELECT count(*)           AS all_rows,      -- 9
       count(email)       AS with_email,    -- 8
       count(department_id) AS with_dept,   -- 8
       count(DISTINCT department_id) AS depts, -- 4
       sum(salary)         AS payroll,
       round(avg(salary), 2) AS mean_salary
FROM   employees;
```

all_rows		with_email		with_dept		depts		payroll		mean_salary
-----+-----+-----+-----+-----+-----										
9		8		8		4		593500.00		65944.44

COMMON MISTAKE · AVG AND NULL

`avg(bonus)` over ten employees where four have no bonus divides by **six**, not ten. If the business meaning is “average bonus across all staff, counting no bonus as zero”, you must write `avg(coalesce(bonus, 0))` or `sum(bonus)/count(*)`. Both are defensible; they answer different questions, and nobody can tell which one a bare `avg` meant. State the intent in a comment.

NOTE · `COUNT(*)` VS `COUNT(1)`

Identical in every modern engine — the planner treats them the same, and `count(1)` has no performance advantage. It is a myth from very old Oracle versions. Use `count(*)`; it is the standard idiom and it reads as “count rows”.

INTERVIEW · BEGINNER

Q. Difference between `count(*)` and `count(column)` ? — `count(*)` counts rows; `count(column)` counts non- `NULL` values in that column. The follow-up worth pre-empting: on a `LEFT JOIN`, use `count(right_table.id)`, because `count(*)` counts the all- `NULL` padding row and reports 1 where the answer is 0.

8.3 GROUP BY, and 8.4 the grouping rule

`GROUP BY` partitions rows into groups by the values of some columns, then applies aggregates within each group. One output row per group, always.

```
SELECT d.department_name,
       count(e.employee_id)      AS headcount,
       round(avg(e.salary), 2)   AS avg_salary,
       max(e.salary)             AS top_salary
FROM   departments d
LEFT   JOIN employees e ON e.department_id = d.department_id
GROUP  BY d.department_id, d.department_name
ORDER  BY headcount DESC, d.department_name;
```

department_name	headcount	avg_salary	top_salary
Engineering	3	81333.33	92000.00
Sales	2	64750.00	71500.00
Support	2	49500.00	52000.00
Finance	1	76000.00	76000.00
Research	0		

(5 rows)

Research appears with a headcount of 0 and `NULL` averages — correct on both counts. It appears because of the `LEFT JOIN`; its headcount is 0 rather than 1 because of `count(e.employee_id)`; and its average is `NULL` rather than 0 because the average of no values is genuinely undefined, not zero.

THE GROUPING RULE

THE RULE

Every column in the `SELECT` list must either appear in `GROUP BY` or be inside an aggregate function. No exceptions.

```
-- ERROR: column "e.full_name" must appear in the GROUP BY clause
--          or be used in an aggregate function
SELECT department_id, full_name, avg(salary)
FROM   employees
GROUP  BY department_id;
```

The error is the database refusing to guess. The group “department 1” contains three names; the query asks for one `full_name` and has not said which. Three legitimate resolutions, depending on what you actually meant:

```
-- (a) You wanted a list
SELECT department_id, string_agg(full_name, ' ' ORDER BY full_name) AS names,
       round(avg(salary),2) AS avg_salary
FROM   employees GROUP BY department_id;

-- (b) You wanted one specific row per group – the top earner
SELECT DISTINCT ON (department_id) department_id, full_name, salary
FROM   employees ORDER BY department_id, salary DESC; -- PostgreSQL

-- (c) You wanted the detail rows AND the group average side by side
SELECT department_id, full_name, salary,
       round(avg(salary) OVER (PARTITION BY department_id), 2) AS dept_avg
FROM   employees; -- a window function: Part 13
```

Option (c) is the important one. A window function aggregates **without collapsing rows**, which is precisely what people want when they hit this error and try to fight `GROUP BY`. If you find yourself wanting detail and a total in the same result, you want Part 13, not a cleverer `GROUP BY`.

DIALECT · MYSQL'S PERMISSIVENESS

MySQL historically allowed the illegal query above, silently returning an arbitrary name from each group. Since 5.7 `ONLY_FULL_GROUP_BY` is on by default and it errors like everyone else — but legacy code and legacy configurations still run in permissive mode, producing results that look right and are not reproducible. If you inherit a MySQL codebase, check that setting first. PostgreSQL does allow one relaxation: columns functionally dependent on a grouped primary key may be omitted, so `GROUP BY d.department_id` alone permits selecting `d.department_name`.

8.5 WHERE vs HAVING

The most-asked question in SQL interviews has a one-line answer that follows entirely from the logical evaluation order (Volume I, §1.11): **WHERE filters rows before grouping; HAVING filters groups after.**

```
SELECT d.department_name,
       count(*)           AS headcount,
       round(avg(e.salary), 2) AS avg_salary
FROM   employees e
JOIN   departments d ON d.department_id = e.department_id
WHERE  e.hire_date < '2023-01-01'      -- rows: exclude recent hires
GROUP BY d.department_id, d.department_name
HAVING count(*) >= 2                  -- groups: only departments with 2+ such staff
ORDER BY avg_salary DESC;
```

department_name	headcount	avg_salary
Engineering	3	81333.33
Sales	2	64750.00

Both filters are needed and neither can do the other's job. `hire_date` is a property of a row, so it must be filtered before the rows are combined — putting it in `HAVING` would be wrong even where the engine permits it. `count(*) >= 2` is a property of a group and does not exist until the group does, so it cannot go in `WHERE`.

BEST PRACTICE · PERFORMANCE

Filter as early as possible. A condition that could go in either clause belongs in `WHERE`, because it removes rows before the expensive grouping work. `HAVING` is only for conditions on aggregates.

INTERVIEW · BEGINNER

Q. WHERE vs HAVING ? — Answer with the evaluation order, not a definition: “`WHERE` runs at step 3, before `GROUP BY` at step 4, so it filters individual rows and cannot see aggregates. `HAVING` runs at step 5, after grouping, so it filters groups and can. If a condition can go in `WHERE`, put it there — fewer rows reach the grouping.” **Common wrong answer:** “`HAVING` is for aggregates and `WHERE` is for columns” — true but shallow; it does not explain *why*, and the follow-up question is always *why*.

8.6 Conditional aggregation and FILTER

Counting different things in one pass over the table — the technique behind almost every dashboard row.

```
SELECT c.city,
       count(*)                               AS customers,
       count(*) FILTER (WHERE o.status = 'delivered') AS delivered,
       count(*) FILTER (WHERE o.status = 'cancelled') AS cancelled,
       coalesce(sum(o.total_amount) FILTER
                (WHERE o.status <> 'cancelled'), 0) AS realised
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id
GROUP  BY c.city
ORDER  BY realised DESC;
```

The portable equivalent, for MySQL, SQL Server and Oracle, which have no `FILTER` :

```
SELECT c.city,
       sum(CASE WHEN o.status = 'delivered' THEN 1 ELSE 0 END) AS delivered,
       sum(CASE WHEN o.status <> 'cancelled'
              THEN o.total_amount ELSE 0 END) AS realised
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id
GROUP  BY c.city;
```

COMMON MISTAKE

`count(CASE WHEN ... THEN 1 END)` works because `count` skips the `NULL` from the missing `ELSE` .
`sum(CASE WHEN ... THEN 1 END)` also works, but returns `NULL` rather than 0 when nothing matches — so always write `ELSE 0` with `sum` . Mixing the two conventions in one query is how a dashboard ends up with blank cells next to zeroes.

AGGREGATING TEXT AND ARRAYS

```
SELECT d.department_name,
       string_agg(e.full_name, ', ' ORDER BY e.salary DESC) AS team_by_seniority,
       array_agg(e.employee_id ORDER BY e.employee_id) AS ids
FROM   departments d
JOIN   employees e ON e.department_id = d.department_id
GROUP  BY d.department_id, d.department_name;
```

department_name	team_by_seniority	ids
Engineering	Asha Menon, Imran Khan, Rahul Iyer	{101,102,106}
Sales	Meera Nair, Vikram Rao	{103,104}
Support	Divya Pillai, Sneha Das	{105,107}
Finance	Arjun Shetty	{108}

The `ORDER BY` *inside* the aggregate is the detail worth knowing: without it, the order of concatenated values is undefined and will change. It is a common source of test failures that reproduce only on the build server.

8.7 GROUPING SETS, ROLLUP and CUBE

Several levels of aggregation in one pass, instead of a `UNION ALL` of three near-identical queries.

```
-- Subtotals per status, plus a grand total
SELECT coalesce(status, 'ALL STATUSES') AS status,
       count(*)                          AS orders,
       sum(total_amount)                  AS revenue
FROM   orders
GROUP  BY ROLLUP (status)
ORDER  BY grouping(status), status;
```

status	orders	revenue
cancelled	1	9200.00
delivered	3	31450.00
paid	1	21200.00
pending	1	3100.00
shipped	1	11000.00
ALL STATUSES	7	75950.00

Clause	Produces
<code>GROUP BY ROLLUP (a,b)</code>	Hierarchical subtotals: <code>(a,b)</code> , <code>(a)</code> , <code>()</code> — for drill-down reports
<code>GROUP BY CUBE (a,b)</code>	Every combination: <code>(a,b)</code> , <code>(a)</code> , <code>(b)</code> , <code>()</code> — for cross-tabs
<code>GROUP BY GROUPING SETS (...)</code>	Exactly the combinations you list
<code>grouping(col)</code>	1 if this row is a subtotal over that column, 0 otherwise

COMMON MISTAKE

A subtotal row has `NULL` in the columns it aggregates over — indistinguishable from a genuine `NULL` in the data. That is what `grouping(col)` is for: use it to label subtotals and to sort them to the bottom, never `coalesce` alone. `coalesce(status, 'ALL')` above is safe only because `orders.status` is `NOT NULL`.

DIALECT

All three are standard SQL and supported by PostgreSQL 9.5+, SQL Server and Oracle. MySQL supports only `WITH ROLLUP`, with different syntax and no `GROUPING SETS`.

Part 8 Review

Quick Check ● BEGINNER

- 1 Difference between `count(*)`, `count(col)` and `count(DISTINCT col)` ?
- 2 Does `avg` include `NULL` s in its denominator?
- 3 What does `sum` return over zero rows?
- 4 State the grouping rule.
- 5 `WHERE` vs `HAVING` , in terms of evaluation order.
- 6 Why does a `LEFT JOIN` with `count(*)` report 1 instead of 0?
- 7 What is the portable equivalent of `FILTER (WHERE ...)` ?
- 8 What does `grouping(col)` return and why do you need it?
- 9 Difference between `ROLLUP` and `CUBE` ?
- 10 Why put `ORDER BY` inside `string_agg` ?

Practice ● EASY

- 1 Count employees, employees with an email, and distinct departments, in one row.
- 2 Headcount and average salary per department, including departments with nobody.
- 3 Total revenue per order status, excluding cancelled orders.
- 4 Departments with more than one employee, using `HAVING` .
- 5 Departments whose average salary exceeds 60,000, counting only staff hired before 2023.
- 6 Per customer: order count, total spend, average order value, and date of last order.
- 7 Per product: units sold and revenue, from `order_items` . Include products never ordered.
- 8 Order counts by status in a single row using `FILTER` , then rewrite portably.
- 9 List each department's members as a comma-separated string, ordered by salary.
- 10 Revenue per status with a grand total row, using `ROLLUP` .
- 11 Average salary per department where `NULL` departments are grouped as 'Unassigned'.
- 12 Find the discrepancy: for each order, compare `total_amount` against the sum of its items and list the ones that disagree.

Challenge ● INTERMEDIATE

- 1 Build a one-row executive summary of the whole business: customers, active customers, orders, realised revenue, average order value, and delivery rate — with no division-by-zero risk and no `NULL` s in the output.

- 2 Per city, show total customers and the percentage of all customers they represent, in one query.
- 3 Produce a category × status cross-tab of revenue using `GROUPING SETS`, labelling every subtotal correctly.
- 4 A report shows each department's headcount as double the truth after a join was added. Explain the mechanism and give two fixes.

Interview Drill ● ADVANCED

- 1 Explain `WHERE` vs `HAVING` using evaluation order.
- 2 Give three ways `NULL` changes an aggregate's answer.
- 3 Why does the grouping rule exist? What is the database refusing to do?
- 4 How would you show detail rows and a group total in the same result?
- 5 When does a join inflate an aggregate, and how do you detect it?

Part 8 Solutions

Quick Check

- 1 Rows; non- `NULL` values in that column; distinct non- `NULL` values.
- 2 No — `NULL` s are excluded from both numerator and denominator.
- 3 `NULL` , not 0. Wrap in `coalesce` when a zero is the meaningful display.
- 4 Every select-list column must be in `GROUP BY` or inside an aggregate.
- 5 `WHERE` at step 3 filters rows before grouping; `HAVING` at step 5 filters groups after.
- 6 The left join emits one all- `NULL` padding row for an unmatched left row, and `count(*)` counts rows. Use `count(right.id)` .
- 7 `sum(CASE WHEN ... THEN 1 ELSE 0 END)` or `count(CASE WHEN ... THEN 1 END)` .
- 8 1 if the row is a subtotal over that column, else 0. You need it because subtotal rows carry `NULL` in the aggregated column, indistinguishable from real `NULL` s.
- 9 `ROLLUP` gives hierarchical subtotals; `CUBE` gives every combination of the grouping columns.
- 10 Without it the concatenation order is undefined and can change between runs.

Practice — selected

7 • Products never ordered.

```
SELECT p.product_name,  
       coalesce(sum(oi.quantity), 0)           AS units_sold,  
       coalesce(sum(oi.quantity * oi.unit_price), 0) AS revenue  
FROM   products p  
LEFT   JOIN order_items oi ON oi.product_id = p.product_id  
GROUP  BY p.product_id, p.product_name  
ORDER  BY revenue DESC, p.product_name;
```

Grouping by `p.product_id` as well as the name is deliberate: two products could share a name, and grouping by name alone would silently merge them. Group by the key, select the label.

11 • Unassigned as a group.

```
SELECT coalesce(d.department_name, 'Unassigned') AS department,  
       count(*)                               AS headcount,  
       round(avg(e.salary), 2)                 AS avg_salary  
FROM   employees e  
LEFT   JOIN departments d ON d.department_id = e.department_id  
GROUP  BY d.department_name  
ORDER  BY headcount DESC, department;
```

This works because `GROUP BY` gathers all `NULL` s into a single group — one of the two places `NULL` s are treated as equal (Volume I, §5.5 rule 5). Note the direction of the join has flipped from earlier examples: here we want all *employees* including the unassigned one, so `employees` is on the left.

12 · The discrepancy.

```
SELECT o.order_id,
       o.total_amount                AS recorded,
       sum(oi.quantity * oi.unit_price) AS line_sum,
       o.total_amount - sum(oi.quantity * oi.unit_price) AS difference
FROM   orders o
JOIN   order_items oi ON oi.order_id = o.order_id
GROUP BY o.order_id, o.total_amount
HAVING o.total_amount <> sum(oi.quantity * oi.unit_price)
ORDER BY o.order_id;
```

order_id	recorded	line_sum	difference
1003	11000.00	10950.00	50.00

Order 1003 was seeded deliberately inconsistent (Volume I, §2.9). This query is the template for every reconciliation task you will ever write: aggregate the detail, compare against the recorded summary, filter to the disagreements with `HAVING`. In a real system you would run it nightly and alert on a non-empty result.

Challenge — worked

1 · The executive summary.

```
SELECT count(DISTINCT c.customer_id) AS customers,
       count(DISTINCT o.customer_id) AS active_customers,
       count(DISTINCT o.order_id)    AS orders,
       coalesce(sum(o.total_amount) FILTER
                 (WHERE o.status <> 'cancelled'), 0)::numeric(12,2) AS realised_revenue,
       coalesce(round(avg(o.total_amount) FILTER
                     (WHERE o.status <> 'cancelled'), 2), 0) AS avg_order_value,
       coalesce(round(100.0 * count(*) FILTER (WHERE o.status = 'delivered')
                  / nullif(count(o.order_id), 0), 1), 0) AS delivered_pct
FROM   customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id;
```

customers	active_customers	orders	realised_revenue	avg_order_value	delivered_pct
5	4	7	66750.00	11125.00	42.9

Four defensive choices worth naming. `count(DISTINCT c.customer_id)` rather than `count(*)`, because the join multiplies customer rows by their order count. `count(DISTINCT o.customer_id)` gives active customers for free, and the `NULL` from Priya's unmatched row is skipped, so she is correctly excluded. `nullif` guards the percentage. And `coalesce` wraps everything, so an empty database returns zeroes rather than a row of blanks — which matters because a dashboard cannot distinguish “no data” from “broken query”.

2 · Percentage of total in one query.

```
SELECT city,
       count(*)                               AS customers,
       round(100.0 * count(*) / sum(count(*) OVER (), 1) AS pct_of_all
FROM   customers
GROUP BY city
ORDER BY customers DESC, city;
```

`sum(count(*) OVER ())` looks impossible and is not: the aggregate `count(*)` is computed per group first, then the window function sums those group counts across all groups. Aggregates run before window functions, so a window function may take an aggregate as its argument. This is the standard idiom for “percentage of total” and it avoids the subquery or self-join that beginners reach for. Part 13 explains the mechanism properly.

4 · The doubled headcount. The mechanism is **join fan-out**. Joining `departments` to `employees` to `order_items` — or to any table with several rows per employee — multiplies each employee row by the number of matches, and `count(*)` then counts the multiplied rows. Every aggregate downstream of a one-to-many join is suspect. Two fixes: **(a)** `count(DISTINCT e.employee_id)`, which is correct but masks the problem and is slow on large results; **(b)** aggregate the many-side *first* in a subquery or CTE so it presents one row per employee, then join — correct, fast, and self-documenting:

```
SELECT d.department_name, count(*) AS headcount, coalesce(sum(s.total_sales), 0) AS sales
FROM   departments d
JOIN   employees e ON e.department_id = d.department_id
LEFT   JOIN (SELECT employee_id, sum(amount) AS total_sales
            FROM   sales GROUP BY employee_id) s ON s.employee_id = e.employee_id
GROUP BY d.department_id, d.department_name;
```

How to detect it: run the query without the aggregate and count the raw rows. If that number exceeds the number of entities you think you are counting, you have fan-out. Part 9 treats this as its central topic.

Parts 1–8

Closed book, 90 minutes, database open. Twenty-five questions: ten conceptual, twelve query-writing, three diagnostic. Below 70% means re-read rather than continue — Part 9 assumes all of this.

Section A — Concepts ● 10 MARKS

- 1 Give the nine-step logical evaluation order of a `SELECT`, and use it to explain two things beginners find arbitrary.
- 2 Why is `NULL = NULL` not true, and name three consequences.
- 3 Explain sargability and give two predicates that are not sargable and their rewrites.
- 4 Why should money never be a `float`? Give the two acceptable alternatives.
- 5 What does `timestampz` store, and when would you deliberately use plain `timestamp`?
- 6 Explain why `OFFSET` pagination degrades, and what replaces it.
- 7 State the grouping rule and explain what the database is refusing to guess.
- 8 When does `count(*)` give the wrong answer, and what do you use instead?
- 9 Name four differences between `DELETE` and `TRUNCATE`.
- 10 Give three places PostgreSQL and MySQL disagree in ways that silently change results rather than erroring.

Section B — Query writing ● 12 MARKS

- 11 Every employee with their department name and salary, unassigned employees included, highest paid first.
- 12 Departments with average salary above 60,000, counting only employees hired before 2023, ordered by average descending.
- 13 Per customer: order count, total spend excluding cancelled orders, and average order value — customers with no orders included, showing zeroes not `NULL`s.
- 14 Monthly order counts and revenue for January to July 2024, with no missing months.
- 15 Products never ordered.
- 16 The second page of five orders sorted by date descending, using keyset pagination. State your cursor.
- 17 Each employee's name, their manager's name, and `'no manager'` where applicable.
- 18 Order counts by status in one row, with the delivered percentage, safe on an empty table.

- 19** Orders whose recorded total disagrees with the sum of their line items, with the difference.
- 20** Each department's members as a comma-separated list ordered by seniority, plus headcount.
- 21** Revenue per product category with a grand total row, correctly labelled.
- 22** Customers who have ordered, and the number of distinct products they have bought.

Section C — Diagnosis ● 3 MARKS

- 23** A report of "departments with no employees" returns nothing, though Research clearly has none. The query uses `NOT IN`. Diagnose and fix.
- 24** A monthly revenue chart shows a flat line for two months, then a jump. The SQL is a `GROUP BY date_trunc`. Nothing errors. What is wrong?
- 25** A dashboard tile reads `0%` for conversion rate although conversions clearly exist. One character fixes it. Which, and why?

Cumulative Test — Solutions

Section A

- 1 `FROM`, `JOIN`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `DISTINCT`, `ORDER BY`, `LIMIT`. It explains why a select-list alias is unavailable in `WHERE` but available in `ORDER BY`, and why `HAVING` must exist separately from `WHERE`.
- 2 `NULL` is unknown, and two unknowns cannot be shown equal. Consequences: `WHERE x = 1` and `WHERE x <> 1` together do not cover the table; `NOT IN` with a `NULL` returns nothing; aggregates silently exclude `NULL`s, changing averages.
- 3 A predicate an index can serve, requiring the column bare on one side. `created_at::date = d` → a half-open range; `upper(city) = 'X'` → compare the untransformed column, or index the expression.
- 4 Binary floats cannot represent decimal fractions exactly, so values drift and equality is unreliable. Use `numeric(p,s)`, or `bigint` in minor units.
- 5 An absolute instant, stored as UTC and rendered in the session zone. Plain `timestamp` only for a wall-clock intention with no location — “07:00 local wherever the user is”.
- 6 `OFFSET` computes and discards the skipped rows, so cost grows with page number; concurrent writes also shift offsets, skipping and duplicating rows. Keyset pagination with a unique compound sort key and a row-value comparison.
- 7 Every select-list column must be grouped or aggregated. The database refuses to pick an arbitrary row's value for a column that has many values within the group.
- 8 After a `LEFT JOIN`, where it counts the all-`NULL` padding row. Use `count(right_table.key)`. Also after a one-to-many join, where fan-out inflates it — use `count(DISTINCT ...)` or pre-aggregate.
- 9 DML vs DDL; filterable vs not; row triggers fire vs not; slow vs instant; rollback everywhere vs engine-dependent; identity preserved vs resettable.
- 10 `||` is concatenation in PostgreSQL and logical OR in MySQL; `10/3` truncates in PostgreSQL and returns a float in MySQL; `LIKE` is case-sensitive in PostgreSQL and usually not in MySQL; `NULL`s sort last vs first. Any three.

Section B — model answers

13 ·

```

SELECT c.customer_id,
       c.full_name,
       count(o.order_id) AS order_count,
       coalesce(sum(o.total_amount) FILTER
                 (WHERE o.status <> 'cancelled'), 0)::numeric(12,2) AS total_spend,
       coalesce(round(avg(o.total_amount) FILTER
                     (WHERE o.status <> 'cancelled'), 2), 0) AS avg_order_value
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id
GROUP  BY c.customer_id, c.full_name
ORDER  BY total_spend DESC, c.customer_id;

```

Marks for: `LEFT JOIN` keeping Priya; `count(o.order_id)` giving her 0; `FILTER` excluding cancelled orders from the money but not from the count; `coalesce` turning the `NULL` aggregates into zeroes; grouping by the key as well as the name.

17 ·

```

SELECT e.full_name AS employee,
       coalesce(m.full_name, 'no manager') AS manager
FROM   employees e
LEFT   JOIN employees m ON m.employee_id = e.manager_id
ORDER  BY e.employee_id;

```

A self-join with a `LEFT`, because Asha has no manager and an `INNER JOIN` would drop the CEO. Full marks require noticing that.

21 ·

```

SELECT CASE WHEN grouping(p.category) = 1 THEN 'ALL CATEGORIES'
           ELSE p.category END AS category,
       sum(oi.quantity * oi.unit_price)::numeric(12,2) AS revenue
FROM   order_items oi
JOIN   products p ON p.product_id = oi.product_id
GROUP  BY ROLLUP (p.category)
ORDER  BY grouping(p.category), revenue DESC;

```

Using `grouping()` rather than `coalesce` is the marked distinction: it labels the subtotal correctly even if a genuine `NULL` category existed, and it sorts the total to the bottom.

22 ·

```

SELECT c.customer_id, c.full_name,
       count(DISTINCT oi.product_id) AS distinct_products
FROM   customers c
JOIN   orders o      ON o.customer_id = c.customer_id
JOIN   order_items oi ON oi.order_id = o.order_id
GROUP BY c.customer_id, c.full_name
ORDER BY distinct_products DESC, c.customer_id;

```

DISTINCT inside the count is essential: a customer who bought the same product across three orders has bought one distinct product, and the fan-out of two joins would otherwise report three.

Section C

23 • **NOT IN** against a subquery containing a **NULL** — employee 109 has no department, so the value list includes **NULL**, every comparison becomes unknown, and no row can be true. Fix with **NOT EXISTS**, which asks a two-valued question. Adding **WHERE department_id IS NOT NULL** to the subquery also works but leaves the fragility in place for the next person.

24 • Nothing is wrong with the SQL and everything is wrong with the chart. **GROUP BY** cannot produce rows for months with no orders, so those months are absent from the result and the charting library connects March to June with a straight line — which reads as flat sales rather than no sales. Fix by generating the month series and left-joining the orders onto it, with **coalesce** on the aggregates.

25 • Integer division. **100 * conversions / visits** with integer columns truncates to zero whenever conversions are fewer than visits — which is always. The one-character fix is **100.0**, forcing numeric arithmetic. Add **nullif(visits, 0)** as well and the tile is also safe on a day with no traffic.

Where you are, and what comes next

You can now write, filter, sort, transform and aggregate against a single table, and you know the three things that most often make a correct-looking query wrong: `NULL` s in comparisons, `NULL` s in aggregates, and integer division. Before Part 9, confirm you can do all four of these without looking anything up:

1. Explain why `WHERE x = 1` and `WHERE x <> 1` do not partition a table.
2. Write a gap-free monthly report from memory, including why the status filter goes in `ON`.
3. Convert an `OFFSET` page to a keyset page, including the tie-break column.
4. State the grouping rule and the three ways to satisfy it.

NEXT

Volume III — Parts 9 to 14. Joins in full depth with 100 practice problems; subqueries and the `EXISTS / IN / JOIN` decision; CTEs including recursive traversal of the manager hierarchy; set operations; window functions as a major chapter — ranking, running totals, top-N per group, gaps and islands, sessionization; and the advanced toolkit of `FILTER`, `DISTINCT ON`, `LATERAL` and pivoting. It closes with a cumulative test over Parts 1–14.

Volume III is the hardest and most valuable part of the book — joins and window functions are where SQL interviews are won. Say the word and I will write it at this depth.



SQL MASTERY

Volume III — Combining & Analysing Data

Parts 9 to 14. Joins, subqueries, CTEs, set operations, window functions and the advanced toolkit. This is the volume that decides SQL interviews.

-
- 09** **Joins** — the concept before the syntax, all six kinds, fan-out, semi and anti joins
 - 10** **Subqueries** — scalar, correlated, `EXISTS`, and the `JOIN / IN / EXISTS` decision
 - 11** **Common Table Expressions** — `WITH`, chaining, recursion, hierarchies, cycles
 - 12** **Set Operations** — `UNION`, `INTERSECT`, `EXCEPT`
 - 13** **Window Functions** — frames, ranking, running totals, top-N per group, gaps and islands
 - 14** **Advanced SQL** — `FILTER`, `DISTINCT ON`, `LATERAL`, pivoting
-

Practice database: Volume I, §2.9. Reset it before starting — several examples depend on the deliberate quirks in the seed data.

Joins

Joins are where SQL becomes powerful and where most people's mental model quietly breaks. This part teaches the mechanism first — what a join *does* to rows, before any keyword — because once you can predict the row count, every join type is a one-line variation.

IN THIS PART

9.1 What a join actually does

9.2 INNER JOIN

9.3 LEFT and RIGHT JOIN

9.4 FULL OUTER JOIN

9.5 CROSS JOIN and NATURAL JOIN

9.6 Self joins

9.7 Joining three or more tables

9.8 Fan-out — the duplication trap

9.9 Semi joins and anti joins

9.10 ON vs WHERE · Review

9.1 What a join actually does

Forget the Venn diagrams. They are the standard teaching aid and they are actively misleading, because they describe *sets of keys* while a join operates on *rows* — and the number of output rows is exactly what people get wrong.

Here is the real mechanism, and it is only three steps:

1. **Form every possible pairing** of a row from the left table with a row from the right table. Three left rows and four right rows gives twelve candidate pairs — the Cartesian product.
2. **Keep the pairs where the `ON` condition is true.** Everything else is discarded.
3. **For an outer join, add back** the rows from the preserved side that found no partner, padded with `NULL` s in the other side's columns.

That is the entire model. Every join type differs only in step 3.

FIGURE 9.1 — THE THREE STEPS ON TINY TABLES

L

id	name
1	Asha
2	Ravi
3	Sara

R

lid	item
1	pen
1	book
2	desk

Step 1 – 9 candidate pairs (3 × 3)

Step 2 – ON L.id = R.lid keeps 3: (1,pen) (1,book) (2,desk)

Step 3 – INNER: stop here. → 3 rows

LEFT: add Sara, padded with NULLs. → 4 rows

Notice that Asha appears **twice** in the inner join. She did not become two people; she matched two rows. This is fan-out (§9.8), and it is the single most consequential fact about joins.

BEST PRACTICE · PREDICT BEFORE YOU RUN

Before running any join, say out loud how many rows you expect and why. “Seven orders, each with exactly one customer, so seven rows.” When the answer differs, you have learned something about your data — usually that a relationship you believed was one-to-one is not. Engineers who do this catch fan-out bugs in seconds; engineers who don't ship them.

9.2 INNER JOIN

Keeps only pairs that matched. Rows on either side with no partner disappear — which is both its purpose and its danger.

```
SELECT e.full_name,  
       d.department_name,  
       e.salary  
FROM   employees AS e  
INNER JOIN departments AS d ON d.department_id = e.department_id  
ORDER BY d.department_name, e.salary DESC;
```

full_name	department_name	salary
Arjun Shetty	Finance	76000.00
Asha Menon	Engineering	92000.00
Imran Khan	Engineering	88000.00
Rahul Iyer	Engineering	64000.00
Meera Nair	Sales	71500.00
Vikram Rao	Sales	58000.00
Divya Pillai	Support	52000.00
Sneha Das	Support	47000.00

(8 rows)

Eight rows, but there are nine employees. Nisha Verma is gone, because her `department_id` is `NULL` and `NULL = anything` is never true. Research is gone too, because no employee points at it. Neither omission produced an error, a warning, or anything else a careless reader would notice — the result simply looks like a complete staff list and is not.

COMMON MISTAKE · THE SILENT SHRINK

An `INNER JOIN` is also a filter. Every time you add one to a working query, you may be removing rows. Check the count: if `SELECT count(*) FROM employees` is 9 and your joined query returns 8, that difference is either intentional or a bug, and you must know which. Make this a reflex — count before, count after.

THE WORD INNER IS OPTIONAL

`JOIN` alone means `INNER JOIN`. This book writes the short form for inner joins and always spells out `LEFT JOIN`, which is the common convention: the default needs no announcement, the deviation does.

9.3 LEFT JOIN and RIGHT JOIN

A **LEFT JOIN** keeps every row of the left table whether or not it matched, padding unmatched rows with **NULL**s from the right.

```
SELECT e.full_name,  
       coalesce(d.department_name, '- unassigned -') AS department  
FROM   employees AS e  
LEFT   JOIN departments AS d ON d.department_id = e.department_id  
ORDER  BY e.employee_id;
```

full_name	department
Asha Menon	Engineering
Rahul Iyer	Engineering
Meera Nair	Sales
Vikram Rao	Sales
Sneha Das	Support
Imran Khan	Engineering
Divya Pillai	Support
Arjun Shetty	Finance
Nisha Verma	- unassigned -

(9 rows)

Nine rows — every employee. Reverse the tables to see the other side of the data:

```
SELECT d.department_name,  
       count(e.employee_id) AS headcount  
FROM   departments AS d  
LEFT   JOIN employees AS e ON e.department_id = d.department_id  
GROUP  BY d.department_id, d.department_name  
ORDER  BY headcount DESC, d.department_name;
```

department_name	headcount
Engineering	3
Sales	2
Support	2
Finance	1
Research	0

Research now appears with zero — the answer an **INNER JOIN** cannot give, and precisely the row a “which departments need hiring?” report exists to surface. Note **count(e.employee_id)** rather than **count(*)**: the padding row is still a row, and **count(*)** would report 1 (Volume II, §8.2).

RIGHT JOIN

`A RIGHT JOIN B` is exactly `B LEFT JOIN A`. It exists for completeness and is rarely used, because reading a query is easier when the preserved table is the one you named first. Convention: **write every outer join as `LEFT`** and reorder the tables instead. The exception is when you are adding a table to the end of a long existing join chain and rewriting the whole thing would be riskier than one `RIGHT JOIN` — a real situation, and the only one that justifies it.

INTERVIEW · BEGINNER

Q. Difference between `INNER JOIN` and `LEFT JOIN` ? — Inner keeps only matched pairs; left keeps every row of the left table, padding unmatched ones with `NULL`. Then give the consequence, which is what they are actually testing: “so an inner join silently drops rows, and if I’m counting or reporting on the left table I almost always want a left join — otherwise entities with no related rows vanish from the report.”

9.4 FULL OUTER, 9.5 CROSS and NATURAL

FULL OUTER JOIN keeps unmatched rows from **both** sides. It is uncommon in application code and genuinely useful for one thing: reconciling two datasets that should agree.

```
SELECT coalesce(a.order_id, b.order_id) AS order_id,  
       a.total_amount                AS in_system_a,  
       b.total_amount                AS in_system_b,  
       CASE WHEN a.order_id IS NULL THEN 'missing from A'  
            WHEN b.order_id IS NULL THEN 'missing from B'  
            ELSE 'amount differs' END AS problem  
FROM   orders_system_a a  
FULL   OUTER JOIN orders_system_b b ON b.order_id = a.order_id  
WHERE  a.order_id IS NULL  
       OR b.order_id IS NULL  
       OR a.total_amount IS DISTINCT FROM b.total_amount;
```

One query, three classes of discrepancy: present only in A, present only in B, present in both but disagreeing. That is the entire job of a reconciliation report.

DIALECT

MySQL has no **FULL OUTER JOIN** (as of 8.x). Emulate it with **LEFT JOIN ... UNION ... RIGHT JOIN ...**. PostgreSQL, SQL Server, Oracle and SQLite 3.39+ all support it.

CROSS JOIN

Every pairing, no condition — step 1 of §9.1 with no step 2. Left rows × right rows. Deliberate uses are narrow but real: generating a calendar × products grid so a report has a row for every combination, or building test data.

```
-- Every product on every day of March: the skeleton of a gap-free sales matrix  
SELECT d.day::date, p.product_name  
FROM   generate_series(DATE '2024-03-01', DATE '2024-03-31', INTERVAL '1 day') AS d(day)  
CROSS JOIN products p;
```

COMMON MISTAKE · THE ACCIDENTAL CROSS JOIN

An old-style comma join with a forgotten condition — **FROM orders o, customers c** with no **WHERE o.customer_id = c.customer_id** — produces a full Cartesian product. On 10,000 orders and 5,000 customers that is 50 million rows, and the query does not error; it simply consumes the server. This is why this book insists on explicit **JOIN ... ON**: a missing **ON** is a syntax error you find in one second, while a missing **WHERE** is an incident.

NATURAL JOIN

Joins automatically on *every* column with the same name in both tables. Never use it. It reads as a convenience and behaves as a time bomb: the day someone adds a `created_at` column to both tables, the join condition silently gains a term and your results change. Understand it so you can recognise it in inherited code, then replace it with an explicit `ON`.

`USING (column)` is the safe middle ground — it joins on a named column and merges it into one output column: `FROM employees JOIN departments USING (department_id)`. Explicit about which column, and slightly shorter than `ON`.

9.6 Self joins

A table joined to itself, with two aliases. The mechanism is unchanged — the aliases simply let SQL treat one table as two. It is how you traverse a hierarchy stored as a self-referencing foreign key.

```
SELECT e.full_name           AS employee,
       coalesce(m.full_name, '- none -') AS manager,
       e.salary,
       m.salary              AS manager_salary
FROM   employees AS e
LEFT   JOIN employees AS m ON m.employee_id = e.manager_id
ORDER  BY e.employee_id;
```

employee	manager	salary	manager_salary
Asha Menon	- none -	92000.00	
Rahul Iyer	Asha Menon	64000.00	92000.00
Meera Nair	Asha Menon	71500.00	92000.00
Vikram Rao	Meera Nair	58000.00	71500.00
Sneha Das	Meera Nair	47000.00	71500.00
Imran Khan	Asha Menon	88000.00	92000.00
Divya Pillai	Sneha Das	52000.00	47000.00
Arjun Shetty	Asha Menon	76000.00	92000.00
Nisha Verma	Arjun Shetty	45000.00	76000.00

LEFT is essential: Asha has no manager, and an inner join would drop the CEO from an org chart — a bug that survives review because the output still looks plausible.

Look at Divya Pillai: she earns more than her manager. That is a classic interview question, and it is now a one-line filter:

```
SELECT e.full_name, e.salary, m.full_name AS manager, m.salary AS manager_salary
FROM   employees e
JOIN   employees m ON m.employee_id = e.manager_id
WHERE  e.salary > m.salary;
```

Here **INNER** is correct — an employee with no manager cannot out-earn one. Choosing the join type by reasoning about the question, not by habit, is the skill.

NOTE · SELF JOINS REACH ONE LEVEL

A self join gives you an employee and their manager. It cannot give you the whole chain to the top, because you would need one join per level and you do not know the depth. That requires recursion — Part 11.

9.7 Joining three or more tables

Joins chain left to right: the result of the first join becomes the left input of the second. Read a long chain as a pipeline, not as a set of independent statements.

```
SELECT c.full_name      AS customer,
       o.order_id,
       o.order_date,
       p.product_name,
       oi.quantity,
       oi.unit_price,
       (oi.quantity * oi.unit_price) AS line_total
FROM   customers AS c
JOIN   orders     AS o  ON o.customer_id = c.customer_id
JOIN   order_items AS oi ON oi.order_id  = o.order_id
JOIN   products   AS p  ON p.product_id = oi.product_id
WHERE  o.status <> 'cancelled'
ORDER BY o.order_id, p.product_name;
```

customer	order_id	order_date	product_name	qty	unit_price	line_total
Kavya Reddy	1001	2024-01-05	27-inch Monitor	1	18900.00	18900.00
Kavya Reddy	1001	2024-01-05	Mechanical Keyboard	1	5400.00	5400.00
Rohan Gupta	1002	2024-01-19	Mechanical Keyboard	1	5400.00	5400.00
Kavya Reddy	1003	2024-02-11	Laptop Stand	1	1750.00	1750.00
...						
(10 rows)						

BEST PRACTICE · THE OUTER-JOIN CONTAMINATION RULE

Once you use a **LEFT JOIN**, every subsequent join to that branch must also be **LEFT**. An inner join further down the chain will discard the **NULL**-padded rows the left join just preserved, converting the whole chain back to inner behaviour. This is the most common way an outer join is silently defeated:

```
-- BROKEN: the inner join to products removes customers with no orders
FROM   customers c
LEFT   JOIN orders o      ON o.customer_id = c.customer_id
JOIN   order_items oi ON oi.order_id = o.order_id  -- ← kills it

-- CORRECT
LEFT   JOIN order_items oi ON oi.order_id = o.order_id
```


9.8 Fan-out — the duplication trap

If you learn one thing from this part, learn this. **Joining to the many side of a one-to-many relationship multiplies rows**, and every aggregate computed afterwards is inflated by that multiplication.

```
-- WRONG: order 1005 has three items, so its total_amount is counted three times
SELECT c.full_name, sum(o.total_amount) AS spend
FROM   customers c
JOIN   orders o      ON o.customer_id = c.customer_id
JOIN   order_items oi ON oi.order_id   = o.order_id
GROUP BY c.customer_id, c.full_name;
```

full_name		spend	
-----+-----			
Kavya Reddy		61550.00	← should be 37050.00
Rohan Gupta		69000.00	← should be 26600.00
...			

The mechanism: joining `orders` to `order_items` turns one order row into one row per item. Order 1005 has three items, so `21200.00` appears three times and `sum` dutifully adds it three times. Nothing is broken; the query is asking a different question from the one you meant.

THREE FIXES, IN ORDER OF QUALITY

```
-- (1) Don't join what you don't need. The order total is on orders.
SELECT c.full_name, sum(o.total_amount) AS spend
FROM   customers c
JOIN   orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.full_name;

-- (2) Aggregate the many side FIRST, then join one row to one row.
SELECT c.full_name,
       coalesce(sum(i.line_total), 0) AS spend
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id
LEFT   JOIN (SELECT order_id, sum(quantity * unit_price) AS line_total
             FROM   order_items GROUP BY order_id) AS i
       ON i.order_id = o.order_id
GROUP BY c.customer_id, c.full_name;

-- (3) Deduplicate the value being summed. Correct, but obscure — avoid.
SELECT c.full_name, sum(DISTINCT o.total_amount) AS spend -- breaks if two orders tie!
FROM   ...
```

COMMON MISTAKE

Fix (3) is what people reach for, and it is a trap: `sum(DISTINCT ...)` collapses two genuinely different orders that happen to have the same amount. `count(DISTINCT id)` is safe because ids are unique; `sum(DISTINCT amount)` is not, because amounts are not. Pre-aggregation (fix 2) is the professional answer.

BEST PRACTICE · HOW TO DETECT FAN-OUT

Remove the `GROUP BY` and count the raw rows. If a query you believe returns one row per customer returns more, you have fan-out. A faster habit: after any join, ask “can the right side have more than one match per left row?” If yes, no aggregate on the left side's columns is trustworthy.

INTERVIEW · ADVANCED

Q. A revenue report is showing roughly triple the real figure. Where do you look first? — Join fan-out. Find the one-to-many join in the chain and check whether an aggregate is summing a column from the *one* side. Fix by pre-aggregating the many side into a subquery or CTE that yields one row per key, then joining. **Interview tip:** mention that you diagnose it by dropping the aggregate and counting rows — a concrete diagnostic method is what distinguishes someone who has actually debugged this from someone who has read about it.

9.9 Semi joins and anti joins

Two patterns with no dedicated keyword in SQL, which is why they are worth naming. Both answer existence questions, and crucially **neither multiplies rows**.

Pattern	Question	Written as
Semi join	"Left rows that have at least one match" — return left columns only, once each	<code>WHERE EXISTS (...)</code>
Anti join	"Left rows that have no match"	<code>WHERE NOT EXISTS (...)</code>

```
-- Semi join: customers who have ordered. One row each, regardless of order count.
```

```
SELECT c.full_name
FROM   customers c
WHERE  EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

```
-- Anti join: customers who have never ordered.
```

```
SELECT c.full_name
FROM   customers c
WHERE  NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

```
full_name
```

```
-----
```

```
Priya Anand
```

The alternative anti-join spelling, using an outer join and testing for the padding:

```
SELECT c.full_name
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id
WHERE  o.order_id IS NULL;  -- the tell-tale "left join / is null" idiom
```

Both are correct and both are common. `NOT EXISTS` reads closer to the intent and is immune to the `NULL` problem that destroys `NOT IN` (Volume II, §5.5). Prefer it, and recognise the `LEFT JOIN ... IS NULL` form when you meet it.

BEST PRACTICE

Use a semi join whenever you want to *filter* by the existence of related rows but do not need any column from them. An `INNER JOIN` would work but risks fan-out — filtering customers by “has an order” with a join gives one row per order, and the bug only appears when someone adds a `count`.

9.10 ON versus WHERE

For an **INNER JOIN**, a condition in **ON** and the same condition in **WHERE** give identical results. **For an outer join, they are completely different**, and this is one of the highest-value distinctions in SQL.

```
-- (A) Filter in ON: applied while matching. Unmatched left rows are still preserved.
SELECT d.department_name, count(e.employee_id) AS senior_staff
FROM   departments d
LEFT   JOIN employees e
        ON e.department_id = d.department_id
        AND e.salary > 70000          -- ← in ON
GROUP BY d.department_id, d.department_name;

-- (B) Filter in WHERE: applied after the join. NULL-padded rows fail it and vanish.
SELECT d.department_name, count(e.employee_id) AS senior_staff
FROM   departments d
LEFT   JOIN employees e ON e.department_id = d.department_id
WHERE  e.salary > 70000          -- ← in WHERE
GROUP BY d.department_id, d.department_name;
```

(A) — five rows, zeroes preserved

department_name	senior_staff
Engineering	2
Sales	1
Finance	1
Support	0
Research	0

(B) — three rows, the zeroes are gone

department_name	senior_staff
Engineering	2
Sales	1
Finance	1

Version B has silently become an inner join. The **WHERE** clause runs after the join has produced its **NULL**-padded rows for Support and Research, and **NULL > 70000** is unknown, so those rows are discarded. If the report was meant to show every department, B is wrong — and it is wrong in the most dangerous way, by omitting rows rather than by miscounting visible ones.

THE RULE

In an outer join, a condition on the **optional** table belongs in **ON**. A condition on the **preserved** table belongs in **WHERE**. Put it in the wrong place and you either destroy the outer join or fail to filter.

INTERVIEW · ADVANCED

Q. What is the difference between putting a condition in `ON` versus `WHERE` ? — For inner joins, none. For outer joins, `ON` filters during matching so unmatched rows are still preserved and padded; `WHERE` filters after, and because the padded columns are `NULL`, any condition on them removes those rows and turns the outer join into an inner one. **Interview tip:** offer the concrete case — “that's why a gap-filled report puts the status filter in `ON` ” — because the abstract statement alone doesn't demonstrate you have hit it in practice.

Part 9 Review

Forty problems here; the remaining sixty of the promised hundred are distributed through the pattern library in Part 35 and the challenge bank in Part 34, where they can build on window functions too.

Quick Check ● BEGINNER

- 1 State the three steps a join performs.
- 2 Why can an `INNER JOIN` return fewer rows than the left table has?
- 3 Rewrite `A RIGHT JOIN B` as a left join.
- 4 What is fan-out, and what does it do to a `sum`?
- 5 Why should you never use `NATURAL JOIN`?
- 6 Give the two ways to write an anti join.
- 7 Why must a self join on `manager_id` be a `LEFT JOIN` for an org chart?
- 8 What happens to a `LEFT JOIN` if a later join in the chain is inner?
- 9 Where does a filter on the optional table of an outer join belong?
- 10 Which engine lacks `FULL OUTER JOIN`?

Practice ● EASY

- 1 Every employee with their department name — all nine.
- 2 Every department with its headcount — all five.
- 3 Every order with its customer's name and city.
- 4 Every order line with the product name and computed line total.
- 5 Customers who have never ordered, both ways (`NOT EXISTS` and `LEFT JOIN ... IS NULL`).
- 6 Products never ordered.
- 7 Each employee with their manager's name, CEO included.
- 8 Employees who earn more than their manager.
- 9 Managers with the number of direct reports, including employees who manage nobody.
- 10 Customer, order and product in one four-table join, cancelled orders excluded.
- 11 Total spend per customer — correct, with no fan-out. Then write the broken version and show the difference.
- 12 Per department, the count of employees earning over 70,000, showing zero for departments with none.
- 13 Departments with no employees, three different ways.

- 14 Every product paired with every category name (a deliberate cross join) and explain the row count.
- 15 Customers who have bought the Mechanical Keyboard.
- 16 Customers who have bought both a keyboard and a monitor.
- 17 Orders containing more than one distinct product.
- 18 For each employee, their department name and their department's average salary.
- 19 Pairs of employees in the same department (each pair once, not twice).
- 20 Every customer with their order count and total spend, non-orderers showing zeroes.

Challenge ● INTERMEDIATE

- 1 A report of “revenue per customer” joins customers → orders → order_items → products and reports figures roughly 1.6× too high. Explain precisely why, and give the corrected query.
- 2 Write a query listing every department and, for each, the count of employees hired in 2024 — every department must appear, including those with zero 2024 hires and those with no employees at all. Explain each clause placement.
- 3 Build a reconciliation between `orders.total_amount` and the summed line items, reporting orders present in one but not the other, and orders present in both but disagreeing.
- 4 Find every pair of products that have appeared together in the same order, with a count of how often, each pair listed once.
- 5 Explain, with a worked example on the practice data, why `count(*)` and `count(DISTINCT o.order_id)` differ in a customers→orders→items join.

Interview Drill ● ADVANCED

- 1 Explain joins without using the word Venn.
- 2 `ON` vs `WHERE` in an outer join, with an example.
- 3 How do you diagnose an inflated aggregate?
- 4 When would you use a semi join rather than an inner join?
- 5 Why is `NOT EXISTS` preferred to `NOT IN`?

Part 9 Solutions

Quick Check

- 1 Form all pairings; keep those satisfying `ON`; for outer joins, add back unmatched rows from the preserved side padded with `NULL`.
- 2 Left rows with no matching right row are discarded — including rows whose join column is `NULL`, which can never match.
- 3 `B LEFT JOIN A` with the same `ON`.
- 4 One row on the left matching many on the right is duplicated. A `sum` of a left-side column then adds that value once per match.
- 5 It joins on every same-named column, so adding a column to either table silently changes the join condition.
- 6 `WHERE NOT EXISTS (...)`, or `LEFT JOIN ... WHERE right.key IS NULL`.
- 7 Because the top of the hierarchy has a `NULL manager_id` and an inner join would drop them.
- 8 The inner join discards the `NULL`-padded rows, converting the whole chain to inner behaviour.
- 9 In `ON`. A filter on the preserved table goes in `WHERE`.
- 10 MySQL.

Practice — selected

9 · Managers and direct reports.

```
SELECT m.full_name          AS manager,
       count(e.employee_id) AS direct_reports
FROM   employees m
LEFT   JOIN employees e ON e.manager_id = m.employee_id
GROUP  BY m.employee_id, m.full_name
ORDER  BY direct_reports DESC, m.full_name;
```

manager	direct_reports
Asha Menon	4
Meera Nair	2
Arjun Shetty	1
Sneha Das	1
Divya Pillai	0
Imran Khan	0
...	

Everyone appears, because the question said “including employees who manage nobody”. The join direction is the interesting part: `e.manager_id = m.employee_id` with `m` preserved. Swap the aliases and you get each employee's manager instead — same table, same syntax, opposite question.

16 · Customers who bought both. The tempting `WHERE product_name = 'Mechanical Keyboard' AND product_name = '27-inch Monitor'` returns nothing — no single row can be both. Three correct approaches; the third is the one to remember:

```
-- (a) Two EXISTS clauses – clearest for exactly two items
SELECT c.full_name FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
              JOIN products p ON p.product_id = oi.product_id
              WHERE o.customer_id = c.customer_id AND p.product_name =
'Mechanical Keyboard')
AND EXISTS (SELECT 1 FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
              JOIN products p ON p.product_id = oi.product_id
              WHERE o.customer_id = c.customer_id AND p.product_name = '27-inch Monitor');

-- (b) Group and count distinct matches – scales to any number of items
SELECT c.customer_id, c.full_name
FROM   customers c
JOIN   orders o      ON o.customer_id = c.customer_id
JOIN   order_items oi ON oi.order_id   = o.order_id
JOIN   products p    ON p.product_id   = oi.product_id
WHERE  p.product_name IN ('Mechanical Keyboard', '27-inch Monitor')
GROUP BY c.customer_id, c.full_name
HAVING count(DISTINCT p.product_name) = 2;
```

Approach (b) is the general pattern — “relational division” — and the `DISTINCT` inside `count` is what makes it correct: a customer who bought two keyboards and no monitor would otherwise reach a count of 2 and pass. Change the list and the number and it answers “bought all five of these”.

19 · Pairs in the same department.

```
SELECT a.full_name AS employee_a, b.full_name AS employee_b, a.department_id
FROM   employees a
JOIN   employees b ON b.department_id = a.department_id
              AND b.employee_id > a.employee_id      -- the key line
ORDER BY a.department_id, a.employee_id, b.employee_id;
```

`b.employee_id > a.employee_id` does two jobs at once: it stops a row pairing with itself, and it prevents each pair appearing twice in both orders. Using `<>` instead gives you every pair twice — the classic wrong answer.

Challenge — worked

1 · The 1.6× inflation. The chain `customers → orders → order_items` multiplies each order row by its number of items. Across the practice data there are 7 orders and 11 items, so order-level values are counted $11/7 \approx 1.57$ times on average — matching the reported 1.6×. The ratio being non-integral is itself the diagnostic clue: a uniform 2× or 3× suggests a duplicate join key, while a fractional multiplier means variable fan-out. Correct version:

```

SELECT c.customer_id, c.full_name,
       coalesce(sum(o.total_amount) FILTER (WHERE o.status <> 'cancelled'), 0) AS revenue
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id
GROUP  BY c.customer_id, c.full_name
ORDER  BY revenue DESC;

```

The items and products tables were never needed — the total is already on `orders`. Half of all fan-out bugs are fixed by deleting a join rather than by compensating for it.

2 · Every department, 2024 hires.

```

SELECT d.department_name,
       count(e.employee_id) AS hires_2024
FROM   departments d
LEFT   JOIN employees e
       ON e.department_id = d.department_id
       AND e.hire_date >= DATE '2024-01-01'
       AND e.hire_date <  DATE '2025-01-01'
GROUP  BY d.department_id, d.department_name
ORDER  BY hires_2024 DESC, d.department_name;

```

Three placements, each deliberate. The date filter is in `ON`, so departments whose only employees were hired earlier still appear with zero rather than vanishing. The range is half-open, so a hire at any time on 31 December 2024 is included. And `count(e.employee_id)`, not `count(*)`, so an unmatched department reports 0 rather than 1. Move the date filter to `WHERE` and all three departments with no 2024 hires disappear — the same silent failure as §9.10.

4 · Product pairs bought together.

```

SELECT p1.product_name AS product_a,
       p2.product_name AS product_b,
       count(*)         AS times_together
FROM   order_items a
JOIN   order_items b ON b.order_id  = a.order_id
                AND b.product_id > a.product_id      -- each pair once, no self-pair
JOIN   products p1 ON p1.product_id = a.product_id
JOIN   products p2 ON p2.product_id = b.product_id
GROUP  BY p1.product_name, p2.product_name
ORDER  BY times_together DESC, product_a, product_b;

```

product_a	product_b	times_together
Mechanical Keyboard	27-inch Monitor	1
Laptop Stand	27-inch Monitor	1
Laptop Stand	USB-C Hub	1
...		

This is market-basket analysis in six lines, and it is the same `>` trick as practice 19 applied to a junction table. In production you would add a `HAVING count(*) >= n` threshold and join to order dates for a time window — but the shape does not change.

5 • `count(*)` versus `count(DISTINCT o.order_id)` . On Kavya Reddy: she has three orders (1001, 1003, 1007) containing 2, 2 and 1 items respectively. The three-table join produces 5 rows for her. `count(*)` reports 5 — the number of *line items* she has bought. `count(DISTINCT o.order_id)` reports 3 — her order count. Both are correct answers to different questions, and neither is “the number of rows in the join”, which is a number with no business meaning at all. Whenever a join chain crosses a one-to-many boundary, every count in the query needs an explicit decision about which entity it is counting.

Subqueries

A query inside a query. Three positions, two kinds, and one decision that comes up in every code review: join, `IN`, or `EXISTS`? §10.7 answers it properly.

IN THIS PART

10.1 Scalar subqueries

10.2 Subqueries in WHERE

10.3 Subqueries in FROM

10.4 Subqueries in SELECT

10.5 Correlated subqueries

10.6 EXISTS and NOT EXISTS

10.7 JOIN vs IN vs EXISTS

Review · Solutions

10.1 Scalar subqueries, 10.2 in WHERE

A **scalar subquery** returns exactly one row and one column, so it can be used anywhere a single value can.

```
-- Employees earning above the company average
SELECT full_name, salary
FROM   employees
WHERE  salary > (SELECT avg(salary) FROM employees)
ORDER BY salary DESC;
```

full_name	salary
Asha Menon	92000.00
Imran Khan	88000.00
Arjun Shetty	76000.00
Meera Nair	71500.00

This is the answer to a question a single-pass query cannot express: you need the aggregate over the whole table *before* you can compare each row against it. The subquery runs once, produces 65944.44, and the outer query filters against that constant.

COMMON MISTAKE

A scalar subquery that returns more than one row raises `more than one row returned by a subquery used as an expression`. If it returns *zero* rows it yields `NULL` instead — and `salary > NULL` is unknown, so the outer query silently returns nothing. The empty case fails quietly; the multi-row case fails loudly. Guard the empty case when the subquery is filtered.

MULTI-ROW SUBQUERIES IN WHERE

```
-- IN: match any of the returned values
SELECT full_name FROM employees
WHERE department_id IN (SELECT department_id FROM departments WHERE location
= 'Bengaluru');
```



```
-- ANY / ALL: compare against every returned value
SELECT full_name, salary FROM employees
WHERE salary > ALL (SELECT salary FROM employees WHERE department_id = 3);  -- above ALL
Support staff
SELECT full_name, salary FROM employees
WHERE salary > ANY (SELECT salary FROM employees WHERE department_id = 3);  -- above at
least one
```

`= ANY` is identical to `IN`; `<> ALL` is identical to `NOT IN`. The `ANY / ALL` forms earn their place with inequality operators, where `IN` cannot express the comparison at all.

COMMON MISTAKE · NOT IN, AGAIN

`NOT IN` against a subquery that can produce a `NULL` returns zero rows (Volume II, §5.5). It is worth repeating here because the subquery form is where it actually bites — with a literal list you can see there is no `NULL` ; with a subquery you cannot. **Default to `NOT EXISTS` .**

10.3 Subqueries in FROM, 10.4 in SELECT

A subquery in `FROM` — a **derived table** — behaves exactly like a table. It must be aliased.

```
-- Departments whose average salary beats the company average.
SELECT d.department_name, s.avg_salary
FROM (SELECT department_id, round(avg(salary), 2) AS avg_salary
      FROM employees
      WHERE department_id IS NOT NULL
      GROUP BY department_id) AS s
JOIN departments d ON d.department_id = s.department_id
WHERE s.avg_salary > (SELECT avg(salary) FROM employees)
ORDER BY s.avg_salary DESC;
```

department_name	avg_salary
Engineering	81333.33
Finance	76000.00

Two aggregations at different grain in one query: per-department in the derived table, company-wide in the scalar subquery. This is the standard shape for “above average” questions, and Part 11 will rewrite it as a CTE that reads better.

SUBQUERIES IN SELECT

```
SELECT c.full_name,
       (SELECT count(*) FROM orders o WHERE o.customer_id = c.customer_id)
AS order_count,
       (SELECT max(o.order_date) FROM orders o WHERE o.customer_id = c.customer_id)
AS last_order
FROM customers c
ORDER BY order_count DESC;
```

This works and reads well, and it has a real cost: each subquery is *correlated*, potentially executing once per outer row. Two subqueries over five customers is fine; two subqueries over five million is two extra index lookups per row. PostgreSQL's planner often rewrites simple cases into joins, but do not rely on it — the equivalent `LEFT JOIN` with `GROUP BY` scans each table once:

```
SELECT c.full_name, count(o.order_id) AS order_count, max(o.order_date) AS last_order
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.full_name
ORDER BY order_count DESC;
```

BEST PRACTICE

Use a select-list subquery when you need **one** derived value and clarity matters more than microseconds. Switch to a join with `GROUP BY` the moment you need two or more, or the table is large. The join computes all the aggregates in a single pass; N subqueries make N passes.

10.5 Correlated subqueries

A subquery that references a column from the outer query is **correlated**. It cannot be evaluated once; conceptually it re-runs for each outer row.

```
-- Employees earning above their OWN department's average
SELECT e.full_name, e.department_id, e.salary
FROM   employees e
WHERE  e.salary > (SELECT avg(x.salary)
                  FROM   employees x
                  WHERE  x.department_id = e.department_id)  -- ← e, from outside
ORDER BY e.department_id, e.salary DESC;
```

full_name	department_id	salary
Asha Menon	1	92000.00
Imran Khan	1	88000.00
Meera Nair	2	71500.00
Divya Pillai	3	52000.00

Note the alias discipline: the inner table needs its own alias (`x`) so `x.department_id = e.department_id` is unambiguous. Forget it and the condition becomes self-referential, always true, and you silently get the company-wide average instead of the departmental one — a wrong answer with no error.

BEST PRACTICE

The window-function form of this query is both clearer and faster, because it needs one pass instead of one subquery execution per row:

```
SELECT full_name, department_id, salary
FROM   (SELECT full_name, department_id, salary,
              avg(salary) OVER (PARTITION BY department_id) AS dept_avg
        FROM   employees) t
WHERE  salary > dept_avg;
```

Part 13. Recognise the correlated form — you will meet it constantly in existing code — and reach for windows when writing new code.

10.6 EXISTS and NOT EXISTS

EXISTS asks a yes/no question: did the subquery produce at least one row? It returns a boolean, never data, and it **stops at the first match** — so the subquery's select list is irrelevant. The convention is **SELECT 1**.

```
-- Customers who have at least one delivered order
SELECT c.full_name
FROM   customers c
WHERE  EXISTS (SELECT 1
               FROM   orders o
               WHERE  o.customer_id = c.customer_id
               AND    o.status = 'delivered');

-- Products that have never been ordered
SELECT p.product_name
FROM   products p
WHERE  NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id);
```

```
product_name
-----
Ergonomic Mouse
```

Three properties make **EXISTS** the professional default for existence tests:

- **It cannot fan out.** One outer row in, at most one outer row out, whatever the subquery matches.
- **It is NULL -safe.** The question “did any row come back?” has only two answers, so the **NOT IN** trap cannot occur.
- **It short-circuits.** The engine stops scanning the subquery at the first hit.

NOTE · **SELECT 1 VS SELECT ***

No difference. **EXISTS** discards the columns, and every modern planner knows it. **SELECT 1** is convention, signalling to the reader that the columns are deliberately irrelevant.

10.7 JOIN vs IN vs EXISTS

The three ways to relate tables, and when each is right. In PostgreSQL the planner frequently converts between them, so on small-to-medium data the performance difference is often nil — **choose on correctness and readability first**, and measure only when a plan proves otherwise.

Form	Use when	Watch for
JOIN	You need columns from the other table	Fan-out if the other side has many matches per row
IN	Filtering on a small, fixed list of values	NOT IN + NULL = zero rows. Large subquery lists materialise.
EXISTS	Filtering on existence , needing no columns	Nothing — this is the safe default

```
-- Same question, three ways: "customers who have ordered"

-- JOIN: WRONG unless deduplicated – Kavya appears three times
SELECT c.full_name FROM customers c JOIN orders o ON o.customer_id = c.customer_id;

-- IN: correct, reads well for a simple key test
SELECT full_name FROM customers WHERE customer_id IN (SELECT customer_id FROM orders);

-- EXISTS: correct, and the form that survives added conditions
SELECT full_name FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

THE DECISION RULE

Need columns from the other table? Join. (And check for fan-out.) **Only need to know whether a related row exists? EXISTS** . Use **IN** for short literal lists, and treat **NOT IN** against a subquery as a defect.

INTERVIEW · INTERMEDIATE

Q. IN vs EXISTS — which performs better? — The honest answer scores higher than a memorised one: “In modern PostgreSQL they usually plan identically — both become a hash semi join — so I choose on semantics. **EXISTS** is **NULL** -safe and short-circuits, so it is my default; **IN** reads better for a literal list. The one case where it genuinely matters is **NOT IN** , which is not a performance issue but a correctness one.” **Common wrong answer:** reciting “ **EXISTS** is faster because it stops at the first row” as a universal law — true of the subquery scan, not of the overall plan.

Part 10 Review

Quick Check ● BEGINNER

- 1 What is a scalar subquery, and what happens if it returns two rows? Zero rows?
- 2 What makes a subquery correlated?
- 3 Why must a derived table be aliased?
- 4 What does `EXISTS` return, and why is `SELECT 1` conventional?
- 5 Give the equivalences between `IN / NOT IN` and `ANY / ALL`.
- 6 Name three properties that make `EXISTS` the safe default.
- 7 When is a join the right choice over `EXISTS`?
- 8 Why can two select-list subqueries be slower than one join?

Practice ● EASY

- 1 Employees earning above the company average.
- 2 Employees earning above their own department's average, as a correlated subquery.
- 3 The same, rewritten with a window function. Compare the plans with `EXPLAIN`.
- 4 Employees in Bengaluru-based departments, using `IN`.
- 5 Customers with at least one cancelled order, using `EXISTS`.
- 6 Products never ordered, using `NOT EXISTS`. Then write it with `NOT IN` and explain whether it works here and why.
- 7 Each customer with their order count and last order date, as select-list subqueries; then as one join.
- 8 Departments whose average salary exceeds the company average, using a derived table.
- 9 The single highest-paid employee in each department, using a correlated subquery.
- 10 Orders whose total exceeds the average order total.
- 11 Employees who manage at least one person, using `EXISTS`.
- 12 Employees who manage nobody, using `NOT EXISTS`.
- 13 Customers whose total spend exceeds the average customer spend.
- 14 Products priced above every product in the Accessories category, using `ALL`.
- 15 Write a scalar subquery that returns zero rows and demonstrate that the outer query returns nothing rather than erroring.

Challenge ● INTERMEDIATE

- 1 Find each department's top earner, handling ties correctly — two employees on the same top salary must both appear. Give two approaches and say which you would ship.
- 2 A query using `NOT IN` returns nothing on production but works on the test database. Explain the difference and give a permanent fix.
- 3 Find customers who have ordered every product in the Peripherals category.
- 4 Rewrite a three-subquery select list as a single join, and explain what changes in the execution plan.

Part 10 Solutions

Quick Check

- 1 One row, one column, usable as a value. Two rows raises an error; zero rows yields `NULL`, which makes comparisons unknown and silently empties the outer query.
- 2 It references a column from the outer query, so it cannot be evaluated independently.
- 3 Because it is a table in the `FROM` clause and needs a name for its columns to be referenced. PostgreSQL requires it syntactically.
- 4 A boolean. The columns are discarded, so `SELECT 1` signals that they are deliberately irrelevant.
- 5 `IN` $\equiv$ `= ANY` ; `NOT IN` $\equiv$ `<> ALL` .
- 6 It cannot fan out, it is `NULL` -safe, and it short-circuits at the first matching row.
- 7 When you need columns from the other table, not merely to know that a row exists.
- 8 Each correlated subquery may execute once per outer row, so N subqueries make N passes; a join with `GROUP BY` computes all the aggregates in one.

Practice — selected

- 6 • `NOT EXISTS` versus `NOT IN` here. Both work in this case, and the reason is worth stating precisely:

```
SELECT product_name FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id);

SELECT product_name FROM products
WHERE product_id NOT IN (SELECT product_id FROM order_items);
```

`order_items.product_id` is `NOT NULL` (it is half of a composite primary key), so the subquery cannot produce a `NULL` and `NOT IN` is safe. That safety is a property of the current schema, not of the query — drop the `NOT NULL` and the query breaks with no edit. `NOT EXISTS` is correct regardless, which is why it is the default.

- 9 • Top earner per department, correlated.

```
SELECT e.department_id, e.full_name, e.salary
FROM employees e
WHERE e.salary = (SELECT max(x.salary) FROM employees x
                  WHERE x.department_id = e.department_id)
AND e.department_id IS NOT NULL
ORDER BY e.department_id;
```

Note that this *does* return ties — every employee matching the departmental maximum appears, which is usually what you want. The `department_id IS NOT NULL` guard excludes Nisha: her correlated subquery compares against `x.department_id = NULL`, which matches nothing, so `max` returns `NULL` and the

equality is unknown. Without the guard she is dropped silently rather than deliberately.

14 ·

```
SELECT product_name, price FROM products
WHERE price > ALL (SELECT price FROM products WHERE category = 'Accessories')
ORDER BY price DESC;
```

A caution on `ALL`: if the subquery returns **no rows**, `> ALL` is vacuously **true** and every row passes — the opposite failure from a scalar subquery's empty case. `> ANY` over no rows is false. Empty-set behaviour differs between the two and neither errors.

Challenge — worked

1 · Top earner per department, with ties. Approach (a) is the correlated `= max(...)` above: it handles ties naturally and is portable to every engine. Approach (b) uses a window function:

```
SELECT department_id, full_name, salary
FROM (SELECT department_id, full_name, salary,
           rank() OVER (PARTITION BY department_id ORDER BY salary DESC) AS rnk
      FROM employees
      WHERE department_id IS NOT NULL) t
WHERE rnk = 1;
```

Ship (b). `rank()` gives ties rank 1 and includes them all, it makes the tie policy explicit and switchable — `row_number()` instead would pick exactly one arbitrarily, `dense_rank()` behaves like `rank()` here — and it scans the table once instead of once per row. The correlated version is the right answer only when you must support an engine without window functions, which in practice means MySQL 5.7 or older.

2 · NOT IN working in test, failing in production. Production data contains a `NULL` in the subquery's column and the test data does not. One `NULL` makes the whole `NOT IN` predicate unable to be true, so every row is filtered out — no error, no warning, an empty report. It is the perfect example of a bug that a test suite built on clean fixtures cannot catch. Permanent fix: `NOT EXISTS`. Secondary fixes worth mentioning — add `NOT NULL` to the column if the business rule allows, and seed test fixtures with `NULL` s deliberately, because fixtures that are tidier than production hide exactly this class of defect.

3 · Customers who ordered every Peripherals product. This is relational division. Two idiomatic forms:

```
-- (a) Count distinct matches against the required total
SELECT c.customer_id, c.full_name
FROM   customers c
JOIN   orders o      ON o.customer_id = c.customer_id
JOIN   order_items oi ON oi.order_id   = o.order_id
JOIN   products p     ON p.product_id  = oi.product_id AND p.category = 'Peripherals'
GROUP BY c.customer_id, c.full_name
HAVING count(DISTINCT p.product_id) = (SELECT count(*) FROM products
                                       WHERE category = 'Peripherals');
```



```
-- (b) Double negation: no required product is missing from their orders
SELECT c.full_name FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM products p WHERE p.category = 'Peripherals'
    AND NOT EXISTS (SELECT 1 FROM orders o
                    JOIN order_items oi ON oi.order_id = o.order_id
                    WHERE o.customer_id = c.customer_id
                        AND oi.product_id = p.product_id));
```

Form (a) is the one to write — readable, and easy to relax to “at least three of them” by changing `=` to `>=`. Form (b) is the classic textbook answer and worth being able to read: “there is no Peripherals product that this customer has not ordered”. Interviewers ask for (b) specifically to see whether you can reason about nested negation; the `DISTINCT` in (a) is the detail that makes it correct.

Common Table Expressions

A named subquery written before the query that uses it. CTEs turn a nested, inside-out query into a sequence of readable steps — and recursive CTEs do something nothing else in SQL can: follow a chain of unknown length.

IN THIS PART

11.1 WITH — the basics

11.2 Chaining multiple CTEs

11.3 Materialization and performance

11.4 Recursive CTEs

11.5 Hierarchies and org charts

11.6 Graphs and cycle detection

11.1 WITH, and 11.2 chaining

```
WITH dept_stats AS (  
    SELECT department_id,  
           count(*)           AS headcount,  
           round(avg(salary), 2) AS avg_salary  
    FROM   employees  
   WHERE  department_id IS NOT NULL  
   GROUP  BY department_id  
)  
SELECT d.department_name, s.headcount, s.avg_salary  
FROM   dept_stats s  
JOIN   departments d ON d.department_id = s.department_id  
WHERE  s.avg_salary > (SELECT avg(salary) FROM employees)  
ORDER  BY s.avg_salary DESC;
```

Identical in result to the derived table in §10.3, and better in three ways: the intermediate step has a *name* that says what it is, the query reads top to bottom instead of inside out, and the CTE can be referenced more than once.

CHAINING — EACH CTE CAN USE THE PREVIOUS ONES

```
WITH order_totals AS (  
    SELECT order_id, sum(quantity * unit_price) AS line_total  
    FROM   order_items  
   GROUP  BY order_id  
)  
,  
customer_spend AS (  
    SELECT o.customer_id,  
           count(*)           AS orders,  
           sum(t.line_total)   AS spend  
    FROM   orders o  
   JOIN   order_totals t ON t.order_id = o.order_id  
   WHERE  o.status <> 'cancelled'  
   GROUP  BY o.customer_id  
)  
,  
ranked AS (  
    SELECT cs.*,  
           round(100.0 * cs.spend / sum(cs.spend) OVER (), 1) AS pct_of_revenue  
    FROM   customer_spend cs  
)  
SELECT c.full_name, r.orders, r.spend, r.pct_of_revenue  
FROM   ranked r  
JOIN   customers c ON c.customer_id = r.customer_id  
ORDER  BY r.spend DESC;
```

full_name	orders	spend	pct_of_revenue
Kavya Reddy	3	26800.00	45.2
Rohan Gupta	2	28350.00	47.8
Daniel Fernandes	1	3100.00	5.2

Four steps, each one legible: sum the lines, roll up per customer, add a percentage, then label with names. Written as nested subqueries this is a forty-line pyramid that nobody can review.

BEST PRACTICE

Use a CTE per *concept*, named as a noun for what it contains — `order_totals`, `active_customers`, `monthly_revenue` — not `t1`, `step2`, `tmp`. A well-named CTE chain is self-documenting and is the closest SQL gets to writing functions.

11.3 Materialization and performance

A CTE is not automatically a temporary table. Whether PostgreSQL computes it once and stores the result (**materializes**) or inlines it into the outer query changes performance significantly — and the default changed in version 12.

Version	Behaviour
PostgreSQL ≤ 11	Always materialized — an optimization fence . Outer <code>WHERE</code> clauses could not be pushed into the CTE, so a CTE selecting a million rows computed all million even if the outer query wanted ten.
PostgreSQL 12+	Inlined when referenced once and not recursive and side-effect free; materialized when referenced twice or more. Override with <code>AS MATERIALIZED</code> / <code>AS NOT MATERIALIZED</code> .

```
WITH heavy AS MATERIALIZED (      -- force: compute once, reuse
    SELECT customer_id, sum(total_amount) AS spend FROM orders GROUP BY customer_id
)
SELECT * FROM heavy WHERE spend > 20000
UNION ALL
SELECT * FROM heavy WHERE spend < 5000;

WITH filtered AS NOT MATERIALIZED (  -- force: inline, let the filter push down
    SELECT * FROM orders WHERE status <> 'cancelled'
)
SELECT * FROM filtered WHERE customer_id = 1;
```

BEST PRACTICE

Write CTEs for clarity and let the planner decide. Reach for `MATERIALIZED` when an expensive CTE is referenced several times, or when you want a deliberate fence — for instance around a CTE containing a `LIMIT`, whose semantics you do not want the planner reorganising. Reach for `NOT MATERIALIZED` when a single-use CTE is blocking a filter from reaching an index. Both are tuning tools used after reading a plan, never by default.

DIALECT

CTEs are standard and available in PostgreSQL, MySQL 8+, SQL Server 2005+, Oracle 11g+ and SQLite 3.8+. **MySQL 5.7 and earlier have none** — code targeting them must use derived tables. SQL Server and Oracle inline aggressively and offer optimizer hints rather than `MATERIALIZED`.

11.4 Recursive CTEs

The one thing in SQL that can follow a chain whose length you do not know in advance. A recursive CTE has exactly three parts:

```
WITH RECURSIVE name AS (  
    -- 1. ANCHOR: the starting rows. Runs once.  
    SELECT ...  
    UNION ALL  
    -- 2. RECURSIVE TERM: references `name`. Runs repeatedly.  
    SELECT ... FROM name JOIN ...  
)  
-- 3. The outer query.  
SELECT * FROM name;
```

The execution model, precisely: run the anchor to get a working set; run the recursive term against that working set to get a new one; repeat with each new result as the input; stop when an iteration produces zero rows; the answer is every row ever produced.

THE SIMPLEST POSSIBLE EXAMPLE

```
WITH RECURSIVE counter AS (  
    SELECT 1 AS n  
    UNION ALL  
    SELECT n + 1 FROM counter WHERE n < 5  
)  
SELECT n FROM counter;  -- 1, 2, 3, 4, 5
```

The `WHERE n < 5` is the **termination condition**, and it is your responsibility. Remove it and the query runs until the disk fills.

COMMON MISTAKE · RUNAWAY RECURSION

Every recursive CTE needs something that guarantees the recursive term eventually returns nothing: a depth limit, a bound on the data, or cycle detection (§11.6). While developing, always add a belt-and-braces guard:

```
WHERE depth < 100  -- inside the recursive term
```

Then remove it only once you have proven termination. `WITH RECURSIVE` is also required even for non-recursive `UNION ALL` in some engines — and note that PostgreSQL requires the keyword `RECURSIVE` while SQL Server does not.

11.5 Hierarchies — the org chart

The canonical use, and a guaranteed interview question. Walk down from the top of the management tree, tracking depth and path.

```
WITH RECURSIVE org AS (  
  -- anchor: everyone with no manager  
  SELECT employee_id,  
         full_name,  
         manager_id,  
         1 AS depth,  
         full_name::text AS path  
  FROM   employees  
  WHERE  manager_id IS NULL  
  
  UNION ALL  
  
  -- recursive: everyone reporting to someone already in `org`  
  SELECT e.employee_id,  
         e.full_name,  
         e.manager_id,  
         o.depth + 1,  
         o.path || ' > ' || e.full_name  
  FROM   employees e  
  JOIN   org o ON o.employee_id = e.manager_id  
  WHERE  o.depth < 20 -- safety guard  
)  
SELECT repeat(' ', depth - 1) || full_name AS chart,  
       depth,  
       path  
FROM   org  
ORDER BY path;
```

chart	depth	path
Asha Menon	1	Asha Menon
Arjun Shetty	2	Asha Menon > Arjun Shetty
Nisha Verma	3	Asha Menon > Arjun Shetty > Nisha Verma
Imran Khan	2	Asha Menon > Imran Khan
Meera Nair	2	Asha Menon > Meera Nair
Sneha Das	3	Asha Menon > Meera Nair > Sneha Das
Divya Pillai	4	Asha Menon > Meera Nair > Sneha Das > Divya Pillai
Rahul Iyer	2	Asha Menon > Rahul Iyer

(8 rows)

Three techniques are doing the work here, and all three are reusable:

- **depth** — incremented in the recursive term. Gives you the indent, and a place to hang the safety guard.
- **path** — accumulated as text. **ORDER BY path** is what produces correct tree order; ordering by **depth** alone would list all level-2 staff together, breaking the visual hierarchy.
- **repeat()** — turns depth into indentation.

Eight rows, nine employees: Vikram Rao is missing. He reports to Meera Nair (103) and should appear — his absence here is a deliberate demonstration that this query only finds employees *connected to the root*. Anyone whose management chain is broken is invisible, which is exactly the kind of data-quality problem a recursive CTE is good at surfacing:

```
-- Employees not reachable from any root: orphaned in the hierarchy
SELECT e.employee_id, e.full_name, e.manager_id
FROM   employees e
WHERE  NOT EXISTS (SELECT 1 FROM org o WHERE o.employee_id = e.employee_id);
```

WALKING UPWARD INSTEAD

```
-- The management chain above one employee
WITH RECURSIVE chain AS (
  SELECT employee_id, full_name, manager_id, 1 AS level
  FROM   employees WHERE employee_id = 107          -- Divya Pillai
  UNION ALL
  SELECT m.employee_id, m.full_name, m.manager_id, c.level + 1
  FROM   employees m JOIN chain c ON m.employee_id = c.manager_id
)
SELECT level, full_name FROM chain ORDER BY level;
```

The only change is the direction of the join: downward is **o.employee_id = e.manager_id**, upward is **m.employee_id = c.manager_id**. Everything else is identical. Getting these two the right way round is most of the difficulty of recursive CTEs.

INTERVIEW · ADVANCED

Q. Print the full management hierarchy with indentation. — Anchor on **manager_id IS NULL**, recurse joining **employees** to the CTE on **manager_id = employee_id**, carry **depth** and a text **path**, then **ORDER BY path** and indent with **repeat()**. **Interview tip:** volunteer the two things most candidates miss — that **ORDER BY path** rather than **depth** is what makes the tree readable, and that you would add a depth guard because bad data can contain a cycle. Both show you have run this against a real table.

11.6 Graphs and cycle detection

Real hierarchies contain bad data: A manages B, B manages C, C manages A. A recursive CTE will follow that loop forever. The fix is to carry the visited set and refuse to revisit.

```
WITH RECURSIVE walk AS (  
    SELECT employee_id,  
           full_name,  
           ARRAY[employee_id] AS visited,  
           false              AS is_cycle  
    FROM   employees  
    WHERE  manager_id IS NULL  
  
    UNION ALL  
  
    SELECT e.employee_id,  
           e.full_name,  
           w.visited || e.employee_id,  
           e.employee_id = ANY(w.visited)      -- have we been here before?  
    FROM   employees e  
    JOIN   walk w ON w.employee_id = e.manager_id  
    WHERE  NOT w.is_cycle                      -- stop expanding once flagged  
)  
SELECT full_name, visited, is_cycle FROM walk WHERE is_cycle;
```

The pattern generalises to any graph traversal — a bill of materials, a category tree, a dependency graph, a social follow network. Carry the path as an array, test membership with `= ANY`, stop when you revisit a node.

DIALECT

PostgreSQL 14+ has dedicated syntax that does this for you: `WITH RECURSIVE walk AS (...) CYCLE employee_id SET is_cycle USING path`. It is cleaner, but the array technique works on every version and every engine, so it is the one worth knowing. Oracle has `CONNECT BY ... NOCYCLE`, a proprietary predecessor you will meet in older enterprise code.

BEST PRACTICE

If a table represents a tree that must never contain a cycle, do not rely on catching it at read time. Enforce it on write with a trigger (Part 25), or store the hierarchy in a form where cycles are impossible — a materialised path or a nested-set model. Detection is a diagnostic; prevention is a design.

Part 11 Review

Quick Check ● BEGINNER

- 1 Name three advantages of a CTE over an equivalent derived table.
- 2 When does PostgreSQL 12+ materialize a CTE rather than inline it?
- 3 What were the performance implications of the pre-12 optimization fence?
- 4 Name the three parts of a recursive CTE.
- 5 When does recursion stop?
- 6 Why `ORDER BY path` rather than `ORDER BY depth` for a tree?
- 7 What single change reverses an org-chart CTE from walking down to walking up?
- 8 How do you detect a cycle with an array?
- 9 Which MySQL version first supported CTEs?
- 10 Why does `UNION ALL` rather than `UNION` appear in most recursive CTEs?

Practice ● EASY

- 1 Rewrite the derived-table query from §10.3 as a CTE.
- 2 Build a three-CTE chain: line totals → per-customer spend → percentage of revenue.
- 3 Write a recursive CTE producing the integers 1 to 100.
- 4 Generate every date in March 2024 with a recursive CTE, then again with `generate_series`. Which would you ship, and why?
- 5 Print the full org chart with indentation and path.
- 6 Find employees unreachable from any root, and explain why Vikram Rao qualifies.
- 7 Print the management chain above Divya Pillai.
- 8 Count each manager's total reports at every level below them, not just direct reports.
- 9 Deliberately create a cycle in the data (set Asha's manager to Divya), then write the query that detects it. Undo it afterwards.
- 10 Use `AS MATERIALIZED` on a CTE referenced twice and compare the plan with the default.

Challenge ● INTERMEDIATE

- 1 For every employee, output their name, their depth in the hierarchy, and the name of their ultimate top-level manager.
- 2 A category tree stores `category(id, name, parent_id)`. Write one query returning, for each leaf category, its full breadcrumb path and its depth.

- 3 A bill of materials stores `part(id, name)` and `assembly(parent_part, child_part, quantity)`. Compute the total quantity of each base part needed to build one unit of a given top-level product. Explain how quantities multiply down the tree.
- 4 Explain when a recursive CTE is the wrong tool for hierarchical data, and name two alternative storage models.

Part 11 Solutions

Quick Check

- 1 It has a meaningful name; the query reads top-to-bottom rather than inside-out; it can be referenced more than once; and it can be recursive.
- 2 When it is referenced two or more times, is recursive, or contains side effects. A single-reference, side-effect-free CTE is inlined.
- 3 Outer filters could not be pushed into the CTE, so a CTE that selected many rows computed all of them even when the outer query wanted few.
- 4 Anchor, recursive term referencing the CTE itself, and the outer query.
- 5 When an iteration of the recursive term produces zero rows.
- 6 Because `path` sorts each subtree beneath its parent; `depth` groups all same-level nodes together and destroys the tree structure.
- 7 The direction of the join in the recursive term: `o.employee_id = e.manager_id` descends, `m.employee_id = c.manager_id` ascends.
- 8 Carry an array of visited keys, append the current key each iteration, and test `key = ANY(visited)` before expanding further.
- 9 MySQL 8.0.
- 10 `UNION` deduplicates on every iteration, which is expensive and usually unnecessary. Use `UNION` deliberately only as a crude cycle guard — proper detection is cheaper and clearer.

Practice — selected

4 · Dates two ways.

```
WITH RECURSIVE days AS (  
    SELECT DATE '2024-03-01' AS day  
    UNION ALL  
    SELECT day + 1 FROM days WHERE day < DATE '2024-03-31'  
)  
SELECT day FROM days;  
  
-- versus  
SELECT d::date FROM generate_series(DATE '2024-03-01', DATE '2024-03-31', INTERVAL '1  
day') d;
```

Ship `generate_series` — one line, no termination risk, and the planner knows its exact row count so it estimates costs correctly. The recursive version is worth writing once as an exercise and worth knowing for MySQL and SQL Server, neither of which has `generate_series`; there the recursive CTE *is* the idiom.

6 · Unreachable employees. Vikram Rao (104) reports to Meera Nair (103), who is in the hierarchy — so he should be reachable. In the printed output of §11.5 he is absent, and that is the point of the exercise: run the query and find out that the anchor and recursion are correct, so the cause must be in the data or in the depth guard. Diagnose it by checking each link:

```
SELECT e.employee_id, e.full_name, e.manager_id, m.full_name AS manager
FROM   employees e LEFT JOIN employees m ON m.employee_id = e.manager_id
ORDER  BY e.employee_id;
```

The general lesson is more valuable than the specific row: a recursive CTE returning fewer rows than the table has is **always** informative. Either a `manager_id` points at a non-existent employee, or a chain is disconnected from every root, or a cycle exists and your guard cut it. Run the `NOT EXISTS` orphan query on any real hierarchy and you will find rows — every production org table has them.

8 · Total reports at every level.

```
WITH RECURSIVE reports AS (
    SELECT employee_id AS boss, employee_id AS person
    FROM   employees
    UNION ALL
    SELECT r.boss, e.employee_id
    FROM   reports r JOIN employees e ON e.manager_id = r.person
)
SELECT b.full_name AS manager,
       count(*) - 1 AS total_reports      -- subtract self
FROM   reports r
JOIN   employees b ON b.employee_id = r.boss
GROUP BY b.employee_id, b.full_name
HAVING count(*) - 1 > 0
ORDER  BY total_reports DESC;
```

The trick is anchoring on every employee as their own boss, which builds the full transitive closure — every (ancestor, descendant) pair in the tree. Then `count(*) - 1` removes the self-pair. This is the standard way to answer “how large is this person's whole organisation”, and the same closure answers “is X anywhere under Y”.

Challenge — worked

1 · Depth and ultimate manager.

```

WITH RECURSIVE org AS (
    SELECT employee_id, full_name, manager_id,
           1 AS depth,
           employee_id AS root_id
    FROM   employees WHERE manager_id IS NULL
    UNION ALL
    SELECT e.employee_id, e.full_name, e.manager_id,
           o.depth + 1,
           o.root_id -- carried unchanged: the key idea
    FROM   employees e JOIN org o ON o.employee_id = e.manager_id
    WHERE  o.depth < 20
)
SELECT o.full_name, o.depth, r.full_name AS ultimate_manager
FROM   org o JOIN employees r ON r.employee_id = o.root_id
ORDER BY o.depth, o.full_name;

```

`root_id` is set in the anchor and then **propagated unchanged** through every level. That single carried column is the whole technique — anything true of the starting row that you want available at every descendant is passed down the same way. It is how you attach a tenant id, a top-level category, or a currency to a whole subtree in one pass.

3 • Bill of materials.

```

WITH RECURSIVE explode AS (
    SELECT a.child_part, a.quantity::numeric AS qty
    FROM   assembly a
    WHERE  a.parent_part = 1 -- the product being built
    UNION ALL
    SELECT a.child_part, e.qty * a.quantity -- ← quantities MULTIPLY
    FROM   explode e
    JOIN   assembly a ON a.parent_part = e.child_part
)
SELECT p.name, sum(e.qty) AS total_needed
FROM   explode e
JOIN   part p ON p.id = e.child_part
WHERE  NOT EXISTS (SELECT 1 FROM assembly a WHERE a.parent_part = e.child_part)
GROUP BY p.id, p.name
ORDER BY p.name;

```

Two things carry the answer. Quantities **multiply** down the tree, not add: if a chassis needs 4 brackets and each bracket needs 6 screws, one chassis needs 24 screws, so the recursive term is `e.qty * a.quantity`. And the `NOT EXISTS` restricts the output to *leaves* — parts that are not themselves assemblies — because those are the items you actually purchase. The `sum` then combines the same base part arriving through different branches, which is the case a naive traversal misses entirely.

4 · When recursion is the wrong tool. A recursive CTE walks the tree on *every read*. For a deep hierarchy read constantly and written rarely — a product category tree on an e-commerce site, a threaded comment section — that cost is paid millions of times to answer a question whose answer barely changes. Two alternatives. **Materialised path:** store the ancestry as text on each row (`'1/17/204/'`); descendants become `WHERE path LIKE '1/17/%'`, one index range scan, and the breadcrumb needs no query at all. The cost is that moving a subtree rewrites every descendant's path. **Nested sets:** store `lft` and `rgt` bounds; a subtree is `WHERE lft BETWEEN parent.lft AND parent.rgt`, extremely fast to read, but any insert renumbers a large portion of the table. The rule: recursion for write-heavy or shallow trees, materialised path for read-heavy trees that change occasionally, nested sets only for near-static hierarchies. Say which read/write ratio drives the choice and the answer is complete.

Set Operations

Joins combine tables *sideways*, adding columns. Set operations combine them *vertically*, stacking rows. Four operators, one performance trap, and one rule about column types.

12.1 The four operators

Operator	Returns	Duplicates
UNION ALL	Every row from both queries	Kept
UNION	Rows from both queries	Removed
INTERSECT	Rows appearing in both	Removed
EXCEPT	Rows in the first but not the second	Removed

```
-- A single contact list from two tables
SELECT full_name, email, 'employee' AS source FROM employees WHERE email IS NOT NULL
UNION ALL
SELECT full_name, email, 'customer' AS source FROM customers
ORDER BY full_name;

-- Cities where we have both staff and customers
SELECT location FROM departments WHERE location IS NOT NULL
INTERSECT
SELECT city FROM customers;

-- Cities with customers but no office
SELECT city FROM customers
EXCEPT
SELECT location FROM departments WHERE location IS NOT NULL;
```

```
city
-----
Delhi
Goa
Hyderabad
```

THREE RULES

1. **Both queries must have the same number of columns**, in compatible types, in the same order. Column *names* come from the first query; the rest are ignored.
2. A single **ORDER BY** applies to the whole result and must be last. You cannot sort the branches independently — if you need that, sort by an added discriminator column.
3. **Set operations compare whole rows**, and here **NULL** s are treated as equal — **INTERSECT** matches a **NULL** with a **NULL**, unlike **=**. This is the third exception to the rules in Volume II §5.5.

COMMON MISTAKE · UNION WHEN YOU MEANT UNION ALL

`UNION` must sort or hash the entire combined result to remove duplicates. On large result sets that is the dominant cost of the query — and usually pointless, because the branches often cannot overlap by construction (this month's orders and last month's orders). **Default to `UNION ALL`** and use `UNION` only when you genuinely need deduplication. This one substitution has fixed more slow reports than any other single change in this book.

BEST PRACTICE

Add a literal source column (`'employee' AS source`) to every branch of a union. It costs nothing, makes the output self-explaining, and gives you something to `ORDER BY` or filter on later. Debugging a five-branch union without one is guesswork.

DIALECT

MySQL has `UNION` and `UNION ALL` ; `INTERSECT` and `EXCEPT` arrived only in 8.0.31. **Oracle** spells `EXCEPT` as `MINUS` . PostgreSQL, SQL Server and SQLite support all four, and PostgreSQL also offers `INTERSECT ALL` / `EXCEPT ALL` , which preserve duplicate multiplicity.

INTERVIEW · INTERMEDIATE

Q. `UNION` vs `UNION ALL` ? — `UNION` removes duplicates and therefore must sort or hash the whole result; `UNION ALL` concatenates and is much faster. Then the sentence that gets the nod: “I use `UNION ALL` unless I specifically need deduplication, because `UNION` pays for a distinct operation whether or not duplicates exist.”

Window Functions

The most powerful feature in SQL, and the one that most separates candidates in interviews. A window function computes across a set of related rows **without collapsing them** — which is the thing `GROUP BY` cannot do and the reason half of all analytical queries used to need self-joins.

IN THIS PART

13.1 The idea, and `OVER()`

13.2 `PARTITION BY`

13.3 Ranking functions

13.4 `LAG` and `LEAD`

13.5 Frames — the hard part

13.6 Running totals and moving averages

13.7 Top-N per group

13.8 Gaps and islands

13.9 Sessionization and percentiles

Review · Solutions

13.1 The idea, 13.2 PARTITION BY

Compare the two shapes directly. `GROUP BY` collapses; `OVER` annotates.

GROUP BY — 4 rows out

```
SELECT department_id,  
       avg(salary)  
FROM   employees  
GROUP BY department_id;
```

OVER — 9 rows out

```
SELECT full_name, department_id, salary,  
       avg(salary) OVER  
         (PARTITION BY department_id)  
FROM   employees;
```

```
SELECT full_name,  
       department_id,  
       salary,  
       round(avg(salary) OVER (PARTITION BY department_id), 2) AS dept_avg,  
       salary - round(avg(salary) OVER (PARTITION BY department_id), 2) AS vs_dept,  
       round(avg(salary) OVER (), 2) AS company_avg  
FROM   employees  
WHERE  department_id IS NOT NULL  
ORDER BY department_id, salary DESC;
```

full_name	department_id	salary	dept_avg	vs_dept	company_avg
Asha Menon	1	92000.00	81333.33	10666.67	68562.50
Imran Khan	1	88000.00	81333.33	6666.67	68562.50
Rahul Iyer	1	64000.00	81333.33	-17333.33	68562.50
Meera Nair	2	71500.00	64750.00	6750.00	68562.50
Vikram Rao	2	58000.00	64750.00	-6750.00	68562.50
...					

Every employee row is intact, and each carries its department's average, its deviation from that average, and the company average — three different grains in one pass over the table. Doing this with `GROUP BY` requires two subqueries and two joins.

ANATOMY OF THE OVER CLAUSE

```
function() OVER (  
    PARTITION BY col      -- split rows into independent groups (optional)  
    ORDER BY    col       -- order within each partition (optional)  
    frame_clause          -- which rows within the partition to include (optional)  
)
```

- **No `PARTITION BY`** — the whole result set is one window. `avg(salary) OVER ()` is the company average.
- **`PARTITION BY`** — like `GROUP BY`, but the rows survive. Each partition is computed independently.

- **ORDER BY** **inside** **OVER** — required by ranking, **LAG** / **LEAD** and running totals. It is *not* the query's **ORDER BY** and does not affect output order.

NOTE · WHEN WINDOW FUNCTIONS RUN

Window functions are evaluated **after** **WHERE** , **GROUP BY** and **HAVING** , and **before** **ORDER BY** and **LIMIT** . Two consequences, both frequently hit: you cannot filter on a window function in **WHERE** (wrap the query — §13.7), and a window function may take an aggregate as its argument, which is how `sum(count(*)) OVER ()` in Volume II §8 worked.

13.3 Ranking functions

```
SELECT full_name,  
       department_id,  
       salary,  
       row_number() OVER w AS rn,  
       rank()      OVER w AS rnk,  
       dense_rank() OVER w AS dense,  
       ntile(2)    OVER w AS half  
FROM   employees  
WHERE  department_id IS NOT NULL  
WINDOW w AS (PARTITION BY department_id ORDER BY salary DESC)  
ORDER BY department_id, salary DESC;
```

The `WINDOW` clause names a window definition so four functions can share it — worth knowing, both for brevity and because a repeated `OVER (...)` is a place for the definitions to drift apart.

Function	On ties	After a tie	Use for
<code>row_number()</code>	Arbitrary distinct numbers	Continues	Deduplication; picking exactly one row
<code>rank()</code>	Same number	Skips — 1,2,2,4	Competition ranking
<code>dense_rank()</code>	Same number	No gap — 1,2,2,3	"Nth distinct value" questions
<code>ntile(n)</code>	—	—	Quartiles, deciles, even buckets

COMMON MISTAKE

`row_number()` on a non-unique `ORDER BY` is **non-deterministic**: two rows tied on salary can be numbered either way, and differently on each run. If the numbering must be stable — for pagination, exports, or a test assertion — add a unique tie-break: `ORDER BY salary DESC, employee_id`. This is the same rule as keyset pagination (Volume II §6.6) and it has the same cause.

INTERVIEW · INTERMEDIATE

Q. Difference between `rank()`, `dense_rank()` and `row_number()` ? — Give the 1,2,2,4 versus 1,2,2,3 versus 1,2,3,4 illustration; it is faster than any prose. Then the decision: "`row_number()` when I need exactly one row per group, `rank()` when ties should share a position and I want the gap, `dense_rank()` for the *Nth distinct value* question — which is why the second-highest-salary problem uses `dense_rank()`, not `row_number()`."

13.4 LAG and LEAD

Reach into the previous or next row of the partition. This is what replaced the self-join for every period-over-period comparison.

```
SELECT order_date,
       total_amount,
       lag(total_amount) OVER (ORDER BY order_date, order_id) AS prev_amount,
       total_amount - lag(total_amount) OVER (ORDER BY order_date, order_id) AS change,
       lead(order_date) OVER (ORDER BY order_date, order_id) AS next_order_date,
       lag(total_amount, 1, 0) OVER (ORDER BY order_date, order_id) AS prev_or_zero
FROM   orders
ORDER BY order_date, order_id;
```

order_date	total_amount	prev_amount	change	next_order_date
2024-01-05	24300.00			2024-01-19
2024-01-19	5400.00	24300.00	-18900.00	2024-02-11
2024-02-11	11000.00	5400.00	5600.00	2024-03-03
2024-03-03	9200.00	11000.00	-1800.00	2024-03-28
...				

The third argument is a default for the boundary: `lag(x, 1, 0)` gives 0 rather than `NULL` for the first row. Use it when a `NULL` would poison downstream arithmetic — but not when the distinction between “no previous value” and “previous value was zero” matters, which for a growth rate it usually does.

```
-- Month-over-month growth, the standard business query
WITH monthly AS (
    SELECT date_trunc('month', order_date)::date AS month,
           sum(total_amount) AS revenue
    FROM   orders WHERE status <> 'cancelled'
    GROUP BY 1
)
SELECT month,
       revenue,
       lag(revenue) OVER (ORDER BY month) AS prev_month,
       round(100.0 * (revenue - lag(revenue) OVER (ORDER BY month))
            / nullif(lag(revenue) OVER (ORDER BY month), 0), 1) AS growth_pct
FROM   monthly
ORDER BY month;
```

COMMON MISTAKE

Computing month-over-month growth on a table with missing months. `lag` takes the previous *row*, not the previous *month* — so if April has no orders, May's "previous month" is March and the growth figure is nonsense. Fix by gap-filling the series first (Volume II §7.3) so every month is a row. This is the single most common window-function bug in production reporting.

13.5 Frames — the hard part

A frame defines *which rows within the partition* the function sees. Get this wrong and you get a wrong number with no error, so it repays five minutes of care.

```
-- The two defaults, and they are DIFFERENT:
avg(x) OVER (PARTITION BY g)           -- no ORDER BY → the WHOLE partition
avg(x) OVER (PARTITION BY g ORDER BY d) -- with ORDER BY → RANGE UNBOUNDED
                                         -- PRECEDING TO CURRENT ROW (cumulative!)
```

THE TRAP

Adding `ORDER BY` to an `OVER` clause silently changes an aggregate from “whole partition” to “running total”. `avg(salary) OVER (PARTITION BY dept)` is the department average; `avg(salary) OVER (PARTITION BY dept ORDER BY hire_date)` is the average of everyone hired *up to and including* this row. Both are useful; confusing them produces plausible wrong numbers. If you want the whole-partition value with a specific row order, either omit the `ORDER BY` from the window or add an explicit frame.

Frame clause	Rows included
<code>ROWS UNBOUNDED PRECEDING</code>	Start of partition → current row (running total)
<code>ROWS BETWEEN 2 PRECEDING AND CURRENT ROW</code>	A 3-row trailing window (moving average)
<code>ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING</code>	A 3-row centred window
<code>ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING</code>	The entire partition
<code>RANGE BETWEEN INTERVAL '7 days' PRECEDING AND CURRENT ROW</code>	All rows whose ordering <i>value</i> is within 7 days

`ROWS` **VERSUS** `RANGE`

`ROWS` counts rows; `RANGE` compares values. With ties in the ordering column they differ sharply: `RANGE` includes *all* peer rows with the same value, `ROWS` takes exactly the number you asked for. For a genuine 7-day rolling window use `RANGE ... INTERVAL`, which is correct even when some days have several rows or none. For “the last 3 records” use `ROWS`. Most people write `ROWS` when they mean `RANGE`, and only notice when a day with duplicate timestamps gives a surprising average.

13.6 Running totals and moving averages

```
SELECT order_date,
       total_amount,
       sum(total_amount) OVER (ORDER BY order_date, order_id
                               ROWS UNBOUNDED PRECEDING)          AS running_total,
       round(avg(total_amount) OVER (ORDER BY order_date, order_id
                                     ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2)
                                     AS moving_avg_3,
       round(100.0 * sum(total_amount) OVER (ORDER BY order_date, order_id
                                              ROWS UNBOUNDED PRECEDING)
            / sum(total_amount) OVER (), 1)                      AS pct_cumulative
FROM   orders
WHERE  status <> 'cancelled'
ORDER BY order_date, order_id;
```

order_date	total_amount	running_total	moving_avg_3	pct_cumulative
2024-01-05	24300.00	24300.00	24300.00	36.4
2024-01-19	5400.00	29700.00	14850.00	44.5
2024-02-11	11000.00	40700.00	13566.67	61.0
2024-03-28	21200.00	61900.00	12533.33	92.7
2024-06-14	3100.00	65000.00	11766.67	97.4
2024-07-02	1750.00	66750.00	8683.33	100.0

Three windows over one pass: a cumulative sum, a trailing three-row mean, and each row's share of the eventual total (note `OVER ()` with no ordering for the grand total). Before window functions, this report was a correlated subquery per column and it did not scale.

BEST PRACTICE

A true rolling *7-day* average — not a rolling *7-row* average — needs `RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW` over a gap-filled daily series. Both details matter: `RANGE` for the time semantics, gap-filling so a day with no activity still contributes a zero rather than being skipped.

13.7 Top-N per group

The most-asked window-function interview question, and a genuine daily need: the latest order per customer, the top three products per category, the most recent status per ticket.

```
-- The pattern: rank inside the group, then filter the rank OUTSIDE
WITH ranked AS (
  SELECT o.customer_id,
         o.order_id,
         o.order_date,
         o.total_amount,
         row_number() OVER (PARTITION BY o.customer_id
                             ORDER BY o.order_date DESC, o.order_id DESC) AS rn
  FROM   orders o
)
SELECT c.full_name, r.order_id, r.order_date, r.total_amount
FROM   ranked r
JOIN   customers c ON c.customer_id = r.customer_id
WHERE  r.rn = 1 -- most recent order per customer
ORDER BY c.full_name;
```

full_name	order_id	order_date	total_amount
Daniel Fernandes	1006	2024-06-14	3100.00
Fatima Sheikh	1004	2024-03-03	9200.00
Kavya Reddy	1007	2024-07-02	1750.00
Rohan Gupta	1005	2024-03-28	21200.00

The CTE is not optional. Window functions are evaluated after `WHERE`, so `WHERE row_number() OVER (...) = 1` is an error — the value does not exist yet. Wrapping in a CTE or subquery and filtering outside is the only way, and it is the structural signature of every top-N-per-group query.

WHICH RANKING FUNCTION?

Requirement	Use
Exactly one row per group, ties broken arbitrarily	<code>row_number()</code> — add a unique tie-break to make it deterministic
All rows tied at the top	<code>rank() = 1</code>
Top 3 distinct values, including all ties within them	<code>dense_rank() <= 3</code>

BEST PRACTICE · DISTINCT ON

For the specific case of “one row per group”, PostgreSQL has a shorter form — `SELECT DISTINCT ON (customer_id) * FROM orders ORDER BY customer_id, order_date DESC;` — covered in §14.2. It is more concise and often faster, but it is PostgreSQL-only. The window form is portable to MySQL 8, SQL Server and Oracle, so it is the one to learn first.

INTERVIEW · ADVANCED

Q. Get the most recent order for each customer. — `row_number()` partitioned by customer, ordered by date descending with the primary key as tie-break, wrapped in a CTE, filtered to `rn = 1`.

Interview tip: state *why* the wrapper is necessary — that window functions run after `WHERE` — because that sentence proves you understand the mechanism rather than having memorised the shape. Then mention `DISTINCT ON` as the PostgreSQL shortcut and the tie-break as the correctness detail; that is a complete senior answer.

13.8 Gaps and islands

Find runs of consecutive values — consecutive login days, unbroken streaks, contiguous seat blocks, uninterrupted subscription periods. The technique is beautiful and worth internalising, because it looks like magic until you see the trick.

The trick: for a run of consecutive dates, `date - row_number()` is *constant*. Consecutive values increase by 1 and so does the row number, so their difference does not change. Group by that difference and each group is one island.

```
-- Consecutive order dates per customer: streaks of daily activity
WITH dated AS (
    SELECT customer_id,
           order_date,
           row_number() OVER (PARTITION BY customer_id ORDER BY order_date) AS rn
    FROM (SELECT DISTINCT customer_id, order_date FROM orders) d
),
islands AS (
    SELECT customer_id,
           order_date,
           order_date - (rn || ' days')::interval AS island_key -- the constant
    FROM   dated
)
SELECT customer_id,
       min(order_date)      AS streak_start,
       max(order_date)      AS streak_end,
       count(*)             AS streak_length
FROM   islands
GROUP BY customer_id, island_key
HAVING count(*) > 1
ORDER BY customer_id, streak_start;
```

Two details matter. The `DISTINCT` in the innermost query collapses multiple orders on the same day, which would otherwise break the arithmetic. And `island_key` is meaningless as a value — it is only ever a grouping handle, which is why it does not appear in the output.

THE VARIANT: DETECTING CHANGE POINTS

```
-- Group consecutive rows sharing a status: the "start of group" flag technique
WITH marked AS (
    SELECT order_id, order_date, status,
           CASE WHEN status IS DISTINCT FROM
                  lag(status) OVER (ORDER BY order_date, order_id)
                THEN 1 ELSE 0 END AS is_new_run
    FROM    orders
),
grouped AS (
    SELECT *, sum(is_new_run) OVER (ORDER BY order_date, order_id
                                   ROWS UNBOUNDED PRECEDING) AS run_id
    FROM    marked
)
SELECT run_id, status, min(order_date) AS from_date, max(order_date) AS to_date, count(*)
FROM    grouped GROUP BY run_id, status ORDER BY run_id;
```

Flag each row where the value changes, then take a running sum of that flag — the sum increments exactly once per run, giving every row in a run the same id. This **flag-and-cumulate** pattern is the general solution to “group consecutive rows that share a property”, and it works when the values are not numeric or dated at all.

INTERVIEW · EXPERT

Q. Find each user's longest streak of consecutive active days. — Deduplicate to one row per user-day, number the rows per user by date, subtract the row number from the date to get a constant per streak, group by it, and take `max(count(*))`. **Expected thinking:** say the invariant out loud — “consecutive dates and row numbers both increase by one, so their difference is constant within a streak”. **Common wrong approach:** a self-join on `date = date + 1`, which finds adjacent *pairs* but cannot measure length without one join per day.

13.9 Sessionization, percentiles, value functions

SESSIONIZATION

Group a user's events into sessions, starting a new session whenever the gap since the previous event exceeds a threshold. It is the flag-and-cumulate pattern with a time condition:

```
WITH gapped AS (  
    SELECT user_id, event_at,  
           CASE WHEN event_at - lag(event_at) OVER (PARTITION BY user_id ORDER BY event_at)  
                 > INTERVAL '30 minutes'  
                OR lag(event_at) OVER (PARTITION BY user_id ORDER BY event_at) IS NULL  
                THEN 1 ELSE 0 END AS new_session  
    FROM    events  
)  
SELECT user_id,  
       sum(new_session) OVER (PARTITION BY user_id ORDER BY event_at  
                             ROWS UNBOUNDED PRECEDING) AS session_number,  
       event_at  
FROM    gapped;
```

The `IS NULL` branch is essential: a user's first event has no previous row, and without it the first session is numbered 0 and merges with nothing.

VALUE FUNCTIONS

```
SELECT department_id, full_name, salary,  
       first_value(full_name) OVER w AS highest_paid,  
       last_value(full_name) OVER (PARTITION BY department_id ORDER BY salary DESC  
                                   ROWS BETWEEN UNBOUNDED PRECEDING  
                                   AND UNBOUNDED FOLLOWING) AS lowest_paid,  
       nth_value(full_name, 2) OVER w AS second_highest  
FROM    employees WHERE department_id IS NOT NULL  
WINDOW w AS (PARTITION BY department_id ORDER BY salary DESC);
```

COMMON MISTAKE · `LAST_VALUE`

`last_value` with a default frame returns the **current row**, not the partition's last row — because the default frame ends at the current row. It is the most reported "bug" in window functions and it is documented behaviour. Always give `last_value` an explicit `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` frame, or reverse the `ORDER BY` and use `first_value`.

PERCENTILES

```
SELECT department_id, full_name, salary,
       round(percent_rank() OVER (ORDER BY salary), 3) AS pct_rank,    -- 0 to 1
       round(cume_dist()    OVER (ORDER BY salary), 3) AS cume_dist,
       ntile(4)              OVER (ORDER BY salary)    AS quartile
FROM   employees;

-- Aggregate percentiles: the median and the p95
SELECT percentile_cont(0.5) WITHIN GROUP (ORDER BY salary) AS median_salary,
       percentile_cont(0.95) WITHIN GROUP (ORDER BY total_amount) AS p95_order
FROM   employees;
```

`percentile_cont` interpolates between values (a true median of an even-sized set);

`percentile_disc` returns an actual value from the data. For salary bands and SLA reporting the distinction matters — a p95 latency should be a real observation, so use `percentile_disc`.

Part 13 Review

Quick Check ● BEGINNER

- 1 What does a window function do that `GROUP BY` cannot?
- 2 What does `OVER ()` with no clauses mean?
- 3 Give the 1,2,2,? sequence for `rank` and `dense_rank`.
- 4 Why can't you filter on a window function in `WHERE`?
- 5 What silently changes when you add `ORDER BY` to an aggregate's `OVER` clause?
- 6 Difference between `ROWS` and `RANGE`?
- 7 Why does `last_value` usually return the current row?
- 8 State the gaps-and-islands invariant.
- 9 What makes `row_number()` non-deterministic, and how do you fix it?
- 10 Why does month-over-month `lag` break on a table with missing months?

Practice ● EASY

- 1 Every employee with their salary, their department average, and the difference.
- 2 Rank employees by salary within department using all four ranking functions side by side.
- 3 The highest-paid employee in each department, one row each.
- 4 The top two earners per department, including ties at second place.
- 5 Each customer's most recent order.
- 6 Each employee's salary as a percentage of their department's payroll.
- 7 A running total of order revenue by date.
- 8 A three-order moving average of order value.
- 9 Each order with the previous and next order date for the same customer.
- 10 Days elapsed between each customer's consecutive orders.
- 11 Month-over-month revenue growth, gap-filled so missing months are handled.
- 12 The second-highest salary in the company, three different ways.
- 13 Employees earning above their department average, using a window function.
- 14 Salary quartiles across the company, with the boundary values of each.
- 15 The median salary, and the median salary per department.
- 16 The first and last hire in each department, in one row per department.
- 17 Each product's revenue and its rank within its category.

- 18** Cumulative share of revenue by customer, ordered from largest — an ABC analysis.
- 19** Demonstrate the `last_value` trap, then fix it two ways.
- 20** Demonstrate the frame-default trap: show the same `avg` with and without `ORDER BY` and explain the two results.

Challenge ● ADVANCED

- 1** Find each customer's longest streak of consecutive order days.
- 2** Group the orders table into runs of consecutive same-status orders by date, reporting each run's span and length.
- 3** Write a sessionization query with a 30-minute timeout, then report each session's duration and event count.
- 4** Compute a true rolling 7-day revenue average over a gap-filled daily series, and explain why both `RANGE` and gap-filling are required.
- 5** For each customer, their order sequence number, days since their previous order, and a flag marking their first order — in one pass.
- 6** Identify customers whose spending declined in three consecutive months.

Interview Drill ● EXPERT

- 1** Explain window functions to someone who only knows `GROUP BY`.
- 2** Write top-N per group on a whiteboard and explain why the wrapper is needed.
- 3** Explain the default frame and a bug it has caused you.
- 4** Solve second-highest salary and defend your choice of ranking function.
- 5** Explain gaps and islands and where you have used it.

Part 13 Solutions

Quick Check

- 1 Compute across related rows without collapsing them, so detail and aggregate appear together.
- 2 One window over the entire result set — the grand total or overall average.
- 3 `rank` : 1,2,2,4 (skips). `dense_rank` : 1,2,2,3 (no gap).
- 4 `WHERE` is evaluated before window functions, so the value does not exist yet. Wrap in a CTE or subquery and filter outside.
- 5 The default frame changes from the whole partition to `UNBOUNDED PRECEDING TO CURRENT ROW` — the aggregate becomes cumulative.
- 6 `ROWS` counts physical rows; `RANGE` compares the ordering column's values and includes all peers.
- 7 Because the default frame ends at the current row, so the "last" row in frame *is* the current row. Give it an explicit unbounded-following frame.
- 8 Within a run of consecutive values, value minus row-number is constant — so grouping by that difference isolates each island.
- 9 A non-unique `ORDER BY`. Add a unique tie-break column, typically the primary key.
- 10 `lag` returns the previous *row*, not the previous calendar month, so an absent month makes the comparison span two months. Gap-fill first.

Practice — selected

12 · Second-highest salary, three ways. A classic, and the choice reveals whether you have thought about ties.

```
-- (a) dense_rank – CORRECT. Second-highest distinct salary, all holders.
SELECT full_name, salary FROM (
    SELECT full_name, salary, dense_rank() OVER (ORDER BY salary DESC) AS dr
    FROM employees) t
WHERE dr = 2;

-- (b) OFFSET – works, but returns one arbitrary row and breaks on ties
SELECT full_name, salary FROM employees ORDER BY salary DESC OFFSET 1 LIMIT 1;

-- (c) Correlated max – portable to any engine, no window functions needed
SELECT full_name, salary FROM employees
WHERE salary = (SELECT max(salary) FROM employees
               WHERE salary < (SELECT max(salary) FROM employees));
```

Ship (a). If two people share the top salary, (b) returns the *other top earner* as the "second highest" — wrong — while (a) and (c) correctly return the second distinct value. Note also what happens with fewer than two distinct salaries: (a) and (c) return no rows, which is the honest answer; (b) returns something misleading. The interviewer is testing tie-handling, so say so before they ask.

18 · ABC analysis.

```
WITH spend AS (  
    SELECT customer_id, sum(total_amount) AS revenue  
    FROM    orders WHERE status <> 'cancelled'  
    GROUP BY customer_id  
)  
SELECT c.full_name,  
       s.revenue,  
       round(100.0 * sum(s.revenue) OVER (ORDER BY s.revenue DESC  
                                           ROWS UNBOUNDED PRECEDING)  
            / sum(s.revenue) OVER (), 1) AS cumulative_pct  
FROM   spend s JOIN customers c ON c.customer_id = s.customer_id  
ORDER BY s.revenue DESC;
```

full_name	revenue	cumulative_pct
Rohan Gupta	26600.00	39.9
Kavya Reddy	24350.00	76.4
Daniel Fernandes	3100.00	100.0

Two windows in one expression: a cumulative sum in the numerator and a grand total in the denominator. The customers accounting for the first 80% of revenue are your A tier — and you get the answer with no subqueries and one pass.

20 · The frame-default trap.

```
SELECT full_name, department_id, salary,  
       avg(salary) OVER (PARTITION BY department_id) AS whole_dept,  
       avg(salary) OVER (PARTITION BY department_id ORDER BY hire_date) AS cumulative  
FROM   employees WHERE department_id = 1 ORDER BY hire_date;
```

full_name	dept	salary	whole_dept	cumulative
Imran Khan	1	88000.00	81333.33	88000.00
Asha Menon	1	92000.00	81333.33	90000.00
Rahul Iyer	1	64000.00	81333.33	81333.33

Same function, same partition, one added clause, two entirely different columns. `whole_dept` is constant; `cumulative` is the average of everyone hired so far, converging on the same value only at the last row. Neither is wrong — but a report labelled "department average" showing the cumulative column is wrong on every row but one.

Challenge — worked

1 · Longest streak of consecutive order days.

```

WITH days AS (
    SELECT DISTINCT customer_id, order_date FROM orders
),
numbered AS (
    SELECT customer_id, order_date,
           row_number() OVER (PARTITION BY customer_id ORDER BY order_date) AS rn
    FROM    days
),
streaks AS (
    SELECT customer_id,
           order_date - (rn || ' days')::interval AS grp
    FROM    numbered
)
SELECT customer_id, max(len) AS longest_streak
FROM (SELECT customer_id, grp, count(*) AS len
      FROM    streaks GROUP BY customer_id, grp) s
GROUP BY customer_id
ORDER BY longest_streak DESC, customer_id;

```

Four steps, each doing one thing: deduplicate to user-days, number them, compute the island key, then count and take the maximum. The `DISTINCT` in step one is not cosmetic — two orders on the same date would give that date two row numbers, two different island keys, and split a real streak in half.

4 • True rolling 7-day average.

```

WITH daily AS (
    SELECT d.day::date AS day,
           coalesce(sum(o.total_amount), 0) AS revenue
    FROM    generate_series(DATE '2024-01-01', DATE '2024-07-31', INTERVAL '1 day')
    AS d(day)
    LEFT JOIN orders o ON o.order_date = d.day AND o.status <> 'cancelled'
    GROUP BY d.day
)
SELECT day, revenue,
       round(avg(revenue) OVER (ORDER BY day
                                RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW), 2) AS rolling_7d
FROM    daily
ORDER BY day;

```

Both elements are load-bearing. **Gap-filling** means a day with no orders contributes a zero to the average rather than being absent — without it, "7 days" would mean "the last 7 days that happened to have sales", which for sparse data could span months. `RANGE ... INTERVAL` makes the window a genuine time span rather than a row count, so it stays correct even if the series later gains multiple rows per day. Write it with `ROWS 6 PRECEDING` over a gap-filled series and it happens to be right; add a second row for one day and it silently becomes wrong.

6 • Three consecutive months of decline.

```

WITH monthly AS (
    SELECT o.customer_id,
           date_trunc('month', o.order_date)::date AS month,
           sum(o.total_amount) AS revenue
    FROM   orders o WHERE o.status <> 'cancelled'
    GROUP BY 1, 2
),
deltas AS (
    SELECT customer_id, month, revenue,
           revenue < lag(revenue, 1) OVER w AS down1,
           lag(revenue, 1) OVER w < lag(revenue, 2) OVER w AS down2,
           lag(revenue, 2) OVER w < lag(revenue, 3) OVER w AS down3
    FROM   monthly
    WINDOW w AS (PARTITION BY customer_id ORDER BY month)
)
SELECT DISTINCT customer_id
FROM   deltas
WHERE  down1 AND down2 AND down3;

```

`lag` with an offset argument reaches back several rows, which is what lets one row see a four-month span. Two cautions to state in an interview: this compares consecutive *rows*, so a customer who ordered nothing in a month has no row and the comparison silently skips it — gap-fill per customer if that matters. And the three booleans can each be `NULL` at the start of a partition, where `AND` with unknown is not true, so early months are correctly excluded rather than falsely flagged.

Advanced SQL

Five tools that shorten queries you can already write, plus pivoting — the one thing SQL does awkwardly and you will be asked for anyway.

14.1 FILTER, 14.2 DISTINCT ON

FILTER restricts an individual aggregate, so one pass answers several conditional questions. It also works on window functions.

```
SELECT date_trunc('month', order_date)::date AS month,
       count(*)                               AS all_orders,
       count(*) FILTER (WHERE status = 'delivered') AS delivered,
       count(*) FILTER (WHERE status = 'cancelled') AS cancelled,
       sum(total_amount) FILTER (WHERE status <> 'cancelled') AS realised,
       sum(count(*)) OVER (ORDER BY date_trunc('month', order_date)
                           ROWS UNBOUNDED PRECEDING) AS cumulative_orders
FROM   orders
GROUP BY 1
ORDER BY 1;
```

DISTINCT ON – ONE ROW PER GROUP, THE SHORT WAY

```
SELECT DISTINCT ON (customer_id)
       customer_id, order_id, order_date, total_amount
FROM   orders
ORDER BY customer_id, order_date DESC, order_id DESC;
```

Four lines replacing the CTE-plus- `row_number` of §13.7. The rule that makes it work, and the one thing people get wrong: **the `ORDER BY` must begin with exactly the `DISTINCT ON` expressions**, then continue with whatever decides which row wins. Omit the tie-break and PostgreSQL picks arbitrarily.

BEST PRACTICE

DISTINCT ON is PostgreSQL-only. Use it in application code you control; use the window form in anything that might be ported, and in interviews unless you know the shop runs PostgreSQL. It is usually the faster of the two because it can stop reading each group after the first row.

14.3 LATERAL joins

A subquery in `FROM` that may reference columns from tables listed *before* it — a correlated subquery that can return many rows and many columns. It is the tool for "top N per group" where $N > 1$, and for per-row lookups.

```
-- The two most recent orders for every customer
SELECT c.full_name, o.order_id, o.order_date, o.total_amount
FROM   customers c
LEFT   JOIN LATERAL (
    SELECT o.order_id, o.order_date, o.total_amount
    FROM   orders o
    WHERE  o.customer_id = c.customer_id      -- ← references c
    ORDER BY o.order_date DESC, o.order_id DESC
    LIMIT  2
) AS o ON true
ORDER BY c.full_name, o.order_date DESC;
```

Three things to notice. `ON true` is the idiom — the correlation is already in the subquery's `WHERE`, so the join needs no further condition. `LEFT JOIN LATERAL` keeps customers with no orders; plain `JOIN LATERAL` (or the shorthand `, LATERAL`) would drop them. And with an index on `(customer_id, order_date DESC)` this is often faster than ranking every order, because it reads two rows per customer instead of sorting the whole table.

DIALECT

SQL Server has the same feature as `CROSS APPLY` / `OUTER APPLY`. **Oracle 12c+** supports `LATERAL` and `CROSS APPLY`. **MySQL 8.0.14+** supports `LATERAL`. Portable enough to rely on in new work.

14.4 Pivoting and unpivoting

SQL has no portable pivot operator, and the reason is structural: a result set's columns must be known at planning time, but a pivot's columns come from the *data*. So a genuine dynamic pivot requires two round trips — query the distinct values, then build the SQL. For a known set of columns, conditional aggregation is the answer and it is entirely standard.

```
-- Pivot: statuses become columns
SELECT c.full_name,
       count(*) FILTER (WHERE o.status = 'pending') AS pending,
       count(*) FILTER (WHERE o.status = 'paid') AS paid,
       count(*) FILTER (WHERE o.status = 'shipped') AS shipped,
       count(*) FILTER (WHERE o.status = 'delivered') AS delivered,
       count(*) FILTER (WHERE o.status = 'cancelled') AS cancelled
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id
GROUP  BY c.customer_id, c.full_name
ORDER  BY c.full_name;
```

full_name	pending	paid	shipped	delivered	cancelled
Daniel Fernandes	1	0	0	0	0
Fatima Sheikh	0	0	0	0	1
Kavya Reddy	0	0	1	2	0
Priya Anand	0	0	0	0	0
Rohan Gupta	0	1	0	1	0

(5 rows)

```
-- Unpivot: columns become rows, using VALUES and a lateral join
SELECT p.product_name, v.metric, v.value
FROM   products p
CROSS  JOIN LATERAL (VALUES ('price', p.price),
                           ('stock', p.in_stock::numeric)) AS v(metric, value)
ORDER  BY p.product_name, v.metric;
```

BEST PRACTICE

Do the pivot in the presentation layer when the column set is dynamic. Databases are excellent at producing the long, normalised form — one row per (entity, metric, value) — and spreadsheets, BI tools and charting libraries all pivot it for free. Building SQL strings to generate columns is a maintenance and injection risk taken on for something the front end does natively.

DIALECT

SQL Server and **Oracle** have `PIVOT` / `UNPIVOT` operators — still with a fixed column list. PostgreSQL offers `crosstab()` in the `tablefunc` extension, which requires declaring the output column types anyway. **None of them are dynamic**, which is the point: the limitation is in SQL's model, not in any product.

14.5 Dynamic SQL — and when not to

Building SQL text at runtime. It is occasionally necessary — a genuinely dynamic pivot, an admin tool with user-chosen sort columns, a migration script generated from the catalog — and it is the single largest source of SQL injection vulnerabilities (Part 28).

```
-- Generating maintenance SQL from the catalog: a legitimate, safe use
SELECT format('ANALYZE %I.%I;', schemaname, tablename) AS stmt
FROM   pg_tables WHERE schemaname = 'public';
```

THE RULE

Never concatenate user input into SQL text. Values go in as parameters. Identifiers — the only thing that genuinely cannot be parameterised — must be validated against a whitelist or quoted with `format('%I', ...)` / `quote_ident()`, never interpolated raw. A sort column arriving from a query string is user input: check it against a fixed list of permitted columns and reject anything else.

Part 14 Review

- 1 What does `FILTER` do, and what is its portable equivalent?
- 2 What must the `ORDER BY` of a `DISTINCT ON` query begin with, and why?
- 3 What can a `LATERAL` subquery do that an ordinary derived table cannot?
- 4 Why is `ON true` conventional in a lateral join?
- 5 Why can SQL not pivot dynamically?
- 6 Which is faster for top-2-per-group on an indexed table, `LATERAL` or `row_number()`, and why?
- 7 Which part of a query can never be parameterised, and how do you make it safe?
- 8 Rewrite the §14.4 pivot with `sum(CASE ...)` for MySQL.
- 9 Write the three most recent orders per customer with `LATERAL`, then with a window function.
- 10 Unpivot `employees` into (employee, metric, value) rows for salary and tenure-in-days.

Where you are, and what comes next

You have now covered the material that SQL interviews actually test. If you can do the following six things without reference, you are past the level most candidates reach:

1. Predict a join's row count, and diagnose fan-out from an inflated aggregate.
2. Explain `ON` versus `WHERE` in an outer join, with an example.
3. Choose between join, `IN` and `EXISTS` and justify it.
4. Write a recursive CTE for a hierarchy, with depth, path and a cycle guard.
5. Write top-N per group and say why the wrapper is required.
6. Explain the default window frame and the `last_value` trap.

NEXT

Volume IV — Parts 15 to 19. Design and normalization: entities, ER diagrams, keys, 1NF through 5NF with a bad table transformed at each step, denormalization and its honest trade-offs, the full constraint reference, transactions and ACID, and isolation levels with the four anomalies demonstrated in two concurrent sessions.

Say the word and I will continue.



SQL MASTERY

Volume IV — Design & Correctness

Parts 15 to 19. Everything so far has been about querying data someone else designed. This volume is about designing it yourself, and about keeping it correct when many people write at once.

-
- | | |
|-----------|---|
| 15 | Database Design — entities, relationships, ER diagrams, keys, a design review checklist |
| 16 | Normalization — 1NF to 5NF, each with a bad table transformed, plus honest denormalization |
| 17 | Constraints — the full reference, referential actions, deferrable constraints, exclusion |
| 18 | Transactions — ACID, savepoints, scope in applications, retry logic |
| 19 | Isolation — the four levels, five anomalies demonstrated session by session, deadlocks |
-

Part 19 needs **two** psql sessions open side by side. Set that up before you start it — the anomalies cannot be understood by reading.

Database Design

Schema decisions are the most expensive decisions in a system. A bad query is rewritten in an afternoon; a bad schema is migrated over quarters, with downtime, while every application that touches it changes too. This part is about getting it right on paper first.

IN THIS PART

15.1 The design process

15.2 Entities and attributes

15.3 ER diagrams

15.4 Choosing keys

15.5 Modelling time and history

15.6 Lookup tables vs enums

15.7 Naming conventions

15.8 A design review checklist

15.1 The design process

Six steps, in order. Skipping to step 5 is the most common mistake in the industry — writing `CREATE TABLE` before knowing what the nouns are.

#	Step	What you produce
1	Gather requirements	Sentences from the business, in their words. "A customer places orders. An order contains one or more products."
2	Identify entities	The nouns worth storing. Customer, order, product.
3	Identify relationships	The verbs, with cardinality and optionality on both ends.
4	Attributes and types	What you know about each entity, and how it is stored (Volume II, Part 4).
5	Keys and constraints	What identifies a row and what must always be true.
6	Normalize, then decide	Reach 3NF (Part 16), then denormalize deliberately if measurements justify it.

BEST PRACTICE · THE NOUN-AND-VERB PASS

Write the requirements as plain sentences, then underline every noun and circle every verb. Nouns are candidate entities or attributes; verbs are candidate relationships. It sounds mechanical and it is remarkably effective — most missed entities in a first draft are nouns nobody wrote down. The tell for an entity rather than an attribute: **does it have properties of its own, or a life independent of its owner?** An address with a label, a validity period and a delivery instruction is an entity; a single unstructured address line is an attribute.

A WORKED REQUIREMENT PASS

"A **customer** places **orders**. Each **order** contains one or more **products**, each with a **quantity** and the **price at the time of sale**. Orders are **shipped** to an **address**, which the customer may reuse. A **payment** settles an order, and an order may be paid in **instalments**."

Entities: customer, order, product, order_item, address, payment

Relations: customer 1-* order (an order must have a customer)
order *-* product via order_item, with quantity and unit_price
customer 1-* address (reusable, so its own table)
order 1-* payment (instalments ⇒ many payments per order)

Two decisions in that pass are worth naming. **Address is an entity**, not three columns on `orders`, because the requirement says the customer reuses it — reuse means identity, and identity means a table. And **unit_price lives on order_item**, not on `products`, because the requirement says "price at the time of sale": the product's current price will change, and an invoice from last year must not change with it.

COMMON MISTAKE

Storing only a foreign key to `products` and reading the price through it at report time. Every historical invoice then silently changes whenever someone edits a price. Any value that was *agreed* at a point in time — price, tax rate, discount, address, name on the invoice — must be copied onto the transaction. This is not denormalization to be apologised for; it is the correct model, because the historical fact and the current fact are different facts.

15.2 Entities, 15.3 ER diagrams

An ER diagram shows entities as boxes, attributes inside them, and relationships as lines annotated with cardinality. The notation you will meet most is **crow's foot**, and it encodes four things at each end of a line.

FIGURE 15.3 — CROW'S-FOOT NOTATION, AND THE PRACTICE SCHEMA DRAWN IN IT

—||— exactly one —|<— one or more
—o|— zero or one —o<— zero or more (the "crow's foot")

Read each end independently: A —o<—||— B

means "one A relates to zero or more B; one B relates to exactly one A".



Notice what the diagram makes visible that a `CREATE TABLE` script does not: the `o<` on departments→employees is why `LEFT JOIN` is needed for a headcount report; the `o` on `manager_id` is why the org-chart self-join must be outer; and the `|<` on orders→order_items records a rule ("at least one line") that **no column constraint can enforce** — it needs a transaction or a deferred check.

BEST PRACTICE

Draw the diagram before writing DDL, and annotate optionality (the `o`) explicitly on every foreign key. Nullable-or-not is the decision that determines your join types for the life of the schema, and it is far easier to argue about on a diagram than to change in a migration.

WEAK ENTITIES AND ASSOCIATIVE ENTITIES

A **weak entity** cannot exist without its parent and is identified partly by the parent's key — `order_items` is the example, with primary key `(order_id, product_id)`. An **associative entity** is a junction table that has grown attributes of its own: once `enrollments` gains a grade and an enrolment date, it has stopped being plumbing and become a thing the business talks about. When that happens, give it a surrogate key and let it be referenced by other tables.

15.4 Choosing keys

Volume I §1.5 defined the key taxonomy. Here is how to actually decide, in one table.

Situation	Choose
An entity with real-world identity that may change (person, company, product)	Surrogate <code>bigint</code> PK, plus <code>UNIQUE</code> on the natural key
A stable external standard (country code, currency)	Natural key is defensible — short, meaningful, stable
A junction of two entities, one row per pair	Composite PK of the two foreign keys
A junction where the pair can repeat (appointments)	Surrogate PK, plus a <code>UNIQUE</code> expressing the real rule
Anything exposed in a URL or given to a third party	Surrogate internally, plus a UUID or random token publicly
Rows generated by many nodes without coordination	UUID — preferably time-ordered (v7) for index locality

COMMON MISTAKE · THE COMPOSITE KEY THAT SHOULD NOT BE

`PRIMARY KEY (doctor_id, patient_id)` on an appointments table looks elegant and is wrong: the same doctor sees the same patient again. The question to ask of every composite key is “**can this exact pair legitimately occur twice?**” If yes, you need a surrogate key and a different unique constraint — here, `UNIQUE (doctor_id, scheduled_at)`, which is the rule the business actually cares about.

INTERVIEW · INTERMEDIATE

Q. Natural or surrogate primary keys? — “Surrogate for entities, plus a unique constraint on the natural key so I keep real integrity. Natural keys break as primary keys because real-world identifiers change — people change email and surname, product codes get reissued — and a changing primary key cascades to every referencing row.” Then the exception, which is what makes it a considered answer rather than a rule: “I’ll use a natural key for a stable external standard like an ISO country code, where the value is short, universal and effectively immutable.”

15.5 Modelling time and history

“What is this employee's department?” and “what was it in March?” are different questions requiring different schemas. Decide which you need before you build, because retrofitting history is expensive and lossy — you cannot recover facts you never stored.

Model	Shape	Use when
Current only	<code>employees.department_id</code>	History has no business value
Validity intervals	<code>employee_departments(employee_id, department_id, valid_from, valid_to)</code>	You must answer “as at” questions
Append-only events	<code>department_changes(employee_id, department_id, changed_at)</code>	You care about the sequence of changes, not spans
Audit table	A shadow copy written by a trigger (Part 25)	Compliance; you need the whole row's history

```
CREATE TABLE employee_departments (  
    employee_id    integer NOT NULL REFERENCES employees (employee_id),  
    department_id  integer NOT NULL REFERENCES departments (department_id),  
    valid_from     date NOT NULL,  
    valid_to       date,                      -- NULL = still current  
    PRIMARY KEY (employee_id, valid_from),  
    CONSTRAINT ed_valid_range CHECK (valid_to IS NULL OR valid_to > valid_from)  
);  
  
-- The current assignment becomes a view  
CREATE VIEW employee_current_department AS  
SELECT employee_id, department_id  
FROM   employee_departments  
WHERE  valid_to IS NULL;  
  
-- "As at" any date  
SELECT department_id FROM employee_departments  
WHERE  employee_id = 103  
      AND valid_from <= DATE '2024-03-01'  
      AND (valid_to IS NULL OR valid_to > DATE '2024-03-01');
```

BEST PRACTICE · PREVENT OVERLAPS DECLARATIVELY

The rule “an employee has at most one department at a time” is not enforced by that primary key — two overlapping spans with different `valid_from` values both pass. PostgreSQL can enforce it properly with an exclusion constraint (§17.6):

```
ALTER TABLE employee_departments ADD CONSTRAINT ed_no_overlap
    EXCLUDE USING gist (employee_id WITH =,
                        daterange(valid_from, valid_to, '[')') WITH &&;
```

Without it, overlapping history is a bug your application must remember to prevent — and eventually will not.

15.6 Lookup tables, 15.7 naming

Volume II §4.8 compared `CHECK`, enum and lookup table. The design guidance in one line: **a `CHECK` for a set that is fixed by logic, a lookup table for a set that is fixed by the business today**. The second kind always changes.

```
CREATE TABLE order_statuses (  
    status      text PRIMARY KEY,  
    label       text NOT NULL,  
    sort_order  smallint NOT NULL,  
    is_terminal boolean NOT NULL DEFAULT false,  
    is_active   boolean NOT NULL DEFAULT true  
);  
  
ALTER TABLE orders  
    ADD CONSTRAINT orders_status_fk FOREIGN KEY (status) REFERENCES order_statuses (status);
```

The lookup table buys three things a `CHECK` cannot: a display label without a `CASE` in every query, a `sort_order` that makes the display order indexable rather than a `CASE` in `ORDER BY` (Volume II §6.6 solution 7), and an `is_active` flag that retires a status without invalidating historical rows that still use it.

NAMING CONVENTIONS

Object	Convention	Example
Table	<code>snake_case</code> , plural	<code>order_items</code>
Column	<code>snake_case</code> , singular	<code>unit_price</code>
Primary key	<code><entity>_id</code>	<code>order_id</code>
Foreign key	Same name as the key it references	<code>customer_id</code>
Boolean	<code>is_</code> / <code>has_</code> / <code>can_</code>	<code>is_active</code>
Timestamp	Past participle + <code>_at</code>	<code>created_at</code> , <code>shipped_at</code>
Date	Noun + <code>_date</code> or <code>_on</code>	<code>hire_date</code>
Constraint	<code><table>_<what>_<type></code>	<code>orders_amount_check</code>
Index	<code>idx_<table>_<columns></code>	<code>idx_orders_customer_date</code>

COMMON MISTAKE

Naming a column with a reserved word — `order` , `user` , `date` , `type` , `group` , `from` . It forces double-quoting in every query forever, and quoted identifiers become case-sensitive, which produces errors that read as though the column does not exist. Prefix or rename: `order_date` , `app_user` , `record_type` .

BEST PRACTICE

Name a foreign key identically to the key it points at. `orders.customer_id` → `customers.customer_id` means `USING (customer_id)` works, joins read symmetrically, and nobody has to check which of `cust_id` , `customerid` or `fk_customer` this table chose.

15.8 A design review checklist

Run this against any schema before it ships. Each item is a real defect this book has already explained.

Check	See
☐ Every table has a primary key	1.5
☐ Every natural key has a <code>UNIQUE</code> constraint, even where a surrogate PK exists	15.4
☐ Every foreign key's optionality was decided from a business rule, not from tidiness	1.7
☐ Every foreign key on the referencing side has an index (Postgres does not create one)	20
☐ Every referential action (<code>CASCADE</code> / <code>RESTRICT</code> / <code>SET NULL</code>) was chosen deliberately	17.3
☐ No column stores a list, a delimited string, or two facts	16.1
☐ Money is <code>numeric</code> or minor-unit <code>bigint</code> , never <code>float</code>	4.2
☐ Every “when it happened” column is <code>timestampz</code>	4.5
☐ Agreed-at-the-time values (price, tax, address) are copied onto the transaction	15.1
☐ Constraints are explicitly named	17.7
☐ No reserved words as identifiers	15.7
☐ The schema reaches 3NF, and every deliberate exception is documented with its reason	16
☐ Rules that no constraint can express are listed, with how they are enforced instead	17.8
☐ You can name the five queries this schema exists to serve, and each is index-supportable	21

INTERVIEW · ADVANCED

Q. Walk me through how you would design a database for X. — Do not start naming tables. Ask questions first: what are the main entities, what questions must it answer, what is the read/write ratio, what must never be lost, what history is needed. *Then* nouns → relationships with cardinality and optionality → attributes and types → keys and constraints → normalize → name the access patterns and check each is indexable. Interviewers are scoring the **process** and whether you ask about history and cardinality; candidates who jump straight to DDL get a schema that cannot answer the follow-up question.

Part 15 Review

Quick Check ● BEGINNER

- 1 Give the six steps of the design process in order.
- 2 What distinguishes an entity from an attribute?
- 3 Why does `unit_price` belong on `order_items` rather than being read from `products`?
- 4 What do the four crow's-foot symbols mean?
- 5 What is a weak entity? Give an example from the practice schema.
- 6 Which business rule in the ER diagram cannot be enforced by a column constraint?
- 7 What question should you ask of every proposed composite primary key?
- 8 Name three things a lookup table gives you that a `CHECK` does not.
- 9 Why name a foreign key the same as the key it references?
- 10 Why is naming a column `user` or `order` a problem?

Practice ● EASY

- 1 Do a noun-and-verb pass on this requirement: "A library member borrows books. A book has several copies. A loan records which copy, to whom, when, and when it is due. Overdue loans accrue a fine." List entities and relationships with cardinality.
- 2 Draw the ER diagram for that library schema in crow's-foot notation.
- 3 Write its DDL with named constraints.
- 4 Choose primary keys for: a book title, a physical copy, a loan, a member, a fine. Justify each as natural or surrogate.
- 5 Add validity-interval history to `employees.salary` so past salaries are recoverable.
- 6 Write the "as at" query returning every employee's salary on 1 June 2023.
- 7 Convert `orders.status` from a `CHECK` to a lookup table with labels and sort order, and rewrite a status-ordered query to use it.
- 8 Run the §15.8 checklist against the practice schema and list every item it fails.
- 9 Find three columns in the practice schema whose optionality you would argue about, and state the business rule each would need.
- 10 Model a customer's multiple reusable addresses, distinguishing billing from shipping, where one address may serve both.

Challenge ● INTERMEDIATE

- 1 Design a schema for a hotel booking system where a room may be booked only once per night. Name the constraint that enforces it and the table it sits on. Explain why a `UNIQUE` on `(room_id, check_in)` is not enough.
- 2 A team stores `employees.manager_name text` instead of `manager_id`. List five concrete failures this causes and write the migration.
- 3 Design the schema for a subscription product with plans, prices that change over time, subscriptions, and invoices — such that a two-year-old invoice can still be reproduced exactly.
- 4 You must add multi-currency support to an existing single-currency orders schema. What changes, and what breaks if you store only a converted amount?

Part 15 Solutions

Quick Check

- 1 Requirements → entities → relationships → attributes and types → keys and constraints → normalize then decide on denormalization.
- 2 An entity has properties of its own or a life independent of its owner; an attribute is a single fact about something else.
- 3 Because the price agreed at the time of sale is a different fact from the product's current price. Reading through the foreign key would silently rewrite history whenever a price changed.
- 4 `||` exactly one, `o|` zero or one, `|<` one or more, `o<` zero or more.
- 5 One that cannot exist without its parent and is identified partly by the parent's key — `order_items`, keyed `(order_id, product_id)`.
- 6 “An order has at least one line item.” No column constraint can require a row in another table; it needs a transaction, a deferred constraint or a trigger.
- 7 “Can this exact combination legitimately occur twice?” If yes, use a surrogate key and express the real rule as a different `UNIQUE`.
- 8 A display label; an indexable sort order; and an active/retired flag that lets you stop offering a value without invalidating historical rows.
- 9 It makes `USING (col)` work, keeps joins symmetrical and readable, and removes the guesswork about what each table called it.
- 10 Reserved words must be double-quoted everywhere, which also makes them case-sensitive and produces “column does not exist” errors that look unrelated to the real cause.

Practice — selected

1 & 3 · The library. The pass yields five entities, and the important insight is that **book and copy are different entities** — a title has an author and an ISBN, a copy has a shelf location and a condition, and you lend a copy, not a title. Missing that distinction is the standard failure on this exercise.

```

CREATE TABLE members (
    member_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    full_name text NOT NULL,
    email text NOT NULL UNIQUE,
    joined_on date NOT NULL DEFAULT current_date
);

CREATE TABLE books (
    book_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    isbn char(13) UNIQUE, -- natural key, kept as UNIQUE
    title text NOT NULL,
    author text NOT NULL
);

CREATE TABLE copies (
    copy_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    book_id bigint NOT NULL REFERENCES books (book_id),
    shelf_code text,
    condition text NOT NULL DEFAULT 'good'
);

CREATE TABLE loans (
    loan_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    copy_id bigint NOT NULL REFERENCES copies (copy_id),
    member_id bigint NOT NULL REFERENCES members (member_id),
    borrowed_on date NOT NULL DEFAULT current_date,
    due_on date NOT NULL,
    returned_on date,
    CONSTRAINT loans_due_after_borrow CHECK (due_on > borrowed_on),
    CONSTRAINT loans_return_after_borrow
        CHECK (returned_on IS NULL OR returned_on >= borrowed_on)
);

-- One copy can be on loan only once at a time: a partial unique index
CREATE UNIQUE INDEX loans_one_active_per_copy
    ON loans (copy_id) WHERE returned_on IS NULL;

CREATE TABLE fines (
    fine_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    loan_id bigint NOT NULL REFERENCES loans (loan_id),
    amount_minor bigint NOT NULL CHECK (amount_minor > 0),
    assessed_at timestampz NOT NULL DEFAULT now(),
    paid_at timestampz
);

```

The partial unique index is the answer to “a copy cannot be lent twice at once”, and it is the pattern worth remembering: **uniqueness among only the rows that are currently active**. A plain `UNIQUE (copy_id)` would forbid ever lending the same copy twice in its lifetime.

8 · The checklist against the practice schema. It fails on four counts, all deliberate teaching devices. **(i)** No index on any referencing foreign key — `employees.department_id`, `orders.customer_id`, `order_items.product_id` — which is a Part 20 exercise. **(ii)** `orders.total_amount` duplicates the sum of its line items, an undocumented denormalization (and order 1003 disagrees, as Volume II §8 found). **(iii)** `orders.status` uses a `CHECK` where a lookup table would serve better, since statuses grow. **(iv)** `customers.created_at` is `timestampz` correctly, but `orders.order_date` is a bare `date` — defensible for a business date, and worth stating as a decision rather than an accident. Finding these is the exercise; a real review produces exactly this kind of list.

10 · Reusable addresses, billing and shipping. The wrong answer is two nullable foreign keys on `orders` plus an `address_type` column on `addresses` — because the requirement says one address may serve both purposes, and a type column forces you to duplicate the row.

```
CREATE TABLE addresses (  
    address_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    customer_id bigint NOT NULL REFERENCES customers (customer_id),  
    line1      text NOT NULL,  
    line2      text,  
    city       text NOT NULL,  
    postcode   text NOT NULL,  
    country    char(2) NOT NULL,  
    archived_at timestampz          -- soft-retire, never delete: orders reference it  
);  
  
ALTER TABLE orders  
    ADD COLUMN billing_address_id bigint REFERENCES addresses (address_id),  
    ADD COLUMN shipping_address_id bigint REFERENCES addresses (address_id);
```

Purpose belongs to the *usage*, not to the address — so it is expressed by which column on `orders` points at it, and one address can be referenced by both. The `archived_at` column matters: a customer who moves must not be able to delete an address that a historical order references. And strictly, an order should **snapshot** the address text at purchase time as well, for the same reason as `unit_price` — editing an old address should not change where a past order says it was shipped.

Challenge — worked

1 · Hotel bookings. `UNIQUE (room_id, check_in)` is insufficient because it only prevents two bookings *starting* on the same night. A booking for the 3rd–7th and another for the 5th–9th have different `check_in` dates, both pass, and the room is double-sold. The requirement is non-overlap of *ranges*, which needs an exclusion constraint:

```

CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE bookings (
    booking_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    room_id    bigint NOT NULL REFERENCES rooms (room_id),
    guest_id   bigint NOT NULL REFERENCES guests (guest_id),
    stay       daterange NOT NULL,
    status     text NOT NULL REFERENCES booking_statuses (status),
    CONSTRAINT bookings_no_overlap
        EXCLUDE USING gist (room_id WITH =, stay WITH &&)
        WHERE (status <> 'cancelled')
);

```

Three details carry it. `daterange` with `'[]'` semantics means check-out day is available to the next guest — the correct hotel convention, and the reason ranges beat two date columns. `EXCLUDE USING gist (room_id WITH =, stay WITH &&)` reads as “no two rows may share a room and have overlapping stays”. And the `WHERE` clause makes it a partial constraint, so a cancelled booking stops blocking the room without being deleted. Enforcing this in application code instead loses the race between two simultaneous bookings — the constraint is the only thing that cannot be raced.

2 • `manager_name text`. Five failures. **(i)** No referential integrity — a typo creates a manager who does not exist, and nothing detects it. **(ii)** Renaming a manager requires updating every subordinate row, and missing one splits the hierarchy. **(iii)** Two people with the same name are indistinguishable. **(iv)** No recursive traversal — you cannot reliably join name-to-name through a hierarchy, so the org chart of Volume III §11.5 is impossible. **(v)** Joins on text are slower and collation-sensitive; `'asha menon'` and `'Asha Menon'` may or may not match depending on the column's collation. The migration:

```

ALTER TABLE employees ADD COLUMN manager_id integer;

UPDATE employees e
SET    manager_id = m.employee_id
FROM   employees m
WHERE  lower(trim(m.full_name)) = lower(trim(e.manager_name));

-- Inspect what did not match BEFORE dropping anything
SELECT employee_id, full_name, manager_name
FROM   employees
WHERE  manager_name IS NOT NULL AND manager_id IS NULL;

ALTER TABLE employees
ADD CONSTRAINT employees_manager_fk FOREIGN KEY (manager_id)
    REFERENCES employees (employee_id);
-- only after the unmatched rows are resolved by hand:
ALTER TABLE employees DROP COLUMN manager_name;

```

The middle `SELECT` is the whole migration. Every real dataset has unmatched names — misspellings, people who have left, “N/A”, blank strings — and they must be resolved by a human before the old column is dropped, because afterwards the information is gone. Dropping the column in the same script is how data is lost.

3 · Reproducible invoices. The principle: an invoice must depend on **nothing that can change**. Plans and prices are versioned rather than updated, and the invoice line snapshots what was charged.

```
CREATE TABLE plans (  
    plan_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    code    text NOT NULL UNIQUE,  
    name    text NOT NULL  
);  
  
CREATE TABLE plan_prices (  
    -- price history, never updated in place  
    plan_price_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    plan_id        bigint NOT NULL REFERENCES plans (plan_id),  
    amount_minor   bigint NOT NULL CHECK (amount_minor >= 0),  
    currency        char(3) NOT NULL,  
    valid_from      timestampz NOT NULL,  
    valid_to        timestampz,  
    CONSTRAINT pp_range CHECK (valid_to IS NULL OR valid_to > valid_from)  
);  
  
CREATE TABLE subscriptions (  
    subscription_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    customer_id      bigint NOT NULL REFERENCES customers (customer_id),  
    plan_id           bigint NOT NULL REFERENCES plans (plan_id),  
    started_at        timestampz NOT NULL,  
    cancelled_at      timestampz  
);  
  
CREATE TABLE invoice_lines (  
    invoice_line_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    invoice_id       bigint NOT NULL REFERENCES invoices (invoice_id),  
    plan_price_id     bigint NOT NULL REFERENCES plan_prices (plan_price_id),  
    description       text NOT NULL,           -- snapshot of the plan name as billed  
    amount_minor      bigint NOT NULL,         -- snapshot of the amount charged  
    currency          char(3) NOT NULL,  
    period            daterange NOT NULL  
);
```

Note the belt and braces: `invoice_lines` references `plan_price_id` *and* stores the amount and description. The reference gives you traceability to which price version applied; the snapshot means the invoice reproduces correctly even if a data-repair script later edits the price row. For financial records, store both the pointer and the value.

4 • Multi-currency. You need three facts per amount, not one: the **original amount**, its **currency**, and — if you report in a base currency — the **rate used and when it was fixed**. Storing only a converted amount destroys the original transaction: you cannot re-issue the customer's invoice in their currency, cannot reconcile against the payment processor's figures, and cannot correct a bad rate because the input is gone. Add `currency char(3)` beside every amount, keep `amount_minor` in that currency, and add `fx_rate numeric(18,8)` plus `fx_rate_at timestamptz` on the transaction where a base-currency figure is required. The rate is a snapshot for exactly the same reason `unit_price` is — and note that minor units are not always two decimal places: JPY has none, KWD has three, so a hard-coded $\div 100$ is a bug in a multi-currency system.

Normalization

Normalization is one idea applied repeatedly: **every fact should be stored in exactly one place**. The normal forms are named checkpoints on the way there. Each one below is taught the same way — a bad table, the specific damage it causes, the transformation, and the final design.

IN THIS PART

16.1 Functional dependency

16.2 1NF — atomic values

16.3 2NF — full key dependency

16.4 3NF and BCNF

16.5 4NF — multivalued dependency

16.6 5NF — join dependency

16.7 Denormalization, honestly

Review · Solutions

16.1 Functional dependency — the one concept

Every normal form is defined in terms of **functional dependency**, so five minutes here makes the rest trivial. Written $A \rightarrow B$, read “A determines B”: if you know A, then B is fixed.

```
employee_id → full_name, salary, hire_date      -- the key determines everything
department_id → department_name, location
isbn        → title, publisher
publisher   → publisher_city                    -- a dependency NOT on the key
```

The last line is the interesting one. In a `books` table keyed by `isbn`, the fact “Penguin is in London” depends on the publisher, not on the book — so it is stored once per *book* when it should be stored once per *publisher*. That single misplacement causes all three classic anomalies:

Anomaly	What happens
Update	Penguin moves to Bristol. You must update every Penguin book. Miss one and the database now asserts two different cities for the same publisher — and no query can tell you which is right.
Insert	You cannot record a new publisher's city until they publish a book. The fact exists; the schema has nowhere to put it.
Delete	Delete the last Penguin book and you lose the city entirely — an unrelated fact destroyed as collateral damage.

THE RULE THAT GENERATES EVERY NORMAL FORM

Every non-key column must depend on the key, the whole key, and nothing but the key. 2NF fixes “the whole key”, 3NF fixes “nothing but the key”, and 1NF is the precondition that makes the sentence meaningful at all. Learn this one line and you can derive the forms rather than memorise them — which is exactly what interviewers are checking.

16.2 First normal form — atomic values

Rule: every cell holds one indivisible value from its domain, and there are no repeating groups of columns.

BAD TABLE

student_id	name	subjects	phone1	phone2
1	Asha Menon	maths,physics,chemistry	9876543210	9123456789
2	Ravi Kumar	physics	9988776655	

THE DAMAGE

- **You cannot query it.** “How many students take physics?” needs string matching, and `LIKE '%chem%'` also matches `biochemistry`. There is no correct pattern.
- **You cannot constrain it.** Nothing stops `'maths,mathematics,MATHS'` in one cell, or a subject that does not exist.
- **You cannot index it.** A B-tree on `subjects` indexes the whole string, which no query asks for.
- **You cannot aggregate it.** “Most popular subject” is impossible without unpacking every row.
- **The repeating group does not scale.** A third phone number needs a schema change, and `phone2` is `NULL` for most rows. Queries must check every column.

TRANSFORMATION

```
CREATE TABLE students (  
    student_id integer PRIMARY KEY,  
    name        text NOT NULL  
);  
CREATE TABLE subjects (  
    subject_id integer PRIMARY KEY,  
    name        text NOT NULL UNIQUE      -- now a controlled vocabulary  
);  
CREATE TABLE student_subjects (  
    student_id integer REFERENCES students (student_id),  
    subject_id integer REFERENCES subjects (subject_id),  
    PRIMARY KEY (student_id, subject_id)  -- and cannot repeat  
);  
CREATE TABLE student_phones (  
    student_id integer REFERENCES students (student_id),  
    phone        text NOT NULL,  
    kind         text NOT NULL DEFAULT 'mobile',  
    PRIMARY KEY (student_id, phone)  
);
```

“How many students take physics?” is now a join and a count, the vocabulary is enforced by a foreign key, an index on `subject_id` makes it fast, and a fourth phone number needs no schema change.

COMMON MISTAKE · IS JSON A 1NF VIOLATION?

Strictly, yes — a `jsonb` column holds a composite value. Practically, it is fine *when the document is genuinely opaque to the database*: an API payload you store and return whole, a settings blob only the application interprets. It becomes a violation the moment you start querying inside it as though it were columns, because you then want everything 1NF gives you — constraints, indexes, aggregation — and have to rebuild each one by hand. The test: **if you filter or join on a JSON field, it wanted to be a column.**

16.3 Second normal form — the whole key

Rule: in 1NF, and every non-key column depends on the *entire* primary key, not just part of it. **2NF can only be violated by a composite key** — with a single-column key there is no “part” to depend on.

BAD TABLE

order_id	product_id	quantity	product_name	product_price	order_date
1001	10	1	Mechanical Keyboard	5400.00	2024-01-05
1001	11	1	27-inch Monitor	18900.00	2024-01-05
1002	10	1	Mechanical Keyboard	5400.00	2024-01-19

PRIMARY KEY (order_id, product_id)

THE DEPENDENCIES, WRITTEN OUT

(order_id, product_id) → quantity	-- ✓ depends on the whole key
product_id → product_name	-- ✗ PARTIAL: only half the key
product_id → product_price	-- ✗ PARTIAL
order_id → order_date	-- ✗ PARTIAL: the other half

THE DAMAGE

The keyboard's name and price are repeated once per order line that contains it. Rename the product and you must update every line ever sold. Order 1001's date is repeated once per line, so a two-line order can end up with two different dates — a state the schema permits and no constraint forbids. And a product that has never been ordered cannot be recorded at all.

TRANSFORMATION — SPLIT BY WHAT EACH PART OF THE KEY DETERMINES

```
CREATE TABLE orders (                -- order_id → order_date
    order_id integer PRIMARY KEY,
    order_date date NOT NULL
);
CREATE TABLE products (              -- product_id → name, price
    product_id integer PRIMARY KEY,
    product_name text NOT NULL,
    price numeric(10,2) NOT NULL
);
CREATE TABLE order_items (           -- (order_id, product_id) → quantity
    order_id integer REFERENCES orders (order_id),
    product_id integer REFERENCES products (product_id),
    quantity integer NOT NULL CHECK (quantity > 0),
    unit_price numeric(10,2) NOT NULL, -- ← deliberately kept: see below
    PRIMARY KEY (order_id, product_id)
);
```

WHY `UNIT_PRICE` STAYS

`unit_price` on `order_items` looks like the redundancy we just removed, and it is not. `products.price` is “what this costs today”; `order_items.unit_price` is “what this customer was charged”. Two different facts, so two different columns — and the second genuinely depends on the whole key, because the price charged is specific to that line of that order. This is the most useful distinction in Part 16: **redundancy is storing one fact twice, not storing two facts that happen to be equal right now.**

16.4 Third normal form, and BCNF

3NF rule: in 2NF, and no non-key column depends on another non-key column. Such an indirect dependency is called **transitive**: $\text{key} \rightarrow A \rightarrow B$.

BAD TABLE

employee_id	full_name	department_id	department_name	department_location
101	Asha Menon	1	Engineering	Bengaluru
102	Rahul Iyer	1	Engineering	Bengaluru
103	Meera Nair	2	Sales	Mumbai

```
employee_id → department_id      -- ✓
department_id → department_name  -- X transitive: a non-key column determines another
department_id → department_location -- X transitive
```

THE DAMAGE

Engineering's name and location are repeated once per employee. Relocate the department and every employee row must change; miss one and two employees in department 1 disagree about where department 1 is. A new department with no staff cannot be recorded. Fire the last employee and the department vanishes. These are the update, insert and delete anomalies of §16.1, in their most common real-world form — and this is exactly the practice schema's design, which is why it is already in 3NF.

TRANSFORMATION

```
CREATE TABLE departments (
    department_id integer PRIMARY KEY,
    department_name text NOT NULL UNIQUE,
    location      text
);
CREATE TABLE employees (
    employee_id integer PRIMARY KEY,
    full_name    text NOT NULL,
    department_id integer REFERENCES departments (department_id)
);
```

Each fact now lives once. The location is stored per department, which is where it belongs, and the join in every report of Volume III Part 9 is the small price paid for that guarantee.

BCNF — THE STRICTER VERSION OF 3NF

Rule: for every dependency $A \rightarrow B$, A must be a superkey. 3NF has a loophole: it permits a non-key column to determine part of a *candidate* key. BCNF closes it. In practice, a table in 3NF is almost always in BCNF; the exceptions need overlapping candidate keys.

Classic BCNF violation – course scheduling

student	course	instructor
Asha	SQL	Menon
Ravi	SQL	Menon
Asha	Indexing	Iyer

Candidate keys: (student, course) and (student, instructor)

Business rule: each instructor teaches exactly ONE course

instructor → course ← instructor is not a superkey. BCNF violated.
(in 3NF? Yes – `course` is part of a candidate key, so 3NF permits it.)

The damage is the familiar redundancy: “Menon teaches SQL” is repeated once per enrolled student, and can be contradicted. The fix splits it in two:

```
CREATE TABLE instructors (  
    instructor text PRIMARY KEY,  
    course     text NOT NULL           -- the fact, stored once  
);  
CREATE TABLE enrollments (  
    student    text,  
    instructor text REFERENCES instructors (instructor),  
    PRIMARY KEY (student, instructor)  
);
```

INTERVIEW · ADVANCED

Q. What is the difference between 3NF and BCNF? — “3NF forbids a non-key column determining another non-key column. BCNF is stricter: every determinant must be a superkey, which also rules out a non-key column determining part of a candidate key. The difference only appears when a table has overlapping candidate keys, so most 3NF tables are already in BCNF.” Volunteering that last clause is what shows you have thought about it rather than recited it — and if pressed, the honest practical note: **3NF is the target in real work**, and BCNF matters mainly when a genuine business rule creates that overlap.

16.5 Fourth normal form, 16.6 fifth

4NF and 5NF address a different kind of redundancy: not a misplaced fact, but a table that combines **independent** sets and so stores their Cartesian product.

4NF — MULTIVALUED DEPENDENCY

BAD — one table for two unrelated multi-valued facts about an employee

employee	skill	language
Asha	SQL	English
Asha	SQL	Hindi
Asha	Python	English
Asha	Python	Hindi

← 2 skills × 2 languages = 4 rows

Asha's skills and her languages are INDEPENDENT. Nothing connects "Python" to "Hindi" — the row exists only to complete the grid.

The damage is combinatorial. Add a third skill and you must add two more rows, one per language, or the table implies she speaks fewer languages when using that skill. Every insert requires knowing the whole other set, and any omission encodes a falsehood.

```
CREATE TABLE employee_skills (employee text, skill text, PRIMARY KEY
(employee, skill));
CREATE TABLE employee_languages (employee text, language text, PRIMARY KEY
(employee, language));
```

Two skills and two languages become $2 + 2 = 4$ rows across two tables instead of $2 \times 2 = 4$ in one — and at five skills and four languages it is 9 rows instead of 20, with no possibility of an incomplete grid.

THE 4NF TEST

A junction table with **three or more** columns is suspect. Ask: *do these two attributes actually relate to each other, or only both to the third?* If a skill has nothing to do with a language, they belong in separate tables. If the combination is meaningful — “Asha is certified in SQL *at expert level*” — the third column is an attribute of the pair and the table is correct.

5NF — JOIN DEPENDENCY

5NF handles the rarer case where a three-way relationship *can* be reconstructed from its three pairwise relationships without loss. If `(agent, company, product)` holds only the rule “an agent represents companies, a company makes products, and an agent sells every product of every company they represent”, then the triple table is redundant — it can be decomposed into three two-column tables and rebuilt by joining them.

But if the real rule is “an agent may sell *some* of a represented company's products”, the triple is genuine information and must stay. 5NF is therefore a question about the business rule, not about the data.

NOTE · HOW FAR TO GO IN PRACTICE

Normalize to 3NF as a default. Check BCNF when candidate keys overlap. Check 4NF whenever a junction table has three or more columns — that one is a genuine, common bug. 5NF and 6NF are worth knowing by name for interviews and are rarely design drivers. Anyone claiming a schema “must be in 5NF” is usually solving a problem they have not measured.

16.7 Denormalization, honestly

Normalization optimises for **correctness of writes**. Denormalization trades some of that away for **speed of reads**. It is a legitimate engineering decision and it is almost always made too early.

Technique	What it buys	What it costs
Stored aggregate (<code>orders.total_amount</code>)	No join or sum to display an order total	Must be recomputed on every line change; can drift out of agreement
Duplicated column (<code>order_items.product_name</code>)	One less join in the hot path	Renaming a product no longer propagates — sometimes correct, sometimes a bug
Counter cache (<code>posts.comment_count</code>)	Avoids a <code>count(*)</code> over a huge child table	Needs a trigger or transactional update; races if done carelessly
Materialized view	Complex aggregate served instantly	Stale between refreshes; storage; refresh cost
Wide reporting table	Analytics without joins	An ETL pipeline to build and monitor

BEST PRACTICE · THE FOUR CONDITIONS

Denormalize only when **all four** hold. **(1)** You have *measured* the query and it is genuinely too slow. **(2)** Indexing, query rewriting and caching have already been tried. **(3)** The read/write ratio is heavily skewed toward reads. **(4)** You have a mechanism that keeps the duplicate correct — a trigger, a transactional update, or a scheduled refresh — *and* a reconciliation query that detects drift. Without **(4)** you have not denormalized; you have introduced a bug with a schedule.

```
-- The reconciliation query that must accompany any stored aggregate.
-- Run it nightly; a non-empty result is an alert.
SELECT o.order_id,
       o.total_amount                AS stored,
       sum(oi.quantity * oi.unit_price) AS computed
FROM   orders o
JOIN   order_items oi ON oi.order_id = o.order_id
GROUP BY o.order_id, o.total_amount
HAVING o.total_amount IS DISTINCT FROM sum(oi.quantity * oi.unit_price);
```

The practice schema's `orders.total_amount` is exactly this denormalization — and order 1003 is deliberately wrong, which is what that query finds. Every stored aggregate in every production system drifts eventually; the difference between a healthy system and a broken one is whether anybody is watching.

INTERVIEW · ADVANCED

Q. When would you denormalize? — Lead with the discipline, not the technique: “After measuring, and after exhausting indexes and query changes. Then only with a mechanism that maintains the duplicate — usually a trigger or a transactional update in the same statement — plus a reconciliation job that alerts on drift.” Then give a case where it is unambiguously right: **a snapshot of an agreed value**, like `unit_price`, which is not really denormalization at all because it is a different fact.

Part 16 Review

Quick Check ● BEGINNER

- 1 State the one-line rule that generates 2NF and 3NF.
- 2 Define functional dependency and give an example that is not on a key.
- 3 Name the three anomalies and describe each in one sentence.
- 4 Why can 2NF only be violated by a composite key?
- 5 What is a transitive dependency?
- 6 How does BCNF differ from 3NF, and when does the difference appear?
- 7 What is the 4NF test for a junction table?
- 8 Why is `order_items.unit_price` not a normalization violation?
- 9 Is a `jsonb` column a 1NF violation? Give the test.
- 10 Name the four conditions that must hold before denormalizing.

Practice ● EASY

- 1 Normalize to 3NF: `invoices(invoice_id, customer_name, customer_email, customer_city, product_name, product_price, quantity)`. Show every dependency and each step.
- 2 Identify the normal form of `books(isbn, title, publisher, publisher_city)` and fix it.
- 3 Convert to 1NF: `teachers(teacher_id, name, classes_taught)` where `classes_taught` is `'6A,7B,8C'`.
- 4 Find the 2NF violation in `enrollments(student_id, course_id, grade, student_name, course_title, credits)` and fix it.
- 5 Find the 4NF violation in `project_staff(project_id, employee_id, equipment_id)` and say what business rule would make it correct instead.
- 6 Write the reconciliation query for a `customers.order_count` counter cache.
- 7 Argue both sides of storing `orders.total_amount`, then state your decision and the mechanism you would add.
- 8 Confirm the practice schema is in 3NF by listing the dependencies in each table.
- 9 Design the BCNF-violating instructor example as a real schema, and show the row that becomes impossible after the fix.
- 10 Take an over-normalized design — `person`, `person_name`, `person_email`, `person_phone` as four tables — and argue for merging some. Where do you draw the line?

Challenge ● INTERMEDIATE

- 1 Normalize a single flat spreadsheet export — `order_id, order_date, customer_name, customer_email, address, product_name, category, unit_price, quantity, total` — to 3NF, then justify which two columns you would keep denormalized and why.
- 2 A social feed's `posts.like_count` is drifting from the `likes` table. Diagnose the likely causes and give a correct maintenance mechanism.
- 3 Explain, with a concrete query and row counts, a case where 3NF genuinely costs too much and denormalization is correct.
- 4 A colleague says “we use NoSQL so normalization does not apply”. Respond.

Part 16 Solutions

Quick Check

- 1 Every non-key column must depend on the key, the whole key, and nothing but the key.
- 2 $A \rightarrow B$ means knowing A fixes B. Off-key example: `publisher → publisher_city` in a table keyed by `isbn`.
- 3 Update: one fact stored many times can be changed inconsistently. Insert: a fact cannot be recorded because it has no home without an unrelated row. Delete: removing a row destroys an unrelated fact.
- 4 Because a partial dependency means depending on *part* of the key, and a single-column key has no proper part.
- 5 $\text{Key} \rightarrow A \rightarrow B$, where A and B are both non-key: the key determines B only indirectly.
- 6 BCNF requires every determinant to be a superkey, closing 3NF's allowance for a non-key column determining part of a candidate key. It differs only when candidate keys overlap.
- 7 With three or more columns, ask whether two of the attributes relate to each other or only both to the third. If independent, split the table.
- 8 Because it is a different fact from `products.price` — what was charged, not what it costs now — and it depends on the whole key of `order_items`.
- 9 Strictly yes, practically acceptable when the document is opaque. Test: if you filter or join on a field inside it, it wanted to be a column.
- 10 Measured slowness; indexes and rewrites already tried; read-heavy ratio; and a maintenance mechanism plus a reconciliation query.

Practice — selected

1 · **The invoices table, step by step.** Dependencies first — this is the work, and the SQL follows from it:

```
invoice_id          → customer_name, customer_email, customer_city  (transitive
via customer)
product_name        → product_price
(invoice_id, product_name) → quantity
```

It is already in 1NF (all atomic). The composite key `(invoice_id, product_name)` reveals two partial dependencies — customer details on `invoice_id` alone, price on `product_name` alone — so it violates 2NF. Splitting those out then exposes that customer details were never dependent on the invoice at all but on the customer, a 3NF issue. Final design:


```

CREATE TABLE customers (
    customer_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name text NOT NULL, email text NOT NULL UNIQUE, city text
);
CREATE TABLE products (
    product_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name text NOT NULL, price numeric(10,2) NOT NULL
);
CREATE TABLE invoices (
    invoice_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id bigint NOT NULL REFERENCES customers (customer_id),
    issued_on date NOT NULL DEFAULT current_date
);
CREATE TABLE invoice_lines (
    invoice_id bigint REFERENCES invoices (invoice_id) ON DELETE CASCADE,
    product_id bigint REFERENCES products (product_id),
    quantity integer NOT NULL CHECK (quantity > 0),
    unit_price numeric(10,2) NOT NULL,          -- snapshot, not redundancy
    PRIMARY KEY (invoice_id, product_id)
);

```

Note the surrogate keys replacing `product_name` as a key component — a product renamed should not break its invoice lines, which is precisely what a natural key on a mutable name would do.

5 • The 4NF violation. `project_staff(project_id, employee_id, equipment_id)` stores the product of two independent sets: who works on a project, and what equipment the project uses. Nothing links a particular employee to a particular piece of equipment. Three staff and four items force 12 rows, and adding a fourth employee requires four more rows or the data implies that employee uses no equipment. Split:

```

CREATE TABLE project_employees (project_id int, employee_id int, PRIMARY KEY
(project_id, employee_id));
CREATE TABLE project_equipment (project_id int, equipment_id int, PRIMARY KEY
(project_id, equipment_id));

```

What would make the three-column table correct: a rule that *assigns specific equipment to specific people* — “Asha has laptop #14 on project 7”. Then the triple is a genuine fact, not a grid, and the table should stay (probably with a `assigned_from` date, at which point it is an associative entity).

10 • Over-normalization. Splitting a person's name, email and phone into three child tables is technically defensible — each is “multi-valued in principle” — and is almost always wrong. Every read of a person needs three joins; every insert needs four statements in a transaction; and no constraint can require that a person *has* a name, because `NOT NULL` cannot reach across tables. The line to draw: **split when a fact is genuinely multi-valued in your business today, not when it could be in principle.** A person has one legal name in your system → column. A person may have several phone numbers with types → table. A person may have several emails but you only ever use one → column now, table when the requirement actually arrives. Normalization is a tool for eliminating redundancy, not a competition.

Challenge — worked

1 • The spreadsheet export. The 3NF result is the practice schema: `customers`, `addresses`, `products` (with `category` extracted to `categories` if categories gain attributes), `orders`, `order_items`. The two columns to keep denormalized, with reasons that differ:

`order_items.unit_price` — not really denormalization, as established: it is the agreed price, a distinct fact, and keeping it is *required* for correctness rather than permitted for speed. `orders.total_amount` — genuine denormalization, and justified because the total is displayed on every order list, in every confirmation email and in every financial report, while order lines are written once and then never change. Add the trigger that maintains it and the nightly reconciliation query; without both, do not add the column.

The column to *not* keep: `product_name` on the line item. Tempting, but a product rename should propagate to the catalogue view of past orders in most businesses — and where it should not (invoices), you want a full snapshot of the line description, not one duplicated column.

2 • The drifting `like_count`. Likely causes, in order of frequency. **(i)** The count is updated in application code outside the transaction that inserts the like, so a crash between the two loses the increment. **(ii)** Read-modify-write in the application — `SELECT count`, add one, `UPDATE` — which loses updates under concurrency (Part 19's lost-update anomaly). **(iii)** Deletes handled in one path but not another; an admin bulk-delete of spam likes bypasses the decrement. **(iv)** Retries double-counting after a network timeout. The correct mechanism:

```
CREATE FUNCTION bump_like_count() RETURNS trigger AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        UPDATE posts SET like_count = like_count + 1 WHERE post_id = NEW.post_id;
    ELSIF TG_OP = 'DELETE' THEN
        UPDATE posts SET like_count = like_count - 1 WHERE post_id = OLD.post_id;
    END IF;
    RETURN NULL;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER likes_count_sync
AFTER INSERT OR DELETE ON likes
FOR EACH ROW EXECUTE FUNCTION bump_like_count();
```

Two properties make this correct where application code was not: it runs inside the same transaction as the like, so it cannot be half-applied; and `like_count = like_count + 1` is a single atomic statement, not a read-modify-write, so concurrent likes cannot lose one another. Add `UNIQUE (post_id, user_id)` on `likes` to make double-submission impossible, and a nightly reconciliation to catch anything a manual data fix breaks. The trade-off to state: the trigger serialises writes on the `posts` row, so a viral post becomes a contention hot spot — at which point the answer is a separate counter table with periodic aggregation, not a bigger trigger.

4 · “We use NoSQL so normalization does not apply.” Normalization is not a SQL feature; it is a set of facts about redundancy that hold wherever data is stored. A document store lets you *choose* to denormalize — embedding a customer's address inside every order document — and that choice has exactly the update anomaly of §16.1: the customer moves, and you must find and rewrite every document, without a transaction to make it atomic or a constraint to tell you when you missed one. The engine has not removed the problem; it has removed the tools for managing it and handed the responsibility to your application. The honest framing: relational databases make normalization the default and denormalization deliberate; document databases invert that. Both are legitimate, and in both cases the engineer must know which facts are duplicated and how they are kept in step.

Constraints

A constraint is a business rule that cannot be broken by any client, script or mistake. This part is the complete reference, including the two PostgreSQL features — partial unique indexes and exclusion constraints — that turn rules people enforce in application code into rules the database guarantees.

IN THIS PART

17.1 The six constraints

17.2 CHECK in depth

17.3 Referential actions

17.4 Deferrable constraints

17.5 Partial unique indexes

17.6 Exclusion constraints

17.7 Naming and management

17.8 Rules constraints cannot express

17.1 The six, 17.2 CHECK in depth

Constraint	Guarantees	Creates an index?
NOT NULL	A value is always present	No
UNIQUE	No two rows share the value	Yes — implicitly
PRIMARY KEY	UNIQUE + NOT NULL ; the identifier	Yes
FOREIGN KEY	The referenced row exists	On the referenced side only — never the referencing side
CHECK	A boolean expression holds for the row	No
DEFAULT	(Not a constraint) fills an omitted column	No

COMMON MISTAKE · THE UNINDEXED FOREIGN KEY

PostgreSQL automatically indexes the *referenced* column (it is already a key) but **never** the referencing column. So `orders.customer_id` has no index unless you create one — and without it, every join from customers to orders scans, and every `DELETE` from `customers` scans `orders` to check the constraint. On a large child table a single parent delete can take minutes. **Index every foreign key you join on or cascade through.**

CHECK — WHAT IT CAN AND CANNOT DO

```

CREATE TABLE bookings (
    booking_id    bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    guest_email   text NOT NULL,
    check_in      date NOT NULL,
    check_out     date NOT NULL,
    adults        smallint NOT NULL DEFAULT 1,
    children      smallint NOT NULL DEFAULT 0,
    amount_minor  bigint NOT NULL,
    currency      char(3) NOT NULL,
    status        text NOT NULL,

    -- single column, a range
    CONSTRAINT bookings_adults_sane    CHECK (adults BETWEEN 1 AND 8),
    -- two columns in the same row: only a table-level CHECK can do this
    CONSTRAINT bookings_dates_ordered CHECK (check_out > check_in),
    -- a computed condition
    CONSTRAINT bookings_occupancy     CHECK (adults + children <= 10),
    -- a pattern
    CONSTRAINT bookings_currency_iso  CHECK (currency ~ '^[A-Z]{3}$'),
    -- an enumeration
    CONSTRAINT bookings_status_valid  CHECK (status IN ('held', 'confirmed', 'cancelled')),
    -- conditional: a cancelled booking must have a reason
    CONSTRAINT bookings_cancel_reason
        CHECK (status <> 'cancelled' OR cancel_reason IS NOT NULL)
);

```

THE THREE THINGS `CHECK` CANNOT DO

(1) Reference another table — no subqueries allowed. “The order total must equal its line items” is not a `CHECK`. **(2)** Reference another row of the same table — “only one row may have `is_primary = true` per customer” needs a partial unique index (§17.5). **(3)** Use a non-immutable function. `CHECK (created_at <= now())` is rejected, because a constraint must give the same answer forever — a row valid at insert time cannot become invalid tomorrow, or a restore from backup would fail.

NOTE · `CHECK` AND `NULL`

A `CHECK` passes when it evaluates to **true or unknown** — only an outright `false` rejects the row. So `CHECK (salary > 0)` happily accepts `salary IS NULL`. If the column must be present *and* positive, you need both `NOT NULL` and the `CHECK`. This is Volume II §5.5 arriving in the schema, and it is one of the most commonly missed interactions in DDL review.

17.3 Referential actions

What happens to child rows when the parent is deleted or its key updated. Choose deliberately for every foreign key — the default is not always what you want.

Action	On parent delete	Choose when
<code>NO ACTION</code> (<i>default</i>)	Error — checked at the <i>end</i> of the statement	The safe default; a deliberate decision is required
<code>RESTRICT</code>	Error — checked <i>immediately</i>	Same intent, but cannot be deferred
<code>CASCADE</code>	Children are deleted too	The child cannot exist without the parent — order lines, tokens
<code>SET NULL</code>	Child's FK becomes <code>NULL</code>	The link is optional — an employee's department
<code>SET DEFAULT</code>	Child's FK becomes its default	Rare; needs the default to reference a real row

```
CREATE TABLE order_items (  
    order_id bigint REFERENCES orders (order_id) ON DELETE CASCADE,  
    product_id bigint REFERENCES products (product_id) ON DELETE RESTRICT,  
    ...  
);
```

Two different actions on one table, and both are arguments. `CASCADE` on `order_id`: a line has no meaning without its order, so deleting the order should take the lines. `RESTRICT` on `product_id`: deleting a product that has been sold must fail, forcing whoever tried to think again — the historical record matters more than their tidying.

COMMON MISTAKE · CASCADE CHAINS

`ON DELETE CASCADE` is transitive. Deleting one customer can cascade to orders, then to order items, then to shipments, then to shipment events — thousands of rows, silently, in one statement, holding locks on all of them. Before adding `CASCADE`, trace the whole chain and ask whether you want that. For anything a user can trigger, prefer `RESTRICT` plus an explicit, auditable deletion routine — or soft deletes (Volume II §3.7).

INTERVIEW · INTERMEDIATE

Q. What is the difference between `NO ACTION` and `RESTRICT` ? — Both raise an error; the difference is *when*. `RESTRICT` checks immediately, `NO ACTION` at the end of the statement — which means `NO ACTION` permits a statement that temporarily violates the constraint and fixes it before finishing, and can be deferred to commit time if declared `DEFERRABLE` . `RESTRICT` cannot.

17.4 Deferrable constraints

Some correct operations pass through a temporarily invalid state. Swapping two rows' unique values, or inserting two rows that reference each other, cannot be done one statement at a time. Deferring the check to commit time solves it.

```
ALTER TABLE seats
  ADD CONSTRAINT seats_position_unique UNIQUE (row_label, position)
  DEFERRABLE INITIALLY IMMEDIATE;

BEGIN;
SET CONSTRAINTS seats_position_unique DEFERRED;
UPDATE seats SET position = 2 WHERE seat_id = 1;  -- transiently duplicates
UPDATE seats SET position = 1 WHERE seat_id = 2;  -- resolves it
COMMIT;                                           -- checked here: passes
```

Without deferral the first `UPDATE` fails, and the usual workaround — move one row to a fake value, swap, move back — is three statements and a lie in the data. Deferral also solves circular foreign keys, where two tables reference each other and neither row can be inserted first.

BEST PRACTICE

Declare `DEFERRABLE INITIALLY IMMEDIATE`. The constraint then behaves normally by default — catching errors as early as possible, which is what you want — while remaining available to defer in the specific transaction that needs it. `INITIALLY DEFERRED` delays *every* check to commit time, which makes errors harder to locate. Note that `PRIMARY KEY` and `UNIQUE` can be deferrable in PostgreSQL, but `NOT NULL` and `CHECK` cannot.

17.5 Partial unique indexes

Uniqueness among only *some* rows. This solves a whole family of rules that people otherwise enforce in application code and get wrong under concurrency.

```
-- One primary address per customer, but many non-primary ones
CREATE UNIQUE INDEX addresses_one_primary
  ON addresses (customer_id) WHERE is_primary;

-- Emails unique among live customers only; soft-deleted rows may repeat
CREATE UNIQUE INDEX customers_email_live
  ON customers (email) WHERE deleted_at IS NULL;

-- A copy can be on loan only once at a time
CREATE UNIQUE INDEX loans_one_active_per_copy
  ON loans (copy_id) WHERE returned_on IS NULL;

-- Only one draft invoice per order, unlimited issued ones
CREATE UNIQUE INDEX invoices_one_draft
  ON invoices (order_id) WHERE status = 'draft';
```

Each of these is a rule an application layer typically implements as “check, then insert” — which loses the race between two simultaneous requests. The index cannot lose that race, because uniqueness is enforced at the storage level.

DIALECT

Partial indexes are PostgreSQL and SQLite. **SQL Server** calls them filtered indexes — same feature, `CREATE UNIQUE INDEX ... WHERE ...`. **Oracle** has no direct equivalent; the idiom is a function-based unique index returning `NULL` for rows to exclude, since `NULL`s do not conflict. **MySQL** has neither, so the rule must live in application code or a trigger — a genuine reason to prefer PostgreSQL for schemas with conditional uniqueness.

17.6 Exclusion constraints

The generalisation of `UNIQUE`: instead of “no two rows are equal”, it says “no two rows relate in this way”. Its killer application is **non-overlapping time ranges** — the rule behind every booking, scheduling and validity-period system.

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE room_bookings (
    booking_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    room_id    bigint NOT NULL REFERENCES rooms (room_id),
    stay       daterange NOT NULL,
    status     text NOT NULL DEFAULT 'confirmed',
    CONSTRAINT room_bookings_no_overlap
        EXCLUDE USING gist (room_id WITH =, stay WITH &&)
        WHERE (status <> 'cancelled')
);

INSERT INTO room_bookings (room_id, stay) VALUES (1, '[2024-06-03,2024-06-07)');
INSERT INTO room_bookings (room_id, stay) VALUES (1, '[2024-06-05,2024-06-09)');
-- ERROR:  conflicting key value violates exclusion constraint
--         "room_bookings_no_overlap"
```

Read the constraint as a sentence: no two rows may have `room_id` equal *and* `stay` overlapping. The `&&` operator is range overlap; `'[]'` bounds make the end date exclusive, so a checkout on the 7th leaves the room free from the 7th — the correct hotel convention, and the reason a range type beats two date columns.

BEST PRACTICE

Any time a requirement contains the words “cannot overlap”, “only one at a time”, or “no double-booking”, reach for an exclusion constraint. Application-level checking cannot be made correct without explicit locking, because two concurrent requests both read “no conflict” before either writes. This is the single highest-value constraint feature most engineers do not know exists.

INTERVIEW · EXPERT

Q. How do you prevent double-booking a room? — “An exclusion constraint over the room id and a range type, excluded on overlap. Application-level checks lose the race between two simultaneous bookings, so the guarantee has to be in the database.” **Common wrong answer:** `UNIQUE (room_id, check_in)`, which only catches bookings that start on the same day and lets a 3rd–7th and a 5th–9th both through. Naming that specific failure is what demonstrates you have thought it through.

17.7 Naming and management, 17.8 the limits

```
-- Add with a name
ALTER TABLE orders ADD CONSTRAINT orders_amount_positive CHECK (total_amount >= 0);

-- Add cheaply on a large table: validate afterwards, without blocking writes
ALTER TABLE orders ADD CONSTRAINT orders_amount_positive
    CHECK (total_amount >= 0) NOT VALID;
ALTER TABLE orders VALIDATE CONSTRAINT orders_amount_positive;

-- Inspect
SELECT conname, pg_get_constraintdef(oid) FROM pg_constraint
WHERE conrelid = 'orders'::regclass;

ALTER TABLE orders DROP CONSTRAINT orders_amount_positive;
```

Explicit names matter more than they look. Generated names (`orders_total_amount_check`) appear in error messages your users may see, in migration scripts colleagues must read, and in the `ON CONFLICT` clauses that reference them. A name like `orders_amount_positive` makes an incident five minutes shorter.

RULES A CONSTRAINT CANNOT EXPRESS — AND WHERE THEY GO INSTEAD

Rule	Enforce with
"An order must have at least one line"	A transaction that inserts both, plus a deferred check or a trigger at commit
"The order total must equal its lines"	A trigger maintaining it (Part 25) plus a reconciliation query
"No more than 3 active subscriptions per customer"	A trigger, or a counter column with a <code>CHECK</code> maintained transactionally
"A manager must be in the same department"	A <code>CHECK</code> cannot see another row — trigger, or denormalize the department onto the row
"Only one row may be the current record"	A partial unique index — this one <i>is</i> expressible (§17.5)
"Stays must not overlap"	An exclusion constraint — also expressible (§17.6)

BEST PRACTICE

Write down every business rule your schema *cannot* enforce, and beside each one name the mechanism that does — a trigger, a transaction, a nightly check, or an accepted risk. That list belongs in the repository next to the migrations. Rules that are enforced nowhere and written down nowhere are the ones that turn out to have been violated for two years.

Part 17 Review

Quick Check ● BEGINNER

- 1 Which constraints create an index implicitly, and which side of a foreign key is indexed?
- 2 Does `CHECK (salary > 0)` reject a `NULL` salary? Why?
- 3 Name the three things a `CHECK` cannot do.
- 4 Difference between `NO ACTION` and `RESTRICT` ?
- 5 Why is `ON DELETE CASCADE` risky?
- 6 What problem do deferrable constraints solve? Give two examples.
- 7 Why declare `DEFERRABLE INITIALLY IMMEDIATE` rather than `INITIALLY DEFERRED` ?
- 8 Write the partial unique index for "one primary address per customer".
- 9 Why can an application-level overlap check never be correct?
- 10 Why does `CHECK (created_at <= now())` fail to be created?

Practice ● EASY

- 1 Add named constraints to the practice schema: salary cap, order date not in the future, quantity limit.
- 2 Add the missing indexes on all three referencing foreign keys and confirm with `\d`.
- 3 Add a `CHECK` two ways — directly, and `NOT VALID` then validated — and explain when each is right.
- 4 Demonstrate the `NULL`-passes-`CHECK` behaviour, then fix it.
- 5 Try to delete a product that appears in an order. Read the error and name the constraint.
- 6 Delete an order and observe what happens to its items. Which clause decided that?
- 7 Add soft deletes to `customers` with a partial unique index on email among live rows. Prove a deleted customer's email can be reused.
- 8 Build the `room_bookings` table and prove the exclusion constraint rejects an overlap.
- 9 Make a unique constraint deferrable and swap two rows' values inside one transaction.
- 10 List every constraint on `orders` from `pg_constraint` with its definition.

Challenge ● INTERMEDIATE

- 1 Enforce "an employee's manager must be in the same department" three ways, and argue for one.
- 2 Design the constraints for a seat-reservation system so that no seat can be sold twice for one showing, correct under concurrency.

- 3 Add a `NOT NULL` column to a 200-million-row table with no downtime. Give every statement in order.
- 4 List every business rule in the practice schema that no constraint enforces, and name the mechanism for each.

Part 17 Solutions

Quick Check

- 1 `UNIQUE` and `PRIMARY KEY`. A foreign key indexes only the referenced side, which is already a key — never the referencing column.
- 2 No. A `CHECK` passes on true or unknown, and `NULL > 0` is unknown. Add `NOT NULL` as well.
- 3 Reference another table, reference another row of the same table, or use a non-immutable function.
- 4 Both error; `RESTRICT` checks immediately, `NO ACTION` at end of statement and can be deferred.
- 5 It is transitive — one delete can silently remove thousands of rows across several tables in one locked statement.
- 6 Operations that pass through a transiently invalid state: swapping two unique values, and inserting mutually-referencing rows.
- 7 So the constraint still fails fast by default, while remaining deferrable in the one transaction that needs it. `INITIALLY DEFERRED` pushes all errors to commit, making them harder to locate.
- 8 `CREATE UNIQUE INDEX ON addresses (customer_id) WHERE is_primary;`
- 9 Two concurrent requests both read “no conflict” before either writes. Only the database, holding the write, can decide.
- 10 `now()` is not immutable. A constraint must give the same verdict forever, or restoring a backup would reject rows that were valid when written.

Practice — selected

2 · The missing foreign-key indexes.

```
CREATE INDEX idx_employees_department ON employees (department_id);
CREATE INDEX idx_employees_manager   ON employees (manager_id);
CREATE INDEX idx_orders_customer     ON orders (customer_id);
CREATE INDEX idx_order_items_product ON order_items (product_id);
```

Note what is *not* needed: `order_items.order_id` is the leading column of the composite primary key `(order_id, product_id)`, so that index already serves joins on `order_id`. `product_id` is the *second* column and cannot be used alone — which is why it needs its own index. Column order in a composite index is the whole subject of Part 20.

7 · Soft deletes with reusable emails.

```

ALTER TABLE customers ADD COLUMN deleted_at timestamptz;
ALTER TABLE customers DROP CONSTRAINT customers_email_key; -- the blanket UNIQUE
CREATE UNIQUE INDEX customers_email_live
    ON customers (email) WHERE deleted_at IS NULL;

UPDATE customers SET deleted_at = now() WHERE customer_id = 5;
INSERT INTO customers (customer_id, full_name, email, city)
VALUES (7, 'Priya Anand (new)', 'priya@example.com', 'Hyderabad'); -- succeeds

```

Dropping the blanket `UNIQUE` is the step people forget, and without it the partial index changes nothing. The trade-off to be aware of: two *deleted* rows may now share an email, which is correct here but means any query joining on email must filter to live rows.

9 · The deferred swap.

```

CREATE TABLE seats (seat_id int PRIMARY KEY, row_label text, position int);
ALTER TABLE seats ADD CONSTRAINT seats_pos_unique
    UNIQUE (row_label, position) DEFERRABLE INITIALLY IMMEDIATE;
INSERT INTO seats VALUES (1, 'A', 1), (2, 'A', 2);

BEGIN;
SET CONSTRAINTS seats_pos_unique DEFERRED;
UPDATE seats SET position = 2 WHERE seat_id = 1;
UPDATE seats SET position = 1 WHERE seat_id = 2;
COMMIT;

```

Run the same two updates without the `SET CONSTRAINTS` line and the first fails immediately with a unique violation. The deferral is what makes a two-statement swap possible at all — the alternative is a three-statement dance through a temporary sentinel value.

Challenge — worked

1 · Manager in the same department. Three mechanisms, with real trade-offs. **(a) A trigger** on insert and update of `employees` that looks up the manager's department and raises an exception on mismatch. Correct, but the rule lives in procedural code, fires on every write, and must also fire when the *manager's* department changes — which is a second trigger on the same table and easy to forget. **(b) Denormalize and use a composite foreign key:**

```

ALTER TABLE employees ADD CONSTRAINT employees_manager_same_dept
    FOREIGN KEY (manager_id, department_id)
    REFERENCES employees (employee_id, department_id);
-- requires: ALTER TABLE employees ADD UNIQUE (employee_id, department_id);

```

This is the elegant answer: the constraint is declarative, checked by the engine, and impossible to race. Its cost is the redundant `UNIQUE (employee_id, department_id)` — harmless, since `employee_id` is already unique — and the fact that moving a manager between departments now correctly fails until their reports

are moved too, which is either exactly right or an operational annoyance depending on the business. **(c) A nightly check** that reports violations without preventing them; the right choice only when the rule is advisory. **Ship (b).** A declarative constraint that the database enforces beats procedural code every time it is expressible, and this is a genuinely under-used technique — most engineers do not know a composite foreign key can encode a same-group rule.

3 • **NOT NULL column on 200 million rows.** Every statement, in order, with why:

```
-- 1. Add nullable. Instant: catalog-only change, brief lock.
ALTER TABLE events ADD COLUMN source text;

-- 2. Backfill in batches. No long transaction, no lock queue, vacuum can keep up.
-- Repeat until 0 rows affected.
UPDATE events SET source = 'legacy'
WHERE event_id IN (SELECT event_id FROM events
                   WHERE source IS NULL ORDER BY event_id LIMIT 10000);

-- 3. Set the default for new rows (instant on PG 11+).
ALTER TABLE events ALTER COLUMN source SET DEFAULT 'unknown';

-- 4. Add a NOT VALID check: new and updated rows are enforced from now on,
-- with only a brief lock and no scan.
ALTER TABLE events ADD CONSTRAINT events_source_not_null
CHECK (source IS NOT NULL) NOT VALID;

-- 5. Validate: a full scan that does NOT block reads or writes.
ALTER TABLE events VALIDATE CONSTRAINT events_source_not_null;

-- 6. PG 12+: SET NOT NULL is now cheap, because the validated check proves it.
ALTER TABLE events ALTER COLUMN source SET NOT NULL;
ALTER TABLE events DROP CONSTRAINT events_source_not_null;
```

The wrong answer, and the one most often given: `ALTER TABLE ... ADD COLUMN source text NOT NULL DEFAULT 'legacy'` in a single statement. On PostgreSQL 11+ with a constant default this happens to be fast — but on PostgreSQL 10 or earlier it rewrites the entire table under an exclusive lock, and on other engines it still does. Knowing *why* the fast path exists, and that it depends on the default being constant, is the difference between a memorised recipe and understanding.

4 • **Unenforced rules in the practice schema.** Five, each with its mechanism. **(i)** “An order has at least one line item” — not expressible; enforce by inserting both in one transaction, and add a deferred constraint trigger if it must be guaranteed. **(ii)** “`orders.total_amount` equals the sum of its lines” — a trigger on `order_items` plus the nightly reconciliation query of §16.7. Order 1003 proves it is currently unenforced. **(iii)** “An employee's `hire_date` is not in the future” — expressible as a `CHECK` only against a literal, since `now()` is not immutable; in practice a trigger, or accept it. **(iv)** “A manager must not be their own subordinate” (no cycles) — needs a trigger doing a recursive check, or a redesign. **(v)**

"`order_items.unit_price` should be within some sane distance of `products.price`" — cross-table, so a trigger or a data-quality report. Writing this list out is itself the deliverable: a schema is not documented until the rules it *cannot* keep are written beside the ones it can.

Transactions

A transaction turns several statements into one indivisible unit: all of them happen, or none does. It is the mechanism that makes a database trustworthy, and the one thing an application cannot rebuild for itself.

IN THIS PART

18.1 BEGIN, COMMIT, ROLLBACK

18.2 ACID

18.3 SAVEPOINT

18.4 Transaction scope in applications

18.5 Locking basics

18.6 Retry logic · Review

18.1 BEGIN, COMMIT, ROLLBACK

```
BEGIN;

UPDATE accounts SET balance = balance - 5000 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 5000 WHERE account_id = 2;

COMMIT;  -- or ROLLBACK; to undo everything above
```

The canonical example, and worth stating precisely why it needs a transaction: between the two statements the money exists nowhere. A crash, a network failure, or an error in the second statement would destroy 5,000 rupees. The transaction makes that intermediate state invisible to everyone else and impossible to leave behind.

NOTE · AUTOCOMMIT

Without an explicit `BEGIN`, every statement is its own transaction — this is **autocommit**, and it is the default in `psql` and in nearly every driver. So a single `UPDATE` affecting a million rows is already atomic: it fully succeeds or fully fails. You need `BEGIN` only when *several* statements must succeed or fail together.

WHAT HAPPENS ON ERROR

```
BEGIN;
INSERT INTO departments (department_id, department_name) VALUES (1, 'Duplicate');
-- ERROR:  duplicate key value violates unique constraint "departments_pkey"

SELECT * FROM departments;
-- ERROR:  current transaction is aborted, commands ignored until end of
--         transaction block

ROLLBACK;  -- the only way out
```

This is PostgreSQL-specific and important: **after any error, the whole transaction is poisoned**. Every subsequent statement is refused until you `ROLLBACK` (or roll back to a savepoint). MySQL and Oracle are more permissive — a failed statement leaves the transaction usable — which means code written against MySQL and ported to PostgreSQL breaks in exactly this place.

BEST PRACTICE

Wrap anything destructive, verify, then decide. The habit from Volume II §3.6 formalised:

```
BEGIN;  
DELETE FROM orders WHERE order_date < '2020-01-01';  
-- DELETE 1483 ← is that the number you expected?  
-- SELECT a few rows to confirm the survivors look right  
COMMIT; -- or ROLLBACK;
```

The cost is a few seconds of held locks. The benefit is that a wrong `WHERE` clause becomes a non-event.

18.2 ACID

Four guarantees. Every interview asks for them, and the answer that scores is the one that says *what breaks without each*.

Property	Guarantee	Without it
Atomicity	All statements in the transaction take effect, or none do	The transfer debits one account and never credits the other
Consistency	A transaction moves the database from one valid state to another — every constraint holds at commit	A committed order references a customer who does not exist
Isolation	Concurrent transactions do not see each other's incomplete work	A report sums a balance mid-transfer and the books do not balance (Part 19)
Durability	Once committed, the data survives crash and power loss	A confirmed payment disappears when the server reboots

HOW EACH IS ACTUALLY IMPLEMENTED

- **Atomicity** — via the write-ahead log and MVCC. PostgreSQL never overwrites a row in place; it writes a new version stamped with the transaction id. A rollback simply means that version is never made visible.
- **Consistency** — by checking constraints, and by refusing the commit if any fail. Note that this "C" is the weakest of the four and is largely a consequence of the other three plus your constraint definitions; it is *not* the same "consistency" as in the CAP theorem.
- **Isolation** — via MVCC snapshots and locks. Each transaction sees a consistent view of the database as of a point in time. Which point, and how strictly, is the isolation level.
- **Durability** — the WAL is flushed to disk before the commit is acknowledged. This is why `synchronous_commit = off` makes writes faster and makes durability probabilistic: a crash can lose the last fraction of a second of committed transactions.

INTERVIEW · BEGINNER

Q. Explain ACID. — Give each letter with a one-line failure it prevents, using a bank transfer as the running example; that is far more convincing than four definitions. Then add the detail that shows depth: "In PostgreSQL these are implemented by MVCC and the write-ahead log — the log is flushed before a commit is acknowledged, which is exactly what durability costs."

18.3 SAVEPOINT

A partial rollback point inside a transaction. It lets one part fail without discarding the rest.

```
BEGIN;

INSERT INTO customers (customer_id, full_name, email, city)
VALUES (10, 'Nikhil Rao', 'nikhil@example.com', 'Chennai');

SAVEPOINT before_optional;

INSERT INTO customers (customer_id, full_name, email, city)
VALUES (1, 'Duplicate', 'dup@example.com', 'Pune');      -- fails

ROLLBACK TO SAVEPOINT before_optional;  -- undo just this, transaction lives on

INSERT INTO orders (order_id, customer_id, order_date, status, total_amount)
VALUES (2001, 10, current_date, 'pending', 4500.00);

COMMIT;  -- Nikhil and his order are saved; the duplicate never happened
```

Savepoints are also how a poisoned PostgreSQL transaction is rescued: rolling back to a savepoint clears the error state and the transaction becomes usable again. Every ORM's "nested transaction" is implemented this way — there is no such thing as a real nested transaction in PostgreSQL, only savepoints.

COMMON MISTAKE

Creating a savepoint per row in a large loop. Each one is cheap individually but they accumulate subtransaction state, and past roughly 64 active subtransactions PostgreSQL spills to disk and performance collapses — a well-known cliff. For bulk loads, batch the work and retry whole batches rather than wrapping every row in a savepoint.

18.4 Transaction scope in applications

The most consequential transaction decisions are made in application code, not in SQL. Three rules, each learned expensively by somebody.

Rule	Why
Keep transactions short	An open transaction holds locks and pins a snapshot, which prevents <code>VACUUM</code> from cleaning dead rows <i>anywhere</i> in the database. One long-running transaction can bloat every table.
Never do I/O inside one	An HTTP call, an email send, or a file upload inside a transaction means locks are held for the duration of somebody else's network. A slow third party becomes your database outage.
One transaction per unit of business work	Not per statement (loses atomicity), not per request (too broad). "Place the order" is a transaction; "render the dashboard" is not.

```
-- WRONG: the external call is inside the transaction
BEGIN;
  INSERT INTO orders ...;
  -- call the payment gateway here: 800 ms of held locks, and if it
  -- times out you do not know whether the customer was charged
  UPDATE orders SET status = 'paid' ...;
COMMIT;

-- RIGHT: three short transactions around the external call
BEGIN; INSERT INTO orders (... , status) VALUES (... , 'pending') RETURNING order_id; COMMIT;
-- call the gateway, outside any transaction, with an idempotency key
BEGIN; UPDATE orders SET status = 'paid', gateway_ref = $1 WHERE order_id = $2; COMMIT;
```

BEST PRACTICE

The pattern above is the **outbox** or saga shape, and the `'pending'` state is the point: a durable record that the work started, so a crash mid-flight leaves something to reconcile rather than nothing. Combined with a `UNIQUE` constraint on the gateway reference (Volume IV §15.4), a duplicate webhook cannot create a second payment.

18.5 Locking basics

Locks are how isolation is enforced when transactions actually collide. You rarely take them by hand, but you must know the three you might.

```
-- Read a row intending to update it: blocks other writers of that row
SELECT * FROM accounts WHERE account_id = 1 FOR UPDATE;

-- Same, but skip rows someone else has locked: the job-queue idiom
SELECT * FROM jobs WHERE status = 'queued'
ORDER BY created_at LIMIT 1
FOR UPDATE SKIP LOCKED;

-- Fail immediately rather than wait
SELECT * FROM accounts WHERE account_id = 1 FOR UPDATE NOWAIT;
```

`FOR UPDATE SKIP LOCKED` deserves highlighting: it makes a plain table into a correct work queue for many concurrent workers, with no polling collisions and no external queue system. Each worker takes a different job because locked rows are invisible to the others.

Lock	Effect
<code>FOR UPDATE</code>	Exclusive on the rows read; other <code>FOR UPDATE</code> and writes wait
<code>FOR NO KEY UPDATE</code>	Weaker; allows concurrent foreign-key checks
<code>FOR SHARE</code>	Others may read-lock, nobody may write
<code>LOCK TABLE ... IN ACCESS EXCLUSIVE MODE</code>	Whole table. Almost never correct in application code

COMMON MISTAKE

Plain readers are **not blocked** by writers in PostgreSQL — MVCC means a reader sees the previous row version instead of waiting. This is why "the database is locked up" is usually wrong as a diagnosis: readers are almost never the victims. The real contention is writer-versus-writer on the same row, or a DDL statement waiting for an `ACCESS EXCLUSIVE` lock behind a long-running query — and then *everything* queues behind the DDL, which is the actual outage.

18.6 Retry logic

Under `REPEATABLE READ` and `SERIALIZABLE` (Part 19), PostgreSQL will sometimes abort a transaction that cannot be serialised. This is not a bug and not something to configure away — **it is the mechanism**, and the application's job is to retry.

```
-- Serialization failure: SQLSTATE 40001
-- ERROR:  could not serialize access due to concurrent update

-- Deadlock detected: SQLSTATE 40P01
-- ERROR:  deadlock detected
```

```
# The retry wrapper every application using SERIALIZABLE needs
for attempt in range(1, 6):
    try:
        with conn.transaction():           # BEGIN ... COMMIT
            place_order(conn, cart)
        break
    except SerializationFailure:           # 40001 or 40P01
        if attempt == 5:
            raise
        sleep(random.uniform(0, 0.05 * 2 ** attempt)) # exponential + jitter
```

BEST PRACTICE

Three requirements for correct retries. **Retry the whole transaction**, not the failed statement — the snapshot is gone, so the earlier reads must be redone. **Add jitter** to the backoff, or two competing transactions retry in lockstep and collide again. **Cap the attempts** and surface a real error, because an unbounded retry loop turns a contention problem into an outage. And retry only 40001/40P01: retrying a constraint violation or a syntax error will never succeed.

INTERVIEW · ADVANCED

Q. Your application intermittently fails with "could not serialize access". What is happening and what do you do? — Two transactions under `REPEATABLE READ` or `SERIALIZABLE` touched overlapping data in a way the engine cannot order, so it aborted one to preserve correctness. The fix is a retry loop with exponential backoff and jitter around the whole transaction, not a lower isolation level. **Interview tip:** add that you would also look at *why* contention is high — a hot row, an over-broad transaction, or work that could be done outside it — because retries treat the symptom.

Transaction Isolation

Isolation is the dial between speed and correctness. This part demonstrates each anomaly as two sessions side by side — you cannot understand these by reading, so open two terminals now.

IN THIS PART

19.1 The four levels

19.2 Dirty read

19.3 Non-repeatable read

19.4 Phantom read

19.5 Lost update

19.6 Write skew

19.7 Deadlocks

19.8 Choosing a level · Review

19.1 The four levels

Level	Dirty read	Non-repeat.	Phantom	Write skew
Read Uncommitted	Possible*	Possible	Possible	Possible
Read Committed (<i>PG default</i>)	No	Possible	Possible	Possible
Repeatable Read	No	No	No*	Possible
Serializable	No	No	No	No

Two asterisks, both PostgreSQL-specific and both worth knowing. **PostgreSQL has no true Read Uncommitted** — asking for it silently gives you Read Committed, so dirty reads simply cannot happen. And **PostgreSQL's Repeatable Read is snapshot isolation**, which is stronger than the standard requires: it prevents phantom reads too, where the SQL standard permits them.

DIALECT · DEFAULTS DIFFER, AND IT MATTERS

PostgreSQL and **Oracle** default to Read Committed. **MySQL/InnoDB** defaults to **Repeatable Read** — so the same application behaves differently on the two engines with no code change. **SQL Server** defaults to Read Committed with locking reads, which means readers *can* block writers unless `READ_COMMITTED_SNAPSHOT` is on. Never assume the default; check it.

```
SHOW default_transaction_isolation;           -- read committed

BEGIN ISOLATION LEVEL REPEATABLE READ;
-- or
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

HOW TO RUN THE DEMONSTRATIONS

Open two psql sessions on `sqlmastery`, side by side. In each demonstration, **S1** and **S2** are the two sessions and the steps are numbered in the order you must execute them, alternating between windows. Run them in that exact order — the whole point is what each session sees at each moment.

19.2 Dirty read, 19.3 non-repeatable read

DIRTY READ — READING UNCOMMITTED DATA

A dirty read means seeing a change that may never be committed. It is the one anomaly you cannot demonstrate in PostgreSQL, because the engine does not permit it at any level — MVCC gives readers the last committed version, so there is nothing dirty to read. On SQL Server with `READ UNCOMMITTED` (or the `NOLOCK` hint) it is real, and it means a report can include a transfer that was rolled back.

NON-REPEATABLE READ — THE SAME ROW, TWO DIFFERENT VALUES

FIGURE 19.3 — NON-REPEATABLE READ UNDER READ COMMITTED

S1	S2
1 BEGIN;	
2 SELECT salary FROM employees WHERE employee_id = 101; → 92000.00	
3	UPDATE employees SET salary = 99000 WHERE employee_id = 101;
4	COMMIT;
5 SELECT salary FROM employees WHERE employee_id = 101; → 99000.00 ← SAME ROW, DIFFERENT VALUE	
6 COMMIT;	

S1 read the same row twice inside one transaction and got two answers. Any calculation that reads a value, does something, then re-reads it is now built on sand.

Repeat the demonstration with `BEGIN ISOLATION LEVEL REPEATABLE READ;` in step 1 and step 5 returns 92000.00 — S1's snapshot was fixed when the transaction began, and S2's commit is invisible to it.

WHY THIS MATTERS IN PRACTICE

Read Committed takes a **new snapshot for every statement**. So a multi-statement report — total revenue, then revenue by region, then the top ten customers — can produce three mutually inconsistent numbers, and the regional figures will not add up to the total. If a report's internal consistency matters, run the whole report in one `REPEATABLE READ` transaction.

19.4 Phantom read

Not a changed row, but a changed *set* of rows: the same query returns a different number of rows because someone inserted or deleted.

FIGURE 19.4 — PHANTOM READ UNDER READ COMMITTED

S1	S2
1 BEGIN;	
2 SELECT count(*) FROM employees WHERE department_id = 1; → 3	
3	INSERT INTO employees (employee_id, full_name, department_id, salary, hire_date) VALUES (110, 'New Hire', 1, 70000, current_date);
4	COMMIT;
5 SELECT count(*) FROM employees WHERE department_id = 1; → 4 ← A ROW APPEARED	
6 COMMIT;	

Under **REPEATABLE READ**, step 5 still returns 3 — PostgreSQL's snapshot isolation prevents phantoms, which the SQL standard does not require. This is the asterisk from §19.1 and it is why PostgreSQL's Repeatable Read is genuinely stronger than MySQL's or the standard's.

19.5 Lost update

The most damaging everyday anomaly, and the one most often written into application code without anybody noticing. Two transactions read the same value, each computes a new one from what it read, and the second write silently discards the first.

FIGURE 19.5 — LOST UPDATE: TWO 1,000-RUPEE DEPOSITS, ONE LANDS

S1	S2
1 BEGIN;	BEGIN;
2 SELECT balance FROM accounts WHERE account_id = 1; → 5000	
3	SELECT balance FROM accounts WHERE account_id = 1; → 5000
4 -- app computes 5000 + 1000 UPDATE accounts SET balance = 6000 WHERE account_id = 1;	-- app computes 5000 + 1000
5 COMMIT;	
6	UPDATE accounts SET balance = 6000 WHERE account_id = 1;
7	COMMIT;

Two deposits of 1000 were made. The balance is 6000, not 7000.
Nothing errored. Nothing was logged. The money is simply gone.

THREE FIXES, IN ORDER OF PREFERENCE

```
-- (1) BEST: never read-modify-write. Let the database do the arithmetic.
UPDATE accounts SET balance = balance + 1000 WHERE account_id = 1;

-- (2) Pessimistic lock: read with intent, forcing the second session to wait.
BEGIN;
SELECT balance FROM accounts WHERE account_id = 1 FOR UPDATE;
UPDATE accounts SET balance = $1 WHERE account_id = 1;
COMMIT;

-- (3) Optimistic lock: a version column. The loser's UPDATE affects 0 rows.
UPDATE accounts SET balance = $1, version = version + 1
WHERE account_id = 1 AND version = $2;
-- if rowcount = 0, someone else won: re-read and retry
```

BEST PRACTICE

Fix (1) whenever the new value is a function of the old one — counters, balances, stock levels. It is atomic in a single statement, needs no locking, and cannot lose an update at any isolation level. Fix (3) is right when a human is editing a form over thirty seconds and you want to tell them "someone else changed this" rather than hold a lock. Fix (2) is right when you must read several rows, decide, then write. Reaching for a higher isolation level instead of any of these is the wrong instinct: under `REPEATABLE READ`, PostgreSQL would abort S2 with a serialization failure — correct, but you still need the retry loop of §18.6.

INTERVIEW · ADVANCED

Q. Two users each add an item to a stock count at the same time and one increment is lost. Why, and how do you fix it? — Lost update: both read the old value and both wrote a value computed from it. The fix is to make the write relative — `SET qty = qty + 1` — so the arithmetic happens inside the database atomically. **Common wrong answer:** "wrap it in a transaction" — a transaction alone does not help, because at Read Committed both transactions are perfectly valid; the problem is the read-modify-write in application code, not the absence of a transaction.

19.6 Write skew — why Serializable exists

The subtlest anomaly. Two transactions read an overlapping set, each makes a decision that was valid given what it read, and each writes a *different* row — so no write conflicts, no lock collides, and together they violate a rule neither broke alone.

FIGURE 19.6 — WRITE SKEW: THE ON-CALL RULE BREAKS

Rule: at least one doctor must remain on call.

Currently on call: Asha and Ravi. Both want to go off call.

S1 (Asha)	S2 (Ravi)
1 BEGIN;	BEGIN;
2 SELECT count(*) FROM shifts WHERE on_call; → 2	
3	SELECT count(*) FROM shifts WHERE on_call; → 2
4 -- 2 > 1, so it is safe for me to leave UPDATE shifts SET on_call = false WHERE doctor = 'Asha';	-- 2 > 1, so it is safe for me to leave
5 COMMIT;	
6	UPDATE shifts SET on_call = false WHERE doctor = 'Ravi';
7	COMMIT;

Nobody is on call. Each transaction was individually correct.

They wrote DIFFERENT rows, so no lock and no update conflict occurred.

This is why **REPEATABLE READ** is not enough and why **SERIALIZABLE** exists. PostgreSQL's Serializable Snapshot Isolation tracks read/write dependencies between transactions and aborts one when the interleaving could not have arisen from running them one after another:

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
SELECT count(*) FROM shifts WHERE on_call;
UPDATE shifts SET on_call = false WHERE doctor = 'Ravi';
COMMIT;
-- ERROR: could not serialize access due to read/write dependencies
--         among transactions
-- HINT: The transaction might succeed if retried.
```

BEST PRACTICE

Write skew is the answer to “why would I ever use Serializable?”. Any rule of the form **“at least one”, “at most N”, or “the total must not exceed”** — where the check reads rows other than the one being written — is vulnerable, and no amount of care in application code fixes it without either `SERIALIZABLE` or an explicit lock on a common row. If you cannot use Serializable, the workaround is to force a conflict deliberately: have every such transaction `SELECT ... FOR UPDATE` the same parent row (the ward, the account group, the tenant), so the second one waits.

19.7 Deadlocks

Two transactions each hold a lock the other needs. Neither can proceed, so the engine detects the cycle and kills one.

FIGURE 19.7 — A DEADLOCK IN FOUR STEPS

S1	S2
1 BEGIN;	BEGIN;
2 UPDATE accounts SET balance = balance - 100 WHERE account_id = 1; -- locks row 1	
3	UPDATE accounts SET balance = balance - 50 WHERE account_id = 2; -- locks row 2
4 UPDATE accounts SET balance = balance + 100 WHERE account_id = 2; -- WAITS for S2	
5	UPDATE accounts SET balance = balance + 50 WHERE account_id = 1; -- WAITS for S1

→ ERROR: deadlock detected
DETAIL: Process 1234 waits for ShareLock on transaction 5678;
blocked by process 5679.
One transaction is aborted; the other proceeds.

BEST PRACTICE · THE ONE RULE THAT PREVENTS MOST DEADLOCKS

Always acquire locks in a consistent order. In the figure, S1 locked account 1 then 2 while S2 locked 2 then 1. Had both sorted by `account_id` ascending, the second would simply have waited for the first and no cycle could form. Sort the ids before you write:

```
-- Deadlock-free transfer: touch the lower id first, always
BEGIN;
SELECT * FROM accounts WHERE account_id IN ($from, $to)
ORDER BY account_id FOR UPDATE;
UPDATE accounts SET balance = balance - $amt WHERE account_id = $from;
UPDATE accounts SET balance = balance + $amt WHERE account_id = $to;
COMMIT;
```

The other three sources, in frequency order: an index or foreign key causing lock acquisition you did not write (an unindexed FK makes a parent update lock child rows); long transactions widening the window for a cycle; and an ORM emitting statements in an order that varies with hash iteration. All three are found the same way — read the deadlock detail in the log, which names both statements.

```
-- Diagnose: what is blocking what, right now
SELECT blocked.pid          AS blocked_pid,
       blocked.query        AS blocked_query,
       blocking.pid         AS blocking_pid,
       blocking.query       AS blocking_query,
       blocked.wait_event_type, blocked.wait_event
FROM   pg_stat_activity blocked
JOIN   pg_stat_activity blocking
      ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))
WHERE  cardinality(pg_blocking_pids(blocked.pid)) > 0;

-- And make deadlocks visible in the log permanently:
-- log_lock_waits = on, deadlock_timeout = 1s
```

COMMON MISTAKE

Treating a deadlock as a bug to be eliminated entirely. Under concurrency they will occur; the requirement is that they are **rare and retried**, exactly like serialization failures (§18.6). A system with no retry logic treats a normal, recoverable event as a user-visible error.

19.8 Choosing a level

Use	For	Cost
Read Committed	The default, and right for most OLTP work: single-statement writes, relative updates, reads that tolerate a moving target	Non-repeatable reads, phantoms, write skew all possible
Repeatable Read	Multi-statement reports that must be internally consistent; a batch job reading a stable snapshot	Serialization failures on write conflicts — needs retries
Serializable	Invariants spanning several rows: “at least one on call”, “balance must not go negative across accounts”, booking limits	More aborts under contention; needs retries; some overhead tracking dependencies

THE DECISION PROCEDURE

Start at Read Committed. Then ask: **does any transaction read data to decide what to write, where the decision spans rows it does not itself modify?** If yes, you need **SERIALIZABLE**, an explicit lock on a common row, or a constraint that makes the rule impossible to break (an exclusion constraint, §17.6 — always the best option when it applies). If instead a transaction merely needs a stable view for reading, **REPEATABLE READ** is enough. And if the new value is a function of the old, use a relative update and no level change is needed at all.

INTERVIEW · EXPERT

Q. Which isolation level would you use, and why? — “Read Committed by default, because it is cheap and correct for the common case where writes are relative and reads tolerate change. I move to Serializable where an invariant spans rows — the classic case is write skew, like two doctors each going off call because each read that the other was still on. And where the rule can be expressed as a constraint instead, I prefer that: an exclusion constraint for non-overlapping bookings is stronger than any isolation level, because it cannot be forgotten by the next developer.” **Interview tip:** mention that Serializable requires application retry logic. Candidates who propose it without acknowledging the 40001 retries have not run it in production.

Parts 18–19 Review

Quick Check ● BEGINNER

- 1 What does autocommit mean, and when do you need an explicit `BEGIN` ?
- 2 What happens in PostgreSQL after a statement errors inside a transaction? How does MySQL differ?
- 3 Name each ACID property and one failure it prevents.
- 4 What is a savepoint, and how do ORMs use it?
- 5 Why must you never make an HTTP call inside a transaction?
- 6 Why does one long-running transaction bloat every table in the database?
- 7 What does `FOR UPDATE SKIP LOCKED` make possible?
- 8 Which anomaly cannot occur in PostgreSQL at any level, and why?
- 9 Why is PostgreSQL's Repeatable Read stronger than the standard requires?
- 10 Which isolation level does MySQL default to, and why does that matter?
- 11 Define lost update and give the best fix.
- 12 Define write skew and say why `REPEATABLE READ` does not prevent it.
- 13 What single rule prevents most deadlocks?
- 14 Which SQLSTATE codes should an application retry, and which must it not?

Practice ● EASY

Exercises 3–9 need two psql sessions. Create the helper table first.

- 1 Create `accounts(account_id, owner, balance)` with a `CHECK (balance >= 0)` and two rows of 5,000.
- 2 Write a transfer as a transaction. Then write one that violates the check and confirm the whole transfer rolls back.
- 3 Reproduce the non-repeatable read of §19.3 under Read Committed, then show it disappears under Repeatable Read.
- 4 Reproduce the phantom read of §19.4, then show PostgreSQL's Repeatable Read prevents it.
- 5 Reproduce the lost update of §19.5, then fix it three ways and verify each.
- 6 Build the `shifts` table and reproduce write skew. Then show `SERIALIZABLE` aborting one transaction.
- 7 Cause a deadlock deliberately, read the error detail, then rewrite both transactions so it cannot recur.
- 8 Use the `pg_blocking_pids` query to observe one session blocking another.

- 9 Build a job queue with `FOR UPDATE SKIP LOCKED` and show two sessions taking different jobs.
- 10 Use a savepoint to let one failing insert not abort a batch of five.
- 11 Show that a plain `SELECT` is not blocked by an uncommitted `UPDATE` on the same row, and explain what it returns.
- 12 Run a two-statement report under Read Committed while another session writes, and demonstrate that the two figures disagree. Then fix it.

Challenge ● ADVANCED

- 1 Design the “place an order and decrement stock” transaction so that stock can never go negative under any concurrency. Give three approaches and pick one.
- 2 Implement optimistic locking on `products.price` with a version column, including what the application does when the update affects zero rows.
- 3 A payment webhook may be delivered twice. Design the transaction and constraints so a double delivery cannot double-charge.
- 4 An application uses `SERIALIZABLE` everywhere and is now failing constantly under load. Diagnose and give three remedies in order of preference.
- 5 Enforce “a customer may have at most 3 active subscriptions” correctly under concurrency, without `SERIALIZABLE`.

Interview Drill ● EXPERT

- 1 Explain ACID with a bank transfer, then say how each property is implemented.
- 2 Walk through the four isolation levels and the anomaly each one stops.
- 3 Explain write skew and why it justifies Serializable.
- 4 How do you make a stock decrement safe? Defend your answer against two alternatives.
- 5 How would you diagnose intermittent deadlocks in production?

Parts 18–19 Solutions

Quick Check

- 1 Each statement is its own transaction. You need `BEGIN` only when several statements must succeed or fail together.
- 2 PostgreSQL aborts the transaction and refuses every further statement until `ROLLBACK` or a rollback to a savepoint. MySQL and Oracle leave the transaction usable, so ported code breaks here.
- 3 Atomicity — no half-transfer. Consistency — no order referencing a missing customer. Isolation — no report reading mid-transfer state. Durability — no committed payment lost to a reboot.
- 4 A partial rollback point inside a transaction. ORMs implement "nested transactions" as savepoints, since PostgreSQL has no true nesting.
- 5 Locks and the snapshot are held for the duration of someone else's network call, so a slow third party becomes your database problem.
- 6 It pins a snapshot, so `VACUUM` cannot remove any row version newer than that snapshot — anywhere, not just in the tables the transaction touched.
- 7 A correct multi-worker job queue on an ordinary table: locked rows are invisible to other workers, so each takes a different job with no collisions.
- 8 Dirty read. MVCC always serves the last committed row version, so there is no uncommitted data to read — `READ UNCOMMITTED` silently behaves as Read Committed.
- 9 It is implemented as snapshot isolation, which also prevents phantom reads; the standard permits them at that level.
- 10 Repeatable Read. The same application therefore behaves differently on MySQL and PostgreSQL with no code change — a real portability hazard.
- 11 Two transactions read a value, each computes a new one from it, and the second write discards the first. Best fix: a relative update, `SET x = x + n`.
- 12 Two transactions read an overlapping set, each writes a *different* row, and together they break an invariant. Repeatable Read does not prevent it because there is no write conflict to detect — the writes do not touch the same rows.
- 13 Acquire locks in a consistent order — typically sort the affected keys ascending before writing.
- 14 Retry 40001 (serialization failure) and 40P01 (deadlock). Never retry constraint violations, syntax errors or permission errors — they will fail identically forever.

Practice — selected

2 · The failing transfer.


```

CREATE TABLE accounts (
  account_id integer PRIMARY KEY,
  owner      text NOT NULL,
  balance    numeric(12,2) NOT NULL CONSTRAINT accounts_balance_non_negative
              CHECK (balance >= 0)
);
INSERT INTO accounts VALUES (1,'Asha',5000.00), (2,'Ravi',5000.00);

BEGIN;
UPDATE accounts SET balance = balance - 9000 WHERE account_id = 1;
-- ERROR: new row for relation "accounts" violates check constraint
--       "accounts_balance_non_negative"
UPDATE accounts SET balance = balance + 9000 WHERE account_id = 2;
-- ERROR: current transaction is aborted...
ROLLBACK;
SELECT * FROM accounts;  -- both still 5000.00

```

Atomicity in one screen: the constraint caught the debit, the credit never happened, and the rollback left both balances untouched. Note that the *second* error is the poisoned-transaction behaviour of §18.1 — in MySQL the credit would have succeeded, creating 9,000 rupees from nothing.

9 • The job queue.

```

CREATE TABLE jobs (
  job_id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  payload     jsonb NOT NULL,
  status      text NOT NULL DEFAULT 'queued',
  locked_at   timestamptz,
  created_at  timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX idx_jobs_queued ON jobs (created_at) WHERE status = 'queued';

INSERT INTO jobs (payload) SELECT jsonb_build_object('n', n)
FROM generate_series(1,5) n;

-- Each worker runs this. Two sessions get DIFFERENT jobs.
BEGIN;
SELECT job_id, payload FROM jobs
WHERE status = 'queued'
ORDER BY created_at
LIMIT 1
FOR    UPDATE SKIP LOCKED;

UPDATE jobs SET status = 'running', locked_at = now() WHERE job_id = $1;
COMMIT;

```

Three details make this production-grade. `SKIP LOCKED` is what prevents workers queueing behind each other — without it, ten workers serialise on the oldest row. The **partial index** on queued rows keeps the lookup fast even when the table holds millions of completed jobs. And `locked_at` lets a reaper reset jobs whose worker died: `UPDATE jobs SET status='queued' WHERE status='running' AND locked_at < now() - interval '5 minutes'`. That last part is what people forget, and it is why jobs mysteriously stop processing.

11 • Reader not blocked by writer. In S1: `BEGIN; UPDATE employees SET salary = 1 WHERE employee_id = 101;` and do not commit. In S2: `SELECT salary FROM employees WHERE employee_id = 101;` returns **92000.00** immediately — the last committed version. MVCC means S2 neither waits nor sees the uncommitted value. This is the single most important practical consequence of MVCC and the reason “readers don't block writers, writers don't block readers” is PostgreSQL's headline concurrency property. Contrast SQL Server in its default locking mode, where S2 would block until S1 commits.

12 • The inconsistent report. S1: `BEGIN; SELECT sum(total_amount) FROM orders;` → 75950. S2: `INSERT INTO orders ... VALUES (2002, 1, current_date, 'paid', 10000); COMMIT;`. Back in S1: `SELECT count(*), sum(total_amount) FROM orders;` → 8 rows, 85950 — the count and the earlier total now describe different databases. Fix by opening the report with `BEGIN ISOLATION LEVEL REPEATABLE READ;` so every statement reads the same snapshot. This is the everyday reason to reach for Repeatable Read, and it needs no retry logic because the report only reads.

Challenge — worked

1 • Stock that can never go negative. Three approaches, and the choice is instructive because the best one is the simplest.

```
-- (a) BEST: a relative update with a guard, plus a CHECK as the backstop.
--      Atomic in one statement. If rowcount = 0, there was not enough stock.
UPDATE products
SET   in_stock = in_stock - $qty
WHERE product_id = $id
      AND in_stock >= $qty;
-- rowcount 0 → reject the order, no transaction gymnastics needed

-- (b) Pessimistic: lock the row, decide, write.
BEGIN;
SELECT in_stock FROM products WHERE product_id = $id FOR UPDATE;
-- application checks
UPDATE products SET in_stock = in_stock - $qty WHERE product_id = $id;
COMMIT;

-- (c) SERIALIZABLE plus a retry loop.
```

Ship (a). The condition `AND in_stock >= $qty` is evaluated by the database at write time against the current row, so two concurrent orders for the last unit cannot both succeed — the second sees the decremented value and affects zero rows. It needs no explicit lock, no isolation change and no retry, and the `CHECK (in_stock >= 0)` remains as a backstop proving the invariant can never be violated even by a buggy

code path. Approach (b) is correct but holds a lock across an application round trip; (c) is correct but pays for dependency tracking and needs retry logic. **The general lesson: express the precondition as part of the write, and concurrency stops being your problem.**

3 • The double-delivered webhook. Two mechanisms, and you want both. First, a `UNIQUE` constraint on the provider's idempotency key so a second delivery cannot create a second payment:

```
ALTER TABLE payments ADD CONSTRAINT payments_gateway_ref_unique UNIQUE (gateway_ref);

-- The handler: idempotent by construction
INSERT INTO payments (order_id, gateway_ref, amount_minor, currency, status)
VALUES ($1, $2, $3, $4, 'settled')
ON CONFLICT (gateway_ref) DO NOTHING
RETURNING payment_id;
```

If `RETURNING` yields no row, this delivery was a duplicate and the handler returns 200 without doing anything — which is exactly what the provider needs to stop retrying. Second, the order-status update must be in the *same transaction* as the insert, and expressed conditionally so a replay is harmless: `UPDATE orders SET status='paid' WHERE order_id=$1 AND status='pending'`. The `AND status='pending'` makes it idempotent (Volume II §3.6). Note what makes this work at all: the guarantee lives in a constraint, not in a "have we seen this before?" lookup, which would lose the race between two simultaneous deliveries of the same webhook.

4 • `SERIALIZABLE` everywhere, failing under load. Diagnosis: the abort rate rises with contention, and Serializable aborts on read/write dependency cycles — so applying it to transactions that do not need it manufactures failures. Check `pg_stat_database.xact_rollback` against `xact_commit`, and log the 40001s with their statements to see which transactions actually conflict. Three remedies in order. **(i) Narrow the scope:** use Serializable only for the transactions with a genuine multi-row invariant, and Read Committed for everything else — most systems need it in two or three places, not everywhere. **(ii) Replace the invariant with a constraint:** an exclusion constraint or a guarded relative update (as in challenge 1) removes the need for Serializable entirely and cannot be forgotten later. **(iii) Reduce the conflict footprint:** shorten transactions, move reads that do not need to be in the snapshot outside it, and add indexes — Serializable tracks predicate locks, and a sequential scan takes a far broader predicate lock than an index scan, so a missing index directly increases false conflicts. That last point is the non-obvious one and it is frequently the whole answer.

5 • At most 3 active subscriptions, without `SERIALIZABLE`. The naive check — count, then insert — is write skew: two concurrent requests both count 2 and both insert. Two correct approaches without changing the isolation level. **(a) Serialise on the parent row:** `SELECT 1 FROM customers WHERE customer_id = $1 FOR UPDATE;` at the start of the transaction, so every subscription change for that customer takes the same lock and the second waits. Simple, correct, and the contention is scoped to one customer. **(b) Make it a uniqueness problem,** which is stronger because it needs no discipline from callers: add a `slot` `smallint` `CHECK (slot BETWEEN 1 AND 3)` column with `CREATE UNIQUE INDEX ON subscriptions (customer_id, slot) WHERE cancelled_at IS NULL;`. A fourth concurrent subscription must claim a slot, all three are taken, and the insert fails on the unique index — enforced by storage, unraceable, and impossible for a future

developer to bypass. (b) is the better engineering answer for the same reason the exclusion constraint beat application checking in §17.6: **turn the rule into something the database cannot allow, rather than something the code must remember to check.**

Where you are, and what comes next

You can now design a schema and defend it, and you understand what the database guarantees when many people write at once. That second half is what separates engineers who can build a feature from engineers who can be trusted with money. Confirm you can do these six things:

1. Run a design review from the §15.8 checklist and explain every item.
2. Normalize a flat table to 3NF, naming each dependency you removed.
3. State the four conditions that must hold before denormalizing.
4. Write an exclusion constraint for non-overlapping bookings, and say why a `UNIQUE` is not enough.
5. Explain lost update and write skew, and fix each without raising the isolation level.
6. Explain why acquiring locks in a consistent order prevents deadlocks.

NEXT

Volume V — Parts 20 to 26. Performance and the database's own programmability: B-tree internals and every index type, reading `EXPLAIN ANALYZE`, join algorithms and the planner's choices, advanced tuning (statistics, `VACUUM`, bloat, partitioning, materialized views, pooling, replicas), views, stored procedures and PL/pgSQL, triggers, and JSON/JSONB in depth.

Say the word and I will continue.



SQL MASTERY

Volume V — Performance & Programmability

Parts 20 to 26. Why queries are slow, how to find out rather than guess, and what the database can do for you when you move logic inside it.

-
- | | |
|----|---|
| 20 | Indexes — B-tree internals, every index type, composite order, selectivity, when they hurt |
| 21 | Query Performance — the planner, reading <code>EXPLAIN ANALYZE</code> , scans, join algorithms |
| 22 | Advanced Performance — statistics, <code>VACUUM</code> , bloat, N+1, partitioning, matviews, pooling, replicas |
| 23 | Views — updatable views, materialized views, refresh strategies, security |
| 24 | Functions & Procedures — PL/pgSQL, parameters, control flow, exceptions |
| 25 | Triggers — before/after, audit logging, maintained aggregates, pitfalls |
| 26 | JSON & JSONB — operators, paths, aggregation, GIN indexing, when to use it |
-

Part 21 needs a table large enough for the planner to behave realistically. §21.2 generates one — the six-row practice tables will always be sequentially scanned, and correctly so.

Indexes

An index is a separate data structure that lets the database find rows without reading the whole table. Everything about query performance begins here — and so does a whole category of write-performance problems, because every index is a second copy that must be maintained.

IN THIS PART

20.1 The analogy, then the mechanism

20.2 B-tree internals

20.3 Composite indexes and column order

20.4 Covering indexes and index-only scans

20.5 Partial and expression indexes

20.6 Selectivity and cardinality

20.7 The other index types

20.8 When indexes hurt · Review

20.1 The analogy, then the mechanism

Think of the index at the back of a textbook. To find every mention of “normalization” you do not read all 900 pages; you turn to the index, find the term in alphabetical order, and read off the page numbers. Two properties make that work: the index is **sorted**, so you can find a term without scanning, and it is **much smaller** than the book, so scanning it is cheap even when you must.

A database index is the same structure with the same two properties. It stores the indexed column's values in sorted order, each paired with a pointer to the row's physical location. Technically:

- The table itself — the **heap** — is unordered. Rows sit wherever there was space.
- The index holds `(value, tuple_id)` pairs in sorted order, in its own file.
- A lookup walks the index to find matching values, then fetches those rows from the heap. Two structures read, far fewer pages touched.

The trade-off follows immediately and is the whole subject of §20.8: every `INSERT`, `UPDATE` and `DELETE` must update every affected index as well as the heap. Indexes make reads faster and writes slower, and they consume storage. There is no such thing as a free index.

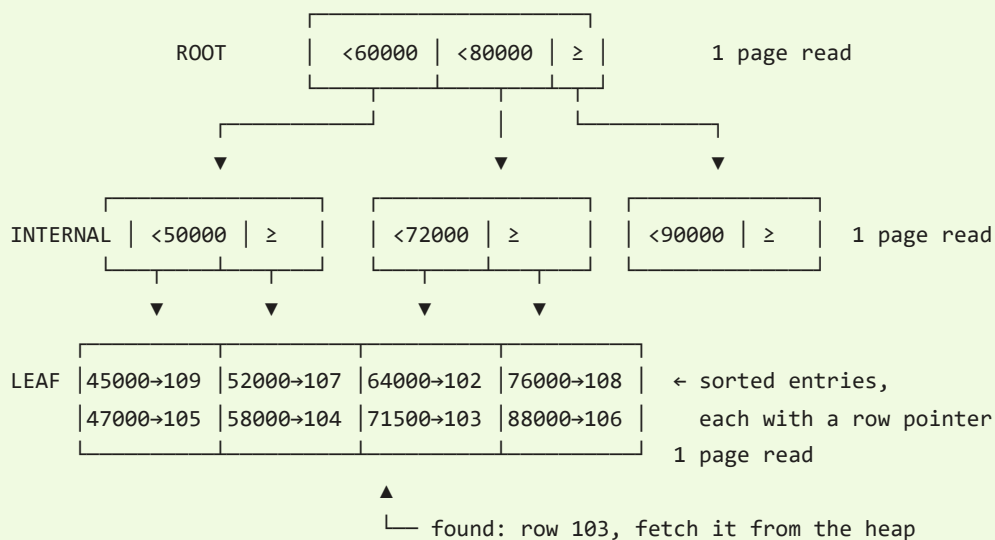
NOTE · WHY THE ANALOGY EVENTUALLY BREAKS

A book index has one entry per term. A database index has one entry per *row* — so a column where 90% of rows share a value produces an index that is nearly as large as the table and answers almost nothing. That is selectivity (§20.6), and it is why `CREATE INDEX ON orders (status)` often does nothing useful.

20.2 B-tree internals

The default index type, and the right choice for perhaps 95% of indexes. A B-tree is a balanced tree of fixed-size pages: one root, some internal levels, and leaves holding the actual entries in sorted order.

FIGURE 20.2 — FINDING `SALARY = 71500` IN THREE PAGE READS



Leaves are linked left-to-right, so a RANGE scan continues sideways without returning to the root.

Depth grows logarithmically. A B-tree over 100 million rows is typically 4–5 levels deep, so any single-value lookup costs about five page reads — and the upper levels are almost always already in memory.

Two structural facts explain most B-tree behaviour you will observe:

- **Sorted leaves** mean a B-tree serves equality (`=`), ranges (`<` `>` `BETWEEN`), prefix matching (`LIKE 'abc%'`), `IS NULL`, and `ORDER BY` — it can return rows already in order, letting the planner skip a sort entirely.
- **Linked leaves** mean a range scan is sequential once it has found its starting point, which is why `WHERE created_at >= x AND created_at < y` is so much cheaper than it looks.

What a B-tree cannot do: a leading wildcard (`LIKE '%abc'`), because the sort order gives no starting point; or a column wrapped in a function, because the index stores the raw values, not the transformed ones. Both are Volume II §6.1's sargability rule, now with its mechanical reason.

INTERVIEW · INTERMEDIATE

Q. How does an index make a query faster? — Answer with the structure, not the analogy alone: “It is a sorted, balanced tree stored separately from the table. A lookup descends about four or five levels instead of reading every page, and because the leaves are sorted and linked it also serves ranges and can satisfy `ORDER BY` without a sort. The cost is that every write must maintain it.” Mentioning the write cost unprompted is what distinguishes the answer.

20.3 Composite indexes and column order

An index on several columns is sorted by the first column, then by the second within equal firsts, and so on — like a phone book sorted by surname then forename. **Column order is the single most consequential decision in indexing**, and it follows directly from that sorting.

```
CREATE INDEX idx_orders_cust_date ON orders (customer_id, order_date);
```

Query	Can it use the index?
WHERE customer_id = 1	Yes — leading column
WHERE customer_id = 1 AND order_date > '2024-01-01'	Yes — ideal; both columns used
WHERE customer_id = 1 ORDER BY order_date	Yes — and no sort needed
WHERE order_date > '2024-01-01'	Poorly — the leading column is missing, so no seek is possible. PostgreSQL may still scan the whole index if that is cheaper than the heap, but it is not a lookup.
ORDER BY order_date	No — the index is not globally sorted by order_date

THE RULE · LEFTMOST PREFIX

A composite index serves any query that uses a **leftmost prefix** of its columns. An index on (a, b, c) helps a ; a, b ; and a, b, c — but not b alone, c alone, or b, c . So one index on (a, b, c) replaces separate indexes on (a) and (a, b) , and those separate indexes are pure waste.

HOW TO ORDER THE COLUMNS

1. **Equality columns first**, range columns last. Once the index reaches a range, it cannot seek on anything after it — the entries are only sorted *within* equal values of the preceding columns.
2. **Among equality columns, most selective first** — usually. The effect is smaller than people think, since a seek on either order lands in the same place, but it matters for index-scan efficiency and for partial matches.
3. **Then the ORDER BY columns**, in the query's direction, so the sort disappears from the plan.

```
-- Query: orders for one customer, one status, in date order
SELECT * FROM orders
WHERE customer_id = 1 AND status = 'delivered'
ORDER BY order_date DESC;

-- The index that serves it completely:
CREATE INDEX idx_orders_cust_status_date
    ON orders (customer_id, status, order_date DESC);
--          ^equality    ^equality  ^ordering
```

COMMON MISTAKE · ONE INDEX PER COLUMN

Creating `(customer_id)`, `(status)` and `(order_date)` separately does *not* serve the query above well. PostgreSQL can combine them with a bitmap scan, but that is far slower than one composite index and it cannot help the `ORDER BY` at all. **Index for your queries, not for your columns.**

20.4 Covering indexes and index-only scans

If every column a query needs is present in the index, the database never touches the heap. The plan shows `Index Only Scan`, and it is typically several times faster than an index scan plus heap fetches.

```
-- This query needs three columns
SELECT customer_id, order_date, total_amount
FROM   orders WHERE customer_id = 1;

-- (a) Include the payload in the key: it can also be used for ordering
CREATE INDEX idx_orders_covering
  ON orders (customer_id, order_date, total_amount);

-- (b) Better: INCLUDE columns are stored in the leaves but not in the key,
--      so the index is smaller and the extra columns need no sort order.
CREATE INDEX idx_orders_covering2
  ON orders (customer_id, order_date) INCLUDE (total_amount);
```

`INCLUDE` (PostgreSQL 11+) is the right tool when a column is needed for *output* but never for filtering or ordering. The distinction matters: key columns make the index bigger and must be maintained in sorted order; included columns are dead weight in the leaves and nothing more.

NOTE · WHY AN INDEX-ONLY SCAN SOMETIMES STILL VISITS THE HEAP

PostgreSQL's index entries do not record whether a row is visible to your transaction — MVCC visibility lives in the heap. So an index-only scan must consult the **visibility map**, and if a page is not marked all-visible, the heap is read after all. This is why an index-only scan gets faster after `VACUUM`: vacuuming is what sets those visibility-map bits. A table with constant churn may never achieve a true index-only scan.

20.5 Partial and expression indexes

Two features that make indexes dramatically smaller and more targeted. Both are heavily used in well-tuned PostgreSQL schemas and largely unknown to people who learned on MySQL.

```
-- PARTIAL: index only the rows you query. Tiny, and much faster to maintain.
CREATE INDEX idx_orders_pending ON orders (order_date)
    WHERE status = 'pending';

CREATE INDEX idx_users_active ON users (last_seen_at)
    WHERE deleted_at IS NULL;

-- EXPRESSION: index the transformed value, so a function call stays sargable
CREATE INDEX idx_customers_email_lower ON customers (lower(email));
CREATE INDEX idx_orders_month ON orders (date_trunc('month', order_date));
```

A partial index on pending orders in a table where 0.5% of rows are pending is roughly 1/200th the size of a full index. It fits in cache, it is quicker to scan, and — the part people miss — **it is not touched at all when a non-pending row is written**, so it costs almost nothing on the write path.

COMMON MISTAKE · THE EXPRESSION MUST MATCH EXACTLY

An expression index on `lower(email)` serves `WHERE lower(email) = $1` and nothing else. It does not serve `WHERE email = $1`, nor `WHERE upper(email) = $1`, nor `WHERE lower(trim(email)) = $1`. The planner matches the expression textually (after normalisation), so pick one canonical form per column and use it everywhere — or better, normalise the value on write and index the column plainly.

BEST PRACTICE

Whenever a query has a `WHERE` clause that is always present — a tenant id, `deleted_at IS NULL`, an active-status filter — consider making it a partial index predicate instead of a leading index column. You get a smaller index and a cheaper write path for the rows the index ignores.

20.6 Selectivity and cardinality

Cardinality is how many distinct values a column has. **Selectivity** is what fraction of the table a typical predicate on it returns. High cardinality means high selectivity means a useful index.

Column	Distinct values	Rows per value (1M table)	Index useful?
<code>order_id</code>	1,000,000	1	Yes — perfect
<code>email</code>	~1,000,000	1	Yes
<code>customer_id</code>	50,000	20	Yes
<code>status</code>	5	200,000	Rarely — unless partial
<code>is_deleted</code>	2	500,000	No — unless partial and skewed

The threshold in practice: if a predicate returns more than roughly 5–10% of the table, a sequential scan is usually *faster* than an index scan. Reading pages in order is far cheaper per row than jumping between random heap pages, so the planner correctly chooses to read everything. This is why `CREATE INDEX ON orders (status)` is so often followed by the discovery that it is never used.

```
-- Measure it before you index it
SELECT count(*)           AS rows,
       count(DISTINCT status) AS distinct_statuses,
       round(count(DISTINCT status)::numeric / count(*), 6) AS selectivity
FROM   orders;

-- What the planner itself believes, from its statistics
SELECT attname, n_distinct, most_common_vals, most_common_freqs
FROM   pg_stats WHERE tablename = 'orders';

-- Which indexes are actually being used
SELECT relname, indexrelname, idx_scan, pg_size_pretty(pg_relation_size(indexrelid))
FROM   pg_stat_user_indexes
ORDER BY idx_scan; -- zeros at the top are candidates for deletion
```

BEST PRACTICE · THE SKEW EXCEPTION

A low-cardinality column is still worth indexing when the values are **heavily skewed** and you query the rare one. If 0.1% of orders are `'failed'` and you query only those, a partial index on `WHERE status = 'failed'` is excellent — tiny, always used, nearly free to maintain. Selectivity is about the *predicate*, not the column.

20.7 The other index types

Type	Good at	Use for
B-tree	Equality, ranges, prefix, ordering	The default. Almost everything.
Hash	Equality only — no ranges, no ordering	Rarely worth it; B-tree is nearly as fast and far more capable
GIN	Values <i>inside</i> a composite column	<code>jsonb</code> containment, arrays, full-text search, trigram <code>LIKE '%x%'</code>
GiST	Geometric and range overlap	Exclusion constraints, ranges, PostGIS geometry, nearest-neighbour
BRIN	Very large, naturally ordered tables	Append-only time-series: a tiny index storing min/max per block range
SP-GiST	Non-balanced partitioned trees	IP prefixes, quadtree-like data. Specialist.

```
-- GIN for jsonb containment and for substring search
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX idx_events_payload ON events USING gin (payload jsonb_path_ops);
CREATE INDEX idx_products_name_trgm ON products USING gin (product_name gin_trgm_ops);
-- now this is fast:
SELECT * FROM products WHERE product_name ILIKE '%board%';

-- GiST for range overlap (the exclusion constraint of Volume IV §17.6)
CREATE INDEX idx_bookings_stay ON room_bookings USING gist (stay);

-- BRIN for an append-only log: often 1000x smaller than a B-tree
CREATE INDEX idx_events_time_brin ON events USING brin (occurred_at);
```

BEST PRACTICE

The trigram GIN index is the fix for the search-box problem of Volume II §5.7 — it makes `ILIKE '%term%'` genuinely fast, which no B-tree can. BRIN is the right answer for a billion-row events table where rows arrive in timestamp order: it indexes block ranges rather than rows, so it is minuscule and still prunes most of the table for a date filter. Both are cases where reaching for the default B-tree gives you a large index that does not help.

20.8 When indexes hurt

Indexes are not free, and over-indexing is as common a production problem as under-indexing.

Cost	Detail
Write amplification	An <code>INSERT</code> into a table with eight indexes performs nine writes. On an update, every index whose columns changed is rewritten — and in PostgreSQL, a non-HOT update rewrites <i>all</i> of them.
Storage	Indexes frequently exceed the size of the table they serve. That is memory not available for caching data.
Cache pressure	Unused indexes occupy shared buffers and compete with the pages you actually need.
Planning time	More candidate paths for the optimizer to cost, on every execution.
Maintenance	<code>VACUUM</code> must clean every index; a bulk load is much slower with indexes present.
Bloat	Heavily updated indexes fragment and grow, needing periodic <code>REINDEX</code> .

```
-- Find indexes that have never been used since the last stats reset
SELECT s.relname AS table, s.indexrelname AS index,
       pg_size_pretty(pg_relation_size(s.indexrelid)) AS size, s.idx_scan
FROM   pg_stat_user_indexes s
JOIN   pg_index i ON i.indexrelid = s.indexrelid
WHERE  s.idx_scan = 0
       AND NOT i.indisunique      -- never drop a uniqueness-enforcing index
ORDER BY pg_relation_size(s.indexrelid) DESC;

-- Find duplicate/redundant indexes: (a) is covered by (a,b)
SELECT indrelid::regclass AS table, array_agg(indexrelid::regclass) AS indexes
FROM   pg_index GROUP BY indrelid, indkey HAVING count(*) > 1;
```

BEST PRACTICE · BUILDING INDEXES ON A LIVE SYSTEM

`CREATE INDEX` takes a lock that blocks writes for its whole duration — minutes or hours on a large table. Always use `CREATE INDEX CONCURRENTLY` in production: it does not block writes, at the cost of two table passes and the possibility of leaving an `INVALID` index if it fails, which you then drop and retry. It cannot run inside a transaction block, which is why migration tools need special handling for it.

INTERVIEW · ADVANCED

Q. A table has 15 indexes and writes are slow. What do you do? — Find the unused ones from `pg_stat_user_indexes` and the redundant ones (any index that is a leftmost prefix of another). Drop them, excluding anything enforcing uniqueness. Then check whether several single-column indexes should be one composite index matching the real query. **Interview tip:** say that you reset the stats and observe for a full business cycle before dropping — an index used only by the monthly report shows zero scans on a Tuesday.

Part 20 Review

Quick Check ● BEGINNER

- 1 What two properties of a B-tree make lookups fast?
- 2 Roughly how deep is a B-tree over 100 million rows, and why does that matter?
- 3 State the leftmost-prefix rule.
- 4 Given `(a, b, c)`, which of `b`, `a, b`, `b, c`, `a, c` can use it?
- 5 Why do equality columns come before range columns?
- 6 What is an index-only scan, and why might it still read the heap?
- 7 What does `INCLUDE` do that adding a key column does not?
- 8 Why is a partial index cheap on the write path?
- 9 Why does an expression index on `lower(email)` not help `WHERE email = $1`?
- 10 Above roughly what result fraction does a sequential scan beat an index scan?
- 11 Which index type makes `ILIKE '%term%'` fast?
- 12 When is BRIN the right choice?
- 13 Name four costs of an index.
- 14 Why use `CREATE INDEX CONCURRENTLY`, and what are its two drawbacks?

Practice ● EASY

- 1 Add the four missing foreign-key indexes to the practice schema and confirm with `\d`.
- 2 Compute the cardinality and selectivity of every column in `orders`. Which are worth indexing?
- 3 Design the single best index for: orders for one customer, one status, newest first. Justify the column order.
- 4 Create a covering index with `INCLUDE` for a three-column query and explain why `INCLUDE` beats a key column here.
- 5 Create a partial index on pending orders and state what fraction of writes touch it.
- 6 Create an expression index on `lower(email)` and write the only query shape that uses it.
- 7 Install `pg_trgm` and index `products.product_name` for substring search.
- 8 Query `pg_stat_user_indexes` for unused indexes. Explain why the practice database shows almost all of them as unused.
- 9 Find redundant indexes with the `pg_index` query, after deliberately creating `(customer_id)` alongside `(customer_id, order_date)`.
- 10 Read `pg_stats` for `orders.status` and interpret `n_distinct` and `most_common_freqs`.

Challenge ● INTERMEDIATE

- 1** A table has indexes on `(a)`, `(a,b)`, `(a,b,c)`, `(b)` and `(c,a)`. Which are redundant, which are not, and what would you keep?
- 2** Design the indexes for a keyset-paginated feed sorted by date descending, filtered by tenant and status. Explain every column and why the sort direction is in the index.
- 3** A billion-row append-only events table is queried only by date range and event type. Design the indexing strategy and justify BRIN versus B-tree versus partitioning.
- 4** Writes on a 40-million-row table degraded after a release. No queries changed. Give three hypotheses and how you would test each.

Part 20 Solutions

Quick Check

- 1 Entries are sorted, so you can seek rather than scan; and the structure is balanced and shallow, so any lookup costs a fixed handful of page reads.
- 2 Four or five levels. It matters because lookup cost grows logarithmically — a table 100× larger costs one extra page read, not 100× the work.
- 3 A composite index serves queries using a leftmost prefix of its columns.
- 4 a, b yes. a, c partially — a seeks, c is only a filter on the retrieved entries. b and b, c no.
- 5 Because entries after a range column are only sorted within equal values of the preceding columns, so once the index reaches a range it cannot seek on anything further.
- 6 A scan satisfied entirely from the index, with no heap access. It may still read the heap because MVCC visibility is not in the index — if the visibility map does not mark a page all-visible, the row must be checked.
- 7 Stores the column in the leaves without making it part of the sorted key, so the index stays smaller and the column needs no ordering semantics.
- 8 Rows that fail the predicate are not in the index at all, so writing them does not touch it.
- 9 The index stores the transformed values. The planner matches the expression, and email is not lower(email) .
- 10 Around 5–10%. Beyond that, sequential page reads beat random heap access.
- 11 A GIN index with gin_trgm_ops from the pg_trgm extension.
- 12 Very large, naturally ordered, append-only tables — time-series — where a tiny min/max-per-block index prunes most of the table.
- 13 Write amplification, storage, cache pressure, planning time, vacuum cost, bloat. Any four.
- 14 It avoids blocking writes for the duration of the build. Drawbacks: it needs two table passes so it is slower, it can leave an INVALID index on failure, and it cannot run inside a transaction block.

Practice — selected

3 · The best single index.

```
-- Query: WHERE customer_id = ? AND status = ? ORDER BY order_date DESC
CREATE INDEX idx_orders_cust_status_date
ON orders (customer_id, status, order_date DESC);
```

Two equality columns first, so the index can seek directly to the matching group; then order_date DESC , so the rows come out already in the required order and the plan contains no Sort node. Putting order_date earlier would break the seek; putting the equality columns in the other order would work almost as well, and

`customer_id` goes first because it is far more selective, which keeps the scanned portion of the index smaller when only it is supplied.

8 · Why everything shows as unused. `idx_scan` is zero for almost every index in the practice database, and the reason is not that the indexes are bad — it is that with five to eleven rows per table, a sequential scan is genuinely cheaper than consulting an index, so the planner correctly never uses one. This is the most important thing to understand about testing index work: **you cannot evaluate an index on a toy dataset.** Generate a realistic table (§21.2) before drawing any conclusion, and treat `SET enable_seqscan = off` as a way to *prove an index is usable*, never as evidence that it is worth having.

10 · Reading `pg_stats`.

```
SELECT attname, n_distinct, most_common_vals, most_common_freqs
FROM   pg_stats WHERE tablename = 'orders' AND attname = 'status';
```

`n_distinct` of 5 tells the planner there are five statuses; `most_common_vals` and `most_common_freqs` tell it their distribution. If `delivered` has frequency 0.43, the planner estimates `WHERE status = 'delivered'` returns 43% of the table and will choose a sequential scan — correctly. If a rare value has frequency 0.001, the *same column* gets an index scan for that value. This is why the planner's choice legitimately differs between two executions of the same query text with different parameters, and why `ANALYZE` keeping these figures current matters so much (§22.1).

Challenge — worked

1 · The five indexes. `(a)` is **redundant** — it is a leftmost prefix of `(a,b,c)`, which serves everything it serves. `(a,b)` is **also redundant** for the same reason. `(a,b,c)` **keep**. `(b)` **keep** — `b` alone cannot use any of the `a`-leading indexes. `(c,a)` **keep** — it serves `c` alone and `c,a`, neither of which the others cover. Final set: `(a,b,c)`, `(b)`, `(c,a)`. The two caveats to state: a narrower index is smaller and slightly faster, so if `(a)` alone is used by a very hot query on a huge table, keeping it can still be justified by measurement — and never drop an index backing a `UNIQUE` or `PRIMARY KEY` constraint, since dropping it drops the constraint.

2 · The keyset feed index.

```
-- Query (Volume II §6.6):
--   WHERE tenant_id = ? AND status = 'published'
--     AND (published_at, post_id) < (?, ?)
--   ORDER BY published_at DESC, post_id DESC LIMIT 21

CREATE INDEX idx_posts_feed
  ON posts (tenant_id, published_at DESC, post_id DESC)
  WHERE status = 'published';
```

Every element earns its place. `tenant_id` leads because it is an equality predicate present in every query and it partitions the index by customer. `published_at DESC, post_id DESC` matches both the row-value cursor comparison *and* the `ORDER BY` exactly — so the plan is a single index scan with no sort, and the

`LIMIT` stops it after 21 entries regardless of table size. `post_id` is in the index because the cursor needs a unique tie-break. And `status` becomes a **partial predicate** rather than a key column: the feed only ever shows published posts, so drafts and deleted posts are excluded from the index entirely, making it smaller and free to maintain when a draft is edited. The `DESC` in the index is what makes the descending scan a forward scan; PostgreSQL can scan backwards, but a matching direction combines more cleanly with the `LIMIT` and avoids surprises when a second ordering column disagrees.

3 · The billion-row events table. The answer is **partitioning plus BRIN**, not a single B-tree. Range-partition by month on `occurred_at` (Part 22): a date-range query then touches only the relevant partitions, old data is dropped by detaching a partition instantly rather than by a 40-million-row `DELETE`, and each partition's indexes stay small enough to cache. Within each partition, a **BRIN** index on `occurred_at` — because rows arrive in timestamp order, the block-range min/max values are tight, so BRIN prunes almost everything while occupying a few hundred kilobytes instead of the tens of gigabytes a B-tree would need. Then a **B-tree on `event_type`** only if the type filter is selective; if there are six event types, it will not be used and a partial index per hot type is better. What to avoid: a plain B-tree on `occurred_at` over the whole table — it works, and it costs enormous storage and write amplification for a pruning job BRIN does for almost nothing.

4 · Writes degraded, no query changes. Three hypotheses, each with a test. **(i) New indexes shipped in the release.** Test: compare `pg_indexes` against the previous migration state; check `pg_stat_user_indexes` for indexes created recently with zero scans. This is by far the most common cause — someone added three indexes “to be safe”. **(ii) Updates stopped being HOT.** A PostgreSQL update can avoid touching indexes entirely if no indexed column changed and the new row version fits on the same page. Adding an index on a frequently-updated column destroys that optimisation for every update. Test: `pg_stat_user_tables.n_tup_hot_upd` versus `n_tup_upd` — a collapsed ratio confirms it. **(iii) Bloat or a fill-factor problem.** Test: check table and index sizes against row counts over time, and `pg_stat_user_tables.n_dead_tup`; a long-running transaction blocking `VACUUM` (Volume IV §18.4) produces exactly this symptom with no code change at all. Hypothesis (ii) is the one that separates a senior answer — most engineers do not know that adding an index can slow down updates to *unrelated* columns.

Query Performance

Optimisation is not a set of tricks. It is one skill: reading the plan the database chose, understanding why it chose it, and changing the inputs so it chooses better. This part teaches that skill.

IN THIS PART

21.1 The planner and the optimizer

21.2 Building a realistic table

21.3 Reading EXPLAIN

21.4 Scan types

21.5 Join algorithms

21.6 Sorts and aggregation

21.7 A worked optimisation

21.8 Finding slow queries · Review

21.1 The planner and the optimizer

SQL is declarative: you state the result, the engine decides the algorithm (Volume I §1.9). That decision is made by the **planner**, and understanding its three inputs tells you everything about how to influence it.

Input	What it is	You influence it by
Available paths	Which scans and join methods are possible given the indexes that exist	Creating the right indexes (Part 20)
Statistics	Row counts, distinct values, value distributions, correlation — sampled by <code>ANALYZE</code>	Keeping them current; raising the sample size
Cost constants	<code>seq_page_cost</code> , <code>random_page_cost</code> , <code>cpu_tuple_cost</code> , <code>effective_cache_size</code>	Configuration, matched to your hardware

The planner enumerates candidate plans, costs each in abstract units, and executes the cheapest. It is a *cost-based* optimizer, so it does not know the truth — it knows its estimates. Almost every planner mistake is an estimate that was wrong, which is why §21.3's central skill is comparing estimated rows against actual rows.

NOTE · `RANDOM_PAGE_COST`

The default is `4.0` — random reads assumed four times more expensive than sequential, which was true of spinning disks. On SSDs and cloud storage the real ratio is closer to `1.1` , and leaving the default makes the planner systematically underuse indexes. It is the single most valuable configuration change on modern hardware, and it explains a surprising number of “why won't it use my index” reports.

21.2 Building a realistic table

Run this before continuing. Everything in the rest of this part depends on having a table large enough that the planner's choices are interesting.

```
CREATE TABLE big_orders (  
    order_id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    customer_id   integer NOT NULL,  
    order_date    date NOT NULL,  
    status        text NOT NULL,  
    total_amount  numeric(12,2) NOT NULL  
);  
  
INSERT INTO big_orders (customer_id, order_date, status, total_amount)  
SELECT (random() * 50000)::int + 1,  
       DATE '2022-01-01' + (random() * 1000)::int,  
       (ARRAY['pending','paid','shipped','delivered','cancelled'])[  
           CASE WHEN random() < 0.70 THEN 4           -- 70% delivered: realistic skew  
                WHEN random() < 0.85 THEN 3  
                WHEN random() < 0.95 THEN 2  
                WHEN random() < 0.99 THEN 1  
                ELSE 5 END],  
       (random() * 50000)::numeric(12,2)  
FROM   generate_series(1, 2000000);  
  
ANALYZE big_orders;                                -- always, after a bulk load  
  
SELECT pg_size_pretty(pg_total_relation_size('big_orders'));  -- ~150 MB
```

Two million rows, 50,000 customers, three years of dates, and a deliberately skewed status distribution — because uniform test data makes the planner look far cleverer than it is on real data.

COMMON MISTAKE

Forgetting `ANALYZE` after a bulk load. Until it runs, the planner has no statistics for the new table and falls back on crude defaults — often choosing a nested loop over two million rows. If a query is inexplicably slow immediately after a data import, run `ANALYZE` before investigating anything else.

21.3 Reading EXPLAIN

```
EXPLAIN -- the plan and its estimates. Does not run the query.
EXPLAIN ANALYZE -- runs it, and reports ACTUAL rows and time
EXPLAIN (ANALYZE, BUFFERS) -- plus page reads: shared hit vs read from disk
EXPLAIN (ANALYZE, BUFFERS, VERBOSE, SETTINGS, FORMAT TEXT) -- everything
```

CAREFUL

`EXPLAIN ANALYZE` **executes the statement**. On a `DELETE` or `UPDATE` that means it really deletes or updates. Wrap it: `BEGIN; EXPLAIN ANALYZE DELETE ...; ROLLBACK;`

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT customer_id, count(*), sum(total_amount)
FROM big_orders
WHERE order_date >= '2024-01-01' AND status = 'delivered'
GROUP BY customer_id
ORDER BY sum(total_amount) DESC
LIMIT 10;
```

```
Limit (cost=98234.11..98234.14 rows=10 width=48)
  (actual time=812.443..812.447 rows=10 loops=1)
    Buffers: shared hit=1204 read=17893
    -> Sort (cost=98234.11..98891.02 rows=262764 width=48)
      (actual time=812.441..812.443 rows=10 loops=1)
        Sort Key: (sum(total_amount)) DESC
        Sort Method: top-N heapsort Memory: 26kB
        -> HashAggregate (cost=89122.44..91750.08 rows=262764 width=48)
          (actual time=780.112..801.334 rows=49873 loops=1)
            Group Key: customer_id
            Batches: 1 Memory Usage: 8241kB
            -> Seq Scan on big_orders
              (cost=0.00..46331.00 rows=281996 width=18)
              (actual time=0.021..521.883 rows=284102 loops=1)
              Filter: ((order_date >= '2024-01-01'::date)
                AND (status = 'delivered'::text))
              Rows Removed by Filter: 1715898
            Planning Time: 0.184 ms
            Execution Time: 812.503 ms
```

HOW TO READ IT – FIVE RULES

1. **Read from the inside out, bottom up.** The most indented node runs first. Here: sequential scan → hash aggregate → sort → limit.

2. **Compare `rows=` estimated against `rows=` actual.** The scan estimated 281,996 and got 284,102 — excellent. A discrepancy of 100× is the single most useful signal in the whole plan, and it means the statistics are wrong or the planner cannot model your predicate.
3. **actual time is cumulative and per loop.** A node's time includes its children. With `loops=N`, multiply: `actual time=2.1` with `loops=5000` is 10.5 seconds, not 2 milliseconds. This is where nested-loop disasters hide.
4. **Find the node where time jumps.** Here the scan alone is 521 ms of the 812 ms — that is the target.
5. **Rows Removed by Filter is the smoking gun.** 1.7 million rows read and thrown away means the filter should have been an index.

BEST PRACTICE · BUFFERS TELLS YOU ABOUT I/O

`shared hit` is pages found in cache; `read` is pages fetched from the operating system. 17,893 reads against 1,204 hits means this query is doing real I/O and will be much slower on a cold cache. Run any benchmark twice and report the second run — the first is measuring your disk, not your query.

21.4 Scan types

Node	What it does	Good sign / bad sign
Seq Scan	Reads every page of the table	Correct for small tables or unselective filters; bad when Rows Removed by Filter is huge
Index Scan	Walks the index, fetches each matching row from the heap	Ideal for few rows; degrades as row count rises because heap access is random
Index Only Scan	Answers entirely from the index	The best case. Requires a covering index and a vacuumed table
Bitmap Heap Scan	Collects matching tuple ids from one or more indexes, sorts them, then reads the heap in physical order	The planner's middle ground for medium selectivity — often optimal. Can combine several indexes
Bitmap Index Scan	The index half of the above	Appears as a child of Bitmap Heap Scan

The bitmap scan deserves attention because people misread it as a failure. It exists precisely because an index scan's random heap access becomes expensive past a few thousand rows: by gathering all the tuple ids first and sorting them, the heap is then read *sequentially*. Seeing **Bitmap Heap Scan** usually means the planner made a good decision, not a bad one.

```
-- Two indexes combined by bitmap AND – no composite index needed
EXPLAIN SELECT * FROM big_orders WHERE customer_id = 42 AND status = 'paid';
--   Bitmap Heap Scan on big_orders
--     Recheck Cond: ((customer_id = 42) AND (status = 'paid'))
--     -> BitmapAnd
--           -> Bitmap Index Scan on idx_big_customer
--           -> Bitmap Index Scan on idx_big_status
```

NOTE · RECHECK COND AND LOSSY BITMAPS

If the bitmap grows too large for **work_mem**, PostgreSQL degrades it to page granularity — it remembers which *pages* contain matches rather than which rows. The **Recheck Cond** then re-evaluates the predicate on every row of those pages. Seeing **lossy=** in a plan with **BUFFERS** is a signal to raise **work_mem** or narrow the query.

21.5 Join algorithms

Three algorithms, chosen by the planner from the estimated row counts. Knowing which is appropriate lets you recognise instantly when the planner has been misled.

Algorithm	How it works	Chosen when
Nested Loop	For each outer row, look up matches in the inner side — ideally via an index	The outer side is small . Devastating if the estimate was wrong and it is actually large
Hash Join	Build a hash table from the smaller side, then probe it once per row of the larger	Both sides substantial, join on equality. The workhorse of analytical queries
Merge Join	Sort both sides, then walk them together in one pass	Both sides large and already sorted — typically by matching indexes

THE CLASSIC PERFORMANCE DISASTER

```
-> Nested Loop (cost=0.42..8934.11 rows=12 width=48)
      (actual time=0.031..48213.882 rows=284102 loops=1)
    -> Seq Scan on a (rows=12) (actual rows=284102 loops=1)  ← 23,000x off
    -> Index Scan using pk_b on b (actual time=0.14..0.16 loops=284102)
```

The planner estimated 12 outer rows and chose a nested loop, which is correct for 12. There were 284,102, so the inner index scan ran 284,102 times — 0.16 ms each, 48 seconds total. **Every catastrophic plan looks like this:** a nested loop whose outer estimate was badly wrong. The fix is never “force a hash join”; it is to find out why the estimate was wrong — stale statistics, a correlated predicate, a function the planner cannot see through, or a join on an expression.

```
-- Diagnostic only, never in production code: prove the alternative is better
SET enable_nestloop = off;
EXPLAIN ANALYZE SELECT ...;      -- if it is now fast, the estimate was the problem
RESET enable_nestloop;
```

21.6 Sorts and aggregation

Node	Meaning
Sort Method: quicksort	Fitted in <code>work_mem</code> . Good.
Sort Method: top-N heapsort	Only the top N kept — the <code>LIMIT</code> optimisation. Very good.
Sort Method: external merge Disk: 42MB	Spilled to disk. Raise <code>work_mem</code> for this session, or avoid the sort with an index.
HashAggregate	Groups via a hash table. Fast; needs memory proportional to group count.
GroupAggregate	Groups sorted input. Chosen when input is already ordered, or when <code>HashAggregate</code> would not fit.
Batches: 5 Memory Usage: ...	The hash spilled to disk in five passes. Same remedy as an external sort.

```
-- Session-level, for one heavy report. Never set this globally to a large value:
-- work_mem is per sort node per connection, so 100 connections × 3 sorts × 256MB
-- is 75 GB of potential allocation.
SET work_mem = '256MB';
```

BEST PRACTICE

The best sort is the one that does not happen. An index whose column order matches the `ORDER BY` lets the engine read rows already sorted, and the `Sort` node vanishes from the plan entirely — which is worth more than any amount of `work_mem` . Check for this before tuning memory.

21.7 A worked optimisation

The query from §21.3, taken from 812 ms to 31 ms in three steps. This is the whole method.

STEP 1 — READ THE PLAN AND NAME THE PROBLEM

The sequential scan reads 2 million rows and discards 1.7 million (Rows Removed by Filter: 1715898). 521 ms of the 812 ms is spent finding 284,102 rows. The filter should be served by an index.

STEP 2 — BUILD THE INDEX THE QUERY ACTUALLY WANTS

```
-- status = 'delivered' matches 70% of rows, so it is useless as a key column.
-- But it is a constant in this query, so it becomes the partial predicate,
-- and order_date – the range – becomes the key.
CREATE INDEX idx_big_delivered_date
  ON big_orders (order_date, customer_id, total_amount)
  WHERE status = 'delivered';

ANALYZE big_orders;
```

```
Limit (actual time=30.882..30.886 rows=10 loops=1)
  Buffers: shared hit=2891
    -> Sort (actual time=30.880..30.882 rows=10 loops=1)
          Sort Method: top-N heapsort  Memory: 26kB
          -> HashAggregate (actual time=24.113..29.442 rows=49873 loops=1)
                Group Key: customer_id
                -> Index Only Scan using idx_big_delivered_date on big_orders
                     (actual time=0.028..12.114 rows=284102 loops=1)
                     Index Cond: (order_date >= '2024-01-01'::date)
Planning Time: 0.211 ms
Execution Time: 30.941 ms
```

Three things changed at once, and each is worth naming. The scan became an **Index Only Scan** — every column the query needs is in the index, so the heap is never touched, which is why **shared read** dropped to zero. The filter became an **Index Cond**, so rows are no longer read and discarded. And the partial predicate means the index contains only delivered orders, making it 30% the size it would otherwise be.

STEP 3 — VERIFY THE TRADE-OFF

```
SELECT pg_size_pretty(pg_relation_size('idx_big_delivered_date'));  -- ~47 MB
```

47 MB of storage and a small write cost, in exchange for 26× on a query that runs constantly. Whether that is worth it is a judgement — but it is now a judgement made with numbers rather than instinct, which is the entire point of this part.

THE METHOD, IN FIVE LINES

(1) Get the plan with `EXPLAIN (ANALYZE, BUFFERS)`. **(2)** Find the node where time accumulates. **(3)** Compare estimated to actual rows — if they diverge, fix the statistics before anything else. **(4)** Ask what the node is doing that it should not: reading and discarding, sorting, looping. **(5)** Change one input — an index, a rewrite, `ANALYZE` — and measure again. Never change two things at once.

21.8 Finding slow queries

You cannot optimise what you have not measured. `pg_stat_statements` is the single most valuable extension in PostgreSQL.

```
-- postgresql.conf: shared_preload_libraries = 'pg_stat_statements' (needs restart)
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- The queries costing you the most TOTAL time – not the slowest single query,
-- which is usually a nightly report nobody is waiting for.
SELECT round(total_exec_time::numeric, 1)    AS total_ms,
       calls,
       round(mean_exec_time::numeric, 2)     AS mean_ms,
       rows,
       left(query, 90)                       AS query
FROM   pg_stat_statements
ORDER  BY total_exec_time DESC
LIMIT 15;

-- Log anything slower than 500 ms, permanently
-- log_min_duration_statement = 500
-- auto_explain.log_min_duration = 1000 (logs the PLAN, not just the text)
```

BEST PRACTICE · OPTIMISE BY TOTAL TIME, NOT MEAN

A query taking 4 ms and running 200,000 times an hour costs 800 seconds. A query taking 6 seconds and running twice costs 12. Sort by `total_exec_time`, not `mean_exec_time` — the biggest wins are almost always in a fast query called constantly, and that query is invisible to a slow-query log with a 500 ms threshold. This is also the N+1 problem's signature (§22.3).

INTERVIEW · ADVANCED

Q. A page is slow. Walk me through diagnosing it. — “First find out *which* query, from `pg_stat_statements` ordered by total time, or the application's traces — not by guessing. Then `EXPLAIN (ANALYZE, BUFFERS)` on the real parameters. I look for three things in order: a large gap between estimated and actual rows, which means a statistics problem; a nested loop with a big outer side; and rows read then discarded by a filter. Then I change one input and re-measure.”

Interview tip: mention that the answer is often not a slow query at all but many fast ones — an N+1 — because that is what a trace shows and a slow-query log hides.

Part 21 Review

Quick Check ● BEGINNER

- 1 Name the planner's three inputs and how you influence each.
- 2 What is the difference between `EXPLAIN` and `EXPLAIN ANALYZE`, and what is the danger of the second?
- 3 In which order do you read a plan?
- 4 Why must you multiply `actual time` by `loops`?
- 5 What does `Rows Removed by Filter` tell you?
- 6 Why is a Bitmap Heap Scan often a good sign?
- 7 When is a nested loop the right choice, and when is it a catastrophe?
- 8 What does `Sort Method: external merge Disk: 42MB` mean, and give two remedies.
- 9 Why should `work_mem` not be set globally to a large value?
- 10 Why sort `pg_stat_statements` by total time rather than mean?
- 11 What does `random_page_cost = 4.0` assume, and why change it?
- 12 What is the first thing to do after a bulk load?

Practice ● EASY

- 1 Build `big_orders` from §21.2 and confirm its size.
- 2 Run the §21.3 query and read its plan. Identify the slowest node and the rows discarded.
- 3 Add the §21.7 index and re-measure. Report both times and the index size.
- 4 Force a sequential scan with `SET enable_indexscan = off` and compare. Explain what this proves and what it does not.
- 5 Write a query the planner serves with a Bitmap Heap Scan, and one it serves with an Index Only Scan.
- 6 Produce a plan containing an external merge sort, then eliminate it with an index.
- 7 Join `big_orders` to a small table and observe a nested loop. Then join it to a large one and observe a hash join.
- 8 Delete the statistics with `ALTER TABLE ... SET STATISTICS 1` and `ANALYZE`, then compare estimated to actual rows. Restore afterwards.
- 9 Install `pg_stat_statements` and find your top five queries by total time.
- 10 Use `EXPLAIN (ANALYZE, BUFFERS)` to show the difference between a cold and warm cache for the same query.

Challenge ● ADVANCED

- 1** A plan shows `Nested Loop` with outer `rows=3` and `actual rows=180000`. Give four possible causes of the misestimate and how you would test each.
- 2** Optimise: `SELECT * FROM big_orders WHERE extract(year FROM order_date) = 2024 AND lower(status) = 'paid'`. Give two approaches.
- 3** A query is fast in `psql` and slow from the application. Give three explanations.
- 4** Explain why a query got slower after the table grew, even though the index is still used.

Part 21 Solutions

Quick Check

- 1 Available paths (create indexes); statistics (keep `ANALYZE` current, raise the sample); cost constants (configure to match hardware).
- 2 `EXPLAIN` shows estimates without running; `EXPLAIN ANALYZE` executes and reports actuals. The danger: it really performs an `UPDATE` or `DELETE` — wrap it in a transaction and roll back.
- 3 Inside out, most-indented first, bottom to top.
- 4 Because the reported time is per loop. A 2 ms node with 5,000 loops is 10 seconds.
- 5 How many rows were read and thrown away — the clearest signal that a filter should have been an index condition.
- 6 It exists to avoid random heap access at medium selectivity: tuple ids are gathered and sorted so the heap is read sequentially. It usually means a good decision.
- 7 Right when the outer side is genuinely small and the inner has an index. A catastrophe when the outer estimate was wrong, because the inner lookup runs once per outer row.
- 8 The sort exceeded `work_mem` and spilled to disk. Remedies: raise `work_mem` for that session, or add an index that makes the sort unnecessary.
- 9 It is allocated per sort or hash node per connection, so the worst case multiplies by connections and by nodes per query.
- 10 Because a 4 ms query called 200,000 times costs far more than a 6-second query called twice — and the frequent one is invisible to a slow-query threshold.
- 11 That random reads cost four times a sequential read, which was true of spinning disks. On SSDs the real ratio is near 1.1, and the default makes the planner underuse indexes.
- 12 `ANALYZE`. Without statistics the planner uses crude defaults and often picks a disastrous plan.

Practice — selected

4 • What forcing a sequential scan proves. `SET enable_indexscan = off` makes the planner choose its next-best plan, and comparing the two times tells you whether the index was actually helping. What it **does not** prove: that the planner was wrong. If the forced plan is slower, the planner was right. And these settings are diagnostic only — they are not hints, they do not survive the session, and shipping them in application code disables the optimizer's ability to adapt as data changes. The legitimate uses are exactly two: proving an index is usable at all, and demonstrating that a misestimate rather than a missing index is the problem.

6 • Creating and eliminating an external merge sort.

```

SET work_mem = '1MB';                                -- make spilling easy to reproduce
EXPLAIN (ANALYZE) SELECT * FROM big_orders ORDER BY total_amount;
-- Sort Method: external merge  Disk: 61256kB          ← 60 MB written and read back

CREATE INDEX idx_big_amount ON big_orders (total_amount);
EXPLAIN (ANALYZE) SELECT * FROM big_orders ORDER BY total_amount;
-- Index Scan using idx_big_amount on big_orders      ← no Sort node at all
RESET work_mem;

```

Note which fix is better and why. Raising `work_mem` to 128 MB also removes the disk spill, but it still sorts two million rows on every execution. The index removes the sort *entirely* — the rows are already in order — so the query can also stop early under a `LIMIT`, which a sort can never do. Structural fixes beat memory fixes.

8 • Sabotaging the statistics.

```

ALTER TABLE big_orders ALTER COLUMN status SET STATISTICS 1;
ANALYZE big_orders;
EXPLAIN ANALYZE SELECT * FROM big_orders WHERE status = 'cancelled';
-- estimated rows=400000, actual rows=19873          ← 20x wrong

ALTER TABLE big_orders ALTER COLUMN status SET STATISTICS -1; -- back to default
ANALYZE big_orders;

```

With a single-bucket histogram the planner cannot see the skew and assumes the five statuses are evenly distributed. The resulting overestimate makes it avoid an index it should use — and in a join, that same overestimate would push it away from a nested loop that was correct. This exercise is the mechanism behind “the query was fine yesterday”: statistics drift, and the plan changes without the query changing. Raising `SET STATISTICS` to 500 or 1000 on a skewed, heavily-queried column is a real tuning technique for exactly this reason.

Challenge — worked

1 • The 60,000× misestimate. Four causes, each with a test. **(i) Stale statistics.** Test: check `pg_stat_user_tables.last_analyze`, run `ANALYZE`, re-explain. Always check this first; it is the most common and the cheapest to fix. **(ii) Correlated predicates.** The planner multiplies selectivities assuming independence, so `WHERE city = 'Mumbai' AND state = 'Maharashtra'` is estimated as $1/1000 \times 1/30$ when in reality the second is implied by the first. Test: compare the estimate for each predicate alone against both together; fix with extended statistics — `CREATE STATISTICS (dependencies) ON city, state FROM addresses; ANALYZE addresses;`. This is the cause most people have never heard of and it is extremely common in real schemas. **(iii) An opaque expression.** A function call or a join on an expression gives the planner no histogram, so it falls back on a fixed default guess. Test: does the estimate look like a round fraction such as 0.5% or 0.33%? Fix with an expression index, which also creates statistics on the

expression. **(iv) A parameter the planner could not see.** A generic plan for a prepared statement, or a value inside a CTE that acts as an optimisation fence. Test: substitute the literal and re-explain; if the estimate becomes accurate, it is a plan-caching issue (see challenge 3).

2 · The unsargable query.

```
-- Original: neither predicate can use an index
SELECT * FROM big_orders
WHERE  extract(year FROM order_date) = 2024 AND lower(status) = 'paid';

-- Approach A (best): rewrite to be sargable, then index normally
SELECT * FROM big_orders
WHERE  order_date >= DATE '2024-01-01' AND order_date < DATE '2025-01-01'
      AND status = 'paid';
CREATE INDEX idx_big_paid_date ON big_orders (order_date) WHERE status = 'paid';

-- Approach B: when you cannot change the query, index the expressions
CREATE INDEX idx_big_year ON big_orders (extract(year FROM order_date));
CREATE INDEX idx_big_status_lower ON big_orders (lower(status));
```

Approach A is correct and B is a workaround. The rewrite gives a range scan on a half-open interval, which is index-friendly and time-zone-safe (Volume II §4.5), and it lets the status become a partial predicate. Approach B produces two indexes that serve only these exact expressions and nothing else — no other date query can use `extract(year ...)`, and `lower(status)` is pure waste given that `status` is already lower-case in the data. B is what you do when the query lives in a vendor application you cannot edit.

3 · Fast in psql, slow from the application. Three explanations, all real. **(i) Generic plans for prepared statements.** After five executions PostgreSQL may switch a prepared statement to a generic plan built without knowledge of the actual parameter values — which is disastrous for a skewed column, where the right plan differs per value. Test: `SET plan_cache_mode = force_custom_plan` and compare. **(ii) It is not one query.** The application issues the query once per row of a previous result — the N+1 problem (§22.3). Each execution is genuinely fast; there are 500 of them plus 500 network round trips. Test: count the queries in the application's trace or in `pg_stat_statements.calls`, which is how you distinguish this from a slow query in one measurement. **(iii) Different session settings or a different transaction context.** The application may run at a different isolation level, hold a long transaction whose snapshot forces more visibility checks, or have a smaller `work_mem` than your interactive session. Test: `SELECT * FROM pg_settings WHERE source <> 'default'` from both, and compare. Explanation (ii) is the most common by a wide margin, and it is the one a database-side investigation alone will never find.

4 · Slower after growth, index still used. Several mechanisms, and they compound. The index is deeper — one or two extra levels of page reads per lookup, which is minor. More significantly, the **working set no longer fits in cache**: at 2 GB the whole table and index sat in shared buffers, and at 200 GB every lookup is a real disk read, so the same plan is an order of magnitude slower with identical row counts. Third, if the query returns a *fraction* of the table, that fraction is now a much larger absolute number of rows — 0.1% of 200 million is 200,000 heap fetches, each potentially a random read. Fourth, **bloat**: if the table has churned, the index contains dead entries and spans more pages than its live data warrants (§22.2). The diagnostic

that distinguishes them is **BUFFERS** — a rising **read** against **hit** ratio points at cache; a stable ratio with rising row counts points at the query legitimately doing more work, which means the fix is a narrower query or a covering index, not more memory.

Advanced Performance Engineering

Part 21 was about one query. This part is about the system: the maintenance that keeps plans good, the application patterns that generate load, and the four architectural moves — partitioning, materialization, pooling, replicas — you make when a single well-tuned database is no longer enough.

IN THIS PART

22.1 Statistics and ANALYZE

22.2 VACUUM and bloat

22.3 The N+1 problem

22.4 SELECT * and pagination at scale

22.5 Partitioning

22.6 Materialized views

22.7 Connection pooling

22.8 Read replicas · Review

22.1 Statistics and ANALYZE

The planner's estimates come from a statistical sample of each table, refreshed by `ANALYZE`. Stale statistics are the most common cause of a plan that was good last month and is terrible now.

```
ANALYZE big_orders;                -- one table
ANALYZE;                          -- the whole database
ANALYZE VERBOSE big_orders (status); -- one column, with progress

-- Who is behind?
SELECT relname, n_live_tup, n_mod_since_analyze, last_analyze, last_autoanalyze
FROM   pg_stat_user_tables ORDER BY n_mod_since_analyze DESC;

-- Raise the sample for a skewed, heavily-queried column (default 100 buckets)
ALTER TABLE big_orders ALTER COLUMN status SET STATISTICS 1000;
ANALYZE big_orders;

-- Teach the planner about correlated columns
CREATE STATISTICS addr_city_state (dependencies, ndistinct)
  ON city, state FROM addresses;
ANALYZE addresses;
```

Autovacuum runs `ANALYZE` automatically once roughly 10% of a table's rows have changed. That default is far too lax for a large table: 10% of 100 million rows is 10 million modifications before the planner learns anything. Tune it per table for the ones that matter:

```
ALTER TABLE big_orders SET (
  autovacuum_analyze_scale_factor = 0.02,  -- 2% instead of 10%
  autovacuum_vacuum_scale_factor  = 0.05
);
```

BEST PRACTICE

Extended statistics (`CREATE STATISTICS`, PostgreSQL 10+) are the fix for the correlated-predicate misestimate of Part 21's challenge 1, and almost nobody uses them. Any pair of columns where one implies the other — city and state, product and category, tenant and region — is a candidate, and the improvement in join plans can be dramatic.

22.2 VACUUM and bloat

PostgreSQL's MVCC never overwrites a row in place. An `UPDATE` writes a new version and marks the old one dead; a `DELETE` only marks. Those dead rows occupy space until `VACUUM` reclaims it — and understanding this explains a whole family of operational surprises.

Command	Does	Lock
<code>VACUUM</code>	Marks dead-row space reusable by this table. Does not shrink the file.	None blocking — safe on a live system
<code>VACUUM ANALYZE</code>	Both, in one pass	None blocking
<code>VACUUM FULL</code>	Rewrites the table compactly and returns space to the OS	ACCESS EXCLUSIVE — blocks everything
<code>REINDEX</code>	Rebuilds a bloated index	Blocking, unless <code>CONCURRENTLY</code> (PG 12+)

```
-- Measure bloat before acting on it
SELECT relname,
       n_live_tup, n_dead_tup,
       round(100.0 * n_dead_tup / nullif(n_live_tup + n_dead_tup, 0), 1) AS dead_pct,
       pg_size_pretty(pg_total_relation_size(relid))                    AS size,
       last_autovacuum
FROM   pg_stat_user_tables
WHERE  n_dead_tup > 1000
ORDER BY n_dead_tup DESC;
```

COMMON MISTAKE · WHY VACUUM IS NOT WORKING

Autovacuum **cannot remove a row version newer than the oldest open snapshot**. So a single forgotten transaction — an idle `BEGIN` in a psql window, a connection-pool session left open, a long-running report — prevents cleanup across the *entire database*, not just the tables it touched. Bloat then grows without limit while autovacuum appears to run normally. Find them:

```
SELECT pid, state, age(clock_timestamp(), xact_start) AS xact_age, left(query,60)
FROM   pg_stat_activity
WHERE  state <> 'idle' OR xact_start IS NOT NULL
ORDER BY xact_start;
-- 'idle in transaction' with a large xact_age is the culprit.
-- Prevention: idle_in_transaction_session_timeout = '5min'
```

NOTE · TRANSACTION ID WRAPAROUND

If vacuuming is prevented for long enough, PostgreSQL approaches transaction-id exhaustion and will eventually refuse writes to protect data. It is rare and it is a genuine outage. Monitor `SELECT max(age(datfrozenxid)) FROM pg_database;` — if it climbs toward 200 million, investigate before it reaches the 2-billion limit. Never disable autovacuum.

22.3 The N+1 problem

The most common performance defect in application code, and it is invisible to a slow-query log because every individual query is fast.

```
# The pattern: 1 query, then N more
orders = db.query("SELECT * FROM orders WHERE order_date >= %s", [start]) # 1
for o in orders:                                                         # N
    customer = db.query("SELECT * FROM customers WHERE customer_id = %s",
                        [o.customer_id])
# 500 orders → 501 queries → 501 network round trips → ~250 ms of pure latency
# for data one join would have returned in 4 ms.
```

```
-- Fix 1: one join
SELECT o.*, c.full_name, c.email
FROM   orders o JOIN customers c ON c.customer_id = o.customer_id
WHERE  o.order_date >= $1;

-- Fix 2: two queries, when the shapes must stay separate (the ORM "eager load")
SELECT * FROM orders WHERE order_date >= $1;
SELECT * FROM customers WHERE customer_id = ANY($2);  -- one query, array of ids
```

Symptom	Where you see it
Huge <code>calls</code> count on a trivial query	<code>pg_stat_statements</code> — the giveaway
Page time scales linearly with rows displayed	Application metrics: 10 rows fast, 500 rows 50× slower
Database CPU low, application latency high	The time is in round trips, not in the database
Hundreds of identical spans in a trace	APM waterfall

BEST PRACTICE

In an ORM this is a lazy-loading default. Learn your framework's eager-load keyword — Django's `select_related` / `prefetch_related`, Rails' `includes`, SQLAlchemy's `joinedload`, JPA's `JOIN FETCH` — and add a test that asserts a query count for your hottest endpoints. A count assertion catches the regression the moment someone adds an innocent-looking attribute access inside a loop.

22.4 SELECT * and pagination at scale

Two application habits with measurable cost, both already introduced and now quantified.

SELECT *

Volume II §3.1 gave four correctness reasons to avoid it. The performance reason is **TOAST**: PostgreSQL stores large values out-of-line in a separate table. A row with a 40 KB `description` column costs one extra page read per row when that column is selected — so `SELECT *` on a listing of 100 rows does 100 unnecessary TOAST fetches for a column nobody displays. It also makes an index-only scan impossible, since no index covers every column.

PAGINATION

```
-- OFFSET: cost grows linearly with page number (Volume II §6.5)
SELECT * FROM big_orders ORDER BY order_date DESC, order_id DESC LIMIT 20 OFFSET 100000;
-- ~180 ms: reads and discards 100,000 rows

-- Keyset: constant cost, any page
SELECT * FROM big_orders
WHERE (order_date, order_id) < ($1, $2)
ORDER BY order_date DESC, order_id DESC LIMIT 20;
-- ~0.4 ms with an index on (order_date DESC, order_id DESC)
```

COMMON MISTAKE · COUNT(*) FOR A TOTAL

A filtered `count(*)` must examine every matching row — there is no shortcut, and on a large table it is often slower than fetching the page itself. Options, cheapest first: drop the exact total and show “load more”; use the planner's estimate (`reltuples`), or the row estimate from `EXPLAIN`) and display “about 17,000 results”; or cache the exact count against the filter set for 30–60 seconds. Reserve the exact figure for cases that genuinely need it, such as an export receipt.

22.5 Partitioning

Splitting one logical table into physical pieces. Its two real benefits are often confused, so state them separately: **partition pruning** (queries touch only relevant partitions) and **instant bulk deletion** (drop a partition rather than delete millions of rows).

```
CREATE TABLE events (  
    event_id    bigint GENERATED ALWAYS AS IDENTITY,  
    occurred_at timestamptz NOT NULL,  
    event_type  text NOT NULL,  
    payload     jsonb NOT NULL,  
    PRIMARY KEY (event_id, occurred_at)      -- the partition key MUST be in the PK  
) PARTITION BY RANGE (occurred_at);  
  
CREATE TABLE events_2024_01 PARTITION OF events  
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');  
CREATE TABLE events_2024_02 PARTITION OF events  
    FOR VALUES FROM ('2024-02-01') TO ('2024-03-01');  
CREATE TABLE events_default PARTITION OF events DEFAULT;  -- catch-all  
  
-- Indexes created on the parent propagate to every partition  
CREATE INDEX ON events (event_type);  
  
-- Retention: instant, no dead rows, no vacuum afterwards  
ALTER TABLE events DETACH PARTITION events_2024_01;  
DROP TABLE events_2024_01;
```

Strategy	Splits by	Use for
RANGE	Ordered value bands	Time-series — by far the most common and most useful
LIST	Explicit value sets	Region, tenant, country
HASH	A hash of the key	Spreading writes evenly when there is no natural range

COMMON MISTAKE · PRUNING REQUIRES THE KEY IN THE WHERE CLAUSE

Partitioning helps *only* when the query filters on the partition key. `WHERE event_type = 'login'` with no date predicate scans every partition — and is now slower than an unpartitioned table, because the planner must consider each one. Verify pruning in the plan: you should see only the expected partitions listed, not all of them. Also note the constraint in the DDL above: **the partition key must be part of every unique constraint**, which is why the primary key is `(event_id, occurred_at)` rather than `event_id` alone — a real modelling consequence people meet late.

BEST PRACTICE

Partition when a table exceeds roughly 100 GB, or when you need time-based retention, or when maintenance operations on the whole table have become impossible to schedule. Do **not** partition for query speed alone — a good index on an unpartitioned table beats partitioning in almost every case, and partitioning adds real operational complexity: partition creation must be automated (`pg_partman` or a scheduled job), and a missing future partition sends rows to the default partition or fails the insert.

22.6 Materialized views

A view whose results are stored on disk. It converts an expensive aggregate from a per-request cost into a per-refresh cost.

```
CREATE MATERIALIZED VIEW mv_daily_revenue AS
SELECT order_date,
       count(*)           AS orders,
       sum(total_amount)   AS revenue,
       count(DISTINCT customer_id) AS customers
FROM   big_orders
WHERE  status <> 'cancelled'
GROUP BY order_date
WITH  DATA;

-- A unique index is REQUIRED for concurrent refresh, and speeds up reads
CREATE UNIQUE INDEX ON mv_daily_revenue (order_date);

REFRESH MATERIALIZED VIEW mv_daily_revenue;           -- locks readers out
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_daily_revenue; -- does not block reads
```

Refresh strategy	Trade-off
Scheduled full refresh	Simplest. Staleness bounded by the interval; whole-view recomputation cost.
<code>CONCURRENTLY</code> on a schedule	No read outage, but roughly twice the work and needs a unique index.
Incremental, hand-rolled	A summary table plus a trigger or a job that updates only changed keys. Cheapest to run, most code to maintain.

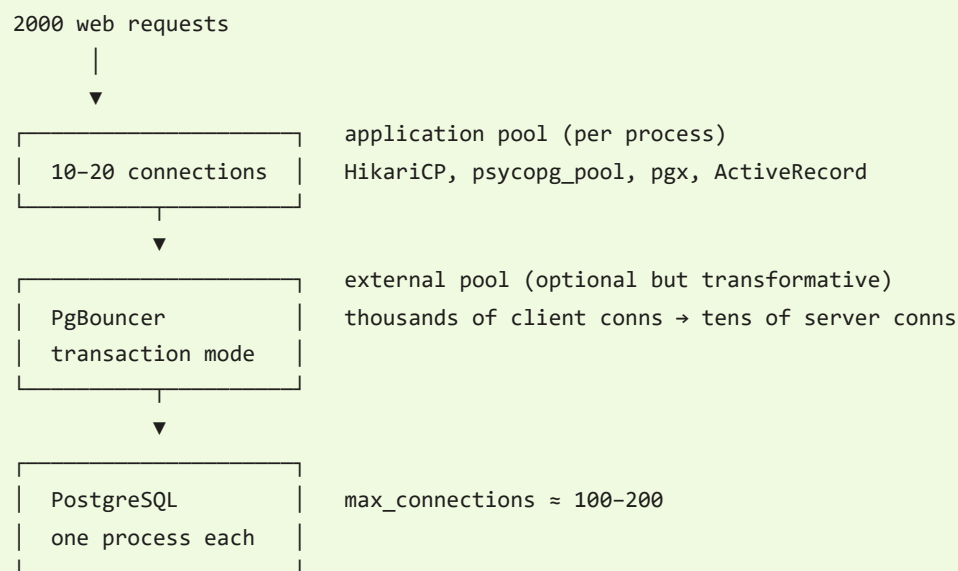
BEST PRACTICE

PostgreSQL has no incremental materialized-view maintenance — a refresh recomputes everything. For a large fact table that makes scheduled refreshes expensive, and the usual answer is a plain summary table maintained by an upsert over the changed date range: `INSERT ... SELECT ... FROM orders WHERE order_date >= current_date - 2 ... ON CONFLICT (order_date) DO UPDATE`. More code, orders of magnitude less work, and you control exactly how fresh each row is.

22.7 Connection pooling

A PostgreSQL connection is an operating-system **process**, costing several megabytes and a fork. Opening one per request is expensive; keeping thousands open is worse, because they contend for locks and memory even while idle.

FIGURE 22.7 — WHERE THE POOL SITS



PgBouncer mode	Reuses a server connection	Breaks
session	Per client connection	Nothing. Least pooling benefit.
transaction	Per transaction — the usual choice	Session state: <code>SET</code> , advisory locks, <code>LISTEN</code> , server-side prepared statements (unless PgBouncer 1.21+)
statement	Per statement	Multi-statement transactions entirely. Rarely usable.

BEST PRACTICE · POOL SIZING

Bigger is not better. A reasonable starting point is `connections ≈ (2 × cores) + effective_spindles` — often 10–30 for a typical server, not 500. Beyond the point where every core is busy, more connections add context switching and lock contention while reducing throughput. If requests are queueing, the fix is usually faster queries or an external pool, not a larger pool. And always set `idle_in_transaction_session_timeout`, or one leaked transaction in a pooled connection blocks vacuuming for the whole database (§22.2).

22.8 Read replicas and caching

The last resort before sharding: move read load off the primary. PostgreSQL streams the write-ahead log to standby servers that serve read-only queries.

Concern	Reality
Replication lag	Milliseconds normally, seconds under load. A read immediately after a write may not see it — the read-your-own-writes problem.
Routing	Writes and read-after-write go to the primary; reports, search and analytics to a replica. The application must decide, per query.
Query cancellation	A long query on a replica can conflict with replayed changes and be cancelled. <code>hot_standby_feedback = on</code> prevents it, at the cost of bloat on the primary.
What it does not fix	Write throughput. Every replica applies every write, so replicas scale reads only.

```
-- On the primary: how far behind is each replica?
SELECT client_addr, state, sent_lsn, replay_lsn,
       pg_wal_lsn_diff(sent_lsn, replay_lsn) AS replay_lag_bytes
FROM   pg_stat_replication;

-- On the replica: lag in seconds
SELECT now() - pg_last_xact_replay_timestamp() AS replication_delay;
```

THE CACHING LADDER

Work down this list before adding infrastructure, because each rung is cheaper to operate than the next:

1. **The buffer cache** — already there. Size `shared_buffers` to about 25% of RAM and `effective_cache_size` to about 75%.
2. **Better queries and indexes** — free, and the only rung that reduces total work rather than moving it.
3. **Materialized views or summary tables** — inside the database, transactionally consistent, no new system to run.
4. **An application cache** — Redis or in-process. Fast, and now you own cache invalidation, which is a genuinely hard problem.
5. **Read replicas** — scales reads, adds lag and routing complexity.
6. **Sharding** — scales writes, and changes everything: no cross-shard joins, no cross-shard transactions, no global uniqueness. Last resort, always.

INTERVIEW · EXPERT

Q. Your database is at capacity. Walk me through your options. — Go up the ladder in order and say why: “First I find out what the load actually is — `pg_stat_statements` by total time, and check for N+1 patterns, because the answer is often that we are issuing ten thousand cheap queries rather than running an expensive one. Then indexes and query rewrites, since those reduce work rather than redistributing it. Then materialized summaries for the reporting load. Then a read replica if reads dominate. Sharding last, because it costs us joins, transactions and global constraints.” **Interview tip:** ask whether the load is read-heavy or write-heavy before answering — replicas do nothing for writes, and a candidate who proposes them without asking has skipped the diagnosis.

Part 22 Review

Quick Check ● BEGINNER

- 1 What does `ANALYZE` collect, and when does autovacuum run it?
- 2 What are extended statistics for?
- 3 Why does a `DELETE` not free disk space?
- 4 Why can one idle transaction bloat the entire database?
- 5 Difference between `VACUUM` and `VACUUM FULL` ?
- 6 What is the N+1 problem, and what is its signature in `pg_stat_statements` ?
- 7 Give the performance reason — not the correctness reason — to avoid `SELECT *`.
- 8 Name partitioning's two real benefits.
- 9 When does partitioning make a query slower?
- 10 Why must the partition key be part of the primary key?
- 11 What does `REFRESH MATERIALIZED VIEW CONCURRENTLY` require, and what does it cost?
- 12 What breaks under PgBouncer transaction mode?
- 13 Why is a larger connection pool not always better?
- 14 What do read replicas *not* fix?

Practice ● EASY

- 1 Find every table whose statistics are furthest out of date.
- 2 Create extended statistics on two correlated columns and show the estimate improving.
- 3 Update 500,000 rows of `big_orders`, then measure `n_dead_tup` and the table size before and after `VACUUM`.
- 4 Open an idle transaction in a second session and demonstrate that `VACUUM` cannot reclaim the dead rows.
- 5 Find long-running and idle-in-transaction sessions with the `pg_stat_activity` query.
- 6 Write an N+1 in any language against the practice database, then fix it two ways and compare the query counts.
- 7 Partition `events` by month, insert rows spanning three months, and prove pruning with `EXPLAIN`.
- 8 Show a query against that partitioned table where pruning does *not* happen, and explain why.
- 9 Detach and drop a partition. Compare the time against a `DELETE` of the same rows.

- 10** Build `mv_daily_revenue`, time a query against it versus the underlying aggregate, then refresh it concurrently.
- 11** Replace that materialized view with a summary table maintained by an upsert over the last two days.
- 12** Compare `OFFSET 100000` against the keyset equivalent on `big_orders` and report both timings.

Challenge ● ADVANCED

- 1** A table has grown to 400 GB, 60% of which is bloat, and you cannot take downtime. Give a plan.
- 2** Design the retention strategy for a 2 TB events table keeping 90 days. Specify partitioning, automation, and what happens if a future partition is missing.
- 3** A dashboard runs 40 aggregate queries and takes 12 seconds. Give three architectural approaches with their trade-offs.
- 4** An application shows a stale value immediately after saving. Diagnose and give three fixes.
- 5** Writes are saturating a single primary. Explain why replicas do not help and what does.

Part 22 Solutions

Quick Check

- 1 Row counts, distinct-value estimates, most-common values and their frequencies, and histograms. Autovacuum analyses at roughly 10% of rows modified — too lax for large tables.
- 2 To teach the planner that two columns are correlated, so it stops multiplying their selectivities as if independent.
- 3 MVCC only marks rows dead. `VACUUM` makes the space reusable; only `VACUUM FULL` returns it to the operating system.
- 4 Vacuum cannot remove any row version newer than the oldest open snapshot, and that snapshot applies database-wide.
- 5 `VACUUM` is non-blocking and marks space reusable in place. `VACUUM FULL` rewrites the table compactly under an `ACCESS EXCLUSIVE` lock.
- 6 One query followed by one more per returned row. Signature: an enormous `calls` count on a trivial query with a low mean time.
- 7 TOAST fetches for large out-of-line columns you never display, plus it makes an index-only scan impossible.
- 8 Partition pruning, and instant bulk deletion by dropping a partition.
- 9 When the query does not filter on the partition key — every partition is scanned and the planner must consider each one.
- 10 Because a unique index cannot span partitions; uniqueness is enforced per partition, so the key must include the partitioning column.
- 11 A unique index on the view. It costs roughly double the work of a plain refresh, in exchange for not blocking readers.
- 12 Session state: `SET`, advisory locks, `LISTEN / NOTIFY`, temp tables, and server-side prepared statements on older versions.
- 13 Past the point where every core is busy, extra connections add context switching and lock contention while lowering throughput.
- 14 Write throughput — every replica applies every write. They scale reads only.

Practice — selected

4 • Proving that an idle transaction blocks vacuum. In S2: `BEGIN; SELECT 1;` and stop. In S1:

```
UPDATE big_orders SET total_amount = total_amount WHERE order_id <= 200000;
VACUUM (VERBOSE) big_orders;
-- INFO:  ... 200000 dead row versions cannot be removed yet,
--       oldest xmin: 12345678
```

That message is the one to recognise on sight — “cannot be removed yet” with an `oldest xmin` means something is holding a snapshot. Commit S2 and vacuum again and they are gone. In production this appears as a table that grows steadily while autovacuum logs show it running normally every few minutes, which is why the `pg_stat_activity` query in §22.2 belongs in your monitoring rather than in your incident response.

8 • When pruning does not happen.

```
-- Prunes: the partition key is in the predicate
EXPLAIN SELECT count(*) FROM events
WHERE occurred_at >= '2024-02-01' AND occurred_at < '2024-03-01';
-- Append -> Seq Scan on events_2024_02          ← one partition

-- Does NOT prune
EXPLAIN SELECT count(*) FROM events WHERE event_type = 'login';
-- Append -> Seq Scan on events_2024_01
--          -> Seq Scan on events_2024_02
--          -> Seq Scan on events_default          ← all of them
```

A subtler failure worth knowing: `WHERE occurred_at >= now() - interval '7 days'` prunes at *execution* time rather than plan time, so a cached generic plan may include every partition. And `WHERE occurred_at::date = '2024-02-15'` does not prune at all, because the cast makes the expression opaque — the same sargability rule as Part 20, now costing you whole partitions instead of an index.

11 • The summary table instead of a matview.

```
CREATE TABLE daily_revenue (
    order_date date PRIMARY KEY,
    orders      integer NOT NULL,
    revenue     numeric(14,2) NOT NULL,
    updated_at  timestamptz NOT NULL DEFAULT now()
);

-- Run every few minutes: recomputes only the recent window
INSERT INTO daily_revenue (order_date, orders, revenue)
SELECT order_date, count(*), sum(total_amount)
FROM   big_orders
WHERE  status <> 'cancelled'
      AND order_date >= current_date - 2      -- late-arriving rows covered
GROUP BY order_date
ON CONFLICT (order_date) DO UPDATE
SET     orders = EXCLUDED.orders,
        revenue = EXCLUDED.revenue,
        updated_at = now();
```


The two-day window is the design decision: it is wide enough to absorb late-arriving or corrected rows, and narrow enough that the job touches a few thousand rows rather than two million. A full `REFRESH MATERIALIZED VIEW` recomputes three years of history every run to update today's number. More code, and typically a thousandfold less work — which is why production reporting is usually built this way rather than on materialized views.

Challenge — worked

1 · 400 GB with 60% bloat, no downtime. `VACUUM FULL` is out — it holds an `ACCESS EXCLUSIVE` lock for hours. The plan: **(i)** First find out *why* it bloated, because otherwise you will do this again — check for a long-running transaction, an autovacuum starved by too-conservative `autovacuum_vacuum_cost_delay`, or an update pattern rewriting whole rows. **(ii)** Rebuild the indexes first with `REINDEX CONCURRENTLY`; index bloat is frequently most of the total and this is safe online. **(iii)** For the heap, use `pg_repack`, which rebuilds the table into a new file using triggers to capture concurrent changes and takes only a brief exclusive lock at the swap. **(iv)** If `pg_repack` is not available, and the table is time-based, the alternative is to partition it going forward and let the bloated old data age out — trading a one-off operation for a gradual one. **(v)** Then fix the cause: per-table `autovacuum_vacuum_scale_factor = 0.02`, `idle_in_transaction_session_timeout`, and monitoring on `n_dead_tup`. Doing (v) first is defensible: there is no point repacking a table that will bloat again next week.

3 · The 12-second dashboard. Three approaches, and the diagnosis comes first: 40 queries at 300 ms each is a different problem from 2 queries at 6 seconds. Check whether they run sequentially — often the single biggest win is issuing them concurrently, which costs nothing on the database side. **(a) Pre-aggregate.** Summary tables refreshed every few minutes, as in practice 11. The dashboard becomes 40 primary-key lookups and renders in under 100 ms. Trade-off: bounded staleness, and a job to operate. This is the right answer for almost every dashboard. **(b) One query instead of 40.** Many dashboard tiles are the same scan with different filters, so `FILTER` and `GROUPING SETS` (Volume II §8.6–8.7) can collapse them into one pass. Trade-off: a complex query and coupling between tiles. **(c) Route to a replica or an analytical store.** Trade-off: does not make it faster, only stops it hurting the transactional workload — which may be the actual requirement. Combine (a) with (c): pre-aggregate on a replica, serve the summaries from the primary.

4 · Stale value immediately after saving. Three causes, distinguished by where you look. **(i) Replication lag.** The write went to the primary, the subsequent read went to a replica that had not yet applied it. Fixes: route reads-after-write to the primary for a short window; use PostgreSQL's `pg_wal_lsn` to wait for the replica to catch up to the LSN of your write; or return the written value from the write request so no read is needed. **(ii) An application or HTTP cache** serving the pre-write value. Fix: invalidate on write, or include a version in the cache key. **(iii) The read happened inside a `REPEATABLE READ` transaction opened before the write committed** — the snapshot is older than the change, correctly. Fix: end the transaction, or do not hold one open across the request. The diagnostic order is worth stating: check for a replica in the read path first, because it is the most common and the easiest to confirm with `pg_last_xact_replay_timestamp()`.

5 · Write saturation. Replicas do not help because **every replica replays every write** — adding one adds write work to the system rather than removing it. What does help, in order: **(i) Reduce the writes.** Batch inserts instead of row-at-a-time, drop unused indexes (every index is a write multiplier — Part 20 §20.8), and stop writing things you do not need such as an `updated_at` touch on every read. This is usually where the whole problem is. **(ii) Make each write cheaper.** Check `n_tup_hot_upd` — if HOT updates have

collapsed, an index on a hot column is forcing full index maintenance on every update; a lower `fillfactor` can restore it. **(iii) Move write load off the critical path.** Queue non-essential writes (analytics events, audit rows) and insert them in batches, or send them to a separate database. **(iv) Faster storage and more WAL throughput** — vertical scaling, which is unglamorous and often the correct answer. **(v) Sharding**, last: it is the only thing that truly scales writes horizontally, and it costs cross-shard joins, cross-shard transactions and global uniqueness. Say the ladder in that order and the answer is complete.

Views

A view is a stored query that behaves like a table. It is the cheapest abstraction in SQL — free at runtime, and the right way to give a name to a query everybody keeps rewriting slightly differently.

23.1 Views, 23.2 updatable views

```
CREATE VIEW active_customers AS
SELECT customer_id, full_name, email, city
FROM   customers
WHERE  deleted_at IS NULL;

CREATE OR REPLACE VIEW order_summary AS
SELECT o.order_id, o.order_date, o.status,
       c.full_name           AS customer,
       count(oi.product_id)  AS line_count,
       sum(oi.quantity * oi.unit_price) AS computed_total
FROM   orders o
JOIN   customers c    ON c.customer_id = o.customer_id
LEFT   JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY o.order_id, o.order_date, o.status, c.full_name;

SELECT * FROM order_summary WHERE status = 'delivered';
```

A view stores no data. Every query against it is expanded into the underlying query by the rewriter, so `WHERE status = 'delivered'` is pushed into the view's own plan — a view costs nothing extra, and it is **not** a performance technique. If you need stored results, that is a materialized view (§22.6).

UPDATABLE VIEWS

A **simple** view — one table, no aggregates, no `DISTINCT`, no `GROUP BY`, no set operations, no window functions — is automatically updatable. `active_customers` above accepts `INSERT`, `UPDATE` and `DELETE`; `order_summary` does not.

```
UPDATE active_customers SET city = 'Pune' WHERE customer_id = 1;  -- works

-- WITH CHECK OPTION: refuse writes that would fall outside the view
CREATE VIEW mumbai_customers AS
SELECT * FROM customers WHERE city = 'Mumbai'
WITH   CHECK OPTION;

INSERT INTO mumbai_customers (customer_id, full_name, email, city)
VALUES (20, 'Test', 't@example.com', 'Delhi');
-- ERROR: new row violates check option for view "mumbai_customers"
```

Without `WITH CHECK OPTION` that insert succeeds and the row promptly vanishes from the view — a genuinely confusing bug. For a complex view, an `INSTEAD OF` trigger (Part 25) can make it writable by hand.

BEST PRACTICE

Views earn their place in three situations: hiding a soft-delete or tenant filter that every query must apply and someone will eventually forget; giving a stable interface over a schema you intend to change; and exposing a subset of columns as a security boundary. They are a poor choice for deeply nested layers — a view on a view on a view produces plans nobody can read and filters that no longer push down. Two levels is a reasonable limit.

NOTE · SECURITY

`GRANT SELECT` on a view without granting anything on the base table gives a role access to exactly the rows and columns the view exposes. By default the view runs with the *definer's* privileges, which is what makes this work. PostgreSQL 15+ adds `security_invoker = true` for the opposite behaviour, and row-level security (Part 28) is the more precise tool when the filter depends on who is asking.

Functions & Procedures

Code that runs inside the database. Used well it eliminates round trips and puts invariants where nothing can bypass them; used badly it becomes business logic nobody can test, version or debug.

24.1 SQL functions, 24.2 volatility

```
CREATE OR REPLACE FUNCTION order_total(p_order_id bigint)
RETURNS numeric
LANGUAGE sql
STABLE                                -- see the table below: this matters
AS $$
    SELECT coalesce(sum(quantity * unit_price), 0)
    FROM    order_items
    WHERE   order_id = p_order_id;
$$;

SELECT order_id, order_total(order_id) FROM orders;

-- Returning a set: usable directly in FROM
CREATE OR REPLACE FUNCTION customer_orders(p_customer_id bigint)
RETURNS TABLE (order_id bigint, order_date date, total numeric)
LANGUAGE sql STABLE AS $$
    SELECT o.order_id, o.order_date, o.total_amount
    FROM    orders o WHERE o.customer_id = p_customer_id
    ORDER  BY o.order_date DESC;
$$;

SELECT * FROM customer_orders(1);
```

Volatility	Promise	Consequence
IMMUTABLE	Same arguments always give the same result; reads no tables	Can be used in an expression index; evaluated once per query
STABLE	Constant within one statement; may read tables	Can be used in an index scan condition
VOLATILE (default)	May return anything, may have side effects	Re-evaluated per row; blocks many optimisations

COMMON MISTAKE

Leaving a function `VOLATILE` by omission. The planner must then call it once per row and cannot push it into an index condition, so `WHERE my_func(x) = 1` becomes a full scan with a function call per row. Declaring the correct volatility is frequently a 100× difference. Also declare `PARALLEL SAFE` where true, or the function silently disables parallel query for the whole statement.

24.3 PL/pgSQL – variables,

control flow, exceptions

```
CREATE OR REPLACE FUNCTION place_order(
    p_customer_id bigint,
    p_product_id  bigint,
    p_quantity    integer
) RETURNS bigint
LANGUAGE plpgsql
AS $$
DECLARE
    v_order_id bigint;
    v_price    numeric(10,2);
    v_stock    integer;
BEGIN
    IF p_quantity <= 0 THEN
        RAISE EXCEPTION 'quantity must be positive, got %', p_quantity
            USING ERRCODE = 'check_violation';
    END IF;

    SELECT price, in_stock INTO v_price, v_stock
    FROM   products WHERE product_id = p_product_id FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'product % does not exist', p_product_id;
    END IF;

    IF v_stock < p_quantity THEN
        RAISE EXCEPTION 'insufficient stock: % available, % requested',
            v_stock, p_quantity;
    END IF;

    INSERT INTO orders (customer_id, order_date, status, total_amount)
    VALUES (p_customer_id, current_date, 'pending', v_price * p_quantity)
    RETURNING order_id INTO v_order_id;

    INSERT INTO order_items (order_id, product_id, quantity, unit_price)
    VALUES (v_order_id, p_product_id, p_quantity, v_price);

    UPDATE products SET in_stock = in_stock - p_quantity
    WHERE  product_id = p_product_id;

    RETURN v_order_id;

EXCEPTION
    WHEN unique_violation THEN
        RAISE NOTICE 'duplicate detected: %', SQLERRM;
        RETURN NULL;
```

```
END;  
$$;
```

Five PL/pgSQL essentials appear there: `DECLARE` for locals, `SELECT ... INTO` to capture a row, `IF NOT FOUND` to test whether it matched, `RAISE EXCEPTION ... USING ERRCODE` to fail with a catchable code, and an `EXCEPTION` block to handle specific SQLSTATEs.

NOTE · FUNCTIONS VERSUS PROCEDURES

A **function** runs inside the calling transaction and cannot `COMMIT`. A **procedure** (PostgreSQL 11+, called with `CALL`) can commit and start new transactions — which is what makes batched maintenance jobs possible:

```
CREATE PROCEDURE purge_old_events(p_before date)  
LANGUAGE plpgsql AS $$  
BEGIN  
    LOOP  
        DELETE FROM events WHERE event_id IN (  
            SELECT event_id FROM events WHERE occurred_at < p_before LIMIT 10000);  
        EXIT WHEN NOT FOUND;  
        COMMIT;                -- only a procedure may do this  
    END LOOP;  
END $$;  
CALL purge_old_events('2023-01-01');
```

COMMON MISTAKE · THE EXCEPTION BLOCK THAT COSTS YOU

Every `BEGIN ... EXCEPTION` block creates a subtransaction. Inside a loop over a million rows that is a million subtransactions, and performance collapses past roughly 64 nested ones (Volume IV §18.3). Put the exception handler *outside* the loop, or handle failure with `ON CONFLICT` instead — which is not an exception at all and costs nothing.

BEST PRACTICE · WHEN TO PUT LOGIC IN THE DATABASE

Yes for: invariants that must hold for every writer (a stock decrement that must never go negative); operations needing many round trips (a loop that would be N+1 from the application); and set-based data manipulation, which SQL does better than any application language. **No** for: business rules that change with product decisions, anything needing external calls, and anything your team cannot version-control, test and review like ordinary code. The deciding question is not capability but *ownership* — logic in the database is invisible to application tests and to most developers, so put only what genuinely belongs there.

Triggers

Code that fires automatically on a data change. Triggers are the right answer for audit trails and maintained aggregates, and the wrong answer for almost everything else — because they run invisibly, and code that runs invisibly is code nobody remembers when debugging.

25.1 Anatomy, 25.2 audit logging

Choice	Options	Use
Timing	BEFORE / AFTER / INSTEAD OF	BEFORE to modify or reject the row; AFTER to react once it is written; INSTEAD OF only on views
Event	INSERT / UPDATE / DELETE / TRUNCATE	Combine with OR ; UPDATE OF col narrows to specific columns
Level	FOR EACH ROW / FOR EACH STATEMENT	Row level for per-row work; statement level for one action per statement

Inside a row-level trigger you get `NEW` (the incoming row), `OLD` (the previous one), `TG_OP` (which operation), and `TG_TABLE_NAME`. A `BEFORE` trigger returning `NULL` cancels the operation silently; returning `NEW` (possibly modified) lets it proceed.

```

-- 1. The universally useful trigger: maintain updated_at
CREATE OR REPLACE FUNCTION set_updated_at() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    NEW.updated_at := now();
    RETURN NEW;
END $$;

CREATE TRIGGER orders_set_updated_at
BEFORE UPDATE ON orders
FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- 2. Audit logging: one generic function for every table
CREATE TABLE audit_log (
    audit_id    bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    table_name  text NOT NULL,
    operation   text NOT NULL,
    row_pk      text,
    old_row     jsonb,
    new_row     jsonb,
    changed_by  text NOT NULL DEFAULT current_user,
    changed_at  timestamptz NOT NULL DEFAULT now()
);

CREATE OR REPLACE FUNCTION audit_changes() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO audit_log (table_name, operation, old_row, new_row)
    VALUES (TG_TABLE_NAME,
            TG_OP,
            CASE WHEN TG_OP <> 'INSERT' THEN to_jsonb(OLD) END,
            CASE WHEN TG_OP <> 'DELETE' THEN to_jsonb(NEW) END);
    RETURN NULL;
END $$;

CREATE TRIGGER employees_audit
AFTER INSERT OR UPDATE OR DELETE ON employees
FOR EACH ROW EXECUTE FUNCTION audit_changes();

```

`to_jsonb(OLD)` is what makes one function serve every table — no column list to maintain, and a new column is captured automatically. This single pattern is the most valuable trigger in the book.

25.3 Maintained aggregates, 25.4 pitfalls

```
-- Keep orders.total_amount correct (the denormalization of Volume IV §16.7)
CREATE OR REPLACE FUNCTION sync_order_total() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE v_order_id bigint := coalesce(NEW.order_id, OLD.order_id);
BEGIN
    UPDATE orders o
    SET     total_amount = coalesce((SELECT sum(quantity * unit_price)
                                     FROM order_items WHERE order_id = v_order_id), 0)
    WHERE  o.order_id = v_order_id;
    RETURN NULL;
END $$;

CREATE TRIGGER order_items_sync_total
AFTER INSERT OR UPDATE OR DELETE ON order_items
FOR EACH ROW EXECUTE FUNCTION sync_order_total();
```

Because the trigger runs in the same transaction as the change, the aggregate can never be half-updated — which is the property application code cannot provide. It still needs the nightly reconciliation query of Volume IV §16.7, because a data-repair script or a disabled trigger will eventually cause drift.

Pitfall	Detail
Invisible cost	A row trigger fires per row. A 500,000-row bulk update runs it 500,000 times, and this is why an import that took 20 seconds now takes 20 minutes.
Hidden behaviour	A developer reads the <code>INSERT</code> and cannot see the three other tables it wrote. Debugging becomes archaeology.
Recursion	A trigger on A updating B, whose trigger updates A. Guard with <code>pg_trigger_depth()</code> or <code>WHEN</code> conditions.
Contention	Every child insert updating one parent row serialises writers on that row — fine normally, a hot spot for a viral record.
Order	Multiple triggers on one event fire in alphabetical order by name. Name them deliberately if order matters.
<code>TRUNCATE</code> bypasses them	Row triggers do not fire on <code>TRUNCATE</code> , so a maintained aggregate silently goes stale.

BEST PRACTICE

Use a `WHEN` clause so the trigger does not even fire when nothing relevant changed: `AFTER UPDATE OF quantity, unit_price ON order_items FOR EACH ROW WHEN (OLD.* IS DISTINCT FROM NEW.*)`. And for bulk work, prefer a statement-level trigger with transition tables (PostgreSQL 10+) — `REFERENCING NEW TABLE AS new_rows FOR EACH STATEMENT` — which lets you do one set-based update instead of N row-level ones. That change alone often recovers the entire cost of a trigger on a bulk load.

INTERVIEW · ADVANCED

Q. When would you use a trigger, and when would you refuse? — “I use them for audit trails, `updated_at` maintenance, and denormalized aggregates that must be transactionally consistent — things that must happen for *every* writer, including a script at 2 a.m. I refuse them for business logic, because the behaviour is invisible at the call site and untestable from the application, and for anything with an external side effect. The cost people underestimate is bulk operations: a per-row trigger turns a fast import into a slow one.”

JSON & Semi-structured Data

PostgreSQL's `jsonb` is a genuine document store inside a relational database. This part covers the operators, the path language, indexing, and — most importantly — where the boundary between a column and a document should sit.

26.1 Operators and extraction

```
CREATE TABLE events (  
  event_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  payload jsonb NOT NULL  
);  
  
INSERT INTO events (payload) VALUES  
( '{"type":"purchase","user":{"id":42,"plan":"pro"},  
  "items":[{"sku":"KB-1","qty":2},{"sku":"MON-27","qty":1}],  
  "total":24300,"tags":["web","promo"]}' );
```

Operator	Returns	Example
->	JSON value (chainable)	payload -> 'user' -> 'plan' → "pro"
->>	Text (comparable, castable)	payload ->> 'type' → purchase
#> / #>>	By path, as JSON / as text	payload #>> '{user,id}' → 42
@> / <@	Contains / is contained by	payload @> '{"type":"purchase"}' → true
? / ? / ?&	Key exists / any / all	payload ? 'total' → true
@? / @@	JSONPath exists / matches	payload @? '\$.items[*] ? (@.qty > 1)'
/ - / #-	Merge / delete key / delete path	payload - 'tags'

```

-- Unnest an array into rows: the workhorse
SELECT e.event_id,
       item ->> 'sku'           AS sku,
       (item ->> 'qty')::integer AS qty
FROM   events e
CROSS JOIN LATERAL jsonb_array_elements(e.payload -> 'items') AS item;

-- Flatten an object into key/value rows
SELECT key, value FROM events, jsonb_each(payload -> 'user');

-- JSONPath: items with quantity above one
SELECT jsonb_path_query_array(payload, '$.items[*] ? (@.qty > 1)') FROM events;

-- Build JSON from relational rows
SELECT jsonb_build_object(
       'customer', c.full_name,
       'orders', coalesce(jsonb_agg(jsonb_build_object(
           'id', o.order_id, 'total', o.total_amount)
       ) FILTER (WHERE o.order_id IS NOT NULL), '[]'::jsonb))
FROM   customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.full_name;

```

COMMON MISTAKE

payload -> 'type' = 'purchase' is false, because -> returns the JSON value "purchase" *with quotes*. Use ->> for comparison, or compare against '"purchase"'::jsonb. Everybody meets this once; the rule is -> **to navigate**, ->> **to compare**.

26.2 Indexing JSONB, 26.3 the design boundary

```
-- GIN, default operator class: supports @>, ?, ?|, ?&, @?, @@
CREATE INDEX idx_events_payload ON events USING gin (payload);

-- jsonb_path_ops: smaller and faster, but supports ONLY @> and the path operators
CREATE INDEX idx_events_payload_path ON events USING gin (payload jsonb_path_ops);

-- B-tree on one extracted field: best when you always filter on the same key
CREATE INDEX idx_events_type ON events ((payload ->> 'type'));

-- Partial + expression: the sharpest tool of all
CREATE INDEX idx_events_purchase_user
  ON events (((payload #>> '{user,id}'))::bigint))
  WHERE payload ->> 'type' = 'purchase';
```

Query shape	Index to build
Containment, varied keys	GIN, <code>jsonb_path_ops</code> if only <code>@></code> is used
Always the same one key	B-tree expression index on that extraction — far smaller than GIN
Range on an extracted number or date	B-tree expression index with a cast; GIN cannot do ranges
Key existence across many keys	GIN, default operator class

THE DESIGN BOUNDARY

Columns for what you know; `jsonb` for what varies. Three tests decide it. **(1)** Does anything *constrain* it — `NOT NULL`, a foreign key, a `CHECK`? Only a column can. **(2)** Do you filter, join or sort on it in more than one shape? A column is cheaper and the planner has real statistics for it. **(3)** Is the set of keys unbounded or per-tenant? Then `jsonb` is right, and a column would be a hundred sparse nullable fields. The failure mode to avoid is the table that is one `jsonb` column: you have built a document database with none of a document database's tooling, and no constraint can protect it (Volume IV §16.2).

COMMON MISTAKE · THE PLANNER CANNOT SEE INSIDE JSON

PostgreSQL keeps no per-key statistics for a `jsonb` column, so it estimates `WHERE payload ->> 'status' = 'active'` with a fixed default guess — typically 0.5% — regardless of the truth. That misestimate propagates into join choices and produces exactly the nested-loop disaster of Part 21 §21.5. Creating an **expression index** fixes both problems at once: it makes the predicate sargable *and* gives `ANALYZE` something to gather statistics on. That second effect is the one people do not know about, and it is often the larger win.

INTERVIEW · ADVANCED

Q. When would you use JSONB instead of columns? — “For genuinely variable structure: third-party API payloads I store whole, per-tenant custom fields, event metadata whose keys I cannot enumerate. Not for anything I constrain, join on, or query in more than one shape — those want columns, because only a column gets constraints and real planner statistics. In practice I use both in one table: known fields as columns, a `jsonb` column beside them for the rest.” **Interview tip:** add that you index JSONB with expression indexes rather than GIN when the access pattern is a single known key — it shows you have tuned it, not just used it.

Parts 23–26 Review

Quick Check ● BEGINNER

- 1 Does a view improve performance? Explain.
- 2 What makes a view automatically updatable?
- 3 What does `WITH CHECK OPTION` prevent?
- 4 Name the three volatility classes and what each permits.
- 5 What can a procedure do that a function cannot?
- 6 Why is an `EXCEPTION` block inside a large loop expensive?
- 7 What does a `BEFORE` trigger returning `NULL` do?
- 8 Why does `to_jsonb(OLD)` make one audit function serve every table?
- 9 In what order do several triggers on one event fire?
- 10 Why does a maintained aggregate go stale after `TRUNCATE` ?
- 11 Difference between `->` and `->>` , and which do you compare with?
- 12 What does `jsonb_path_ops` trade away?
- 13 Give two reasons an expression index on a JSON key helps.
- 14 Give the three tests for column versus `jsonb` .

Practice ● EASY

- 1 Create `active_customers` with soft deletes, update through it, and add `WITH CHECK OPTION` to a city-filtered view.
- 2 Create `order_summary` and confirm it is not updatable. Explain which clause makes it so.
- 3 Write `order_total()` as a `STABLE` SQL function, then as `VOLATILE` , and compare the plans of a query using it.
- 4 Write a set-returning function for a customer's orders and use it in `FROM` .
- 5 Implement `place_order()` and test all three failure paths.
- 6 Write a batched purge procedure with `COMMIT` inside a loop and run it.
- 7 Add `updated_at` to `orders` with a `BEFORE UPDATE` trigger; prove it fires.
- 8 Build the generic `audit_log` and attach it to `employees` . Make three changes and read the log.
- 9 Add the `sync_order_total` trigger, then break the total with a direct `UPDATE` and show the reconciliation query finding it.

- 10** Add a `WHEN (OLD.* IS DISTINCT FROM NEW.*)` clause and demonstrate the trigger not firing on a no-op update.
- 11** Insert the sample event and extract `type`, the nested user id, and the tags array.
- 12** Unnest `items` into rows with `jsonb_array_elements` and sum the quantities.
- 13** Build a GIN index and an expression index on the same table; write a query that uses each.
- 14** Produce nested JSON output: each customer with an array of their orders, empty array when none.

Challenge ● ADVANCED

- 1** A bulk import of 2 million rows into a table with three row-level triggers takes four hours. Give three ways to make it fast and the risk of each.
- 2** Design an audit system capturing who changed what, where "who" is an application user rather than a database role. How does the trigger learn the application user?
- 3** A per-tenant custom-fields feature is proposed. Compare `jsonb`, a key-value table, and per-tenant columns. Recommend one.
- 4** A query filtering on a JSONB key has a good index but a terrible join plan. Explain and fix.

Parts 23–26 Solutions

Quick Check

- 1 No. It is expanded into the underlying query by the rewriter, so it costs nothing extra and saves nothing. Stored results are a materialized view.
- 2 One base table, no aggregates, no `GROUP BY`, no `DISTINCT`, no set operations, no window functions.
- 3 Writes through the view that would produce a row falling outside the view's own `WHERE` clause — rows that would immediately vanish.
- 4 `IMMUTABLE` — usable in an expression index. `STABLE` — usable in an index scan condition, may read tables. `VOLATILE` — re-evaluated per row, blocks optimisations.
- 5 `COMMIT` and begin new transactions, which is what makes batched loops possible.
- 6 Each block creates a subtransaction; performance collapses past roughly 64 and they accumulate state per iteration.
- 7 Cancels the operation for that row, silently.
- 8 It serialises the whole row generically, so no column list is needed and new columns are captured automatically.
- 9 Alphabetical order by trigger name.
- 10 Row-level triggers do not fire on `TRUNCATE`, so the aggregate is never updated.
- 11 `->` returns JSON (quoted, chainable); `->>` returns text. Compare with `->>`.
- 12 Support for key-existence operators — it handles only `@>` and the path operators, in exchange for being smaller and faster.
- 13 It makes the predicate sargable, and it gives `ANALYZE` statistics on the extracted value, fixing the planner's default guess.
- 14 Does anything constrain it? Do you filter, join or sort on it? Is the key set unbounded? Constrained or queried → column; unbounded → `jsonb`.

Practice — selected

3 · Volatility and the plan. Declared `STABLE`, the planner may evaluate the function once per query where the argument is constant and can use it in an index condition. Declared `VOLATILE`, it must be called for every row and cannot be pushed down:

```
CREATE FUNCTION vol_total(bigint) RETURNS numeric LANGUAGE sql VOLATILE AS
  $$ SELECT coalesce(sum(quantity*unit_price),0) FROM order_items WHERE order_id = $1 $$;

EXPLAIN ANALYZE SELECT order_id FROM big_orders WHERE vol_total(order_id) > 1000;
-- Seq Scan, Filter: (vol_total(order_id) > 1000) ← 2,000,000 function calls
```

The volatility declaration is a *promise you make*, and the planner trusts it — so declaring a function `IMMUTABLE` when it reads a table gives you wrong answers, not just slow ones, because the result may be cached or folded into an index. Declare the strictest class that is truthfully correct.

10 · The `WHEN` clause.

```
CREATE TRIGGER order_items_sync_total
AFTER UPDATE OF quantity, unit_price ON order_items
FOR EACH ROW WHEN (OLD.* IS DISTINCT FROM NEW.*)
EXECUTE FUNCTION sync_order_total();

UPDATE order_items SET quantity = quantity WHERE order_id = 1001;
-- trigger does NOT fire: nothing changed
```

Two filters stacked: `UPDATE OF quantity, unit_price` means an update touching only other columns never fires the trigger at all, and `WHEN (OLD.* IS DISTINCT FROM NEW.*)` skips no-op writes. On a table where an ORM rewrites every column on every save — which is the common case — these two clauses can eliminate the great majority of trigger executions.

14 · Nested JSON output.

```
SELECT jsonb_pretty(jsonb_build_object(
    'customer_id', c.customer_id,
    'name',        c.full_name,
    'orders',      coalesce(
        jsonb_agg(jsonb_build_object(
            'order_id', o.order_id,
            'date',     o.order_date,
            'total',    o.total_amount)
        ORDER BY o.order_date DESC)
        FILTER (WHERE o.order_id IS NOT NULL),
        '[]'::jsonb)))
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id
GROUP  BY c.customer_id, c.full_name;
```

The `FILTER (WHERE o.order_id IS NOT NULL)` is the essential detail: without it, a customer with no orders gets `[null]` — an array containing one null object — rather than an empty array, because the left join produced one all-`NULL` row and `jsonb_agg` aggregated it. The `coalesce` then converts the resulting `NULL` aggregate into `[]`. Both are needed, and this exact construction is how you build an API response in one query instead of N+1.

Challenge — worked

1 · The four-hour import. Three approaches, increasingly good. **(a) Disable the triggers for the load** —

`ALTER TABLE t DISABLE TRIGGER USER;`, load, re-enable, then reconcile whatever the triggers would have maintained. Fast and blunt; the risk is that you must remember to reconcile, and if the load fails halfway the

triggers are still disabled. Wrap it in a transaction so the disable rolls back. **(b) Replace row-level triggers with statement-level triggers using transition tables** — `AFTER INSERT ON t REFERENCING NEW TABLE AS new_rows FOR EACH STATEMENT`, then do one set-based `INSERT ... SELECT FROM new_rows`. This keeps the guarantee and turns 2 million function calls into one query. It is the correct answer where the trigger's work is expressible as a set operation, which audit logging and aggregate maintenance both are. **(c) Load into a staging table, transform, then insert** — the pattern from Volume I §2.8, with the triggers doing one bulk pass at the end. Best when the incoming data also needs cleaning. Recommend (b) as the durable fix and (a) as the one-off.

2 · Auditing the application user. The trigger runs as the database role — usually one shared connection-pool role — so `current_user` is useless. The mechanism is a session variable set by the application at the start of each request or transaction:

```
-- Application, once per transaction (works through PgBouncer transaction mode
-- only if set inside the transaction, which SET LOCAL guarantees):
SET LOCAL app.user_id = '4021';

-- In the trigger function:
INSERT INTO audit_log (table_name, operation, new_row, changed_by_app_user)
VALUES (TG_TABLE_NAME, TG_OP, to_jsonb(NEW),
        nullif(current_setting('app.user_id', true), '')::bigint);
```

Three details carry it. `SET LOCAL` scopes the value to the transaction, so a pooled connection cannot leak one user's identity into another's request — using plain `SET` here is a real security bug. The `true` second argument to `current_setting` makes it return `NULL` instead of raising when unset, so a migration script that forgets it still works. And you should keep `current_user` as well, because a direct `psql` change by an engineer has no app user and that is exactly the event an auditor wants to see.

3 · Per-tenant custom fields. jsonb column: one column, arbitrary keys per tenant, indexable with GIN or per-key expression indexes. No constraints, no per-key statistics, and validation must live in the application or a `CHECK` with a JSON schema function. **Key-value table** (`tenant_id, entity_id, field_key, value text`): fully queryable and constrainable, and every read of five fields becomes five joins or a pivot; it also loses typing unless you add a `value_type` column and cast everywhere. **Per-tenant columns:** genuinely queryable and typed, and it means DDL at runtime driven by user input — hundreds of sparse columns, a migration per customer request, and a table that eventually hits the column limit. **Recommend jsonb**, with two additions that answer its weaknesses: a `tenant_field_definitions` table declaring each tenant's field names, types and required-ness so the application can validate against real metadata; and expression indexes created for the specific fields tenants actually filter on, which recovers both sargability and planner statistics. That combination gets the flexibility of documents with most of the discipline of columns — and it is what production multi-tenant systems converge on.

4 · Good index, terrible join plan. The index makes the *scan* fast, but PostgreSQL gathers **no statistics inside a jsonb column** — so the planner estimates `WHERE payload ->> 'status' = 'active'` at a fixed default of roughly 0.5%, whatever the reality. If 60% of rows actually match, that 120× underestimate

propagates upward: the planner believes the node returns a handful of rows and chooses a nested loop for the join above it, which then executes its inner side hundreds of thousands of times. The fix is the same expression index, understood correctly:

```
CREATE INDEX idx_events_status ON events ((payload ->> 'status'));  
ANALYZE events;    -- ← the step that actually fixes the join
```

An expression index causes `ANALYZE` to collect a histogram and most-common-values list *for the expression*, exactly as if it were a column. After that the estimate is accurate and the join method changes on its own. This is the key insight of §26.3's warning box: the expression index's statistical side effect is frequently worth more than its access-path effect, and it is why "I added a GIN index and the query is still slow" is such a common report — GIN makes containment fast but gives the planner nothing to estimate with.

Where you are, and what comes next

You can now diagnose a slow query instead of guessing at it, which is the skill that distinguishes a senior engineer from a competent one. Confirm you can do these six things:

1. Read an `EXPLAIN ANALYZE` plan and name the node where the time goes.
2. Explain what a large gap between estimated and actual rows means, and give three causes.
3. Design a composite index for a given query and justify every column's position.
4. Explain why one idle transaction can bloat the whole database.
5. Recognise an N+1 from `pg_stat_statements` alone.
6. State the three tests for a column versus a `jsonb` field.

NEXT

Volume VI — Parts 27 to 31. SQL in application development (parameterised statements, pooling, ORMs, transaction boundaries in services); security (injection anatomy, roles and privileges, least privilege, row-level security); administration concepts (backup, restore, replication, failover, monitoring); partitioning in depth; and data warehousing — OLTP versus OLAP, fact and dimension tables, star and snowflake schemas, slowly changing dimensions.

Say the word and I will continue.



SQL MASTERY

Volume VI — Production Systems

Parts 27 to 31. How SQL reaches your application, how it is attacked and defended, how it is operated, and how it is shaped differently when the question is analytical rather than transactional.

-
- | | |
|-----------|--|
| 27 | SQL & Application Development — parameterised statements, pooling, ORMs, transaction boundaries |
| 28 | Security — injection anatomy, roles and privileges, least privilege, row-level security |
| 29 | Administration Concepts — backup, restore, replication, failover, monitoring |
| 30 | Partitioning — strategies, pruning, indexes, automated maintenance |
| 31 | Data Warehousing — OLTP vs OLAP, fact and dimension tables, star and snowflake, SCDs |
-

Part 28 is the one part of this book with security consequences. Read §28.1 and §28.2 in full even if you skim the rest.

SQL & Application Development

Almost all SQL in the world is issued by application code, not typed by a person. This part covers the boundary: how a query gets to the database safely, how connections are managed, and what an ORM does for you and to you.

IN THIS PART

27.1 Parameterised queries

27.2 Prepared statements

27.3 Drivers and pooling by language

27.4 ORMs — what they cost

27.5 Transaction boundaries in services

27.6 Migrations · Review

27.1 Parameterised queries

The single most important line of code in this book. **Never build SQL by concatenating values.** Pass values as parameters and the driver sends them separately from the statement text, so they can never be parsed as SQL.

```
# WRONG – string interpolation. This is SQL injection (§28.1).
cur.execute(f"SELECT * FROM users WHERE email = '{email}'")

# RIGHT – the value travels out of band; the query text never changes
cur.execute("SELECT * FROM users WHERE email = %s", (email,))
```

Note what makes this safe: it is not escaping. The statement is parsed once with a placeholder where the value goes, and the value is then bound to that slot as data. There is no string for an attacker's quote character to escape from.

Language	Placeholder	Example
Python (psycopg)	%s or %(name)s	cur.execute(sql, (a, b))
Java (JDBC)	?	ps.setString(1, email)
Node (pg)	\$1, \$2	client.query(sql, [a, b])
Go (pgx)	\$1, \$2	conn.Query(ctx, sql, a, b)
PHP (PDO)	? or :name	\$stmt->execute([\$email])
C# (Npgsql)	@name	cmd.Parameters.AddWithValue(...)

COMMON MISTAKE · THE THREE THINGS YOU CANNOT PARAMETERISE

Placeholders work for **values** only. A table name, a column name, or a sort direction cannot be a parameter — the parser needs them to build the plan. So a user-chosen sort column must be validated against a whitelist:

```
ALLOWED_SORTS = {"date": "order_date", "amount": "total_amount", "id": "order_id"}
col = ALLOWED_SORTS.get(request.args.get("sort"), "order_id")    # never the
raw input
direction = "DESC" if request.args.get("dir") == "desc" else "ASC"
sql = f"SELECT * FROM orders ORDER BY {col} {direction} LIMIT %s" # safe: values from
a fixed map
```

The `f`-string is acceptable here *only* because `col` and `direction` can only ever be one of the literals you wrote. Map, do not sanitise.

BEST PRACTICE · IN LISTS

`WHERE id IN (%s)` with a list does not work — you cannot bind a list to a single placeholder. In PostgreSQL use an array instead: `WHERE id = ANY(%s)` with a Python list or a Postgres array. One placeholder, any number of values, no dynamic SQL, and the plan is cached because the statement text is constant regardless of how many ids you pass.

27.2 Prepared statements, 27.3 pooling

A **prepared statement** is parsed and planned once, then executed many times with different parameters. It saves parsing and planning time on a hot query.

```
PREPARE order_by_customer (bigint) AS
    SELECT order_id, order_date, total_amount FROM orders WHERE customer_id = $1;

EXECUTE order_by_customer(1);
DEALLOCATE order_by_customer;
```

Most drivers do this for you. The behaviour worth knowing is PostgreSQL's **generic plan** switch: after five executions it may build one plan that ignores the actual parameter values, which is disastrous for a skewed column where the right plan differs per value (Volume V §21, challenge 3).

```
-- Diagnose and control it
SET plan_cache_mode = force_custom_plan; -- always re-plan with real values
SET plan_cache_mode = auto;              -- the default
SHOW plan_cache_mode;
```

COMMON MISTAKE · PREPARED STATEMENTS THROUGH A POOLER

Server-side prepared statements are per *connection*. Under PgBouncer in transaction mode, your next transaction may land on a different backend that has never heard of your statement, producing `prepared statement "s1" does not exist`. Fixes: PgBouncer 1.21+ tracks them; or disable server-side preparation in the driver (psycopg's `prepare_threshold=None`, pgx's `QueryExecModeSimpleProtocol`); or use session mode and give up most of the pooling benefit.

POOL CONFIGURATION THAT MATTERS

Setting	Sensible value	Why
Max pool size	10–30 per application process	Volume V §22.7 — beyond core count, throughput falls
Connection timeout	2–5 s	Fail fast rather than queue forever under load
Statement timeout	Per workload: 5 s web, minutes for reports	A runaway query cannot hold locks indefinitely
Idle-in-transaction timeout	Server-side, 1–5 min	One leaked transaction blocks vacuum database-wide
Max connection lifetime	30 min	Recycles connections so a failover or DNS change is picked up

```
-- Set timeouts per role, so they apply however the application connects
ALTER ROLE app_web SET statement_timeout = '5s';
ALTER ROLE app_web SET idle_in_transaction_session_timeout = '60s';
ALTER ROLE app_reports SET statement_timeout = '10min';
```

27.4 ORMs — what they cost

An ORM maps rows to objects and writes your SQL. It is genuinely useful for CRUD and genuinely dangerous when you stop knowing what SQL it emits.

ORM problem	What it looks like	Fix
N+1 queries	Lazy loading inside a loop; 500 fast queries per page	Eager loading — <code>select_related</code> , <code>includes</code> , <code>joinedload</code> , <code>JOIN FETCH</code>
<code>SELECT *</code> always	Every column fetched, TOAST included	<code>.only()</code> / <code>.select()</code> projections
Read-modify-write	<code>obj.count += 1; obj.save()</code> — lost updates (Volume IV §19.5)	Atomic expressions — Django's <code>F()</code> , Rails' <code>increment!</code>
Whole-row updates	<code>save()</code> writes all columns, defeating HOT updates and firing triggers	<code>update_fields</code> , dirty tracking
Opaque transactions	A request-wide transaction holding locks across external calls	Explicit, narrow transaction blocks (§27.5)
Migration surprises	An auto-generated migration that rewrites a 200 M-row table	Read every generated migration before it ships

BEST PRACTICE

Turn on SQL logging in development and *read it*. Every ORM has a one-line setting for this, and it is the difference between using an ORM and being used by one. Then add a test that asserts the query count for your three hottest endpoints — that single assertion catches the N+1 regression the moment someone adds an attribute access inside a loop.

NOTE · WHEN TO DROP TO SQL

Use the ORM for CRUD and simple lookups, where it removes real boilerplate. Write SQL by hand for reports, window functions, recursive CTEs, bulk operations and anything you have had to optimise — every ORM has an escape hatch for raw SQL, and using it is not a defeat. The queries in Volume III are the ones you write by hand.

27.5 Transaction boundaries, 27.6 migrations

Volume IV §18.4 gave the three rules: keep transactions short, never do I/O inside one, one transaction per unit of business work. In service code that translates into a specific shape.

```
# WRONG: a request-scoped transaction spanning an external call
@transaction.atomic                                # opens at request start, closes at response
def checkout(request):
    order = Order.objects.create(...)
    charge = stripe.Charge.create(...) # 800 ms with locks held; and if it times
    order.status = "paid"              # out, you do not know if it succeeded
    order.save()

# RIGHT: three short transactions with the external call between them
def checkout(request):
    with transaction.atomic():            # tx 1: durable intent
        order = Order.objects.create(status="pending",
                                      idempotency_key=key)
    charge = stripe.Charge.create(..., idempotency_key=key) # no transaction open
    with transaction.atomic():          # tx 2: record the outcome
        Order.objects.filter(id=order.id, status="pending") \
            .update(status="paid", gateway_ref=charge.id)
```

The `'pending'` row is the point: a crash between the transactions leaves a record to reconcile rather than nothing at all. Combined with the gateway's idempotency key and a `UNIQUE` constraint on it, a retry cannot double-charge (Volume IV §19, challenge 3).

MIGRATIONS THAT DO NOT CAUSE AN OUTAGE

Operation	Safe?	Do instead
Add a nullable column	Safe	—
Add a column with a constant default	Safe on PG 11+	Older versions: add nullable, backfill, set default
Add <code>NOT NULL</code>	Blocks writes	The six-step recipe of Volume IV §17, challenge 3
Add an index	Blocks writes	<code>CREATE INDEX CONCURRENTLY</code> , outside a transaction
Add a foreign key or <code>CHECK</code>	Full scan	<code>NOT VALID</code> , then <code>VALIDATE CONSTRAINT</code>
Change a column type	Full rewrite	New column, backfill in batches, swap, drop
Rename a column	Instant, but breaks running code	Expand/contract: add new, dual-write, migrate readers, drop old
Drop a column	Instant	Deploy the code that stops using it <i>first</i>

BEST PRACTICE · EXPAND AND CONTRACT

Any schema change that is visible to running code needs two deploys, not one. **Expand:** add the new structure, write to both, read from the old. **Migrate:** backfill, switch readers to the new. **Contract:** stop writing the old, then drop it. Three steps instead of one, and it is the only way to change a schema without a coordinated stop of every process that talks to it.

INTERVIEW · ADVANCED

Q. How do you add a column to a large table in production without downtime? — “Add it nullable, which is instant. Backfill in batches so no single transaction holds locks. Add a `NOT VALID` check, validate it without blocking, then set `NOT NULL` cheaply. And I deploy the code that tolerates the column being absent before the migration, and the code that requires it after — expand and contract, so no deploy order can break.” **Interview tip:** mentioning the deploy ordering is what separates this from a memorised DDL recipe.

SQL Security

SQL injection has been the top-ranked application vulnerability for two decades, and it is completely preventable by one habit. This part covers that habit, then the privilege model that limits the damage when something else goes wrong.

IN THIS PART

28.1 Injection, anatomically

28.2 Defence — and non-defences

28.3 Roles, users and privileges

28.4 Least privilege in practice

28.5 Row-level security

28.6 Data protection · Review

28.1 Injection, anatomically

Injection happens when data is parsed as code. The mechanism is worth seeing precisely once, because understanding it is what makes the defence obvious rather than ritual.

```
# The vulnerable code
sql = "SELECT * FROM users WHERE email = '" + email + "'"

# Normal input: asha@example.com
#  SELECT * FROM users WHERE email = 'asha@example.com'

# Attacker input: ' OR '1'='1
#  SELECT * FROM users WHERE email = '' OR '1'='1'           ← every row returned

# Attacker input: '; DROP TABLE users; --
#  SELECT * FROM users WHERE email = ''; DROP TABLE users; --

# Attacker input: ' UNION SELECT id, password_hash, NULL, NULL FROM admins --
#  → the login form now returns password hashes in the user list
```

Class	How it works
Classic / in-band	Results come back in the response — <code>UNION</code> to read other tables
Error-based	The attacker reads data out of error messages that leak values or schema
Blind boolean	No output, but the page differs for true and false — one bit at a time
Blind time-based	<code>' OR (SELECT pg_sleep(5)) IS NOT NULL --</code> — the response delay is the answer
Second-order	The payload is stored innocuously, then interpolated later by a different query — a report or a batch job

SECOND-ORDER IS THE ONE PEOPLE MISS

A registration form stores the display name `Bob'); DROP TABLE sessions; --` safely via a parameterised insert. Six months later, a nightly report builds SQL by concatenating display names into a string. The vulnerability is in the report, and no amount of validation at the form would have helped — because the value was legitimate data. **Every** query must be parameterised, not just the ones handling direct user input.

28.2 Defence — and non-defences

Measure	Effective?	Why
Parameterised queries	Yes — complete	Values never enter the statement text, so nothing can be parsed as SQL
Whitelisting identifiers	Yes	The only safe way to accept a user-chosen column or sort order
Least privilege	Yes — limits damage	Does not prevent injection; means the injected query can do less
Escaping quotes by hand	No	Encoding, multi-byte characters and numeric contexts all defeat it. Every hand-rolled escaper has been broken.
Blocking keywords	No	Trivially bypassed by comments, case and encoding; also blocks legitimate input like the surname O'Brien
Input validation alone	No	Good defence in depth, but a legitimate value can still be a payload — see second-order
Stored procedures	Only if parameterised	A procedure that concatenates its arguments into dynamic SQL is just as vulnerable
A web application firewall	Partial	Buys time against known patterns; not a fix

```
-- Safe dynamic SQL, when you genuinely need it: quote identifiers properly
CREATE OR REPLACE FUNCTION count_rows(p_schema text, p_table text)
RETURNS bigint LANGUAGE plpgsql AS $$
DECLARE v_count bigint;
BEGIN
    EXECUTE format('SELECT count(*) FROM %I.%I', p_schema, p_table) INTO v_count;
    RETURN v_count;
END $$;
-- %I quotes an identifier safely
-- %L would quote a literal; never use %s
```

THE RULE, IN ONE LINE

Values as parameters; identifiers from a whitelist; nothing else concatenated, ever. That is the whole of injection defence. Everything else in this part reduces the blast radius of a different mistake.

INTERVIEW · INTERMEDIATE

Q. How do you prevent SQL injection? — “Parameterised queries, everywhere, including in code that handles data which originally came from a trusted source — second-order injection is exactly that case. Identifiers cannot be parameterised, so a user-chosen sort column is mapped through a fixed whitelist. And least privilege as defence in depth, so the application role cannot `DROP` anything even if something slips.” **Common wrong answer:** escaping or stripping quotes — say explicitly that this is not a defence and why.

28.3 Roles and privileges

PostgreSQL has one concept — the **role** — which acts as a user when it can log in and as a group when other roles are granted membership of it.

```
-- Group roles carry the privileges; login roles inherit them
CREATE ROLE app_read NOLOGIN;
CREATE ROLE app_write NOLOGIN;

GRANT CONNECT ON DATABASE sqlmastery TO app_read;
GRANT USAGE ON SCHEMA core TO app_read;
GRANT SELECT ON ALL TABLES IN SCHEMA core TO app_read;

GRANT app_read TO app_write; -- write inherits read
GRANT INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA core TO app_write;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA core TO app_write; -- needed for IDENTITY

-- Crucially: cover tables created in the FUTURE
ALTER DEFAULT PRIVILEGES IN SCHEMA core
    GRANT SELECT ON TABLES TO app_read;
ALTER DEFAULT PRIVILEGES IN SCHEMA core
    GRANT INSERT, UPDATE, DELETE ON TABLES TO app_write;

-- Login roles, one per workload
CREATE ROLE app_web LOGIN PASSWORD '...' IN ROLE app_write;
CREATE ROLE app_reports LOGIN PASSWORD '...' IN ROLE app_read;
CREATE ROLE analyst_ravi LOGIN PASSWORD '...' IN ROLE app_read;

-- Column-level grants, when a role must not see one field
GRANT SELECT (customer_id, full_name, city) ON customers TO app_reports;
REVOKE SELECT ON customers FROM app_reports; -- remove the table-wide grant first
```

COMMON MISTAKE · THE `PUBLIC` SCHEMA

Before PostgreSQL 15, every user could create objects in the `public` schema of any database they could connect to — which combined with a permissive `search_path` lets an attacker shadow a function you call (Volume I §2.6, solution 10). On any version, run `REVOKE CREATE ON SCHEMA public FROM PUBLIC;` and put your objects in a named schema.

DIALECT

MySQL: a user is a name *and* a host — `'app'@'10.0.%'` — and privileges are granted per database or table with `GRANT ... ON db.*`. **SQL Server:** a server-level *login* maps to a database-level *user*, with roles at both levels. **Oracle:** a user *is* a schema, and `GRANT CREATE SESSION` is required before the account can even connect.

28.4 Least privilege, 28.5 row-level security

Role	Should be able to	Should not
Web application	<code>SELECT</code> , <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> on its own schema	DDL, <code>TRUNCATE</code> , read other schemas, create roles
Reporting / BI	<code>SELECT</code> only, ideally on a replica, with a long <code>statement_timeout</code>	Write anything; see PII columns
Migration runner	DDL on its schema — used only by CI, never by the app	Be the same role the application connects as
Analyst (human)	<code>SELECT</code> on curated views	Read base tables directly
Superuser	Break-glass only, audited	Be used by any automated process

ROW-LEVEL SECURITY

RLS attaches a predicate to a table so a role sees only its own rows — enforced by the database, so it cannot be forgotten in one query out of four hundred. It is the correct foundation for multi-tenancy.

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;

CREATE POLICY orders_tenant_isolation ON orders
    USING      (tenant_id = current_setting('app.tenant_id', true)::bigint)
    WITH CHECK (tenant_id = current_setting('app.tenant_id', true)::bigint);

-- The application sets the tenant per transaction. SET LOCAL, never SET --
-- a pooled connection would otherwise leak one tenant's context into another's.
BEGIN;
SET LOCAL app.tenant_id = '42';
SELECT * FROM orders;      -- only tenant 42's rows exist, as far as this query knows
COMMIT;
```

`USING` filters what is visible to reads; `WITH CHECK` constrains what may be written — without it, a tenant could insert a row belonging to another tenant and then be unable to see it. Both are needed.

COMMON MISTAKE

RLS is **bypassed by the table owner and by superusers** unless you add `FORCE ROW LEVEL SECURITY`. If your application connects as the role that owns the tables — which is the default in most setups — your policies do nothing. Own the tables with a separate role, connect as a non-owner, and verify by querying as the application role.

28.6 Data protection

Concern	Approach
Passwords	Never encrypted — hashed , with bcrypt, scrypt or Argon2. Reversible encryption of a password is a defect.
Encryption in transit	<code>sslmode=verify-full</code> in the connection string. <code>require</code> alone does not check the certificate.
Encryption at rest	Disk or filesystem encryption for the data directory, backups <i>and</i> WAL archives.
Column-level encryption	<code>pgcrypto</code> , or encrypt in the application. Note the cost: an encrypted column cannot be indexed for range queries or searched by prefix.
Masking for non-production	Never restore production data to a development environment unmasked. Replace names, emails and identifiers on restore.
Audit trail	The trigger-based <code>audit_log</code> of Volume V §25.2, with the application user captured via <code>SET LOCAL</code> .
Retention	Deleting data on request is a legal requirement in many jurisdictions — design for it (soft deletes complicate it, backups more so).

BEST PRACTICE

Keep secrets out of the database and out of the repository — use the environment or a secret manager, and rotate. And prefer `scram-sha-256` over `md5` in `pg_hba.conf`; `md5` authentication is deprecated and its hashes are trivially crackable if the catalog leaks.

INTERVIEW · ADVANCED

Q. How would you isolate tenants in a multi-tenant database? — “A `tenant_id` on every table, plus row-level security with both `USING` and `WITH CHECK` policies keyed on a session variable set with `SET LOCAL` per transaction. The application connects as a non-owner role with `FORCE ROW LEVEL SECURITY`, so a forgotten `WHERE` clause cannot leak across tenants — the guarantee is in the database, not in four hundred queries.” **Interview tip:** mention the owner-bypass trap and the `SET` versus `SET LOCAL` distinction; both are the difference between RLS that works and RLS that is decoration.

Administration Concepts

Not a DBA manual — the concepts a developer needs in order to talk to one, and to avoid the three or four decisions that lose data.

29.1 Backup and restore

Method	What it gives	Restores to
<code>pg_dump</code>	Logical dump of one database; portable across versions and platforms	The moment the dump started
<code>pg_dumpall</code>	All databases plus roles and global objects	Same
Physical base backup (<code>pg_basebackup</code>)	A byte-level copy of the data directory	The backup's point in time
PITR — base backup + archived WAL	Continuous archiving	Any moment since the base backup

```
pg_dump -Fc -d sqlmastery -f sqlmastery.dump      # custom format:
compressed, selective
pg_restore -d sqlmastery_new -j 4 sqlmastery.dump  # parallel restore

# Point-in-time recovery: restore the base backup, then replay WAL to a target
# In postgresql.conf on the restored server:
#   restore_command = 'cp /archive/%f %p'
#   recovery_target_time = '2026-09-04 14:32:00+05:30'
```

THE ONLY RULE THAT MATTERS

An untested backup is not a backup. Restore it, on a schedule, to a separate machine, and run a query against the result. The common failure is not a missing backup — it is a backup that has been silently failing for four months, or one that restores without the roles, or one whose `pg_dump` version cannot read the new server. Test the restore, and time it: your recovery-time objective is whatever the restore actually takes, not what you hoped.

NOTE · RPO AND RTO

RPO — recovery point objective — how much data you can afford to lose. Nightly dumps mean an RPO of up to 24 hours; continuous WAL archiving means seconds. **RTO** — recovery time objective — how long you can be down. A 2 TB logical restore can take many hours; a physical restore or a promoted replica takes minutes. Choose the method from these two numbers, agreed with the business, not from habit.

29.2 Replication and failover, 29.3 monitoring

Type	How	Use for
Streaming (physical)	WAL shipped to a standby, applied byte for byte	High availability, read replicas — whole cluster only
Logical	Row changes decoded and published per table	Selective replication, major-version upgrades, feeding a warehouse
Synchronous	Commit waits for a standby to confirm	Zero data loss, at the cost of commit latency
Asynchronous (default)	Commit returns immediately	Everything else — accept a small RPO

Failover promotes a standby to primary. The parts that go wrong are rarely the promotion itself: it is split-brain (two nodes believing they are primary), applications still holding connections to the old primary, and the fact that an asynchronous standby may be missing the last few committed transactions. Tools — Patroni, repmgr, or a managed service — exist because doing this correctly by hand under pressure is very hard.

```
-- The four numbers to put on a dashboard
SELECT count(*) FROM pg_stat_activity;                -- connections
SELECT now() - pg_last_xact_replay_timestamp();        -- replica lag
SELECT max(age(datfrozenxid)) FROM pg_database;        -- wraparound risk
SELECT sum(n_dead_tup) FROM pg_stat_user_tables;       -- bloat trend

-- Cache hit ratio: below ~99% on an OLTP system means shared_buffers is too small
SELECT round(100.0 * sum(blks_hit) / nullif(sum(blks_hit + blks_read), 0), 2)
FROM   pg_stat_database;

-- Who is waiting on what, right now
SELECT pid, wait_event_type, wait_event, state,
       age(clock_timestamp(), query_start) AS runtime, left(query, 60)
FROM   pg_stat_activity WHERE state <> 'idle' ORDER BY query_start;
```

BEST PRACTICE · WHAT TO ALERT ON

Six things, and no more, or the alerts get ignored: replica lag above a threshold you have agreed; connection count approaching `max_connections`; transaction age above a few minutes (the vacuum blocker of Volume V §22.2); disk above 80%; wraparound age climbing; and backup job failure. Everything else belongs on a dashboard, not in a pager.

Partitioning

Volume V introduced partitioning as a performance tool. This part is the operational reality: choosing a strategy, guaranteeing pruning, handling indexes and constraints, and automating the maintenance that a partitioned table demands forever.

30.1 Why, 30.2 the three strategies

Reason to partition	Genuine?
Instant bulk deletion by dropping a partition	The best reason. A retention policy becomes <code>DROP TABLE</code> instead of deleting a hundred million rows and vacuuming for days
Partition pruning on the key	Real, but a good index often achieves the same
Maintenance on a manageable unit	Real — <code>VACUUM</code> , <code>REINDEX</code> and <code>ANALYZE</code> per partition instead of on a 2 TB table
Smaller indexes per partition	Real — each partition's index may fit in cache where a global one would not
"It will make everything faster"	No. Without a key predicate it is slower

```
-- RANGE: the common case, almost always time
CREATE TABLE events (
    event_id    bigint GENERATED ALWAYS AS IDENTITY,
    occurred_at timestamptz NOT NULL,
    tenant_id   bigint NOT NULL,
    event_type  text NOT NULL,
    payload     jsonb NOT NULL,
    PRIMARY KEY (event_id, occurred_at)    -- key must include the partition column
) PARTITION BY RANGE (occurred_at);

-- LIST: discrete values
CREATE TABLE customers_by_region (...) PARTITION BY LIST (region);
CREATE TABLE customers_apac PARTITION OF customers_by_region
    FOR VALUES IN ('IN', 'SG', 'AU', 'JP');

-- HASH: even spread when there is no natural range
CREATE TABLE sessions (...) PARTITION BY HASH (session_id);
CREATE TABLE sessions_0 PARTITION OF sessions FOR VALUES WITH (MODULUS 4, REMAINDER 0);

-- Sub-partitioning: range by month, then hash by tenant
CREATE TABLE events_2026_09 PARTITION OF events
    FOR VALUES FROM ('2026-09-01') TO ('2026-10-01')
    PARTITION BY HASH (tenant_id);
```


COMMON MISTAKE · THE CONSTRAINT CONSEQUENCES

Three limits that surprise people mid-project. **(1)** A unique constraint or primary key **must include the partition key**, because uniqueness is enforced per partition — so `event_id` alone cannot be the primary key. **(2)** A foreign key *referencing* a partitioned table is supported only from PostgreSQL 12. **(3)** An `ON CONFLICT DO UPDATE` needs the conflict target to include the partition key. Discover these at design time, not after the table has a billion rows.

30.3 Pruning, 30.4 indexes, 30.5 maintenance

Pruning is the whole benefit, and it only happens when the planner can prove which partitions a query needs.

```
-- PRUNES: literal or parameter on the partition key
EXPLAIN SELECT count(*) FROM events
WHERE occurred_at >= '2026-09-01' AND occurred_at < '2026-10-01';
-- Append -> Seq Scan on events_2026_09                                ← one partition

-- DOES NOT PRUNE: no key predicate
EXPLAIN SELECT count(*) FROM events WHERE event_type = 'login';
-- Append -> every partition

-- DOES NOT PRUNE: the cast makes the expression opaque
WHERE occurred_at::date = '2026-09-04'

-- PRUNES AT EXECUTION, not planning – check with EXPLAIN ANALYZE
WHERE occurred_at >= now() - interval '7 days'
-- "Subplans Removed: 34" in the ANALYZE output is what success looks like
```

BEST PRACTICE

Verify pruning with `EXPLAIN` on your *real* queries before you commit to a partition key, and check `enable_partition_pruning` is on (it is by default). If your dominant query does not filter on the key you chose, the key is wrong — and the choice is much harder to change later than an index.

INDEXES AND JOINS

```
-- An index created on the parent propagates to all partitions, existing and future
CREATE INDEX ON events (tenant_id, occurred_at DESC);

-- Partitionwise join and aggregate: off by default, big wins when both sides
-- are partitioned identically
SET enable_partitionwise_join = on;
SET enable_partitionwise_aggregate = on;
```

THE MAINTENANCE YOU NOW OWN FOREVER

```
-- Create next month's partition BEFORE it is needed. Automate this.
CREATE TABLE events_2026_10 PARTITION OF events
    FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');

-- Retention: detach then drop. Detaching first keeps the drop out of the
-- transaction that would otherwise lock the parent.
ALTER TABLE events DETACH PARTITION events_2023_06 CONCURRENTLY;    -- PG 14+
DROP TABLE events_2023_06;

-- Attach an existing table: add a matching CHECK first so validation is instant
ALTER TABLE events_backfill ADD CONSTRAINT ck
    CHECK (occurred_at >= '2026-08-01' AND occurred_at < '2026-09-01');
ALTER TABLE events ATTACH PARTITION events_backfill
    FOR VALUES FROM ('2026-08-01') TO ('2026-09-01');
```

COMMON MISTAKE · THE MISSING FUTURE PARTITION

If no partition covers an incoming row, the insert **fails** — or lands in the `DEFAULT` partition, which then grows unbounded and, worse, blocks you from attaching a proper partition for that range until you move the rows out. Automate creation with `pg_partman` or a scheduled job, create partitions several months ahead, and alert if the newest partition's upper bound is less than a month away. This is the most common partitioning incident and it happens at 00:00 on the first of the month.

INTERVIEW · ADVANCED

Q. When would you partition a table? — “When I need time-based retention, because dropping a partition is instant where deleting a hundred million rows is hours of work plus vacuum; when the table has grown past the point where whole-table maintenance can be scheduled; or when the dominant query always filters on a natural range key. Not for query speed alone — a good index usually beats it, and partitioning adds permanent operational work: partitions must be created ahead of time, and a missing one is an outage.”

Data Warehousing SQL

Everything so far optimised for transactions. Analytical workloads invert nearly every assumption — including normalization, which a warehouse deliberately abandons. This part covers the model and the SQL that goes with it.

IN THIS PART

31.1 OLTP vs OLAP, revisited

31.2 Facts and dimensions

31.3 Grain — the critical decision

31.4 Star and snowflake schemas

31.5 Slowly changing dimensions

31.6 Analytical SQL · Review

31.1 OLTP vs OLAP, 31.2 facts and dimensions

Volume I §1.13 compared the workloads. The modelling consequence is the point here: **a warehouse is deliberately denormalized**, because its write path is a controlled batch load rather than thousands of concurrent users, so the update anomalies of Volume IV §16 cannot arise the same way.

	Fact table	Dimension table
Holds	Measurements of events — the numbers	Descriptive context — the adjectives
Row count	Very large, grows forever	Small to medium, grows slowly
Columns	Foreign keys to dimensions, plus additive measures	Many descriptive attributes, deliberately wide and flat
Example	<code>fact_sales</code> : quantity, amount, discount	<code>dim_customer</code> : name, city, segment, tier
Changes	Append-only	Updated, with history (§31.5)

```

CREATE TABLE dim_date (
    date_key      integer PRIMARY KEY,          -- 20260904, not a date: joins as an int
    full_date     date NOT NULL,
    day_of_week   smallint NOT NULL,
    day_name      text NOT NULL,
    month         smallint NOT NULL,
    month_name    text NOT NULL,
    quarter       smallint NOT NULL,
    year          smallint NOT NULL,
    is_weekend    boolean NOT NULL,
    is_holiday    boolean NOT NULL,
    fiscal_year   smallint NOT NULL            -- the reason this table exists
);

CREATE TABLE dim_customer (
    customer_key  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY, -- surrogate
    customer_id   bigint NOT NULL,          -- the natural key from the source system
    full_name     text NOT NULL,
    city          text,
    country       text,
    segment       text,                    -- denormalized on purpose
    valid_from    timestampz NOT NULL,
    valid_to      timestampz,
    is_current    boolean NOT NULL DEFAULT true
);

CREATE TABLE fact_sales (
    sale_key      bigint GENERATED ALWAYS AS IDENTITY,
    date_key      integer NOT NULL REFERENCES dim_date (date_key),
    customer_key  bigint NOT NULL REFERENCES dim_customer (customer_key),
    product_key   bigint NOT NULL REFERENCES dim_product (product_key),
    order_id      bigint NOT NULL,          -- degenerate dimension: the receipt number
    quantity      integer NOT NULL,
    gross_amount  numeric(14,2) NOT NULL,
    discount      numeric(14,2) NOT NULL DEFAULT 0,
    net_amount    numeric(14,2) NOT NULL,
    PRIMARY KEY (sale_key, date_key)
) PARTITION BY RANGE (date_key);

```

WHY `DIM_DATE` ALWAYS EXISTS

Every warehouse has a date dimension, and it looks redundant until you need it. It lets a report group by fiscal quarter, exclude holidays, or compare like-for-like weekdays without any of that logic living in the query — and it guarantees every date exists, so a report has no gaps (the gap-filling problem of Volume II §7.3 solved once, structurally). The integer `date_key` in `YYYYMMDD` form joins faster than a date and partitions naturally.

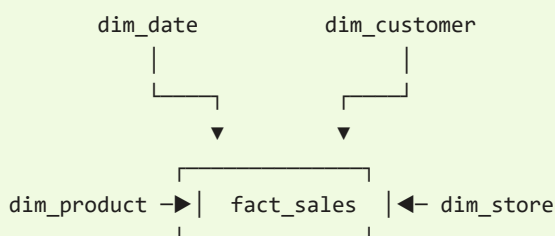
31.3 Grain, 31.4 star and snowflake

DECLARE THE GRAIN FIRST

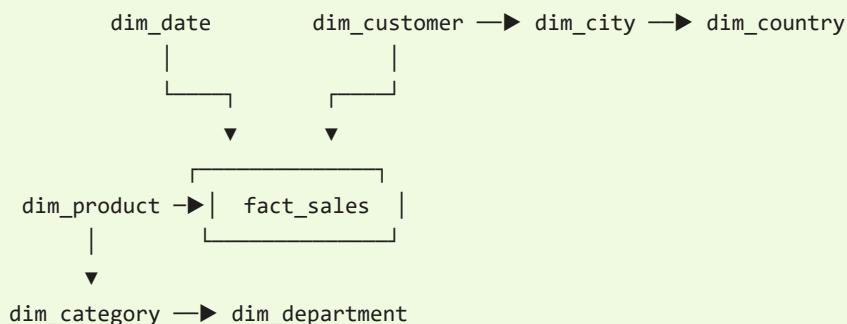
The grain is what one row of the fact table means, and it is the first and most consequential decision in a warehouse. “One row per order line” and “one row per order” are different tables with different measures — and mixing grains in one fact table double-counts every aggregate, irreparably. Write the grain down as a sentence before writing any DDL: “One row of `fact_sales` is one product on one order.”

FIGURE 31.4 — STAR VERSUS SNOWFLAKE

STAR — dimensions flat and denormalized. One join per dimension.



SNOWFLAKE — dimensions normalized into hierarchies. More joins.



Prefer the star. The redundancy in a flat dimension is cheap — dimensions are small — and every removed join is a faster query and a simpler mental model for analysts. Snowflake only when a hierarchy is genuinely large or shared across several dimensions.

```
-- The canonical star query: one fact, several dimensions, one join each
SELECT d.year,
       d.quarter,
       c.segment,
       p.category,
       sum(f.net_amount)           AS revenue,
       sum(f.quantity)            AS units,
       count(DISTINCT f.order_id)  AS orders
FROM   fact_sales   f
JOIN   dim_date     d ON d.date_key   = f.date_key
JOIN   dim_customer c ON c.customer_key = f.customer_key
JOIN   dim_product  p ON p.product_key = f.product_key
WHERE  d.year = 2026 AND c.country = 'IN'
GROUP BY ROLLUP (d.year, d.quarter, c.segment, p.category)
ORDER BY d.year, d.quarter, revenue DESC NULLS LAST;
```

Note `count(DISTINCT f.order_id)` : because the grain is one row per *line*, counting orders requires the distinct. This is the fan-out lesson of Volume III §9.8 arriving as a modelling consequence — and it is why the grain must be written down where analysts can read it.

31.5 Slowly changing dimensions

A customer moves from Mumbai to Pune. Should last year's sales now report as Pune? Both answers are legitimate, and the choice is a **type**.

Type	Behaviour	Use when
Type 0	Never changes — the original value is retained forever	Date of birth, original signup source
Type 1	Overwrite. History is lost.	Correcting a misspelling — the old value was simply wrong
Type 2	A new row with a new surrogate key and validity dates. The workhorse.	Anything where historical reporting must reflect the state at the time
Type 3	A <code>previous_city</code> column beside the current one	When exactly one prior value is needed and simplicity matters
Type 4	Current row in the dimension, history in a separate mini-dimension	Rapidly changing attributes that would bloat a Type 2
Type 6	Type 2 rows plus a current-value column on every row	Reporting that needs both “as at the time” and “as it is now”

```
-- Type 2 load: close the old row, insert the new one
BEGIN;

UPDATE dim_customer
SET    valid_to = now(), is_current = false
WHERE  customer_id = 42 AND is_current
      AND (city, segment) IS DISTINCT FROM ('Pune', 'enterprise');

INSERT INTO dim_customer (customer_id, full_name, city, country, segment,
                          valid_from, valid_to, is_current)
SELECT 42, 'Kavya Reddy', 'Pune', 'IN', 'enterprise', now(), NULL, true
WHERE  NOT EXISTS (SELECT 1 FROM dim_customer
                  WHERE customer_id = 42 AND is_current);

COMMIT;
```

The `IS DISTINCT FROM` guard is what makes the load idempotent: re-running it when nothing changed closes no row and inserts nothing. Without it, every nightly run creates a new version of every customer.

WHY THE FACT TABLE JOINS ON THE SURROGATE KEY

This is the whole mechanism of Type 2. `fact_sales.customer_key` points at the specific dimension *version* that was current when the sale happened — so last year's sales stay attached to Mumbai and this year's to Pune, automatically, with no date logic in any query. Joining the fact table on the natural `customer_id` instead would destroy the history you just went to the trouble of storing. That single design choice is the most commonly botched part of a warehouse.

INTERVIEW · ADVANCED

Q. What is a slowly changing dimension, and how do you implement Type 2? — “A dimension attribute that changes over time, where reporting may need the value as it was. Type 2 adds a new dimension row per change, with `valid_from`, `valid_to`, `is_current` and a fresh surrogate key — and the fact table joins on that surrogate key, so historical facts remain attached to the version that was current when they occurred.” **Interview tip:** the surrogate-key point is the answer they are listening for; anyone can recite the four types.

Parts 27–31 Review

Quick Check ● BEGINNER

- 1 Why is a parameterised query safe? It is not escaping — what is it?
- 2 Name the three things you cannot parameterise, and what to do instead.
- 3 What is second-order injection, and why does input validation not stop it?
- 4 Why do prepared statements break under PgBouncer transaction mode?
- 5 Name four ORM problems and their fixes.
- 6 What does expand-and-contract mean, and why does it need two deploys?
- 7 Why does `ALTER DEFAULT PRIVILEGES` exist?
- 8 Why is RLS ineffective if the application owns its tables?
- 9 Why `SET LOCAL` rather than `SET` for a tenant id?
- 10 Define RPO and RTO, and give a backup method matching each extreme.
- 11 What is the single rule about backups?
- 12 Give three reasons a unique key must include the partition column.
- 13 What happens if no partition covers an incoming row?
- 14 What is the grain of a fact table, and why declare it first?
- 15 Why does a fact table join a Type 2 dimension on the surrogate key?

Practice ● EASY

- 1 Write a parameterised query in any language, then write the vulnerable version and explain what an attacker could send.
- 2 Implement a whitelisted sort column and direction for an orders listing.
- 3 Rewrite an `IN` list query to use `= ANY()` and explain the plan-caching benefit.
- 4 Write a safe dynamic-SQL function using `format('%I')`, then show what `%s` would have allowed.
- 5 Create `app_read`, `app_write` and two login roles with default privileges. Verify each can and cannot do what you intended.
- 6 Grant column-level `SELECT` on `customers` excluding `email`, and prove the exclusion works.
- 7 Add `tenant_id` to `orders`, enable RLS with both policies, and prove isolation from a non-owner role.
- 8 Set per-role `statement_timeout` values and demonstrate one firing.
- 9 Take a `pg_dump`, drop the database, restore it, and time both operations.

- 10** Write the six monitoring queries from §29.3 and interpret each result on your instance.
- 11** Partition `events` by month, prove pruning, then prove a query that does not prune.
- 12** Detach and drop a partition; compare with deleting the same rows.
- 13** Build `dim_date` for 2026 with `generate_series`, including fiscal year and weekend flags.
- 14** Build a star schema over the practice database: `dim_customer`, `dim_product`, `dim_date`, `fact_sales` at line grain. Load it from `order_items`.
- 15** Write the canonical star query with `ROLLUP` against your loaded facts.
- 16** Perform a Type 2 update on `dim_customer`, then show that a historical fact still reports the old city.

Challenge ● ADVANCED

- 1** An audit finds the application connects as a superuser. Write the migration plan to least privilege, in the order you would execute it, without downtime.
- 2** Design multi-tenant isolation for a SaaS product. Compare RLS, schema-per-tenant and database-per-tenant, and recommend one with its failure modes.
- 3** Design the retention and partitioning strategy for a 5 TB events table keeping 13 months, including automation and what you alert on.
- 4** A report needs revenue by customer segment as the segment was at the time of sale, and also as it is today. Design the dimension and write both queries.
- 5** Your nightly warehouse load takes 9 hours and must finish in 3. Give four approaches in order of preference.

Parts 27–31 Solutions

Quick Check

- 1 The statement is parsed once with a placeholder, and the value is bound to that slot as data afterwards. There is no string for a quote character to escape from.
- 2 Table names, column names, and sort direction. Map user input through a fixed whitelist of literals.
- 3 A payload stored safely, then interpolated later by a different query. Validation does not stop it because the stored value is legitimate data — the flaw is in the second query.
- 4 They are per connection, and transaction mode may route your next transaction to a different backend that has never prepared them.
- 5 N+1 (eager load), `SELECT *` (projections), read-modify-write (atomic expressions), whole-row updates (dirty tracking), request-wide transactions (explicit narrow blocks). Any four.
- 6 Add the new structure and dual-write, migrate readers, then drop the old. Two deploys because old and new code run simultaneously during a rolling release.
- 7 `GRANT ... ON ALL TABLES` applies only to tables that exist now. Default privileges cover tables created later.
- 8 Table owners bypass RLS unless `FORCE ROW LEVEL SECURITY` is set. Own tables with a separate role and connect as a non-owner.
- 9 `SET LOCAL` is scoped to the transaction, so a pooled connection cannot carry one tenant's context into another request.
- 10 RPO is acceptable data loss, RTO is acceptable downtime. Nightly dumps give an RPO of a day; continuous WAL archiving gives seconds. A promoted replica gives an RTO of minutes; a large logical restore gives hours.
- 11 An untested backup is not a backup. Restore it on a schedule and time the restore.
- 12 Uniqueness is enforced per partition, so a global unique index is impossible without it; it also affects `ON CONFLICT` targets and foreign-key support.
- 13 The insert fails, or lands in the `DEFAULT` partition — which then blocks attaching a real partition for that range until the rows are moved.
- 14 What one row means. Declaring it first prevents mixing grains, which double-counts every aggregate irreparably.
- 15 Because the surrogate key identifies the dimension *version* current when the fact occurred, so history stays attached automatically.

Practice — selected

13 · `dim_date` .

```

INSERT INTO dim_date (date_key, full_date, day_of_week, day_name, month,
                      month_name, quarter, year, is_weekend, is_holiday, fiscal_year)
SELECT to_char(d, 'YYYYMMDD')::integer,
       d::date,
       extract(isodow FROM d),
       to_char(d, 'Day'),
       extract(month FROM d),
       to_char(d, 'Month'),
       extract(quarter FROM d),
       extract(year FROM d),
       extract(isodow FROM d) IN (6, 7),
       false,                                -- loaded separately
       CASE WHEN extract(month FROM d) >= 4
            THEN extract(year FROM d) ELSE extract(year FROM d) - 1 END
FROM   generate_series(DATE '2026-01-01', DATE '2026-12-31', INTERVAL '1 day') d;

```

`isodow` rather than `dow`, so Monday is 1 and the weekend test is portable reasoning rather than remembering that PostgreSQL's `dow` makes Sunday 0 (Volume II §7.3). The fiscal-year expression assumes an April start — the exact reason this table exists, since no date function knows your company's financial calendar.

14 · Loading the star from the practice database.

```

INSERT INTO dim_customer (customer_id, full_name, city, country, segment,
                          valid_from, is_current)
SELECT customer_id, full_name, city, 'IN',
       CASE WHEN city IN ('Mumbai','Delhi') THEN 'metro' ELSE 'regional' END,
       created_at, true
FROM   customers;

INSERT INTO fact_sales (date_key, customer_key, product_key, order_id,
                       quantity, gross_amount, discount, net_amount)
SELECT to_char(o.order_date, 'YYYYMMDD')::integer,
       dc.customer_key,
       dp.product_key,
       o.order_id,
       oi.quantity,
       oi.quantity * oi.unit_price,
       0,
       oi.quantity * oi.unit_price
FROM   order_items oi
JOIN   orders o      ON o.order_id      = oi.order_id
JOIN   dim_customer dc ON dc.customer_id = o.customer_id AND dc.is_current
JOIN   dim_product  dp ON dp.product_id = oi.product_id AND dp.is_current
WHERE  o.status <> 'cancelled';

```

Three things to notice. The grain is one row per order line, which is why the join starts from `order_items`. The dimension joins carry `AND is_current` — correct for an initial load, and **wrong** for an incremental one, where you must join on the version valid at `o.order_date`. And `order_id` is carried on the fact as a degenerate dimension: it has no attributes of its own but you need it for `count(DISTINCT order_id)`.

16 • Type 2 in action. After the \$31.5 load, `dim_customer` holds two rows for customer 42 with different `customer_key` values. The historical fact rows still reference the old key, so:

```
SELECT c.city, sum(f.net_amount)
FROM   fact_sales f JOIN dim_customer c ON c.customer_key = f.customer_key
WHERE  f.order_id IN (1001, 1003) GROUP BY c.city;
--   Mumbai | 35300.00      ← the old city, because the old key is referenced
```

No date filtering, no validity comparison, no `BETWEEN valid_from AND valid_to` anywhere in the query. That is the payoff, and it is why the surrogate key is not an optional detail.

Challenge — worked

1 • Superuser to least privilege, without downtime. The order matters, because doing it in the wrong sequence breaks the application. **(i) Observe before changing.** Turn on `log_statement = 'ddl'` and audit what the application actually does — you will usually find one endpoint that creates a temp table and one job that truncates something, and those are what break. **(ii) Create the role hierarchy** (`app_read`, `app_write`) and grant privileges plus `ALTER DEFAULT PRIVILEGES`, but change nothing yet. **(iii) Create a new login role** in `app_write` and point a *single* canary instance at it. Run for a day and collect permission errors. **(iv) Move the migration runner to its own DDL-capable role** used only by CI — this is the step that lets the application role lose DDL at all. **(v) Roll the new credentials out** instance by instance. **(vi) Only then revoke** the old role's rights, and keep the superuser as a break-glass account with auditing. The key insight: least privilege is discovered, not designed — you cannot know what the application needs until you have watched it fail in a controlled way.

2 • Multi-tenant isolation. RLS with a shared schema: one set of tables, `tenant_id` everywhere, policies keyed on a session variable. Cheapest to operate at scale — one connection pool, one migration, one set of indexes — and the isolation is enforced by the database rather than by discipline. Failure modes: the owner-bypass trap; a missing policy on a newly added table (mitigate by testing that every table has RLS enabled in CI); noisy-neighbour performance; and a per-tenant restore being genuinely hard. **Schema-per-tenant:** stronger isolation, per-tenant backup and restore is trivial, and migrations must run N times — which is fine at 50 tenants and unmanageable at 5,000, where catalog bloat and connection-pool `search_path` juggling become real problems. **Database-per-tenant:** strongest isolation and the only option for some compliance regimes; costs the most in connections, backups and operational overhead, and cross-tenant analytics requires a separate pipeline. **Recommend RLS with a shared schema** for a product expecting many tenants, with the option to promote a large or regulated tenant to its own database — because that hybrid is what almost every mature SaaS ends up running, and designing for it from the start is cheaper than migrating to it later.

4 • Segment as-at-the-time and as-today. This is precisely what SCD Type 6 exists for: keep Type 2 history rows, and additionally carry the current value on *every* row.

```

ALTER TABLE dim_customer ADD COLUMN current_segment text;

-- Maintained on each load: every version of a customer gets today's segment
UPDATE dim_customer d
SET     current_segment = c.segment
FROM   (SELECT customer_id, segment FROM dim_customer WHERE is_current) c
WHERE  c.customer_id = d.customer_id;

-- Revenue by segment AS AT the time of sale
SELECT c.segment, sum(f.net_amount) FROM fact_sales f
JOIN   dim_customer c ON c.customer_key = f.customer_key GROUP BY c.segment;

-- Revenue by segment AS IT IS TODAY
SELECT c.current_segment, sum(f.net_amount) FROM fact_sales f
JOIN   dim_customer c ON c.customer_key = f.customer_key GROUP BY c.current_segment;

```

Two queries, identical shape, one column different — and the two totals will disagree, which is correct and is exactly the question the business asked. The alternative (joining on the natural key to get today's value) works but requires the analyst to know which join to use; the Type 6 column makes both readings available without that knowledge.

5 • The 9-hour load. Four approaches, best first. **(i) Make it incremental.** Most 9-hour loads are full reloads. Process only rows changed since the last watermark — a `updated_at` high-water mark, or logical replication / CDC from the source. This is usually a 20× reduction and it is the only approach that keeps working as the data grows. **(ii) Parallelise by partition.** If the fact table is partitioned by date, load each partition independently and concurrently, then attach. Dimension loads can run in parallel with each other since they are independent. **(iii) Optimise the load mechanics:** `COPY` rather than `INSERT`; drop or disable fact-table indexes and recreate them after; use `UNLOGGED` staging tables (no WAL) for intermediate steps; raise `maintenance_work_mem` for the index rebuilds. Together these routinely halve a load. **(iv) Move transformation into the database.** If the pipeline pulls rows out, transforms them row-by-row in Python, and writes them back, that is an N+1 at ETL scale — a single `INSERT ... SELECT` with joins will beat it by orders of magnitude. Start with (i): the others make a wasteful load faster, while (i) stops doing the wasteful work.

Where you are, and what comes next

Thirty-one parts done. You can design a schema, query it well, make it fast, keep it correct under concurrency, ship it safely from application code, and model it differently when the question is analytical. Confirm these six:

1. Explain why parameterisation is not escaping, and what second-order injection is.
2. Name the three things that cannot be parameterised and what to do instead.
3. Set up least privilege for a web app, a reporting user and a migration runner.
4. Explain the two traps that make row-level security decorative.
5. State the one rule about backups, and define RPO and RTO.
6. Explain why a fact table joins a Type 2 dimension on the surrogate key.

NEXT

Volume VII — Parts 32 to 40. The applied half: fifteen real-world database projects with full schemas and queries; the interview bank (100 beginner, 150 intermediate, 150 advanced, 100 scenario, 200 query-writing); 500+ graded coding challenges; the pattern library; debugging and find-the-bug exercises; the four-dialect comparison tables; best practices as bad-SQL-beside-good; the cheat sheets; the glossary and quick reference; the roadmaps; and the final capstone project.

That volume is the largest of the seven and will come in several passes. Say the word and I will start it.



SQL MASTERY

Volume VII — Projects & Practice

Parts 32 to 40, complete. The applied half of the book: real schemas to build, 700 questions to answer out loud, 500 problems to solve, the pattern library, and the references you keep open while working.

-
- | | |
|----|---|
| 32 | Real-world Projects — fifteen databases, from requirements to reports · complete |
| 33 | Interview Preparation — 700 questions across five bands · complete |
| 34 | Coding Challenges — 500 graded problems, each naming its topic and pattern · complete |
| 35 | SQL Patterns — the twenty patterns that solve most real problems; read before Part 34 · complete |
| 36 | Debugging SQL — the method, and twelve find-the-bug exercises · complete |
| 37 | Dialect Comparison — PostgreSQL, MySQL, SQL Server, Oracle, side by side · complete |
| 38 | Best Practices — twenty rules as bad SQL beside good SQL · complete |
| 39 | Cheat Sheets — query anatomy, functions, diagnostics, dialects · complete |
| 40 | Mastery Roadmap — 30/60/90-day plans, glossary, capstone · complete |
-

Real-world Projects

Fifteen databases. The first three are worked in full — requirements, design decisions with their reasoning, complete DDL, sample data, and the queries a real system needs. The remaining twelve give you requirements, the design decisions that matter, and the queries to write, because building them yourself is the point.

HOW TO WORK THESE

Read the requirements. Do the noun-and-verb pass of Volume IV §15.1 *before* reading the design. Write your own ER diagram, then compare — the differences are where your instincts need adjusting. Then write the DDL, load the data, and answer the queries without looking at the solutions.

32.1 Library Management System

BEGINNER Worked in full. The classic first project, and it contains one modelling trap almost everyone falls into.

REQUIREMENTS

A library holds **books**. The library owns several physical **copies** of each book. A **member** borrows a copy; the **loan** records which copy, to whom, when it was taken and when it is due. A copy may be on loan to only one member at a time. Returning is recorded. An overdue loan accrues a **fine**, which may be paid. Members may **reserve** a book that is fully out, and are notified in queue order. Staff need: what is overdue, who owes money, which books are most borrowed, and whether a given title has any copy available right now.

THE DESIGN DECISIONS THAT MATTER

Decision	Reasoning
Book and copy are separate entities	The trap. A title has an ISBN, author and publisher; a copy has a shelf location, an acquisition date and a condition. You lend a <i>copy</i> , not a title. Merging them makes "three copies of this book" unrepresentable.
Loan is not a junction table	The same member borrows the same copy repeatedly, so <code>(member_id, copy_id)</code> cannot be the key. Surrogate key, and the real rule expressed as a partial unique index.
One active loan per copy	A partial unique index on <code>copy_id WHERE returned_on IS NULL</code> . A plain <code>UNIQUE</code> would forbid ever lending a copy twice.
Money in minor units	<code>bigint</code> paise, never <code>float</code> (Volume II §4.2).
Reservation queue by timestamp	A <code>requested_at</code> column plus a partial unique index so one member cannot hold two active reservations on one title.
Authors as a junction	A book may have several authors, and an author writes several books. M:N, so <code>book_authors</code> .

SCHEMA

```

CREATE TABLE members (
    member_id    bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    full_name    text NOT NULL,
    email        text NOT NULL,
    phone        text,
    joined_on    date NOT NULL DEFAULT current_date,
    max_loans    smallint NOT NULL DEFAULT 5 CHECK (max_loans BETWEEN 1 AND 20),
    suspended    boolean NOT NULL DEFAULT false
);
CREATE UNIQUE INDEX members_email_key ON members (lower(email));

CREATE TABLE authors (
    author_id    bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    full_name    text NOT NULL
);

CREATE TABLE books (
    book_id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    isbn          char(13) UNIQUE,                -- natural key kept as UNIQUE
    title        text NOT NULL,
    publisher     text,
    published_year smallint CHECK (published_year BETWEEN 1400 AND 2100),
    category     text NOT NULL
);

CREATE TABLE book_authors (
    book_id      bigint REFERENCES books (book_id) ON DELETE CASCADE,
    author_id    bigint REFERENCES authors (author_id),
    author_order smallint NOT NULL DEFAULT 1,
    PRIMARY KEY (book_id, author_id)
);

CREATE TABLE copies (
    copy_id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    book_id      bigint NOT NULL REFERENCES books (book_id),
    shelf_code   text,
    acquired_on  date NOT NULL DEFAULT current_date,
    condition    text NOT NULL DEFAULT 'good'
                CHECK (condition IN ('new', 'good', 'worn', 'damaged', 'lost')),
    withdrawn_on date                    -- soft retire; loans reference it
);
CREATE INDEX idx_copies_book ON copies (book_id);

CREATE TABLE loans (
    loan_id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    copy_id      bigint NOT NULL REFERENCES copies (copy_id),
    member_id    bigint NOT NULL REFERENCES members (member_id),
    borrowed_on  date NOT NULL DEFAULT current_date,
    due_on       date NOT NULL,

```

```

        returned_on date,
        CONSTRAINT loans_due_after_borrow CHECK (due_on > borrowed_on),
        CONSTRAINT loans_return_after_borrow
            CHECK (returned_on IS NULL OR returned_on >= borrowed_on)
    );
-- The business rule, enforced by the database:
CREATE UNIQUE INDEX loans_one_active_per_copy
    ON loans (copy_id) WHERE returned_on IS NULL;
CREATE INDEX idx_loans_member ON loans (member_id);
CREATE INDEX idx_loans_overdue ON loans (due_on) WHERE returned_on IS NULL;

CREATE TABLE fines (
    fine_id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    loan_id      bigint NOT NULL REFERENCES loans (loan_id),
    amount_minor bigint NOT NULL CHECK (amount_minor > 0),
    assessed_on  date NOT NULL DEFAULT current_date,
    paid_on      date,
    CONSTRAINT fines_paid_after_assessed
        CHECK (paid_on IS NULL OR paid_on >= assessed_on)
);
CREATE INDEX idx_fines_unpaid ON fines (loan_id) WHERE paid_on IS NULL;

CREATE TABLE reservations (
    reservation_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    book_id        bigint NOT NULL REFERENCES books (book_id),
    member_id      bigint NOT NULL REFERENCES members (member_id),
    requested_at   timestampz NOT NULL DEFAULT now(),
    fulfilled_at   timestampz,
    cancelled_at   timestampz
);
CREATE UNIQUE INDEX reservations_one_active_per_member_book
    ON reservations (book_id, member_id)
    WHERE fulfilled_at IS NULL AND cancelled_at IS NULL;

```

SAMPLE DATA

```

INSERT INTO members (full_name, email, joined_on) VALUES
('Asha Menon', 'asha@example.com', '2023-02-14'),
('Ravi Kumar', 'ravi@example.com', '2023-08-01'),
('Fatima Sheikh', 'fatima@example.com', '2024-01-09'),
('Daniel Fernandes', 'daniel@example.com', '2024-06-22'),
('Priya Anand', 'priya@example.com', '2025-03-30');    -- never borrows

INSERT INTO authors (full_name) VALUES
('C. J. Date'), ('Markus Winand'), ('Bill Karwin'), ('Joe Celko');

INSERT INTO books (isbn, title, publisher, published_year, category) VALUES
('9780321197849', 'An Introduction to Database Systems', 'Pearson', 2003, 'Databases'),
('9783950307825', 'SQL Performance Explained', 'Winand', 2012, 'Databases'),
('9781680508987', 'SQL Antipatterns', 'Pragmatic', 2010, 'Databases'),
('9780123820228', 'Joe Celko's SQL for Smarties', 'Morgan Kaufmann', 2010, 'Databases'),
('9780596520830', 'The Art of SQL', 'O'Reilly', 2006, 'Databases');

INSERT INTO book_authors (book_id, author_id) VALUES (1,1),(2,2),(3,3),(4,4);

INSERT INTO copies (book_id, shelf_code, condition) VALUES
(1, 'DB-01-A', 'good'), (1, 'DB-01-B', 'worn'), (1, 'DB-01-C', 'good'),
(2, 'DB-02-A', 'new'), (2, 'DB-02-B', 'good'),
(3, 'DB-03-A', 'good'),
(4, 'DB-04-A', 'good'), (4, 'DB-04-B', 'damaged'),
(5, 'DB-05-A', 'good');

-- Two returned, three still out, one of those overdue
INSERT INTO loans (copy_id, member_id, borrowed_on, due_on, returned_on) VALUES
(1, 1, '2026-07-01', '2026-07-22', '2026-07-19'),
(4, 1, '2026-07-25', '2026-08-15', '2026-08-30'),    -- returned late → fine
(2, 2, '2026-08-10', '2026-08-31', NULL),
(6, 2, '2026-08-20', '2026-09-10', NULL),
(7, 3, '2026-06-15', '2026-07-06', NULL);           -- long overdue

INSERT INTO fines (loan_id, amount_minor, assessed_on, paid_on) VALUES
(2, 15000, '2026-08-30', NULL);                     -- ₹150, unpaid

INSERT INTO reservations (book_id, member_id, requested_at) VALUES
(3, 1, '2026-08-21 10:00+05:30'),
(3, 4, '2026-08-21 11:30+05:30');

```

THE QUERIES A REAL LIBRARY NEEDS

-- 1. What is overdue right now, and by how long

```
SELECT m.full_name,  
       b.title,  
       l.due_on,  
       current_date - l.due_on AS days_overdue  
FROM   loans l  
JOIN   copies c ON c.copy_id = l.copy_id  
JOIN   books  b ON b.book_id = c.book_id  
JOIN   members m ON m.member_id = l.member_id  
WHERE  l.returned_on IS NULL  
       AND l.due_on < current_date  
ORDER BY days_overdue DESC;
```

-- 2. Availability per title: the query the front desk runs constantly

```
SELECT b.title,  
       count(c.copy_id) AS total_copies,  
       count(c.copy_id) FILTER (WHERE l.loan_id IS NULL) AS available_now  
FROM   books b  
JOIN   copies c ON c.book_id = b.book_id AND c.withdrawn_on IS NULL  
LEFT   JOIN loans l ON l.copy_id = c.copy_id AND l.returned_on IS NULL  
GROUP BY b.book_id, b.title  
ORDER BY available_now, b.title;
```

-- 3. Who owes money

```
SELECT m.full_name,  
       sum(f.amount_minor) / 100.0 AS owed  
FROM   fines f  
JOIN   loans l ON l.loan_id = f.loan_id  
JOIN   members m ON m.member_id = l.member_id  
WHERE  f.paid_on IS NULL  
GROUP BY m.member_id, m.full_name  
ORDER BY owed DESC;
```

-- 4. Most borrowed titles, last 12 months

```
SELECT b.title, count(*) AS loans  
FROM   loans l  
JOIN   copies c ON c.copy_id = l.copy_id  
JOIN   books  b ON b.book_id = c.book_id  
WHERE  l.borrowed_on >= current_date - INTERVAL '12 months'  
GROUP BY b.book_id, b.title  
ORDER BY loans DESC, b.title  
LIMIT 10;
```

-- 5. Reservation queue position – a window function, not a subquery

```
SELECT b.title,  
       m.full_name,  
       row_number() OVER (PARTITION BY r.book_id ORDER BY r.requested_at) AS queue_pos  
FROM   reservations r  
JOIN   books b ON b.book_id = r.book_id
```



```

JOIN    members m ON m.member_id = r.member_id
WHERE   r.fulfilled_at IS NULL AND r.cancelled_at IS NULL
ORDER  BY b.title, queue_pos;

-- 6. Members who have never borrowed (NOT EXISTS, not NOT IN)
SELECT m.full_name FROM members m
WHERE   NOT EXISTS (SELECT 1 FROM loans l WHERE l.member_id = m.member_id);

-- 7. Can this member borrow? Their active loans against their limit
SELECT m.full_name, m.max_loans,
       count(l.loan_id) FILTER (WHERE l.returned_on IS NULL) AS active,
       m.max_loans - count(l.loan_id) FILTER (WHERE l.returned_on IS NULL) AS remaining
FROM   members m LEFT JOIN loans l ON l.member_id = m.member_id
WHERE  m.member_id = 2
GROUP BY m.member_id, m.full_name, m.max_loans;

-- 8. Assess fines for everything overdue: ₹5/day, idempotent
INSERT INTO fines (loan_id, amount_minor, assessed_on)
SELECT l.loan_id, (current_date - l.due_on) * 500, current_date
FROM   loans l
WHERE  l.returned_on IS NULL
      AND l.due_on < current_date
      AND NOT EXISTS (SELECT 1 FROM fines f
                      WHERE f.loan_id = l.loan_id AND f.assessed_on = current_date);

```

NOTE THE TECHNIQUE IN QUERY 2

`count(c.copy_id) FILTER (WHERE l.loan_id IS NULL)` over a `LEFT JOIN` to active loans. The left join attaches an active loan if one exists; the filter counts the copies where it did not. This "left join then count the misses" shape answers every availability question — seats free, rooms open, licences unassigned — and it is far clearer than a correlated subquery.

INTERVIEW QUESTIONS FROM THIS PROJECT

1. Why are book and copy separate tables? What breaks if they are merged?
2. How do you enforce that a copy is on loan to only one member at a time, correctly under concurrency?
3. Why is `(member_id, copy_id)` the wrong primary key for `loans` ?
4. Write the availability query. Why `FILTER` over a left join rather than a subquery?
5. The fine-assessment job runs twice by accident. Why does it not double-charge?

32.2 E-commerce Database

● **INTERMEDIATE** Worked in full. The project that exercises price history, inventory under concurrency, and the order state machine.

REQUIREMENTS

Customers browse **products** in **categories** (which nest arbitrarily deep).
A product has **variants** – size, colour – each with its own SKU, price and stock.
A customer adds variants to a **cart**, then **checks out**, creating an **order** with one **line** per variant at the price agreed then. Orders ship to a saved **address**, move through a **status** lifecycle, and are settled by one or more **payments**. Stock must never go negative. Prices change over time and old invoices must reproduce exactly. Customers leave **reviews**, one per product.
The business needs: revenue by period and category, best sellers, cart abandonment, repeat-purchase rate, and stock that needs reordering.

THE DESIGN DECISIONS THAT MATTER

Decision	Reasoning
Variant is the stocked unit	Stock, price and SKU belong to "medium blue t-shirt", not to "t-shirt". Getting this wrong is the most expensive e-commerce modelling error, because retrofitting variants touches every table.
Categories self-reference	<code>parent_category_id</code> , traversed with a recursive CTE (Volume III §11.5). Arbitrary depth was a requirement.
Price snapshot on the line	<code>unit_price_minor</code> on <code>order_lines</code> . Volume IV §15.1 — the agreed price is a different fact from the current price.
Address snapshot too	The shipping address is copied onto the order as text, not only referenced. Editing a saved address must not change where a past order says it went.
Stock as a guarded relative update	<code>SET stock = stock - \$1 WHERE stock >= \$1</code> , plus <code>CHECK (stock >= 0)</code> as backstop. Volume IV §19 challenge 1 — no locking, no isolation change, unraceable.
Status as a lookup table	With <code>sort_order</code> and <code>is_terminal</code> . The set grows (<code>refunded</code> , <code>partially_shipped</code>), and a <code>CHECK</code> would need a migration each time.
One review per customer per product	<code>UNIQUE (product_id, customer_id)</code> — a constraint, not an application check.
Payments are many-to-one	Instalments and partial refunds mean an order has several payment rows, with a <code>UNIQUE</code> on the gateway reference for idempotency.

SCHEMA

```

CREATE TABLE categories (
    category_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name         text NOT NULL,
    slug         text NOT NULL UNIQUE,
    parent_category_id bigint REFERENCES categories (category_id)
);
CREATE INDEX idx_categories_parent ON categories (parent_category_id);

CREATE TABLE products (
    product_id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    category_id bigint NOT NULL REFERENCES categories (category_id),
    name        text NOT NULL,
    description  text,
    is_active   boolean NOT NULL DEFAULT true,
    created_at  timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX idx_products_category ON products (category_id) WHERE is_active;

CREATE TABLE variants (
    variant_id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    product_id  bigint NOT NULL REFERENCES products (product_id),
    sku         text NOT NULL UNIQUE,
    size        text,
    colour      text,
    price_minor bigint NOT NULL CHECK (price_minor >= 0),
    stock       integer NOT NULL DEFAULT 0 CHECK (stock >= 0),
    reorder_at  integer NOT NULL DEFAULT 5,
    UNIQUE (product_id, size, colour)
);
CREATE INDEX idx_variants_product ON variants (product_id);
CREATE INDEX idx_variants_reorder ON variants (product_id) WHERE stock <= 5;

CREATE TABLE customers (
    customer_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    full_name   text NOT NULL,
    email       text NOT NULL,
    created_at  timestamptz NOT NULL DEFAULT now(),
    deleted_at  timestamptz
);
CREATE UNIQUE INDEX customers_email_live
    ON customers (lower(email)) WHERE deleted_at IS NULL;

CREATE TABLE addresses (
    address_id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id bigint NOT NULL REFERENCES customers (customer_id),
    line1       text NOT NULL,
    line2       text,
    city        text NOT NULL,
    postcode    text NOT NULL,

```

```

country      char(2) NOT NULL,
is_default   boolean NOT NULL DEFAULT false,
archived_at  timestampz
);
CREATE UNIQUE INDEX addresses_one_default
ON addresses (customer_id) WHERE is_default AND archived_at IS NULL;

CREATE TABLE order_statuses (
    status      text PRIMARY KEY,
    label       text NOT NULL,
    sort_order  smallint NOT NULL,
    is_terminal boolean NOT NULL DEFAULT false
);
INSERT INTO order_statuses VALUES
('cart','In cart',10,false),      ('placed','Placed',20,false),
('paid','Paid',30,false),        ('shipped','Shipped',40,false),
('delivered','Delivered',50,true), ('cancelled','Cancelled',60,true),
('refunded','Refunded',70,true);

CREATE TABLE orders (
    order_id    bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id bigint NOT NULL REFERENCES customers (customer_id),
    status      text NOT NULL REFERENCES order_statuses (status) DEFAULT 'cart',
    placed_at   timestampz,
    shipped_at   timestampz,
    -- address SNAPSHOT: the reference for traceability, the text for history
    address_id  bigint REFERENCES addresses (address_id),
    ship_to     text,
    total_minor bigint NOT NULL DEFAULT 0 CHECK (total_minor >= 0),
    currency    char(3) NOT NULL DEFAULT 'INR',
    CONSTRAINT orders_placed_has_address
        CHECK (status = 'cart' OR (address_id IS NOT NULL AND ship_to IS NOT NULL))
);
CREATE INDEX idx_orders_customer_placed ON orders (customer_id, placed_at DESC);
CREATE INDEX idx_orders_open ON orders (placed_at) WHERE status IN ('placed','paid');
-- One open cart per customer
CREATE UNIQUE INDEX orders_one_cart ON orders (customer_id) WHERE status = 'cart';

CREATE TABLE order_lines (
    order_id    bigint NOT NULL REFERENCES orders (order_id) ON DELETE CASCADE,
    variant_id   bigint NOT NULL REFERENCES variants (variant_id),
    quantity     integer NOT NULL CHECK (quantity > 0),
    unit_price_minor bigint NOT NULL CHECK (unit_price_minor >= 0), -- snapshot
    product_name text NOT NULL, -- snapshot
    PRIMARY KEY (order_id, variant_id)
);
CREATE INDEX idx_order_lines_variant ON order_lines (variant_id);

CREATE TABLE payments (
    payment_id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

```

```
order_id    bigint NOT NULL REFERENCES orders (order_id),
amount_minor bigint NOT NULL,                -- negative for a refund
method      text NOT NULL,
gateway_ref text NOT NULL UNIQUE,           -- idempotency key
settled_at  timestampz NOT NULL DEFAULT now()
);

CREATE TABLE reviews (
  review_id    bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  product_id   bigint NOT NULL REFERENCES products (product_id),
  customer_id  bigint NOT NULL REFERENCES customers (customer_id),
  rating       smallint NOT NULL CHECK (rating BETWEEN 1 AND 5),
  body         text,
  created_at   timestampz NOT NULL DEFAULT now(),
  UNIQUE (product_id, customer_id)
);
```

SAMPLE DATA

```

INSERT INTO categories (name, slug, parent_category_id) VALUES
('Electronics','electronics',NULL),
('Peripherals','peripherals',1),
('Keyboards','keyboards',2),
('Displays','displays',1),
('Apparel','apparel',NULL);

INSERT INTO products (category_id, name, description) VALUES
(3,'Mechanical Keyboard','Tactile switches, hot-swappable'),
(4,'27-inch Monitor','1440p, 165 Hz'),
(2,'USB-C Hub','7 ports'),
(2,'Webcam 1080p','Auto-focus');

INSERT INTO variants (product_id, sku, size, colour, price_minor, stock) VALUES
(1,'KB-TKL-BLK','TKL','black', 540000, 42),
(1,'KB-TKL-WHT','TKL','white', 540000, 8),
(1,'KB-FULL-BLK','Full','black', 620000, 0),
(2,'MON-27-BLK','27in','black',1890000, 15),
(3,'HUB-7C','-','grey', 230000, 3),
(4,'CAM-1080','-','black', 310000, 23);

INSERT INTO customers (full_name, email, created_at) VALUES
('Kavya Reddy','kavya@example.com','2025-01-14'),
('Rohan Gupta','rohan@example.com','2025-03-02'),
('Fatima Sheikh','fatima@example.com','2025-06-19'),
('Daniel Fernandes','daniel@example.com','2026-02-27');

INSERT INTO addresses (customer_id, line1, city, postcode, country, is_default) VALUES
(1,'12 Jubilee Hills','Hyderabad','500033','IN',true),
(2,'4 Connaught Place','Delhi','110001','IN',true),
(3,'88 Bandra West','Mumbai','400050','IN',true),
(4,'7 Calangute Rd','Goa','403516','IN',true);

INSERT INTO orders (customer_id, status, placed_at, address_id, ship_to,
total_minor) VALUES
(1,'delivered','2026-01-05','12 Jubilee Hills, Hyderabad 500033 IN',NULL,2430000),
(1,'delivered','2026-03-11','12 Jubilee Hills, Hyderabad 500033 IN',NULL, 540000),
(2,'shipped','2026-06-28','4 Connaught Place, Delhi 110001 IN', NULL,1890000),
(3,'cancelled','2026-07-03','88 Bandra West, Mumbai 400050 IN', NULL, 310000),
(4,'cart', NULL, NULL, NULL, 0);

```

DELIBERATE ERROR ABOVE

Those `INSERT` s put the address text in the `address_id` column — the column list and the values are misaligned. Fixing it is exercise 1: write the correct statement with `address_id` and `ship_to` in the right positions. Reading a column list against its values is a real skill, and this is the most common cause of a load that "runs but is wrong".

THE QUERIES THE BUSINESS NEEDS

```

-- 1. Revenue by month and top-level category
WITH RECURSIVE cat_tree AS (
    SELECT category_id, name AS root_name, category_id AS root_id
    FROM   categories WHERE parent_category_id IS NULL
    UNION ALL
    SELECT c.category_id, t.root_name, t.root_id
    FROM   categories c JOIN cat_tree t ON t.category_id = c.parent_category_id
)
SELECT date_trunc('month', o.placed_at)::date AS month,
       t.root_name                          AS category,
       sum(ol.quantity * ol.unit_price_minor) / 100.0 AS revenue
FROM   orders o
JOIN   order_lines ol ON ol.order_id = o.order_id
JOIN   variants v     ON v.variant_id = ol.variant_id
JOIN   products p     ON p.product_id = v.product_id
JOIN   cat_tree t     ON t.category_id = p.category_id
WHERE  o.status NOT IN ('cart','cancelled','refunded')
GROUP BY 1, 2
ORDER BY 1, revenue DESC;

-- 2. Best sellers with revenue share (ABC analysis)
WITH sales AS (
    SELECT v.sku, p.name,
           sum(ol.quantity)                AS units,
           sum(ol.quantity * ol.unit_price_minor) AS revenue_minor
    FROM   order_lines ol
    JOIN   orders o     ON o.order_id = ol.order_id
    JOIN   variants v   ON v.variant_id = ol.variant_id
    JOIN   products p   ON p.product_id = v.product_id
    WHERE  o.status NOT IN ('cart','cancelled','refunded')
    GROUP BY v.sku, p.name
)
SELECT sku, name, units, revenue_minor / 100.0 AS revenue,
       round(100.0 * sum(revenue_minor) OVER (ORDER BY revenue_minor DESC
                                              ROWS UNBOUNDED PRECEDING)
            / sum(revenue_minor) OVER (), 1) AS cumulative_pct
FROM   sales ORDER BY revenue_minor DESC;

-- 3. Stock needing reorder, including what is committed to unshipped orders
SELECT v.sku, p.name, v.stock, v.reorder_at,
       coalesce(sum(ol.quantity) FILTER (
           WHERE o.status IN ('placed','paid')), 0) AS committed
FROM   variants v
JOIN   products p ON p.product_id = v.product_id
LEFT JOIN order_lines ol ON ol.variant_id = v.variant_id
LEFT JOIN orders o     ON o.order_id = ol.order_id
GROUP BY v.variant_id, v.sku, p.name, v.stock, v.reorder_at
HAVING v.stock - coalesce(sum(ol.quantity) FILTER (
           WHERE o.status IN ('placed','paid')), 0) <= v.reorder_at

```



```

ORDER BY v.stock;

-- 4. Repeat-purchase rate
SELECT count(*) FILTER (WHERE orders >= 2) AS repeat_customers,
       count(*) AS customers_with_orders,
       round(100.0 * count(*) FILTER (WHERE orders >= 2) / count(*), 1) AS repeat_pct
FROM (SELECT customer_id, count(*) AS orders
      FROM orders
      WHERE status NOT IN ('cart', 'cancelled')
      GROUP BY customer_id) t;

-- 5. Each customer's most recent order (top-N per group)
SELECT DISTINCT ON (o.customer_id)
       c.full_name, o.order_id, o.placed_at, o.status, o.total_minor / 100.0 AS total
FROM orders o JOIN customers c ON c.customer_id = o.customer_id
WHERE o.status <> 'cart'
ORDER BY o.customer_id, o.placed_at DESC, o.order_id DESC;

-- 6. Abandoned carts: a cart with lines and no activity for 7 days
SELECT c.full_name, o.order_id,
       sum(ol.quantity * ol.unit_price_minor) / 100.0 AS cart_value
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
JOIN order_lines ol ON ol.order_id = o.order_id
WHERE o.status = 'cart'
GROUP BY c.full_name, o.order_id
ORDER BY cart_value DESC;

-- 7. Rating summary per product, including products with no reviews
SELECT p.name,
       count(r.review_id) AS reviews,
       round(avg(r.rating), 2) AS avg_rating,
       count(*) FILTER (WHERE r.rating = 5) AS five_star
FROM products p LEFT JOIN reviews r ON r.product_id = p.product_id
GROUP BY p.product_id, p.name
ORDER BY avg_rating DESC NULLS LAST;

-- 8. Checkout: decrement stock unraceably, then add the line
UPDATE variants SET stock = stock - 2
WHERE variant_id = 1 AND stock >= 2;
-- rowcount 0 → out of stock, reject; rowcount 1 → proceed with the insert

```

NOTE QUERY 3

Physical stock is not available stock. A variant with 5 units and 4 committed to unshipped orders has 1 available, and reordering at 5 would be too late. The `HAVING` compares `stock - committed` against the threshold — and this distinction between on-hand and available is the difference between an inventory report and an inventory system.

INTERVIEW QUESTIONS FROM THIS PROJECT

1. Why does stock live on `variants` rather than `products` ? What breaks otherwise?
2. Why store `unit_price_minor` on the order line when `variants.price_minor` exists?
3. How do you guarantee stock never goes negative with 500 concurrent checkouts?
4. Why is the shipping address both referenced *and* snapshotted?
5. How do you enforce one open cart per customer? One review per customer per product?
6. Write revenue by top-level category when categories nest arbitrarily deep.

32.3 Banking Database

● **ADVANCED** Worked in full. The project where correctness is the entire requirement — and where a stored balance is a design error.

REQUIREMENTS

A **customer** holds one or more **accounts** of various **types**. Money moves by **transactions**. A **transfer** moves money between two accounts and must be atomic. An account's **balance** must always be reconstructible and must never be wrong. Overdrafts are permitted only up to an account's limit. Every movement is **immutable** once recorded — a correction is a new compensating entry, never an edit. Statements are produced per account per month. The bank needs: current balances, monthly statements, unusual-activity detection, and the guarantee that total debits equal total credits at all times.

THE DESIGN DECISIONS THAT MATTER

Decision	Reasoning
Double-entry ledger	Every movement is two rows — one debit, one credit — summing to zero. This makes "debits equal credits" a checkable invariant rather than a hope, and it is how real accounting systems work.
No stored balance as truth	The balance is <code>sum(amount_minor)</code> over the ledger. A stored balance is a cache (see below), never the source of truth — because a cache can drift and a sum cannot.
Entries are immutable	No <code>UPDATE</code> , no <code>DELETE</code> . Revoke those privileges from the application role (Volume VI §28.4). A reversal is a new entry with the opposite sign.
Transfer is the parent entity	Both entries reference one <code>transfer_id</code> , so the pair is identifiable and the zero-sum rule is checkable per transfer.
Overdraft as a guarded write	The debit is only permitted if the resulting balance stays above the negative limit — checked in the same statement, so it cannot be raced.
Lock ordering on transfer	Both accounts locked in ascending id order, or two simultaneous mutual transfers deadlock (Volume IV §19.7).
Balance cache with reconciliation	A materialized balance for speed, maintained by trigger, with a nightly query that compares it to the ledger sum and alerts on any difference.

SCHEMA

```

CREATE TABLE customers (
    customer_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    full_name    text NOT NULL,
    kyc_ref      text NOT NULL UNIQUE,
    created_at   timestampz NOT NULL DEFAULT now()
);

CREATE TABLE account_types (
    type_code    text PRIMARY KEY,
    label        text NOT NULL,
    interest_bps integer NOT NULL DEFAULT 0,      -- basis points
    allows_overdraft boolean NOT NULL DEFAULT false
);
INSERT INTO account_types VALUES
    ('SAV','Savings',350,false), ('CUR','Current',0,true), ('FD','Fixed Deposit',700,false);

CREATE TABLE accounts (
    account_id    bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    account_no    text NOT NULL UNIQUE,          -- natural key, kept as UNIQUE
    customer_id   bigint NOT NULL REFERENCES customers (customer_id),
    type_code     text NOT NULL REFERENCES account_types (type_code),
    currency      char(3) NOT NULL DEFAULT 'INR',
    overdraft_minor bigint NOT NULL DEFAULT 0 CHECK (overdraft_minor >= 0),
    opened_on     date NOT NULL DEFAULT current_date,
    closed_on     date,
    CONSTRAINT accounts_closed_after_opened
        CHECK (closed_on IS NULL OR closed_on >= opened_on)
);
CREATE INDEX idx_accounts_customer ON accounts (customer_id);

CREATE TABLE transfers (
    transfer_id   bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    reference     text NOT NULL UNIQUE,          -- idempotency key
    description   text NOT NULL,
    created_at    timestampz NOT NULL DEFAULT now(),
    reverses_id   bigint REFERENCES transfers (transfer_id) -- corrections
);

-- The ledger. Append-only. This table is the truth.
CREATE TABLE entries (
    entry_id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    transfer_id   bigint NOT NULL REFERENCES transfers (transfer_id),
    account_id    bigint NOT NULL REFERENCES accounts (account_id),
    amount_minor  bigint NOT NULL CHECK (amount_minor <> 0), -- +credit, -debit
    posted_at     timestampz NOT NULL DEFAULT now(),
    value_date    date NOT NULL DEFAULT current_date
);
CREATE INDEX idx_entries_account_date ON entries (account_id, value_date, entry_id);
CREATE INDEX idx_entries_transfer ON entries (transfer_id);

```

```
-- The balance CACHE. Never the source of truth.  
CREATE TABLE account_balances (  
    account_id    bigint PRIMARY KEY REFERENCES accounts (account_id),  
    balance_minor bigint NOT NULL DEFAULT 0,  
    updated_at    timestamptz NOT NULL DEFAULT now()  
);
```

```

-- Maintain the cache inside the same transaction as the entry
CREATE OR REPLACE FUNCTION apply_entry_to_balance() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO account_balances (account_id, balance_minor)
    VALUES (NEW.account_id, NEW.amount_minor)
    ON CONFLICT (account_id) DO UPDATE
    SET     balance_minor = account_balances.balance_minor + NEW.amount_minor,
           updated_at     = now();
    RETURN NULL;
END $$;

CREATE TRIGGER entries_apply_balance
AFTER INSERT ON entries
FOR EACH ROW EXECUTE FUNCTION apply_entry_to_balance();

-- The transfer, done correctly: idempotent, deadlock-free, overdraft-safe
CREATE OR REPLACE FUNCTION transfer_money(
    p_reference text, p_from bigint, p_to bigint,
    p_amount_minor bigint, p_description text
) RETURNS bigint
LANGUAGE plpgsql AS $$
DECLARE
    v_transfer_id bigint;
    v_available    bigint;
BEGIN
    IF p_amount_minor <= 0 THEN
        RAISE EXCEPTION 'amount must be positive';
    END IF;
    IF p_from = p_to THEN
        RAISE EXCEPTION 'cannot transfer to the same account';
    END IF;

    -- Idempotency: a replayed request returns the original transfer
    SELECT transfer_id INTO v_transfer_id FROM transfers WHERE reference = p_reference;
    IF FOUND THEN RETURN v_transfer_id; END IF;

    -- Lock BOTH accounts in ascending id order: prevents deadlock
    PERFORM 1 FROM accounts WHERE account_id IN (p_from, p_to)
    ORDER BY account_id FOR UPDATE;

    -- Overdraft check against the ledger, not the cache
    SELECT coalesce(sum(e.amount_minor), 0) + a.overdraft_minor
    INTO   v_available
    FROM   accounts a LEFT JOIN entries e ON e.account_id = a.account_id
    WHERE  a.account_id = p_from
    GROUP BY a.overdraft_minor;

    IF v_available < p_amount_minor THEN

```

```

        RAISE EXCEPTION 'insufficient funds: % available, % requested',
            v_available, p_amount_minor USING ERRCODE = 'check_violation';
    END IF;

    INSERT INTO transfers (reference, description)
    VALUES (p_reference, p_description) RETURNING transfer_id INTO v_transfer_id;

    INSERT INTO entries (transfer_id, account_id, amount_minor) VALUES
        (v_transfer_id, p_from, -p_amount_minor),
        (v_transfer_id, p_to,   +p_amount_minor);

    RETURN v_transfer_id;
END $$;

```

SAMPLE DATA AND A RUN

```

INSERT INTO customers (full_name, kyc_ref) VALUES
    ('Asha Menon','KYC-1001'), ('Ravi Kumar','KYC-1002'), ('Meera Nair','KYC-1003');

INSERT INTO accounts (account_no, customer_id, type_code, overdraft_minor) VALUES
    ('SAV0000001',1,'SAV',      0),
    ('CUR0000002',1,'CUR',5000000),      -- ₹50,000 overdraft
    ('SAV0000003',2,'SAV',      0),
    ('SAV0000004',3,'SAV',      0);

-- Opening deposits: a transfer from outside needs a single-sided entry,
-- so model the bank's own cash account as account 5.
INSERT INTO accounts (account_no, customer_id, type_code) VALUES
    ('BNK0000005',1,'CUR');

SELECT transfer_money('OPEN-1', 5, 1, 1000000, 'Opening deposit');  -- ₹100,000
SELECT transfer_money('OPEN-2', 5, 3,  450000, 'Opening deposit');
SELECT transfer_money('TXN-100', 1, 3,  250000, 'Rent share');
SELECT transfer_money('TXN-100', 1, 3,  250000, 'Rent share');  -- replay: no-op
SELECT transfer_money('TXN-101', 3, 4,   80000, 'Dinner');

```

THE QUERIES

```

-- 1. Current balance from the LEDGER (the truth)
SELECT a.account_no, c.full_name,
       coalesce(sum(e.amount_minor), 0) / 100.0 AS balance
FROM   accounts a
JOIN   customers c ON c.customer_id = a.customer_id
LEFT   JOIN entries e ON e.account_id = a.account_id
GROUP  BY a.account_id, a.account_no, c.full_name
ORDER  BY a.account_no;

-- 2. Reconciliation: the cache against the ledger. Non-empty = alert.
SELECT a.account_no,
       b.balance_minor           AS cached,
       coalesce(sum(e.amount_minor), 0) AS actual
FROM   accounts a
JOIN   account_balances b ON b.account_id = a.account_id
LEFT   JOIN entries e     ON e.account_id = a.account_id
GROUP  BY a.account_no, b.balance_minor
HAVING b.balance_minor IS DISTINCT FROM coalesce(sum(e.amount_minor), 0);

-- 3. The system-wide invariant: every transfer must sum to zero
SELECT t.transfer_id, t.reference, sum(e.amount_minor) AS imbalance
FROM   transfers t JOIN entries e ON e.transfer_id = t.transfer_id
GROUP  BY t.transfer_id, t.reference
HAVING sum(e.amount_minor) <> 0;      -- must return zero rows, always

-- 4. Monthly statement with a running balance
SELECT e.value_date,
       t.description,
       CASE WHEN e.amount_minor < 0 THEN -e.amount_minor / 100.0 END AS debit,
       CASE WHEN e.amount_minor > 0 THEN  e.amount_minor / 100.0 END AS credit,
       sum(e.amount_minor) OVER (ORDER BY e.value_date, e.entry_id
                                ROWS UNBOUNDED PRECEDING) / 100.0 AS running_balance
FROM   entries e JOIN transfers t ON t.transfer_id = e.transfer_id
WHERE  e.account_id = 1
      AND e.value_date >= '2026-09-01' AND e.value_date < '2026-10-01'
ORDER  BY e.value_date, e.entry_id;

-- 5. Accounts in overdraft, with how deep
SELECT a.account_no, c.full_name,
       coalesce(sum(e.amount_minor), 0) / 100.0 AS balance,
       a.overdraft_minor / 100.0              AS limit_
FROM   accounts a
JOIN   customers c ON c.customer_id = a.customer_id
LEFT   JOIN entries e ON e.account_id = a.account_id
GROUP  BY a.account_id, a.account_no, c.full_name, a.overdraft_minor
HAVING coalesce(sum(e.amount_minor), 0) < 0
ORDER  BY 3;

-- 6. Unusual activity: a day's outflow above 4x the account's 90-day average

```



```

WITH daily AS (
    SELECT account_id, value_date,
           sum(-amount_minor) FILTER (WHERE amount_minor < 0) AS outflow
    FROM   entries
    WHERE  value_date >= current_date - 90
    GROUP BY account_id, value_date
),
stats AS (
    SELECT account_id, avg(outflow) AS avg_outflow
    FROM   daily GROUP BY account_id
)
SELECT d.account_id, d.value_date, d.outflow / 100.0 AS outflow,
       round(s.avg_outflow / 100.0, 2) AS avg_daily
FROM   daily d JOIN stats s ON s.account_id = d.account_id
WHERE  d.outflow > 4 * s.avg_outflow
ORDER BY d.outflow DESC;

-- 7. Reverse a transfer: a new compensating transfer, never an edit
INSERT INTO transfers (reference, description, reverses_id)
VALUES ('REV-TXN-101', 'Reversal of TXN-101',
        (SELECT transfer_id FROM transfers WHERE reference = 'TXN-101'))
RETURNING transfer_id;

INSERT INTO entries (transfer_id, account_id, amount_minor)
SELECT (SELECT transfer_id FROM transfers WHERE reference = 'REV-TXN-101'),
       account_id, -amount_minor
FROM   entries
WHERE  transfer_id = (SELECT transfer_id FROM transfers WHERE reference = 'TXN-101');

```

WHY QUERY 3 IS THE MOST IMPORTANT LINE IN THE PROJECT

`HAVING sum(e.amount_minor) <> 0` must return zero rows on every run, forever. It is a single query that proves the entire ledger is internally consistent — no lost half-transfer, no partial write, no rounding leak. Run it in monitoring, not just in tests. A financial system without a continuously-checked global invariant is a system that will one day be wrong in a way nobody can reconstruct.

WHY NOT JUST STORE THE BALANCE

Because a stored balance can only ever be as correct as the last code path that wrote it, and there will be paths you did not think of — a migration, a support script, a partially-applied batch. The ledger sum is correct by construction: to change a balance you must post an entry, and every entry belongs to a transfer that must sum to zero. The cache exists purely because summing millions of entries per page load is slow, and it is safe *only* because query 2 is watching it.

INTERVIEW QUESTIONS FROM THIS PROJECT

1. Why is a stored balance a design error? What is stored instead?
2. Write the invariant query proving debits equal credits.
3. Why does `transfer_money` lock both accounts in ascending id order?
4. The transfer request is retried after a timeout. Why is money not moved twice?
5. How do you correct a mistaken transfer without violating immutability?
6. Write a statement with a running balance. Which frame clause, and why?

32.4–32.15 The remaining twelve

Requirements, the design decisions that matter, and the queries to write. Build each one from the requirements before reading the decisions — that order is what makes it practice rather than reading.

32.4 Student Management System ● BEGINNER

Requirements. Students enrol in courses each term. A course is taught by an instructor in a room at a timetable slot. Enrolment produces attendance records and one grade per assessment. Compute term GPA, attendance percentage, and who is at risk of failing.

Decisions. `course` versus `course_offering` — the same course runs each term with a different instructor, so the offering is the entity you enrol in. Grades belong to `(enrolment, assessment)`, not to the enrolment. A room cannot host two offerings in one slot: an exclusion constraint on `(room_id, slot_range)`. Grade points as a lookup table, not a `CASE` in every query.

Queries. Term GPA per student, weighted by credits · attendance percentage per enrolment · students below 75% attendance · courses with the widest grade spread · instructor load per term · timetable clashes for a given student.

32.5 Employee Management System ● BEGINNER

Requirements. Employees belong to departments, report to a manager, hold a position at a salary, and take leave from an annual allowance. Salary and department change over time and history must be recoverable.

Decisions. Salary and department as validity-interval tables (Volume IV §15.5), with an exclusion constraint preventing overlapping spans. Self-referencing `manager_id`, nullable for the CEO. Leave balance is derived from an allowance minus approved leave, never stored. A composite foreign key to enforce "manager in the same department" if that rule holds (Volume IV §17 challenge 1).

Queries. Org chart with depth and path · salary as at any date · employees above their department average · leave balance per employee · reports at every level below a manager · salary change history with the percentage step each time.

32.6 Hospital Database ● INTERMEDIATE

Requirements. Patients book appointments with doctors, are admitted to beds in wards, receive prescriptions and lab tests, and generate bills. A doctor cannot be double-booked; a bed holds one patient at a time.

Decisions. Two exclusion constraints carry the whole design: `(doctor_id, appointment_range)` and `(bed_id, occupancy_range)`. Prescription lines snapshot the drug name and dose as prescribed. Diagnoses use a coded lookup, so reporting is possible. Admission and discharge as a range, not two nullable dates, so "who was in hospital on this date" is one overlap test.

Queries. Ward occupancy on any date · average length of stay by diagnosis · doctors with the most appointments per month · patients readmitted within 30 days · unbilled discharges · drug interaction checks across a patient's active prescriptions.

32.7 Food Delivery Database ● INTERMEDIATE

Requirements. Customers order from restaurants; a courier collects and delivers. Menus change; items go out of stock during service. Orders pass through a status timeline with timestamps. Couriers have shifts and locations. Delivery-time promises must be measured against reality.

Decisions. The status *timeline* is an append-only `order_events` table, not a single mutable `status` column — you need the durations between transitions, and a column throws them away. Menu-item price snapshot on the line. Restaurant opening hours as ranges per weekday. Courier assignment via `FOR UPDATE SKIP LOCKED` (Volume IV §18.5) — a job queue by another name.

Queries. Median and p95 delivery time by city and hour · orders breaching their promise · courier utilisation per shift · restaurants by acceptance rate · sessionize a customer's ordering behaviour · funnel from cart to delivered with drop-off at each stage.

32.8 Movie Booking Database ● INTERMEDIATE

Requirements. Cinemas have screens with fixed seats. A movie plays at showings on a screen. Customers book seats for a showing; a seat may be sold once per showing. Bookings expire if unpaid within ten minutes. Prices vary by seat class and by time of day.

Decisions. The unraceable seat rule is `showing_seats(showing_id, seat_id, booking_id)` with `(showing_id, seat_id)` as the primary key — the seat-per-showing *is* the entity, and its key *is* the business rule (Volume I §1, challenge 2). Expiry via a `held_until` timestamp and a partial index, not a background scan of everything. Price as a rule table, resolved at booking and snapshotted.

Queries. Seat map for a showing with availability · occupancy rate per showing and per movie · revenue by seat class · expired holds to release · best-performing time slots · movies whose occupancy declined three weeks running.

32.9 Social Media Database ● ADVANCED

Requirements. Users follow users, post content, like and comment. Comments nest. A user's feed shows recent posts from those they follow, paginated. Counts of likes and comments are displayed on every post.

Decisions. Follows are a self-referencing M:N with a composite key and a `CHECK (follower_id <> followee_id)`. Nested comments as an adjacency list plus a materialised path — recursion on every read does not survive contact with a popular thread (Volume III §11 challenge 4). Counter caches maintained by trigger, with the hot-row contention caveat. Keyset pagination on `(created_at DESC, post_id DESC)`, never `OFFSET`.

Queries. The feed, keyset-paginated · mutual follows · suggested users (followed by people you follow, not yet followed by you) · a comment thread in display order · engagement rate per post · 7-day and 30-day retention cohorts.

32.10 Inventory Management System ● INTERMEDIATE

Requirements. Products are stocked in multiple warehouses. Stock moves in (receipts), out (shipments) and between warehouses (transfers). Stock levels must be reconstructible at any past date. Reorder points trigger purchase orders to suppliers.

Decisions. Same shape as the bank: an append-only `stock_movements` ledger is the truth, and a per-warehouse level is a reconciled cache. A transfer is two movements sharing a parent id. Cost tracked per receipt so FIFO or weighted-average valuation is possible. Never a mutable `quantity` column as the source of truth.

Queries. Stock on hand by product and warehouse · stock as at any historical date · inventory valuation FIFO and weighted-average · items below reorder point net of inbound purchase orders · slow-moving stock (no outbound movement in 90 days) · shrinkage by warehouse.

32.11 Job Portal Database ● INTERMEDIATE

Requirements. Companies post jobs; candidates hold profiles with skills and experience and apply to jobs. Applications move through a pipeline with stages. Search is by skill, location, salary band and full text.

Decisions. Skills as a controlled vocabulary with a junction table carrying proficiency and years — not a text list, and not a free-text field per candidate. The application pipeline as an event table so stage durations are measurable. Salary as a range type, so "jobs overlapping my expectation" is one overlap test. Full-text search with a GIN index on a `tsvector`; skill filtering with a plain junction join, because relational beats full-text when the vocabulary is closed.

Queries. Candidates matching all required skills (relational division) · candidates matching at least N of them, ranked · average time in each pipeline stage · jobs with no applicants after 14 days · salary distribution by role and city · companies by offer-acceptance rate.

32.12 Learning Management System ● INTERMEDIATE

Requirements. Courses contain ordered modules containing ordered lessons. Students enrol, watch lessons, take quizzes with multiple attempts, and earn certificates on completion. Progress is shown as a percentage.

Decisions. Ordering by a sortable position column, sparsely numbered so an insertion between two lessons needs no renumbering. Quiz attempts append-only; the score shown is derived by policy (best, last, or first) — store the policy, not the answer. Progress derived from completed lessons over total, never stored. Certificate issued once, enforced by a unique constraint.

Queries. Progress percentage per enrolment · the next incomplete lesson for a student · quiz pass rates per question (to find bad questions) · drop-off by lesson position (where students abandon) · time-to-completion distribution · students inactive for 14 days mid-course.

32.13 Expense Management System ● EASY

Requirements. Employees submit expense claims with line items, attach receipts, and route them through an approval chain that depends on amount. Reimbursement is paid in batches. Multi-currency, with the conversion rate fixed at submission.

Decisions. Original amount, currency, rate and rate timestamp all stored (Volume IV §15 challenge 4) — a converted amount alone destroys the claim. Approval as an event table so the chain and its timings are auditable. Approval thresholds as data, not code. Category as a lookup with a spending policy attached.

Queries. Claims pending approval beyond SLA · spend by category and cost centre per quarter · claims breaching policy limits · average approval turnaround by approver · duplicate-receipt detection · reimbursement batch reconciliation.

32.14 Railway Booking Database ● ADVANCED

Requirements. Trains run on routes calling at stations in order on given days. Passengers book a seat in a class between two stations on a date. A seat may be sold twice on the same journey if the two bookings cover non-overlapping station spans. Waitlists and cancellations apply.

Decisions. The hardest booking rule in this part: availability depends on *segment* overlap, not on the whole journey. Model station calls with a sequence number, express a booking as an integer range over those sequences, and use an exclusion constraint on `(train_run_id, seat_id, seq_range)` with `&&`. Waitlist position by timestamp, promoted on cancellation. Fare as a function of segment distance and class, snapshotted.

Queries. Seats available between two stations on a date · waitlist position and promotion order · occupancy per segment (which legs are full) · revenue per route per class · cancellation rate by days-before-departure · trains where a mid-route segment is the bottleneck.

32.15 SaaS Application Database ● ADVANCED

Requirements. Organisations subscribe to plans; users belong to organisations with roles. Usage is metered and billed monthly. Plans and prices change; invoices from two years ago must reproduce exactly. Tenants must not see each other's data. Per-tenant custom fields are required.

Decisions. Every tenant-scoped table carries `org_id`, with row-level security keyed on a `SET LOCAL` session variable and both `USING` and `WITH CHECK` policies — and the application connecting as a non-owner (Volume VI §28.5). Plan prices versioned, never updated; invoice lines snapshot amount and description. Usage events partitioned by month with BRIN. Custom fields as `jsonb` plus a field-definitions table (Volume V §26 challenge 3).

Queries. MRR and its month-over-month movement, split into new, expansion, contraction and churn · usage against plan limits per org · invoice reproduction for any past month · seat utilisation per org · churn cohorts by signup month · orgs approaching a plan limit.

THE THROUGH-LINE

Notice how few ideas these fifteen projects actually use. **An append-only ledger with a reconciled cache** answers banking and inventory. **An exclusion constraint over a range** answers hospital beds, cinema seats, hotel rooms, room bookings, timetables and railway segments. **A snapshot of an agreed value** answers every project with money in it. **An event table instead of a mutable status** answers food delivery, job pipelines, expense approvals and order lifecycles. Learn those four and you have the structural vocabulary for almost any transactional system.

Interview Preparation

Seven hundred questions across five bands. The ones that carry weight are worked in full: the question, the thinking an interviewer is listening for, the answer, the wrong approach most candidates give, and the one sentence that turns a correct answer into a convincing one.

HOW TO USE THIS PART

Answer **out loud**, to a wall if necessary. SQL interviews are spoken, and the failure mode is not ignorance — it is knowing the answer and constructing it badly under pressure. Give yourself two minutes per conceptual question and five per query.

A worked answer here is a *model*, not a script. Say it in your own words or it will sound rehearsed, which reads worse than a hesitant original answer.

33.1 What interviewers are actually testing

Four things, and only one of them is SQL knowledge. Knowing this changes how you answer.

They are testing	How it shows	How to satisfy it
Whether you know the mechanism	They ask "why" after you answer "what"	Always volunteer one sentence of mechanism — "because <code>WHERE</code> runs before the <code>SELECT</code> list"
Whether you notice edge cases	The question has a <code>NULL</code> , a tie, or an empty set hidden in it	Say the edge case before they ask. Ties, <code>NULL</code> s and zero rows are the three
Whether you ask before assuming	The requirement is deliberately ambiguous	Ask one clarifying question, then state your assumption and proceed
Whether you have shipped this	They ask what went wrong last time	Mention the operational consequence — the lock, the retry, the migration

THE THREE SENTENCES THAT IMPROVE ALMOST ANY ANSWER

"Let me check the grain first" — before writing an aggregate, say what one row means. "There's a tie case here" — before choosing `row_number()` over `rank()`, say why. "That works, but in production I'd..." — after a correct answer, name the operational caveat. Each takes five seconds and each signals experience.

THE FIVE MOST COMMON WAYS CANDIDATES LOSE

1. Writing SQL immediately without confirming the schema or the grain.
2. Using `NOT IN` with a subquery. It signals you have not been bitten by `NULL`s yet.
3. Answering "how would you optimise this" with "add an index" and stopping.
4. Reciting normal forms without being able to spot a violation in a table put in front of you.
5. Silence while thinking. Narrate — "I need one row per customer, so I'm going to rank inside each customer and filter outside" — because they are scoring reasoning, not typing.

33.2 Beginner questions

● **BEGINNER** 100 questions. Fifteen worked in full; the rest are answerable from Volumes I and II — if one stops you, that is a reading list, not a gap in this book.

B1 — ASKED IN ALMOST EVERY SCREEN

Q. What is the difference between `WHERE` and `HAVING` ?

Answer. `WHERE` filters rows before grouping; `HAVING` filters groups after. That is the whole distinction, and it follows from the evaluation order: `WHERE` is step 3, `GROUP BY` step 4, `HAVING` step 5.

Expected thinking. They want the order, not just the words. Give an example where both appear: "count the delivered orders per customer, but only customers with more than two — the status filter is `WHERE` because it is per row, the count filter is `HAVING` because it is per group."

Common wrong answer. "`HAVING` is for aggregates." Nearly right and it misses the point — you can write `HAVING department_id = 3`, which is legal and simply less efficient than the `WHERE` equivalent.

Tip. Add: "So a filter that does not involve an aggregate belongs in `WHERE`, because it removes rows before the grouping work happens."

B2

Q. What does `NULL` mean, and how do you test for it?

Answer. Unknown or not applicable — not zero, not an empty string, not false. Because it is unknown, `NULL = NULL` is not true but unknown, which is why `IS NULL` exists. Any arithmetic or comparison involving `NULL` yields `NULL`.

Tip. Volunteer the three exceptions where `NULL` s are treated as equal: `GROUP BY` puts them in one group, `DISTINCT` collapses them, and set operations like `INTERSECT` match them. Knowing the exceptions is what marks real experience.

B3

Q. What is the difference between `DELETE`, `TRUNCATE` and `DROP` ?

Answer. `DELETE` is DML: removes rows one at a time, takes a `WHERE`, fires row triggers, is logged per row, and is transactional. `TRUNCATE` is DDL: removes all rows by deallocating storage, takes no `WHERE`, does not fire row triggers, and may reset identity sequences. `DROP` removes the table itself.

Tip. The nuance that earns the point: in PostgreSQL `TRUNCATE` is still transactional and can be rolled back; in MySQL and Oracle it cannot. And add the operational consequence — `TRUNCATE` bypassing row triggers means any trigger-maintained aggregate silently goes stale.

B4

Q. What is a primary key? Can a table have two?

Answer. One column or set of columns that uniquely identifies each row: unique and never `NULL`. A table has at most one primary key, though that key may be *composite* — several columns. Additional uniqueness is expressed with `UNIQUE` constraints.

Common wrong answer. "Yes, a composite key." The vocabulary is exactly what is being tested; a composite key is *one* key made of several columns.

B5

Q. Explain the join types.

Answer. `INNER` keeps rows matching on both sides. `LEFT` keeps all left rows, padding unmatched right columns with `NULL`. `RIGHT` is the mirror. `FULL OUTER` keeps everything from both. `CROSS` is every combination. A `SELF` join is a table joined to itself with two aliases.

Expected thinking. Do not draw Venn diagrams — they describe set membership, not row multiplication, and they mislead on duplicates. Say instead: "For each left row, find the matching right rows; if there are three matches, that left row appears three times."

Tip. Add when you would use `LEFT`: "whenever the answer must include the zero case — customers with no orders, employees with no manager. An `INNER JOIN` silently drops them, which is the most common cause of totals that do not match."

B6

Q. What is the difference between `UNION` and `UNION ALL` ?

Answer. `UNION` removes duplicates, which requires sorting or hashing the whole combined result. `UNION ALL` concatenates and is much faster.

Tip. "I default to `UNION ALL` unless I specifically need deduplication, because `UNION` pays for a distinct operation whether or not duplicates exist — and often the branches cannot overlap by construction."

B7

Q. What does `DISTINCT` do, and when is it a smell?

Answer. Removes duplicate rows from the result, considering all selected columns together. It is a smell when it appears *after* a join — usually the join is fanning out and `DISTINCT` is hiding it rather than fixing it.

Tip. "If I find myself adding `DISTINCT` to fix a row count, I go back and ask which join multiplied the rows — because `DISTINCT` will also silently collapse legitimately duplicate rows, and it costs a sort."

B8

Q. What is a foreign key and what does it guarantee?

Answer. A column whose values must exist as a key in another table — referential integrity. It makes an orphan row impossible: you cannot have an order referencing a customer who does not exist, and you cannot delete that customer without deciding what happens to the order.

Tip. Volunteer the referential actions and that the choice is deliberate: "`CASCADE` where the child cannot exist without the parent, like order lines; `RESTRICT` where deletion should be refused, like a product that has been sold." Then the detail almost nobody mentions: PostgreSQL does **not** index the referencing column, so every foreign key you join on needs an explicit index.

B9

Q. What is the difference between `CHAR` and `VARCHAR` ? Which should you use?

Answer. `CHAR(n)` is blank-padded to a fixed length; `VARCHAR(n)` stores only what you put in, up to a limit. In PostgreSQL there is no performance advantage to either, and `text` is identical to `varchar` without a limit.

Tip. "In PostgreSQL I use `text` and enforce length with a `CHECK` if there is a real business rule, because changing a `varchar(50)` to `varchar(100)` is a migration while a check constraint is a one-line change. `CHAR` only for genuinely fixed-width codes like a currency code."

B10

Q. How do you get the second-highest salary?

Answer.

```
SELECT full_name, salary FROM (  
    SELECT full_name, salary, dense_rank() OVER (ORDER BY salary DESC) AS dr  
    FROM employees) t  
WHERE dr = 2;
```

Expected thinking. The question is about ties. `dense_rank()` gives the second *distinct* salary and returns everyone who holds it. `OFFSET 1 LIMIT 1` looks simpler and is wrong when two people share the top salary — it returns the other top earner as the "second highest".

Tip. Say the tie case before they ask, and mention the portable alternative for engines without window functions: `WHERE salary = (SELECT max(salary) FROM employees WHERE salary < (SELECT max(salary) FROM employees))`.

B11

Q. Find duplicate rows in a table.

Answer.

```
-- Which values are duplicated, and how many times  
SELECT email, count(*) AS copies  
FROM customers GROUP BY email HAVING count(*) > 1;  
  
-- Delete all but the oldest of each group  
DELETE FROM customers c  
WHERE EXISTS (SELECT 1 FROM customers k  
              WHERE k.email = c.email AND k.customer_id < c.customer_id);
```

Tip. Add: "Then I'd add the unique index that should have been there, so it cannot recur — the query is the cleanup, the constraint is the fix."

B12

Q. What is the difference between an aggregate function and a window function?

Answer. An aggregate collapses many rows into one. A window function computes across a set of related rows without collapsing them — the detail rows survive and each carries the computed value.

Tip. The one-line example that lands it: "`avg(salary)` with `GROUP BY department_id` gives me four rows; `avg(salary) OVER (PARTITION BY department_id)` gives me every employee with their department average beside their own salary."

B13

Q. What is an index, and what does it cost?

Answer. A sorted, balanced structure stored separately from the table, holding indexed values with pointers to rows. A lookup descends four or five levels instead of reading every page, and because the leaves are sorted and linked it also serves ranges and can satisfy `ORDER BY` without a sort. The cost is that every write must maintain it, plus storage and cache pressure.

Tip. Mentioning the write cost unprompted is what distinguishes this answer. Then: "which is why I check `pg_stat_user_indexes` for indexes with zero scans before adding another."

B14

Q. What is a transaction? Explain ACID.

Answer. A group of statements that either all take effect or none do. **A**tomicity — no half-transfer. **C**onsistency — every constraint holds at commit. **I**solation — concurrent transactions do not see each other's incomplete work. **D**urability — once committed, it survives a crash.

Tip. Use one running example — a bank transfer — for all four letters, and add the implementation: "In PostgreSQL these come from MVCC and the write-ahead log; the log is flushed before a commit is acknowledged, which is exactly what durability costs."

B15

Q. What is normalization? Why would you not do it?

Answer. Organising a schema so every fact is stored in exactly one place. The rule that generates the forms: every non-key column must depend on the key, the whole key, and nothing but the key. You denormalize deliberately when a measured read is too slow, indexes and rewrites are exhausted, the workload is read-heavy, *and* you have a mechanism keeping the duplicate correct plus a reconciliation query detecting drift.

Tip. The distinction that impresses: "A price snapshot on an order line looks like denormalization and is not — the price charged and the current price are two different facts. Real redundancy is storing one fact twice."

THE REMAINING 85 — ANSWER EACH IN TWO MINUTES

- 16** What is SQL, and what does "declarative" mean?
- 17** Name the five SQL sublanguages with one command each.
- 18** Difference between a database, a DBMS and an RDBMS.
- 19** Where does the word "relational" come from?
- 20** Define degree and cardinality of a table.

- 21 Give the logical evaluation order of a `SELECT`, all nine steps.
- 22 Why can you use a column alias in `ORDER BY` but not in `WHERE`?
- 23 What is the difference between `GROUP BY` and `DISTINCT`?
- 24 What does `count(*)` count that `count(column)` does not?
- 25 Why does `avg()` ignore `NULL`s, and when is that wrong?
- 26 What is a composite key? Give an example.
- 27 Distinguish super key, candidate key, primary key and alternate key.
- 28 Natural or surrogate primary key? Defend a choice for a person and for a country.
- 29 How do you model a many-to-many relationship?
- 30 On which side of a one-to-many does the foreign key live, and why?
- 31 What is a self-join and when do you need one?
- 32 What is a `CROSS JOIN` and when is it legitimate?
- 33 What does `ON` do that `WHERE` does not, in an outer join?
- 34 Write a query listing every customer and their order count, including those with none.
- 35 What is the difference between `LIKE` and `ILIKE`?
- 36 Why is `LIKE '%term%'` slow, and what fixes it?
- 37 What does `COALESCE` do? And `NULLIF`?
- 38 Write a `CASE` expression bucketing salaries into three bands.
- 39 What is the difference between `BETWEEN` and two comparisons?
- 40 Why is `BETWEEN` dangerous with timestamps?
- 41 Why should money never be stored as `float`?
- 42 Difference between `timestamp` and `timestampz`. Which do you default to?
- 43 What is an `INTERVAL`? Add three months to a date.
- 44 How do you extract the year from a date? Name two ways.
- 45 What is the difference between `DATE_TRUNC` and `EXTRACT`?
- 46 What is a `UUID` and when would you use one as a key?
- 47 What is a `BOOLEAN` in PostgreSQL, and which engine lacks one?
- 48 What does `LIMIT` without `ORDER BY` return?
- 49 How do you paginate? Why is `OFFSET` slow on page 5,000?
- 50 What is keyset pagination?
- 51 Sort so that `NULL`s come last.

- 52** What is a subquery? Name the three positions one can occupy.
- 53** What is a correlated subquery?
- 54** What does `EXISTS` return, and why `SELECT 1` ?
- 55** Why is `NOT IN` with a subquery dangerous?
- 56** What is a CTE, and name three advantages over a derived table.
- 57** What is a view? Does it improve performance?
- 58** Difference between a view and a materialized view.
- 59** What makes a view updatable?
- 60** Name the six constraint types and what each guarantees.
- 61** Does `CHECK (salary > 0)` reject a `NULL` salary?
- 62** What is a `DEFAULT` , and why is it not really a constraint?
- 63** Why enforce a rule in the database rather than the application?
- 64** What is `AUTO_INCREMENT` 's PostgreSQL equivalent?
- 65** What is a sequence? What happens to gaps after a rollback?
- 66** What does `RETURNING` do?
- 67** How do you do an upsert in PostgreSQL?
- 68** What is the difference between `ALTER` and `UPDATE` ?
- 69** How do you copy a table's structure without its data?
- 70** How do you insert the results of a query into a table?
- 71** What is a schema, in both senses of the word?
- 72** What is `search_path` ?
- 73** Name the four levels of containment from server to table in PostgreSQL.
- 74** What is client–server architecture? Which database is not?
- 75** Why is one query returning 500 rows faster than 500 returning one?
- 76** What is SQL injection, in one sentence?
- 77** How do parameterised queries prevent it? (It is not escaping.)
- 78** What is OLTP versus OLAP? Name three differences.
- 79** When would you choose NoSQL? Be specific about the access pattern.
- 80** What are the three grades of data structure, and which is SQL for?
- 81** Write a query for the top five products by revenue.
- 82** Write a query for orders placed in the last 30 days.

- 83** Write a query for the average order value per customer.
- 84** Write a query for departments with more than three employees.
- 85** Write a query for employees hired in the same month as anyone else.
- 86** Write a query for the total revenue per month for the current year.
- 87** Write a query for customers who ordered in January but not February.
- 88** Write a query counting orders by status, including statuses with zero.
- 89** Write a query for the longest-serving employee per department.
- 90** Write a query concatenating first and last name, handling a `NULL` middle name.
- 91** Write a query finding email addresses that are not valid-looking.
- 92** Write a query rounding an average to two decimal places.
- 93** Write a query returning each employee's tenure in whole years.
- 94** Write a query listing products never ordered.
- 95** Write a query for the percentage of orders that were cancelled.
- 96** Write a query showing each department's headcount and its share of the total.
- 97** Write a query for the day of the week with the most orders.
- 98** Write a query deleting orders older than five years, safely.
- 99** Write a query updating all prices in one category by 10%.
- 100** Explain a query you have written that you were proud of, and why.

33.3 Intermediate questions

● **INTERMEDIATE** 150 questions. This band is where most candidates are actually sorted — the answers require a mechanism, not a definition.

I1 — THE HIGHEST-VALUE QUESTION IN THIS PART

Q. You join orders to order items and the revenue total doubles. What happened?

Answer. Join fan-out. Each order row is repeated once per matching line item, so any aggregate over a column belonging to the *order* — its `total_amount` — is counted once per line. The order's total is at order grain; the join produced line grain.

Expected thinking. Diagnose it, do not just fix it: "I'd check whether the row count changed after the join. If it did, the aggregate is now at a different grain than the column I'm summing."

Answer in SQL. Aggregate before joining, or sum the line-level columns instead:

```
-- Aggregate to order grain first, THEN join
SELECT c.full_name, sum(t.line_total) AS revenue
FROM   customers c
JOIN   orders o ON o.customer_id = c.customer_id
JOIN   (SELECT order_id, sum(quantity * unit_price) AS line_total
        FROM   order_items GROUP BY order_id) t ON t.order_id = o.order_id
GROUP BY c.customer_id, c.full_name;
```

Common wrong approach. Adding `DISTINCT`, or `sum(DISTINCT total_amount)`. The first does not fix an aggregate; the second is worse, because two orders with the same total collapse into one.

Tip. Say the word "grain". It is the vocabulary of people who have debugged this in production.

I2

Q. Write "the most recent order per customer".

Answer.

```
WITH ranked AS (  
    SELECT o.*,  
           row_number() OVER (PARTITION BY o.customer_id  
                               ORDER BY o.order_date DESC, o.order_id DESC) AS rn  
    FROM   orders o  
)  
SELECT * FROM ranked WHERE rn = 1;
```

Expected thinking. Two things must be said aloud. **Why the wrapper:** window functions are evaluated after `WHERE`, so `WHERE row_number() = 1` is an error — you must rank in a CTE and filter outside. **Why the tie-break:** `ORDER BY order_date DESC` alone is non-deterministic if a customer has two orders on the same date, so the primary key goes in as a second key.

Tip. Then offer the PostgreSQL shortcut and say why you would not always use it: "`SELECT DISTINCT ON (customer_id) ... ORDER BY customer_id, order_date DESC` is shorter and often faster, but it is PostgreSQL-only, so the window form is what I'd write in portable code."

I3

Q. `IN` versus `EXISTS` versus a join — which do you use?

Answer. Decide on semantics, not folklore: **need columns from the other table** → **join**, and check for fan-out. **Only need to know whether a related row exists** → `EXISTS`, which cannot fan out, is `NULL`-safe and short-circuits. `IN` for a short literal list.

Common wrong answer. "`EXISTS` is faster because it stops at the first row." True of the subquery scan, not of the overall plan — in modern PostgreSQL `IN` and `EXISTS` usually plan identically as a hash semi join.

Tip. "The one case where it genuinely matters is `NOT IN`, and that is a correctness issue, not a performance one."

I4

Q. Why does `NOT IN` return no rows on production but work in test?

Answer. Production data contains a `NULL` in the subquery's column and the test data does not. `x NOT IN (1, 2, NULL)` expands to `x <> 1 AND x <> 2 AND x <> NULL`, and the last term is unknown — so the whole predicate can never be true and every row is filtered out. No error, no warning, an empty report.

Answer. Use `NOT EXISTS`, which asks "did any row come back" — a two-valued question that `NULL`s cannot poison.

Tip. Add the second-order lesson: "It also tells me my test fixtures are tidier than production, which hides exactly this class of bug — so I'd seed `NULL`s deliberately."

I5

Q. Explain the default window frame, and a bug it has caused.

Answer. With no `ORDER BY` in the `OVER` clause, the frame is the whole partition. **Adding `ORDER BY` silently changes it** to `RANGE UNBOUNDED PRECEDING TO CURRENT ROW` — so the aggregate becomes cumulative. `avg(salary) OVER (PARTITION BY dept)` is the department average; `avg(salary) OVER (PARTITION BY dept ORDER BY hire_date)` is the average of everyone hired up to and including this row.

The bug. A report labelled "department average" showing the cumulative column — plausible numbers, wrong on every row but the last, and no error anywhere.

Tip. Volunteer the related trap: "`last_value` with a default frame returns the current row, not the partition's last, for the same reason — I always give it an explicit `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`."

I6

Q. Write a running total and a 7-day moving average.

```
SELECT order_date, revenue,
       sum(revenue) OVER (ORDER BY order_date
                          ROWS UNBOUNDED PRECEDING)           AS running_total,
       round(avg(revenue) OVER (ORDER BY order_date
                                RANGE BETWEEN INTERVAL '6 days' PRECEDING
                                AND CURRENT ROW), 2)          AS rolling_7d
FROM   daily_revenue ORDER BY order_date;
```

Expected thinking. The `RANGE ... INTERVAL` is the point. `ROWS 6 PRECEDING` gives the last 7 rows, which is only the last 7 days if every day has exactly one row. Say both halves: "a true 7-day window needs `RANGE` for the time semantics *and* a gap-filled series so a day with no sales contributes a zero rather than being skipped."

I7

Q. What is a recursive CTE? Write one for an org chart.

```
WITH RECURSIVE org AS (
  SELECT employee_id, full_name, manager_id, 1 AS depth,
         full_name::text AS path
  FROM   employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.employee_id, e.full_name, e.manager_id, o.depth + 1,
         o.path || ' > ' || e.full_name
  FROM   employees e JOIN org o ON o.employee_id = e.manager_id
  WHERE  o.depth < 20
)
SELECT repeat('  ', depth - 1) || full_name AS chart, depth
FROM   org ORDER BY path;
```

Expected thinking. Three parts: anchor, recursive term referencing the CTE, and the outer query. Recursion stops when an iteration returns zero rows.

Tip. Two details most candidates miss, and both should be volunteered: `ORDER BY path , not depth` — path sorts each subtree under its parent, depth groups all level-2 staff together and destroys the tree; and **the depth guard**, because real hierarchy data contains cycles.

18

Q. Explain gaps and islands, and find each user's longest streak of active days.

Answer. The invariant: within a run of consecutive dates, `date - row_number()` is constant, because both increase by one. Group by that difference and each group is one streak.

```
WITH days AS (SELECT DISTINCT user_id, activity_date FROM activity),
numbered AS (
    SELECT user_id, activity_date,
           row_number() OVER (PARTITION BY user_id ORDER BY activity_date) AS rn
    FROM   days),
islands AS (
    SELECT user_id, activity_date - (rn || ' days')::interval AS grp
    FROM   numbered)
SELECT user_id, max(len) AS longest_streak
FROM   (SELECT user_id, grp, count(*) AS len FROM islands GROUP BY user_id, grp) s
GROUP BY user_id ORDER BY longest_streak DESC;
```

Tip. The `DISTINCT` in step one is load-bearing — two events on one day would give that day two row numbers and split a real streak in half. Say the invariant out loud before writing anything.

Common wrong approach. A self-join on `date = date + 1`, which finds adjacent pairs but cannot measure length without one join per day.

19

Q. How do you read an `EXPLAIN ANALYZE` plan?

Answer. Inside out, most-indented first. Then four things in order: **(1)** compare estimated `rows=` against actual — a large gap means a statistics problem and everything downstream is suspect; **(2)** find the node where time accumulates, remembering that `actual time` is per loop, so multiply by `loops=`; **(3)** look for `Rows Removed by Filter`, which means rows were read and thrown away; **(4)** check `BUFFERS` for whether the cost is I/O.

Tip. Name the signature disaster: "A nested loop whose outer estimate was 12 and whose actual was 280,000 — the inner index scan then ran 280,000 times. Every catastrophic plan I've seen looks like that, and the fix is never to force a hash join; it's to find out why the estimate was wrong."

I10

Q. Design a composite index for this query, and justify the column order.

```
-- WHERE customer_id = ? AND status = ? ORDER BY order_date DESC  
CREATE INDEX idx_orders_cust_status_date  
ON orders (customer_id, status, order_date DESC);
```

Answer. Equality columns first, so the index can seek straight to the matching group; the ordering column last, so the rows emerge already sorted and the plan contains no `Sort`. Once an index reaches a range column it cannot seek on anything after it, because entries are only sorted within equal values of the preceding columns.

Tip. State the leftmost-prefix rule — an index on `(a,b,c)` serves `a`, `a,b` and `a,b,c` but not `b` alone — and then: "which means separate single-column indexes on the same three columns would not serve this query well, and would not help the `ORDER BY` at all."

I11

Q. Two users each add stock and one increment is lost. Why, and how do you fix it?

Answer. Lost update. Both transactions read the old value and both wrote a value computed from it; the second write discarded the first. Fix: make the write relative, so the arithmetic happens inside the database atomically — `UPDATE products SET stock = stock + 1 WHERE product_id = $1`.

Common wrong answer. "Wrap it in a transaction." A transaction alone does not help — at Read Committed both transactions are individually valid. The problem is the read-modify-write in application code, not the absence of a transaction.

Tip. Name the two alternatives and when each applies: `SELECT ... FOR UPDATE` when you must read several rows and then decide, and an optimistic version column when a human is editing a form for thirty seconds and you want to tell them someone else changed it.

I12

Q. Which isolation level would you use, and why?

Answer. Read Committed by default — cheap, and correct where writes are relative and reads tolerate a moving target. Repeatable Read for a multi-statement report that must be internally consistent, since Read Committed takes a new snapshot per statement and your regional figures will not sum to your total. Serializable where an invariant spans rows the transaction does not itself write.

Tip. Give the write-skew example — two doctors each going off call because each read that the other was still on — and then the sentence that shows production experience: "Serializable needs application retry logic on SQLSTATE 40001, with exponential backoff and jitter. And where the rule can be a constraint instead, I prefer that: an exclusion constraint is stronger than any isolation level because it cannot be forgotten by the next developer."

I13

Q. Normalize this table. `invoices(invoice_id, customer_name, customer_email, product_name, product_price, quantity)`

Answer. Write the dependencies first — that is the work, and the tables follow:

```
(invoice_id, product_name) → quantity      -- ✓ the whole key
product_name                → product_price -- X partial
invoice_id                  → customer_*    -- X partial, and transitive
                                -- via the customer
```

Two partial dependencies violate 2NF; splitting them out reveals that the customer details depended on the customer, not the invoice — 3NF. Result: `customers`, `products`, `invoices`, `invoice_lines`, with `unit_price` snapshot on the line.

Tip. Explain that last column unprompted: "`unit_price` on the line looks like the redundancy I just removed and isn't — the price charged is a different fact from the price today, and keeping it is required for correctness, not permitted for speed."

I14

Q. How do you prevent double-booking a room?

Answer. An exclusion constraint over the room and a range type:

```
CREATE EXTENSION IF NOT EXISTS btree_gist;  
ALTER TABLE bookings ADD CONSTRAINT bookings_no_overlap  
    EXCLUDE USING gist (room_id WITH =, stay WITH &&  
    WHERE (status <> 'cancelled');
```

Common wrong answer. `UNIQUE (room_id, check_in)`. It only catches bookings starting on the same day — a 3rd–7th and a 5th–9th both pass and the room is double-sold. Naming that specific failure is what demonstrates you have thought it through.

Tip. "Application-level checking cannot be made correct without explicit locking, because two concurrent requests both read 'no conflict' before either writes. The guarantee has to be in the database."

I15

Q. A page is slow. Walk me through diagnosing it.

Answer. "First find out *which* query — `pg_stat_statements` ordered by **total** time, or the application's traces. Not by guessing, and not by mean time, because a 4 ms query called 200,000 times an hour costs far more than a 6-second query called twice. Then `EXPLAIN (ANALYZE, BUFFERS)` with the real parameters, and look for the estimate gap, the nested loop, and rows removed by filter. Then change one input and re-measure."

Tip. The sentence that wins this question: "Often the answer is not a slow query at all but many fast ones — an N+1 — which a trace shows and a slow-query log with a 500 ms threshold hides completely."

THE REMAINING 135 — TWO MINUTES EACH, OUT LOUD

- 16** Predict the row count of an inner join given the multiplicities on each side.
- 17** Why does putting a filter in `ON` versus `WHERE` change a `LEFT JOIN`'s result?
- 18** Write an anti-join three ways and say which you would ship.
- 19** What is a semi-join? Which SQL construct expresses one?
- 20** When is a `FULL OUTER JOIN` genuinely the right answer?
- 21** Join three tables where the middle one may have no rows. Which join types?
- 22** Why is `NATURAL JOIN` discouraged?
- 23** What does `USING (col)` do differently from `ON a.col = b.col`?

- 24** Explain relational division and write one.
- 25** Write "customers who ordered every product in a category".
- 26** Write "employees earning above their department average" two ways.
- 27** Write "top three products per category, including ties".
- 28** Write "the Nth highest salary" as a parameterised query.
- 29** Difference between `rank()`, `dense_rank()` and `row_number()`, with the sequences.
- 30** When is `row_number()` non-deterministic, and how do you fix it?
- 31** What does `ntile(4)` do, and how does it handle uneven division?
- 32** Explain `LAG` and `LEAD`. What is the third argument for?
- 33** Why does month-over-month growth via `lag` break with missing months?
- 34** Difference between `ROWS` and `RANGE` in a frame clause.
- 35** What is `percentile_cont` versus `percentile_disc`? Which for a p95 latency?
- 36** Compute a median in SQL. Two ways.
- 37** What does the `WINDOW` clause do, and why use it?
- 38** Can a window function take an aggregate as its argument? Give an example.
- 39** Write a query sessionizing events with a 30-minute timeout.
- 40** Explain the flag-and-cumulate pattern.
- 41** Write a cohort retention query by signup month.
- 42** Write a funnel query with drop-off at each stage.
- 43** What is `GROUPING SETS`? How does it relate to `ROLLUP` and `CUBE`?
- 44** How do you tell a `ROLLUP` subtotal row from a real `NULL`?
- 45** What does `FILTER` do, and what is its portable equivalent?
- 46** Pivot statuses into columns. Why can SQL not pivot dynamically?
- 47** Unpivot columns into rows.
- 48** What is a `LATERAL` join and when do you need one?
- 49** Why is `ON true` conventional in a lateral join?
- 50** What is `DISTINCT ON`? What must its `ORDER BY` start with?
- 51** When does PostgreSQL materialize a CTE rather than inline it?
- 52** What was the pre-12 CTE optimization fence, and why did it matter?
- 53** Detect a cycle in a recursive CTE.
- 54** Compute the transitive closure of a hierarchy — everyone under a manager.

- 55** Explode a bill of materials. Why do quantities multiply, not add?
- 56** When is a recursive CTE the wrong tool? Name two alternative models.
- 57** Generate every date in a month, two ways. Which would you ship?
- 58** Explain three-valued logic and what `AND` / `OR` do with unknown.
- 59** What does `IS DISTINCT FROM` do that `<>` does not?
- 60** Name the three places `NULL` s are treated as equal.
- 61** Why does `count(*)` differ from `count(column)` on nullable data?
- 62** What is sargability? Give three unsargable predicates and their fixes.
- 63** Why does wrapping a column in a function defeat its index?
- 64** What is an expression index, and give two reasons it helps.
- 65** What is a partial index? Give three real uses.
- 66** What is a covering index? What does `INCLUDE` add?
- 67** Why might an index-only scan still read the heap?
- 68** What is selectivity? Above what fraction does a seq scan win?
- 69** When is indexing a low-cardinality column still worthwhile?
- 70** Which index type for `ILIKE '%term%'` ? For range overlap? For a huge append-only table?
- 71** Name four costs of an index.
- 72** Find unused and redundant indexes. Which must you never drop?
- 73** Why `CREATE INDEX CONCURRENTLY` , and what are its drawbacks?
- 74** Explain the three join algorithms and when the planner picks each.
- 75** What is a bitmap heap scan, and why is it usually a good sign?
- 76** What does `Sort Method: external merge` mean? Two remedies, and which is better?
- 77** Why should `work_mem` not be set globally high?
- 78** What does `random_page_cost = 4.0` assume, and why change it?
- 79** What does `ANALYZE` collect? When does autovacuum run it?
- 80** What are extended statistics for? Give a schema that needs them.
- 81** Why does a `DELETE` not free disk space?
- 82** Why can one idle transaction bloat the entire database?
- 83** Difference between `VACUUM` and `VACUUM FULL` . When would you use neither?
- 84** What is transaction ID wraparound?
- 85** What is the N+1 problem? What is its signature in `pg_stat_statements` ?

- 86** Give the performance argument against `SELECT *`, not the correctness one.
- 87** Why is a filtered `count(*)` expensive? Three alternatives.
- 88** What is a materialized view? What does `CONCURRENTLY` require?
- 89** Why might a summary table beat a materialized view?
- 90** Why is a bigger connection pool not always better?
- 91** What breaks under PgBouncer transaction mode?
- 92** What do read replicas *not* fix?
- 93** Explain read-your-own-writes and three fixes.
- 94** Name the caching ladder from cheapest to most expensive.
- 95** What is a savepoint, and how do ORMs use it?
- 96** Why is an exception block inside a large loop expensive?
- 97** What happens in PostgreSQL after an error inside a transaction? How does MySQL differ?
- 98** Why must you never make an HTTP call inside a transaction?
- 99** Explain the outbox pattern and why the pending row matters.
- 100** What does `FOR UPDATE SKIP LOCKED` make possible?
- 101** Which SQLSTATEs should an application retry, and which must it not?
- 102** Why does a retry loop need jitter?
- 103** What single rule prevents most deadlocks?
- 104** Are readers blocked by writers in PostgreSQL? Explain.
- 105** Name the four isolation levels and the anomaly each one stops.
- 106** Which anomaly cannot occur in PostgreSQL at any level, and why?
- 107** Why is PostgreSQL's Repeatable Read stronger than the standard requires?
- 108** Which isolation level does MySQL default to? Why does that matter?
- 109** Explain write skew and why Repeatable Read does not stop it.
- 110** Define 1NF, 2NF and 3NF in one sentence each.
- 111** Why can 2NF only be violated by a composite key?
- 112** What is a transitive dependency? Spot one in a table shown to you.
- 113** How does BCNF differ from 3NF, and when does the difference appear?
- 114** What is the 4NF test for a junction table?
- 115** Name the three anomalies and give an example of each.
- 116** When would you denormalize? Name the four conditions.

- 117** Write the reconciliation query for a stored aggregate.
- 118** What is a partial unique index? Give three real uses.
- 119** What problem do deferrable constraints solve? Two examples.
- 120** Difference between `NO ACTION` and `RESTRICT`.
- 121** Why is `ON DELETE CASCADE` risky?
- 122** Name three things a `CHECK` cannot do.
- 123** Why does `CHECK (created_at <= now())` fail to be created?
- 124** List three business rules no constraint can express, and how you enforce each.
- 125** When would you use a trigger, and when would you refuse?
- 126** How does one audit function serve every table?
- 127** In what order do several triggers on one event fire?
- 128** How does a trigger learn the *application* user rather than the database role?
- 129** Name the three volatility classes and what each permits.
- 130** What can a procedure do that a function cannot?
- 131** Difference between `->` and `->>`. Which do you compare with?
- 132** When would you use JSONB instead of columns? Give the three tests.
- 133** Why does a GIN index sometimes not fix a slow JSONB query?
- 134** Name the three things you cannot parameterise, and what to do instead.
- 135** What is second-order injection? Why does validation not stop it?
- 136** Why is escaping quotes not a defence against injection?
- 137** Why does `ALTER DEFAULT PRIVILEGES` exist?
- 138** Name the two traps that make row-level security decorative.
- 139** Why `SET LOCAL` rather than `SET` for a tenant id?
- 140** Define RPO and RTO. Give a backup method for each extreme.
- 141** What is the single rule about backups?
- 142** What is point-in-time recovery and what does it require?
- 143** Why must a unique key include the partition column?
- 144** When does partitioning make a query *slower*?
- 145** What happens if no partition covers an incoming row?
- 146** What is the grain of a fact table? Why declare it first?
- 147** Star or snowflake? Defend your default.

- 148** Explain SCD Type 2 and why the fact joins on the surrogate key.
- 149** Why does a warehouse have a date dimension when dates are a type?
- 150** How do you add a column to a 200-million-row table without downtime?

WHERE THE ANSWERS ARE

Every question in these lists is answered somewhere in Volumes I–VI, and the worked boxes above model the *shape* of a strong answer. If a question stops you, find it in the volume rather than looking for a one-line answer — the mechanism is what the follow-up question tests.

33.4 Advanced questions

● **ADVANCED** 150 questions. This band separates people who have operated a database from people who have used one. Almost every answer here should include an operational consequence.

A1

Q. A query was fast last month and is slow now. Nothing changed in the code. Why?

Answer. The plan changed, because the planner's inputs changed. Five candidates, in the order I'd check them: **(1)** statistics drifted, so an estimate is now wrong and the join method flipped — `last_analyze` and a manual `ANALYZE` settle it in a minute; **(2)** the data crossed a selectivity threshold, so a value that used to match 200 rows now matches 200,000 and a nested loop became catastrophic; **(3)** the working set outgrew cache, so the same plan now does real I/O — visible as a rising `read` versus `hit` ratio in `BUFFERS` ; **(4)** bloat, from a long-running transaction blocking vacuum; **(5)** a prepared statement switched to a generic plan.

Tip. The framing that scores: "Queries don't get slower — plans do, or the data does. So I compare estimated against actual rows first, because that one number tells me whether it's a statistics problem or a genuine volume problem, and those have completely different fixes."

A2

Q. Your database is at capacity. Walk me through your options.

Answer. Ask first whether the load is read-heavy or write-heavy — replicas do nothing for writes, and proposing them without asking skips the diagnosis. Then up the ladder: find what the load actually is from `pg_stat_statements` ordered by total time and check for N+1 patterns; then indexes and query rewrites, which reduce work rather than redistributing it; then materialized summaries for the reporting load; then a read replica if reads dominate; then sharding, last, because it costs joins, cross-shard transactions and global uniqueness.

For writes specifically: reduce the writes (batch inserts, drop unused indexes — each index is a write multiplier), make each write cheaper (check `n_tup_hot_upd` — an index on a hot column destroys HOT updates), move non-essential writes off the critical path, then vertical scaling, then sharding.

Tip. "The most common finding is that we're issuing ten thousand cheap queries rather than running one expensive one — so I look at call counts before I look at durations."

Q. Design a job queue in PostgreSQL. Why would you not use a message broker?

```
CREATE TABLE jobs (
  job_id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  payload     jsonb NOT NULL,
  status      text NOT NULL DEFAULT 'queued',
  locked_at   timestampz,
  attempts    smallint NOT NULL DEFAULT 0,
  run_after   timestampz NOT NULL DEFAULT now(),
  created_at  timestampz NOT NULL DEFAULT now()
);
CREATE INDEX idx_jobs_ready ON jobs (run_after)
  WHERE status = 'queued';

-- Each worker:
BEGIN;
SELECT job_id, payload FROM jobs
WHERE status = 'queued' AND run_after <= now()
ORDER BY run_after LIMIT 1
FOR    UPDATE SKIP LOCKED;
UPDATE jobs SET status='running', locked_at=now(), attempts=attempts+1
WHERE job_id = $1;
COMMIT;
```

Expected thinking. Three details make it production-grade, and naming all three is the answer. `SKIP LOCKED` stops workers queueing behind each other on the oldest row. The **partial index** keeps the lookup fast when the table holds millions of completed jobs. And `locked_at` plus a reaper resets jobs whose worker died — without it, jobs mysteriously stop processing and nobody knows why.

Why not a broker. Because the job and the business data can then be written in *one transaction*. With a separate broker you have two systems and no atomicity: the row commits and the message is lost, or the message is delivered and the row rolled back. That is the outbox problem, and a database queue avoids it entirely. Use a broker when you need fan-out to many consumers, cross-service delivery, or throughput beyond what one Postgres can absorb.

A 4

Q. How would you isolate tenants in a multi-tenant SaaS database?

Answer. Compare the three honestly. **Shared schema with RLS:** `tenant_id` on every table, policies keyed on a `SET LOCAL` session variable, both `USING` and `WITH CHECK`, application connecting as a non-owner with `FORCE ROW LEVEL SECURITY`. Cheapest to operate at scale — one pool, one migration — and the isolation is enforced by the database, so a forgotten `WHERE` cannot leak.

Schema-per-tenant: stronger isolation and trivial per-tenant restore; migrations run N times, which is fine at fifty tenants and unmanageable at five thousand. **Database-per-tenant:** strongest, required by some compliance regimes, most expensive in connections and backups.

Recommendation. "Shared schema with RLS, with the option to promote a large or regulated tenant to its own database — because that hybrid is what almost every mature SaaS ends up running, and designing for it early is cheaper than migrating to it later."

Tip. Name the two traps unprompted: the owner bypass, and `SET` versus `SET LOCAL` on a pooled connection. Both are the difference between RLS that works and RLS that is decoration.

A 5

Q. Why is a stored account balance a design error? What do you store instead?

Answer. Because a stored balance is only as correct as the last code path that wrote it, and there will be paths you did not anticipate — a migration, a support script, a partially-applied batch. Store an append-only double-entry ledger: every movement is two rows summing to zero, and the balance is `sum(amount)`. That is correct by construction.

The invariant. `SELECT transfer_id, sum(amount) FROM entries GROUP BY transfer_id HAVING sum(amount) <> 0` must return zero rows, forever. One query proves the whole ledger is internally consistent.

Tip. "I'd still keep a materialised balance as a cache, because summing millions of entries per page load is slow — but it's a cache maintained by trigger in the same transaction, with a nightly reconciliation that alerts on any difference. It is safe only because something is watching it."

A 6

Q. An application intermittently fails with "could not serialize access". What is happening?

Answer. Two transactions under Repeatable Read or Serializable touched overlapping data in a way the engine could not order, so it aborted one to preserve correctness. This is the mechanism working, not a bug. The fix is a retry loop around the **whole** transaction — the snapshot is gone, so the earlier reads must be redone — with exponential backoff, jitter so competing transactions do not collide again in lockstep, and a capped attempt count.

Tip. "I'd also ask why contention is that high. Often it's a hot row, an over-broad transaction, or a missing index — Serializable takes predicate locks, and a sequential scan takes a far broader predicate lock than an index scan, so a missing index directly manufactures false conflicts. Retries treat the symptom."

A 7

Q. Add a NOT NULL column to a 200-million-row table with no downtime.

```
-- 1. Add nullable: instant, catalog-only.
ALTER TABLE events ADD COLUMN source text;
-- 2. Backfill in batches. Repeat until 0 rows affected.
UPDATE events SET source = 'legacy'
WHERE event_id IN (SELECT event_id FROM events
                   WHERE source IS NULL ORDER BY event_id LIMIT 10000);
-- 3. Default for new rows: instant on PG 11+.
ALTER TABLE events ALTER COLUMN source SET DEFAULT 'unknown';
-- 4. NOT VALID check: enforced from now on, brief lock, no scan.
ALTER TABLE events ADD CONSTRAINT events_source_nn
    CHECK (source IS NOT NULL) NOT VALID;
-- 5. Validate: full scan that does NOT block reads or writes.
ALTER TABLE events VALIDATE CONSTRAINT events_source_nn;
-- 6. PG 12+: now cheap, because the validated check proves it.
ALTER TABLE events ALTER COLUMN source SET NOT NULL;
ALTER TABLE events DROP CONSTRAINT events_source_nn;
```

Tip. Add the deploy ordering, which is what separates this from a memorised recipe: "I deploy code that tolerates the column being absent *before* the migration, and code that requires it after — expand and contract, so no deploy order can break." **Common wrong answer:** a single `ADD COLUMN ... NOT NULL DEFAULT 'x'`, which happens to be fast on PG 11+ with a constant default and rewrites the whole table under an exclusive lock on older versions or other engines.

A 8

Q. A table is 400 GB, 60% bloat, and you cannot take downtime.

Answer. `VACUUM FULL` is out — an `ACCESS EXCLUSIVE` lock for hours. In order: **(1)** find out *why* it bloated, or you will do this again — a long-running transaction, an autovacuum starved by a conservative cost delay, or an update pattern rewriting whole rows; **(2)** rebuild indexes with `REINDEX CONCURRENTLY`, since index bloat is frequently most of the total and this is safe online; **(3)** `pg_repack` for the heap, which rebuilds into a new file using triggers to capture concurrent changes and takes only a brief lock at the swap; **(4)** if unavailable and the table is time-based, partition going forward and let the bloated data age out; **(5)** fix the cause — per-table `autovacuum_vacuum_scale_factor`, `idle_in_transaction_session_timeout`, monitoring on `n_dead_tup`.

Tip. "Arguably step 5 comes first — there is no point repacking a table that will bloat again next week."

A 9

Q. Design retention for a 5 TB events table keeping 13 months.

Answer. Range-partition by month on the event timestamp, with the partition key in the primary key. Retention becomes `ALTER TABLE ... DETACH PARTITION CONCURRENTLY` then `DROP TABLE` — instant, no dead rows, no vacuum afterwards, versus hours of `DELETE` plus days of vacuum. BRIN on the timestamp inside each partition, because rows arrive in order so block ranges are tight and the index is minuscule. Automate partition creation with `pg_partman` or a scheduled job, several months ahead.

What I alert on. "That the newest partition's upper bound is less than a month away. A missing future partition means inserts fail or land in the default partition — which then blocks attaching a real one for that range. It is the most common partitioning incident and it happens at midnight on the first of the month."

A10

Q. Enforce "at most 3 active subscriptions per customer" correctly under concurrency, without Serializable.

Answer. The naive count-then-insert is write skew — two concurrent requests both count 2 and both insert. Two correct options. **(a) Serialise on the parent:** `SELECT 1 FROM customers WHERE customer_id = $1 FOR UPDATE` at the start, so every subscription change for that customer takes the same lock. Simple, and contention is scoped to one customer. **(b) Turn it into uniqueness:** a `slot` `smallint CHECK (slot BETWEEN 1 AND 3)` column with `CREATE UNIQUE INDEX ON subscriptions (customer_id, slot) WHERE cancelled_at IS NULL`. A fourth concurrent subscription must claim a slot, all three are taken, and the insert fails on the index.

Tip. "(b) is the better engineering answer for the same reason an exclusion constraint beats application checking: turn the rule into something the database cannot allow, rather than something the code must remember to check."

A11

Q. A payment webhook may be delivered twice. Design so it cannot double-charge.

```
ALTER TABLE payments ADD CONSTRAINT payments_ref_unique UNIQUE (gateway_ref);

-- Idempotent by construction, in ONE transaction with the status update
INSERT INTO payments (order_id, gateway_ref, amount_minor, status)
VALUES ($1, $2, $3, 'settled')
ON CONFLICT (gateway_ref) DO NOTHING
RETURNING payment_id;
-- no row returned → duplicate delivery → return 200 and do nothing

UPDATE orders SET status = 'paid'
WHERE order_id = $1 AND status = 'pending'; -- guard makes replay harmless
```

Expected thinking. The guarantee is in a *constraint*, not in a "have we seen this before?" lookup — a lookup loses the race between two simultaneous deliveries of the same webhook. And the `AND status = 'pending'` makes the update idempotent, so a replay changes nothing.

Tip. "Returning 200 on the duplicate matters — it's what makes the provider stop retrying. A 500 turns one duplicate into an infinite loop."

A12

Q. A plan shows a nested loop with outer rows=3 and actual rows=180000 . Diagnose.

Answer. Four causes, each with a test. **(1) Stale statistics** — check `last_analyze` , run `ANALYZE` , re-explain. Cheapest and most common. **(2) Correlated predicates** — the planner multiplies selectivities assuming independence, so `city = 'Mumbai' AND state = 'Maharashtra'` is estimated at $1/1000 \times 1/30$ when the second is implied by the first. Test each predicate alone versus together; fix with `CREATE STATISTICS (dependencies)` . **(3) An opaque expression** — a function call or a join on an expression gives no histogram, so the planner uses a fixed default. Tell-tale: the estimate is a round fraction like 0.5%. Fix with an expression index, which also creates statistics. **(4) A generic plan** for a prepared statement — substitute the literal and re-explain.

Tip. Cause (2) is the one most people have never heard of and it is extremely common in real schemas. Naming extended statistics is a strong signal.

A13

Q. Writes on a 40-million-row table degraded after a release. No queries changed.

Answer. Three hypotheses. **(1) New indexes shipped** — every index is a write multiplier; check `pg_indexes` against the previous migration and look for recent indexes with zero scans. **(2) Updates stopped being HOT** — a PostgreSQL update can skip index maintenance entirely if no indexed column changed and the new version fits on the same page. Adding an index on a frequently-updated column destroys that for *every* update, including ones touching unrelated columns. Test: `n_tup_hot_upd` against `n_tup_upd` . **(3) Bloat or a blocked vacuum** — a long-running transaction produces exactly this symptom with no code change at all.

Tip. Hypothesis (2) is the senior answer. Most engineers do not know that adding one index can slow down updates to columns it does not contain.

A14

Q. A dashboard runs 40 aggregate queries and takes 12 seconds.

Answer. Diagnose first: 40 queries at 300 ms is a different problem from 2 at 6 seconds, and check whether they run sequentially — issuing them concurrently often costs nothing and is the single biggest win. Then: **(a) Pre-aggregate** into summary tables refreshed every few minutes; the dashboard becomes 40 primary-key lookups and renders in under 100 ms. Trade-off: bounded staleness and a job to operate. This is the right answer for almost every dashboard. **(b) Collapse the queries** — many tiles are the same scan with different filters, so `FILTER` and `GROUPING SETS` can make one pass serve them all; costs coupling between tiles. **(c) Route to a replica** — does not make it faster, only stops it hurting the transactional workload, which may be the actual requirement.

Tip. "I'd combine (a) and (c): pre-aggregate on a replica, serve the summaries from wherever the dashboard lives."

A15

Q. A bulk import into a table with three row-level triggers takes four hours.

Answer. Three options, increasingly good. **(a) Disable the triggers** for the load, then reconcile what they would have maintained — fast and blunt; wrap it in a transaction so a mid-load failure does not leave them disabled. **(b) Replace row-level with statement-level triggers using transition tables** — `AFTER INSERT ... REFERENCING NEW TABLE AS new_rows FOR EACH STATEMENT`, then one set-based operation instead of two million function calls. This keeps the guarantee, and audit logging and aggregate maintenance are both expressible this way. **(c) Stage, transform, then insert**, with the triggers doing one bulk pass at the end.

Tip. "(b) is the durable fix and (a) is the one-off. And I'd drop and recreate the fact-table indexes around the load too — that is usually the other half of the four hours."

THE REMAINING 155 — ANSWER EACH IN THREE MINUTES

- 16** Design a schema for a system where a seat may be sold once per showing, correct under concurrency.
- 17** Design railway seat booking where a seat may be sold twice on non-overlapping station segments.
- 18** Design an inventory system where stock levels are reconstructible at any past date.
- 19** Design an audit system capturing the *application* user, not the database role.
- 20** Design per-tenant custom fields. Compare three approaches and recommend one.
- 21** Design a schema supporting "revenue by segment as at the time of sale, and as it is today".
- 22** Design a soft-delete strategy. What breaks, and what must you change?
- 23** Design a schema where an employee's manager must be in the same department, declaratively.
- 24** Design an approval workflow where thresholds are data, not code.
- 25** Design a versioned document store where any past version is retrievable.
- 26** Design rate limiting in the database. What are its failure modes?
- 27** Design an idempotent bulk-import pipeline. What makes a re-run safe?
- 28** Design a feed for a social product. Fan-out on write or on read? Defend it.
- 29** Design a nested comment thread that does not recurse on every read.
- 30** Design a schema for prices that vary by customer, quantity and date.
- 31** Explain MVCC. What does it buy, and what does it cost?
- 32** What is the write-ahead log, and which ACID properties depend on it?
- 33** What is a HOT update, and what defeats it?
- 34** What is the visibility map, and why does it matter for index-only scans?
- 35** What is TOAST, and when does it hurt you?

- 36** What is `fillfactor` and when would you lower it?
- 37** Explain `shared_buffers` versus `effective_cache_size`.
- 38** What does `synchronous_commit = off` trade away?
- 39** Explain predicate locks and how Serializable uses them.
- 40** Why does a missing index increase serialization failures?
- 41** Explain lock modes and which pairs conflict.
- 42** Why does a DDL statement behind a long query block everything?
- 43** What is `lock_timeout` and why set it in migrations?
- 44** Diagnose a deadlock from a log entry. What do you change?
- 45** How do you find what is blocking what, right now?
- 46** Why can advisory locks be the right tool? Give a use.
- 47** Explain the difference between physical and logical replication.
- 48** How would you do a major-version upgrade with minutes of downtime?
- 49** What is split-brain, and how is it prevented?
- 50** What is `hot_standby_feedback` and what does it cost?
- 51** Why can a long query on a replica be cancelled?
- 52** Design a zero-downtime failover plan. What does the application need?
- 53** How do you test a restore? What do you measure?
- 54** You must recover to 14:32 yesterday. What do you need in place already?
- 55** What are the six things worth alerting on, and why no more?
- 56** Interpret a cache hit ratio of 94% on an OLTP system.
- 57** Rewrite an `OFFSET`-paginated API to keyset, including the cursor format.
- 58** Why is a filtered `count(*)` sometimes slower than fetching the page?
- 59** Design search over a million products. Full-text, trigram, or both?
- 60** When is `tsvector` the wrong tool for search?
- 61** Explain a GIN index's build and update cost. What is `fastupdate`?
- 62** Why is BRIN useless on a table with random insert order?
- 63** Design indexing for a table with six very different query shapes.
- 64** When would you accept a full table scan as the correct plan?
- 65** Explain partitionwise joins and what they require.
- 66** Why does execution-time pruning not help a cached generic plan?

- 67** How do you attach an existing table as a partition without a long lock?
- 68** Explain sub-partitioning and when it is justified.
- 69** Your nightly warehouse load takes 9 hours and must finish in 3.
- 70** Design an incremental load with a watermark. What breaks with late-arriving data?
- 71** Explain SCD Types 0, 1, 2, 3, 4 and 6 and when each applies.
- 72** Why is mixing grains in one fact table irreparable?
- 73** What is a degenerate dimension? Give one.
- 74** Design a factless fact table and say what it answers.
- 75** How do you handle a late-arriving dimension row during a fact load?
- 76** Explain semi-additive measures with an example.
- 77** Design the ETL idempotency guarantee for a fact table.
- 78** Why does the warehouse abandon 3NF, and why is that safe there?
- 79** Write MRR movement split into new, expansion, contraction and churn.
- 80** Write a cohort retention matrix by signup month and month offset.
- 81** Write a query attributing revenue across a multi-touch funnel.
- 82** Write a query detecting anomalous daily volume per account.
- 83** Write a query finding customers whose spending declined three months running.
- 84** Write a query computing p50, p95 and p99 latency by endpoint and hour.
- 85** Write a query finding overlapping date ranges in a table that should have none.
- 86** Write a query merging adjacent and overlapping ranges into consolidated spans.
- 87** Write a query computing per-day occupancy from admission and discharge dates.
- 88** Write a query pivoting twelve months into columns without dynamic SQL.
- 89** Write a query finding the first and last event per session in one pass.
- 90** Write a query producing a nested JSON API response in one statement.
- 91** Write a query deduplicating a table in place, keeping the newest of each group.
- 92** Write a batched delete that will not hold locks for hours.
- 93** Write a query backfilling a column in batches, resumably.
- 94** Write a query comparing two tables and reporting rows that differ.
- 95** Write a query validating a foreign-key relationship before adding the constraint.
- 96** Write a query finding every table without a primary key.
- 97** Write a query finding every foreign key without a supporting index.

- 98** Write a query listing every table with RLS disabled.
- 99** Write a query estimating the size and bloat of every table.
- 100** Write a query summarising the top queries by total time with their mean and calls.
- 101** Explain volatility classes and what declaring one wrongly does.
- 102** When does a function need `PARALLEL SAFE`, and what happens without it?
- 103** Write safe dynamic SQL. Why `%I` and never `%s`?
- 104** When would you put business logic in the database, and when not?
- 105** Design a stored procedure that batches and commits inside a loop.
- 106** Why does a trigger on a child table updating one parent row create a hot spot?
- 107** How do you guard against trigger recursion?
- 108** Why does `TRUNCATE` silently stale a trigger-maintained aggregate?
- 109** Design an `INSTEAD OF` trigger making a complex view writable.
- 110** What does `security_invoker` change about a view?
- 111** Why revoke `CREATE ON SCHEMA public FROM PUBLIC`?
- 112** Explain the `search_path` function-shadowing attack.
- 113** Design least privilege for a web app, a report user and a migration runner.
- 114** Why is least privilege discovered rather than designed? How do you discover it?
- 115** Explain column-level grants and their ordering gotcha.
- 116** Why is a reversibly-encrypted password a defect?
- 117** Why is `sslmode=require` insufficient?
- 118** What does column-level encryption cost you in query capability?
- 119** Design data masking for a production restore into staging.
- 120** How do you honour a deletion request when backups exist?
- 121** Explain why generic plans exist and when to force custom ones.
- 122** Why does the same query text get different plans for different parameters?
- 123** What is a lossy bitmap, and what does it signal?
- 124** Explain parallel query. What disables it?
- 125** Why is `SET enable_nestloop = off` a diagnostic and not a fix?
- 126** How would you benchmark two query variants credibly?
- 127** Why report the second run of a benchmark, not the first?
- 128** How do you decide whether an index is worth its write cost? Show the numbers.

- 129** Your team wants to add an ORM to a hand-written SQL codebase. Advise them.
- 130** Your team wants to move all logic into stored procedures. Advise them.
- 131** A colleague says "we use NoSQL so normalization doesn't apply". Respond.
- 132** A colleague proposes a single `jsonb` column per table. Respond.
- 133** A colleague wants to shard at 100,000 users. Respond.
- 134** You inherit a schema with no foreign keys. What do you do first?
- 135** Describe the worst database incident you have seen and what changed afterwards.
- 136** How does logical replication handle a conflict, and what breaks a subscription?
- 137** What does `pg_stat_statements` normalise away, and when does that hide the culprit?
- 138** An index-only scan regressed to an index scan after a week of churn. Why?
- 139** When is a foreign data wrapper the right answer, and what does the planner lose?
- 140** Size a connection pool from core count and query profile. Show the arithmetic.
- 141** Compare `COPY`, multi-row `INSERT` and row-at-a-time throughput. Why the gap?
- 142** Quantify statement-level versus row-level trigger cost on a two-million-row load.
- 143** How would you rehearse a major-version upgrade before doing it for real?
- 144** Forecast when you will run out of disk and IOPS, from `pg_stat_*` alone.
- 145** Write the schema-change review checklist you would require before any migration merges.
- 146** What does `maintenance_work_mem` affect, and when would you raise it temporarily?
- 147** Explain autovacuum cost-based delay and the symptom of it being too conservative.
- 148** How do you decide between a partial index and a partitioned table for hot-row access?
- 149** Design the observability you would want before a database migration to new hardware.
- 150** You have one week to make an unfamiliar production database safe. What do you do, in order?

33.5 Scenario-based questions

● **EXPERT** 100 situations. There is no single right answer — you are scored on the order in which you investigate. Say what you would look at **first**, and why.

S1 — THE ARCHETYPE

Scenario. "Checkout is timing out. It started twenty minutes ago. The deploy was three hours ago. What do you do?"

The order that scores. (1) Is it the database at all? Check application latency against database latency — if the database is idle, stop looking here. (2) `pg_stat_activity` : how many active sessions, what are they waiting on, is anything blocking. A blocked queue behind one lock looks exactly like this. (3) Is there a long-running transaction or a DDL statement waiting? A migration that started twenty minutes ago and is queued behind a slow query will block every subsequent query on that table. (4) Connection count against `max_connections` — pool exhaustion presents as timeouts with an idle database. (5) Only then look at plans.

What they are listening for. That you do not start with `EXPLAIN` . The timing — twenty minutes ago, not three hours ago — argues against the deploy and for something that *accumulated*: a lock queue, a leaked transaction, connection exhaustion, or a data threshold crossed. Say that reasoning out loud.

Tip. "And I'd note what to fix afterwards regardless: `statement_timeout` and `lock_timeout` per role, so one slow statement cannot take out checkout again."

S2

Scenario. "Finance says last month's revenue report is wrong. The query hasn't changed in a year."

The order. (1) Wrong in which direction and by how much? Too high suggests fan-out or double-counting; too low suggests an `INNER JOIN` dropping rows or a filter excluding a new status value. (2) Has the *data* changed shape — a new order status, a new payment method, a nullable column that started receiving `NULL` s? A year-old query encodes last year's assumptions. (3) Check the grain: did anyone add a table to the join? (4) Run the query with the aggregate removed and count rows against the expected count. (5) Check for a `NOT IN` against a column that now contains `NULL` s — the report would have gone silently empty or short.

Tip. "The most common cause of a correct query becoming wrong is a new value in a filtered column — a status nobody told the report about. So I'd write the reconciliation query that compares the report's total against an independent path, and put it in monitoring."

S3

Scenario. "Two customers were charged for the same order. It has happened four times this month."

The order. **(1)** Get the payment rows for one affected order and look at the timestamps — milliseconds apart means concurrency, minutes apart means a retry. **(2)** Is there a unique constraint on the gateway reference? If not, that is the bug and the fix. **(3)** If there is, is the payment insert in the same transaction as the status update? A commit between them leaves a window. **(4)** Is the charge decision a read-modify-write — "if status is pending, charge" — evaluated in application code? That is lost update, and two concurrent requests both read pending.

The fix. A `UNIQUE` on the idempotency key, `ON CONFLICT DO NOTHING`, and the status update guarded by `AND status = 'pending'` in the same transaction. Then backfill: find every duplicate and refund.

Tip. "Four times this month and only now reported means there is no alert. I'd add the invariant query — orders with more than one settled payment — to monitoring before I fix the cause, so I know whether the fix worked."

S4

Scenario. "Disk is 92% full. The database is 2 TB and growing 40 GB a day. You have a few hours."

The order. **(1)** What is actually consuming it — tables, indexes, WAL, or logs? A WAL directory filling up because a replication slot is inactive is a different emergency, and the fix is dropping the slot. **(2)** Check bloat: `n_dead_tup` and whether autovacuum is blocked by a long transaction. A stuck vacuum can account for most of the growth, and killing the transaction reclaims space over the next hour. **(3)** Fastest safe wins: drop unused indexes (identified from `pg_stat_user_indexes`, never uniqueness-enforcing ones), truncate log tables, drop old partitions if the table is partitioned. **(4)** Buy time — add disk if the platform allows, because it is the only action with no correctness risk.

Tip. "What I would *not* do is `VACUUM FULL` — it needs as much free space as the table it rewrites, so on a 92%-full disk it fails, and it holds an exclusive lock while doing so. That is the trap in this question."

S5

Scenario. "A migration has been running for 40 minutes and the site is down."

The order. (1) Identify the statement and its lock from `pg_stat_activity` and `pg_blocking_pids`. (2) Decide: can it be cancelled safely? A `CREATE INDEX` (non-concurrent) can be cancelled with no data risk. A table rewrite mid-flight can also be cancelled — it rolls back. (3) Cancel it. The site comes back. (4) Then work out why it locked: an `ALTER TABLE` needing `ACCESS EXCLUSIVE` queued behind a long-running read, and once *it* is queued, every subsequent query queues behind it. That is why the whole site fell over for a change that should have been instant.

The prevention, which is the real answer. "Set `lock_timeout` in every migration — a few seconds — so the DDL gives up instead of queueing and taking the site with it. Then retry. And use `CONCURRENTLY` for indexes, and `NOT VALID` plus `VALIDATE` for constraints."

S6

Scenario. "A user reports seeing another customer's data."

The order. This is an incident, not a bug — treat it as one. (1) Establish scope before cause: which endpoint, which two accounts, how many others could be affected. (2) Is there a tenant filter at all, and is it in the application or the database? An application-level filter means every query is a potential leak and you must audit all of them. (3) Check RLS: is it enabled on the table, does the policy have both `USING` and `WITH CHECK`, and — the usual culprit — **does the application connect as the table owner**, which bypasses policies unless forced. (4) Check whether the tenant context is set with `SET` rather than `SET LOCAL` on a pooled connection, which leaks one request's tenant into another's.

Tip. "Cause (3) and (4) are the two that make RLS look present and be decorative, and they are the first things I check because they are silent."

S7

Scenario. "The team wants to shard. We have 200 GB and 50,000 users."

Answer. Push back with numbers, not opinions. 200 GB fits in memory on a commodity server; 50,000 users is not a scale problem. Ask what symptom is driving it — if the answer is "we're worried about the future", that is not a requirement. Then walk the ladder they have not tried:

`pg_stat_statements` by total time, N+1 patterns, missing indexes, an unpartitioned events table, vertical scaling, a read replica.

What sharding costs, said plainly. "No cross-shard joins, no cross-shard transactions, no global uniqueness, no simple `count(*)`, and every future migration runs N times. Those are permanent constraints on the product, not a one-time engineering cost."

Tip. "If they genuinely expect a hundredfold growth, the useful preparation is *not* sharding now — it is making sure every table has a tenant or entity key that could become a shard key later, and avoiding cross-tenant queries in the product design. That keeps the door open at almost no cost."

S8

Scenario. "Replication lag has been climbing for an hour and reports are showing stale data."

The order. **(1)** Is the standby receiving or applying slowly? `sent_lsn` versus `replay_lsn` distinguishes network from apply. **(2)** Apply lag is usually one of: a long-running query on the standby conflicting with replay, a single enormous transaction on the primary being replayed serially, or the standby being I/O-starved. **(3)** Network lag points at bandwidth or a saturated primary. **(4)** Check whether a bulk operation started an hour ago on the primary — a large `UPDATE` generates WAL the standby must apply single-threaded.

Tip. "And I'd fix the reporting symptom separately from the cause: route the affected reports to the primary temporarily, or surface the lag in the UI. Users tolerating known staleness is much better than users doubting the numbers."

THE REMAINING 92 SCENARIOS — STATE YOUR INVESTIGATION ORDER

- 9** An endpoint is fast for most users and slow for your three largest customers.
- 10** A report ran in 2 seconds in staging and 4 minutes in production.
- 11** A nightly job that took 20 minutes now takes 6 hours and nothing changed.
- 12** Users occasionally see a 500 that disappears on refresh.
- 13** The application logs "prepared statement does not exist" intermittently.
- 14** Connection count is at maximum but the database CPU is 4%.
- 15** An inventory count is negative in the database despite a `CHECK` constraint.
- 16** A counter cache disagrees with the table it counts, by different amounts each day.

- 17** A customer says their order vanished. It is not in the table.
- 18** Two rows exist that a unique constraint should have prevented.
- 19** A foreign key references a row that does not exist.
- 20** An audit log has a gap covering exactly one deploy window.
- 21** A soft-deleted user's email cannot be reused at signup.
- 22** A search feature times out for one-character queries.
- 23** Page 1 of a list is fast; page 400 takes eight seconds.
- 24** An export of 2 million rows exhausts the application's memory.
- 25** A dashboard shows different totals depending on which tile loads first.
- 26** A month-over-month growth chart shows a 400% spike that is not real.
- 27** Revenue by region does not sum to total revenue.
- 28** A cohort retention number improved suddenly with no product change.
- 29** A "customers with no orders" report returns zero rows, which is implausible.
- 30** An average order value dropped 30% overnight.
- 31** A daily active users metric counts some users twice.
- 32** A p95 latency metric is lower than the p50.
- 33** A timezone-sensitive report is off by one day for some users.
- 34** A date-range filter excludes rows on the last day of the range.
- 35** Sorting by name puts lowercase entries after all uppercase ones.
- 36** A currency total is off by a few paise across a million rows.
- 37** A percentage column shows 0 where it should show 33.
- 38** An import loaded 40,000 of 40,000 rows but three columns are all `NULL`.
- 39** A CSV import fails on row 12,000 with an encoding error.
- 40** An import ran twice and now every row is duplicated.
- 41** A backfill has been running for six hours and is holding one transaction open.
- 42** Autovacuum appears to run normally but a table grows without limit.
- 43** A table is 8 GB and contains 40,000 rows.
- 44** An index is 3× the size of its table.
- 45** Transaction age has been climbing for three days.
- 46** Writes are refused with "database is not accepting commands".
- 47** A replica has fallen so far behind that WAL is being retained on the primary.

- 48** A failover happened and the application kept writing to the old primary.
- 49** After a failover, the last 90 seconds of orders are missing.
- 50** A restore succeeded but every user's login fails.
- 51** A restore test has been passing for four months; the real restore fails.
- 52** Someone dropped a table in production twenty minutes ago.
- 53** An `UPDATE` without a `WHERE` clause was committed.
- 54** A support engineer needs to fix one row and has no read-write access.
- 55** An ex-employee's database credentials are still in a config file.
- 56** A penetration test reports a blind time-based injection in a report filter.
- 57** A sort-column parameter is interpolated into SQL and cannot be parameterised.
- 58** A stored procedure concatenates its arguments into dynamic SQL.
- 59** A user-supplied display name broke a nightly report six months after signup.
- 60** The application connects as a superuser and an audit has flagged it.
- 61** Production data was restored into a developer's laptop unmasked.
- 62** A GDPR deletion request arrives for a customer with two years of orders.
- 63** An engineer wants read access to production to debug. What do you offer?
- 64** An ORM upgrade changed the SQL for your hottest endpoint.
- 65** An auto-generated migration proposes rewriting a 200-million-row table.
- 66** A rename migration broke the previous release still running on half the fleet.
- 67** A migration succeeded in CI and timed out in production.
- 68** Two migrations were merged and applied in the wrong order.
- 69** A column was dropped and one background job still selects it.
- 70** A new index made writes 40% slower and reads no faster.
- 71** A query uses an index in staging and a sequential scan in production.
- 72** Adding a `LIMIT` made a query dramatically slower.
- 73** Removing a `WHERE` clause made a query faster.
- 74** A query is fast the first time and slow the second.
- 75** A query plan changed after a routine `ANALYZE`, for the worse.
- 76** A join on two indexed columns is not using either index.
- 77** A GIN index was added and the JSONB query is still slow.
- 78** A partitioned table's query scans every partition.

- 79** Inserts started failing at midnight on the first of the month.
- 80** A partition cannot be attached because of the default partition.
- 81** A materialized view refresh blocks the dashboard for four minutes.
- 82** A trigger-maintained total went stale after a data-repair script.
- 83** A trigger fires twice for one logical change.
- 84** A bulk load that took 20 seconds now takes 20 minutes after a feature shipped.
- 85** Deadlocks appear only during the nightly batch window.
- 86** A job queue processes the same job in two workers simultaneously.
- 87** Queued jobs stopped processing and nothing is erroring.
- 88** A webhook handler returns 500 on duplicates and the provider retries forever.
- 89** An external API call inside a transaction times out under load.
- 90** A saved record shows its old value immediately after saving.
- 91** Two users editing the same form silently overwrite each other.
- 92** A booking system double-sold a hotel room during a traffic spike.
- 93** A seat map shows a seat as free that is actually sold.
- 94** A voucher with a single-use limit was redeemed twice.
- 95** A "last one in stock" was sold to two customers.
- 96** An on-call rota ended up with nobody on call.
- 97** An employee's salary history has two overlapping validity spans.
- 98** A recursive org-chart query returns fewer employees than the table holds.
- 99** A hierarchy query never terminates.
- 100** You have inherited an undocumented 300-table schema and one week. Plan it.

HOW THESE ARE SCORED

Nobody expects a diagnosis from a one-line scenario. They are watching whether you **narrow before you dig**: is it the database at all, what changed and when, what does the timing rule out, and what is the cheapest observation that splits the possibilities in half. Say that reasoning aloud and a wrong guess still scores well; a lucky right guess with no reasoning does not.

33.6 Query-writing questions

200 questions against the practice schema of Volume I §2.9 — `employees`, `departments`, `customers`, `orders`, `products`, `order_items`. Write each one out fully. The twelve worked below are the shapes that recur; everything else is a variation on one of them.

BEFORE YOU WRITE ANYTHING

Say three things out loud, every time. **The grain** — what does one row of my result mean? **The join direction** — must the zero case appear, and does any join fan out? **The edge cases** — ties, `NULL` s, empty sets. Candidates who say those three sentences write correct SQL more often, and are scored higher even when they do not.

Q1 — EVERY CUSTOMER WITH THEIR ORDER COUNT, INCLUDING THOSE WITH NONE

```
SELECT c.full_name,  
       count(o.order_id) AS orders  
FROM   customers c  
LEFT   JOIN orders o ON o.customer_id = c.customer_id  
GROUP  BY c.customer_id, c.full_name  
ORDER  BY orders DESC, c.full_name;
```

Why it is a test. Two traps in six lines. `LEFT JOIN`, or Priya — who has never ordered — disappears. And `count(o.order_id)`, not `count(*)`: the left join gives her one all- `NULL` row, so `count(*)` would report 1 order instead of 0. Getting this pair right is the single most reliable signal of someone who has debugged a report.

Group by the key too. `GROUP BY c.customer_id, c.full_name` rather than the name alone, so two customers with the same name do not merge.

Q2 — REVENUE PER CUSTOMER, FROM THE LINE ITEMS

```
SELECT c.full_name,  
       coalesce(sum(t.line_total), 0) AS revenue  
FROM   customers c  
LEFT   JOIN orders o ON o.customer_id = c.customer_id  
                AND o.status <> 'cancelled'  
LEFT   JOIN (SELECT order_id, sum(quantity * unit_price) AS line_total  
              FROM   order_items GROUP BY order_id) t ON t.order_id = o.order_id  
GROUP  BY c.customer_id, c.full_name  
ORDER  BY revenue DESC;
```

Why the derived table. Joining `customers` → `orders` → `order_items` directly and summing works here, but the moment you also want `count(o.order_id)` in the same query it is wrong — the order is repeated once per line, so the count inflates. Aggregating `order_items` to order grain *first* keeps every subsequent aggregate at the grain you expect.

The status filter is in `ON`, not `WHERE` — deliberately. In `WHERE` it would discard the `NULL` row for customers with no orders and turn the left join back into an inner one.

Q3 — THE TOP EARNER IN EACH DEPARTMENT, TIES INCLUDED

```
SELECT department_id, full_name, salary  
FROM   (SELECT department_id, full_name, salary,  
              rank() OVER (PARTITION BY department_id ORDER BY salary DESC) AS rnk  
              FROM   employees  
              WHERE  department_id IS NOT NULL) t  
WHERE  rnk = 1  
ORDER  BY department_id;
```

Say the tie policy. `rank()` returns everyone on the top salary; `row_number()` would pick exactly one arbitrarily. State which the question wants before writing — and if the interviewer has not said, ask.

The `IS NOT NULL` guard excludes Nisha deliberately rather than dropping her silently. Naming that is worth as much as the window function.

Q4 — CUSTOMERS WHO ORDERED IN JANUARY BUT NOT FEBRUARY

```
SELECT c.full_name
FROM   customers c
WHERE  EXISTS (SELECT 1 FROM orders o
              WHERE  o.customer_id = c.customer_id
                    AND  o.order_date >= DATE '2024-01-01'
                    AND  o.order_date <  DATE '2024-02-01')
AND    NOT EXISTS (SELECT 1 FROM orders o
                  WHERE  o.customer_id = c.customer_id
                        AND  o.order_date >= DATE '2024-02-01'
                        AND  o.order_date <  DATE '2024-03-01');
```

Two deliberate choices. `NOT EXISTS` rather than `NOT IN`, so a `NULL` cannot empty the result. And half-open date ranges rather than `BETWEEN`, so nothing on the boundary is double-counted or missed — `BETWEEN '2024-01-01' AND '2024-01-31'` silently drops anything timestamped later that day if the column ever becomes a timestamp.

Alternative worth mentioning: `EXCEPT` over two date-filtered customer lists. Shorter, and it deduplicates for free — a good answer if the interviewer wants set operations.

Q5 — MONTH-OVER-MONTH REVENUE GROWTH

```
WITH monthly AS (  
    SELECT date_trunc('month', d.day)::date AS month,  
           coalesce(sum(o.total_amount), 0) AS revenue  
    FROM    generate_series(DATE '2024-01-01', DATE '2024-12-01',  
                           INTERVAL '1 month') AS d(day)  
  
    LEFT JOIN orders o  
        ON date_trunc('month', o.order_date) = d.day  
        AND o.status <> 'cancelled'  
    GROUP BY 1  
)  
SELECT month,  
       revenue,  
       lag(revenue) OVER (ORDER BY month) AS prev_month,  
       round(100.0 * (revenue - lag(revenue) OVER (ORDER BY month))  
            / nullif(lag(revenue) OVER (ORDER BY month), 0), 1) AS growth_pct  
FROM    monthly  
ORDER BY month;
```

The gap-fill is the answer. `lag` returns the previous *row*, not the previous month — so with a month missing from the data, May's "previous month" becomes March and the growth figure is nonsense. Generating the month series first guarantees every month is a row. This is the most common window-function bug in production reporting, and volunteering it is what distinguishes the answer.

`nullif(..., 0)` in the denominator, so a zero-revenue month gives `NULL` rather than a division error.

Q6 — EACH CUSTOMER'S MOST RECENT ORDER

```
-- Portable: window function, filtered outside
WITH ranked AS (
    SELECT o.*, row_number() OVER (PARTITION BY o.customer_id
                                   ORDER BY o.order_date DESC, o.order_id DESC) AS rn
    FROM   orders o WHERE o.status <> 'cancelled'
)
SELECT c.full_name, r.order_id, r.order_date, r.total_amount
FROM   ranked r JOIN customers c ON c.customer_id = r.customer_id
WHERE  r.rn = 1 ORDER BY c.full_name;

-- PostgreSQL: shorter, usually faster
SELECT DISTINCT ON (o.customer_id) c.full_name, o.order_id, o.order_date
FROM   orders o JOIN customers c ON c.customer_id = o.customer_id
WHERE  o.status <> 'cancelled'
ORDER BY o.customer_id, o.order_date DESC, o.order_id DESC;
```

Two sentences that must be said. The CTE is not optional — window functions run after `WHERE`, so `WHERE row_number() = 1` is an error. And `order_id` is in the `ORDER BY` as a unique tie-break, or a customer with two orders on one date gets an arbitrary and unstable answer.

Q7 — PRODUCTS NEVER ORDERED

```
SELECT p.product_name
FROM   products p
WHERE  NOT EXISTS (SELECT 1 FROM order_items oi
                   WHERE oi.product_id = p.product_id);
```

Then say why not the alternatives. `NOT IN` works *here* because `order_items.product_id` is `NOT NULL` — but that is a property of today's schema, not of the query, and dropping the constraint would break it silently. A `LEFT JOIN ... WHERE oi.product_id IS NULL` is also correct and reads less clearly. `NOT EXISTS` is right regardless of the schema, which is why it is the default.

Q8 — RUNNING TOTAL AND EACH ORDER'S SHARE OF CUMULATIVE REVENUE

```
SELECT order_date,
       total_amount,
       sum(total_amount) OVER (ORDER BY order_date, order_id
                               ROWS UNBOUNDED PRECEDING) AS running_total,
       round(100.0 * sum(total_amount) OVER (ORDER BY order_date, order_id
                                              ROWS UNBOUNDED PRECEDING)
            / sum(total_amount) OVER (), 1) AS cumulative_pct
FROM   orders
WHERE  status <> 'cancelled'
ORDER BY order_date, order_id;
```

Two windows, one pass. The cumulative sum uses an ordered frame; the grand total uses `OVER ()` with no ordering at all. Mixing them in one expression is the shape behind every ABC analysis and every "share of total" column.

The frame is explicit on purpose. `ROWS UNBOUNDED PRECEDING` rather than relying on the default — the default with `ORDER BY` is `RANGE`, which includes all rows tied on the ordering value, so two orders on the same date would each show the pair's total. Writing the frame removes the ambiguity.

Q9 — THE FULL MANAGEMENT HIERARCHY, INDENTED

```
WITH RECURSIVE org AS (
  SELECT employee_id, full_name, manager_id, 1 AS depth,
         full_name::text AS path
  FROM   employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.employee_id, e.full_name, e.manager_id, o.depth + 1,
         o.path || ' > ' || e.full_name
  FROM   employees e JOIN org o ON o.employee_id = e.manager_id
  WHERE  o.depth < 20
)
SELECT repeat('  ', depth - 1) || full_name AS chart, depth, path
FROM   org ORDER BY path;
```

Three details, all volunteered. `ORDER BY path`, not `depth` — path sorts each subtree beneath its parent; depth lists all level-2 staff together and destroys the tree. The depth guard, because real hierarchy data contains cycles. And the join direction: `o.employee_id = e.manager_id` descends, the reverse ascends — getting that the right way round is most of the difficulty of recursive CTEs.

Q10 — ORDERS BY STATUS PER MONTH, AS COLUMNS

```
SELECT date_trunc('month', order_date)::date      AS month,
       count(*)                                  AS all_orders,
       count(*) FILTER (WHERE status = 'delivered') AS delivered,
       count(*) FILTER (WHERE status = 'cancelled') AS cancelled,
       sum(total_amount) FILTER (WHERE status <> 'cancelled') AS realised
FROM   orders
GROUP BY 1 ORDER BY 1;
```

Say the limitation. SQL cannot pivot dynamically — a result set's columns must be known at planning time, and a pivot's columns come from the data. With a fixed status list, conditional aggregation is the answer and it is entirely standard. With an unknown list, the database should produce the long form (one row per month-status) and the presentation layer should pivot it.

Portable variant: `count(CASE WHEN status = 'delivered' THEN 1 END)` for MySQL and older engines. `FILTER` is clearer where available.

Q11 — EACH EMPLOYEE AGAINST THEIR DEPARTMENT AVERAGE

```
SELECT full_name,
       department_id,
       salary,
       round(avg(salary) OVER (PARTITION BY department_id), 2) AS dept_avg,
       salary - round(avg(salary) OVER (PARTITION BY department_id), 2) AS vs_dept,
       round(avg(salary) OVER (), 2)                                AS company_avg
FROM   employees
WHERE  department_id IS NOT NULL
ORDER BY department_id, salary DESC;
```

Why not `GROUP BY`. The question needs detail *and* aggregate on the same row — three grains in one pass. `GROUP BY` collapses; `OVER` annotates. Doing this with `GROUP BY` takes two subqueries and two joins.

Note the absent `ORDER BY` inside `OVER`. Adding one would silently turn the department average into a running average — the frame-default trap. Say that you left it out deliberately.

Q12 — CUSTOMERS WHO ORDERED EVERY PRODUCT IN A CATEGORY

```
SELECT c.customer_id, c.full_name
FROM   customers c
JOIN   orders o      ON o.customer_id = c.customer_id
JOIN   order_items oi ON oi.order_id   = o.order_id
JOIN   products p     ON p.product_id  = oi.product_id
                        AND p.category  = 'Peripherals'
WHERE  o.status <> 'cancelled'
GROUP BY c.customer_id, c.full_name
HAVING count(DISTINCT p.product_id) = (SELECT count(*) FROM products
                                       WHERE category = 'Peripherals');
```

Relational division. The `DISTINCT` is what makes it correct — a customer who bought the same keyboard three times must not count as three products. And the shape relaxes usefully: change `=` to `>=` for "at least this many", or drop the `HAVING` and rank by the count for "closest to complete".

The double-negation form — `NOT EXISTS` a product that this customer has `NOT EXISTS` ordered — is the classic textbook answer. Be able to read it; ship the counting form.

THE REMAINING 188 — WRITE EACH ONE FULLY

Grouped by the shape they exercise. If one takes more than ten minutes, find its pattern in Part 35 rather than grinding at it.

- 13** All employees, sorted by salary descending, showing name and department name.
- 14** Employees hired before 2021, with tenure in whole years.
- 15** Employees whose name starts with a vowel.
- 16** Employees with no email recorded.
- 17** Employees with no manager, and separately, employees who manage nobody.
- 18** Every employee with their manager's name, keeping the CEO.
- 19** Managers with a count of their direct reports.
- 20** Managers with a count of reports at every level below them.
- 21** The management chain above one named employee.
- 22** Employees unreachable from any root in the hierarchy.
- 23** Each department's headcount, including departments with none.
- 24** Departments with more than three employees.
- 25** Departments with no employees, two ways.
- 26** Each department's payroll and its share of the company total.
- 27** Each employee's salary as a percentage of their department's payroll.

- 28** Departments whose average salary beats the company average.
- 29** The second-highest salary in the company, three ways.
- 30** The Nth highest salary, parameterised.
- 31** The second-highest salary *in each department*.
- 32** Employees earning above their own department average.
- 33** The top two earners per department, ties at second place included.
- 34** Salary quartiles across the company, with each quartile's boundary values.
- 35** The median salary, and the median salary per department.
- 36** The first and last hire in each department, in one row per department.
- 37** Employees hired in the same month as at least one other employee.
- 38** Hires per year, with a running cumulative headcount.
- 39** The gap in days between each employee's hire and the previous hire.
- 40** Employees whose salary is within 10% of the company average.
- 41** Salaries bucketed into three named bands with a count each.
- 42** Employees ranked by salary, with their percentile.
- 43** A pay-rise simulation: 8% for Engineering, 5% elsewhere, without updating.
- 44** Every department paired with every job level (a deliberate cross join).
- 45** Employees and departments as one combined contact list.
- 46** Cities where both an office and a customer exist.
- 47** Cities with customers but no office.
- 48** All customers with their city, marking those in an office city.
- 49** Customers who have never ordered, three ways, with the trade-offs.
- 50** Customers with exactly one order.
- 51** Customers with more than one order in the same month.
- 52** Customers whose first order was more than a year ago.
- 53** Days between each customer's signup and their first order.
- 54** Days between each customer's consecutive orders.
- 55** Each customer's longest gap between orders.
- 56** Customers who have not ordered in the last 90 days but did before.
- 57** Each customer's first and most recent order in one row.
- 58** Each customer's average order value, and the company average, side by side.

- 59** Customers whose total spend exceeds the average customer spend.
- 60** The top five customers by revenue, with their cumulative share.
- 61** Customers ranked within their city by spend.
- 62** The highest-spending customer in each city.
- 63** Customer count and revenue per city, sorted by revenue.
- 64** Cities with more than one customer and above-average revenue per customer.
- 65** New versus returning customers per month.
- 66** A retention matrix: signup month against months-since-signup.
- 67** Repeat-purchase rate as a single percentage.
- 68** Each customer's order sequence number and days since their previous order.
- 69** Customers whose spending declined in three consecutive months.
- 70** Each customer's longest streak of consecutive order days.
- 71** Orders grouped into runs of consecutive same-status dates.
- 72** All orders with the previous and next order date for the same customer.
- 73** Orders whose total exceeds the average order total.
- 74** Orders whose stored total disagrees with the sum of their line items.
- 75** Orders with more than two distinct products.
- 76** Orders containing a specific product.
- 77** Orders containing *only* products from one category.
- 78** Orders with no line items (an integrity check).
- 79** The line count and value of every order, largest first.
- 80** The most expensive line on each order.
- 81** Each order's lines as a comma-separated product list.
- 82** Each order as a nested JSON object with an array of its lines.
- 83** Order counts by status, including statuses with zero orders.
- 84** The percentage of orders cancelled, per month.
- 85** Revenue per month for the current year, gap-filled.
- 86** Revenue per quarter with a `ROLLUP` subtotal and grand total.
- 87** Revenue by month and category as a `GROUPING SETS` query.
- 88** Distinguish a `ROLLUP` subtotal row from a genuine `NULL`.
- 89** A 3-order moving average of order value.

- 90** A true rolling 7-day revenue average over a gap-filled daily series.
- 91** Revenue this month against the same month last year.
- 92** Year-to-date revenue as at any given date.
- 93** The day of the week with the most orders.
- 94** Orders per weekday name, in weekday order, not alphabetical.
- 95** The busiest hour of the day for orders.
- 96** Orders in the last 30 days, and separately, in the last complete calendar month.
- 97** Orders placed on a weekend.
- 98** Orders placed in the last quarter of the financial year (April start).
- 99** Every date in a month, with order counts, including zero days.
- 100** Dates in a range with *no* orders at all.
- 101** Each product's total units sold and revenue.
- 102** Products never ordered, and products ordered exactly once.
- 103** The top five products by revenue.
- 104** Each product's rank within its category by revenue.
- 105** The best-selling product in each category.
- 106** Categories with revenue above the average category revenue.
- 107** Products priced above every product in another named category.
- 108** Products priced above their own category average.
- 109** Products out of stock that appear on unshipped orders.
- 110** Products whose sale price ever differed from their list price.
- 111** The average discount per product, where sale price is below list.
- 112** Product pairs frequently bought together (market-basket).
- 113** Customers who bought product A but never product B.
- 114** Customers who bought at least three products from one category.
- 115** Products bought by more than half of all customers.
- 116** Each category's contribution to monthly revenue, as a percentage.
- 117** Revenue by category with a cumulative percentage (ABC classification).
- 118** Stock cover in days, from current stock and 30-day average sales.
- 119** Products needing reorder, net of quantities on unshipped orders.
- 120** The revenue lost to cancelled orders, per month and per product.

- 121** Concatenate name and city, handling a `NULL` city.
- 122** Extract the domain from every email address.
- 123** Count customers per email domain.
- 124** Find email addresses that do not look valid.
- 125** Normalise names to title case.
- 126** Find names with leading or trailing whitespace.
- 127** Split a full name into first and last, handling middle names.
- 128** Find rows where two text columns differ only by case or whitespace.
- 129** Search products by substring, case-insensitively, using an index.
- 130** Mask all but the last four characters of a phone number.
- 131** Find duplicate customers by normalised email.
- 132** Delete duplicates, keeping the oldest of each group.
- 133** Find customers with the same name but different emails.
- 134** Find orders referencing a customer that does not exist.
- 135** Find order items referencing a product that does not exist.
- 136** Find employees whose manager is in a different department.
- 137** Find employees who are their own manager.
- 138** Find cycles in the management hierarchy.
- 139** Find salaries outside a plausible range for their department.
- 140** Find hire dates in the future.
- 141** Find orders whose date precedes their customer's signup.
- 142** Find line items whose unit price is zero.
- 143** Find negative or zero quantities.
- 144** Write one query summarising every integrity violation above as a report.
- 145** Insert a new customer and their first order in one transaction.
- 146** Insert an order and its lines, computing the total from the lines.
- 147** Upsert a customer by email, updating the city if they exist.
- 148** Upsert daily revenue into a summary table.
- 149** Update all prices in one category by 10%, returning the changed rows.
- 150** Update each order's total from its line items, in one statement.
- 151** Decrement stock for a checkout so it can never go negative.

- 152** Cancel an order and restore its stock, atomically.
- 153** Soft-delete a customer and prove their email can be reused.
- 154** Delete orders older than five years, in batches.
- 155** Delete a customer's data entirely, respecting foreign keys.
- 156** Merge two duplicate customer records, reassigning their orders.
- 157** Copy last year's prices into a price-history table.
- 158** Backfill a new `updated_at` column in resumable batches.
- 159** Move an employee to a new department, closing the old validity span.
- 160** Perform an SCD Type 2 update on a customer dimension.
- 161** Create the index that makes query 6 an index-only scan.
- 162** Create the partial index for "pending orders by date".
- 163** Create the index supporting keyset pagination of the order list.
- 164** Rewrite an unsargable date filter so it can use an index.
- 165** Rewrite `OFFSET 100000` pagination as keyset, with the cursor.
- 166** Rewrite a correlated subquery as a window function and compare plans.
- 167** Rewrite three select-list subqueries as one join.
- 168** Rewrite a query so its `ORDER BY` needs no sort node.
- 169** Find the foreign keys in the practice schema with no supporting index.
- 170** Find every index in the database that has never been used.
- 171** Report table and index sizes with dead-row percentages.
- 172** Report the top ten queries by total execution time.
- 173** Report which sessions are currently blocking which.
- 174** Report every table without a primary key.
- 175** Report every column that is nullable and probably should not be.
- 176** Create a view of active customers and query through it.
- 177** Create a materialized view of daily revenue with a unique index.
- 178** Write the reconciliation query for `orders.total_amount` .
- 179** Write a function returning a customer's lifetime value.
- 180** Write a trigger maintaining `orders.total_amount` .
- 181** Write a generic audit trigger and attach it to `employees` .
- 182** Write the statement-level equivalent of that trigger using transition tables.

- 183** Add an exclusion constraint preventing overlapping employee department spans.
- 184** Add a partial unique index for "one primary address per customer".
- 185** Add a composite foreign key enforcing "manager in the same department".
- 186** Add row-level security to `orders` keyed on a session variable.
- 187** Grant a reporting role read access excluding customer emails.
- 188** Store an order's metadata as `jsonb` and query inside it.
- 189** Index that JSONB field so the filter is sargable and estimated correctly.
- 190** Unnest a JSONB array of line items into rows.
- 191** Partition `orders` by month and prove pruning.
- 192** Build a star schema over the practice tables at line grain.
- 193** Build `dim_date` for the current year with a fiscal year.
- 194** Write the canonical star query with `ROLLUP`.
- 195** Write a query answering "revenue by segment as at the time of sale".
- 196** Write the same query for "as the segment is today".
- 197** Write an incremental fact-table load with a watermark.
- 198** Write a funnel from cart to delivered with drop-off per stage.
- 199** Write a sessionization query with a 30-minute timeout.
- 200** Take any query you have written above and optimise it end to end: plan, index, re-measure, and report both timings.

A BETTER WAY TO PRACTISE THESE

Do not work through them in order. Pick ten at random, write them, then **run each against the practice data and check the answer by hand** — six-row tables exist precisely so you can verify a result without trusting the query. A query you believe is right and have not checked is the thing that produces a wrong report in production.

THE THREE-MINUTE RULE

In an interview you will be given a schema and one question, and you will have five minutes. Practise the constraint: set a timer, write the query on paper without a database, then check it. Writing SQL without autocomplete or error messages is a genuinely different skill from writing it in an editor, and it is the one being tested.

SQL Patterns

Twenty patterns solve most real SQL problems. Each one gets the same treatment: what it is, the generic template, a worked example, **why it works**, and how to recognise the next problem that wants it. Learn these and the 500 challenges in Part 34 stop being 500 separate problems.

READ THIS PART BEFORE PART 34

Patterns come first deliberately. A challenge you cannot start is almost always a pattern you have not met; a challenge you solve badly is usually a pattern you half-remember. When a problem stalls you, the productive question is not "what is the answer" but "**which of the twenty is this?**"

35.1 The ranking family

Five patterns that are one idea in different clothes: **number the rows inside a group, then filter or read off the number**. Recognise the family and you have collapsed a third of all SQL interview questions into one move.

PATTERN 1 · TOP-N PER GROUP

```
-- TEMPLATE
WITH ranked AS (
    SELECT t.*,
           row_number() OVER (PARTITION BY <group> ORDER BY <rank_by> DESC,
                               <unique_tiebreak>) AS rn
    FROM   <table> t
)
SELECT * FROM ranked WHERE rn <= <N>;
```

Why it works. Window functions are evaluated after `WHERE` and before `ORDER BY`, so the rank exists as a value only *outside* the query that computed it. The CTE is not stylistic — it is the only way to filter on a window result.

The tie decision, which is the real content of the pattern. `row_number()` for exactly N rows, ties broken arbitrarily. `rank()` for "everyone at the top position", which may return more than N. `dense_rank()` for "the top N *distinct* values", which is what "second-highest salary" actually means.

Recognise it by: the words *per, each, latest, best, most recent* attached to a group. "The latest order per customer." "The three best-selling products in each category." "The most recent status of each ticket."

POSTGRESQL SHORTCUT, AND WHEN NOT TO USE IT

For N = 1 exactly, `DISTINCT ON` is shorter and often faster because it can stop reading each group after the first row: `SELECT DISTINCT ON (customer_id) * FROM orders ORDER BY customer_id, order_date DESC, order_id DESC`. The `ORDER BY` must begin with the `DISTINCT ON` expressions. It is PostgreSQL-only, so the window form is what you write in portable code and in interviews unless you know the shop.

PATTERN 2 · LATEST RECORD PER ENTITY

Top-N per group with N = 1 and a timestamp as the ranking key. It deserves its own name because it is the single most common query in production software — the current status, the latest price, the most recent reading, the newest version.

```
-- The three forms, in order of preference
-- (a) DISTINCT ON – Postgres, fastest with a matching index
SELECT DISTINCT ON (device_id) device_id, reading, taken_at
FROM readings ORDER BY device_id, taken_at DESC, reading_id DESC;

-- (b) Window function – portable
SELECT * FROM (SELECT r.*, row_number() OVER (PARTITION BY device_id
      ORDER BY taken_at DESC, reading_id DESC) rn
      FROM readings r) t WHERE rn = 1;

-- (c) LATERAL – best when the entity list is small and readings are huge
SELECT d.device_id, r.reading, r.taken_at
FROM devices d
LEFT JOIN LATERAL (SELECT reading, taken_at FROM readings
      WHERE device_id = d.device_id
      ORDER BY taken_at DESC, reading_id DESC LIMIT 1) r ON true;
```

Why (c) can beat both. With an index on `(device_id, taken_at DESC)`, the lateral form reads *one row per device* and stops. The window form must rank every reading before discarding all but the newest. On a billion-row readings table with a thousand devices, that is a thousand index seeks against a billion-row sort.

THE TIE-BREAK IS NOT OPTIONAL

`ORDER BY taken_at DESC` alone is non-deterministic when two readings share a timestamp — and they will, because timestamps are coarser than events. Two runs return different answers, tests flake, and a paginated list silently skips or repeats rows. Always append a unique column.

PATTERN 3 · NTH HIGHEST VALUE

```
-- TEMPLATE: dense_rank, because "Nth highest" means Nth DISTINCT
SELECT <cols> FROM (
      SELECT t.*, dense_rank() OVER (ORDER BY <col> DESC) AS dr FROM <table> t
) x WHERE dr = <N>;
```

Why `dense_rank` and not the others. If three people earn the top salary, the "second-highest salary" is the next distinct figure — not the second of those three people. `row_number()` would return one of the top earners; `rank()` would leave rank 2 empty and return nothing. Only `dense_rank()` answers the question as asked.

Recognise it by: an ordinal on a *value* rather than on a row — second-highest salary, third-largest order, fifth-most-recent login.

PATTERN 4 · ABOVE THE GROUP AVERAGE

```
-- TEMPLATE: annotate with the group aggregate, then compare
SELECT * FROM (
    SELECT t.*, avg(<metric>) OVER (PARTITION BY <group>) AS grp_avg
    FROM <table> t
) x WHERE <metric> > grp_avg;
```

Why the window form beats the correlated subquery. The classic answer is `WHERE salary > (SELECT avg(salary) FROM employees x WHERE x.department_id = e.department_id)`, which conceptually re-runs the subquery per row. The window form computes every group average in one pass. Same answer, one scan instead of N.

THE NULL GROUP TRAP

An employee with no department gets `PARTITION BY NULL` — a partition of its own, whose average is their own salary, so they can never exceed it. They vanish from the result *silently*. Decide explicitly: exclude with `WHERE department_id IS NOT NULL` and say you did, or handle the group deliberately. Silent disappearance is the failure mode of every grouped comparison.

PATTERN 5 · PERCENTILES AND BUCKETS

```
-- Bucket rows into N equal groups
SELECT customer_id, revenue,
       ntile(4)      OVER (ORDER BY revenue DESC) AS quartile,
       percent_rank() OVER (ORDER BY revenue)      AS pct_rank
FROM   customer_revenue;

-- Percentile VALUES (an aggregate, not a window function)
SELECT percentile_disc(0.95) WITHIN GROUP (ORDER BY duration_ms) AS p95,
       percentile_cont(0.50) WITHIN GROUP (ORDER BY duration_ms) AS median
FROM   requests;
```

Why the two spellings differ. `ntile` and `percent_rank` label rows; `percentile_cont` and `percentile_disc` return a *value* from the distribution. And `_disc` returns an actual observation while `_cont` interpolates — so a p95 latency should be `_disc`, because an interpolated latency is a number that never happened.

35.2 The existence family

Four patterns about presence and absence. All four are `EXISTS` or `NOT EXISTS` wearing different hats, and all four have a wrong answer that looks right.

PATTERN 6 · MISSING RECORDS (THE ANTI-JOIN)

```
-- TEMPLATE – the form that is correct regardless of schema
SELECT p.* FROM parent p
WHERE NOT EXISTS (SELECT 1 FROM child c WHERE c.parent_id = p.parent_id);
```

Why not the alternatives. `NOT IN` against a subquery is a correctness defect: one `NULL` in the subquery makes the predicate unable to be true for any row, and the query returns nothing — no error, no warning. `LEFT JOIN ... WHERE c.id IS NULL` is correct and reads less clearly, and it materialises matched rows before discarding them.

Recognise it by: *never, no, without, missing, orphaned*. "Customers who have never ordered." "Products with no reviews." "Employees not assigned to a project."

PATTERN 7 · DUPLICATE DETECTION

```
-- Find them
SELECT <key_cols>, count(*) AS copies
FROM <table> GROUP BY <key_cols> HAVING count(*) > 1;

-- Delete all but the survivor of each group
DELETE FROM <table> t
WHERE EXISTS (SELECT 1 FROM <table> k
              WHERE k.<key_cols> = t.<key_cols>
              AND k.<id> < t.<id>);      -- keep the lowest id

-- Or with a window, when "survivor" is a policy rather than min(id)
DELETE FROM <table> WHERE <id> IN (
    SELECT <id> FROM (SELECT <id>, row_number() OVER (PARTITION BY <key_cols>
                                                    ORDER BY <keep_rule>) rn FROM <table>) x
    WHERE rn > 1);
```

Why normalise before comparing. Duplicates in real data are rarely byte-identical — `'Asha@example.com '` and `'asha@example.com'` are the same customer. Group on `lower(trim(col))`, not the raw column, or you will report zero duplicates on a table full of them.

THE QUERY IS THE CLEANUP; THE CONSTRAINT IS THE FIX

Always finish a deduplication by adding the unique index that should have existed — `CREATE UNIQUE INDEX ON customers (lower(email)) WHERE deleted_at IS NULL`. Without it you will run the same cleanup again next quarter.

PATTERN 8 · RELATIONAL DIVISION — "HAS ALL OF"

```
-- TEMPLATE: count distinct matches against the required total
SELECT e.<entity_id>
FROM   <entity> e
JOIN   <bridge> b ON b.<entity_id> = e.<entity_id>
JOIN   <required> r ON r.<req_id> = b.<req_id> AND <required_filter>
GROUP BY e.<entity_id>
HAVING count(DISTINCT r.<req_id>) = (SELECT count(*) FROM <required>
                                   WHERE <required_filter>);
```

Why the `DISTINCT` is load-bearing. Without it, a customer who bought the same product three times counts as three products and passes a test they should fail. This one word is the whole correctness of the pattern.

Why it relaxes so well. Change `=` to `>=` for "at least N of them". Drop the `HAVING` and order by the count for "ranked by completeness". Neither is possible with the double-negation form.

THE TEXTBOOK FORM, WORTH BEING ABLE TO READ

```
WHERE NOT EXISTS (SELECT 1 FROM required r WHERE NOT EXISTS (SELECT 1 FROM bridge b WHERE
b.entity_id = e.entity_id AND b.req_id = r.req_id)) — "there is no required item that this
entity has not got". Interviewers ask for it specifically to see whether you can reason about nested
negation. Ship the counting form.
```

PATTERN 9 · SEMI-JOIN FOR FILTERING WITHOUT FAN-OUT

```
-- WRONG: the join multiplies rows, so the count is inflated
SELECT c.full_name, count(*) FROM customers c JOIN orders o USING (customer_id)
WHERE o.status = 'delivered' GROUP BY 1;    -- counts ORDERS, maybe intended

-- RIGHT when you only need to FILTER: existence, no multiplication
SELECT c.full_name FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
              AND o.status = 'delivered');
```

The decision rule. Need columns or aggregates from the other table → join, and check the row count changed as expected. Need only to know a related row exists → `EXISTS`. One in, at most one out, whatever the other side contains.

35.3 The sequence family

Six patterns about rows that relate to their neighbours. This family is where SQL stops being a reporting language and starts being an analysis language.

PATTERN 10 · RUNNING TOTALS

```
-- TEMPLATE
SELECT <dim>, <metric>,
       sum(<metric>) OVER (PARTITION BY <group> ORDER BY <seq>
                          ROWS UNBOUNDED PRECEDING) AS running_total,
       round(100.0 * sum(<metric>) OVER (PARTITION BY <group> ORDER BY <seq>
                          ROWS UNBOUNDED PRECEDING)
            / sum(<metric>) OVER (PARTITION BY <group>), 1) AS cumulative_pct
FROM   <table>;
```

Write the frame explicitly. The default frame with `ORDER BY` is `RANGE UNBOUNDED PRECEDING TO CURRENT ROW`, which includes *all rows tied on the ordering value* — so two orders on the same date each show the pair's cumulative total. `ROWS` counts rows and does what you meant. This is the most common silent error in window SQL.

Note the two windows. The cumulative sum is ordered; the denominator uses `OVER (PARTITION BY ...)` with no ordering to get the group total. Mixing an ordered and an unordered window in one expression is the shape behind every "share of total" and every ABC analysis.

PATTERN 11 · MOVING AVERAGES

```
-- A rolling 7 ROWS
avg(x) OVER (ORDER BY d ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)

-- A rolling 7 DAYS — different, and usually what was meant
avg(x) OVER (ORDER BY d RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW)
```

Why the distinction matters. `ROWS 6 PRECEDING` is the last seven *records*, which equals seven days only if every day has exactly one row. With gaps it silently spans weeks; with duplicates it silently covers three days. A genuine time window needs `RANGE` **and** a gap-filled series, so a day with no activity contributes a zero rather than being absent.

PATTERN 12 · PERIOD-OVER-PERIOD COMPARISON

```
-- TEMPLATE: gap-fill the periods FIRST, then lag
WITH periods AS (
    SELECT g.p AS period, coalesce(sum(t.<metric>), 0) AS metric
    FROM   generate_series(<start>, <end>, INTERVAL '1 month') AS g(p)
    LEFT   JOIN <table> t ON date_trunc('month', t.<date>) = g.p
    GROUP  BY g.p
)
SELECT period, metric,
       lag(metric) OVER (ORDER BY period) AS prev,
       round(100.0 * (metric - lag(metric) OVER (ORDER BY period))
            / nullif(lag(metric) OVER (ORDER BY period), 0), 1) AS growth_pct
FROM   periods ORDER BY period;
```

Why gap-filling is the pattern, not a detail. `lag` returns the previous **row**, not the previous month. A month with no sales has no row, so the following month's "previous month" is two months back and the growth figure is wrong — plausibly wrong, which is worse than an error. Generating the period series guarantees one row per period.

`nullif(..., 0)` in the denominator turns a division-by-zero into a `NULL`, which renders as blank rather than crashing the report.

PATTERN 13 · GAPS AND ISLANDS

```
-- TEMPLATE – the invariant: value - row_number is constant within a run
WITH deduped AS (SELECT DISTINCT <entity>, <date> FROM <table>),
numbered AS (
    SELECT <entity>, <date>,
           row_number() OVER (PARTITION BY <entity> ORDER BY <date>) AS rn
    FROM   deduped),
islands AS (
    SELECT <entity>, <date>, <date> - (rn || ' days')::interval AS island_key
    FROM   numbered)
SELECT <entity>, min(<date>) AS run_start, max(<date>) AS run_end, count(*) AS run_len
FROM   islands GROUP BY <entity>, island_key ORDER BY 1, 2;
```

Why it works. Consecutive dates increase by one and so does the row number, so their difference is constant across a run and changes at every gap. The difference is meaningless as a value — it exists only to be grouped by, which is why it never appears in the output.

The `DISTINCT` is essential. Two events on one day would give that day two row numbers, two island keys, and split a real streak in half.

Recognise it by: *consecutive, streak, unbroken, contiguous, run of.* Login streaks, unbroken subscription periods, contiguous seat blocks, days without an incident.

PATTERN 14 · FLAG AND CUMULATE — GROUPING CONSECUTIVE SAME-VALUE ROWS

```
-- TEMPLATE: mark where the value changes, then cumulative-sum the marks
WITH marked AS (
    SELECT t.*,
           CASE WHEN <col> IS DISTINCT FROM lag(<col>) OVER (ORDER BY <seq>)
                THEN 1 ELSE 0 END AS is_new_run
    FROM   <table> t),
grouped AS (
    SELECT *, sum(is_new_run) OVER (ORDER BY <seq> ROWS UNBOUNDED PRECEDING) AS run_id
    FROM   marked)
SELECT run_id, <col>, min(<seq>) AS from_, max(<seq>) AS to_, count(*) AS len
FROM   grouped GROUP BY run_id, <col> ORDER BY run_id;
```

Why it works. The flag is 1 exactly once per run, so a running sum of it increments once per run — giving every row in a run the same identifier. Unlike gaps-and-islands it needs no arithmetic on the values, so it works for text, booleans, statuses, anything.

IS DISTINCT FROM, not **<>** — because the first row's **lag** is **NULL**, and **x <> NULL** is unknown, so the first run would never be flagged.

PATTERN 15 · SESSIONIZATION

```
WITH gapped AS (
    SELECT user_id, event_at,
           CASE WHEN event_at - lag(event_at) OVER w > INTERVAL '30 minutes'
                OR lag(event_at) OVER w IS NULL
                THEN 1 ELSE 0 END AS new_session
    FROM   events
    WINDOW w AS (PARTITION BY user_id ORDER BY event_at))
SELECT user_id,
       sum(new_session) OVER (PARTITION BY user_id ORDER BY event_at
                              ROWS UNBOUNDED PRECEDING) AS session_no,
       event_at
FROM   gapped;
```

It is flag-and-cumulate with a time condition. The **IS NULL** branch is essential: a user's first event has no predecessor, and without it session numbering starts at 0 and the first session merges with nothing.

35.4 The analytical family

Five patterns that answer business questions rather than data questions. Each one is a composition of the earlier patterns — which is the point of learning them in order.

PATTERN 16 · COHORT RETENTION

```
WITH cohorts AS (  
    SELECT customer_id,  
           date_trunc('month', min(order_date))::date AS cohort_month  
    FROM   orders GROUP BY customer_id),  
activity AS (  
    SELECT c.cohort_month,  
           c.customer_id,  
           (extract(year FROM o.order_date) - extract(year FROM c.cohort_month)) * 12  
           + (extract(month FROM o.order_date) - extract(month FROM c.cohort_month))  
           AS month_offset  
    FROM   cohorts c JOIN orders o ON o.customer_id = c.customer_id),  
sizes AS (  
    SELECT cohort_month, count(DISTINCT customer_id) AS cohort_size  
    FROM   cohorts GROUP BY cohort_month)  
SELECT a.cohort_month, s.cohort_size, a.month_offset,  
       count(DISTINCT a.customer_id) AS active,  
       round(100.0 * count(DISTINCT a.customer_id) / s.cohort_size, 1) AS retention_pct  
FROM   activity a JOIN sizes s USING (cohort_month)  
GROUP BY a.cohort_month, s.cohort_size, a.month_offset  
ORDER BY a.cohort_month, a.month_offset;
```

The three steps are the pattern. Assign each entity to a cohort by its *first* event; compute an offset from that cohort's start rather than an absolute date; divide by the cohort's size, not by the total. Skip the third and every cohort looks like it is shrinking because later cohorts are smaller.

Why `count(DISTINCT customer_id)` — a customer with three orders in month 2 is one retained customer, not three. Retention counts people, not events.

PATTERN 17 · FUNNEL WITH DROP-OFF

```

WITH stages AS (
    SELECT session_id,
           max(CASE WHEN step = 'view'      THEN 1 ELSE 0 END) AS s1,
           max(CASE WHEN step = 'cart'      THEN 1 ELSE 0 END) AS s2,
           max(CASE WHEN step = 'checkout' THEN 1 ELSE 0 END) AS s3,
           max(CASE WHEN step = 'paid'      THEN 1 ELSE 0 END) AS s4
    FROM   funnel_events GROUP BY session_id)
SELECT sum(s1) AS viewed, sum(s2) AS carted, sum(s3) AS checked_out, sum(s4) AS paid,
       round(100.0 * sum(s2) / nullif(sum(s1), 0), 1) AS view_to_cart_pct,
       round(100.0 * sum(s3) / nullif(sum(s2), 0), 1) AS cart_to_checkout_pct,
       round(100.0 * sum(s4) / nullif(sum(s3), 0), 1) AS checkout_to_paid_pct
FROM   stages;

```

Why `max(CASE ...)` per session. It collapses many events into one row per session with a boolean per stage — a pivot (pattern 20) used as a funnel. Counting events instead of sessions inflates every stage by however many times a user reloaded.

The honest caveat to state. This measures *reaching* a stage, not passing through them in order. If order matters, compare timestamps between stages, and be explicit that a session which skipped a step is excluded rather than silently counted.

PATTERN 18 · TIME-SERIES WITH GAP FILLING

```

-- TEMPLATE: the calendar is the driving table, not the data
SELECT d.day::date AS day,
       coalesce(count(t.<id>), 0)      AS events,
       coalesce(sum(t.<metric>), 0)    AS total
FROM   generate_series(<start>, <end>, INTERVAL '1 day') AS d(day)
LEFT   JOIN <table> t ON t.<date_col> >= d.day
       AND t.<date_col> < d.day + INTERVAL '1 day'
GROUP  BY d.day ORDER BY d.day;

```

Why the series must be on the left. Grouping the data alone produces rows only for days that have data, so a chart shows a continuous line across a gap and a `lag` comparison reads the wrong neighbour. Driving from the calendar guarantees one row per period whether or not anything happened.

The half-open join condition (`>= day AND < day + 1`) rather than `date_trunc` on the column keeps the predicate sargable, so an index on the timestamp can be used.

PATTERN 19 · RECONCILIATION — PROVING A STORED VALUE

```
-- TEMPLATE: recompute from source, compare, return only disagreements
SELECT p.<id>,
       p.<stored_value> AS stored,
       coalesce(sum(c.<source_value>), 0) AS computed
FROM   parent p LEFT JOIN child c ON c.<fk> = p.<id>
GROUP BY p.<id>, p.<stored_value>
HAVING p.<stored_value> IS DISTINCT FROM coalesce(sum(c.<source_value>), 0);
```

Why this pattern earns its place. Every denormalized aggregate, counter cache and materialised balance drifts eventually — a data-repair script, a disabled trigger, a `TRUNCATE` that bypassed row triggers. This query is the difference between a system that is wrong and a system that *knows* it is wrong. A non-empty result is an alert, and it belongs in monitoring rather than in a test.

`IS DISTINCT FROM`, not `<>` — or a `NULL` stored value compares as unknown and the broken row is the one row the check misses.

PATTERN 20 · CONDITIONAL AGGREGATION (PIVOT)

```
-- TEMPLATE
SELECT <group_col>,
       count(*) AS total,
       count(*) FILTER (WHERE <cond_a>) AS col_a,
       count(*) FILTER (WHERE <cond_b>) AS col_b,
       sum(<metric>) FILTER (WHERE <cond_a>) AS metric_a
FROM   <table> GROUP BY <group_col>;

-- Portable form for MySQL and older engines
count(CASE WHEN <cond_a> THEN 1 END) AS col_a
```

Why SQL cannot pivot dynamically. A result set's columns must be known when the query is planned, and a pivot's columns come from the data. So a dynamic pivot needs two round trips — query the distinct values, then build the SQL — which is why the presentation layer should usually do it. For a known column set, conditional aggregation is complete and standard.

Recognise it by: a request for a cross-tab, a "by status" or "by month" table with one column per value, or several near-identical aggregates over the same table. Four `FILTER` clauses in one pass replace four subqueries.

THE COMPOSITIONS WORTH MEMORISING

The analytical family is built from the earlier patterns, and seeing that is the skill: **cohorts** = latest-per-entity (the first event) + gap-filled periods + conditional aggregation. **Funnels** = pivot + ratio. **Streaks** = gaps-and-islands. **Sessions** = flag-and-cumulate. **ABC analysis** = running total + an unordered window for the denominator. When a new problem arrives, decompose it into this vocabulary before writing anything — that decomposition *is* the solution.

35.5 Pattern recognition drill

Name the pattern for each. No SQL — just the number, and the one-line reason. If you can do this quickly, Part 34's five hundred problems become twenty patterns with variations.

- 1 "Show each warehouse's most recent stock count."
- 2 "Which suppliers have we never placed an order with?"
- 3 "Employees paid more than their department's average."
- 4 "The longest run of days with no failed deployments."
- 5 "Revenue this month versus last, per region."
- 6 "Customers who have bought every item in the starter bundle."
- 7 "What proportion of signups are still active after three months?"
- 8 "Orders per status as columns, per month as rows."
- 9 "The third-largest transaction value."
- 10 "Group a device's readings into periods where it stayed in one state."
- 11 "Does `orders.total` still match its line items?"
- 12 "Cumulative revenue by day, and each day's share of the year."
- 13 "Split a user's clicks into visits, thirty minutes apart."
- 14 "The top three earners in each office, including ties."
- 15 "Which customers appear twice under different email spellings?"
- 16 "A daily chart with zeroes on days we sold nothing."
- 17 "p95 response time per endpoint."
- 18 "Users who added to cart but never checked out."
- 19 "A 7-day rolling average of active users."
- 20 "Accounts whose cached balance disagrees with the ledger."

ANSWERS

1 · P2 latest-per-entity. 2 · P6 anti-join. 3 · P4 above-group-average. 4 · P13 gaps and islands. 5 · P12 period-over-period, gap-filled. 6 · P8 relational division. 7 · P16 cohort retention. 8 · P20 conditional aggregation. 9 · P3 Nth highest, `dense_rank`. 10 · P14 flag and cumulate. 11 · P19 reconciliation. 12 · P10 running total with an unordered denominator. 13 · P15 sessionization. 14 · P1 top-N per group with `rank()`. 15 · P7 duplicate detection, normalised. 16 · P18 gap-filled time series. 17 · P5 percentiles, `percentile_disc`. 18 · P17 funnel, or P6 if you only need the names. 19 · P11 moving average with `RANGE`. 20 · P19 reconciliation.

Coding Challenges

Five hundred problems across five levels. Every problem names its topic and, from Level 2 onward, the Part 35 pattern it exercises — so a problem you cannot start tells you exactly which page to re-read.

HOW TO WORK THESE

All problems run against the practice schema of Volume I §2.9 unless one defines its own tables.

Check every answer by hand against the six-row data — that is why the tables are small. A query you believe is right and have not verified is what produces a wrong report in production.

Ten minutes is the limit. Past that, look up the named pattern rather than grinding: the goal is recognition, not endurance.

34.1 Level 1 — Beginner

● **BEGINNER** 100 problems. Single table, one clause at a time.

L1.1 — WORKED

Problem. How many employees are there, and how many have an email recorded?

```
SELECT count(*)      AS employees,  
       count(email) AS with_email  
FROM   employees;
```

```
employees | with_email  
-----+-----  
          9 |          8
```

Explanation. The whole problem is the difference between the two counts. `count(*)` counts rows; `count(email)` counts non-`NULL` values, so Nisha's missing email makes it 8. This one distinction causes more wrong dashboard numbers than any other beginner error.

Topic. Aggregates and `NULL`.

L1.2 — WORKED

Problem. Employees hired before 2021, with tenure in whole years.

```
SELECT full_name,  
       hire_date,  
       extract(year FROM age(current_date, hire_date))::int AS years  
FROM   employees  
WHERE  hire_date < DATE '2021-01-01'  
ORDER  BY hire_date;
```

Explanation. `age()` returns an interval; extracting the year gives completed years. Two habits to form here: compare against a typed `DATE` literal rather than a string, and use the half-open form `< '2021-01-01'` rather than `<= '2020-12-31'` — correct whether the column is a date or a timestamp.

Common wrong answer. `(current_date - hire_date) / 365`, which drifts a day every four years.

L1.3 — WORKED

Problem. Each product with a price band: under ₹2,000 "budget", under ₹10,000 "mid", else "premium".

```
SELECT product_name, price,  
       CASE WHEN price < 2000 THEN 'budget'  
            WHEN price < 10000 THEN 'mid'  
            ELSE 'premium'  
       END AS band  
FROM   products ORDER BY price;
```

Explanation. `CASE` stops at the first true branch, so the second needs no `AND price >= 2000`. Ordering branches narrowest to widest is what makes it readable.

PROBLEMS 4–100 · TOPIC IN BRACKETS

- 4 Every employee's name and salary, highest paid first. [`ORDER BY`]
- 5 All employees in department 1. [`WHERE`]
- 6 All employees *not* in department 1, including those with none. [`NULL`]
- 7 Customers whose city is unknown. [`IS NULL`]
- 8 Customers in Hyderabad or Mumbai. [`IN`]
- 9 The five most recently hired employees. [`LIMIT`]
- 10 The same, skipping the first two. [`OFFSET`]
- 11 Distinct departments that have at least one employee. [`DISTINCT`]
- 12 Employee names in uppercase, aliased. [string functions]
- 13 Employees whose name contains "an", case-insensitively. [`ILIKE`]
- 14 Employees whose name starts with a vowel. [pattern matching]
- 15 Employees whose email domain is example.com. [`split_part`]
- 16 Each name's first letter and length. [`substring` , `length`]
- 17 Salaries rounded to the nearest thousand. [`round`]
- 18 Annual salary, aliased, sorted by the alias. [expressions]
- 19 Employees earning between 50,000 and 80,000. [`BETWEEN`]
- 20 Highest, lowest and average salary in one row. [aggregates]
- 21 Total payroll, and payroll excluding the top earner. [aggregates]
- 22 Average salary to two decimals. [`round`]
- 23 Employee count per department. [`GROUP BY`]

- 24** Departments with more than two employees. [`HAVING`]
- 25** Average salary per department, highest first. [`GROUP BY`]
- 26** Employees hired per year. [`date_trunc`]
- 27** Employees hired on a weekend. [`isodow`]
- 28** Hire-date month names in calendar, not alphabetical, order. [dates]
- 29** Employees hired in the last five years. [`INTERVAL`]
- 30** Hire dates formatted DD Mon YYYY. [`to_char`]
- 31** Days until each employee's next work anniversary. [date arithmetic]
- 32** All products, cheapest first. [`ORDER BY`]
- 33** Products out of stock. [`WHERE`]
- 34** Products under 10 in stock, with the shortfall to 20. [expressions]
- 35** Total inventory value. [aggregates]
- 36** Product count and average price per category. [`GROUP BY`]
- 37** Categories with more than one product. [`HAVING`]
- 38** Most and least expensive product in one query. [`UNION ALL`]
- 39** Each price as a percentage of the most expensive. [scalar subquery]
- 40** All orders, most recent first. [`ORDER BY`]
- 41** Orders above 10,000. [`WHERE`]
- 42** Orders in Q1 2024. [half-open range]
- 43** Order count by status. [`GROUP BY`]
- 44** Total and average order value, excluding cancelled. [aggregates]
- 45** The single largest order. [`LIMIT`]
- 46** Orders per month in 2024. [`date_trunc`]
- 47** The percentage of orders cancelled. [`FILTER`]
- 48** Orders whose status is not terminal. [`IN`]
- 49** Distinct customers who have ordered. [`DISTINCT`]
- 50** All customers, newest signup first. [`ORDER BY`]
- 51** Customers who signed up in 2024. [dates]
- 52** Customer count per city, largest first. [`GROUP BY`]
- 53** Cities with more than one customer. [`HAVING`]
- 54** Names and emails as one formatted string. [`concat_ws`]

- 55** Customers sorted by city, `NULL` s last. [`NULLS LAST`]
- 56** All line items, largest line value first. [expressions]
- 57** Total units sold. [aggregates]
- 58** Line count per order. [`GROUP BY`]
- 59** Orders with more than two lines. [`HAVING`]
- 60** Each order's computed total from its lines. [`GROUP BY`]
- 61** Lines with quantity above one. [`WHERE`]
- 62** Average unit price across all lines. [aggregates]
- 63** Departments with no location recorded. [`IS NULL`]
- 64** Departments sorted by location then name. [multi-column sort]
- 65** Count of distinct locations. [`count(DISTINCT)`]
- 66** Replace a `NULL` location with "Unassigned". [`COALESCE`]
- 67** Treat an empty-string location as unknown. [`NULLIF`]
- 68** The three highest salaries without window functions. [`LIMIT`]
- 69** Whether any employee earns above 100,000, as a boolean. [`EXISTS`]
- 70** Headcount per department as a percentage of the total. [scalar subquery]
- 71** Employees earning above the company average. [scalar subquery]
- 72** Employees earning exactly the maximum. [scalar subquery]
- 73** Products above the average product price. [scalar subquery]
- 74** Orders above the average order value. [scalar subquery]
- 75** Show that a scalar subquery returning no rows empties the outer query. [`NULL`]
- 76** Insert a department, then confirm it exists. [`INSERT`]
- 77** Insert an employee relying on the hire-date default. [`DEFAULT`]
- 78** Insert three products in one statement, returning ids. [`RETURNING`]
- 79** Update one salary, returning old and new. [`RETURNING`]
- 80** Give one department a 5% rise. [`UPDATE`]
- 81** Null out the department of employees in a closing department. [`UPDATE`]
- 82** Delete one product; explain why it may fail. [foreign keys]
- 83** Wrap a delete in a transaction, check the count, roll back. [`BEGIN`]
- 84** Violate the salary check and read the error. [constraints]
- 85** Violate a foreign key and read the error. [constraints]

- 86** Violate a primary key and read the error. [constraints]
- 87** Create a certifications table with sensible constraints. [`CREATE TABLE`]
- 88** Add a nullable `phone` column. [`ALTER TABLE`]
- 89** Add a named check requiring salary above 20,000. [constraints]
- 90** Rename a column; explain what it breaks. [`ALTER TABLE`]
- 91** Drop the column you added, safely. [`ALTER TABLE`]
- 92** Describe `employees` and read off its constraints. [`\d`]
- 93** List every index in the schema. [catalog]
- 94** Copy `employees` to a backup table with data. [`CREATE TABLE AS`]
- 95** Copy only the structure. [`WITH NO DATA`]
- 96** Export employees to CSV. [`\copy`]
- 97** Re-import it and verify the row count. [`\copy`]
- 98** Count rows in every table in one query. [catalog]
- 99** Report each table's size, prettily formatted. [catalog]
- 100** Write the five queries you would run first on an unfamiliar database. [orientation]

34.2 Level 2 — Easy

● **EASY** 100 problems. Joins and grouping across two or three tables — where fan-out first bites.

L2.1 — WORKED

Problem. Every customer with their order count, including those who have never ordered.

```
SELECT c.full_name, count(o.order_id) AS orders
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id
GROUP  BY c.customer_id, c.full_name
ORDER  BY orders DESC, c.full_name;
```

full_name	orders
Kavya Reddy	3
Rohan Gupta	2
Daniel Fernandes	1
Fatima Sheikh	1
Priya Anand	0

Explanation. Two decisions carry it. `LEFT JOIN`, or Priya vanishes. And `count(o.order_id)` rather than `count(*)` — the left join gives Priya one row with all- `NULL` order columns, so `count(*)` would report 1. Getting that pair right is the most reliable signal of someone who has debugged a report. Also `GROUP BY c.customer_id, c.full_name` : grouping by name alone would merge two customers who share one.

Topic. Outer joins, aggregate `NULL` handling. **Pattern.** Base of P6.

L2.2 — WORKED

Problem. Revenue per customer from line items, excluding cancelled orders, including customers with none.

```
SELECT c.full_name, coalesce(sum(t.line_total), 0) AS revenue
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id
        AND o.status <> 'cancelled'
LEFT   JOIN (SELECT order_id, sum(quantity * unit_price) AS line_total
              FROM   order_items GROUP BY order_id) t ON t.order_id = o.order_id
GROUP  BY c.customer_id, c.full_name
ORDER  BY revenue DESC;
```

Explanation — two non-obvious choices. The status filter is in `ON`, not `WHERE`: in `WHERE` it would discard the `NULL` row belonging to customers with no orders and silently convert the left join into an inner one. And `order_items` is aggregated to order grain *before* joining — joining it directly works for a single `sum`, but the moment you also want `count(o.order_id)` the order is repeated once per line and the count inflates.

Topic. Grain, `ON` versus `WHERE`.

L2.3 — WORKED

Problem. Order count by status, including statuses with zero orders.

```
SELECT s.status, count(o.order_id) AS orders
FROM   (VALUES ('pending'),('paid'),('shipped'),('delivered'),('cancelled'))
        AS s(status)
LEFT   JOIN orders o ON o.status = s.status
GROUP  BY s.status
ORDER  BY count(o.order_id) DESC, s.status;
```

Explanation. The zero rows cannot come from `orders` — a status with no orders has no row to group. So the *list of statuses* must be the driving table. This is precisely why a lookup table beats a `CHECK` for a status column: with `order_statuses` in the schema this joins a real table instead of a literal list.

Pattern. P18 gap filling, categorical form.

PROBLEMS 4–100 · TOPIC AND PATTERN IN BRACKETS

- 4 Every employee with their department name. [inner join]
- 5 The same, keeping employees with no department. [outer join]
- 6 Every department with its headcount, keeping empty ones. [outer join]
- 7 Departments with no employees, two ways. [P6]

- 8** Employees whose department is in Bengaluru. [join filter]
- 9** Average salary per department name. [join + group]
- 10** Departments whose average salary beats the company average. [derived table]
- 11** Each department's payroll and its share of the total. [window]
- 12** Every employee with their manager's name; the CEO must appear. [self-join]
- 13** Managers with a count of direct reports. [self-join + group]
- 14** Employees who manage nobody. [P6]
- 15** Employees whose manager is in another department. [self-join]
- 16** Pairs of employees in the same department, no duplicate pairs. [self-join]
- 17** Employees earning more than their manager. [self-join]
- 18** Every order with its customer's name and city. [join]
- 19** Orders from customers in Hyderabad. [join filter]
- 20** Customers with no orders, three ways, with trade-offs. [P6]
- 21** Customers with exactly one order. [**HAVING**]
- 22** Customers with more than one order in a single month. [group by month]
- 23** Each customer's first order date. [**min**]
- 24** Days between signup and first order. [join + date math]
- 25** Customers whose first order came over a year after signup. [**HAVING**]
- 26** Each customer's order count and total value. [group]
- 27** Customers spending above the average customer spend. [derived table]
- 28** The top three customers by revenue. [**LIMIT**]
- 29** Customer count and revenue per city. [group]
- 30** Cities with above-average revenue per customer. [**HAVING** + subquery]
- 31** Every order with its line count and computed total. [join + group]
- 32** Orders whose stored total disagrees with its lines. [P19]
- 33** Orders with no line items. [P6]
- 34** Orders containing a named product. [join filter]
- 35** Orders with more than two distinct products. [**HAVING**]
- 36** Every line with its product and order detail. [three-table join]
- 37** Each product's units sold and revenue. [join + group]
- 38** Products ordered exactly once. [**HAVING**]

- 39** Products never ordered, with `NOT EXISTS` . [P6]
- 40** The same with `NOT IN` ; explain why it works *here* and would not generally. [P6]
- 41** The top five products by revenue. [`LIMIT`]
- 42** Revenue per category. [join + group]
- 43** Categories above the average category revenue. [derived table]
- 44** Each category's share of total revenue. [window]
- 45** Products whose sale price ever differed from list price. [join comparison]
- 46** Average discount per product where sale price was lower. [expressions]
- 47** Customers who bought a specific product. [join]
- 48** Customers who bought A but never B. [P6]
- 49** Customers who bought from more than one category. [`HAVING`]
- 50** Products bought by more than one customer. [`count(DISTINCT)`]
- 51** Product pairs appearing on the same order. [self-join on lines]
- 52** The most frequently co-purchased pair. [self-join + group]
- 53** Revenue per month, excluding cancelled. [`date_trunc`]
- 54** Revenue per month per category. [multi-key group]
- 55** Order count and revenue by status and month. [multi-key group]
- 56** Statuses as columns, months as rows. [P20]
- 57** Revenue by month with a `ROLLUP` total. [`ROLLUP`]
- 58** Distinguish a `ROLLUP` subtotal from a real `NULL` . [`GROUPING`]
- 59** Revenue by month and category with `GROUPING SETS` . [grouping sets]
- 60** Every date in March 2024 with its order count, zeroes included. [P18]
- 61** Dates in a range with no orders at all. [P18 + P6]
- 62** Every month of 2024 with revenue, gap-filled. [P18]
- 63** Cities with both an office and a customer. [`INTERSECT`]
- 64** Cities with customers but no office. [`EXCEPT`]
- 65** A combined contact list with a source column. [`UNION ALL`]
- 66** Show why `UNION` and `UNION ALL` differ on that list. [set operations]
- 67** Flag each customer as being in an office city or not. [`EXISTS` in `SELECT`]
- 68** Employees in departments located where we have customers. [`IN` subquery]
- 69** Customers with at least one delivered order. [P9]

- 70** Customers with at least one cancelled order. [P9]
- 71** Customers with only cancelled orders. [`HAVING`]
- 72** Customers whose every order was delivered. [`HAVING` + `FILTER`]
- 73** Each customer's order count and delivered count side by side. [P20]
- 74** Each customer's largest single order value. [`max`]
- 75** Each customer's largest order with its date and id. [P1]
- 76** Each customer's most recent order. [P2]
- 77** Each customer's first and most recent order in one row. [double aggregate]
- 78** The highest-spending customer in each city. [P1]
- 79** The best-selling product in each category. [P1]
- 80** The highest-paid employee per department, ties included. [P1 with `rank`]
- 81** The second-highest salary, three ways. [P3]
- 82** The second-highest salary in each department. [P3 partitioned]
- 83** Employees above their own department average. [P4]
- 84** Each salary as a percentage of department payroll. [window]
- 85** Each employee with own, department and company average salary. [P4]
- 86** Insert a customer and their first order in one transaction. [transactions]
- 87** Insert an order and lines, computing the total from the lines. [`INSERT ... SELECT`]
- 88** Upsert a customer by email, updating city if present. [`ON CONFLICT`]
- 89** Update every order's total from its lines in one statement. [`UPDATE ... FROM`]
- 90** Delete a customer with orders; resolve the constraint error. [foreign keys]
- 91** Add soft deletes and a partial unique index on email. [constraints]
- 92** Prove a soft-deleted email can be reused. [partial index]
- 93** Create an active-customers view and query through it. [views]
- 94** Create an order-summary view; show it is not updatable. [views]
- 95** Add `WITH CHECK OPTION` and demonstrate a rejected insert. [views]
- 96** Add the four missing foreign-key indexes. [indexes]
- 97** Explain why `order_items.order_id` needs no separate index. [composite keys]
- 98** Find every foreign key with no supporting index. [catalog]
- 99** Write one query reporting every integrity violation you can think of. [P19]
- 100** Take any five Level 1 problems and rewrite them across two tables. [synthesis]

34.3 Levels 3–5

300 problems, each stated with its pattern — because at this level, knowing *which* pattern applies is most of the work. Three worked, one per level.

L3-1 — WORKED ● MEDIUM

Problem. For each customer: order sequence number, days since their previous order, and a first-order flag — in one pass.

```
SELECT c.full_name, o.order_date,
       row_number() OVER w          AS seq,
       o.order_date - lag(o.order_date) OVER w AS days_since_prev,
       lag(o.order_date) OVER w IS NULL      AS is_first
FROM   orders o
JOIN   customers c ON c.customer_id = o.customer_id
WHERE  o.status <> 'cancelled'
WINDOW w AS (PARTITION BY o.customer_id ORDER BY o.order_date, o.order_id)
ORDER BY c.full_name, o.order_date;
```

Explanation. Three window functions sharing one named window — worth doing not only for brevity but because a repeated `OVER (...)` is where definitions drift apart. The `order_id` tie-break makes the sequence deterministic when a customer orders twice in a day.

Pattern. P12, per entity.

L4.1 — WORKED ● **HARD**

Problem. A true rolling 7-day revenue average, correct on days with no sales.

```
WITH daily AS (  
  SELECT d.day::date AS day, coalesce(sum(o.total_amount), 0) AS revenue  
  FROM   generate_series(DATE '2024-01-01', DATE '2024-06-30',  
                        INTERVAL '1 day') AS d(day)  
  LEFT   JOIN orders o ON o.order_date = d.day AND o.status <> 'cancelled'  
  GROUP  BY d.day  
)  
SELECT day, revenue,  
       round(avg(revenue) OVER (ORDER BY day  
                               RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW), 2)  
       AS rolling_7d  
FROM   daily ORDER BY day;
```

Explanation — both halves are load-bearing. **Gap filling** means a day with no orders contributes a zero rather than being absent; without it, "7 days" means "the last seven days that happened to have sales", which on sparse data can span months. **RANGE ... INTERVAL** makes the window a genuine time span rather than a row count, so it stays correct if a day later gains several rows. Write it with **ROWS 6 PRECEDING** over a gap-filled series and it happens to be right; add a second row for one day and it silently becomes wrong.

Pattern. P11 + P18.

L5.1 — WORKED ● **EXPERT**

Problem. Each customer's longest streak of consecutive order days.

```
WITH days AS (  
    SELECT DISTINCT customer_id, order_date  
    FROM    orders WHERE status <> 'cancelled'  
),  
numbered AS (  
    SELECT customer_id, order_date,  
           row_number() OVER (PARTITION BY customer_id ORDER BY order_date) AS rn  
    FROM    days  
),  
islands AS (  
    SELECT customer_id, order_date - (rn || ' days')::interval AS island_key  
    FROM    numbered  
),  
runs AS (  
    SELECT customer_id, island_key, count(*) AS len  
    FROM    islands GROUP BY customer_id, island_key  
)  
SELECT c.full_name, max(r.len) AS longest_streak  
FROM    runs r JOIN customers c ON c.customer_id = r.customer_id  
GROUP BY c.customer_id, c.full_name  
ORDER BY longest_streak DESC, c.full_name;
```

Explanation. Four steps, each doing one thing. The invariant: consecutive dates and consecutive row numbers both increase by one, so their difference is constant within a streak and changes at every gap. `island_key` is a grouping handle with no meaning as a value, which is why it never appears in the output.

The `DISTINCT` is not cosmetic. Two orders on one date would give that date two row numbers, two island keys, and split a real streak in half — the detail that separates a working gaps-and-islands query from one that looks right on tidy data.

Common wrong approach. A self-join on `order_date = order_date + 1`, which finds adjacent pairs but cannot measure length without one join per day.

Pattern. P13.

LEVEL 3 · MEDIUM — 2-100

- 2** Top three products per category, ties at third included. [P1]
- 3** Each customer's most recent order, three ways, plans compared. [P2]
- 4** The Nth highest salary, parameterised and tie-correct. [P3]
- 5** Salary quartiles with each quartile's boundary values. [P5]

- 6** Median salary overall and per department. [P5]
- 7** p95 order value, using the form returning a real observation. [P5]
- 8** Employees ranked by salary with their percentile. [P5]
- 9** A running total of revenue by date. [P10]
- 10** Cumulative revenue with each day's share of the total. [P10]
- 11** An ABC analysis of customers by revenue. [P10]
- 12** A three-order moving average of order value. [P11]
- 13** Demonstrate `ROWS` versus `RANGE` on tied dates. [P11]
- 14** Month-over-month revenue growth, gap-filled. [P12]
- 15** The same without gap filling; explain the wrong figures. [P12]
- 16** Revenue this month against the same month last year. [P12]
- 17** Year-to-date revenue as at any given date. [P10]
- 18** Each order with previous and next order date for the same customer. [P12]
- 19** Each customer's longest gap between orders. [P12]
- 20** Customers inactive 90 days who ordered before. [P6]
- 21** Group orders into runs of consecutive same-status dates. [P14]
- 22** Show why `IS DISTINCT FROM` is needed in that flag. [P14]
- 23** Customers who ordered every product in one category. [P8]
- 24** Customers who ordered at least three from one category. [P8]
- 25** Customers ranked by how close they are to a whole category. [P8]
- 26** The double-negation form of 23; compare readability. [P8]
- 27** Find duplicate customers by normalised email. [P7]
- 28** Delete duplicates keeping the oldest, then add the preventing index. [P7]
- 29** Customers with the same name but different emails. [P7]
- 30** Reconcile `orders.total_amount` against its lines. [P19]
- 31** One query reporting six classes of integrity violation. [P19]
- 32** Statuses as columns, months as rows. [P20]
- 33** Unpivot products into (product, metric, value) rows. [P20]
- 34** Each order's lines as a comma-separated product list. [`string_agg`]
- 35** Each order as nested JSON, empty array when no lines. [`jsonb_agg`]
- 36** The full org chart with indentation and path. [recursive]

- 37** The management chain above one employee. [recursive]
- 38** Each manager's total reports at every level below. [closure]
- 39** Employees unreachable from any root, and why. [recursive + P6]
- 40** Create a cycle deliberately, then detect it. [cycle detection]
- 41** All dates in a month two ways; which would you ship? [`generate_series`]
- 42** A three-CTE chain: line totals, spend, revenue share. [CTEs]
- 43** `AS MATERIALIZED` on a twice-referenced CTE; compare plans. [CTEs]
- 44** Rewrite a correlated subquery as a window function. [P4]
- 45** Rewrite three select-list subqueries as one join. [grain]
- 46** The two most recent orders per customer with `LATERAL` . [P2]
- 47** The same with a window function; when does each win? [P1 vs P2]
- 48** New versus returning customers per month. [P16]
- 49** Repeat-purchase rate as one percentage. [P20]
- 50** A retention matrix by signup month and offset. [P16]
- 51** Why retention must divide by cohort size, with a wrong version. [P16]
- 52** A funnel from cart to delivered with per-stage drop-off. [P17]
- 53** Make that funnel order-sensitive using timestamps. [P17]
- 54** Sessionize orders with a 30-day inactivity boundary. [P15]
- 55** Report each session's duration and order count. [P15]
- 56** Customers whose spending declined three consecutive months. [P12]
- 57** Products whose sales declined two quarters running. [P12]
- 58** Stock cover in days from stock and 30-day average sales. [aggregates]
- 59** Products needing reorder net of unshipped commitments. [`FILTER`]
- 60** Revenue lost to cancellations per month and product. [`FILTER`]
- 61** Market basket: top five co-purchased pairs. [self-join]
- 62** Products bought by more than half of all customers. [subquery]
- 63** Each city's top product by revenue. [P1]
- 64** Each month's top customer by revenue. [P1]
- 65** Employees within 10% of the company average. [subquery]
- 66** Salary bands with counts and percentages. [P20]
- 67** Simulate a differential rise without updating anything. [`CASE`]

- 68** Add salary history as validity intervals with an overlap constraint. [design]
- 69** Query each salary as at a past date. [temporal]
- 70** Salary change history with the percentage step each time. [P12]
- 71** Add department history and query "as at" any date. [temporal]
- 72** Find overlapping validity spans that should not exist. [P19]
- 73** Merge adjacent and overlapping ranges into consolidated spans. [P14]
- 74** Per-day occupancy from admission and discharge dates. [P18]
- 75** Rewrite an unsargable date filter; prove the index is used. [sargability]
- 76** Rewrite `OFFSET 100000` as keyset with a cursor. [pagination]
- 77** Build the index making a keyset feed query index-only. [indexes]
- 78** A partial index for pending orders; state its write cost. [indexes]
- 79** An expression index fixing both plan and estimate. [indexes]
- 80** Eliminate a `Sort` node from a plan. [indexes]
- 81** Every unused index, excluding unique ones. [catalog]
- 82** Redundant indexes that are prefixes of others. [catalog]
- 83** Table sizes with dead-row percentages. [catalog]
- 84** Top ten queries by total execution time. [`pg_stat_statements`]
- 85** Which sessions are blocking which. [`pg_blocking_pids`]
- 86** A function returning a customer's lifetime value. [functions]
- 87** Declare its volatility correctly; show the plan difference. [volatility]
- 88** A set-returning function for a customer's orders. [functions]
- 89** A trigger maintaining `orders.total_amount` . [triggers]
- 90** Add a `WHEN` clause so it skips no-op updates. [triggers]
- 91** A generic audit trigger using `to_jsonb` . [triggers]
- 92** Capture the application user via `SET LOCAL` . [triggers]
- 93** Convert it to statement-level with transition tables. [triggers]
- 94** Store order metadata as `jsonb` and query inside it. [JSONB]
- 95** Index that key so the filter is sargable and well estimated. [JSONB]
- 96** Unnest a JSONB array of lines into rows. [JSONB]
- 97** Aggregate relational rows into a nested JSON document. [JSONB]
- 98** An exclusion constraint preventing overlapping department spans. [constraints]

- 99** A partial unique index for "one primary address per customer". [constraints]
- 100** A composite foreign key enforcing "manager in the same department". [constraints]

LEVEL 4 · HARD — 2-100

- 2** Each customer's longest streak of consecutive order months. [P13]
- 3** The longest run of days with no orders at all. [P13]
- 4** Every gap in a supposedly contiguous invoice sequence. [P13]
- 5** Sessionize events with a 30-minute timeout; report per-session metrics. [P15]
- 6** Cohort retention as a percentage matrix with cohort sizes. [P16]
- 7** Weekly rather than monthly cohorts; what changes? [P16]
- 8** Revenue retention rather than customer retention. [P16]
- 9** MRR movement split into new, expansion, contraction, churn. [P12 + P20]
- 10** Multi-touch attribution across a funnel. [P17]
- 11** Anomalous daily volume per account against its own baseline. [P4 + P18]
- 12** p50, p95, p99 latency by endpoint and hour. [P5 + P18]
- 13** A rolling 28-day active-user count. [P11]
- 14** Week-on-week change in a rolling average. [P11 + P12]
- 15** Explode a bill of materials, multiplying quantities down the tree. [recursive]
- 16** Breadcrumbs and depth for every leaf of a category tree. [recursive]
- 17** Detect cycles in a graph using an array path. [recursive]
- 18** Shortest path between two nodes in a small graph. [recursive]
- 19** Propagate a root attribute down a whole hierarchy. [recursive]
- 20** Compare recursion against a materialised path, with timings. [design]
- 21** Relational division with a dynamic required set. [P8]
- 22** "Has all of" where the required set is another customer's basket. [P8]
- 23** Seat assignments overlapping on the same showing. [P19]
- 24** Double-booked rooms via range overlap. [P19]
- 25** Design and prove the exclusion constraint that prevents them. [constraints]
- 26** Railway segment availability between two stations. [ranges]
- 27** Waitlist position and promotion order on cancellation. [P1 + P14]
- 28** Build a double-entry ledger; prove the zero-sum invariant. [P19]
- 29** A statement with a running balance from that ledger. [P10]

- 30** Reconcile a cached balance against the ledger. [P19]
- 31** Reverse a transfer with compensating entries, not edits. [design]
- 32** FIFO inventory valuation from a movements ledger. [P10 + P14]
- 33** Weighted-average valuation from the same ledger. [aggregates]
- 34** Stock on hand as at any historical date. [temporal]
- 35** Slow-moving stock: no outbound movement in 90 days. [P6]
- 36** An SCD Type 2 load that is idempotent when nothing changed. [design]
- 37** Query a fact table for "segment as at the time of sale". [star]
- 38** The same for "segment as it is today", via a Type 6 column. [star]
- 39** Build `dim_date` with fiscal year and holiday flags. [warehousing]
- 40** Load a star schema from the practice tables at line grain. [warehousing]
- 41** The canonical star query with `ROLLUP`. [warehousing]
- 42** An incremental fact load with a watermark and late-arrival window. [ETL]
- 43** Make it idempotent; prove a re-run changes nothing. [ETL]
- 44** Detect a fact row whose dimension key is missing. [P19]
- 45** Partition `orders` by month; prove pruning. [partitioning]
- 46** Show a query that fails to prune; explain why. [partitioning]
- 47** Detach and drop a partition; compare with `DELETE`. [partitioning]
- 48** Attach an existing table as a partition without a long lock. [partitioning]
- 49** Automate next month's partition; alert on a missing one. [operations]
- 50** Replace a materialized view with an incremental summary table. [design]
- 51** Refresh a matview concurrently; explain the unique-index requirement. [matviews]
- 52** A batched purge procedure committing inside a loop. [procedures]
- 53** A resumable backfill that survives being killed. [operations]
- 54** The six-step zero-downtime `NOT NULL` addition. [migrations]
- 55** A column type change with no downtime. [migrations]
- 56** An expand-and-contract column rename across two deploys. [migrations]
- 57** Add a foreign key to a huge table without a long scan. [migrations]
- 58** A deadlock-free transfer using consistent lock ordering. [concurrency]
- 59** Reproduce lost update, then fix it three ways. [concurrency]
- 60** Reproduce write skew, then fix it two ways without Serializable. [concurrency]

- 61** A guarded stock decrement that cannot go negative. [concurrency]
- 62** An idempotent webhook handler that cannot double-charge. [concurrency]
- 63** A job queue with `SKIP LOCKED`, retries and a reaper. [concurrency]
- 64** Enforce "at most N active per parent" unraceably. [constraints]
- 65** Enforce "exactly one current row per entity". [partial index]
- 66** A deferred unique constraint permitting a two-statement swap. [constraints]
- 67** Enforce tenant isolation with RLS; prove it from a non-owner. [security]
- 68** Demonstrate the owner-bypass trap and fix it. [security]
- 69** Demonstrate the `SET` versus `SET LOCAL` leak on a pooled connection. [security]
- 70** Column-level grants excluding a PII field; prove the exclusion. [security]
- 71** Build least privilege for web, reporting and migration roles. [security]
- 72** Safe dynamic SQL with `format('%I')`; show what `%s` allows. [security]
- 73** Find every table without a primary key. [catalog]
- 74** Find every table with RLS disabled. [catalog]
- 75** Estimate bloat for every table and rank by wasted bytes. [catalog]
- 76** Compare two tables and report only rows that differ. [set operations]
- 77** Validate a relationship before adding its foreign key. [P19]
- 78** Deduplicate a table in place keeping the newest per group. [P7]
- 79** A batched delete that holds no long locks. [operations]
- 80** Diagnose a nested-loop misestimate and fix it with statistics. [performance]
- 81** Create extended statistics on correlated columns; show the improvement. [performance]
- 82** Sabotage statistics deliberately; observe the plan flip. [performance]
- 83** Prove an idle transaction blocks vacuum. [operations]
- 84** Produce and then eliminate an external merge sort. [performance]
- 85** Produce a bitmap heap scan and an index-only scan deliberately. [performance]
- 86** Show a cold-cache versus warm-cache measurement of one query. [performance]
- 87** Take one query from 800 ms to under 50 ms; document each step. [performance]
- 88** Quantify an index's write cost against its read benefit. [performance]
- 89** Demonstrate HOT updates being defeated by a new index. [performance]
- 90** Build a trigram index and show `ILIKE '%x%'` becoming fast. [indexes]
- 91** Build a BRIN index on an append-only table; compare sizes. [indexes]

- 92** Full-text search with ranking and highlighting. [search]
- 93** Combine full-text with structured filters in one plan. [search]
- 94** Design search where the vocabulary is closed; justify relational over FTS. [design]
- 95** Nested JSON API response for a customer and their orders, in one query. [JSONB]
- 96** Validate a JSONB document against required keys. [JSONB]
- 97** Migrate a JSONB field into a real column with no downtime. [migrations]
- 98** Per-tenant custom fields with a definitions table. [design]
- 99** Model prices varying by customer, quantity and date. [design]
- 100** Take five Level 3 problems and make each 10× larger; re-measure. [synthesis]

LEVEL 5 · EXPERT — 2-100

- 2** Longest streak with a tolerance of one missed day. [P13 variant]
- 3** Streaks that survive weekends and holidays. [P13 + `dim_date`]
- 4** Merge overlapping ranges across three tables into one timeline. [P14]
- 5** Detect the exact moment concurrent capacity was exceeded. [P14 + ranges]
- 6** Compute peak concurrent sessions per hour from start/end times. [P14]
- 7** Sessionize with a per-user adaptive timeout. [P15]
- 8** Path analysis: the three most common event sequences per session. [P15]
- 9** First-touch and last-touch attribution in one query. [P17]
- 10** Time-decayed multi-touch attribution. [P17]
- 11** Retention with a resurrection state: churned then returned. [P16]
- 12** Net revenue retention per cohort per month. [P16]
- 13** Predict next-order date per customer from their own cadence. [P12 + P5]
- 14** Detect a level shift in a daily series, not just an outlier. [P11 + P4]
- 15** Seasonal comparison: this week against the same week's median over 3 years. [P5]
- 16** A full RFM segmentation in one query. [P5 + P20]
- 17** Basket affinity with lift and confidence, not just counts. [self-join + aggregates]
- 18** Recommend products from co-purchase, excluding already-owned. [P6 + P8]
- 19** Longest path through a dependency graph. [recursive]
- 20** Topologically order a dependency graph, detecting cycles. [recursive]
- 21** Critical-path duration through a project network. [recursive]
- 22** Transitive closure with edge weights summed along paths. [recursive]

- 23** Connected components in an undirected graph. [recursive]
- 24** Mutual-follow clusters in a social graph. [recursive + self-join]
- 25** Bill of materials with substitutions and effective dates. [recursive + temporal]
- 26** Explode a BOM and cost it from a price-history table. [recursive + temporal]
- 27** Bi-temporal query: what we believed on date X about date Y. [temporal]
- 28** Reconstruct an entity's full state at any past instant from an event log. [P10]
- 29** Detect an event log whose replay disagrees with current state. [P19]
- 30** Build a snapshot table from an event log, incrementally. [P10 + ETL]
- 31** A ledger supporting multi-currency with rates fixed at posting. [design]
- 32** Prove multi-currency zero-sum per currency per transfer. [P19]
- 33** Interest accrual by day from a balance ledger. [P10 + P18]
- 34** Amortisation schedule generated in SQL. [recursive]
- 35** Allocate a payment across the oldest outstanding invoices. [P10]
- 36** Aged-debt bucketing with partial payments. [P10 + P20]
- 37** FIFO cost of goods sold for a specific shipment. [P10 + P14]
- 38** Reserve stock across warehouses minimising split shipments. [P10]
- 39** Railway availability with waitlist promotion in one transaction. [ranges]
- 40** Seat allocation preferring adjacent seats. [P14 on seat numbers]
- 41** Shift rostering that guarantees minimum cover. [ranges + P14]
- 42** Detect a rota that violates minimum cover, before it is saved. [constraints]
- 43** Timetable clash detection across rooms, staff and cohorts. [ranges]
- 44** Appointment slot availability from working hours minus bookings. [ranges]
- 45** Compute SLA breach minutes excluding non-business hours. [P18 + ranges]
- 46** Business-days-between, honouring a holiday calendar. [`dim_date`]
- 47** A leaderboard with dense ranking, ties and stable pagination. [P1 + pagination]
- 48** Keyset pagination over a computed ranking. [pagination]
- 49** Pagination stable under concurrent inserts. [pagination]
- 50** An approximate total for a filtered list, and when it is acceptable. [performance]
- 51** A feed query for a user following 5,000 accounts. [design + indexes]
- 52** Compare fan-out on write against fan-out on read, with numbers. [design]
- 53** Counter caches for a viral row without serialising writers. [concurrency]

- 54** Sharded counters aggregated on read. [design]
- 55** Rate limiting in the database; state its failure modes. [concurrency]
- 56** A distributed lock with advisory locks, and its dangers. [concurrency]
- 57** Exactly-once processing across a queue and a database. [design]
- 58** The outbox pattern implemented end to end. [design]
- 59** A saga with compensating transactions across three tables. [design]
- 60** Optimistic locking with a version column and conflict reporting. [concurrency]
- 61** Reproduce all five isolation anomalies and defeat each. [concurrency]
- 62** Measure the throughput cost of Serializable under contention. [performance]
- 63** Write the retry wrapper, then prove it under induced conflicts. [concurrency]
- 64** Partition a 500 GB table with no downtime. [migrations]
- 65** Convert a table to partitioned by attaching the original. [migrations]
- 66** Sub-partition by tenant within monthly ranges; justify it. [partitioning]
- 67** Repack a bloated table online; state the risks. [operations]
- 68** Design a retention policy honouring deletion requests *and* backups. [design]
- 69** A logical-replication major-version upgrade plan. [operations]
- 70** Detect and resolve a stuck replication slot. [operations]
- 71** A read-your-own-writes strategy across replicas using LSN. [design]
- 72** Route queries by workload, with per-role timeouts. [design]
- 73** Capacity-forecast disk and IOPS from catalog statistics. [operations]
- 74** Estimate the write amplification of the current index set. [performance]
- 75** Justify dropping four indexes with numbers, not opinions. [performance]
- 76** Design an index set for a workload of twelve query shapes. [performance]
- 77** Rewrite three unsargable production-style queries. [sargability]
- 78** Diagnose a plan that regressed after `ANALYZE`. [performance]
- 79** Fix a query fast in `psql` and slow from the application. [performance]
- 80** Find and fix an N+1 from `pg_stat_statements` alone. [performance]
- 81** Turn 40 dashboard queries into one summary lookup. [design]
- 82** Make a 9-hour warehouse load finish in 3. [ETL]
- 83** Design ETL idempotency and prove it by double-running. [ETL]
- 84** Handle a late-arriving dimension row during a fact load. [ETL]

- 85** Semi-additive measures: month-end balances in a fact table. [warehousing]
- 86** A factless fact table answering "who was eligible but did not act". [warehousing]
- 87** Bridge table for a many-valued dimension without double-counting. [warehousing]
- 88** Reproduce a two-year-old invoice exactly from versioned prices. [design]
- 89** Multi-currency reporting where minor units differ per currency. [design]
- 90** Model tax rates that change and must not retroactively alter invoices. [design]
- 91** Audit trail proving who saw a record, not only who changed it. [security]
- 92** Design masking for a production restore into staging. [security]
- 93** Detect an injection attempt from query logs. [security]
- 94** Find every place identifiers are interpolated in a codebase, and fix them. [security]
- 95** Migrate an application off superuser without downtime. [security]
- 96** Write the schema-change review checklist you would enforce. [process]
- 97** Write the monitoring you would want before a hardware migration. [process]
- 98** Make an unfamiliar production database safe in one week. [process]
- 99** Review a colleague's 60-line query and give three specific improvements. [process]
- 100** Design, build, load and optimise a database of your own choosing end to end. [capstone]

IF A PROBLEM STOPS YOU

Look up the bracketed pattern in Part 35, or the topic in the volume that owns it: joins and windows in III, design and constraints in IV, indexes and plans in V, security and warehousing in VI. Every one of these 500 problems is solvable with material already in the book — none needs a feature it has not taught.

Debugging SQL

A syntax error tells you where it is. A logic error does not — it returns a plausible number and waits six months. This part is about the second kind: how to find it, and the twelve shapes it takes.

36.1 The method

Wrong-answer bugs are found by **bisection on the query**, not by staring at it. Six steps, in order.

#	Step	Why it works
1	Count rows at every stage. Strip the <code>SELECT</code> list to <code>count(*)</code> and remove one clause at a time from the bottom.	A row count that changes when it should not localises the fault to one join or filter
2	Remove the aggregates. Look at the raw joined rows for one known entity.	Aggregates hide duplication; raw rows show it immediately
3	Name the grain. Say aloud what one row of the result means.	Most wrong totals are an aggregate applied at the wrong grain
4	Check one entity by hand. Pick a customer, compute the answer manually, compare.	Six-row practice tables exist for exactly this
5	Hunt the <code>NULL</code> s. <code>count(*)</code> against <code>count(col)</code> on every column in play.	<code>NULL</code> s change joins, filters, aggregates and <code>NOT IN</code> — silently
6	Write the invariant. A query that returns zero rows when the data is correct.	Turns a one-off investigation into permanent monitoring

THE SINGLE MOST USEFUL DIAGNOSTIC

`SELECT count(*) FROM a JOIN b ...;` against `SELECT count(*) FROM a;` . If the join changed the count, every aggregate above it is suspect. Run this before anything else — it is five seconds and it answers the most common question.

36.2 Find the bug — twelve exercises

Each query below is syntactically valid and returns wrong results. Find the fault before reading the diagnosis.

BUG 1 · THE DOUBLED TOTAL

```
SELECT c.full_name, sum(o.total_amount) AS revenue, count(*) AS orders
FROM   customers c
JOIN   orders o      ON o.customer_id = c.customer_id
JOIN   order_items oi ON oi.order_id  = o.order_id
GROUP BY c.customer_id, c.full_name;
```

Diagnosis. `order_items` is joined but never used. Its only effect is to repeat each order once per line, so `revenue` is multiplied by the line count and `orders` counts lines rather than orders. Fan-out.

Fix. Drop the unused join. If line detail is genuinely needed, aggregate `order_items` to order grain in a derived table first, and use `count(DISTINCT o.order_id)`.

Tell. A joined table that appears in no expression. Search every query for tables named only in the `FROM`.

BUG 2 · THE MISSING CUSTOMERS

```
SELECT c.full_name, count(o.order_id) AS orders
FROM   customers c
LEFT   JOIN orders o ON o.customer_id = c.customer_id
WHERE  o.status <> 'cancelled'
GROUP BY c.customer_id, c.full_name;
```

Diagnosis. The filter is in `WHERE`. For a customer with no orders the left join produces `status = NULL`, and `NULL <> 'cancelled'` is unknown — so the row is discarded and the left join silently becomes an inner one.

Fix. Move it into the join: `LEFT JOIN orders o ON o.customer_id = c.customer_id AND o.status <> 'cancelled'`.

Tell. Any `WHERE` predicate on the nullable side of an outer join.

BUG 3 · THE EMPTY REPORT

```
SELECT p.product_name FROM products p
WHERE p.product_id NOT IN (SELECT product_id FROM discontinued_items);
```

Diagnosis. One `NULL` in `discontinued_items.product_id` makes the predicate unable to be true for any row, so the query returns nothing. It works in test because the fixtures are tidier than production.

Fix. `NOT EXISTS` . Always.

Tell. The report went from many rows to exactly zero, with no error. That signature is `NOT IN` almost every time.

BUG 4 · THE CUMULATIVE AVERAGE

```
SELECT full_name, department_id, salary,
       avg(salary) OVER (PARTITION BY department_id ORDER BY hire_date) AS dept_avg
FROM   employees;
```

Diagnosis. Adding `ORDER BY` to an aggregate's `OVER` clause changes the default frame from the whole partition to `UNBOUNDED PRECEDING TO CURRENT ROW` . The column is a running average of everyone hired so far, not the department average — plausible on every row, correct only on the last.

Fix. Remove the `ORDER BY` , or add an explicit `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` .

Tell. A column labelled as a group aggregate that differs row to row within the group.

BUG 5 · THE LAST VALUE THAT ISN'T

```
SELECT department_id, full_name, salary,
       last_value(full_name) OVER (PARTITION BY department_id ORDER BY salary DESC)
       AS lowest_paid
FROM   employees;
```

Diagnosis. Same frame default. `last_value` sees a frame ending at the current row, so it returns the current row's own name.

Fix. Give it `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` , or reverse the sort and use `first_value` .

BUG 6 · THE NONSENSE GROWTH RATE

```
WITH monthly AS (  
    SELECT date_trunc('month', order_date) AS m, sum(total_amount) AS revenue  
    FROM orders GROUP BY 1)  
SELECT m, revenue,  
    round(100.0 * (revenue - lag(revenue) OVER (ORDER BY m))  
        / lag(revenue) OVER (ORDER BY m), 1) AS growth_pct  
FROM monthly;
```

Diagnosis. Two faults. `lag` takes the previous *row*, not the previous month — if April has no orders, May compares against March and the figure is meaningless. And a zero previous month divides by zero.

Fix. Gap-fill the month series with `generate_series`, and wrap the divisor in `nullif(..., 0)`.

Tell. Any period-over-period comparison built on rows rather than on a complete calendar.

BUG 7 · THE SPLIT STREAK

```
WITH numbered AS (  
    SELECT customer_id, order_date,  
        row_number() OVER (PARTITION BY customer_id ORDER BY order_date) AS rn  
    FROM orders)  
SELECT customer_id, count(*) AS streak  
FROM numbered  
GROUP BY customer_id, order_date - (rn || ' days')::interval;
```

Diagnosis. No `DISTINCT` on the day. Two orders on one date get two row numbers, two different island keys, and a genuine streak is cut in half.

Fix. Deduplicate to one row per customer-day before numbering.

BUG 8 · THE WRONG SECOND-HIGHEST

```
SELECT full_name, salary FROM employees  
ORDER BY salary DESC OFFSET 1 LIMIT 1;
```

Diagnosis. If two people share the top salary, this returns the *other top earner* as the second-highest. It answers "the second row" when the question was "the second distinct value".

Fix. `dense_rank() OVER (ORDER BY salary DESC)` filtered to 2.

BUG 9 · THE SHUFFLING PAGE

```
SELECT * FROM orders ORDER BY order_date DESC LIMIT 20 OFFSET 40;
```

Diagnosis. The sort is not unique. Orders sharing a date can appear in either order, so rows drift between pages — some seen twice, some never. The same fault makes `row_number()` non-deterministic.

Fix. Add the primary key as a tie-break, and prefer keyset pagination.

BUG 10 · THE LOST DAY

```
SELECT count(*) FROM events
WHERE occurred_at BETWEEN '2026-09-01' AND '2026-09-30';
```

Diagnosis. `occurred_at` is a timestamp. `BETWEEN` is inclusive, and the upper bound becomes midnight — so everything after 00:00:00 on the 30th is excluded. Roughly a day of data, missing quietly.

Fix. The half-open form: `>= '2026-09-01' AND < '2026-10-01'`. Never `BETWEEN` on a timestamp.

BUG 11 · THE AVERAGE THAT LIES

```
SELECT avg(rating) AS avg_rating FROM reviews WHERE product_id = 7;
```

Diagnosis. `avg` ignores `NULL`s, so unrated reviews are excluded from the divisor rather than counted as zero — and if every rating is `NULL` the result is `NULL`, not 0, which then propagates through any arithmetic downstream.

Fix. Decide deliberately: `avg(coalesce(rating, 0))` if unrated means zero, or report `count(rating)` beside the average so the reader can judge. Wrap in `coalesce(avg(...), 0)` if a number is required.

BUG 12 · THE VANISHING CEO

```
SELECT e.full_name AS employee, m.full_name AS manager
FROM employees e JOIN employees m ON m.employee_id = e.manager_id;
```

Diagnosis. An inner self-join drops every employee whose `manager_id` is `NULL` — the CEO, and anyone whose manager is not yet recorded. Headcount comes out one or more short.

Fix. `LEFT JOIN`.

Tell. Any self-join on a nullable foreign key.

THE PATTERN BEHIND ALL TWELVE

Nine of these twelve are one of three root causes: a **NULL behaving as unknown** (2, 3, 11, 12), a **grain mismatch** (1, 7), or a **default that is not what you assumed** (4, 5, 6, 9, 10). Check those three before anything else and you will find most wrong-answer bugs in minutes.

Dialect Comparison

Roughly 80% of the SQL in this book is portable unchanged. The other 20% is where interviews and migrations go wrong. This is that 20%, tabulated.

37.1 The comparison tables

PAGINATION

Engine	Syntax
PostgreSQL	<code>SELECT ... ORDER BY x LIMIT 10 OFFSET 20</code>
MySQL	Same, plus <code>LIMIT 20, 10</code> (offset first — a common trap)
SQL Server	<code>SELECT TOP 10 ...</code> , or <code>ORDER BY x OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY</code> (<code>ORDER BY</code> required)
Oracle	<code>OFFSET 20 ROWS FETCH FIRST 10 ROWS ONLY</code> (12c+); older: <code>ROWNUM</code> in a subquery

STRINGS

Need	PostgreSQL	MySQL	SQL Server	Oracle
Concatenate	<code> </code> , <code>concat</code>	<code>concat (</code> is OR)	<code>+</code> , <code>concat</code>	<code> </code>
Length	<code>length</code>	<code>char_length</code>	<code>len</code>	<code>length</code>
Substring	<code>substring(s from 2</code> for 3)	<code>substring(s,2,3)</code>	<code>substring(s,2,3)</code>	<code>substr(s,2,3)</code>
Position	<code>position(x in s)</code>	<code>locate</code>	<code>charindex</code>	<code>instr</code>
Case-insensitive like	<code>ILIKE</code>	<code>LIKE</code> (default collation)	<code>LIKE</code> (collation)	<code>upper()</code> both sides
Split	<code>split_part</code>	<code>substring_index</code>	<code>string_split</code>	<code>regexp_substr</code>
Aggregate to string	<code>string_agg</code>	<code>group_concat</code>	<code>string_agg</code>	<code>listagg</code>

DATES

Need	PostgreSQL	MySQL	SQL Server	Oracle
Now	<code>now()</code> , <code>current_date</code>	<code>now()</code> , <code>curdate()</code>	<code>sysdatetime()</code> , <code>getdate()</code>	<code>systimestamp</code> , <code>sysdate</code>
Add a month	<code>d + INTERVAL '1 month'</code>	<code>date_add(d, INTERVAL 1 MONTH)</code>	<code>dateadd(month,1,d)</code>	<code>add_months(d,1)</code>
Difference	<code>d2 - d1</code> (days)	<code>datediff(d2,d1)</code>	<code>datediff(day,d1,d2)</code>	<code>d2 - d1</code>
Truncate to month	<code>date_trunc('month',d)</code>	<code>date_format(d, '%Y-%m-01')</code>	<code>datetrunc(month,d)</code> (2022+)	<code>trunc(d, 'MM')</code>
Extract part	<code>extract(year FROM d)</code>	<code>year(d)</code>	<code>datepart(year,d)</code>	<code>extract(year FROM d)</code>
Format	<code>to_char</code>	<code>date_format</code>	<code>format</code> , <code>convert</code>	<code>to_char</code>

EVERYTHING ELSE

Feature	PostgreSQL	MySQL 8	SQL Server	Oracle
Auto id	GENERATED ... AS IDENTITY , serial	AUTO_INCREMENT	IDENTITY(1,1)	GENERATED AS IDENTITY (12c+)
NULL fallback	coalesce	ifnull , coalesce	isnull , coalesce	nvl , coalesce
Boolean	Native	tinyint(1)	bit	None in SQL until 23c
Upsert	ON CONFLICT DO UPDATE	ON DUPLICATE KEY UPDATE	MERGE	MERGE
Set difference	EXCEPT	EXCEPT (8.0.31+)	EXCEPT	MINUS
Window functions	Full	8.0+	2012+	Full
CTEs / recursive	Full	8.0+	2005+	11g+
JSON	jsonb + GIN	json	json functions	json (12c+)
Lateral	LATERAL	8.0.14+	CROSS/OUTER APPLY	12c+
Procedures	PL/pgSQL	SQL/PSM	T-SQL	PL/SQL
Partial index	Yes	No	Filtered index	Function-based workaround
Exclusion constraint	Yes	No	No	No
Transactional DDL	Yes	No	Yes	No (implicit commit)
Default isolation	Read Committed	Repeatable Read	Read Committed (locking)	Read Committed
Empty string	Distinct from NULL	Distinct	Distinct	Equals NULL
FROM -less select	Allowed	Allowed	Allowed	FROM dual (pre-23c)

PORTABLE VERSUS SPECIFIC

Portable: `SELECT` , joins, `GROUP BY / HAVING` , subqueries, `EXISTS` , CTEs and recursive CTEs, window functions, `CASE` , `COALESCE` , `UNION / INTERSECT` , constraints, transactions, `ORDER BY` . **Specific:** pagination, string and date functions, auto-increment, boolean, upsert, JSON, procedures, index types, and the isolation default. Write the portable core by hand and isolate the rest — in application code, behind one query builder or one file of dialect helpers.

THE FOUR THAT BITE HARDEST IN A MIGRATION

Oracle's empty string is `NULL` — a `NOT NULL` column accepts `''` nowhere else and fails there.

MySQL defaults to Repeatable Read, so the same code sees different concurrency behaviour.

MySQL and Oracle have no transactional DDL, so a failed migration leaves a half-applied schema.

No engine but PostgreSQL has exclusion constraints, so every non-overlap rule becomes application code plus explicit locking.

Best Practices

Twenty rules, each as bad SQL beside good SQL. Not style preferences
— every one of these has a failure mode behind it.

38.1 Writing queries

BAD

```
select * from orders o, customers c
where o.customer_id=c.customer_id
and o.status!='cancelled'
```

GOOD

```
SELECT o.order_id, o.order_date, c.full_name
FROM   orders o
JOIN   customers c ON c.customer_id
      = o.customer_id
WHERE  o.status <> 'cancelled'
```

Four rules at once. Explicit `JOIN` rather than comma-join, so the join condition cannot be forgotten and turn into a cross product. Named columns rather than `SELECT *`. Uppercase keywords and aligned clauses, so the shape is readable at a glance. And `<>` rather than `!=` — both work in every engine, one is the standard.

Rule		Failure it prevents
1	Name every column in the <code>SELECT</code> list	A new column silently changes results, breaks index-only scans, drags TOAST
2	Alias every table, consistently and meaningfully	Ambiguous columns; unreadable multi-table queries
3	Qualify every column with its alias	A new column on either table makes the query ambiguous and it stops compiling
4	Use half-open date ranges, never <code>BETWEEN</code> on timestamps	A silently missing day (Bug 10)
5	<code>NOT EXISTS</code> , never <code>NOT IN</code> against a subquery	An empty report from one <code>NULL</code>
6	Make every <code>ORDER BY</code> unique for pagination or ranking	Shuffling pages; non-deterministic <code>row_number()</code>
7	Keep predicates sargable — no functions on indexed columns	A full scan on an indexed table
8	Name CTEs for what they contain, not <code>t1</code> , <code>step2</code>	Unreviewable queries
9	State the grain in a comment on any query with two aggregates	The next person aggregating at the wrong level
10	<code>UNION ALL</code> unless you need deduplication	A needless sort of the whole result

38.2 Schema and operations

BAD

GOOD

```
CREATE TABLE Orders (
  ID int,
  Cust varchar(50),
  Total float,
  Date varchar(20),
  Status varchar(20)
);
```

```
CREATE TABLE orders (
  order_id      bigint GENERATED ALWAYS
  AS IDENTITY
              PRIMARY KEY,
  customer_id   bigint NOT NULL
              REFERENCES
customers (customer_id),
  total_minor   bigint NOT NULL
              CONSTRAINT
orders_total_non_negative
              CHECK (total_minor >= 0),
  placed_at     timestamptz NOT NULL
  DEFAULT now(),
  status        text NOT NULL
              REFERENCES
order_statuses (status)
);
CREATE INDEX idx_orders_customer
  ON orders (customer_id);
```

Eight faults in six lines. No primary key; a reserved word (`Date`) as a column name; mixed case forcing quotes; a date stored as text so it cannot be compared or indexed correctly; money as `float` , which cannot represent 0.01; no `NOT NULL` anywhere; no foreign key, so orphans are possible; no constraint name, so errors are unreadable; and no index on the referencing column, so every join and every parent delete scans.

Rule		Failure it prevents
11	<code>snake_case</code> , plural tables, singular columns, <code><entity>_id</code> keys	Quoting, case-sensitivity errors, guesswork
12	Money as <code>numeric</code> or minor-unit <code>bigint</code> ; times as <code>timestampz</code>	Rounding loss; timezone bugs
13	Name every constraint explicitly	Unreadable error messages during incidents
14	Index every foreign key you join on or cascade through	Minutes-long parent deletes; scanning joins
15	Snapshot agreed values — price, tax, address — onto the transaction	History rewriting itself when a record is edited
16	Enforce rules in the database, not only in the application	The 2 a.m. script that bypasses every check
17	Keep transactions short; never do I/O inside one	Lock queues; a third party's latency becoming your outage
18	Parameterise values; whitelist identifiers; concatenate nothing	SQL injection, including second-order
19	Migrate with <code>CONCURRENTLY</code> , <code>NOT VALID</code> , <code>lock_timeout</code> , expand-and-contract	A one-line change taking the site down
20	Every denormalization gets a maintenance mechanism <i>and</i> a reconciliation query	Silent drift nobody is watching

IF YOU KEEP ONLY ONE

Rule 20. Every other rule prevents a bug you will eventually see and fix. Rule 20 prevents the bug you will never see — a number that has been quietly wrong for two years, in a report someone made a decision on.

Cheat Sheets

Everything worth keeping open while you work, in the fewest pages that will hold it.

39.1 Query anatomy

WRITTEN ORDER

```
SELECT  DISTINCT cols
FROM    table
JOIN    other ON cond
WHERE   row_filter
GROUP BY cols
HAVING  group_filter
ORDER BY cols
LIMIT n OFFSET m
```

EVALUATION ORDER

```
1 FROM      pick tables
2 JOIN      combine
3 WHERE     filter rows
4 GROUP BY  form groups
5 HAVING    filter groups
6 SELECT    compute columns
7 DISTINCT  dedupe
8 ORDER BY  sort
9 LIMIT     slice
           (windows run between 5 and 6)
```

JOINS, AGGREGATES, WINDOWS

```
-- JOINS
INNER JOIN  matching rows both sides      LEFT JOIN   all left + matches
RIGHT JOIN  all right + matches           FULL JOIN   everything both sides
CROSS JOIN  every combination             self-join   two aliases, one table
-- filter on the nullable side goes in ON, not WHERE

-- AGGREGATES                                -- with a filter
count(*) count(col) sum avg min max        count(*) FILTER (WHERE cond)
count(DISTINCT col)                        sum(x) FILTER (WHERE cond)
string_agg(col, ', ' ORDER BY col)         jsonb_agg(...) FILTER (...)
percentile_cont(0.5) WITHIN GROUP (ORDER BY x) -- median, interpolated
percentile_disc(0.95) WITHIN GROUP (ORDER BY x) -- p95, a real value

-- WINDOWS
fn() OVER (PARTITION BY g ORDER BY o frame)
row_number 1,2,3,4  rank 1,2,2,4  dense_rank 1,2,2,3  ntile(n)
lag(x, n, default) lead(x) first_value(x) last_value(x) nth_value(x, n)
-- FRAMES: no ORDER BY -> whole partition; with ORDER BY -> cumulative!
ROWS UNBOUNDED PRECEDING                -- running total
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW -- 3-row trailing
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING -- whole partition
RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW -- true 7 days
```

SUBQUERIES, CTES, SETS

```

-- Need columns from the other table? JOIN    (check fan-out)
-- Only need "does a row exist"?      EXISTS (NULL-safe, no fan-out)
-- Short literal list?                IN      (never NOT IN on a subquery)

WITH step_one AS (...), step_two AS (SELECT ... FROM step_one)
SELECT ... FROM step_two;                -- name CTEs as nouns

WITH RECURSIVE t AS (
    SELECT ... anchor ...
    UNION ALL
    SELECT ... FROM t JOIN ... WHERE depth < 50    -- always guard
) SELECT * FROM t ORDER BY path;          -- path, not depth

UNION ALL (keep dupes, fast) · UNION · INTERSECT · EXCEPT
DISTINCT ON (cols) ... ORDER BY cols, tiebreak    -- PostgreSQL only

```

DDL, CONSTRAINTS, INDEXES, TRANSACTIONS

```

CREATE TABLE t (
    id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    parent  bigint NOT NULL REFERENCES p (id) ON DELETE CASCADE,
    amount  bigint NOT NULL CONSTRAINT t_amount_pos CHECK (amount > 0),
    created timestamptz NOT NULL DEFAULT now(),
    UNIQUE (parent, amount)
);
ALTER TABLE t ADD COLUMN c text;                -- instant
ALTER TABLE t ADD CONSTRAINT n CHECK (...) NOT VALID; -- then VALIDATE
ALTER TABLE t DROP CONSTRAINT n;

CREATE INDEX CONCURRENTLY i ON t (a, b DESC) INCLUDE (c) WHERE active;
-- equality cols first, range/order last; leftmost prefix only
-- gin for jsonb/arrays/trigram · gist for ranges · brin for append-only

BEGIN; ... SAVEPOINT s; ... ROLLBACK TO s; ... COMMIT;
SELECT ... FOR UPDATE [SKIP LOCKED | NOWAIT];
BEGIN ISOLATION LEVEL REPEATABLE READ | SERIALIZABLE;    -- retry 40001/40P01
UPDATE t SET n = n + 1 WHERE id = $1;                    -- never read-modify-write

EXPLAIN (ANALYZE, BUFFERS) SELECT ...;    -- estimated vs actual rows first
ANALYZE t;  VACUUM t;  REINDEX CONCURRENTLY INDEX i;

```

FUNCTIONS WORTH REMEMBERING

```

-- STRING          -- NUMERIC          -- NULL
concat_ws(',',a,b)  round(x, 2)          coalesce(a, b, 0)
lower upper trim    ceil floor abs    nullif(a, '')
length substring    mod(a,b) power    x IS [NOT] NULL
replace position    random()          a IS DISTINCT FROM b
split_part(s,'@',2) greatest least    -- treats NULL as a value

-- DATE / TIME
current_date now()  date_trunc('month', d) extract(year FROM d)
d + INTERVAL '3 days' age(a, b)  to_char(d, 'DD Mon YYYY')
generate_series(start, stop, INTERVAL '1 day') -- gap filling
-- always: timestampz for instants, half-open ranges for filters

-- JSONB
p -> 'k'          JSON value (chain)    p @> '{"k":1}'    contains
p ->> 'k'         text (COMPARE with)    p ? 'k'          key exists
p #>> '{a,b}' text by path              jsonb_array_elements(p -> 'arr')
jsonb_build_object(...) jsonb_agg(... ORDER BY ...) FILTER (WHERE ...)

```

CATALOG AND DIAGNOSTICS

```

\d t \dt \di \dn \df \l \x \timing \i file.sql \copy

-- top queries by TOTAL time (not mean)
SELECT calls, round(total_exec_time::numeric,1) AS ms, left(query,80)
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 15;

-- who blocks whom
SELECT pid, wait_event, left(query,60) FROM pg_stat_activity
WHERE cardinality(pg_blocking_pids(pid)) > 0;

-- bloat and stale stats
SELECT relname, n_live_tup, n_dead_tup, last_analyze FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;

-- unused indexes (never drop unique ones)
SELECT indexrelname, idx_scan, pg_size_pretty(pg_relation_size(indexrelid))
FROM pg_stat_user_indexes WHERE idx_scan = 0;

-- long transactions: the vacuum blocker
SELECT pid, state, age(clock_timestamp(), xact_start) FROM pg_stat_activity
WHERE xact_start IS NOT NULL ORDER BY xact_start;

```


THE ONE-LINE DIALECT CARD

Pagination: PG/MySQL `LIMIT n OFFSET m` · SQL Server `OFFSET ... FETCH NEXT` · Oracle `FETCH FIRST` . **Concat:** `||` except MySQL `concat()` and SQL Server `+` . **Now:** `now()` / `getdate()` / `sysdate` . **Null fallback:** `coalesce` everywhere; `isnull` / `nvl` locally. **Upsert:** `ON CONFLICT` / `ON DUPLICATE KEY` / `MERGE` . Full tables in Part 37.

Mastery Roadmap

Plans, a glossary, and the capstone. Read the plan that matches your deadline, ignore the others.

40.1 The plans

30 DAYS · EMPLOYABLE

Days	Do	Volume
1–3	Fundamentals and setup. Load the practice schema. Learn the evaluation order cold.	I
4–10	Basics, types, operators, filtering, functions, aggregation. Every Level 1 problem.	II · 34.1
11–18	Joins to fluency, then subqueries and CTEs. Every Level 2 problem.	III · 34.2
19–24	Window functions. Then patterns P1–P14.	III · 35
25–27	Design, normalization, constraints.	IV
28–30	Indexes and <code>EXPLAIN</code> . The beginner and intermediate interview bands, out loud.	V · 33

60 DAYS · INTERVIEW-READY

The 30-day plan, then: transactions and isolation with two sessions open (IV); performance engineering and the debugging exercises (V · 36); the advanced and scenario bands (33.4–33.5); Level 3 and 4 problems; two projects built from scratch (32); the dialect tables if you are interviewing outside PostgreSQL (37).

90 DAYS · PRODUCTION-READY

The 60-day plan, then: applications, security, administration, partitioning, warehousing (VI); routines, triggers and JSONB (V); Level 5 problems; the capstone (§40.4); and the habit that outlasts the plan — read one `EXPLAIN ANALYZE` plan a day from your own work.

PRACTICE AND REVISION STRATEGY

Type every query — 4,000 keystrokes is the difference between a reader and a practitioner. **Break each example on purpose:** remove a `GROUP BY`, drop a join condition, misspell a column, and read the error. **Spaced revision:** re-do a problem the next day, then a week later, then a month later; anything you get right three times is learnt. **Keep an error log** — every bug you cause, one line on the cause — because your own list of mistakes is worth more than any checklist in this book.

CAREER TRACK

Role	Read closely
Data analyst	II–III in full, 13 especially · 31 · 35 · 34 Levels 3–4
Backend engineer	IV in full · 20–22 · 27–28 · 32 projects · 36
Data engineer	III · 22 · 30–31 · 34 Level 5 · the ETL and warehouse problems
DBA / platform	V–VI in full · 29 · 37 · 38 rules 11–20
Interviewing now	33 in full · 35 · 39 · then 9, 13, 16, 20, 21

40.2 Glossary

Term	Meaning
ACID	Atomicity, Consistency, Isolation, Durability — the four transaction guarantees.
Atomicity	All statements in a transaction take effect, or none do.
Bloat	Space occupied by dead row versions not yet reclaimed by <code>VACUUM</code> .
Candidate key	A minimal set of columns that uniquely identifies a row.
Cardinality	A table's row count; also a relationship's multiplicity; also a column's distinct-value count.
Constraint	A rule the database enforces on every write, from every client.
Correlated subquery	A subquery referencing a column from the outer query, so it cannot be evaluated once.
Covering index	An index containing every column a query needs, enabling an index-only scan.
CTE	Common table expression — a named subquery written with <code>WITH</code> before the query using it.
Deadlock	Two transactions each holding a lock the other needs; the engine aborts one.
Degree	A table's column count.
Denormalization	Deliberately storing a fact twice to speed reads, at the cost of maintenance.
Fan-out	A join repeating rows of one side, inflating any aggregate over that side's columns.
Functional dependency	<code>A → B</code> : knowing A fixes B. The basis of every normal form.
Grain	What one row of a table or result means. The first thing to establish in any aggregate.
HOT update	An update that touches no indexed column and fits on the same page, so indexes are untouched.
Index	A sorted structure stored separately from a table, letting rows be found without a full scan.
Isolation level	How much of other transactions' in-flight work a transaction may observe.
Join	Combining rows from two tables on a condition, producing columns from both.
Lost update	Two transactions read a value, each writes one computed from it, and the second discards the first.

Term	Meaning
MVCC	Multi-version concurrency control — writes create new row versions, so readers never block writers.
Normalization	Structuring a schema so every fact is stored in exactly one place.
N+1	One query followed by one more per returned row; the commonest application performance defect.
NULL	Unknown or not applicable. Not zero, not empty, not false, and not equal to another NULL.
OLAP	Analytical workload — few long read-mostly queries over many rows.
OLTP	Transactional workload — many short reads and writes touching few rows by key.
Primary key	The chosen candidate key: unique, never NULL, one per table, possibly composite.
Query planner	The component choosing an execution strategy by costing alternatives against statistics.
Referential integrity	The guarantee, from a foreign key, that a referenced row exists.
RPO / RTO	Acceptable data loss / acceptable downtime. These two numbers choose your backup method.
Sargable	A predicate an index can satisfy — the column is not wrapped in a function or expression.
SCD	Slowly changing dimension — a warehouse attribute changing over time; Type 2 keeps history.
Schema	Both a database's whole structure, and a named container of objects inside a database.
Selectivity	The fraction of a table a predicate returns. Above roughly 5–10%, a scan beats an index.
Serializable	The strictest isolation level: results are as if transactions ran one at a time.
Surrogate key	A meaningless generated identifier, as opposed to a natural key with real-world meaning.
Transaction	A group of statements that either all take effect or none do.
Transitive dependency	Key → A → B, where A and B are both non-key. The 3NF violation.
View	A stored query behaving like a table; stores no data unless materialized.

Term	Meaning
WAL	Write-ahead log — sequential record of changes flushed before commit; the basis of durability.
Window function	A calculation across related rows that does <i>not</i> collapse them.
Write skew	Two transactions each write different rows after reading an overlapping set, together breaking an invariant.

40.3 Capstone project

Build a production-style database for a city bike-share scheme. No solution is printed here on purpose — you have everything you need, and the value is in making the decisions yourself. What follows is the brief, the eighteen deliverables, and the review checklist you will be marked against.

THE BRIEF

A city runs a bike-share scheme. **Docks** at **stations** hold **bikes**. A **member** on a **plan** unlocks a bike from a dock, rides, and returns it to a dock at another station — one **trip**. A dock holds one bike at a time. A bike is at one dock, or on a trip, or in **maintenance** — never two of those. Trips are charged by duration against the plan's included minutes, with overage per minute; plan prices change over time and old invoices must reproduce exactly. Bikes report **faults**; a faulty bike must not be rentable. Operations need to **rebalance** — move bikes from full stations to empty ones — and want to know which stations run dry, when.

Scale: 700 stations, 12,000 bikes, 40,000 members, 25,000 trips a day, five years of history retained. Reporting must not affect the unlock path.

DELIVERABLES

#	Produce	Reference
1	A noun-and-verb pass, with the entity list and every relationship's cardinality <i>and</i> optionality	IV §15.1
2	An ER diagram in crow's-foot notation, with nullable keys marked	IV §15.3
3	A written statement of the grain of every table that will be aggregated	VI §31.3
4	Full DDL in 3NF, every constraint named, every deliberate exception documented	IV §16–17
5	The rule "one bike per dock" enforced so it cannot be raced	IV §17.5
6	The rule "a bike is docked, on a trip, or in maintenance — never two" enforced declaratively	IV §17.6
7	Plan prices versioned, with invoice lines that reproduce exactly two years later	IV §15.5
8	A list of every business rule no constraint can express, with the mechanism for each	IV §17.8
9	Sample data: 12 stations, 60 bikes, 40 members, 400 trips spanning three months, including the awkward cases	VII §32
10	The unlock transaction: correct under 200 concurrent unlocks, idempotent under retry	IV §19
11	CRUD, joins, aggregations, subqueries, CTEs and window functions across the schema	II–III
12	Eight operational reports, including station dry-out times by hour and rebalancing candidates	III §13
13	The index set, with each column's position justified and every foreign key covered	V §20
14	A partitioning and retention plan for the trips table, with automation	VI §30
15	One report taken from slow to fast, with both <code>EXPLAIN ANALYZE</code> plans and the reasoning	V §21.7
16	Reconciliation queries: at minimum, dock occupancy against bike location, and invoice totals against trip minutes	IV §16.7
17	Roles and privileges for the app, reporting and migrations, plus tenant-style isolation if the city franchises	VI §28
18	Ten interview questions <i>about your own schema</i> , with answers	VII §33

WHERE THE DIFFICULTY ACTUALLY IS

FOUR DECISIONS THAT WILL MAKE OR BREAK IT

- 1. Bike location is a state machine, not a column.** A `current_dock_id` on `bikes` seems obvious and cannot express "on a trip" or "in a van being rebalanced" without contradicting itself. Consider an append-only movement ledger with a reconciled current-location cache — the inventory pattern of §32.10.
- 2. "One bike per dock" and "one dock per bike" are two different constraints.** Both are needed. One is a partial unique index; the other is not, and that is the interesting half.
- 3. A trip has two stations and two times, and is open while in progress.** An open trip has no end — so every duration calculation, every report and every constraint must handle a `NULL` end deliberately. Decide whether an open trip blocks the bike by constraint or by convention, and say why.
- 4. "Which stations run dry, and when" is a gaps-and-islands question over occupancy, not a `count(*)`.** You need per-minute or per-hour occupancy derived from the movement ledger, gap-filled, then runs of zero identified. This is the single hardest query in the project and the one worth doing properly.

MARK YOURSELF AGAINST THIS

Check

- ☐ Every table has a primary key; every natural key has a `UNIQUE`
- ☐ Every foreign key's optionality came from a business rule, and is indexed
- ☐ Every referential action was chosen deliberately, not defaulted
- ☐ No column stores a list, and no money is a `float`
- ☐ Every instant is `timestamptz`; every range filter is half-open
- ☐ Agreed values are snapshotted onto the transaction
- ☐ The unlock path cannot double-book a bike under concurrency — and you can explain why in one sentence
- ☐ Every denormalization has a maintenance mechanism and a reconciliation query
- ☐ Every report's grain is stated, and no aggregate is applied at the wrong one
- ☐ Every index's column order is justified against a named query
- ☐ You can name the five queries this schema exists to serve, and each is index-supportable
- ☐ The application role cannot execute DDL

WHEN YOU ARE DONE

Hand the schema and your eight reports to someone else and ask them to break it. Specifically: ask them to find a sequence of operations that leaves two bikes at one dock, or a trip with no bike, or an invoice that changes when a price is edited. If they cannot, you have built something production-grade — and you have also just run the design review of §15.8 from the other side, which is the skill this book was actually teaching.

Forty parts, seven volumes

The curriculum is complete. From "what is data?" in Volume I to a production-style capstone here, every part of the original forty is written.

Volume	Contents
I	Front matter · Parts 1–2 — fundamentals, the relational model, environment, the practice database
II	Parts 3–8 — basics, data types, operators, filtering, functions, aggregation
III	Parts 9–14 — joins, subqueries, CTEs, set operations, window functions, advanced SQL
IV	Parts 15–19 — design, normalization, constraints, transactions, isolation
V	Parts 20–26 — indexes, plans, performance, views, routines, triggers, JSONB
VI	Parts 27–31 — applications, security, administration, partitioning, warehousing
VII	Parts 32–40 — projects, 700 interview questions, 500 problems, patterns, debugging, dialects, practices, cheat sheets, roadmap, capstone

THE THROUGH-LINE, IF YOU REMEMBER NOTHING ELSE

Say the grain before you aggregate. Most wrong numbers are a correct aggregate at the wrong level.

NULL is unknown, and it propagates. It changes joins, filters, aggregates and **NOT IN** — silently, every time.

Put the rule where it cannot be forgotten. A constraint the database enforces beats a check the next developer must remember.

Express the precondition in the write. `SET stock = stock - $1 WHERE stock >= $1` — and concurrency stops being your problem.

Measure before you tune. Estimated rows against actual rows answers more questions than any amount of staring at SQL.

Everything else in these seven volumes is those five ideas applied to a specific case. The examples will date; the reasoning will not.

WHERE TO GO NOW

Pick the plan in §40.1 that matches your deadline and work it. When it is done, build the capstone in §40.3 — and hand it to someone else to break, because the last skill this book teaches is reviewing a schema rather than writing one.

